

Database Design for Smarties

USING UML FOR DATA MODELING

Robert J. Muller



MORGAN KAUFMANN PUBLISHERS

AN IMPRINT OF ACADEMIC PRESS

A Harcourt Science and Technology Company

SAN FRANCISCO SAN DIEGO NEW YORK BOSTON
LONDON SYDNEY TOKYO

Базы данных и UML

ПРОЕКТИРОВАНИЕ



Роберт Дж. Мюллер



Издательство "ЛОРИ"

Database Design for Smarties
Using UML for Data Modeling
Robert J. Muller
Copyright © 1999
All rights reserved

Базы данных и UML
Проектирование
Роберт Дж. Мюллер
Переводчик Е. Молодцова
Научный редактор О. Труфанов
Корректор Т. Гладштейн
Верстка Л. Федерякиной

© 1999 by ACADEMIC PRESS
A Harcourt Science and Technology Company
525 B Street, Suite 1900, San Diego, CA 92101-4495, USA
ISBN 1-55860-515-0

© Издательство "ЛОРИ", 2002

Изд. №: ОАИ (03)
ЛР №: 070612 30.09.97 г.
ISBN 5-85582-168-4

Подписано в печать 12.11.2002 Формат 70х100/16
Гарнитура Нью-Баскервиль Печать офсетная
Печ. л. 27 Тираж 3200 Заказ N 3ак.704
Цена договорная

Изд-во "ЛОРИ". Москва, 123100 Шмитовский пр., д. 13/6, стр. 1 (пом. ТАРП ЦАО)
Телефон для оптовых покупателей: (095)259-01-62
WWW.LORY-PRESS.RU

Отпечатано в типографии ООО "Типография ИПО профсоюзов Профиздат"
109044, Москва, ул. Крутицкий вал, д. 18

*Посвящается Тео, чьи разработки
в области баз данных расширяются изо дня в день*

Предисловие

Эта книга иллюстрирует простой тезис: базу данных (БД) любого типа можно разработать с помощью стандартных методов объектно-ориентированного программирования (ООП). Как всегда, все скрыто в деталях, и при работе с БД детали имеют особое значение.

Отход от традиции

В книге рассматриваются реляционные, объектно-реляционные (ОР) и объектно-ориентированные (ОО) базы данных. В ней нет сравнительного анализа всех методов разработки БД, а также существующих методов информационного моделирования. Повторим еще раз главный тезис — в большинстве случаев при использовании ООП можно обойтись без этих методов. Если вы хотите узнать все о методах разработки IDEFIX, или использовании диаграмм SSADM, или проектировании в среде Oracle Designer/2000, обратитесь к приведенной в конце книги библиографии.

Автор воспользовался унифицированным языком моделирования (UML) и его методами моделирования по двум причинам. Во-первых, это принятый стандарт Рабочей группы по развитию стандартов объектного программирования (OMG). Во-вторых, это кульминация многих лет работы трех очень умных разработчиков моделей объектов, которые решили создать из предлагаемых каждым из них методов единый, всеобъемлющий стандарт нотации. В главе 7 приводится подробное описание UML. Тем не менее можно использовать и любой другой объектно-ориентированный метод моделирования. Знание их облегчит освоение UML, так как они являются объединением почти всех концепций объектно-ориентированных методов, которые встречались автору на практике. Изучая UML, можно получить систематическое представление о концепциях ООП. Затем будет нетрудно преобразовать нотацию UML и знания о ее применении, полученные из данной книги, в любую объектно-ориентированную нотацию и метод, который вы сможете использовать в своей практике.

Данная книга не относится к разряду справочников по теории баз данных; она предназначена для практического использования при создании БД. У автора не было цели представить совершенно новый взгляд на БД или академическое исследование. Эта книга посвящена продуктивному применению современных технологий для построения полезных систем программного обеспечения. Автор делает упор на адаптацию современных технологий к ООП, а не на их замену объектно-ориентированным проектированием.

И, наконец, как вы заметите, в этой книге приводятся примеры на основе системы управления базой данных Oracle. Автор почти все время работал именно с ней, хотя использовал также и другие базы данных — от Sybase до Informix и SQL Server, здесь приведены примеры на основе всех этих

DBMS-продуктов. Концепции данной книги имеют вполне общий характер. Можно преобразовать любой пример на основе Oracle в эквивалент на основе любой другой DBMS, по крайней мере настолько, насколько это позволяет сделать реляционная структура. Но после перехода к реалиям объектно-реляционной DBMS или объектно-ориентированной DBMS можно обнаружить, что выбранный продукт во многом будет определять ваши возможности (см. главы 12 и 13). Точка зрения автора состоит в следующем: не совершите ошибку, полагая, что методы, описанные в книге, изменятся в зависимости от того, будет ли использован Informix или MS Access. Именно разработка является предметом нашей книги, но не реализация. Как и для UML, при понимании концепций можно легко реализовать особенности базы данных с помощью любой выбранной технологии. Если у вас есть конкретные вопросы о применении методов на практике, пожалуйста, напишите по адресу muller@computer.org, чтобы разрешить свои проблемы.

Технология информационных хранилищ

Блюстители теории БД вскоре обнаружат, что в данной книге не рассказано о технологии информационных хранилищ, информационных витрин и о схеме звезда. Однако надо было где-то подвести черту, чтобы получить книгу реального объема, и мы с издателем решили не включать сюда проблемы, связанные с технологией информационных хранилищ.

Основной концепцией технологии информационных хранилищ является размерность — набор информационных атрибутов, относящихся к основным объектам информационного хранилища. В классическом анализе данных, например, приходится часто структурировать данные в многомерные таблицы с ячейками, находящимися на пересечении различных размерностей или категорий. Эти таблицы становятся основой для дисперсионного анализа и других методов статистического моделирования. Одним из наиболее важных способов организации размерностей является схема звезда, в которой таблицы размерностей окружают таблицу фактов (объект) в конфигурации звезды с отношениями один-ко-многим. Такая конфигурация позволяет аналитику данных просматривать факты в базе данных (основные объекты) с точки зрения различных размерностей.

В классическом ООП схема звезда является образцом взаимозависимых объектов, которые вместе находятся в центральном объекте некоторого рода. Центральному объекту не принадлежат другие объекты; напротив, он связывает их в многомерную структуру. Схема звезда реализуется в реляционной БД как набор таблиц один-ко-многим, в объектно-реляционной базе данных — как набор объектных ссылок, а в объектно-ориентированной БД — как объект со средствами доступа к размерностям и атрибутам, которые ссылаются на другие объекты.

Дополнительные ссылки на ресурсы Web

Если вы хотите больше узнать об управлении базами данных, предлагаем вашему вниманию широко известные реляционные, объектно-реляционные и объектно-ориентированные продукты. На этих Web-сайтах можно найти как текущие, так и пробные версии продуктов.

Инструмент	Компания	Web-сайт
Rational Rose 98	Rational Software	www.rational.com
Object Team	Cayenne Software	www.cool.sterling.com
Oracle Designer Object Extension	Oracle Corp.	www.oracle.com
ObjectStore PSE Pro Java	Object Design	www.odi.com
POET Object Database System	POET Software	www.poet.com
Jasmine	Computer Associates	www.cai.com
Objectivity	Objectivity, Inc.	www.objectivity.com
Versant ODBMS	Versant Corp.	www.versant.com
Personal Oracle8	Oracle Corp.	www.oracle.com
Personal Oracle7	Oracle Corp.	www.oracle.com
Informix Universal Data Option	Informix Software, Inc.	www.informix.com
Informix Dynamic Server, Personal Edition	Informix Software, Inc.	www.informix.com
Informix SE	Informix Software, Inc.	www.informix.com
Sybase Adaptive Server	Sybase, Inc.	www.sybase.com
Sybase Adaptive Server Anywhere	Sybase, Inc.	www.sybase.com
SQL Server 7	Microsoft Corp.	www.microsoft.com
DB2 Universal Database	IBM Corp.	www.ibm.com

Содержание

Глава 1 Жизненный цикл базы данных	1
Анализ информационных требований	2
Создание модели данных	3
Проектирование и оптимизация базы данных	5
Качество, проверка и тестирование базы данных	8
Сертификация баз данных	10
Поддержка и доработка баз данных	10
Глава 2 Системная архитектура и проектирование	12
Виды системной архитектуры	14
Трехсхемная архитектура	14
Многозвенные архитектуры	18
Резюме по системной архитектуре	30
Архитектуры данных	30
Реляционные базы данных	32
Объектно-ориентированные базы данных	36
Объектно-реляционные базы данных	45
Итоги	56
Глава 3 Сбор требований	57
Неопределенность и устойчивость	57
Неопределенность	58
Наблюдение и правильная постановка вопросов	62
Устойчивость	65
Четкая расстановка приоритетов	69
Понимание требований	69
Распределение требований по категориям	70
Зависимости между требованиями	73
Определение приоритетов требований	74
Определение стиля базы данных	74
Итоги	76
Глава 4 Моделирование требований	
с помощью вариантов использования	77
Весь мир — сцена	78
Актеры на сцене	81
Диаграммы вариантов использования	81
Краткий пример	83
Транзакции и варианты использования	85

Внешние связи	298
Двоичные ассоциации	298
Генерализации	303
Особые ситуации	310
Жизнь по правилам	317
Инварианты классов	317
Системные инварианты	319
Отношения нормализации	320
Элементарные значения	320
Зависимости и нормализация	322
Денормализация созданной схемы	327
Язык мира	329
Правила согласования	329
Опасности несоответствия	335
Итоги	341
Глава 12 Разработка схемы объектно-реллционной базы данных	343
Так что же нового?	343
Свойства	344
Обратная сторона	345
Процесс преобразования для продуктов ORDBMS	346
Разнообразие объектов: типы	355
Определенные пользователем и объектные типы	357
Ассоциации	366
Поведение	370
Кто диктует правила?	371
Язык войны	371
Устойчивые классы	372
Операции	378
Итоги	380
Глава 13 Разработка схемы объектно-ориентированной базы данных	381
Процесс преобразования для продуктов OODMBS	381
Действительно простота?	385
Классы	386
Генерализации и реализации	392
Ассоциации	393
Дисциплинирование объектов	403
Проблемы поведения	403
Установка границ	405
Целевой язык	406
Устойчивые классы и интерфейсы	406
Операции	412
Итоги	415
Ссылки на истории о Шерлоке Холмсе	416
Библиография	417

Глава 1

Жизненный цикл базы данных

Когда бы выбор я имел, я бы провел остаток жизни, наслаждаясь
ее тихими часами.

Шекспир. Король Генрих IV, часть I

Базы данных (БД), как и все прочие разновидности объектов программного обеспечения (ПО), постоянно подвергаются изменениям. Эта глава представляет собой введение в жизненный цикл базы данных. Разработка БД — только первый шаг, и необходимо представлять весь ее жизненный цикл, чтобы сразу не наделать ошибок. Вы быстро убедитесь, что проблемы могут возникнуть в самые неожиданные моменты разработки.

Жизненный цикл базы данных, как и большинства продуктов, можно разделить на множество периодов. Успешная разработка БД не движется напролом, как Годзилла, сокрушая все на своем пути. Это — итеративный, пошаговый процесс, что особенно заметно при использовании ОО-технологий. Каждый шаг ведет к созданию рабочей базы данных; каждая итерация идет от моделирования к созданию структуры, а затем обратно в таком порядке, в каком это имеет смысл делать. Разработка БД, как и любое системное проектирование, использует многоуровневый процесс [Нотманн, 1997]. Многоуровневость — это когнитивный эквивалент воды, ищущей свой уровень. Когда ситуация меняется, происходит возвращение к той части жизненного цикла, которая соответствует нуждам программиста в данный момент. Иногда требуется создать физические структуры; в другие моменты времени следует построить модели и разработать новые структуры.

ВНИМАНИЕ

Избегайте терминологических заблуждений. Автор посчитал целесообразным определить свои термины так, как счел нужным, хотя существует много других способов описания тех же самых вещей. В частности, обратите внимание, как он использует термины "логический" и "физический". Зачастую поставщики CASE-средств и прочие применяют термин "физическая" разработка для того, чтобы отделить проектирование реляционной структуры от модели данных "объект — отношение". Автор называет последнее моделированием процесса, а первое — процессом логической или концептуальной разработки, следуя стандартам архитектуры ANSI (см. главу 2). Физический проект — это процесс создания физической структуры, набора путей доступа и структур хранения БД.

Это полностью отличается от создания реляционной структуры, хотя в обоих процессах часто используются сходные операторы языка определения данных. Сосредоточьтесь на настоящей цели, выполняя работу, а не на формальном разделении работы на эти категории. Следует также понимать, что эти терминологические отличия имеют чисто культурную природу; их изучение является частью социализации разработчика в определенной культуре проектирования, в которой он будет работать. Нужно всегда иметь в виду действительную работу в рамках языка определенной культуры, чтобы эффективно обращаться с частными особенностями.

Анализ информационных требований

Базы данных начинаются с людей и их потребностей. Поскольку разрабатывается база данных, следует заботиться о нуждах пользователей. *Конечный пользователь* — это конечный потребитель ПО, тот, кто смотрит на экран компьютера, когда запросы к базе данных совершают перебор тысяч или миллионов объектов в системе. Системный пользователь — непосредственный потребитель созданной БД, которая ему необходима при построении системы, предназначенной для конечного пользователя. *Системный пользователь* — это программист, который использует SQL или OQL или любой другой язык для доступа к базе данных, чтобы доставить результаты конечному пользователю.

Как конечный пользователь, так и системный пользователь имеют специфические потребности, о которых должен знать разработчик БД до начала проектирования. Эти потребности разработчик должен преобразовать в структуру определенного типа в базе данных. Информационные требования переплетаются и образуют требования к более крупной системе, частью которой является БД.

В системе с централизованной базой данных требования к данным являются критически важными. Например, если система предназначена в основном для предоставления стабильного набора информационных объектов с целью поиска и доступа, нужно потратить значительное время на понимание информационных требований. Чаще встречающаяся система — это такая, в которой БД поддерживает в основном текущее использование системы. В таком случае большую часть времени разработчик тратит на

выяснение требований, выходящих за рамки простых потребностей базы данных. Применяя стандартные ОО-варианты использования и другие средства ОО-анализа, разрабатываются требования, предъявляемые к системе. В главах 3 и 4 эти методы будут рассмотрены подробно, что позволит разработчику упростить представление конечного пользователя о БД. Они также позволят разработчику понять нужды системных пользователей данных, поскольку он сможет составить представление о том, что будет делать БД. Конечным пользователям требуются объекты, отражающие их мир; системным пользователям нужны структуры, с помощью которых они смогут выполнять свою работу эффективно и продуктивно.

Один класс системных пользователей более важен, чем остальные: это повторный пользователь. Действительное преимущество системной ОО-разработки состоит в возможности пользователя системы изменять функции созданной базы данных. Разработчик должен проектировать БД так, как будто некто, смотрящий из-за его плеча, будет добавлять что-то новое после того, как разработка закончена, — возможно, новые структуры базы данных, соединения с другими базами данных или новые системы, использующие созданную БД. Ключом к пониманию повторного использования является комбинация потенциала повторного использования и сертификация повторного использования.

Потенциал повторного использования — это степень возможности повторного использования базы данных в конкретной ситуации [Muller, 1998]. Потенциал повторного использования измеряет присущую системе способность повторного использования, повторное использование системы на определенном домене и повторное использование системы в организации. По мере разработки нужно следить за каждым из этих компонентов потенциала повторного использования для создания оптимальной БД с повторным использованием.

Сертификация повторного использования говорит системному пользователю, что можно ожидать от базы данных. Сертификация повторного использования БД состоит в том, чтобы проинформировать системных пользователей о том, какова степень риска при повторном использовании, каковы функции базы данных и кто несет ответственность за систему.

В главе 9 подробно рассматривается потенциал и сертификация повторного использования БД.

Создание модели данных

Выяснив требования пользователей, разработчик может формально смоделировать проблему. Создание модели данных преследует несколько целей: помогает разработчику систематизировать представление о данных, проясняя их значение и практическое применение, связать потребности с возможными способами их удовлетворения, а также предоставляет платформу, на которой можно приступить к разработке и конструированию с определенной степенью уверенности в успехе.

Создание модели данных — это первый шаг при разработке БД. Оно позволяет связать потребности пользователей с программными решениями, удовлетворяющими эти потребности. Это начальная абстракция, скрывающая сложность системы. Модель данных снижает сложность до уровня, с которым разработчик в состоянии справиться и которым он может управлять. Поскольку БД и структуры данных становятся все сложнее и многочисленнее, моделирование данных приобретает все большее значение, обусловливаемое способностью продемонстрировать, с одной стороны, суть системы для конкретных физических и концептуальных структур, с другой — многообразие сфер применения.

В настоящее время при моделировании БД используют одну из разновидностей моделирования "объект — отношение" (ER) [Teorey, 1999]. Такие модели помещают в центр внимания некоторые сущности и связи между ними (объекты и отношения). Большинство инструментов разработки баз данных являются инструментами ER-моделирования. Нельзя написать книгу о разработке баз данных без обсуждения ER-моделирования (см. главу 6 также главу 7, предлагающую произвести изменения системы мышления).

В главе 2 представлена идея того, что разработка системной архитектуры и разработка базы данных — это одно и то же. ER-моделирование не совсем подходит для архитектуры системного моделирования. Как разрешить противоречие? В главе 7 даны основы UML, нотации моделирования, инструменты для моделирования всех аспектов системы программного обеспечения, начиная с анализа требований и до реализации. Объектное моделирование с помощью UML заменяет собой ER-моделирование в современном процессе разработки БД, по крайней мере, это предлагается сделать в данной книге.

Объектное моделирование использует при моделировании системы стандартные ОО-концепции сокрытия и наследования данных. Эта модель частично покрывает потребности системы в данных. По мере разработки структуры классов и объектов моделируются данные, которые система предоставит своим пользователям для удовлетворения их потребностей.

Но объектное моделирование позволяет осуществить гораздо больше, нежели простое моделирование статической структуры системы. Объектное моделирование включает в себя также описание динамического поведения системы. Наследование отражает структуру данных системы, а также распределение труда через наследование поведения и полиморфизм. Это динамическое свойство влияет на разработку БД двумя способами. Во-первых, структура системы отражает поведенческие нужды так же, как и структурные различия. Это внимание к поведению часто отражает различное понимание трансформации результатов разработки в реальный мир, что не было бы очевидным при более статичной модели данных. Во-вторых, при усилении связи приложения с БД через правила, триггеры, хранимые процедуры и активные объекты статические методы часто терпят неудачу в области сохранения жизненно важной части разработанной базы данных. Как, например, ER-модель отразит бизнес-правило, которое выходит за рамки простой ссылочной константы внешнего ключа целостности?

В главах 8 – 10 отступаем от темы объектного моделирования, рассматривая интеграционные модели в целом с точки зрения пользователя. Соотнесение результатов разработки с требованиями – важный аспект при проектировании баз данных потому, что он проясняет причины, стоявшие за принятыми при разработке решениями. Оно выявляет также те места, где конфликтуют различные части системы, возможно, из-за противоречивости ожиданий пользователя от системы. Основной частью моделирования данных является разрешение подобных конфликтов на верхнем уровне модели.

Процесс моделирования – только начало разработки. Когда модель получена, следующий шаг состоит в том, чтобы соотнести модель с исходными требованиями, а затем двинуться вперед к добавлению структур, поддерживающих как повторное использование, так и системные функции.

Проектирование и оптимизация базы данных

Когда начинается проектирование? Проектирование начинается с любого места в процессе, когда разработчик начинает размышлять о том, как сущности соотносятся друг с другом. Моделирование плавно переходит в разработку. Добавление новой сущности или класса является моделированием; принятие решения о том, как эта сущность или класс соотносится с другими, – это проектирование.

Где начинается проектирование? Обычно проектирование начинается где-то в другом месте, т. е. когда приступают к созданию БД, почти всегда берут структуры из чьей-то работы, является ли это анализом требований унаследованной БД, архитектурой предыдущей системы или чем-то еще. Качество и объем генетического материала, образующего основу разработки, часто может определить ее успех. И то, как удастся справиться с данной задачей, в значительной степени влияет на окончательный результат всего проекта.

Например, можно начать с унаследованной системы на основе реляционной базы данных, которую нужно преобразовать в объектно-ориентированную БД. Эта унаследованная система может даже не находиться в третьей нормальной форме (см. главу 11), она может являться результатом работы шести комитетов за 20-летний период (как, например, налоговый код США). Подходящая исходная система помогает созданию БД, но место возникновения проблем, определяется не столько тем, где началась разработка, а на какой основе она происходит. В главе 10 дается несколько рекомендаций о том, как развивать разработку в случае разных начальных условий, а также рассматривает культурный контекст, в рамках которого осуществляется проектирование. Организационная культура может повлиять на результат больше, чем технологии.

Момент истины при разработке наступает во время преобразования модели данных в схему. Зачастую CASE-средства предоставляют способ создания реляционной схемы непосредственно из модели данных. До тех пор пока эти средства не столкнутся с текущими реалиями, они не принесут большой пользы, если не осуществляется стандартное ER моделирование и

создание стандартной реляционной схемы. Автор, например, не имеет никакого предпочтения в отношении средств создания ОО- или ОР-моделей на базе ОО-разработки. В главах 11–13 показано, как создать реляционную, ОО- или ОР-разработку на основании ОО-модели данных. Это преобразование использует вариации стандартного алгоритма создания схем из моделей, но слегка отличается в трех случаях. Кроме того, существуют определенные приемы, которые можно использовать для улучшения схем в процессе преобразования.

Наводите мосты впереди себя и не давайте им сгореть после того, как перейдете реку. Так как разработка БД является итеративным и пошаговым процессом, нельзя допускать, чтобы модель стала недействительной. Если произойдет десинхронизация модели данных со схемой, обнаружится, что все труднее и труднее вернуться к ранее созданной части разработки. Здесь снова могут помочь CASE-средства, если они содержат способы обратного проектирования для создания моделей из схем, но опять же эти средства не могут поддерживать большинство методик, описанных в данной книге. Кроме того, поскольку ОО-модель поддерживает больше, чем просто обычное определение схемы, отсутствие сопровождения модели выйдет за рамки проекта и потребует создания общего проекта системы, а не просто разработки БД.

В определенный момент рассматриваемая разработка переходит от логического проектирования к физическому. В данной книге рассматривается только логическое проектирование, которое является итерационным процессом, а не строгой последовательностью шагов. По мере развития физической структуры происходит осознание того, что определенные аспекты логического проекта негативно влияют на физическую реализацию и требуют пересмотра. Изменения в логической разработке по мере повторного пересмотра требований и моделирования также требуют изменения физической реализации. Например, многие разработчики БД оптимизируют производительность путем денормализации логической структуры. Денормализация — это процесс комбинирования таблиц или объектов для ускорения доступа, который обычно производится путем ликвидации объединения данных. Снижение производительности происходит из-за выполнения большего количества работы для поддержки целостности, так как данные могут появляться больше, чем в одном месте БД. Поскольку это негативно влияет на результаты проектирования, необходимо рассмотреть возможность денормализации как итеративного процесса, управляемого требованиями в большей степени, нежели стандартная процедура работы. В главе 11 денормализация рассматривается более подробно.

Физическая разработка состоит в основном из построения путей доступа и структур памяти в физической модели БД. Например, в реляционной базе данных создаются индексы в наборе столбцов и нужно решить, использовать ли В*-деревья, хеш-индексы или битовые отображения или же принять решение о предварительном объединении таблиц в группы. В объектно-ориентированной БД может быть принято решение о группировке определенных объектов или об индексировании части памяти в зонах объектов. В объектно-реляционной БД можно установить необязательное

управление памятью или модули путей доступа для расширенных типов данных, создав определенную их конфигурацию для каждой конкретной ситуации. Попробуйте размножить таблицу на нескольких дисках. Выходя за рамки этой простой конфигурации физической структуры, можно распределить БД на нескольких серверах, применить стратегии репликации или создать системы безопасности для управления доступом.

По мере следования от логического к физическому проектированию все больше внимания надо уделять переходу от моделирования реального мира к улучшению производительности системы — оптимизации и настройке БД. Большинство аспектов физической разработки имеют непосредственное влияние на производительность базы данных. В частности, на этом этапе следует принять во внимание, каким образом конечные пользователи будут осуществлять доступ к данным. Необходимость знать способ доступа конечных пользователей означает, что надо выполнить некоторое физическое проектирование в ходе разработки и построения системы, использующей БД. Неплохо также провести ряд мозговых атак для определения будущего системы. Так, при разработке хранилищ данных критичного назначения для поддержки принятия решений или систем обработки транзакций в режиме реального времени с необходимостью предоставления постоянных ответов нужно четко представлять себе требования к производительности до завершения физического проектирования. Кроме того, при разработке физических моделей с использованием комбинаций развитых продуктов программного/аппаратного обеспечения, таких как симметричная мультиобработка (SMP), массивная параллельная обработка (MPP) или процессоры с кластерами, физическое проектирование критично по отношению к настройке БД.

СОВЕТУЕМ

Разработчик баз данных может получить очень многое из Интернета. Имеется множество телеконференций в Usenet в группе по интересам *comp.databases*, например *comp.databases.oracle.server*. Существует несколько Web-сайтов, специализирующихся на рекомендациях по определенным поставщикам и методам; для поиска таких сайтов используйте поисковые машины. Есть также списки почтовой рассылки (электронная почта, доставляемая автоматически с результатами дискуссий на определенную тему), такие как список почтовой рассылки по моделированию данных. Эти списки могут быть полезны в большей или меньшей степени в зависимости от уровня активности сервера, варьирующегося от нуля до сотен сообщений в месяц. Обычно можно узнать о таких списках через телеконференции Usenet, относящиеся к определенной тематической области. И, наконец, рассмотрите возможность вступления в любые группы пользователей в своей предметной области, например в группу пользователей инструментов разработчика Oracle (www.odtug.com); они обычно проводят конференции, поддерживают Web-сайты и имеют почтовые списки для своих членов.

Разработка БД не завершена до тех пор, пока не будут учтены риски, связанные с использованием этой базы данных и определены методы управления рисками, которые помогут их избежать. *Риск* — это возможность события с негативными последствиями. Вероятность риска можно оценить с помощью объективных или субъективных показателей. В сфере БД риски включают в себя такие понятия, как катастрофы, отказы аппаратных средств, отказы средств программного обеспечения и ошибки, случайное разрушение данных и преднамеренные нападения на данные или серверы. Чтобы справиться с рисками, необходимо сначала определить их допустимый уровень, а затем управлять рисками в рамках этого уровня. Например, если допустимы частые многочасовые простои системы, то нет смысла использовать возможности многих отказоустойчивых средств современных продуктов DBMS. Если небольшие проблемы с БД не имеют особого значения, можно не прилагать колоссальных усилий по программированию, чтобы изолировать проблемы на каждом уровне ввода и модификации данных. Применяемые методы управления рисками должны отражать устойчивость к риску, а не являться магическими ритуалами, выполняемыми для поддержания созданной среды в безопасности от возможного воздействия богов базы данных (в главе 10 рассматриваются некоторые варианты влияния шаманских культур на проектирование БД). В рамках этого процесса нужно начать рассматривать непосредственный способ управления рисками — тестирование.

Качество, проверка и тестирование базы данных

Качество БД определяется тремя исходными параметрами: требованиями, способами разработки и способами построения. Для оценки качества разработки и степени удовлетворения требований используют методы анализа, в то время как качество построения оценивается с помощью тестирования. В главе 5 рассматриваются требования и тестирование БД, а также в различных главах, касающихся разработки, представлены проблемы, которые следует рассмотреть при анализе базы данных. Планы тестирования БД используют модели тестирования, отражающие эти компоненты: модель содержимого, структурная модель и модель проекта.

Содержимое — это то, что работающие с базами данных люди называют обычно "качество данных". При создании БД имеется множество альтернативных способов получения данных из БД. Многие базы данных включают пакетированное содержимое — базы данных изображений и текстов для Интернета, поисковые базы данных или части БД, заполненные данными, отражающими варианты и/или возможности выбора в пределах программного продукта. Нужно разработать модель, описывающую возможные допущения и правила для каждой БД. Часть этой модели можно позаимствовать из созданной модели данных, но ни один из имеющихся в настоящее время методов моделирования не описывает полностью всю семантику и прагматику содержимого БД. Хороший план тестирования включает в себя все содержимое, а не только ограниченный взгляд с точки зрения модели данных.

Модель данных предоставляет часть структуры базы данных, а физическая схема — остальное. Убедитесь, что реально созданная БД содержит структуры, объявленные в модели данных. Следует также проверить, что БД содержит физические структуры (индексы, кластеры, расширенные типы данных, контейнеры объектов, наборы символов, полномочия безопасности, роли и пр.), определенные физическим проектом. В данном случае также требуется тестирование нагрузки, производительности и конфигурации. В продаже имеется ряд инструментов тестирования, которые могут помочь при тестировании физических возможностей БД, хотя большинство из них применимы только к реляционным базам данных.

Модель поведения возникает из проектной спецификации поведения устойчивых объектов. Чаще всего это поведение воплощается в хранимых процедурах, триггерах, правилах либо в методах, расположенных на сервере объектов. Используются обычные методы моделирования процедурного тестирования. А затем создаются наборы тестовых сценариев, полностью отражающие эти модели для снижения уровня риска до приемлемого уровня. В некотором роде происходит перекрытие со стандартным тестированием объектов и интеграционным тестированием, однако зачастую методики тестирования отличаются, включая выполнение отдельных программ помимо основного программного кода.

Как структурное, так и поведенческое тестирование требует наличия тестовых данных в БД. Большинство разработчиков считают, что "настоящие" данные — это все, что нужно иметь в тестовой базе данных. К сожалению, как и в случае тестирования кодов, "настоящие" данные покрывают лишь небольшую часть всех возможностей, и это касается не только семантики. Используя созданные модели данных, нужно разработать единообразные семантические наборы данных, покрывающие все возможные комбинации, которые необходимо протестировать. Зачастую для этого нужно несколько тестовых БД, поскольку требования создают противоречивые данные в одних и тех же структурах. Создание экземпляра тестовой БД не является простой, прямолинейной загрузкой данных из реального мира.

Разработка методов тестирования идет параллельно с проектированием БД и ее реализацией, как это происходит и со всеми другими типами ПО. Об усилиях, затрачиваемых на тестирование, необходимо думать таким же образом, как и об усилиях, затраченных на разработку. Нужно применять те же итеративные, пошаговые подходы, что и при проектировании, с тем же анализом результатов, что использовался при разработке, а также проверять само тестирование.

Тестирование дает ясное представление о рисках при использовании созданной БД. Это, в свою очередь, позволяет довести сведения о риске до тех, кто хочет ее использовать: происходит *сертификация*.

Сертификация баз данных

Очень редко можно встретить сертифицированную БД. А жаль, потому что в этом существует огромная потребность. Автор постоянно встречает пользователей систем с централизованной БД, которые хотят повторно использовать ее самую или ее конструкцию. Обычно они не могут этого сделать либо потому, что не понимают, как она работает, либо потому, что поставщик ПО отказывается разрешить доступ к ней из страха "разрушения".

Это частный случай более общей проблемы: отсутствие возможности повторного использования ПО. Одним из безусловных преимуществ ООП является возросшая продуктивность в случае повторного использования [Muller, 1998]. На практике трудно повторно использовать БД, только в небольшом числе проектов это прошло успешно. Ключ к решению проблемы имеет две стороны: проектирование повторного использования и сертификация повторного использования.

Вся эта книга посвящена разработке, предназначенной для повторного использования. Все представленные методы учитывают требование сделать ПО и БД более пригодными для повторного использования. В предыдущем разделе данной главы "Анализ информационных требований" кратко рассказано о природе потенциала повторного использования, а в главе 9 подробно рассмотрен как потенциал повторного использования, так и сертификация.

Сертификация состоит из трех частей: риск, функция и ответственность. В результате предпринятых усилий по анализу и тестированию получают данные, которые можно применять для оценки риска повторного использования БД и ее проекта. Отсутствие сертификации риска приводит к тому, что большинство разработчиков думают, что только им должен быть разрешен доступ к БД для ее использования. Кроме того, отсутствие анализа риска может обмануть людей, занимающихся поддержкой системы: они полагают, что легко осуществить изменения или что эти изменения слабо влияют на системы. Функциональная часть сертификации состоит в создании ясной документации по концептуальной и физической схемам и четкого представления о преследуемых целях разработки БД. Наконец, четкое представление о том, кто владеет базой данных и кто отвечает за ее поддержку, позволяет другим использовать ее, практически не заботясь о будущем. Без этого пользователи вряд ли смогут обосновать возможность повторного применения кода, проекта и данных без каких бы то ни было изменений, как есть. Это может серьезно затормозить поддержку и доработку БД при повторном использовании.

Поддержка и доработка баз данных

В книге этому уделяется мало внимания, но поддержка и доработка являются финальной стадией жизненного цикла БД. Когда база данных создана, все закончено, не правда ли? Не совсем так.

Часто процесс разработки начинается, когда есть рабочая БД либо в наследуемой системе, либо как результат наследования разработки от предыдущей версии системы. Иногда проект базы данных зависит от логики поддержки и доработки. В течение многих лет жалобы от программистов по этому поводу звучат чаще всего. Инерция существующей системы просто сводит программистов с ума. Им хочется проявить себя, работая над интересными проблемами, а кто-то ограничивает их творчество, создав систему, которую им предстоит только модифицировать. В главе 10 подробно рассматривается, как наилучшим образом разрешить эту ситуацию.

Повторим еще раз, что разработка БД является итеративным, пошаговым процессом. Пошаговая природа остается и после поставки совершенно новой БД, ее не будет, только если прекратит свое существование база данных. БД претерпевает многие изменения, и разработчик никогда, по сути, не бывает спокоен, наслаждаясь ее безотказной работой. В нескольких следующих главах возвращаемся к первой части жизненного цикла, рождению БД в соответствии с потребностями пользователей.

Глава 2

Системная архитектура и проектирование

Произведения искусства являются единственными объектами материальной вселенной, обладающими внутренним порядком. Поэтому, хоть я и не убежден в том, что только искусство имеет значение, я действительно верю в Искусство ради Искусства.

Форстер Е. М. Искусство ради Искусства

Словосочетание "создать архитектуру" имеет длинную и ясную историю, берущую начало в шестнадцатом веке. Но есть ли разница между понятиями "разработать" и "создать архитектуру"?

В мире современных БД имеется небольшая разница между этими понятиями в теории и значительное различие на практике. Администраторы БД и создатели архитектуры данных "проектируют" базы данных и системы, а разработчики прикладного ПО "создают архитектуру" систем, которые их используют. Можно легко отличить инструменты разработки БД от средств создания системной архитектуры.

Основной тезис данной книги состоит в том, что *никакой разницы нет*. Разработка БД с использованием методов, представленных здесь, неразличимым образом переплетается с созданием архитектуры всей системы в целом, частью которой является и база данных. Архитектура многомерна, но составляющие ее компоненты взаимодействуют по большей части как комплексная система, а не являются самостоятельными составляющими элементами. Разработка БД, как и все архитектурные построения, — искусство, а не наука.

Этот вид искусства преследует вполне практическую цель: сделать информацию доступной клиентам системы программного обеспечения. Идеи баз данных имеют глубокие корни, относящиеся к временам самаритян и египтян,

начавших использовать клинообразные знаки и иероглифы для записи счетов в форме, которая могла быть сохранена и повторно просмотрена по требованию [Diamond, 1997]. Это суть БД: разумно постоянный и доступный механизм памяти для хранения информации. Разработка баз данных в докомпьютерную эпоху предстает перед нами как искусство, о чем свидетельствуют находки, обнаруженные в местах поселения самаритян, египтян, народа майя и китайцев. Компьютер дал нам нечто большее: *систему управления базой данных*, ПО, которое позволяет оживить базу данных в руках клиента. Заменяв собой глиняную табличку или пыльную стену, БД превратилась в абстрактный набор битов, организованных в рамках структур данных, операций и ограничений. Разработка таких систем ПО, охватывающих как данные, так и их использование, и есть предмет изучения данной книги.

Системная архитектура, первая ступень проектирования БД, является архитектурной абстракцией, которая применяется для моделирования системы в целом: приложений, серверов, баз данных и чего бы то ни было еще, что служит частью системы. Системная архитектура для систем БД прошла извилистый путь развития за три последние десятилетия. Ранние иерархические БД и БД с плоскими файлами преобразованы в сетевые совокупности указателей отношений объектов и комбинации всего этого. Такие модели данных полностью совместимы с более медленно развивающейся системной архитектурой базы данных. Архитектура прошла путь от простых внутренних моделей к сетевой модели 60-х годов прошлого века CODASYL DBTG (Конференция по языкам обработки данных рабочей группы по базам данных) [CODASYL DBTG, 1971], через трехсхемную архитектуру ANSI/SPARC (Американский национальный институт стандартов/Комитет по требованиям и планированию стандартов) 70-х годов [ANSI, 1975] до многоуровневых клиент/серверных моделей и моделей распределенных объектов 80-х и 90-х годов XX века. Мы ни в коей мере не приблизились к завершению истории создания архитектуры БД, и то, к чему приведет использование объектов, скрывается в тумане будущего.

Архитектура данных, архитектурная абстракция, используемая для моделирования устойчивых данных, представляет собой второй компонент разработки БД. Хотя имеются и другие системы управления БД, в данной книге рассматриваются три наиболее популярных типа: реляционная (RDBMS), объектно-реляционная (ORDBMS) и объектно-ориентированная (OODBMS). Архитектура данных предоставляет не только структуры (таблицы, классы, типы и т. д.), необходимые при разработке БД, но также язык для описания поведения и бизнес-правил или ограничений.

Современная разработка БД не просто отражает лежащую в их основе выбранную системную архитектуру, но сама суть ее зависит от выбранных архитектурных решений. Принятие решений в области создания архитектуры занимает такое же место в жизни разработчика, как и изображение сущностей и отношений или преодоление трудностей SQL, стандартизованного языка реляционной БД. Начнем с архитектуры, прежде чем перейти к рассмотрению непосредственной проблемы — проектирования.

Виды системной архитектуры

Системная архитектура является абстрактной структурой объектов и отношений, образующих систему. Системная архитектура БД отражает объекты, образующие систему ПО с централизованной базой данных. Такие объекты включают в себя прикладные компоненты и их представления о данных, уровни базы данных (часто называемые *серверной архитектурой*) и *промежуточное ПО* (ПО, соединяющее клиенты с серверами и добавляющее возможности по мере необходимости), которое устанавливает связь между приложением и БД. Любая архитектура содержит объекты и отношения между ними. Архитектурные различия зачастую обусловлены этими отношениями.

Изучение истории и теории системной архитектуры очень много дает разработчикам БД. В данной книге описаны средства создания архитектуры, которые автор использовал в своей практике. Дойдя до конца главы, читатели смогут различать основные архитектурные элементы своих проектов. Можно еще более развить понимание разработки, углубив изучение системной архитектуры с помощью других источников.

Трехсхемная архитектура

Самой ранней и влиятельной попыткой создать стандартную системную архитектуру явилась архитектура ANSI/SPARC [ANSI, 1975; Date, 1977]. ANSI/SPARC выделяет в системах с центральной базой данных три модели: внутреннюю, концептуальную и внешнюю (см. рис. 2.1). *Схема* – это описание модели (метамодель). Любая схема имеет структуры и отношения, отражающие ее роль. Цель состояла в том, чтобы сделать все три схемы независимыми друг от друга. Архитектура реализуется в системах, устойчивых к изменениям физических или концептуальных структур. Вместо необходимости перестраивать всю систему целиком при любом изменении структуры хранения, можно просто изменить структуру, что не повлияет на использующую ее систему. Эта концепция *независимости данных* была важна в ранние годы применения систем управления БД и их разработки, она все еще важна и сегодня. Это основа всего, что делают разработчики БД.

Например, представьте себе, какой могла бы быть бухгалтерская система, не обладающая независимостью данных. Всякий раз, когда разработчику приложения потребуется получить доступ к главной учетной книге, он должен будет писать код для доступа к данным на диске, определяя секторы диска и форматы памяти аппаратных средств. Ему придется следить за индексами и использовать их, адаптируясь к "оптимальным" структурам памяти, отличающимся для каждого типа элементов данных, кодировать логику, метод доступа к подмножествам данных и программы сортировки для их упорядочения (опять с помощью индексов и средств промежуточного хранения, если данные не могут быть размещены в оперативной памяти). Затем приступит к работе инженер БД и все переделает. В результате

систему для управления новыми структурами. Без уровней инкапсуляции и независимости, предоставляемых системой управления базой данных, программирование больших БД было бы невозможным.

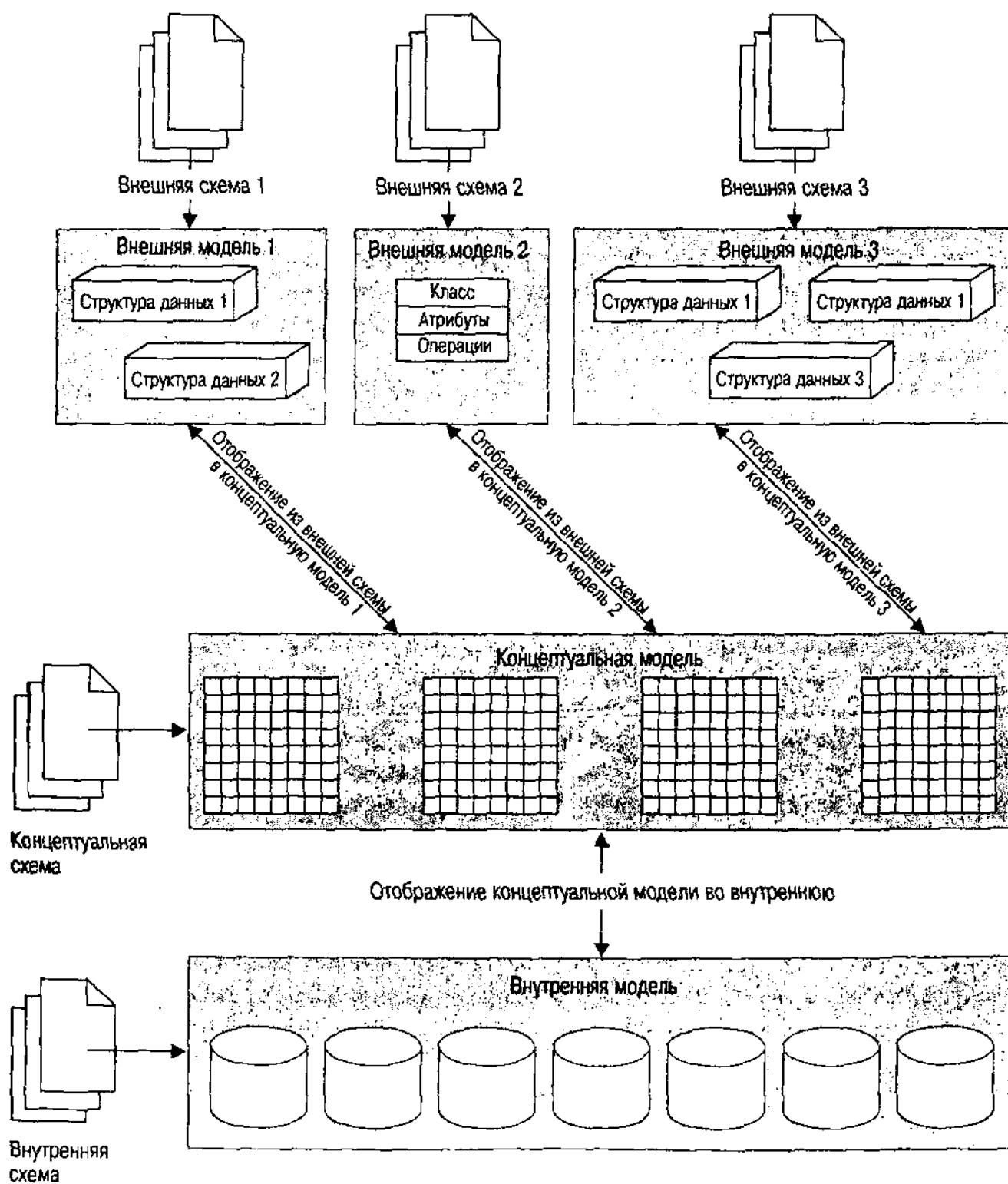


Рис. 2.1. Архитектура ANSI/SPARC

программист решил хранить данные в плоских файлах, вместо того чтобы ускорить доступ. Результат: система, которая связывает свои фундаментальные структуры данных непосредственно с физическими файлами в памяти. В случае, если немного изменится приложение или файлы с данными превысят текущие размеры, компания вынуждена будет полностью переделать свои подпрограммы доступа к данным, чтобы включить структуры данных новых файлов.

ВНИМАНИЕ

Чтобы прояснить ситуацию, отметим, что в рассмотренном примере использована реляционная БД для случая, требующего определения пути доступа. Замена реляционного проектирования объектно-ориентированным было бы лучшим решением. Инженеры в этой маленькой компании никак не использовали ООП и лишь изредка — реляционную технологию создания БД. Отсутствие знаний очень сильно затруднило понимание ими допущенных компромиссов.

Многозвенные архитектуры

В 80-е годы XX века появились персональные компьютеры и совсем небольшие серверные машины, а также локальные сети, объединившие их. Эти технологии сделали возможным распределение компьютерной обработки на нескольких машинах, а не осуществление ее на одной большой центральной машине или мини-компьютере. Сначала эта архитектура имела вид *клиент/серверных вычислений*, где сервер БД поддерживал несколько клиентских машин. Со временем это превратилось в *распределенную архитектуру клиент/сервер*, в которой несколько серверов совместно образуют распределенную БД.

В начале 90-х годов эта архитектура изменилась еще больше после появления концепции *декомпозиции прикладного обеспечения*, детализации основного клиент/серверного подхода. Имея сервер БД, можно выполнить часть приложения на стороне клиента, а другую часть — на прикладном сервере, используемом несколькими клиентами. Одной из известных форм этой архитектуры является *архитектура монитора обработки транзакций (ТР)*, когда управление транзакциями осуществляет промежуточное ПО. Сервер базы данных считает ТР-монитор клиентом, а ТР-монитор в свою очередь обслуживает своих клиентов. Другие типы промежуточного ПО появились для предоставления различной поддержки приложений. Такая архитектура называется *трехзвенной*.

В конце 90-х годов эта архитектура снова трансформировалась в связи с появлением Интернет-браузеров тонких клиентов, промежуточного ПО распределенных объектов и других технологий. Теперь стало возможным перевести еще большую часть обработки с клиента на сервер и распределить объекты на многих машинах. Это привело к созданию *многозвенной архитектуры распределенных объектов*.

Архитектуры многозвенных систем существенно повлияли на системные и сетевые аппаратные средства, а также на ПО [Berson, 1992]. Однако данная книга уделяет основное внимание программным аспектам этой архитектуры. Существенное влияние системной архитектуры на результаты разработки происходит из системной архитектуры ПО, что является предметом обсуждения остальной части данного раздела.

Серверы баз данных: архитектура типа клиент/сервер

Архитектура типа клиент/сервер [Berson, 1992] выделяет в системе две части: ПО, выполняемое на сервере, отвечает на запросы многих клиентов, выполняющих другую часть ПО. Основной целью архитектуры типа клиент/сервер является снижение количества данных, проходящих по сети. При помощи стандартного файлового сервера во время доступа к файлу происходит копирование всего файла через сеть в систему, которой требуется получить к нему доступ.

Архитектура типа клиент/сервер позволяет разработчику структурировать как запрос, так и ответ с помощью серверного ПО, что, в свою очередь, дает возможность серверу посылать в ответ только необходимые данные. Рис. 2.2 иллюстрирует классическую схему клиент/сервер с системой управления БД в качестве сервера и приложения БД в качестве клиента.

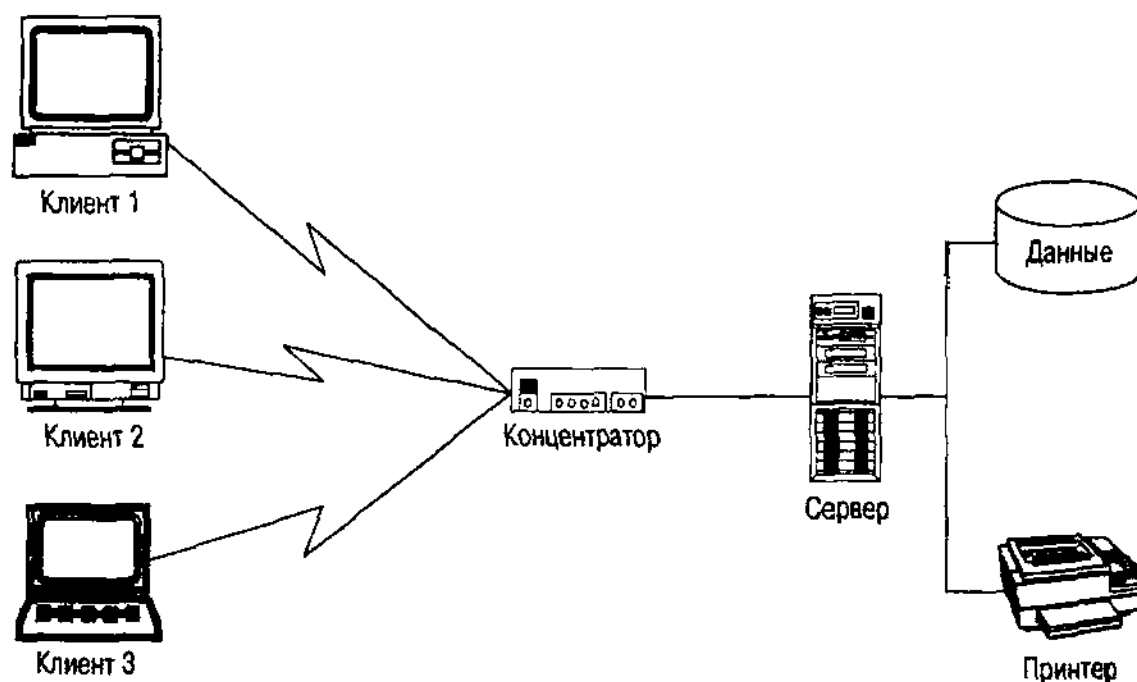


Рис. 2.2. Архитектура клиент/сервер

На самом деле архитектуру ПО можно разбить на уровни и распределить уровни различными способами. Один из них разбивает ПО на три части, например: представление, обработка бизнес-процессов и управление данными [Berson, 1992]. Так, система X-Windows является в чистом виде клиент/серверной системой уровня представления. X-терминал — это система ПО на основе клиента, которая выполняет программы уровня представления и посылает запросы серверу, производящему бизнес-обработку. Программное обеспечение X-терминала делает терминал интеллектуальным.

Более современный пример — браузер "Всемирной паутины", соединенный с сетью и управляющий данными уровня представления, которые он запрашивает на Web-сервере. Web-сервер работает как клиент сервера БД, который может выполняться или нет на том же аппаратном средстве. Пользователь взаимодействует с Web-браузером, посылающим запрос на Web-сервер

на любом языке программирования или языке сценариев, используемом на этом сервере. Web-сервер затем подключается к БД и посылает запрос SQL, осуществляет удаленные вызовы процедур (RPC) или делает то, что требуется для обслуживания базы данных, а сервер БД отвечает действиями на базе данных и/или данными. После этого Web-сервер выводит результаты на экран дисплея с помощью Web-браузера (рис. 2.3).

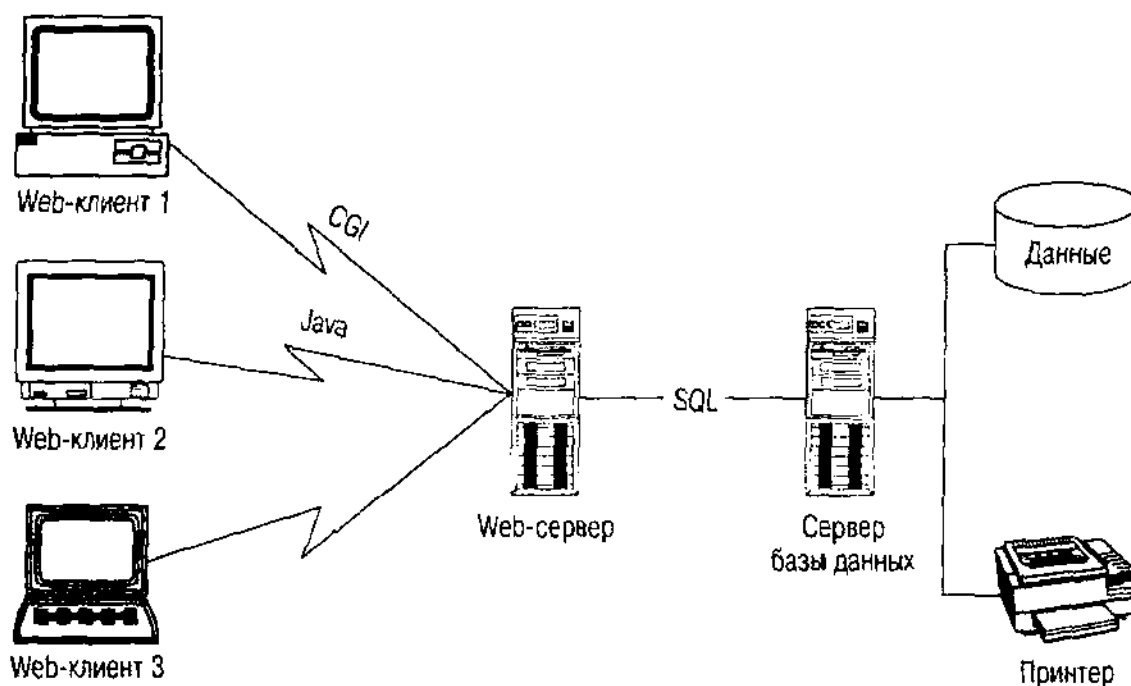


Рис. 2.3. Система клиент/сервер на базе Web

Web-архитектура иллюстрирует распределение бизнес-обработки между клиентом и сервером. Это делают в том случае, когда одни элементы бизнес-обработки имеют интенсивный доступ к БД, а другие – нет. С помощью размещения части с интенсивным доступом на сервере БД можно снизить поток информационного обмена и воспользоваться преимуществами инкапсуляции относящегося к базе данных кода в одном месте. К преимуществам данного подхода можно отнести также большую безопасность БД, клиентские интерфейсы более высокого уровня, чем поддерживались раньше, разработку целостных подсистем на стороне сервера. Web предоставляет один подход к подобному распределению обработки, но это не единственная возможность. Такой подход неизбежно приводит к архитектуре управления обработкой транзакций, упоминавшейся ранее, в которой программное обеспечение ТР-монитора является промежуточным звеном между БД и клиентом. Если ТР-монитор и база данных работают на одном и том же сервере, это и есть архитектура клиент/сервер. Если они находятся на разных серверах, получится многозвенная архитектура (см. рис. 2.4). *Разбиение приложения* – это процесс разбиения прикладного ПО на модули, которые выполняются на разных клиентах и серверах.

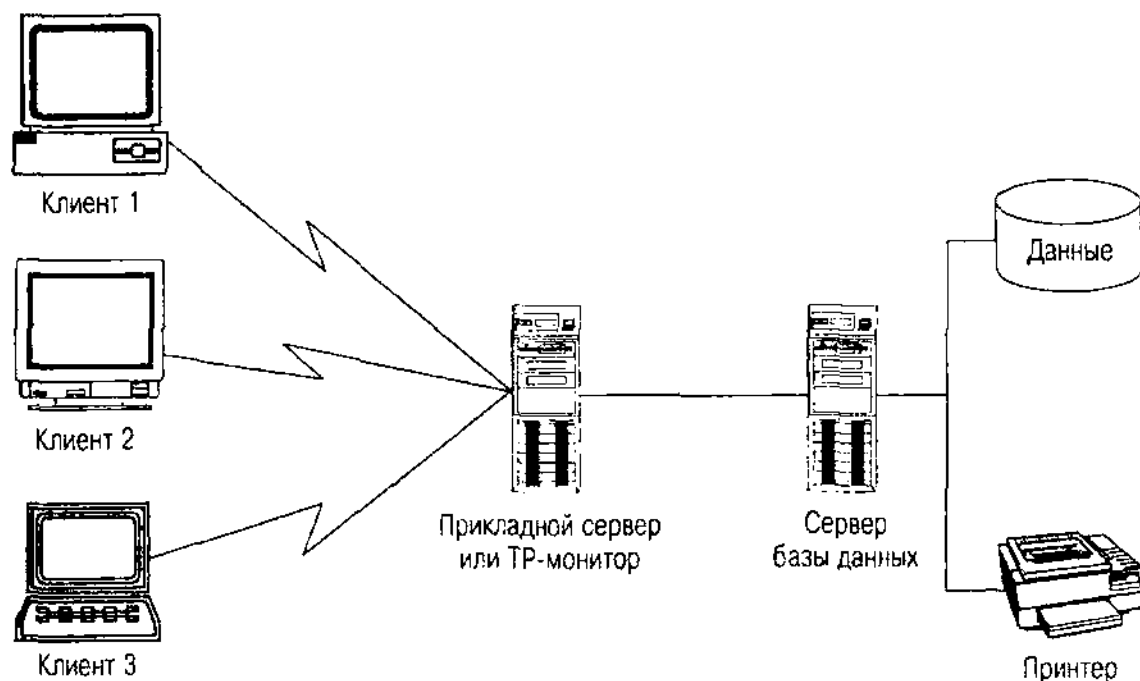


Рис. 2.4. Разбиение приложения в системе клиент/сервер

Распределенная архитектура баз данных

Одновременно с разработкой реляционных БД идет разработка распределенных БД, данные размещаются в географически распределенной сети, связанной коммуникационными линиями [Date, 1983; Ullman, 1988]. На рис. 2.5 приведен пример архитектуры распределенной БД с двумя серверами, тремя базами данных, несколькими клиентами и рядом локальных баз данных на клиентах. Таблицы со стрелками показывают наличие репликации таблиц, существующих на нескольких серверах, которые их автоматически синхронизируют.

ВНИМАНИЕ

Хранилища данных часто инкапсулируют архитектуру распределенной БД, особенно если они создаются путем ссылок на данные, копирования и/или агрегации данных из нескольких БД в хранилище данных. Получение статических множеств, например, позволяет извлечь данные из таблицы и скопировать их на другой сервер для использования там; исходная таблица меняется, а статическое множество — нет. Хотя в данной книге и не рассматриваются подробно проблемы разработки хранилищ данных, архитектура распределенных БД и ее влияние на проектирование часто позволяют решить большинство проблем, связанных с созданием хранилищ данных.

Распределенная БД имеет три рабочих элемента: прозрачность, управление транзакциями и оптимизация.

Прозрачность распределенной базы данных — это степень, в какой функционирование БД проявляется при работе на единственной унифицированной базе данных с точки зрения ее пользователя. В полностью прозрачной системе приложение видит только стандартную модель данных и интерфейсы и ему не нужно знать, где на самом деле осуществляются

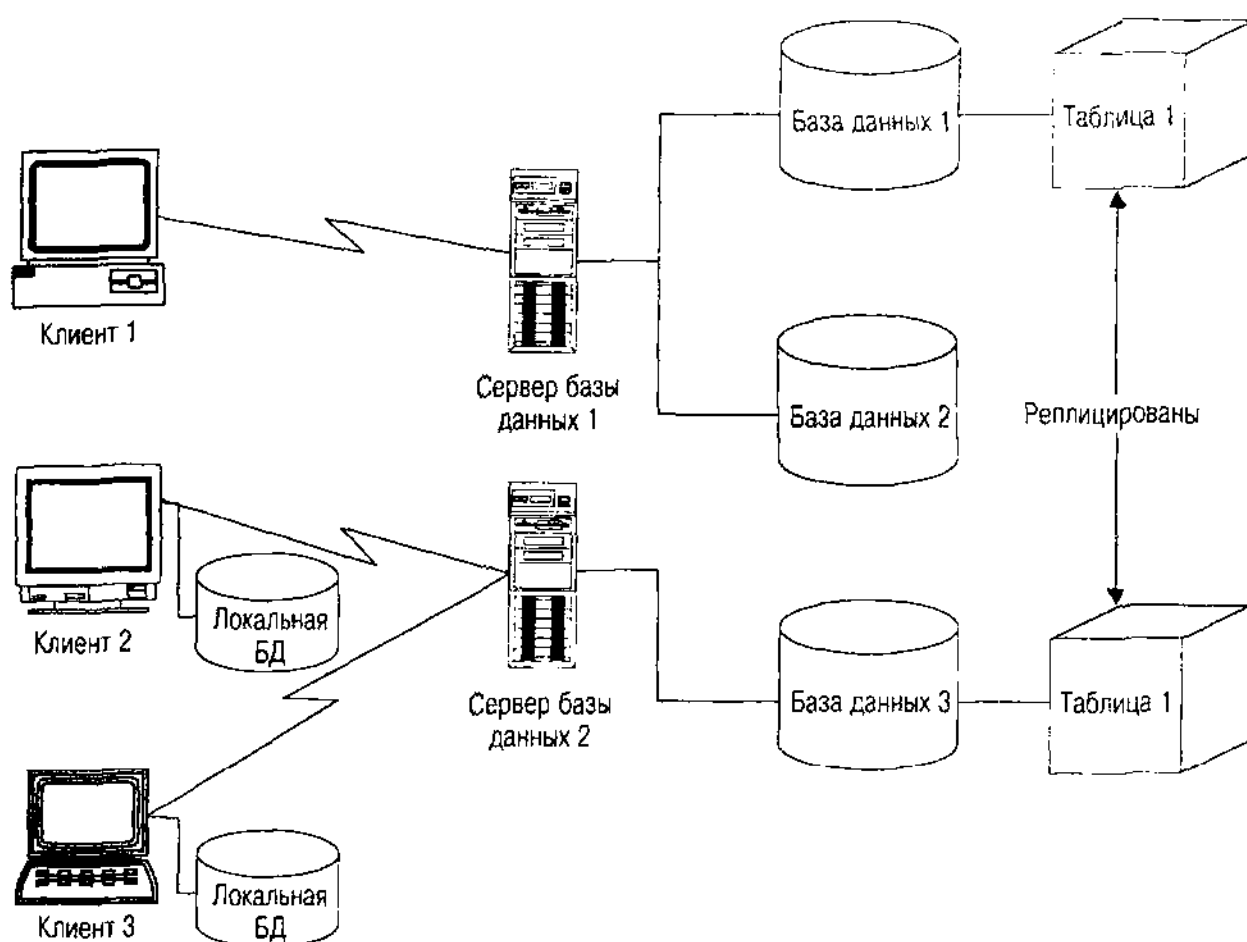


Рис. 2.5. Архитектура распределенной базы данных

действия. Никогда не требуется предпринимать каких-то специальных действий, чтобы получить доступ к таблице, осуществить транзакцию или установить соединение. Например, запрос получает доступ к БД на нескольких серверах, менеджер запроса должен разбить запрос на отдельные запросы к каждому серверу, а затем объединить результаты (см. рассмотренный ниже процесс оптимизации). Приложение запускает на выполнение один оператор SQL, но на серверах на самом деле выполняются несколько операторов. Другой аспект прозрачности — это *фрагментация*, распределение данных в таблице во многих местах (по-другому это называется *разбиением*). Большинство распределенных систем достигают разумного уровня прозрачности вплоть до уровня администратора БД. Затем они отказываются от прозрачности, чтобы облегчить жизнь бедному администратору, который должен управлять всеми последующими проблемами распределения данных и поведения. Недостаток, касающийся прозрачности, связан с гетерогенной распределенной БД, т. е. базой данных, включающей в себя ПО различных систем управления БД, работающее на разных серверах.

ВНИМАНИЕ

Фрагментация БД не имеет никакого отношения к файловой фрагментации — режиму, который создается в файловых системах, таких, как DOS или NTFS, когда сегменты, входящие в состав файлов, распределяются случайным образом на диске вместо того, чтобы сгруппироваться вместе. Рекомендуется еженедельно для повышения производительности дефрагментировать диск. Дефрагментацию БД выполнять не требуется.

Управление транзакциями в системах с распределенной БД отличается от управления транзакциями в системах с единой БД из-за возможности базы данных стать недоступной в процессе завершения транзакций, что в свою очередь приведет к неполному завершению транзакции. Таким образом, для распределенных БД требуется расширение процесса управления транзакциями, способное гарантировать завершение выполнения или полную отмену транзакции. Для осуществления этого имеется множество стратегий [Date, 1983; Elmagarmid, 1991; Gray and Reuter, 1993; Papadimitriou, 1986], наиболее популярные из них: двухэтапное завершение и распределенный оптимистический параллелизм.

При *двухэтапном завершении* обычный процесс завершения транзакций разбивается на две части [Date, 1983; Gray and Reuter, 1993; Ullman, 1988]. Во-первых, распределенные серверы взаимодействуют друг с другом до тех пор, пока все они не будут готовы завершить свои части транзакции. Затем каждый завершает свою часть и информирует остальных об успешном или неуспешном ее завершении. Если все серверы завершат обработку, транзакция завершается успешно; в противном случае система отменяет произведенные на всех серверах изменения. Имеется много тонкостей при администрировании подобных систем, например восстановление утраченных серверов и др.

Оптимистический параллелизм использует противоположный подход [Ullman, 1988; Kung and Robinson, 1981]. Вместо того чтобы обеспечить корректность всего происходящего при обработке транзакции либо с помощью блокировки, либо путем управления маркером времени, оптимистические методы позволяют делать что угодно, а затем по завершении анализируют возникшие конфликты. Используя определенные правила разрешения конфликтов, например сравнение маркеров времени или свойств транзакций, оптимистический подход позволяет избежать ситуаций зависания и допускает значительный параллелизм, особенно в случае операций только на чтение. Oracle7 и Oracle8 имеют версию оптимистического параллелизма, называемого параллелизмом чтения, который допускает доступ на чтение однородной БД вне зависимости от изменений, произведенных с тех пор, как данные были считаны.

Оптимизация распределенных БД — это процесс выполнения оптимизированных запросов на различных серверах. Он требует расширенной оптимизации на основе затрат, учитывающей, где находятся данные, где могут происходить операции и каковы настоящие затраты, связанные с распределением [Ullman, 1989]. В случае, когда менеджер запросов разбивает их на части, например чтобы выполнить их на отдельных серверах, он должен оптимизировать запросы как для выполнения на соответствующих серверах, так и для передачи и приема в сети. Современная технология не очень подходит в

данном случае, и есть хороший выход для того, чтобы сделать автоматическую оптимизацию эффективной. Вывод: проект должен принимать во внимание требования оптимизации, особенно на физическом уровне.

Основное влияние распределенного управления транзакциями на проектирование состоит в том, чтобы применить преимущества разрабатываемого языка при планировании логики транзакций и размещении данных. Прозрачность влияет на это существенным образом: чем меньше приложению нужно знать о происходящем на сервере, тем лучше. Если логика прикладной транзакции прозрачна, приложению не нужно самому заботиться о проблемах разработки, относящихся к управлению транзакциями. Однако проект логической и физической БД потребует принять во внимание распределенные транзакции.

Например, известно, что информационный поток в определенном канале связи будет намного меньше, чем в других. Можно оценить приложения, используя подход цена — выгода для того, чтобы решить, превышает ли локальный доступ потребности удаленного доступа. Для этого используется таблица, содержащая объединение локальных данных из нескольких точек. Каждое локальное местоположение получает преимущества благодаря наличию у него таблицы на локальном сайте. Другие локальные местоположения извлекают выгоду из того, что имеют удаленно созданные данные на своем сайте. Особенно в том случае, когда не все каналы одинаковы, нужно принять решение, какой сервер лучше подходит для всех. Есть и более сложные подходы к решению проблемы. Можно создать отдельные таблицы, переложив проблему разработки на прикладной язык, который должен их перекомбинировать. Можно реплицировать данные, сделав проблему разработки заботой администратора БД и разработчиков поставщика. Можно использовать разбиение таблиц, переложив проблему разработки на Oracle8, единственную БД, поддерживающую это средство. Следовательно, влияние оптимизации на разработку является непосредственным и мгновенным, а также довольно трудным, если БД сложна.

Holmes PLC, например, использует Oracle7 и Oracle8 для управления определенными транзакциями распределенной БД. В обеих системах полностью внедрен протокол двухэтапного завершения транзакций относительно прозрачным способом как на стороне клиента, так и на сервере. Есть два важных момента: где физический проект должен реализовать требования прозрачности и административный интерфейс. Oracle реализует распределенные серверы через стратегию связывания, при которой объект связи одной схемы ссылается на строку соединения удаленной БД. В результате при ссылке на таблицу удаленного сервера, чтобы найти таблицу, нужно определить имя канала. Если ссылка должна быть прозрачной, можно использовать один из трех подходов. Следует ввести синоним, инкапсулирующий имя канала, сделав его общедоступным или частным для определенного пользователя или роли Oracle. Альтернативный вариант — реплицировать таблицу, разрешив локальное управление транзакциями со скрытыми затратами для конечного пользователя в связи с согласованием копий. Либо можно определить хранимые процедуры и триггеры, которые инкапсулируют ссылки канала, с затратами, переходящими на поддержку процедур на различных серверах.

Как видно из примера, архитектура распределенных БД в наибольшей степени влияет на проектные решения, особенно на физическом уровне. Исключительно важно учитывать это в случае выбора распределенной БД.

Объекты повсюду: многозвенная архитектура распределенных объектов

По мере роста популярности ООП концепция распределения таких объектов оказалась в центре внимания. Если можно разделить приложения на части, выполняемые на различных серверах, то почему бы не разбить ОО-приложения на отдельно выполняемые объекты на этих серверах? Рабочая группа по развитию стандартов объектного программирования (OMG) определила эталонную модель объектов и множество стандартных моделей для обобщенной архитектуры посредника объектных запросов (CORBA) [Soley, 1992; Siegel, 1996]. Конкурентом этого отраслевого стандарта является распределенная модель с общими объектами (DCOM) и различные средства доступа к БД, такие, как удаленные объекты данных (RDO), объекты доступа к данным (DAO), база данных с компоновкой и внедрением объектов (OLE DB), активные объекты данных (ADO) и ODBC-Direct [Baarns, 1997; Lassesen, 1995], часть архитектуры Active X компании Microsoft и Open Group, аналогичный стандарт для распределенных

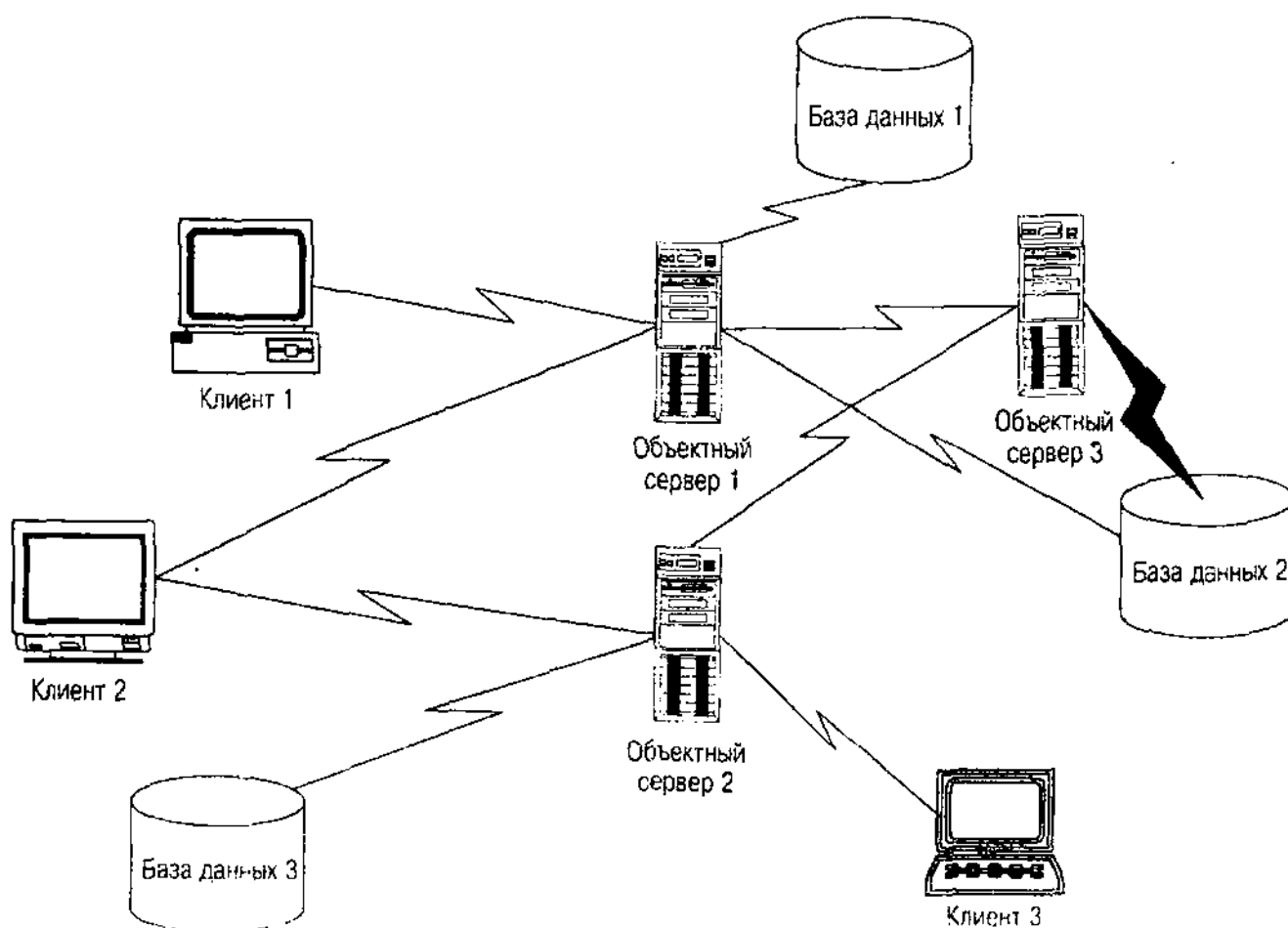


Рис. 2.6. Архитектура клиент-сервер

объектов на серверах в сети [Chappell, 1996; Grimes, 1997; Lee, 1997]. Эта модель постепенно начинает переходить в новую модель Microsoft COM+ или COM 3 [Vaughan-Nichols, 1997]. Каковы бы ни были "за" и "против" различных видов создания ссылочных архитектур [Mowbray and Zahavi, 1995, p. 135–149], все они влияют на проектирование БД одинаково: позволяют скрыть доступ к БД внутри объекта, а затем поместить эти объекты на серверы вместо размещения в клиентских приложениях. Это приложение затем получает данные от объектов по требованию через сеть. На рис. 2.6 показана типичная архитектура распределенных объектов с использованием CORBA.

ОСТОРОЖНО

Данная область технологии разработки ПО определенно не для слабонервных, что становится понятным после беглого просмотра нескольких последних страниц. В частности, компания Microsoft внесла свой вклад в создание исключительно запутанного множества технологий (и аббревиатур их названий) за последние несколько лет. Хотите познакомиться с методами доступа к данным компании Microsoft? Выбирайте между MFC, DAO, RDO, ADO, или добрым старым ODBC, или используйте все их вместе. Автор вынужден высказать свое мнение: он думает, что Microsoft усложняет больше, чем необходимо, разработку приложений БД с помощью всей этой чепухи. Из-за неразберихи, вызванной множеством технологий и способов их использования и навязывающей путанные представления поставщика о проблемах проектирования приложений БД, сам разработчик оказывается в затруднительном положении.

Если подойти с позиций здравого смысла (см. рис. 2.6), то, размещая их на одном уровне, архитектура распределенных объектов превращает БД и ее содержимое в нечто подобное прикладным объектам. БД становится просто еще одним объектом, осуществляющим связь через распределенную сеть. Эта прозрачность объекта имеет некоторое влияние на разработку БД. Часто прослеживается тенденция выполнить разработку системы с присвоением ведущей роли либо БД, либо приложению. В системах распределенных объектов ни один из компонентов не является лидирующим все время. Когда БД считается в большей степени компонентом, реализующим кооперацию, а не является основой системы или дополнительным хранилищем устойчивых данных, обнаруживаются и другие способы использования и получения данных. Вместо применения единственной DBMS и ее серверов можно объединить несколько продуктов DBMS, комбинируя даже систему с объектно-ориентированной базой данных с реляционной БД, если в этом будет смысл. Вместо последовательности прикладных моделей данных, отображаемых в концептуальную модель, как в архитектуре ANSI/SPARC, можно будет увидеть последовательности объектных моделей, отображаемых в последовательности концептуальных моделей через распределенные сети.

ВНИМАНИЕ

Некоторые адвокаты OODBMS хотели бы дать понять разработчикам, что основное преимущество объектно-ориентированного программирования состоит в том, что БД пропадает. Честно говоря, это чепуха. При определенных обстоятельствах и в специальных случаях разработчик может не заботиться о том, находится ли объект в основной памяти или в БД. Если просматривать код, который не использует базу данных, и тот код, который использует ее, можно видеть существенную разницу между ними вне зависимости от примененной технологии. БД никогда не исчезает. Полезнее рассматривать БД как равноценный объект, с которым будет работать код, а не как невидимый подчиненный робот, занятый тяжелым трудом в застенках.

Например, в разрабатываемом приложении имеется требование создать структуру дерева (последовательность предков и потомков, что-то вроде генеалогического древа). Тот, кто проектировал исходную реляционную БД, вошедшую в текущую разработку, представили эту структуру в базе данных как таблицу пар предок — потомок. Один столбец таблицы был предком, а другой — одним из его потомков, т.е. каждый ряд представлял связь между двумя элементами дерева. Клиент определяет корень или точку входа дерева, а затем приложение создает дерево на основании определения пути от этого корня по связям предок — потомок.

Если разработка ведется с использованием подхода на основе приложения, необходимо определить способ сохранения дерева в БД. Например, это может означать наличие специальных таблиц для каждого дерева или даже больших двоичных объектов для сохранения деревьев из основной памяти для быстрого получения их данных. Если разработка происходит на основе централизованной БД, тогда в память просто извлекается таблица связей и из нее строится дерево на базе алгоритма создания графа. В качестве альтернативного варианта можно использовать для извлечения данных в форме дерева такие специальные инструменты БД, как предложение Oracle CONNECT BY.

Осуществляя разработку с точки зрения распределенного объекта, проектировщик создал подсистему в базе данных, которая запросила из базы данных необработанную информацию. Из нескольких запросов эта подсистема сформировала полную базу для дальнейшего анализа. Клиентский объект запрашивал затем эти данные, используя предложения ORDERED BY и WHERE для получения только запрашиваемой информации в необходимом формате. Этот подход представляет собой кооперативный подход к разработке системы на основе распределенных объектов, а не подход, определяющий БД или приложения основной движущей силой проектирования.

Другое приложение имело две БД — одна была хранилищем изображений, а другая — стандартной реляционной базой данных, их описывающей. Приложение использовало стандартную трехзвенную клиент/серверную модель с двумя отдельными серверами БД — один для системы управления документами, а другой — для реляционной БД, а также код на стороне клиента и на сервере для перемещения данных в нужное место. Использование архитектуры распределенных объектов позволило бы получить более гибкую компоновку. Серверы БД могли бы представлять собой кэши объектов.

доступные любому опознаваемому клиенту. Этот архитектурный стиль позволил бы разработчикам создать объектные серверы для перемещения данных между двумя БД и множеством клиентов.

Архитектура управления объектами OMG (OMA) [Soley, 1992; Siegel, 1996] служит стандартным примером разновидности программных объектов, которые можно встретить в архитектурах распределенных объектов (рис. 2.7). Open Group Architectural Framework [Open Group, 1997] содержит другие примеры инфраструктур для создания таких архитектур.

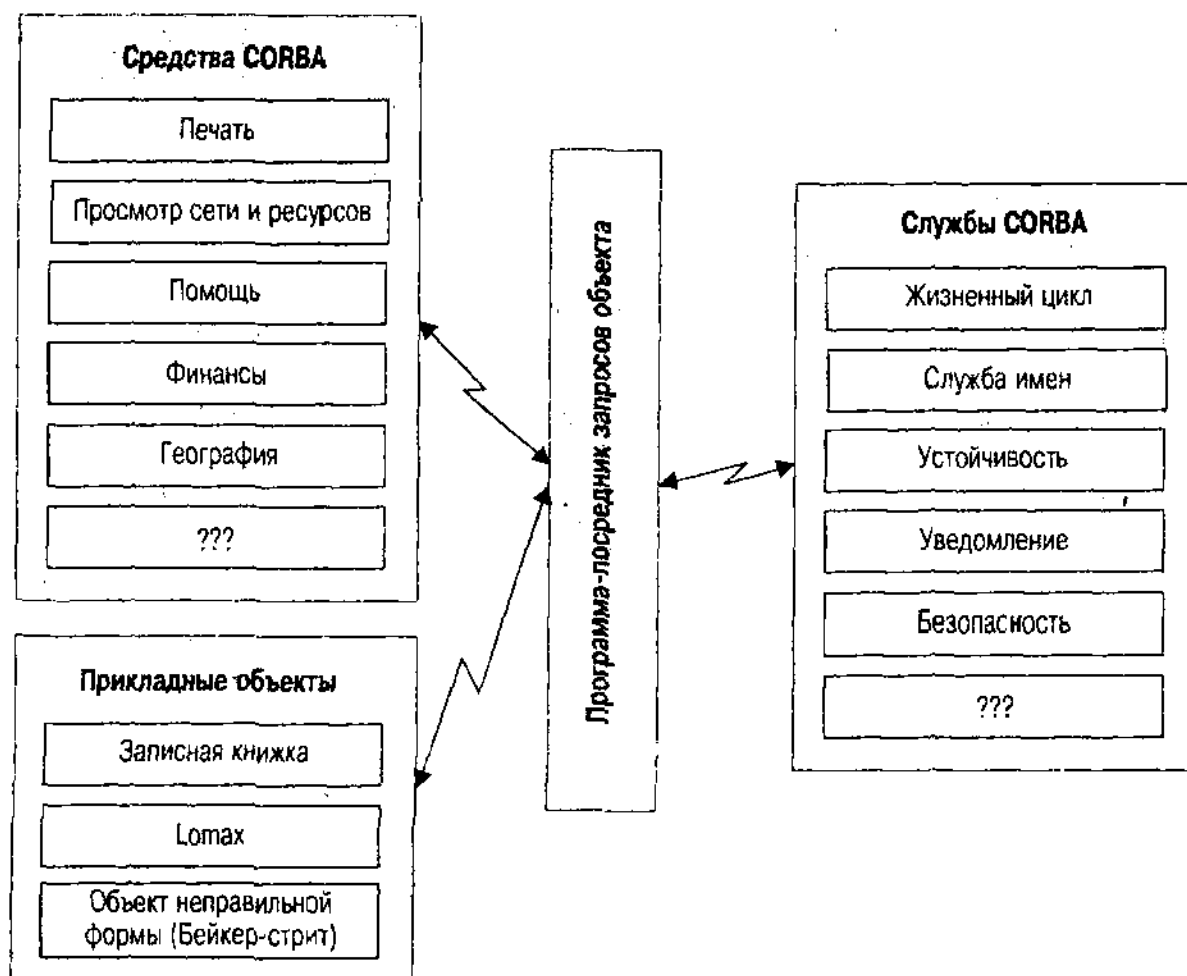


Рис. 2.7. Архитектура управления объектами группы управления объектами

Уровень служб CORBA предоставляет инфраструктуру для создания архитектурных блоков, обеспечивающих всем необходимым инструментарием для создания и управления объектами. Служба жизненного цикла управляет разработкой, перемещением, копированием и утилизацией. Служба имен предназначена для управления уникальными именами объектов в сети (это главная служба, которая долго была узким местом сетевых служб под именем службы каталогов). Служба устойчивости предоставляет постоянную или временную память для объектов, включая объекты, которые CORBA использует для управления прикладными объектами.

Уровень программы-посредника запросов объекта (ORB) предоставляет основные средства связи для диспетчеризации сообщений, маршализации данных в разнородных машинных архитектурах, активации объектов, обработки исключений и безопасности. Он также интегрирует основные взаимодействия в сети с помощью реализации протокола TCP/IP или уровня распределенной вычислительной среды (DCE).

Уровень средств CORBA предоставляет объекты предметной области как по горизонтали, так и по вертикали. Горизонтальные средства — это объекты для управления определенными видами поведения приложений, такими, как пользовательский интерфейс, просмотр сети и ресурсов, печать, работа с электронной почтой, составными документами, управление системой и т. д. Вертикальные средства предоставляют решения для определенных видов отраслевых приложений (финансы, здравоохранение, производство и т. д.).

Уровень прикладных объектов состоит из коллекций объектов в отдельных приложениях, использующих шину ПО CORBA для взаимосвязи со средствами и службами CORBA. Это может быть сведено к минимуму, обеспечивающему графический пользовательский интерфейс для средства, или быть таким полным, что сможет обеспечить разработку всего спектра взаимодействующих объектов для конкретного сайта.

Где же во всем этом место БД? Она может быть где угодно, как уже упоминаемая Годзилла. БД входят в постоянные службы CORBA; они обычно являются объектно-ориентированными БД, такими, как POET, ObjectStore или Versant/DB. Они могут также быть горизонтальными средствами CORBA, предоставляющими память для средств управления определенного рода, или вертикальным средством, предлагающего постоянную память для финансовых или производственных данных. Они даже могут быть прикладными объектами, как локальная БД того или иного типа. Эти объекты работают через адаптеры объектов уровня ORB, например базовый объектный адаптер или адаптер объектно-ориентированной БД [Siegel, 1996; Cattell and Balgu, 1997]. Эти компоненты активизируют и деактивизируют БД и ее объекты, отображают объектные ссылки и управляют безопасностью через средства безопасности OMG. Все они являются в архитектуре равноправными объектами, взаимодействуя друг с другом через ORB.

В качестве примера рассмотрим БД изображений и фактов, поддерживаемую агентством Holmes PLC, а именно обычную систему учета преступлений. Эта БД содержит изображения и текст, относящиеся к преступникам, информационным ресурсам и другим объектам, которые могут представлять интерес для проводимой консультативной детективной работы по всему миру. Хотя агентство Holmes PLC могло бы создать свою БД исключительно в пределах объектно-реляционной или объектно-ориентированной DBMS (и некоторые примеры из данной книги используют такие реализации в качестве примеров), архитектура распределенных объектов дает Holmes PLC большую степень гибкости при организации БД, особенно в области безопасности и производительности на своих серверах по всему миру. Это позволяет им объединить специализированную систему управления документами, содержащую фотографии и изображения документов, с объектно-ориентированной БД,

хранящей данные об отпечатках пальцев и ДНК, а также включить сюда реляционную БД, содержащую сведения об объектах со сложной конфигурацией: от данных о людях до информации о местах и событиях (судебные процессы, статус тюрьмы и т. д.).

Резюме по системной архитектуре

Системная архитектура на высшем уровне предоставляет контекст для разработки БД. Этот контекст так же разнообразен, как и системы, его образующие. В данном разделе мы пытались представить архитектурные решения, наиболее ощутимо влияющие при разработке БД на основные свойства и размещение:

- Трехзвенная архитектура привносит концепцию *независимости данных*, разделяя концептуальное, физическое и прикладное представления. Независимость данных является принципом, на котором основывается разработка современных БД.
- Клиент/серверная архитектура предлагает *разделение* приложения на клиентскую и серверную части, некоторые из них располагаются на сервере или даже в БД. Это может повлиять как на концептуальную, так и на физическую схемы, которые должны принимать во внимание разделение для обеспечения лучшей безопасности, доступности и производительности.
- Архитектура распределенных БД непосредственно влияет на физическое размещение базы данных через требования *фрагментации* и *параллелизма*.
- Архитектура распределенных объектов влияет на все уровни разработки БД, повышая (или понижая, в зависимости от точки зрения) ее статус до сущности, *равноправной* с приложением. Обращение с БД (и потенциально с несколькими разными БД), как со взаимодействующими объектами, требует другой стратегии создания структур данных. Разработка улучшается благодаря снижению связности структур БД, снова возвращаясь к концепции независимости данных.

Архитектуры данных

Системная архитектура — это сцена для разработчика; архитектура данных предоставляет сценарий и реплики, которые разработчик выводит на сцену. Имеется три основных вида архитектуры данных, в настоящее время претендующих на внимание разработчиков БД: реляционные, объектно-реляционные и объектно-ориентированные модели данных. Необходимость выбора одной из этих моделей влияет на любой аспект проектируемой системной архитектуры:

- Язык доступа к данным
- Структуру и отображение интерфейса между приложением и БД

- Схему концептуального проекта
- Схему внутреннего проекта

Действительно, невозможно переоценить последствия выбора архитектуры данных системы. Однако вполне возможно ограничить эти последствия определенными рамками. Одна гипотеза, имеющая многих сторонников в среде специалистов в области вычислительной техники, утверждает, что целью разработчика должна быть подгонка системной архитектуры и инструментария к модели данных: это гипотеза *импедансного несоответствия*. Если архитектура данных идет вразрез с системной архитектурой, продуктивность разработки значительно уменьшится, потому что потребуется все время строить для них соответствия и интерфейсы. Например, можно было бы использовать архитектуру распределенных объектов для проектируемого приложения, а не реляционную БД.

На самом деле все происходит иначе. При наличии адекватного проекта и аккуратной системной структуризации можно скрыть почти все, включая кухонную раковину. Современный пример — стандарт связности базы данных Java (JDBC) для доступа к базам данных из языка Java. JDBC — это набор классов Java, который представляет собой объектно-ориентированную версию стандарта ODBC, первоначально разработанного для использования в языке C. Стандарт JDBC представляет надежную ОО-разработку для мира Java. В основе он может иметь несколько разных форм. Исходный подход состоял в написании интерфейсного уровня для драйверов ODBC, скрыв лежащую в его основе функциональную природу интерфейса БД. Для сохранения производительности возник более прямой подход, заменивший драйвер ODBC собственными драйверами JDBC. Таким образом, на уровне программного интерфейса все было в порядке. К сожалению, основная функция JDBC состоит в извлечении реляционных данных в результирующие реляционные множества, а не управление объектами. Значит, по-прежнему имеется импедансное несоответствие между реализованным на Java полностью ОО-приложением и реляционными данными, которые оно использует.

Сам автор не считает эту проблему слишком серьезной. Написание апплета JDBC не очень сложно, и дополнительная разработка, необходимая для развития методов управления реляционными данными не потребует больших усилий при разработке или программировании. Основой продуктивности при программировании БД является способность языка разработки выразить все, что потребуется. Более трудно постоянно писать новые доработки кода построения дерева на C++ или Java, чем использовать расширение CONNECT BY в Oracle для стандартного SQL. Но, если дерево имеет в своем составе циклы (когда потомок связан обратной связью со своими предками на каком-то уровне), CONNECT BY просто не работает. Некоторые ненавидят необходимость "связывать" SQL с программами при помощи повторяющихся отображающих вызовов ODBC или других API. Использование JSQL или других встроенных стандартов предкомпилятора SQL для того, чтобы скрыть такое отображение с помощью простого синтаксиса ссылок, ликвидирует эту проблему, не отнимая преимуществ применения

SQL высокого уровня вместо языков низкого уровня C++ или Java для запросов БД. Как обычно, соответствие инструментов потребностям приводит к разным решениям в различном контексте.

Оставшаяся часть данного раздела представляет три основные парадигмы архитектуры данных. Мы намерены суммировать основные структуры по каждому виду архитектуры БД, которые формируют часть инструментального набора разработчика. Заключительные главы соотносят некоторые проблемы проектирования с определенными частями этих видов архитектуры данных.

Реляционные базы данных

Реляционная модель данных возникла в плодотворной работе Эдгара Кодда, опубликованной в 1972 г. [Codd, 1972]. Основная идея Кодда состояла в том, чтобы применить концепцию математических отношений к моделированию данных. Отношение — это таблица из строк и столбцов. Рис. 2.8 представляет простую реляционную структуру, в которой многие таблицы соотносятся друг с другом при помощи отображения значений данных

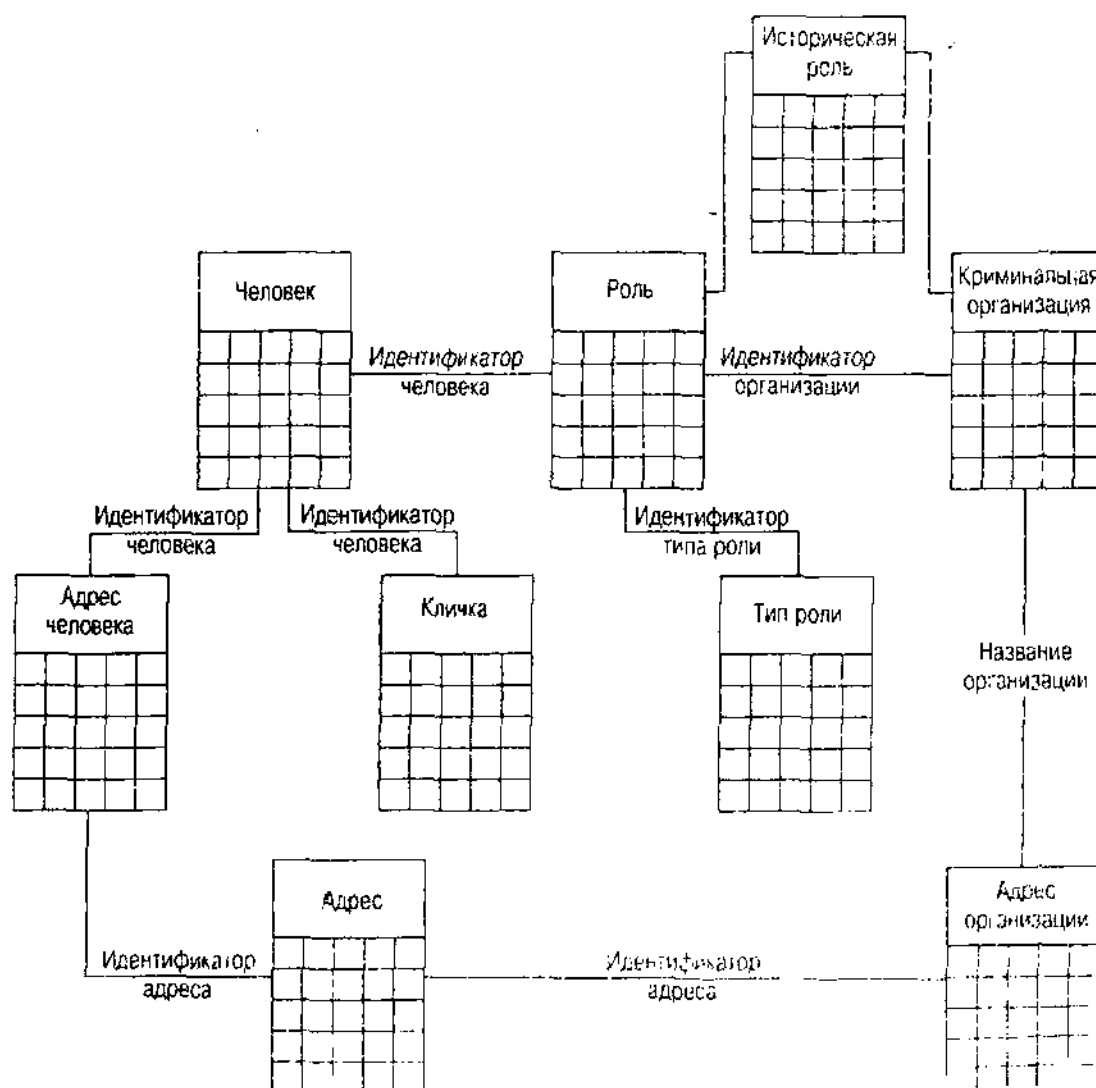


Рис. 2.8. Реляционная схема: БД криминальной сети агентства Holmes PLC

между таблицами, и такие отображения сами являются отношениями. *Ссылочная целостность* — это совокупность ограничений, обеспечивающих корректность отображений между таблицами при завершении транзакции. *Нормализация* — это процесс установления оптимальной табличной структуры, основанной на внутренних зависимостях данных (см. главу 11).

Отношение — это таблица из строк и столбцов. *Отношение* (называемое также *таблицей*) — это конечное подмножество декартова произведения множества *доменов*, каждый из которых является набором значений [Ulman, 1988]. Каждый *атрибут* отношения (называемый также *столбцом*) соответствует домену (*типу столбца*). Таким образом, отношение — это набор *записей* (называемых тоже *строками*). Можно также представить строки отношений как отображение имен атрибутов в значения в доменах атрибутов [Codd, 1970].

Например, таблица "Криминальная организация", представленная на рис. 2.8, имеет пять столбцов:

- *Название организации*: название организации (символьная строка)
- *Юридический статус*: текущий юридический статус организации, поддомен строк, включая "юридически определенный", "на судебном рассмотрении", "подозреваемый", "неизвестный"
- *Стабильность*: насколько стабильна организация, поддомен строк, включая "очень стабильна", "средней стабильности", "нестабильная"
- *Приоритет расследования*: уровень внимания следствия в агентстве Holmes PLC к организации, поддомен строк, включая "пристальное", "текущее", "под наблюдением", "прекращено"
- *Статус обвинения*: текущий статус организации с учетом стратегий уголовного обвинения, применяемых для борьбы с организацией, поддомен строк, включая "история", "на привязи", "туда проникают", "небольшой прогресс", "не продвигается"

Большинство характеристик криминальной организации заключаются в ее отношениях с другими таблицами, такими, как роли, которые играют люди в организации, и различные адреса, связанные с работой организации. Это отдельные таблицы "Адрес организации" и "Роль", связанные с "Названием организации", идентифицирующим организацию в обеих таблицах. С помощью отображения таблиц через "Название организации" можно получить сведения из всех таблиц вместе как ответ на единственный запрос.

Каждый столбец можно ограничить многими способами, включая требование содержать только уникальные значения в каждой строке отношения (*ограничение уникального первичного ключа доступа* или *потенциального ключа поиска*); превращение его в подмножество всего домена (*ограничение домена*), как это сделано с поддоменами в таблице "Криминальная организация"; или ограничение домена как множества значений в строках в другом отношении (*ограничение внешнего ключа*) как ограничение на "Название организации" в таблице "Адрес организации", которое должно присутствовать в таблице "организация". Можно также ограничить несколько

атрибутов совместно, таких, как первичный ключ, состоящий из нескольких атрибутов (например, "Идентификатор адреса" и "Название организации") или условное ограничение по двум или более атрибутам. Можно даже выразить отношения между строками как логические ограничения, хотя большинство продуктов RDMBS и SQL не имеют для этого никаких средств. Другим термином, часто употребляющимся для всех этих типов ограничений, является "бизнес-правило", который, вероятно, основан на способности ограничений выражать правила и основополагающие действия бизнеса.

Эти простые структуры и ограничения на самом деле не касаются основных проблем создания БД, их поддержки и использования. Для этого нужен набор операций на структурах. Благодаря математическому основанию реляционной теории логика предоставляет операции с помощью реляционной алгебры и реляционного исчисления, математических моделей для описания способа доступа к данным в отношениях [Date, 1977; Ullman, 1988]. Несколько поставщиков пытались продавать такие языки; большинство так или иначе проиграло на рынке. К массовому сознанию пробил себе дорогу более простой и легкий для понимания язык SQL.

Язык SQL начинается с определения доменов для столбцов и литералов [ANSI, 1992]:

- Character, varying character и national varying character (строки)
- Numeric, decimal, integer, и smallint
- Float, real, double
- Date, time, timestamp
- Interval (интервал времени, либо год — месяц, либо день — час)

Таблицы создают со столбцами и ограничениями при помощи оператора CREATE TABLE, такие определения изменяют с помощью оператора ALTER TABLE, а удаление таблицы делают с помощью оператора DROP TABLE. Имена таблиц уникальны в пределах схемы (БД, пользователя или любого количества других граничных условий в различных системах).

Наибольшая часть языка SQL — это язык работы с запросами и данными. Оператор SELECT запрашивает данные из таблиц при помощи следующих предложений:

- *SELECT*: Создает список выводимых выражений или "проекцию" запроса
- *FROM*: Определяет входные таблицы и при желании условия объединения этих таблиц
- *WHERE*: Определяет подмножество ввода на основе формы исчисления предикатов первого порядка и также содержит условия объединения, если их нет в предложении FROM
- *GROUP BY* и *HAVING*: Определяют объединение строк вывода и условие выбора в объединенной строке вывода
- *ORDER BY*: Определяет порядок строк вывода

Можно также создать комбинацию нескольких таких операторов в одном запросе, используя операции на множествах UNION, DIFFERENCE и INTERSECT.

Имеются три операции для работы с данными:

- *INSERT*: Добавляет строки к таблице
- *UPDATE*: Обновляет столбцы в строках таблицы
- *DELETE*: Удаляет строки из таблицы

Стандарт ANSI/ISO для реляционных БД посвящен "языку программирования" SQL для работы с данными [ANSI, 1992]. SQL является очень популярным языком, одним из тех, которые можно рекомендовать без ограничений, но и он не лишен недостатков, если рассмотреть теоретические проблемы реляционной модели. Серия книг и статей Дейта и Кодда содержит серьезную критику ограничений SQL [Date, 1986; Codd, 1990]. Любому разработчику БД нужно знать об этих проблемах, чтобы наиболее эффективно использовать данную технологию, хотя они необязательно существенно скажутся на разработке базы данных. Когда язык предлагает средства, которые зависят от выбора модели разработки, то почти всегда это связано с тем, что SQL либо не предоставляет некоторых средств (например, функций для работы со строками), либо реально вмешивается в способ что-то сделать (нет способа удаления столбцов, нет возможности получить списки значений в запросах GROUP BY и т. д.). Эти ограничения могут вынудить разработчика спроектировать таблицы для удовлетворения потребностей и запросов приложения.

Версия SQL, которая наиболее широко предлагается поставщиками RDBMS, соответствует начальному уровню стандарта SQL-92 [ANSI, 1992]. Бесспорно, этому уровню SQL многого не хватает, чтобы его применять в качестве практического инструмента работы с БД. Все его используют, но было бы гораздо лучше, если бы крупные поставщики RDBMS внедрили стандарт SQL-92 полностью. Язык в его полном объеме имеет гораздо лучший синтаксис объединений, позволяет использовать SELECT гораздо чаще, чем допускает простой стандарт, и он включает в себя всеобъемлющий подход к управлению транзакциями, сессиями и национальными наборами символов.

Существенное влияние SQL на проектирование заключается в его способности выражать запросы и работать с данными. Каждая RDBMS имеет отличный диалект SQL. Например, предложение Oracle CONNECT BY уникально в мире RDBMS, оно предоставляет возможность запрашивать транзитивное замыкание через таблицу связей предок — потомок (запрос расширения частей). Sybase имеет интересные агрегативные функции для хранилищ данных, такие как CUBE, отсутствующей в Oracle. Только Oracle поддерживает возможность использования вложенного SELECT с оператором IN, который производит сравнение более чем с одним возвращаемым значением:

```
WHERE (col1, col2) IN (SELECT x, y FROM TABLE1 WHERE z = 3)
```

Не все различия в диалектах влияют на разработку, но структурные различия, подобные этому, имеют большое значение.

Поскольку SQL сочетает в себе язык запросов с языком управления схемой и ее использованием, то SQL также непосредственно влияет на разработку физической БД опять же через свою способность выражать структуры и ограничения такого проекта. Физическая разработка БД во многом зависит от того, какая RDBMS используется. Oracle строит свою среду, ориентируясь на множество пользователей, каждый из которых имеет схему таблиц, свои представления и другие объекты Oracle. А например, Sybase Adaptive Server и Microsoft SQL Server имеют концепцию БД, отдельной области хранения таблиц, и пользователи квазинезависимы от схемы БД. Система обработки транзакций SQL Server блокирует страницы, а не строки с различными исключениями, свойствами и преимуществами. Oracle блокирует строки, а не страницы. БД будет разрабатываться по-разному, потому что для SQL Server значительно проще столкнуться с одновременной взаимной блокировкой, чем при использовании Oracle.

Oracle имеет концепцию согласованного чтения, при которой пользователь, читающий данные из таблицы, продолжает видеть данные в неизменном виде независимо от того, изменены ли они другими пользователями. При обновлении данных первоначальный пользователь может получить сообщение, указывающее на то, что исходные данные изменились и что нужно сделать запрос, чтобы внести эти изменения. Другие основные RDBMS не имеют такой концепции, хотя они обладают другими концепциями, которых нет у Oracle. И снова это приводит к интересным проектировочным решениям. В качестве завершающего примера скажем, что каждая RDBMS поддерживает различный набор методов доступа к физической памяти, варьирующийся от стандартных схем индексирования В*-деревьев до хеш-индексов, индексов битовых карт и индексированных соединений кластеров.

Существует также проблема множества символов национального языка и того, каким образом они реализуются в каждой системе. Имеется стандарт ANSI [ANSI, 1992] для представления различных наборов символов, который не реализует ни один из поставщиков, и способ реализации национальных множеств символов у каждого поставщика полностью отличается от других. Использование специальных средств данной RDBMS может непосредственно повлиять на конкретную разработку.

Объектно-ориентированные базы данных

Объектно-ориентированные модели данных для управления объектно-ориентированными БД формально не существуют, хотя несколько авторов предложили такие модели. Структура этой модели исходит из объектно-ориентированного программирования, имеющего концепции наследования, инкапсуляции и абстракции, а также полиморфного структурирования данных.

Движущей силой объектно-ориентированных БД является гипотеза импедансного несоответствия, упомянутая в приведенном выше разделе по архитектуре распределенных объектов. По мере роста популярности ОО-языков программирования стала очевидной необходимость в интегрированной среде БД, которая одновременно делает ОО-данные устойчивыми и предоставляет все средства для обработки транзакций, множественного доступа и обеспечения целостности данных современным менеджерам БД. Проблема разработчиков таких баз данных состояла в том, что прикладные программисты, применяя устойчивые данные, должны были переходить от ОО-представлений к мышлению SQL для использования реляционных БД. В частности, ОО-системы и системы SQL используют системы разного типа, требующие, чтобы разработчики производили преобразование от одной к другой. Объектно-ориентированные БД ликвидировали необходимость такого преобразования с помощью непосредственной поддержки программных моделей популярных ОО-языков программирования как моделей данных для БД.

Есть два способа сделать объекты устойчивыми в рамках подходов ODBMS. Лидер рынка ObjectStore компании Object Design использует модель памяти. Этот подход предполагает, что объект обязательно работает с постоянной памятью, что в C++ означает добавление спецификатора `persist` (постоянства) памяти в дополнение к другим спецификаторам памяти — `volatile`, `static` и `automatic`. Этот подход плох тем, что требует предварительной компиляции программы, так как он изменяет реальный язык программирования, добавляя спецификатор устойчивости. Сначала выполняется предварительная компиляция программы, а затем она пропускается через стандартный компилятор C++. POET добавляет ключевое слово `persistent` перед ключевым словом `class` и также использует предкомпиляцию. Другие поставщики предлагают подход наследования, когда устойчивые классы наследуются из некоторого корневого устойчивого класса. Недостатком этого способа является необходимость сделать устойчивость характеристикой иерархии типа, что означает невозможность для класса создавать как объекты в оперативной памяти, так и устойчивые объекты (что всегда хочется так или иначе сделать).

Невозможно описать ОО-модель данных, не сталкиваясь с полярными точками зрения по поводу используемых средств или отсутствия таковых. В этом разделе описываются определенные средства, обычно общие для объектно-ориентированных БД, но в каждой системе реализуется модель, существенно отличающаяся от других. Лучше всего начать с модели объектов ODMG из стандарта ODMG для объектных БД [Cattell and Barry, 1998; ODMG, 1998] и ее привязки к C++, Smalltalk и Java. Это единственный реально существующий стандарт ODBMS; сообщество ODBMS пока еще не предложило никаких формальных стандартов, прошедших стандартизацию ANSI, IEEE или ISO.

Объектная модель определяет конструкции, поддерживаемые ODBMS:

- Основными понятиями модели являются *объект* и *литерал*. Каждый объект имеет уникальный идентификатор. У литерала нет идентификатора.

- Объекты и литералы могут быть разбиты на категории по *типам*. Все элементы данного типа имеют общие пределы изменения состояний (т. е. одинаковый набор свойств) и одинаковое поведение (т. е. одинаковый набор определенных операций). К объекту иногда обращаются как к *экземпляру* его типа.
- Состояние объекта определяется значениями набора *свойств*, которыми он обладает. Эти свойства могут быть *атрибутами* самого объекта или *отношениями* между объектами. Обычно значения свойств объекта со временем меняются.
- Поведение объекта определено набором *операций*, которые могут быть выполнены над объектом или самим объектом. Операции могут иметь список входных и выходных параметров с определенным типом. Каждая операция возвращает результат определенного типа.
- БД хранит объекты, позволяя их совместное использование многими пользователями и приложениями. БД основана на *схеме*, определенной в ODL, и содержит *экземпляры типов*, определенных ее схемой.

Объектная модель ODMG определяет, что подразумевается под объектами, литералами, типами, операциями, свойствами, атрибутами, отношениями и т. д. Разработчик приложения использует логическую структуру объектной модели ODMG для создания объектной модели приложения. Объектная модель приложения определяет конкретные типы, например Документ, Автор, Издатель и Глава, а также операции и свойства каждого из этих типов. Объектная модель приложения является (логической) схемой БД [Cattell and Barry, 1997, p.11–12].

Это итоговое утверждение касается всех частей объектной модели. Как и в большинстве случаев, проблема состоит в реализации деталей. На рис. 2.9 продемонстрирована упрощенная UML-модель БД криминальной сети, ОО эквивалент реляционной БД, представленной на рис. 2.8.

ВНИМАНИЕ

В главе 7 подробно представлена нотация UML, а также содержатся ссылки на литературу по UML.

Если разработчик не хочет вступать в дискуссию или пускаться в опасное сравнение средств, предлагаемых на рынке, ему нужно усвоить несколько основных понятий ODBMS, применяемых практически во всех продуктах ODBMS: структура объектных типов, наследование, жизненный цикл объектов, стандартная иерархия классов коллекций, отношения и структура операций [Cattell and Barry, 1997]. Понимание этих концепций создаст минимальную основу для принятия решения о том, с помощью чего ему лучше решать свои проблемы: OODBMS, RDBMS или ORDBMS.

Объекты и структуры типов

Любой объект в объектно-ориентированной БД имеет тип, а каждый тип — внутреннее и внешнее определение. *Внешнее определение*, называемое также *спецификацией*, состоит из операций, свойств или атрибутов и исключений, к которым может получить доступ пользователь объекта. *Внутреннее определение*,

невидимое пользователю объекта (называемое также *реализацией* или *телом*), содержит подробное описание операций и всего, что требуется объекту. ODMG 2.0 определяет *интерфейс* как "спецификацию, которая определяет только абстрактное поведение типа объекта" [Cattell and Barry, 1997, p. 12]. Класс – это "спецификация, определяющая абстрактное поведение и абстрактное состояние типа объекта". Спецификация литерала описывает только абстрактное состояние литерального типа. На рис. 2.9 представлен ряд спецификаций классов с операциями и свойствами. Класс Криминальная-Организация, например, имеет пять свойств (как и столбцы в реляционной таблице) и несколько операций.

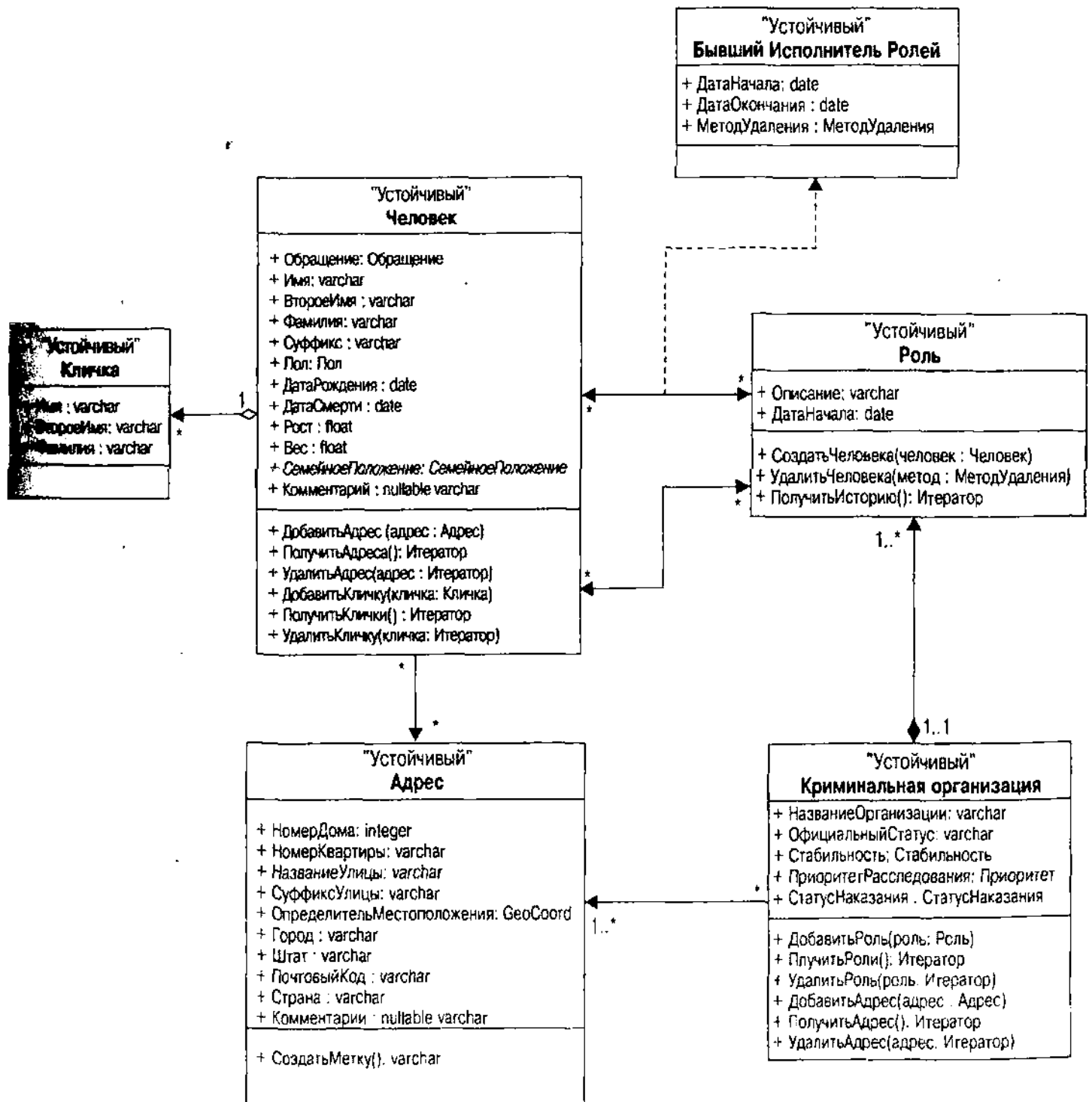


Рис. 2.9. ОО-схема: База данных криминальной сети агентства Holmes PLC

Операция — это абстрактное поведение объекта. Реализацию операции определяет *метод*, описываемый на выбранном языке программирования. Например, операция *ДобавитьРоль* позволяет назначить человека на роль в организации. Реализация этой операции на C++ происходит при помощи функции `insert()`, связанной с шаблонами `set<>` или `map<>`, содержащими набор ролей. Аналогично *свойство* — это абстрактное состояние объекта и его реализацией является *представление*, основанное на привязке к языку (в C++ перечислимый тип или тип класса, например, для свойства *ОфициальныйСтатус*). Литеральные реализации также отображаются в определенных языковых логических конструкциях. Ключом к пониманию ODL (языка определения объектов ODMG) является знание того, что он представляет собой спецификацию, а не реализацию объекта. Привязки к языку определяют, как реализовать абстракции ODL для определенных ОО-языков. Это разделение делает спецификацию ОО базы данных независимой от языков, ее реализующих.

ODMG определяет следующие литеральные типы:

- Long и unsigned long
- Short и unsigned short
- Float и double
- Boolean
- Octet (восемь битов)
- Character
- String
- Enum (перечисление или конечное множество различных значений)

Помимо этих типов имеется четыре структурных типа:

- Date, time и timestamp
- Interval

И, наконец, ODMG позволяет определить любую структуру этих типов, используя формат структуры, очень похожий на тот, который применяется в языке C.

Поскольку большая часть работы в объектно-ориентированных БД должна быть сделана с помощью коллекций объектов, ODMG предоставляет также иерархию классов коллекций для работы с методами и отношениями (см. ниже, раздел "Отношения и коллекции").

Наследование

Наследование может называться по-разному. Наиболее употребительные термины: отношение подтип — супертип, отношение "является..." или отношение генерализации — специализации. Идея состоит в том, чтобы выразить отношения между типами как специализацию типа. Каждый подтип наследует операции и свойства своего супертипа и добавляет новые операции и свойства к собственным определениям. Чашка кофе — это одна из разновидностей напитков.

Например, система регистрации преступлений содержит подсистему, относящуюся к идентификационным документам людей. Каждый человек может иметь любое количество идентификационных документов (включая и документы с кличками и т. д.). Имеется много видов идентификационных документов и, следовательно, ОО-схема должна представлять эти данные с иерархией наследования. Одна из разработок представлена на рис. 2.10.

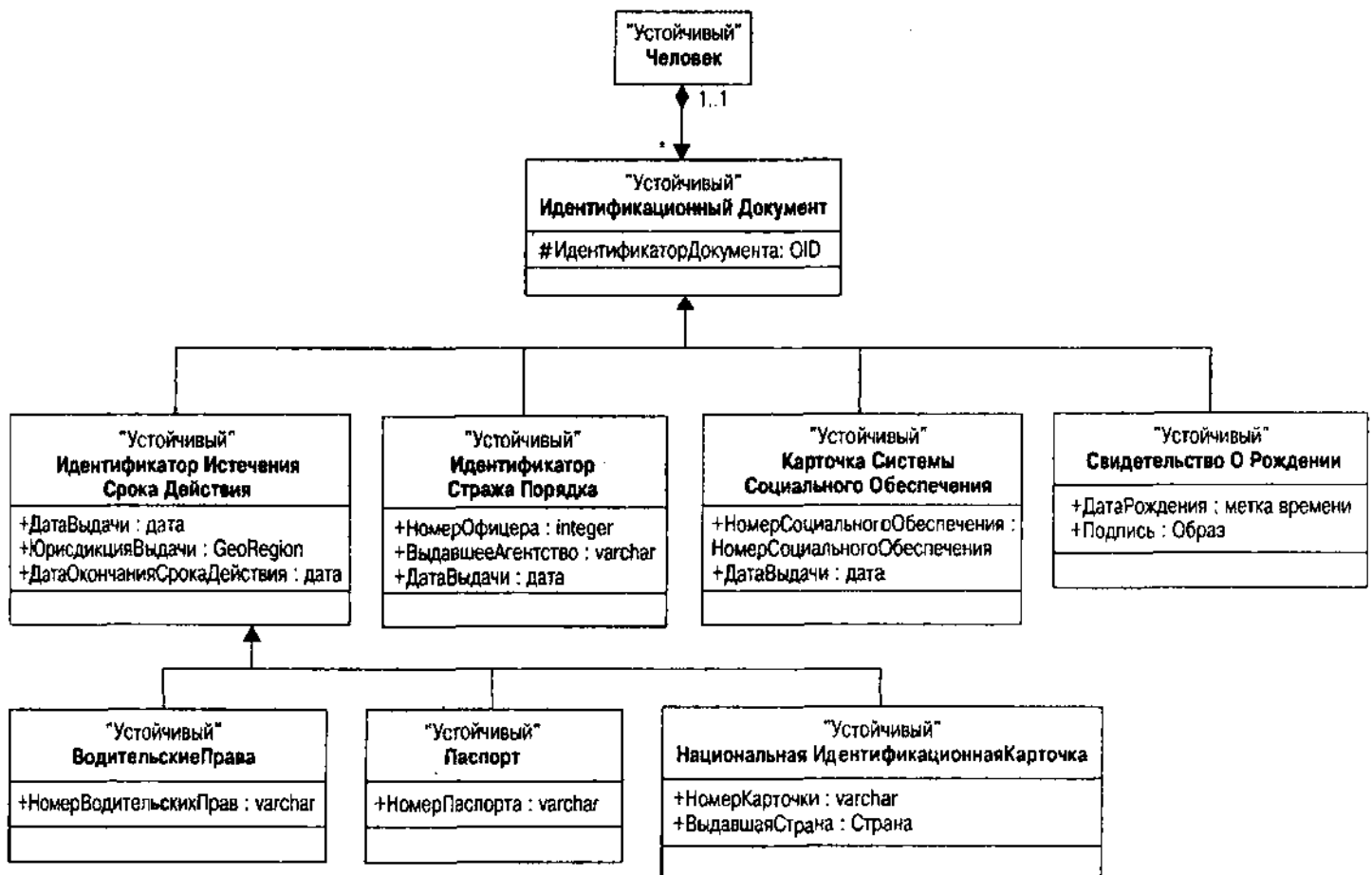


Рис. 2.10. Пример наследования: Идентификационные документы

Абстрактный класс *ИдентификационныйДокумент* представляет любой документ и имеет внутренний идентификатор объекта и отношение к классу *Человек*. *Абстрактный класс* – это класс, у которого нет объектов, или экземпляров, потому что он представляет собой генерализацию реальных классов объектов.

При данном подходе имеются четыре подкласса *ИдентификационногоДокумента*:

- *ИдентификаторИстеченияСрокаДействия*: идентификационный документ, у которого есть срок окончания действия
- *ИдентификаторСтражаПорядка*: идентификационный документ, который идентифицирует стража порядка

- *Карточка Системы Социального Обеспечения*: карточка системы социального обеспечения США
- *Свидетельство О Рождении*: свидетельство о рождении, выданное полномочным органом

Все они, кроме карточки системы социального обеспечения, имеют свои собственные подклассы; на рис. 2.10 для наглядности представлены только относящиеся к Идентификатору Истечения Срока Действия. Идентификатор Истечения Срока Действия наследует отношение с Человеком от Идентификационного Документа вместе с любыми операциями, совершаемыми для того, чтобы дополнить данный класс. Он добавляет дату окончания действия, дату выдачи и полномочный орган, выдавший документ, поскольку все карточки со сроком действия имеют полномочие, которое вызывает истечение срока действия карточки. Подкласс Водительские Права добавляет номер свидетельства к дате истечения срока действия, дате выдачи и полномочному органу, выдавшему документ; паспорт добавляет номер паспорта; Национальная Идентификационная Карточка добавляет номер карточки и выдавшую страну, которая по умолчанию содержит полномочный орган, выдавший документ. Таким образом, каждый подкласс наследует первичные характеристики всех идентификационных документов плюс характеристики подкласса документов с ограниченным сроком действия. Паспорт, например, принадлежит человеку через отношение, которое он наследует через суперкласс Идентификационного Документа.

ВНИМАНИЕ

Приведенный пример прежде всего относится к состоянию наследования, однако наследование при ОО-разработке часто концентрируется главным образом на поведении наследования. Зачастую ОО-разработка имеет дело, прежде всего, с интерфейсам, а не с классами, поэтому разработчик даже не видит переменных состояний. Поскольку в данной книге предлагается использовать ОО-методы для разработки БД, то здесь гораздо большее внимание будет уделяться классам и абстрактным состояниям, чем это делается при классическом ООП.

Жизненные циклы объектов

Самым простым способом познакомиться с жизненным циклом объекта является исследование интерфейса классов ObjectFactory и Object в ODMG [Cattell and Barry, 1997, p. 17]:

```
interface ObjectFactory {
    Object    new();
};

Interface Object {
    enum      Lock_Type{read, write, upgrade}
    exception LockNotGranted{}
    void      lock(in Lock_Type mode) raises (LockNotGranted);
    boolean   try_lock(in Lock_Type mode);
```

```
boolean    same_as(in Object anObject);  
Object     copy();  
void       delete();  
};
```

Оператор `new()` создает объект. Каждый объект имеет уникальный идентификатор или идентификатор объекта (OID). Во время жизни объекта можно заблокировать его или попытаться это сделать, можно сравнить его с другими объектами на равенство OID или скопировать объект для создания нового объекта с теми же значениями свойств. В конце своей жизни объект уничтожается с помощью операции `delete()`. Объект может быть либо *временным* (управляемым системой времени выполнения языка программирования), либо *постоянным* (управляемым ODBMS). ODMG определяет, что *время жизни* объекта (временного или постоянного) не зависит от его типа.

Отношения и коллекции

Отношение отображает объект в другие объекты. Стандарт ODMG определяет двоичные отношения между двумя типами, которые могут иметь стандартные многообразия один-к-одному, один-ко-многим или многие-ко-многим.

Отношения в данном релизе объектной модели не поименованы и не безупречны. Отношение само по себе не является объектом и не имеет идентификатора объекта. Оно явно определяется через объявление *путей прохождения*, позволяющих приложениям использовать логические связи между объектами, участвующими в отношении. Пути прохождения объявляются парами, по одной в каждом направлении следования двоичного отношения. [Cattell and Barry, 1997, p. 36].

Например, Криминальная Организация имеет отношение один-ко-многим с объектами класса Роль: роль относится к одной криминальной организации, которая в свою очередь имеет, по крайней мере, одну, а возможно, несколько ролей. В ODL это превращается в следующий путь прохождения к Криминальной Организации:

```
relationship set<Role> has_roles inverse Role::pertains_to;
```

На практике ODBMS управляет отношениями как наборами связей через внутренние OID во многом так же, как это раньше происходило в сетевых базах данных. ODBMS обеспечивают целостность ссылок путем обновления связей, когда состояние объектов изменяется. Цель — предотвратить попытки обращения с помощью ссылки к несуществующему объекту.

В ситуациях, когда необходимо обратиться к отдельному объекту только в одном направлении, можно объявить атрибут или свойство того типа, к которому нужно обратиться вместо того, чтобы определять явное отношение с инверсией. Эта ситуация не соответствует полностью стандарту ODMG и не гарантирует целостности ссылок, что приводит к образованию *висячих ссылок* (эквивалентов неверных указателей в БД).

Работа с отношениями происходит с помощью стандартных операций над отношениями. Это приводит к операциям формирования или

ликвидации отношения, добавления единственного объекта или добавления или удаления дополнительных объектов из определенного отношения. Сторона отношения ко-многим соответствует одному из нескольких стандартных классов коллекций:

- *Set<>*: Неупорядоченная коллекция объектов или литералов, не допускающая дубликатов
- *Bag<>*: Неупорядоченная коллекция объектов или литералов, которая может содержать дубликаты
- *List<>*: Упорядоченная коллекция объектов или литералов
- *Array<>*: Упорядоченная коллекция объектов или литералов динамического размера, доступная по позиции
- *Dictionary<>*: Неупорядоченная последовательность пар ключ – значение (*ассоциаций*) без дубликатов ключей

Эти объекты коллекций используются при помощи стандартных интерфейсов (*insert*, *remove*, *is_empty* и т. д.). При необходимости перемещаться по коллекции используется объект *Iterator* (итератор) с операциями *create_iterator* или *create_bi-directional_iterator*. Эти итераторы поддерживают стандартный набор операций для доступа (*next_position*, *previous_position*, *get_element*, *at_end*, *at_beginning*). Например, чтобы произвести какие-либо действия с использованием данных о людях из криминальной организации, сначала нужно получить итератор для ролей в организации. А затем в цикле получить данные о людях через текущие ролевые отношения данного человека (*Person*).

Невозможно переоценить важность коллекций и итераторов для объектно-ориентированных БД. Хотя также существует язык запросов (OQL), большая часть ОО-кода получает данные через отношения путем навигации с помощью итераторов, а не путем запроса наборов данных, как это делается в реляционной БД. Даже язык запросов извлекает коллекции объектов, по которым затем нужно совершать итерации. Кроме того, большинство продуктов OODBMS не имеют языка запросов, нет необходимости обращаться с запросами к базе данных (как средство, противоположное навигации) у тех, кто занимается приложениями OODBMS.

Операции

ODMG включает в себя стандарт OMG CORBA для операций и поддерживает *перегрузку* операций, которая происходит при создании операции в классе с тем же именем и сигнатурой (комбинацией типов параметров), как и операция в другом классе. Некоторые ОО-языки позволяют перегрузку между любыми классами, как в Smalltalk. Другие ограничивают перегрузку отношением подкласс – суперкласс, когда операция в подклассе перегружает только операцию с тем же именем и сигнатурой в суперклассе.

Стандарт ODMG поддерживает также исключения и управление исключениями, следуя модели управления *исключениями* или завершения языка C++. Существует иерархия объектов *Exception*, подклассы которых

используются для создания своих собственных исключений. Правила управления исключениями сложны:

1. Программист объявляет обработчик исключений в области действия *s*, способного обрабатывать исключения типа *t*.
2. Операция внутри вложенной области действия *sn* может активизировать исключение типа *t*.
3. Исключение перехватывается в ближайшей области действия, имеющей обработчик исключений. Стек запросов автоматически раскручивается системой времени выполнения до уровня обработчика. Память освобождается для всех объектов, размещенных в промежуточных кадрах стека. Все транзакции, начавшиеся во вложенной области действия, т. е. раскрученные системой времени выполнения в процессе поиска по стеку обработчика исключений, аварийно прерываются.
4. Когда управление доходит до обработчика, он может либо начать обрабатывать исключение, либо передать его (снова сделать активным) охватывающему обработчику [Cattell and Barry, 1997, p. 40].

Объектно-реляционные базы данных

Модель объектно-реляционных данных разработана еще меньше, чем модель ОО-данных. Будучи гибридом, она берет за основу реляционную модель данных и расширяет ее с помощью некоторых объектно-ориентированных концепций. Они зависят от определенного поставщика. Стандарт ISO SQL3, который близок к принятию, не имеет пока большого влияния на поставляемые системы [ISO, 1997; Melton, 1998].

ВНИМАНИЕ

С. Дж. Дейт, один из наиболее известных сторонников реляционной модели, написал манифест вместе с Х. Дарвеном об идеях, касающихся интеграции объектных и реляционных технологий [Date and Darwen, 1998]. Версия ОР-модели данных, представленная здесь, существенно от нее отличается. Тот, кто всерьез захочет использовать ОР-модель данных (или более практичную ORDBMS) должен прочесть книгу Дейта. Она одновременно раздражающая, разъясняющая и ухудшающая понимание. Раздражающая, потому что Дейт и Дарвен приносят едкий и самонадеянный британский юмор в свою книгу, которая разрушает практически все аспекты ОР-мира. Разъясняющая, потому что они объясняют несколько серьезных проблем ОР-теории, если так можно ее назвать, с реляционной, а не с ОО точки зрения. Ухудшающая понимание, потому что вряд ли поставщики ORDBMS усвоят что-либо из этой книги, что наносит ущерб и нам и им. Здесь не приводится подробный текст манифеста этих авторов, потому что требуемая ими система действительно реализует преимущества объектно-ориентированной интеграции с реляционной технологией и потому, что есть серьезные сомнения, что эта система когда-либо станет работающей ORDBMS.

В зависимости от выбранного поставщика система БД в большей или меньшей степени будет напоминать объектно-ориентированную систему. Она также обращена к миру и своим реляционным лицом, теоретически давая разработчикам лучшее, что есть в обоих мирах. На рис. 2.11 показана эта гибридная разновидность, как комбинация ОО и реляционной структур, представленных на рис. 2.8 и 2.9. Таблицы имеют соответствующие типы объектов, и отношения являются наборами или коллекциями объектов. Проблемы, создаваемые этими моделями данных, настолько новы, что поставщики только приступили к работе над ними, а большинство текущих решений по сути ориентированы на них. Время покажет, насколько созрела объектно-реляционная модель.

В настоящее время можно работать с инфраструктурой, представленной Майклом Стоунбрейкером в его книге по технологии ORDBMS. В ней для определения настоящей ORDBMS предлагаются следующие средства [Stonebraker, 1999, p. 268].

1. Расширение базовых типов

- a.** Динамическое связывание определенных пользователем функций
- b.** Клиентская или серверная активизация определенных пользователем функций
- c.** Интеграция определенных пользователем функций с системами промежуточного ПО
- d.** Безопасные функции, определенные пользователем
- e.** Обратный вызов в определенных пользователем функциях
- f.** Определяемые пользователем методы доступа
- g.** Типы данных произвольной длины
- h.** Менеджер открытой памяти

2. Сложные объекты

- a.** Конструкторы типов
 - набор
 - запись
 - ссылка
- b.** Определенные пользователем функции
 - динамическая компоновка
 - клиентская или серверная активизация
 - безопасные функции, определенные пользователем
 - обратный вызов
- c.** Сложные типы данных произвольной длины
- d.** Поддержка SQL

3. Наследование

- a. Наследование данных и функций
 - b. Перегрузка
 - c. Наследование типов, а не таблиц
 - d. Множественное наследование
4. Система правил
- a. События и действия являются извлечениями, а также обновлениями
 - b. Интеграция правил с наследованием и расширением типов
 - c. Богатая семантика выполнения для правил
 - d. Отсутствие бесконечных циклов

ВНИМАНИЕ

Будучи прекрасной основой для академических дебатов по поводу истины и красоты, данное определение, вероятно, не является необходимым и достаточным определением модели данных с точки зрения практикующего архитектора данных. Определенно, оно не решает всех этих проблем и является образцом логики того же рода, как и та, которая привела к десятилетней войне по поводу истинной сущности "объекта". Полагаю, что совсем не продуктивно считать список характеристик определением. Пока не появится формальная математическая модель, расширяющая реляционную модель любыми подходящими конструкциями, я предпочитаю оставить выяснение истины представителям академической среды. Важно также отметить, что эти характеристики отражают интерес Стоунбрейкера к Illustra, продукту, который он создал, когда работал в области ORDBMS, будучи знакомым с Informix и владея Informix Dynamic Server с режимом универсальных данных.

В следующих разделах рассмотрим основы этих характеристик. Там, где это окажется полезным, абстракция будет проиллюстрирована реализацией в одном или нескольких коммерческих ORDBMS-продуктах, включая Oracle8 с режимом объектов, универсальную базу данных DB2 [Chamberlin, 1998] и Informix с его дополнительным динамическим сервером Dynamic Server (известным как Illustra) [Stonebraker and Brown, 1999].

Типы и наследование

Архитектура реляционных данных поддерживает типы с помощью ссылки на домены столбцов. Стандарт ANSI имеет ограниченное количество очень примитивных типов: NUMERIC, CHARACTER, TIMESTAMP, RAW, GRAPHIC, DATE и INTERVAL, а также подтипов (INTEGER, VARYING CHARACTER, LONG RAW), которые являются ограничениями более общих типов. Это базовые типы модели данных.

ОР-модель данных добавляет *расширенные* или *определяемые пользователем* типы к базовым типам реляционной модели. Имеются три разновидности расширенных типов:

- Подтипы или явные типы данных
- Типы данных записей

- Инкапсулированные типы данных

Подтипы *Подтип* — это базовый тип с определенным ограничением. Стандартный SQL поддерживает комбинацию ограничений по размеру и логике. Например, можно использовать тип NUMERIC, ограничив точность до 11 знаков и выбрав двоичную систему счисления для представления денежных сумм вплоть до \$999 999 999,99. Можно также включить константу CHECK, которая ограничивает значение чем-то между 0 и 999 999 999,99, делая ее неотрицательной денежной суммой. Однако эти ограничения можно применять только к определению столбца. Их нельзя создать отдельно. ОР-модель позволяет создавать и именовать отдельный тип с ограничениями.

Например, DB2 UDB имеет такой оператор:

```
CREATE DISTINCT TYPE <name> AS <type declaration> WITH COMPARISONS
```

Этот синтаксис позволяет именовать объявление типа. После этого система считает новый тип совершенно отдельным (особым) типом от его базового типа, что может значительно помочь при поиске ошибок в коде SQL. Такие типы являются частью стандарта SQL3. Предложение WITH COMPARISONS, в лучших традициях IBM, ничего не делает. Оно включено просто для того, чтобы напоминать, что данный тип поддерживает реляционные операции, такие как + и <, и все базовые типы, кроме BLOB, требуют его присутствия. Informix имеет подобный оператор CREATE DISTINCT TYPE, но не имеет WITH COMPARISONS. Обе системы позволяют привести значения к типу, чтобы сообщить системе, что значение должно иметь определенный тип. DB2 имеет для этого функцию CAST, тогда как Informix использует :: для литерала: например, 82::fahrenheit приводит число 82 к типу fahrenheit. Обе системы позволяют создать преобразующие функции, которые используются в операциях преобразования к типу для преобразования значений из одного типа в другой надлежащим образом. Oracle8 не имеет концепции подтипа.

Типы данных записей *Типы данных записей* (или *структурный тип* в стандарте ISO SQL3) — это определение таблицы, которое, возможно, сопровождается методами или функциями. После определения типа можно создавать объекты этого типа или определять таблицы таких объектов. ОР-системы обычно не имеют управления доступом к членам записи, поэтому программы могут получать прямой доступ к атрибутам данных объекта. В данной книге эти типы отличаются от типов инкапсулированных данных, которые скрывают данные за системой защиты доступа из методов или функций.

ВНИМАНИЕ

SQL3 определяет тип так, что каждый атрибут генерирует функции наблюдателя и мутатора (функции, которые получают и задают значения атрибутов). Таким образом, стандарт строго поддерживает полную инкапсуляцию, хотя и непосредственно раскрывает базовые атрибуты, подобно тому, как кто-то одновременно имеет торт и ест его.

Oracle8 содержит типы данных записей как первичный способ объявления структуры объектов в системе. Оператор `CREATE TYPE AS OBJECT` позволяет определить атрибуты и методы данного типа. В DB2 нет концепции типа записей. Динамический сервер Informix предлагает тип строки для определения атрибутов (`CREATE ROW TYPE` с синтаксисом, подобным `CREATE TABLE`), но без методов. Можно, однако, создать определяемые пользователем подпрограммы, которые берут объекты любого типа и действуют как методы. С определенной натяжкой это означает, что объектные типы Oracle8 напоминают инкапсулированные типы, описанные в следующем разделе, за исключением того, что они позволяют получать доступ ко всем атрибутам данных объекта непосредственно.

Инкапсулированные типы данных и BLOB (большие двоичные объекты) ОР-системы по-настоящему начинают представлять интерес, когда в них добавляют типы инкапсулированных данных — типы, которые полностью скрывают свою реализацию. Informix предоставляет так называемые *DataBlade* (вероятно, имеется в виду лезвие, вставляющееся в бритвы); Oracle8 имеет программные компоненты для поддержки типов данных (*data cartridges*) в сетевой компьютерной архитектуре (NCA). Эти технологии позволяют расширить систему базовых типов данных новыми типами и поведением, которые с ними связаны. Например, *spatial DataBlade* в Informix предоставляет всеобъемлющий способ обращения с пространственной и географической информацией. Он позволяет сохранять данные и запрашивать их естественными способами, не вынуждая проектировщика создавать реляционные структуры. Программные компоненты поддержки типов данных Oracle8 выполняют аналогичные функции, но с интересными ограничениями проектирования (см. главу 12). Эти дополнительные модули позволяют не только представлять данные и поведение, но они также предоставляют индексирование и другой инструментарий для обеспечения методов доступа, которые интегрируются с оптимизатором DBMS [Stonebraker, 1999, p. 117–149].

Критически важной частью головоломки инкапсулированных типов данных является конструктор, функция, которая работает как строительная фабрика, создающая объекты. Informix, например, предоставляет функцию `row()` и оператор приведения типов `cast` для создания сущности строкового типа в операторе `INSERT`. Если, скажем, применяется триплет строкового типа для объявления колонки трех целых в таблице, используется `row(1,2,3)::triplet` в качестве значения в предложении `VALUES`, чтобы привести целые к строковому типу. В Oracle8 типы создаются с помощью методов-конструкторов, имеющих то же имя, что и тип и набор параметров. Затем этот метод используется как значение: `triplet(1,2,3)`. Oracle8 поддерживает также методы для сравнения с помощью стандартного индексирования.

ОР-системы предоставляют также широкую поддержку для LOB (large objects) или больших объектов. Они представляют собой инкапсулированные типы в том смысле, что их внутренняя структура абсолютно недоступна для SQL. Обычно LOB извлекают в программу, затем преобразуют его содержимое в объект определенного типа. Как преобразование, так и поведение, связанное с новым объектом, находятся в клиентской программе, а

не в базе данных. Oracle8 предоставляет типы BLOB, CLOB, NCLOB и Bfile. BLOB — это двоичная строка любой структуры, какая потребуется. CLOB и NCLOB — это символьные объекты для хранения очень больших текстовых объектов. CLOB содержит однобайтовые символы, NCLOB — многобайтовые. Bfile — это ссылка на BLOB во внешнем файле; функции Bfile позволяют работать с файлом обычными способами, но через SQL, а не с помощью программных операторов. Динамический сервер Informix также предоставляет BLOB и CLOB. DB2 V2 предоставляет BLOB, CLOB и DBCLOB (двоичный, однобайтовые и многобайтовые символы соответственно). V2 предоставляет также файловые ссылки, позволяющие читать и записывать LOB из и в файлы.

Наследование Наследование в ОР-системах имеет пару особенностей по сравнению с наследованием в OODBMS. Первая — отрицательная: Oracle8 и DB2 V2 не поддерживают никакого наследования. Oracle8 может иметь некоторую форму наследования в будущих релизах, но первый релиз не имеет наследования. Динамический сервер Informix предоставляет наследование и имеет вторую особенность: наследование типов и таблиц. Определение Стоунбрейкера относится к наследованию типов, а не таблиц; под этим он, видимо, подразумевает, что наследование, основанное только на типах, недостаточно хорошо, поскольку его книга подробно рассматривает также и механизм наследования таблиц. Наследование типов так же, как и ОО-наследование, относится к строковым типам. Наследуется как структура данных, так и использование любых определяемых пользователем функций, использующих строковый тип в качестве аргумента. При перезагрузке функции для наследуемых типов динамический сервер выполнит надлежащую функцию на надлежащих данных.

Особенность возникает, когда дело доходит до структуры данных в системе. В OODBMS расширение типа — это множество всех объектов данного типа. Обычно имеются способы перебора всех этих объектов. В ORDBMS, однако, данные находятся в таблицах. В таких системах данные используются двумя способами. Можно либо объявить таблицу принадлежащей какому-то типу и придать ей типовую структуру, либо объявить определенный тип столбца в таблице, придав столбцу структуру типа. Следовательно, можно объявить несколько таблиц с одним типом, разбивая расширение типа. В современных системах не существует другого способа, кроме UNION, для работы над расширением типа как целым.

Наследование обычного вида работает с типами и расширениями типов. Чтобы удовлетворять потребности таблиц, Informix расширяет концепцию наследования на таблицы на основе наследования типа. При создании таблицы подтипа можно создать ее под таблицей супертипа. Это двухшаговое наследование позволяет создать отдельные иерархии данных с использованием иерархий одного и того же типа данных. Оно позволяет также использовать запросы ORDBMS на подтипах.

На рис. 2.10 в приведенном выше разделе об OODBMS представлена иерархия наследования идентификационных документов. Используя динамический сервер Informix, для представления иерархии типов можно объявить строковые типы для Идентификационного Документа,

ДокументаСоСрокомДействия, Паспорта и т. д. Затем можно объявить таблицу для каждого из этих типов, соответствующую конкретному объекту. В этом случае ИдентификационныйДокумент, ДокументСоСрокомДействия и ИдентификаторСтражаЗакона будут абстрактными классами и не потребуют таблиц, в то время как остальные будут конкретными и потребуют таблицы. Можно создать экземпляры любого из этих классов путем создания множественных таблиц для хранения данных (Паспорт США, Паспорт Объединенного Королевства и т. д.).

В связи с четким различием абстрактных и конкретных структур эта иерархия не требует объявления наследования таблиц. Можно рассматривать иерархию Ролей как контрпример. На рис. 2.9 Роль – это класс, представляющий связь между Человеком и КриминальнойОрганизацией. Можно создать иерархию классов, представляющую различные виды ролей (например, Босс, Заместитель, Исполнитель, Советник, Партнер), и можно интерпретировать роль как разновидность более общей ассоциации. Можно создать как таблицу ролей, так и таблицу для каждого из ее подтипов, используя компонент UNDER для установления иерархии типов. Во время выборки из таблицы Role, на самом деле сканируется не просто таблица, но все ее таблицы подтипов. Если в запросе использовалась функция, SQL применит соответствующую перегруженную функцию к действительной строке, основываясь на ее настоящем типе (динамическое связывание и полиморфизм). Использование описателя ONLY в компоненте FROM ограничит запрос единственной таблицей вместо просмотра всех таблиц подтипов.

Продукты ORDBMS неоднородны в смысле использования ими наследования. Кто на самом деле предлагает это средство, делает это с определенными ограничениями концепции наследования OODBMS. Эти ограничения имеют вполне определенное влияние на разработку БД, на концептуальную и физическую схемы. Но влияние ОР-архитектуры данных не ограничивается типами. Они предлагают множество возможностей структурирования с использованием также сложных объектов и коллекций.

Сложные объекты и коллекции

Все ОР-архитектуры данных предлагают сложные объекты различных видов:

- *Вложенные таблицы:* Таблицы со столбцами, которые сами определены с помощью многих компонентов как таблицы
- *Столбцы с типами:* Таблицы со столбцами определенного пользователем типа
- *Ссылки:* Таблицы со столбцами, ссылающимися на объекты других таблиц
- *Коллекции:* Таблицы со столбцами, являющиеся коллекциями объектов, такие, как множества массивов переменной длины

ВНИМАНИЕ

Тот, кто знаком с некоторыми проблемами математического моделирования структур данных, заметит некоторые трудности вышеприведенной классификации. Например, можно моделировать вложенные таблицы, используя типы, и можно рассматривать их как специальные виды коллекций (скажем, набор записей). Это опять указывает на сложности определения модели, которая не имеет формальной основы. С точки зрения практической разработки вышеприведенные категории отражают разницу в выборе свойств продукта целевой DBMS.

Новые средства для работы со структурой таблиц Oracle8 во многом основаны на вложенных структурах. Сначала создается тип таблицы, определяющий тип как таблицу объектов определенного пользователем типа:

```
CREATE TYPE <table type> AS TABLE OF <user-defined type>
```

Вложенная таблица — это столбец таблицы, объявленный типом таблицы. Например, можно сохранить таблицу Alias внутри таблицы Person, если воспользоваться следующими определениями:

```
CREATE TYPE ALIAS_TYPE (...);  
CREATE TYPE ALIAS AS TABLE OF ALIAS_TYPE;  
CREATE TABLE Person (  
    PersonID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100) NOT NULL,  
    Aliases ALIAS)
```

Динамический сервер Informix для представления сложных объектов опирается исключительно на типы. Создается определяемый пользователем тип, затем объявляется таблица, использующая этот тип для типа столбца в таблице. Informix не имеет возможности сохранять таблицы в столбцах, но он поддерживает наборы определяемых пользователем типов, которые приводят к тем же решениям.

Как Oracle8, так и динамический сервер Informix предоставляют ссылки на типы с определенными практическими различиями. Ссылка в данном контексте — это неизменяемый указатель на объект, хранимый вне таблицы. Ссылки используют инкапсулированный OID для обращения к определяющему ее объекту. Ссылки зачастую занимают место отношений внешнего ключа в ОР-архитектурах. Можно скомбинировать их с типами для существенного снижения сложности запросов. Как Oracle8, так и Informix обладают навигационным синтаксисом для использования ссылок в выражениях SQL, известным как точечная нотация. Например, в реляционной модели на рис. 2.8 имеется отношение внешнего ключа между CriminalOrganization и Address через таблицу отношений OrganizationAddress. Чтобы получить почтовые коды организации, можно использовать стандартный SQL:

```
SELECT a.PostalCode  
FROM CriminalOrganization o, OrganizationAddress oa, Address a  
WHERE o.OrganizationID = oa.OrganizationID AND  
      oa.AddressID = a.AddressID
```

Получить ту же информацию из ORDBMS можно было бы, представив адресное отношение как набор ссылок к адресам, которые являются отдельным типом. Чтобы их получить, можно использовать следующий запрос SQL в Informix:

```
SELECT deref(*).PostalCode  
FROM (SELECT Addresses  
      FROM CriminalOrganization)
```

SELECT в предложении FROM возвращает множество ссылок на объекты типа Address, ссылка из этого множества разыменовывается и выполняется переход к атрибуту PostalCode типа в основном выражении предложения SELECT.

Oracle8 действует способом, более близким реляционной модели, поскольку он не поддерживает этот вид получения множества. Вместо этого можно получить ссылки на адреса и разыменовывать их в контекст представления объекта и его тип. Представление объекта — это представление, определяемое с помощью оператора CREATE VIEW для объекта определенного типа. Это позволяет инкапсулировать запрос неопределенной сложности, который создает объекты. В этом случае, например, можно создать представление объектов адресов криминальной организации, которое включает название организации и VARRAY адресов для каждой организации. Затем обычно выбирается объект в структуре данных PL/SQL и используется стандартная точечная нотация для доступа к элементу почтового кода индивидуальных членов VARRAY.

VARRAY в Oracle8 — это массивы переменной длины объектов одного типа, включая ссылки к объектам. Массивы переменной длины получали и теряли популярность в различных продуктах и подходах к представлению структур данных. Они предоставляют базовую возможность структурировать данные в последовательно организованный список. Динамический сервер Informix имеет более точные коллекции SET, MULTiset и LIST. SET — это коллекция уникальных элементов без какого бы то ни было упорядочения. MULTiset — это коллекция элементов без какого бы то ни было упорядочения, допускающая дублирование значений. LIST — это коллекция элементов с последовательным упорядочением и разрешенным дублированием значений. Доступ к элементам LIST можно получать с помощью целочисленного индекса. LIST и VARRAY имеют похожие характеристики, хотя и различны в реализации.

DB2 V2 не обладает подобными характеристиками. Она не предлагает ни возможности создавать сложные типы, ни каких бы то ни было видов коллекций. Эта ORDBMS основана исключительно на больших объектах (LOB) и внешне определенных функциях, действующих на них.

Правила

Правило — это комбинация обнаружения события (ON EVENT x) и обработки (действие DO). Когда сервер базы данных обнаруживает событие (обычно INSERT, UPDATE или DELETE, но также возможно SELECT), он

запускает действие. Комбинация пары событие – действие и есть правило [Stonebraker, 1999, p. 101–111]. Большинство менеджеров БД называет правила *триггерами*.

Правила интересны, но они на самом деле не образуют существенного, различного базиса для ORDBMS. Большинство RDBMS и некоторые OODBMS также имеют триггеры, и расширения, которые перечисляет Стоунбрейкер, не относятся к характеристикам ОО в DBMS. Желательно, чтобы стандарт SQL3 включил бы наконец триггеры и/или правила так, чтобы можно было разрабатывать переносимые триггеры. Сейчас это невозможно. В результате многие компании избегают триггеров, потому что они мешают переходу к другой DBMS в случае необходимости, вызываемой экономическими или техническими причинами. Это заставляет реализовывать бизнес-правила на сервере приложений или на стороне клиента, а не в БД, которой они принадлежат.

Решения

Объектно-реляционная модель вносит большой вклад в разработку прикладного ПО. Реляционные средства модели позволяют предыдущим реляционным разработкам мигрировать в новые модели данных в такой мере, в какой эта модель поддерживает полную реляционную модель данных. Однако чтобы полностью использовать модель данных, следует выбрать, по крайней мере, одно из двух направлений.

Первое – можно выбрать использование типов данных со многими значениями в реляционных таблицах с помощью вложенных таблиц или типизированных атрибутов. Для определенных целей, таких, как средства быстрой разработки приложений, использующих эти свойства, это очень полезно. Однако следует избегать этих средств за исключением того случая, когда есть надежное тому обоснование. Внутренние реализации этих средств все еще примитивны, а такие понятия, как оптимизация запросов, индексирование, уровни вложения и логика запросов все еще проблематичны. Более важно то, что использование этих средств приводит к чрезмерному уровню сложности разработки. Ближайший пример – использование вложенных классов в C++. Единственное веское основание для вложения классов в C++ – это инкапсуляция вспомогательного класса в том классе, которому он помогает, защищая его от опасного внешнего мира. Аналогично, объявляя вложенную таблицу или множество объектов коллекции, которая скрывает сложность данных в пределах столбца таблицы, нельзя использовать их повторно вне таблицы. Вместо этих средств следует создать отдельные таблицы для каждого вида объектов и использовать ссылки для связи таблицы с этими объектами.

Второе – можно использовать объектно-ориентированные средства (наследование, методы и объектные ссылки) для создания схемы, которая хорошо вписывается в объектно-ориентированный концептуальный проект. Взаимодействие с реляционными средствами модели данных обеспечивает мост к реляционным операциям (таким, как возможность эффективно использовать SQL с БД). ОО-средства позволяют осуществить связность и инкапсуляцию, необходимых в хороших ОО-проектах.

Итоги

В данной вводной главе структурируется контекст, в котором разрабатываются базы данных с использованием ОО-методов. Частью контекста является системная архитектура, существенно влияющая на физическую разработку БД. Другая часть контекста — архитектура данных — заметно влияет на концептуальную разработку и выбор, осуществляемый в разрабатываемых приложениях, использующих БД.

Остальная часть данной главы представляет три основные разновидности систем управления базой данных: RDBMS, ORDBMS и OODBMS. Эти разделы содержат обзор услуг, предоставляемых системами по хранению данных и некоторых основных проблем разработки, относящихся к любой системе.

Далее в книге рассматриваются проблемы и решения, которые встречаются во время типично организованного процесса разработки. Однако следует помнить, что порядок проистекает из искусства — искусства разработки.

Глава 3

Сбор требований

Нашим является тот мир, в котором люди не знают, чего хотят, и желают пройти через ад, чтобы получить это.

Дон Маркус

Ад требований — это тот особый бесконечный путь, по которому Сизиф толкает камень к вершине горы только для того, чтобы увидеть, как он опять катится вниз. Часто неверно интерпретируемый этот миф имеет корни в реальности. На самом деле Сизиф достигает вершины горы много раз; просто он снова и снова спрашивает, остановиться ему или начинать сначала.

Возможно, раз получив указание, правильнее было бы не переспрашивать, чтобы не услышать в ответ приказание вновь и вновь катить этот камень требований в гору. Данная глава описывает местность с той целью, чтобы разработчик, по крайней мере, не поскользнулся и не скатился снова вниз.

Потребности заказчика должны явиться отправной точкой для разработки БД. Неопределенность и переход от нее к ясным и четким требованиям — тот путь, по которому идет разработка. Определение приоритетных требований позволяет предложить осмысленный план проекта, отложив низкоприоритетные вопросы до следующего раза. А понимание масштабов требований позволяет понять, какая нужна архитектура БД. В этой главе раскрыты основы сбора требований к данным на примере системы учета преступлений, принятой в агентстве Holmes PLC.

Неопределенность и устойчивость

Сбор требований — часть любого проекта по разработке ПО, определенные методы такого сбора применяются вне зависимости от того, имеет ли

система централизованную базу данных или база данных отсутствует вовсе. Этот раздел суммирует некоторые общие рекомендации, касающиеся требований, и адаптирует их по отношению к БД.

Неопределенность

Неопределенность может сделать жизнь интересной. Однако, несмотря на вечные претензии рассерженных пользователей и программистов, целью сбора требований должно быть снижение неопределенности до уровня, при котором может быть предоставлен проект БД, отражающий запросы клиента.

В качестве примера рассмотрим картотеку. Она представляет собой коллекцию справочных материалов, созданную Шерлоком Холмсом для записи фактов в дополнение к своей феноменальной памяти. Некоторые относящиеся к делу цитаты иллюстрируют основные требования.

Этот отрывок описывает природу картотеки:

"— Будьте добры, поищите ее в моем указателе, доктор,— пробормотал Холмс, не открывая глаз. За многие годы он усвоил систему пометок всех параграфов, касающихся людей и вещей, поэтому трудно было назвать предмет или личность, по которым он не мог бы сразу же предоставить сведения. В данном случае я нашел ее биографию между данными о еврейском раввине и главнокомандующем, написавшем монографию о глубоководных рыбках.

— Ну-ка, ну-ка,— сказал Холмс.— Гм! Родилась в Нью-Джерси в 1858 году. Контральто — гм! "Ла Скала" — гм! Примадонна Имперской оперы в Варшаве — да! Ушла на пенсию с оперной сцены — ха! Живет в Лондоне — так-так! Ваше Величество, как я понимаю, запутались с этой молодой персоной, написали ей несколько компрометирующих его писем и теперь жаждет получить эти письма обратно". [SCAN]

Не каждая попытка найти информацию увенчивается успехом:

"Мой друг слушал с приятным удивлением эту длинную речь, которая текла дальше с исключительной силой и глубокомыслием, каждое утверждение подкреплялось шлепком коричневатой руки по колену оратора. Когда наш посетитель замолк, Холмс нашел букву "S" в своей картотке. Но он напрасно копался в этой груде разнообразной информации.

— Имеется Артур Х Стаутон, начинающий молодой фальшивомонетчик,— сказал он,— и был также Генри Стаутон, которому я помог повеситься, но Годфри Стаутон — для меня новое имя.

Теперь пришла очередь нашего посетителя удивляться". [MISS]

Следующий отрывок иллюстрирует вежи биографий людей и их отношение к криминальной организации.

"— Подайте, пожалуйста, мой биографический указатель с полки. Он лениво перелистал страницы, откинувшись назад в кресле и выдувая огромные клубы дыма.

— Моя коллекция на "М" замечательна,— сказал он. Только один Мориарти мог бы прославить ее, а здесь есть также отравитель Морган и Мерридю с отвратительной памятью, и Мэтью, который выбил мой левый клык в зале ожидания Черинг-Кросс, и здесь есть наш сегодняшний друг.

Он провел рукой по записной книжке и прочел:

— *Моран, Себастьян, полковник*. Безработный. Бывший первопроходец Бангалора. Родился в Лондоне в 1840 году. Сын сэра Августо Морана, бывшего британского посла в Персии. Закончил Итон и Оксфорд. Принимал участие в Джовакской кампании, Афганской кампании, жил в Чарасибе (недолго), Шерпуре и Кабуле. Автор книг "Влияние Западных Гималаев" (1881), "Три месяца в джунглях" (1884). Адрес: ул. Кондуит. Клубы: англо-индийский, танкервильский, карточный багательский клуб. На полях было написано аккуратным почерком Холмса:

Второй из наиболее опасных людей в Лондоне". [EMPT]

А вот пример практического использования картотеки в криминальном расследовании:

"Мы оба сидели в тишине в течение короткого периода времени после того, как замолк необычный рассказчик. Затем Шерлок Холмс снял с полки одну увесистую записную книжку, куда он помещал свои вырезки.

— Вот объявление, которое заинтересует вас,— сказал он.— Оно появилось во всех газетах около года назад. Послушайте:

"Потерялся 9-го числа текущего месяца мистер Джереми Хэйлинг, 26 лет, инженер-гидравлик. Покинул дом в десять часов вечера и с тех пор ничего о нем неизвестно. Был одет в..."

И т. д. Ха! Я полагаю, что это был последний раз, когда машина полковника дала сбой". [ENGR]

А вот другой пример, показывающий, как Холмс добавлял пометки на полях к исходной статье:

"Не успел наш посетитель вперевалку выйти из комнаты — никакие другие слова не могут описать способ передвижения мистера Меррилоу,— как Шерлок Холмс бросился к стопке записных книжек в углу. В течение нескольких минут оттуда доносилось шуршание страниц, а затем восторженное похрюкивание — он нашел то, что искал. Он был так взволнован, что не поднялся, а сидел на полу, как какой-то странный Будда со скрещенными ногами, огромные книги были разбросаны вокруг него, а одна — открыта на его коленях.

— Данный случай заинтересовал меня сразу, Ватсон. Вот мои пометки на полях для подтверждения этого. Должен признаться, что мне они ни к чему. И все же я убежден, что полковник был не прав. Разве вы не помните о трагедии аббата Парвы?

— Нет, Холмс.

— И тем не менее вы были тогда со мной. Но мое собственное впечатление было слишком поверхностным. Тогда не за что было зацепиться, и никто не захотел воспользоваться моими услугами. Вероятно, вы захотите прочитать газеты?" [VEIL]

Холмс также использует записную книжку, чтобы фиксировать случаи, аналогичные предыдущим, к которым клиенты смогли привлечь его внимание:

"— Довольно занятный предмет для изучения эта девушка,— заключил он.— Я нахожу ее более интересной, чем ее маленькая проблема, которая, кстати, довольно банальна. Вы сможете найти аналогичные случаи, если обратитесь к моему указателю. В Андувре в '77 было что-то подобное, в Гааге в прошлом году. Идея стара, но была парочка новых для меня деталей. Однако сама девушка более интересна". [IDEN]

А здесь используется другой вид записей — картотека преступлений:

"— Весь ход событий,— сказал Холмс,— с точки зрения человека, называющего себя Степлтоном, был простым и прямым, хотя для нас, не имевших никакой возможности вначале определить мотивы его действий и знавших только часть фактов, все казалось исключительно сложным. У меня было преимущество благодаря двум беседам со Степлтоном, и сейчас в этом деле все прояснилось, не осталось для нас ничего секретного. Вы найдете несколько пометок по этому делу в моем индексированном списке преступлений". [HOUN]

Определенные ограничения такого носителя данных, как записная книжка, безусловно, лимитировали способы упорядочения преступлений Холмсом:

"— Имя молодой женщины не было Матильда Бриггс, Ватсон,— сказал Холмс.— Был корабль, ассоциирующийся с гигантской крысой с Суматры, история, к которой мир все еще не готов. Но что мы знаем о вампирах? Входит ли это также в сферу нашего понимания? Все лучше, чем стагнация, но определенно нас направили в русло сказок братьев Гримм. Протяните руку, Ватсон, и посмотрите, что там есть на букву V.

Я откинулся назад и взял огромный том, на который он ссылался. Холмс положил его на колени, и его глаза двинулись медленно и с любовью по записям о старых преступлениях, смешанным с аккумулированной информацией о его жизни.

— Вояж "Глории Скотт",— прочел он.— Это был нехороший бизнес. Припоминаю, что вы занимались этим, Ватсон, хотя и безрезультатно. Виктор Линч, фальшивомонетчик. Ядовитая ящерица или змея. Замечательное преступление, да! Виттория, цирковая красавица. Вандербильд и Йеггман. Гадюки. Вигор, чудо Хаммерсмита. Ау! Ау! Добрый старый указатель. Ты должен дать ответ. Вот послушайте, Ватсон. Вампиризм в Венгрии. И снова вампиризм в Трансильвании. Он с интересом перелистывал страницы, но вскоре бросил.

— Чепуха, Ватсон, чепуха! Что нам делать с гуляющими мертвецами, которые захоронены в могиле с колом, пронзающим их сердца? Это чистый лунатизм". [SUSS]

Эти цитаты показывают, как Холмс строил свои записные книжки:

"Он взял большую книжку, в которую день за днем записывал данные из разделов убийств различных Лондонских газет.

— Боже! — воскликнул он, переворачивая страницы, — что за стоны, крики и мычание! Что за схожесть! Но определенно это самая ценная основа для поиска, которая когда бы то ни было предоставлялась изучающему необычное! Этот человек один, и к нему нельзя приблизиться ни на йоту, не нарушая абсолютной секретности. Как обойтись без известий или сообщения, которое может дойти до него? Очевидно, можно воспользоваться объявлением в газете. И, кажется, нет другого пути, к счастью, мы можем ограничиться только одной газетой". [REDC]

"Однажды зимней ночью мы сидели вдвоем у огня, он закончил помещать выдержки в свою записную книжку, и я предложил ему ближайшие два часа посвятить тому, чтобы сделать нашу комнату немного более обитаемой". [MUSG]

"Первый день Холмс провел в составлении перекрестных ссылок в огромном указателе". [BRUC]

"По мере наступления вечера шторм становился все сильнее, ветер в трубе кричал и плакал, как ребенок. Шерлок Холмс, задумавшись, составлял перекрестные ссылки на свои записи о преступлениях..." [FIVE]

Эти отрывки из классических рассказов о Холмсе иллюстрируют как тщательность подхода Холмса к расследованию, так и природу неопределенности при сборе требований. Холмс, несмотря на все старание, не смог составить свою записную книжку в терминах, лишенных неопределенности. "Вояж "Глории Скотт", например, как и запись в томе "V", кажется слегка загадочной.

Неопределенность — это состояние, характеризующееся многочисленными противоречивыми интерпретациями текста или ситуации. Неопределенность возникает при изучении требований для системы или БД, которые можно интерпретировать по-разному. Gause и Weinberg выделяют три типа неопределенности, важных для сбора требований [Gause and Weinberg, 1989]:

- *Пропущенные требования*: Они являются ключевой причиной появления различий в интерпретации
- *Неопределенные слова*: Слова в требованиях, допускающие различную интерпретацию
- *Введенные элементы*: Слова, которые приписываются объекту исследования

При разработке системы каталога доступ к Холмсу был ограничен в связи с его напряженным расписанием и обычной немногословностью. Такая ситуация привела к наличию пропущенных требований и элементов. При определенной настойчивости можно продумать детали.

Например, из вышеприведенных отрывков видно, что система включает несколько разновидностей фактов:

- *Биографические факты*: Факты, относящиеся к истории конкретных людей
- *Истории преступления*: Факты, относящиеся к истории преступления из практики Холмса и интересные случаи из мировой практики
- *Энциклопедические статьи*: Факты, относящиеся к интересным объектам для консультации детективов (вампиры, например, или змеи)
- *Платные объявления*: Разделы об убийствах, объявления о потерянных членах семьи и др.
- *Периодика*: Газетные и журнальные статьи, которые добавляют важные факты о людях, местах и событиях, интересных для детектива

Разумеется, источников информации, важных для исследователя, может быть намного больше. Исключил ли их Холмс по каким-то причинам или они просто не всплыли в беседе? А как же статьи из Интернета? А как насчет данных правительства? А сведения о корпорациях, предприятиях с ограниченной ответственностью и других аспектах бизнеса? Телевизионные рекламные ролики (имеющие отношение к обману потребителей, возможно, в крупном масштабе)?

Кроме того, нужно понимать, что сведения в картотеке не являются случайными или универсальными. Однако сложно утверждать, что в ней содержатся только те факты, которые относятся к сфере интересов детектива. Эта неопределенность происходит как из-за неопределенности терминологии (что представляет интерес?), так и из-за того, что мы приписываем некоторые слова Холмсу. Как найти ответы на эти вопросы?

Наблюдение и правильная постановка вопросов

Одним словом: исследование. Нужно исследовать требования, чтобы выяснить как можно больше относительно проблемы, которую предстоит решить [Gause and Weinberg, 1989]. Первый вопрос прост: существует ли решение проблемы? Если вопрос поставлен правильно, имеется ли реальный способ его решения.

Что представляет собой суть проблемы, которую решает картотека? В работе детектива очень важно судить о ситуации с точки зрения фактов, относящихся как к самой ситуации, так и к контексту этой ситуации. Согласно приведенным цитатам, Холмс использует записи из своей книжки для получения фактов, нужных ему при интерпретации различных событий и ситуаций. Проблема, следовательно, может быть определена таким образом:

Агентству Holmes PLC требуется предоставить факты, относящиеся к его практике консультирующего частного детектива.

Это определение дает представление о проблеме, но не содержит достаточного количества подробностей для формулировки решения. В данном случае, однако, все начинается с образца: существующего решения. Имея

такую отправную точку, хочется улучшить определение проблемы, используя в качестве руководства имеющуюся систему. Для этого идентифицируются ограничения системы и, возможно, лежащие в ее основе проблемы, которые пытается разрешить система.

Следующий шаг исследования состоит в задании не связанных с контекстом вопросов, начинающихся с обычных вопросительных слов: *кто, что, почему, когда, где и как*. Вопросы, касающиеся процесса, относятся к сущности процесса разработки, а вопросы, касающиеся продукта, — к сущности продукта разработки. Метавопросы — это вопросы о вопросах. Например, вот несколько проблем, которые могут возникнуть в первом множестве вопросов о каталоге:

- Клиент каталога — консультирующий частный детектив из агентства Holmes PLC. Это частная система и она является интеллектуальной собственностью Holmes PLC.
- В результате роста доходов прибыльность данной системы для клиента составляет, по крайней мере, 20 млн фунтов стерлингов в течение пяти лет. Эти доходы образуются благодаря лучшей идентификации возможных статей дохода и лучших результатов расследований, приводящих к растущей конкурентоспособности услуг детектива.
- Имеется возрастающее давление со стороны клиентов, требующих автоматизированного решения проблем. Клиенты хотят получать более быстрый доступ к лучшей информации, чем это делает имеющаяся система. За последнее время было слишком много Годфри Стаутонов.
- Содержимое системы варьируется от критически важной информации о преступниках и совершенных преступлениях до "миленькой" информации о вампирах и змеях. Критически важные данные включают в себя биографические данные о преступниках и имеющих к ним отношение людях и организациях, истории преступлений, составленные агентством Holmes PLC и полицией, а также сведения из газетной криминальной хроники (этот старый замечательный бумажный институт объявлений в качестве помощника детективов сейчас замещается конференциями в Интернете).
- Не все данные могут быть доступны немедленно. Последние сведения более важны, а исторические данные можно добавлять, когда позволит время. Биографические данные о всех известных агентству преступниках должны присутствовать в первой версии. Информация в текущей бумажной системе важнее других сведений.
- Клиенты хотели бы иметь доступ к текущей бумажной системе через компьютерный поиск в течение года. Старая система должна продолжить свое существование до тех пор, пока новая система не заработает с эквивалентными данными.
- Информация важнее времени в данном случае. Лучше иметь более полные данные, чем получить работающую систему через год.

- Информация предназначена для удовлетворения нужд детективов в фактических данных о преступниках и других людях, имеющих отношение к совершенным преступлениям. Эти факты предоставляют основу для дедуктивной и индуктивной работы детектива; без них нельзя вести эффективные расследования. Компьютерная система решает проблему быстрого доступа к большому количеству данных, а также перекрестного индексирования данных для доступа в соответствии с концепцией. Здесь имеется потенциальная возможность конфликта, потому что разные клиенты могут обращаться к различным отображениям одних и тех же данных: у Холмса может быть интерес к "Вояжам", в то время как Ватсон может пожелать найти "Глорию Скотт".
- Жизненно важно качество данных. Неверные данные не просто бесполезны, они фактически затрудняют дедуктивные и индуктивные умозаключения. Это означает, что средства оценки данных являются существенным компонентом системы, как и способы проведения оценок, а также то, что БД должна быть защищена от случайных или преднамеренных разрушений.
- Безопасность во время системного доступа крайне важна для достоверности данных, она гарантирует клиенту конфиденциальность и секретность при проведении расследования. Объекты расследования не должны иметь возможности определить, имеет ли система о них данные и обращаются ли клиенты к такой информации.
- Наиболее важная проблема для этой системы — комбинированное посягательство на конфиденциальность данных и интеллектуальную собственность. Большая часть материалов, которые являются источниками для сбора информации агентством Holmes PLC, общедоступны. Через некоторое время Holmes PLC будет все чаще использовать информацию, собранную из индивидуальных источников, полицейских агентств и других частных организаций. Это может привести к проблемам с законами и предписаниями о секретности, в частности с законами о прослушивании телефонных переговоров и электронном прослушивании, принятыми различными юридическими системами по всему миру. Кроме того, материалы, собранные из общедоступных источников, могут охраняться законами об авторском праве или праве на интеллектуальную собственность.

Учитывая все это, можно проанализировать, где же существует неопределенность. В любом вышеприведенном предложении есть неопределенность; вопрос в том, насколько она велика, не будет ли риск недопустимо высок. Например, одно требование говорит о том, что полнота данных жизненно важна, даже ценой получения системы через год. Нельзя удовлетвориться требованием в этой форме. Разберитесь, что понимается под термином "полнота данных"? Это может означать многое в зависимости от интересующего множества данных. Например, для биографий преступников полнота может означать наличие всех полицейских записей, относящихся к данному человеку, или может подразумевать полицейские записи, данные газет, отчеты информаторов и любое количество других

источников данных. И когда же отчет информатора "полон"? Это вопросы, на которые можно получить ответ только в том случае, если снова обратиться к клиентам, изложив ситуацию и исследовав их запросы более глубоко. И все же вы, вероятно, закончите этот процесс пониманием того, как получить доступ к достаточной информации по предмету, хотя и с некоторой оговоркой, указывающей, насколько адекватна информация или надежность информаторов. Этот вид оценки метаданных предоставляет действительные данные клиенту о том, насколько они могут полагаться на сведения из БД. Существенной ошибкой было бы теоретизировать до появления данных, и знание о том, каковы данные на самом деле, становится жизненной необходимостью для суждения об истинности таких теорий.

Устойчивость

Устойчивость в данном случае означает две вещи: продолжение вкатывания этого камня и определение, что означают требования устойчивости данных.

Можно провести всю свою жизнь, собирая требования, вместо того, чтобы разрабатывать системы. Хотя бесконечное приобретение знаний полезно, даже если кто-то платит тебе только за это, в некоторый момент необходимо начать разработку программного обеспечения. Знание, когда нужно прекратить сбор и перейти к интерпретации требований, приходит с опытом. Можно использовать тонкие методы оценки, чтобы судить об адекватности требований, или воспользоваться интуицией. Опыт требуется как для сбора начальных данных, так и для развития интуиции, помогающей определить момент, когда информации собрано уже достаточно. Важно исследовать уровень разрешения неопределенности, соответствующий потребностям проекта.

Устойчивость распространяется и на сами данные. Эта книга посвящена разработке БД, и в ней сосредоточено внимание на правилах такой разработки. В последнем разделе перечислены различные требования к системе в целом. И в них всегда имеются скрытые предположения об устойчивости данных. Анализ некоторых требований показывает, что это значит.

- *Клиент каталога – консультирующий частный детектив из агентства Holmes PLC. Это частная система, и она является интеллектуальной собственностью Holmes PLC.*

Ничего особенного, относящегося к базе данных, здесь нет.

- *В результате роста доходов прибыльность данной системы для клиента составляет по крайней мере, 20 млн фунтов стерлингов в течение пяти лет. Эти доходы образуются благодаря лучшей идентификации возможных статей дохода и лучшим результатам расследований, приводящих к растущей конкурентоспособности услуг детектива.*

И здесь тоже..

- *Имеется возрастающее давление со стороны клиентов, требующих автоматизированного решения проблем. Клиенты хотят получить более быстрый доступ к лучшей информации, чем это делает имеющаяся система. За последнее время было слишком много Годфри Стаутонов.*

Здесь появляется первое предположение о базовой технологии. "Более быстрый доступ к лучшей информации" относится к памяти для хранения данных и наличию путей доступа к ним. Одним из квалификаторов является "более быстрый", что может означать все, что угодно. В данном случае "более быстрый" относится к имеющейся бумажной системе. Дополнительный анализ требований, вероятно, выявит некоторые подробности: необходимость доступа из мобильных точек, необходимость немедленного ответа через сеть значительного масштаба и распределение данных в офисах Holmes PLC по всему миру. "Более быстрый" затем начинает относиться к имеющейся системе, при которой детектив вынужден звонить в центральный офис для получения информации путем поиска в бумажной памяти. Термин "более", вероятно, относится к последнему требованию о "полноте" данных, это значит, что мы не только можем перевести БД в электронную форму, но и должны улучшить ее содержимое.

- *Содержимое системы варьируется от критически важной информации о преступниках и совершенных преступлениях до "миленькой" информации о вампирах и змеях. Критически важные данные включают в себя биографические данные о преступниках и имеющих к ним отношение людей и организациях, истории преступлений, собранные агентством Holmes PLC и полицией, а также сведения из газетной криминальной хроники (хотя этот замечательный старый бумажный институт объявлений в качестве помощника детективов сейчас замещается конференциями в Интернете).*

Здесь есть некоторые специфические данные, которые должны быть устойчивыми: биографические данные и данные полиции о преступниках. Имеются также сведения о других имеющих к ним отношение людям, информация об организациях (криминальных, корпоративных, некоммерческих и т. д.), информация об истории преступлений и публикации в средствах массовой информации, содержащие данные о потенциальном преступнике или "интересной" активности.

- *Не все данные могут быть доступны немедленно. Последние сведения более важны, а исторические данные можно добавлять, когда позволит время. Биографические данные обо всех известных агентству преступниках должны присутствовать в первом релизе. Информация в текущей бумажной системе важнее других сведений.*

Это требование аналогично требованию "более быстрого" доступа, рассмотренного ранее.

- *Клиенты хотели бы иметь доступ к текущей бумажной системе через компьютерный поиск в течение года. Старая система должна продолжить свое существование до тех пор, пока новая система не заработает с эквивалентными данными.*

Это требование помогает определить приоритет требований к данным. Подробности в следующем разделе.

- *Информация важнее времени в данном случае. Лучше иметь более полные данные, чем получить работающую систему через год.*

Это общие требования, относящиеся к содержимому данных. В последнем разделе обсуждается огромная степень неопределенности в этом требовании, поэтому его необходимо проверить, чтобы сделать полезным. Оно, однако, критически важно для устойчивости данных. "Полное" в данном случае относится к устойчивым данным, а не ПО, которое осуществляет к ним доступ. Важно понимать, что это требование необычно; время так же или еще более важно, чем полнота информации. Иногда лучше иметь неполные данные сейчас, чем полные во вторник, когда подозреваемый фальшивомонетчик будет освобожден судом присяжных из-за отсутствия доказательств. Эта БД основана на более длительной перспективе, что жизненно важно для понимания, как структурировать приложение и БД.

- *Информация предназначена для удовлетворения нужд детективов в фактических данных о преступниках и других людях, имеющих отношение к совершенным преступлениям. Эти факты составляют основу для дедуктивной и индуктивной работы детектива; без них нельзя эффективно вести расследования. Компьютерная система решает проблему быстрого доступа к большому количеству данных, а также проблему перекрестного индексирования данных для доступа в соответствии с принятой концепцией. Здесь имеется потенциальная возможность конфликта, потому что требования разных клиентов во многом различны: у Холмса может быть интерес к "Вояжам" в то время, как Ватсон может пожелать найти "Глорию Скотт".*

Здесь имеются два элемента, относящиеся к устойчивым данным: методы доступа и вторичные данные. "Огромное количество" означает терабайты данных, которые потребуют специального ПО для управления БД, широкого анализа аппаратных средств и специфических требований для определения путей доступа, таких, как индексирование, кластеризация и декомпозиция. Все это является частью разработки физической БД. Кроме того, перекрестное индексирование требует данные о данных — данные, которые описывают данные способами, позволяющими осуществлять быстрый доступ к требуемым данным. Имеется несколько различных способов организации данных второго порядка. Можно осуществлять поиск в тексте, что не очень удобно для нахождения относящейся к делу информации. Можно разработать систему поиска по ключам, она трудоемка, но увеличивает скорость доступа; это то, что делал Холмс со своими бумажными каталогами. Можно использовать простую нумерационную схему, например десятичную систему Девея, применяемую в библиотеках; наконец, можно разработать улучшенную систему, используя фацеты (*facets*), размерности домена проблем в различных комбинациях. Это классический стиль хранилищ данных. Небольшое исследование может выявить в этом случае требования для расширяемого индексирования, позволяя клиенту добавлять фацеты или классификации к схеме по мере надобности, а не полагаться на статичное представление.

Основное влияние этих требований состоит в необходимости сохранять лицензии и разрешения, касающиеся частных и ограниченных данных. Это другая разновидность данных второго порядка. Это относится также к рассмотренному требованию безопасности, поскольку хранение интеллектуальной собственности в безопасности часто обязательно, как и секретность данных, касающихся индивидуумов.

Четкая расстановка приоритетов

На сборе требований работа не завершается. Не все требования одинаково важны. Следующей задачей является расстановка приоритетов требований, включающая их четкое понимание, соотнесение друг с другом и распределение по категориям. Это итеративный процесс, требующий зачастую более глубокого анализа с помощью вариантов использования, что является предметом следующей главы. До получения необходимого уровня детализации можно пережить сложный период, пытаясь понять, что является истинными приоритетами и что от чего зависит. По мере разработки часто будут встречаться технологические зависимости, которые также следует принимать в расчет, и это потребует пересмотра приоритетов еще раз. Когда основные требования получены, можно начать создавать скелет разработки системы и БД.

Понимание требований

Не следует считать, что достигнуто понимание требований только потому, что имеется список требований к системе. Логика отказов требует, чтобы любая система сопротивлялась усилиям человека по ее изменению. Первое, что нужно сделать — прекратить сбор требований и обдумать их. И думать не только о собранных требованиях, но и о тех, которые могут возникнуть в будущем, об их побочных эффектах. Привлекайте для этой деятельности людей, понимающих домен и имеющих большой опыт решения проблем домена. Проведите занятия по решению проблем. Рассмотрите следующие рекомендации психологии познания [Dörner, 1996]:

- Четко сформулируйте цели
- Определите, где возможен компромисс конфликтующих целей
- Определите приоритеты, затем измените их, если потребуется
- Смоделируйте систему, включая побочные эффекты и долгосрочные изменения
- Определите, сколько требуется данных, и получите их
- Не создавайте излишних абстракций
- Не сокращайте все до оболочки, единственной причины или факта
- Прислушивайтесь к тем, кто говорит, что вы ошибаетесь
- Когда это полезно, используйте методы и избегайте их, если они мешают

- Изучайте свою историю и извлекайте из нее уроки при создании новой системы
- Думайте в терминах системы, а не в терминах простых требований

Чем в большей степени это используется и чем больше вы учитесь на ошибках, тем легче будет собрать требования.

Распределение требований по категориям

На самом базовом уровне требования можно разложить на рабочие цели, свойства объектов, правила и предпочтения.

Рабочие цели

Рабочие цели отражают цель создания системы и подход, применяемый для достижения этой цели. Цели, функции, поведение, операции — все это является синонимами рабочих целей. Цели — это наиболее важный вид требований, потому что они отражают основное предназначение системы, отвечают на вопрос, зачем все это делается [Muller, 1998].

Рассмотрите, например, следующее требование из предыдущего раздела:

- *Содержимое системы варьируется от критически важной информации о преступниках и совершенных преступлениях до "миленькой" информации о вампирах и змеях. Критически важные данные включают в себя биографические данные о преступниках и имеющих к ним отношение людях и организациях, истории преступлений, собранные агентством Holmes PLC и полицией, а также сведения из газетной криминальной хроники (хотя этот замечательный старый бумажный институт объявлений в качестве помощника детективов сейчас замещается конференциями в Интернете).*

Это требование выражает несколько перекрывающихся рабочих целей системы каталога, которые касаются информационных потребностей:

- Предоставить детективам данные, в которых они нуждаются для выполнения своей работы, через подходящий компьютерный интерфейс к БД
- Предоставить сведения об известных преступниках в виде биографических и исторических данных
- Предоставить сведения о людях, интересующих детективов по работе, в виде биографических данных и заметок, собранных из различных средств массовой информации
- Предоставить сведения о преступлениях в виде информации, собранной агентством Holmes PLC, во время полицейских расследований, и в виде новостей, помещаемых в различных средствах массовой информации
- Предоставить сведения о фактах, которые могут пригодиться в криминальном расследовании, опубликованных в энциклопедических справочниках (вампиры и змеи)

- Осуществить быструю и простую интеграцию текущей информации с помощью технологии гибкого интерфейса, связывающего систему с другими подобными системами, принадлежащими полиции и средствам массовой информации и обладающими основными данными, требующимися клиентам

Эти требования выражают цель (сделать что-то) и способ достижения этой цели (через данные или поведение). Некоторые из этих целей касаются пользователей системы (как перечисленные выше), другие могут быть скрыты от пользователя, существуя только ради достижения других целей или для того, чтобы сделать систему более гибкой и легко используемой. В данном случае собака, которая не лает по ночам (эта подсказка навела Холмса на мысль о том, что похищение Сильвера Блейза было его собственной работой [SILV]), – требование к интерфейсу. Системе нужен гибкий интерфейс передачи данных, позволяющий Холмсу легко интегрировать новые сведения в БД. На самом деле детективу все равно, как данные поступают в систему, просто они должны быть доступны, когда потребуются.

Свойства объектов

По мере разработки требований к данным для системы начинают вырисовываться объекты (сущности, таблицы, классы и т. д.), которые будут существовать в системе, а затем и свойства. *Свойства объектов* – это вторая категория требований и, вероятно, наиболее важная для разработчика БД. Свойства могут быть столбцами таблицы, атрибутами объектов или членами данных класса.

Например, ключевой объект в информационных требованиях последнего раздела – это история преступления. Сразу предложим некоторые базовые свойства этого объекта:

- Идентификатор преступления
- Набор юрисдикций, относящихся к делу
- Множество людей, связанных с этим делом в разных ролях (преступник, соучастник, следователь, криминалист, медицинский эксперт, свидетель, жертва, судья, член жюри и т. д.)
- Набор доказательств (просмотрите случай убийства Николь Симпсон, чтобы понять, что важны даже мелочи)
- Стоимость собственности, фигурирующей в деле
- Расходы агентства по расследованию дела

Это классические элементы данных. Конечно, не все свойства объектов непосредственно связаны с элементами БД: достоверность, удобство использования, нетоксичность являются свойствами объекта, применимые к системе программного обеспечения в целом. В основном хорошие свойства объекта очень специфичны: система должна быть доступна 24 ч в сутки, или данные должны отражать все известные сведения, относящиеся к смерти или ранению, или ответ системы должен быть получен в течение двух секунд или меньше для любого элемента данных. В свойствах объекта заключена большая неопределенность: ликвидируйте ее.

Правила

Правила — это условные требования к свойствам объектов. Чтобы, например, стоимость собственности была доступной, она должна быть представлена определенным образом.

Большинство ограничений ссылочной целостности попадает в эту категорию требований. Так, если история преступления ссылается на определенного преступника с определенной кличкой, то как преступник, так и кличка должны храниться в БД.

Правила в отношении времени также принадлежат этой категории. Если имеется рабочая цель — на запрос данных система должна отвечать в течение двух секунд, — ее следует преобразовать в правило о том, что вся информация в БД должна иметь характеристики доступа, обеспечивающие извлечение данных не более чем за две секунды.

К третьей категории относится правило качества. Можно определить устойчивость к ошибкам, что станет частью требований. Например, если сохраняются данные ДНК, ошибка в идентификации отдельного человека должна быть меньше 0,0001, т. е. результаты статистически значимы с вероятностью 0,9999. Эти виды правил устанавливают допуски или границы, за которые не могут выходить данные.

Прежде чем создавать правило, вы должны подумать. Правила имеют тенденцию становиться законами, особенно в проектах по созданию программ с жесткой датой окончания. Убедитесь, что правило необходимо, прежде чем ввести его в проект. Может оказаться, что это правило придется использовать в течение долгого времени.

Предпочтения

Предпочтение — это условие, относящееся к свойству объекта, которое выражает предпочтительное состояние. Например, детектив предпочел бы знать, кто совершил любое преступление. Реально в системе должно находиться описание нераскрытых преступлений — дел, по которым нет преступника. Требование соединения истории преступлений с преступником невыполнимо. Однако можно объединить предпочтение, чтобы истории преступлений имели преступника, с требованием, что информационный источник приложит все усилия для нахождения этих данных. Как альтернативный вариант можно квалифицировать "подозреваемых" или что-то подобное с некоторой долей вероятности, чтобы позволить детективам продолжить расследование. Можно также использовать начальные значения или значения по умолчанию — значения, присваиваемые системой, если другого не определено — для представления предпочтений.

Обычно предпочтения приобретают форму "насколько возможно" или "оптимально" или "в конечном счете" или что-то подобное. Надо понимать разницу между правилами и предпочтениями. Правило говорит "должен", или "будет", или "нужно". Это разница между необходимостью и желанием (как и во многих других аспектах жизни).

Зависимости между требованиями

Требования — это не изолированные жемчужины мудрости, существующие независимо одна от другой. Следующий шаг для понимания требований — объединение их в систему и определение зависимости друг от друга. Согласованная комбинация требований часто может изменить сущность подхода к разработке.

Вместе с требованиями к данным отношения выражаются обычно непосредственно через модели данных, содержащие объекты и их отношения. В последующих главах подробно показано, как это сделать. В нескольких словах — более общие требования имеют и более общие отношения.

Например, сохранение истории преступлений, связанных с преступником, требует создание отношения преступника с некоторым видом событий. Преступник в определенном смысле является собственностью события. Существование преступника как отдельного объекта делает его объектом в системе, связанным с другими объектами, такими, как преступления.

В более общем случае система каталога связывает истории дел по аналогии, т.е. некоторые дела подобны другим. Детективы хотели бы (и очень) получать все дела, подобные тому, которое они расследовали. Это трудная проблема, и вряд ли она может стать серьезной целью в случае каталога. Тем не менее существуют отношения между делами, требующие дополнительных правил в БД. Например, единый язык классификации преступлений мог бы выразить много подобного в различных преступлениях, включая мотив, способы и возможность осуществления. На самом деле довольно сложно выразить это в формате БД [Muller, 1982], но можно попробовать.

Важно понимать, как группируются требования. Связи, которые можно установить между требованиями, показывают обычно, насколько системная архитектура способна организовать свои подсистемы. Если определенный набор требований существенно зависит от одного требования и относительно независим от других, то имеется кандидат в подсистемы. С другой стороны, если проектировщик интуитивно чувствует, что несколько требований идут вместе, но не обнаружено никакого отношения между ними, нужно подумать еще. Значит, что-то упущено.

В системе каталога, например, история дел — это главное требование. Можно связать ее с преступниками, преступлением и другими событиями, людьми, организациями и расследованиями. А вампиры имеют к ним очень слабое отношение. Предшествующие объекты — это сердце подсистемы, относящейся к основной цели каталога: записывать данные о прошлых событиях и людях, чтобы помочь в проведении текущих расследований. Истории дел объединяют разрозненные нити в единое целое. А данные о вампирах случайны и не представляют интереса. Такой элемент данных и подобные ему, возможно, принадлежат отдельной энциклопедической системе, не входящей в подсистему истории дел.

Близкая внутренняя связь существует также между людьми, преступниками и организациями. Эти требования, вероятно, относятся и к подсистеме, показывая, что системы могут содержать другие системы и могут перекрещиваться с ними. Преступная организация, например, отображает

определенные типы людей (как полковник Себастьян Моран) в организации, истории дел и события. Преступная организация, в которую входил Моран, была основной целью расследований Холмса. Значит, она заслуживает отдельной подсистемы данных для выявления сложной сети отношений между людьми, организациями и событиями. Эта система перекликается с биографической системой и системой истории дел, а также с системой событий. Можно увидеть поле деятельности для архитектурного дракона даже на самой ранней стадии разработки.

Все подобные отношения являются областью действия аналитика. Иногда часть такой системы, как эта, включает разработку данных, статистический анализ и другие инструменты для помощи аналитику при идентификации и определении требований.

Возрастающее влияние понимания требований на масштаб и направление проекта, естественно, приводит к следующей теме: определение приоритетов того, что удалось понять.

Определение приоритетов требований

Требования имеют разную форму. Некоторые жизненно важны для проекта, другие неверны. К сожалению, приходится пройти через ряд неудач, чтобы натренировать свой мозг для того, чтобы отличать оплошность от чего-то стоящего. Один из аспектов этого — относительность приоритетов. Они зависят от природы людей, вовлеченных в проект. Сочетание мнений и отношений может радикально меняться от проекта к проекту. Во многих проектах приоритеты необходимо рассматривать полностью заново. Каждый из типов требований имеет разный набор приоритетов.

Приоритеты рабочих целей определяются по их вкладу в основное назначение системы. Если для успеха системы требуется успешное достижение цели, то цель является критически важной. Если можно опустить цель, рассмотрев лишь небольшую проблему, она несущественна. Если цель не имеет отношения к предназначению системы, то это плохое требование.

Приоритеты свойств объектов определяются категориями: необходимо, желательно и игнорировать. Свойство необходимо", если без него система не осуществит свое предназначение. Желательное свойство было бы неплохо иметь. Свойство, которое можно проигнорировать, — это нечто, что может оказаться полезным, но либо его нет, либо оно не стоит необходимых для его создания усилий.

Приоритеты правил и предпочтений определяются по их природе. Правила применяются к необходимым ситуациям. Если правило выражено, его необходимо создать. Предпочтения, с другой стороны, приходят в серых тонах, а не в белом и черном. Определение возможного и оптимального является одним из сложнейших и необходимых моментов при сборе требований.

Определение стиля базы данных

На этом этапе жизненного цикла проекта необходимо создать в своем воображении технологию, которая будет использоваться. Разработчик уже

разглядел подсистемы, начал идентифицировать элементы модели данных, и его техническая интуиция, возможно, заработала. Можно ошибиться в своих ожиданиях, но неплохо хотя бы попытаться подвести в этом месте некоторую итоговую черту. По крайней мере, следует прислушаться к тем, кто думает, что произошли ошибки, и выяснить, почему они так считают. Обратная связь — это важный момент хорошей разработки.

Существует несколько основных стилей БД, применяемых в совершенно разных ситуациях. *База данных OLTP* (обработка транзакций в реальном масштабе времени) — одна из тех, которые поддерживают множество небольших транзакций, обычно в очень крупной базе данных. Например, создание ввода для нового идентификационного документа или ввод данных о новом преступнике в криминальной организации — это небольшая транзакция, повторяющаяся параллельно много раз. Хранилище данных — это БД, которая поддерживает специальные запросы на очень больших БД. Можно отнести исторические характеристики каталога к хранилищу данных. Киоск данных — это хранилище данных небольшого масштаба, возможно, на уровне отдела компании. Например, к этой категории относятся локальные биографические или географические базы данных. Кроме того, имеются специализированные БД: мультимедийные, CAD, географические справочные системы (GIS) и статистические. Системы GIS, в частности, могут содержать в каталоге средства представления данных местоположения преступлений, адреса и другие географические сведения (см. главу 12 в качестве примера такого использования).

Такие категории могут подойти только для решения наиболее общих и специфических проблем. Так, Holmes PLC определенно не является системой OLTP. Эту систему можно отнести к разряду хранилищ данных, но стиль анализа данных в хранилище данных и в киоске данных не тот, который рассчитывают использовать в Holmes PLC. Модель данных в целом сложнее, чем простая топология звезды, отражающая размерности. Вероятно, лучше всего считать базу данных Holmes PLC мультимедийной поисковой БД, но делая ударение на классификации по определенным параметрам, а не на основе текстового поиска.

Очевидными также станут отношения, которые приведут к выбору данного типа модели данных (реляционной, объектно-реляционной или объектно-ориентированной). Если разрабатываемая система будет иметь навигационный доступ с помощью сложного объектно-ориентированного ПО, написанного на C++ или Java, нужно будет использовать продукты OODBMS. Если эта система будет приложением OLTP, основанным на простых формах и специальных запросах, используйте реляционные системы. Если имеется потребность в хранении сложных данных, таких как мультимедийные или географические данные, и нужен доступ через специализированные запросы к этим данным, обратите внимание прежде всего на продукты ORDBMS.

По мере ознакомления с этими моделями данных и технологиями, реализующими их, можно будет использовать более сложные правила для принятия решений при выборе продукта DBMS. Данная книга поможет ориентироваться в этой области.

Итоги

Джеральд Вейнберг цитирует Бернарда Рассела по поводу отношения между тем, что имеется в виду, и тем, что сделано на самом деле:

Я полностью согласен с тем, что в данном случае дискуссия по поводу того, что подразумевалось, важна и исключительно необходима как первый шаг к рассмотрению существенных вопросов, но если ничего нельзя сказать по существу, то обсуждение подразумеваемого кажется пустой тратой времени. Такие философы напоминают мне владельца магазина, у которого я однажды спросил, как скорее всего попасть в Винчестер. Он позвал человека с заднего двора:

- Джентльмен желает знать кратчайший путь до Винчестера.
- Винчестер? — спросил невидимый голос.
- Да.
- Путь до Винчестера?
- Да.
- Кратчайший путь?
- Да.
- Не знаю.

Он хотел прояснить вопрос, но не был заинтересован в ответе. Это как раз то, что делает современная философия для добросовестного исследователя истины [Russell, 1956, p. 169-170].

Требования должны приводить к решениям, а не к новым требованиям. Собрав их, нужно перейти к следующему шагу — чуть более строгому анализу требований с помощью анализа вариантов использования.

Глава 4

Моделирование требований с помощью вариантов использования

Двор собирается. Гамлет сидит около Офелии и делает непристойные жесты, задевающие ее, а также отпускает едкие замечания, мешая королю и королеве. Актеры разыгрывают сцены убийства, напоминающего убийство отца Гамлета, и ухаживания за королевой. Король бросается прочь в смнении, и двор расходится в смущении. Гамлет говорит, что слова призрака заслуживают доверия.

Сценарий, описывающий акт 3, сцену 2 из пьесы Шекспира "Гамлет"
[Fuller, 1966]

Сценарий в литературе — это изложение сцены или истории, в котором сюжет представляется в терминах, понятных читателю. *Вариант использования* в языке UML выполняет подобную функцию для системы программного обеспечения (ПО). Подобно сценариям для пьес, варианты использования могут наилучшим образом описать отражение реальной деятельности системы. Вот почему они не просто полезны, они — существенны. Сделанные надлежащим образом варианты использования на высоком уровне описывают, что должна делать система. Вариант использования имеет, по крайней мере, четыре основных вида воздействия на разработку БД:

- Вариант использования представляет атомарную транзакцию в системе [Rational Software, 1997b]. Знание системных транзакций позволяет разработать логическую и физическую БД и архитектуру, реализующие требования обработки транзакций в системе.

- Вариант использования показывает, применение системой элементов данных как внутри себя через включение, обновление и уничтожение, так и внешне через запросы. Знание наборов элементов данных позволяет организовать их в пользовательские схемы и соотнести с общей концептуальной схемой системы. Вариант использования является основой бизнес-правил, ограничивающих разработку БД.
- Вариант использования предоставляет инфраструктуру для измерения продукта БД. Функциональные оценки, которые делаются для варианта использования, позволяют измерить вклад, вносимый базой данных в проект.
- Вариант использования предоставляет платформу для обоснования БД. По мере постепенного развития проекта и конструкции БД сделанное проверяется с помощью соотнесения с вариантами использования, которые направляют проект. По мере изменения области действия варианты использования также меняются, отражая новые требования и контролируя действия разработчика в соответствии с этими требованиями.

Варианты использования впервые предложены в оригинальной работе Якобсона, до сих пор являющейся актуальной [Jacobson et al., 1992]. UML использует термин *сценарий* для экземпляра варианта использования, определенной реализации варианта использования [Rational Software, 1997a].

Весь мир — сцена

Справочник нотаций UML определяет актера [Rational Software, 1997a, p. 77]:

Актер — это роль объекта или объектов вне системы, напрямую взаимодействующих с ней как часть блока когерентной работы (варианта использования). Элемент Актер характеризует роль, исполняемую внешним объектом; один физический объект может исполнять несколько ролей и поэтому моделируется несколькими актерами.

UML использует изображение фигурки, состоящей из палочек, для обозначения актера в диаграммах вариантов использования (см. рис. 4.1).

Актеры представляют контекст варианта использования. С помощью тщательной разработки актерской модели можно лучше понять, кто или что собирается использовать систему или ее часть. По мере отображения вариантов использования, когда становится понятно, что некоторым актерам нужен доступ к определенным элементам данных специфическим образом, в проекте БД будут заполняться пробелы.

Актер может быть и пользователем, и другой программной подсистемой или даже частью аппаратного обеспечения [Rational Software, 1997b]. Варианты использования не ограничиваются представлением всей системы. Можно представить использование любого интерфейса, внутреннего или внешнего, с помощью варианта использования. Например, рассмотрим подсистему, описывающую обработку изображений фотографий места

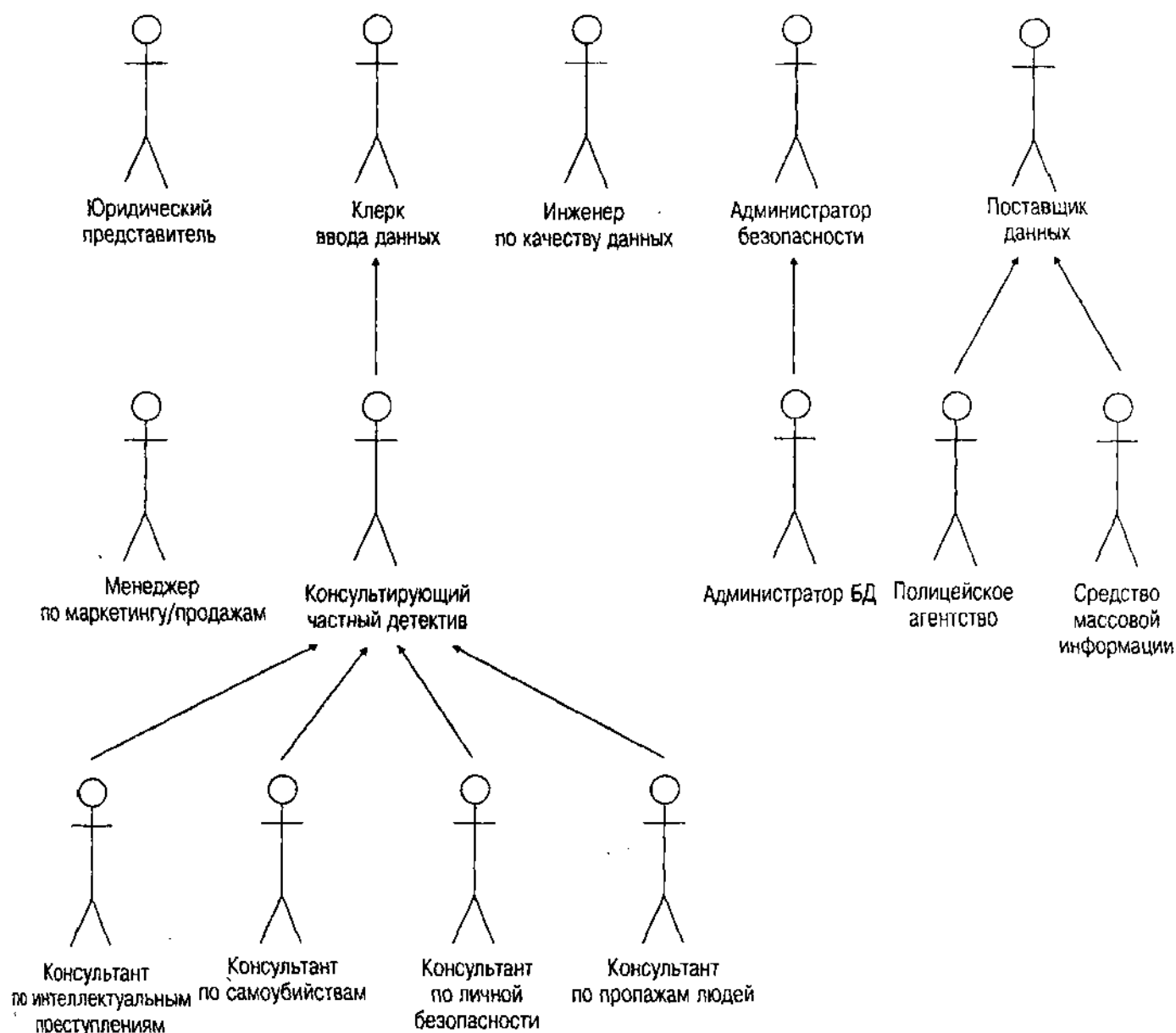


Рис. 4.1. Иерархия актеров вариантов использования

преступления. Эта подсистема предоставляет ряд служб другим подсистемам всей системы каталога. Разработка диаграммы вариантов использования для этой подсистемы обеспечивает измеримые, проверяемые требования для подсистемы места преступления как части всей системы. Конечный пользователь, однако, может не быть актером для данного варианта использования. Вместо этого другие подсистемы каталога становятся ее актерами.

Чтобы разработать модель актеров системы, сначала нужно тщательно определить границы системы. Процесс сбора требований, описанный в главе 3, приводит к общему описанию системы. Для создания начального варианта модели актеров можно использовать эти требования.

Например, для системы каталога к определенным ролям в системе относятся разнообразные требования:

- Консультирующий частный детектив (основной клиент системы)
- Менеджер по маркетингу и продажам (определяет возможную прибыль и собирает рекомендации)
- Поставщики данных из полиции и средств массовой информации (поставляют данные в систему)
- Клерк, вводящий данные (вводит новые данные)
- Инженер по качеству данных (гарантирует высокое качество данных в базе)
- Администратор БД (поддерживает базу данных)
- Администратор по безопасности (поддерживает безопасность системы)
- Юридический представитель (разрешает проблемы секретности и интеллектуальной собственности)

Все эти актеры имеют некоторый повод использовать систему, а система должна предоставлять для них транзакции (варианты использования).

При рассмотрении отношений в целом может оказаться, что определенные актеры связаны с другими актерами. Например, администратор по безопасности может отчасти быть чем-то вроде администратора БД, а администратор БД может также использовать систему как консультирующий частный детектив. Детектив может вводить информацию о делах по мере ее получения, если будет решено объединить ввод данных с интерфейсом запросов. На рис. 4.1 показана иерархическая модель актеров. В данном случае модель разбивает роль консультирующего частного детектива на несколько типов в зависимости от типа преступления, с которым он имеет дело. Разные виды актеров будут использовать различные части каталога. Можно также классифицировать детективов по степени безопасности, если это будет иметь определяющее влияние на структуру системы.

Моделирование актеров в иерархии позволяет упростить описания вариантов использования. В данном контексте наследование означает, что подактер наследует роль, исполняемую его суперактером-предком. Все, что может делать консультирующий частный детектив, делает и администратор. Если такие перекрытия в ролях не смоделированы, нужно подключить администраторов к тем же вариантам использования, что и детективов. Теперь нужно смоделировать варианты использования детективов, затем показать, в каких случаях администраторы делают что-то дополнительно к тому, что могут делать детективы. Возможен и обратный порядок. Детектив применяет определенный набор вариантов использования в системе, разделяя их с администратором. Но администратор владеет определенными вариантами использования, недоступными для детектива.

Актеры также играют пассивные роли в системе. Можно определить различные внешние элементы, которые система контролирует или использует как актеров, если это влияет на требования. Важный момент этого подхода состоит в определении конкретной БД в качестве актера. Например, может оказаться, что система каталога требует четыре БД. Первая — база данных США, содержащая факты, относящиеся к детективной практике в

Соединенных Штатах. Вторая — база данных Великобритании, содержащая информацию о детективной практике по всему миру. Третья — база данных изображений, фотографий, отпечатков пальцев, профилей ДНК и др. Четвертая — текстовая БД, содержащая тексты и графику из средств массовой информации. Две последние системы предоставляют также различные интерфейсы (процессоры поиска изображений и поиска текста), отличающие их от стандартных SQL в базах данных с фактами. Можно задействовать этих актеров, чтобы показать, как различные варианты использования обращаются к системам баз данных.

Актеры баз данных предоставляют также способ для определения элементов данных из БД в вариантах использования. В описании вариантов использования можно перечислить таблицы и столбцы или требующиеся объекты и связать их со специфической схемой БД, в которой они появляются. См. подробнее в разделе "Сводка элементов данных и бизнес-правил".

Определение иерархии актеров предоставляет разработчику штат с целью использования для заполнения сцены: модели вариантов использования.

Актеры на сцене

Имея модель с актерами, теперь можно разработать основную часть модели вариантов использования. Поскольку это итеративный процесс, первоочередное определение актеров, как правило, помогает определить варианты использования более четко. Обычно происходит пересмотр модели актеров при разработке вариантов использования. Можно также найти возможности для соединения актеров с вариантами использования неожиданным образом, где некоторые актеры становятся не имеющими отношения к системе.

Диаграммы вариантов использования

Диаграмма вариантов использования является контекстной диаграммой, показывающей отношения верхнего уровня между актерами и вариантами использования. Можно также включить в эти диаграммы отношения между вариантами использования (отношения применения и расширения). Рис. 4.2 иллюстрирует диаграмму вариантов использования очень высокого уровня для системы каталога. Прямоугольник со скругленными краями — это граница системы каталога. Актеры находятся вне пределов границы системы, а варианты использования — внутри нее. Варианты использования помечены овалами без какой бы то ни было внутренней структуры. Идея состоит в том, чтобы смоделировать транзакции системы на самом верхнем уровне.

Определив каждого актера, подумайте о транзакциях, ими инициированных. Создайте вариант использования в системе для каждой из них и свяжите его с актером ассоциацией (связью от актера к варианту использования). Ассоциация — это связывающее отношение, демонстрирующее, что актер взаимодействует с вариантом использования некоторым образом (активно или пассивно). Обычно начинают с актеров верхнего уровня в актерской иерархии и идут вниз, добавляя специализированные варианты использования для подактеров.

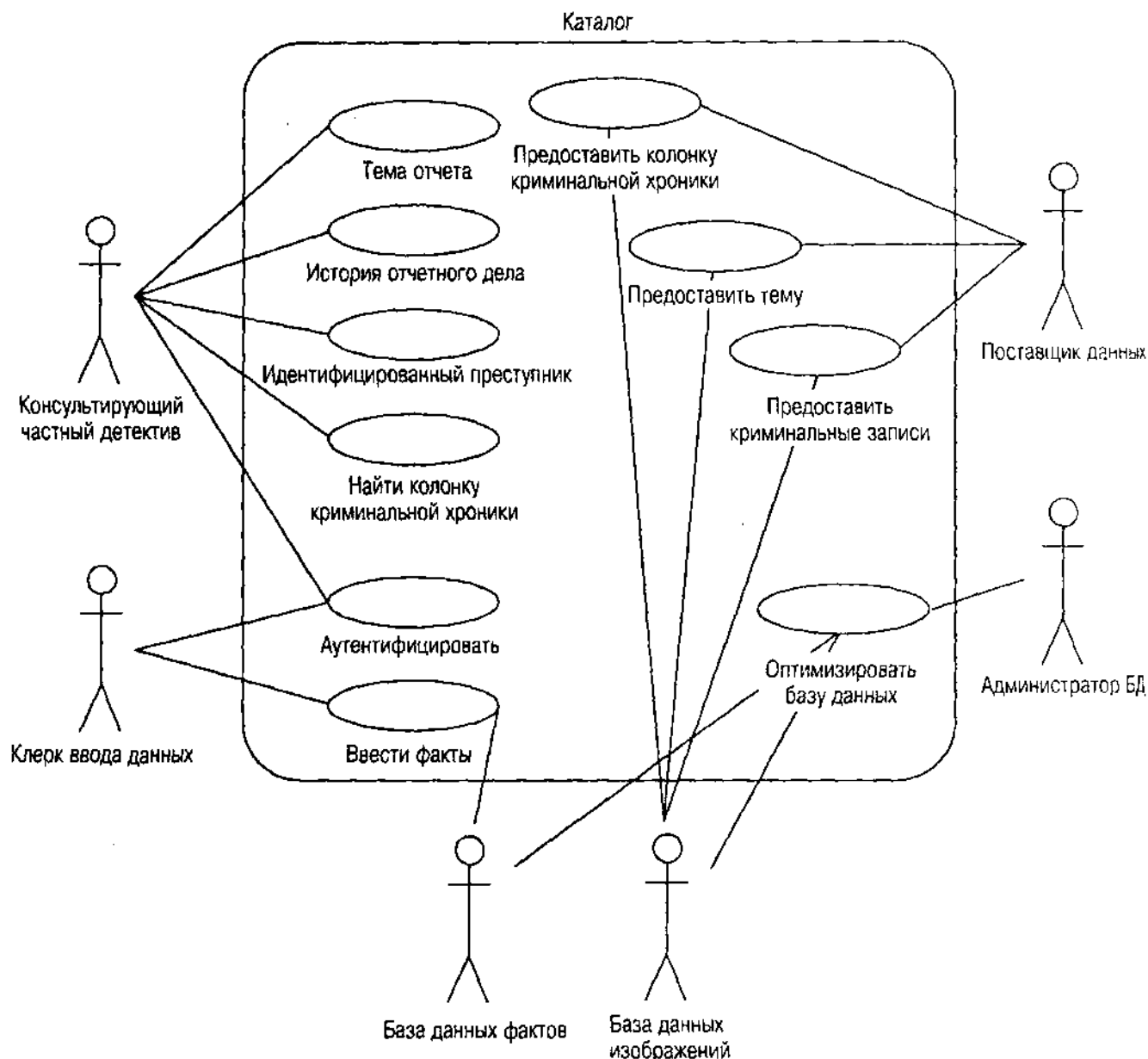


Рис. 4.2. Диаграмма вариантов использования каталога

Теперь подумайте об уже созданных транзакциях и определите, являются ли они на самом деле неделимыми. Это критическая часть определения как системы, так и ее данных. Элементарность — это свойство транзакции, относящееся к ее когерентности как набора операций. Транзакция имеет последовательность операций, которую описывает вариант использования. Сделать эту последовательность атомарной означает, что нужно выполнить всю последовательность целиком или полностью удалить из системы. Нельзя выполнить только часть последовательности. Эта концепция атомарности лежит в основе транзакций БД, а также в основе системных транзакций и образует базис варианта использования.

И, наконец, решите, должен ли каждый из других актеров взаимодействовать с вариантом использования, определенным разработчиком? Если

да, соедините их. Помните, что нужно только связать актеров, которые не наследуют от других актеров в модели актеров. Значит, если актер является подактером для актера, имеющего заданные варианты использования, то этому актеру будут автоматически присвоены варианты использования, уже определенные для первого актера.

Теперь, когда все это продумано, возьмите варианты использования и представьте их реальным людям, играющим роли созданных актеров. Наблюдайте за их работой, опросите их, поговорите с ними о вариантах использования. Завершите петлю обратной связи интеграцией реального мира с моделью этого мира.

Краткий пример

Начнем с консультирующего частного детектива. В первом приближении его транзакции выглядят так:

- *Аутентификация*: подключиться к системе как аутентифицированный пользователь
- *Создать отчет по истории дела*: отчет строится как текст с мультимедийными отрывками
- *Создать отчет о событиях*: отчет о ряде событий на основе критерия поиска в тексте и фактах
- *Создать отчет о преступнике*: создать отчет о биографии преступника в виде текста с мультимедийными отрывками, включая данные полиции и истории дел, относящиеся к преступнику
- *Идентификация клички*: идентифицировать преступника по кличке
- *Идентификация доказательств*: идентифицировать преступника на основе анализа доказательств, таких, как отпечатки пальцев, ДНК и т. д.
- *Создать отчет по теме*: создать энциклопедически полный отчет по теме
- *Найти раздел криминальной хроники*: найти раздел криминальной хроники (объявление личного свойства в газете или в Интернете) по критерию текстового поиска
- *Найти принадлежность*: найти данные о преступной организации на основе критерия поиска по фактам
- *Исследовать*: исследовать отношения между людьми, организациями и фактами в базе данных

На рис. 4.3 показано первое приближение с ассоциациями. Рисунок демонстрирует некоторые связи с другими актерами, такими как БД и поставщики данных из полиции и средств массовой информации. Эти актеры в свою очередь поставляют варианты использования: добавление полицейских записей, отпечатки пальцев, фотографии, тексты средств массовой информации и другие крупницы фактов, которые необходимо иметь в каталоге для удовлетворения нужд детективов. Полная система вариантов использования для каталога может включать 150–200 вариантов использования с пятью-шестью уровнями и больше.

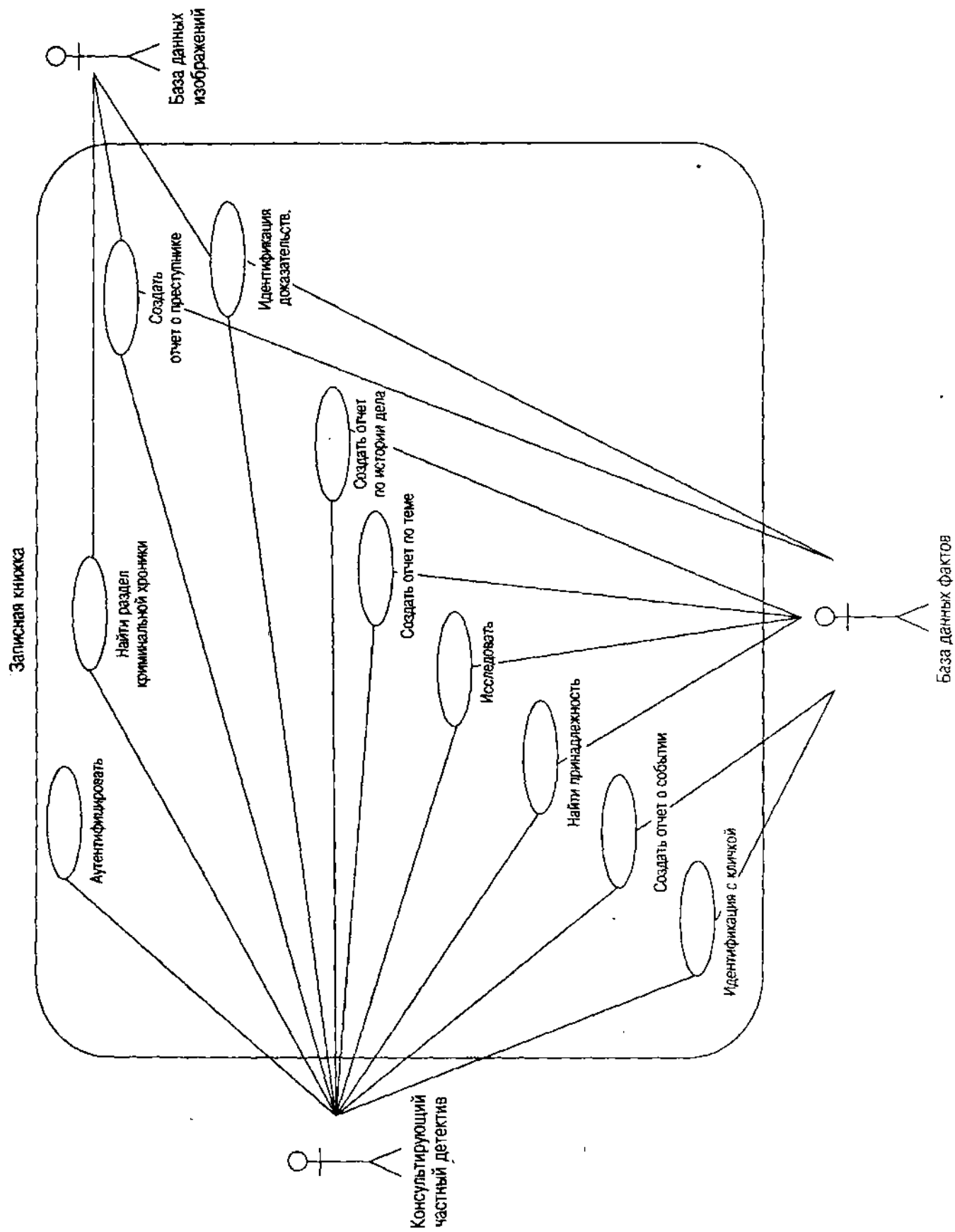


Рис. 4.3. Диаграммы вариантов использования каталога с ассоциациями

Когда основные варианты использования обрastут плотью, обсудите их. Представьте обзор на высоком уровне с описанием важных транзакций детективам всего мира. Бесспорно, они смогут предложить дополнительные варианты использования, которые нужно будет рассмотреть и вставить в систему. Кроме того, обнаружится, что некоторые требования были неправильно поняты, что приведет к необходимости изменить варианты использования.

Транзакции и варианты использования

По мере продолжения системного анализа каждый вариант использования разбивается на несколько, создавая вложенную последовательность вариантов использования, объединяемых для формирования системы в целом. Так, создание отчета по истории дела разбивается на несколько вариантов использования, по одному для каждого основного элемента истории дела. Сюда входит создание отчета по криминальным данным (еще один вариант использования высокого уровня), отчетов о полицейской истории дела, об истории дела в средствах массовой информации, о доказательствах по делу и о перекрестных ссылках дела (вплоть до сведений о криминальных организациях, например).

Эти варианты использования иллюстрируют некоторые аспекты анализа транзакций. Большинство вариантов использования высокого уровня — это транзакции основных запросов. Каждый из них — это транзакция, так как привлекает определенный набор данных с определенным началом и концом. Вариант использования "добавить полицейскую историю дела" — это пример транзакции, которая изменяет БД. Полицейская организация, которая вводит историю дела, делает это во время одной атомарной транзакции. Если транзакция не проходит (либо из-за сбоев аппаратного обеспечения, либо из-за нарушения бизнес-правила, например отсутствия авторизационной подписи), никакие данные из истории дела не попадут в БД. Система полностью аннулирует транзакцию.

Внутреннее разбиение вариантов использования создает проблему управления транзакциями. На самом деле есть две разновидности вариантов использования с точки зрения транзакций — атомарные и податомарные. Их можно легко различить: атомарные варианты использования имеют связующие ассоциации с актерами, а податомарные — расширяют другие варианты использования или используются ими (об отношениях "расширения" и "использования" см. в разделе "Отношения вариантов использования"). Некоторые продукты OODBMS поддерживают модель вложенных транзакций, позволяющую вкладывать атомарные транзакции внутрь транзакций. Большинство продуктов RDBMS предоставляет концепцию контрольной точки — точки в последовательности действий в транзакции, к которой можно вернуться. При использовании любой модели создаются транзакции (или частичные транзакции до контрольной точки), представляющие податомарные транзакции. Можно закончить или аннулировать любой уровень транзакции.

ОСТОРОЖНО

Использование вложенных транзакций и контрольных точек — продвинутый вид управления БД. Не пытайтесь сделать это самостоятельно и не ждите, что отладка будет легкой.

Можно избежать некоторые из этих проблем, поместив все варианты использования на самый верхний уровень. Однако этот вид модели трудно быстро освоить. Проект "разделяй и властвуй" — исключительно удобный способ ограничить сложность и ускорить понимание. Нужно сравнить пользу от этого с возможной нелогичностью, вводимой в модель системных транзакций. Как реализовать вариант использования через пользовательский интерфейс системы? Можно сделать вариант одним из пунктов в меню или кнопкой, управляющей вариантом использования, и затем обращаться с ним как с вариантом использования высокого уровня. Но подумайте о том, как трудно будет работать с системой, имеющей в меню 200 пунктов. Переместите часть системы в другие системы и модули, к которым предоставляется доступ по требованию. Это приводит к более сложной конфигурации системы, однако ее значительно легче использовать.

Рассмотрим также некоторые последствия обработки транзакций при их определении. Основное правило разработки транзакций — сделать их максимально короткими. При обновлении строки заблокируйте ее непосредственно перед обновлением, а не когда она читается. Используйте блокировку обновления, а не исключительную блокировку, чтобы позволить другим читать даже тогда, когда происходит обновление. При включении или уничтожении строки не пользуйтесь исключительной блокировкой таблицы в начале транзакции; позвольте DBMS прозрачно управлять блокировкой. Если применяемая DBMS не управляет блокировкой приемлемым образом, попробуйте получить другую DBMS, потому что эта устарела. Удивительно, но проблемы с блокировкой (особенно с *взаимоблокировкой*, когда несколько пользователей имеют конфликтующие блокировки, взаимно блокирующие друг друга, приводя к отказу транзакции или бесконечному ожиданию) имеются даже у ведущих продуктов RDBMS и OODBMS. Нужно исследовать и создать прототип выбранной технологии на ранних стадиях проекта, чтобы избежать неожиданностей в конце. Особенно если имеются транзакции с большим количеством данных (100 000 строк или более в одном запросе — это много), не предполагайте, что тесты с простыми и ограниченными данными скажут, что на самом деле произойдет, когда начнется промышленная эксплуатация. Эти технологические проблемы кажутся проблемами внедрения, а не проблемами требований, но можно обнаружить, что транзакции вариантов использования являются источником многих проблем, если не принять во внимание основы обработки транзакций.

Отношения вариантов использования

Имеющиеся отношения между вариантами использования в UML: "расширения" и "использования" — являются стереотипными расширениями UML, позволяющими создать особое отношение к уже имеющимся между двумя вариантами использования. Отношение "расширения" говорит, что расширяющий вариант использования случается при определенных условиях

в пределах расширяемого варианта использования. Одно из следствий состоит в том, что расширение — это не вся транзакция, а ее условная часть. Можно создать транзакции совместно применяемых вариантов использования, которые вместе используют большие транзакции, снижая сложность и избыточность всей системы в целом.

Справочник по семантике UML определяет два стереотипа [Rational Software, 1997b, p. 95]:

Общность вариантов использования выражается отношениями "использования" (т. е. обобщениями со стереотипом "использует"). Отношение означает, что последовательность поведения, описанная в одном варианте использования, включена в последовательность другого варианта использования. Последний вариант может ввести новые элементы поведения в любом месте последовательности, пока он не изменит порядок первоначальной последовательности. Более того, если вариант использования имеет несколько отношений использования, его последовательность будет результатом чередования данных последовательностей вместе с новыми элементами поведения. Комбинирование частей для образования новой последовательности определено в применяемом варианте использования.

Отношение расширения, т. е. обобщение со стереотипом "расширение", определяет, что вариант использования может быть расширен с помощью некоторого дополнительного поведения, определенного в другом варианте использования. Отношение расширения включает как *условие* для расширения, так и *ссылку на точку* расширения в соответствующем варианте использования, т. е. позицию в варианте использования, где можно сделать добавления. Когда экземпляр варианта использования достигает *точки расширения*, на которую ссылается отношение расширения, проверяется условие данного отношения. Если условие справедливо, последовательность, подчиненная экземпляру варианта использования, расширяется, чтобы включить последовательность расширяющего варианта использования. Различные части расширяющей последовательности варианта использования могут быть включены в разных точках расширения первоначальной последовательности. Если имеется только одно условие (если условие отношения расширения выполнено в первой точке расширения), тогда все расширяющее поведение в целом включается в исходную последовательность.

Отношение "использования" позволяет выделить часть варианта использования в отдельный вариант для применения во многих транзакциях. Это позволяет идентифицировать совместно используемое поведение, соответствующее в проекте общему коду, а также упростить до некоторой степени модель вариантов использования путем сокращения частей описания варианта использования из нескольких вариантов использования. Самое общее отношение "использования" — это совместное использование простого элемента поведения, такого как вывод на экран графики или вычисление алгоритмического значения.

В модели каталога имеется отношение "использования" между вариантом использования Идентификация преступника с кличкой и вариантом

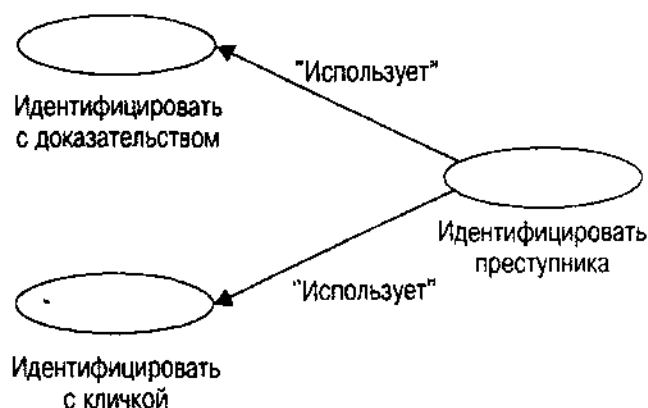


Рис. 4.4. Отношение "использования"

использования Доказательство (рис. 4.4). Оба варианта использования содержат вариант использования Идентификация преступника, представляющий данные актеру, идентифицирующему преступную личность с помощью записи о преступнике, включая имя и кличку преступника, отпечатки пальцев, ДНК (если есть) и фотографии. В первом случае транзакции запрашивают идентификацию преступника, используя кличку. Во втором случае вариант использования идентифицирует преступника с помощью профиля доказательств, например ДНК или отпечатков пальцев.

Отношение "расширения" позволяет упростить вариант использования путем исключения условных элементов, которые отступают от основной цели транзакции: "исключения". Наиболее общий пример этого — обработка ошибок. При определенных условиях вариант использования существует с ошибкой. Например, в системе каталога можно попробовать найти преступника по его кличке и потерпеть неудачу. Вместо исключения этого в описание варианта использования расширим вариант использования путем добавления расширения с условием ошибки и результирующим поведением. Например, можно вывести на экран предупреждение с текстом "Невозможно идентифицировать подозреваемого". Рис. 4.5 иллюстрирует это отношение между вариантом использования Идентифицировать преступника с кличкой и вариантом использования Невозможно идентифицировать подозреваемого. Можно, конечно, расширяя различные варианты использования одним и тем же расширением, создать библиотеку расширений для процедур управления ошибками, которая обычно превращается в библиотеку управления ошибками в проекте системы.



Рис. 4.5. Отношение "расширения"

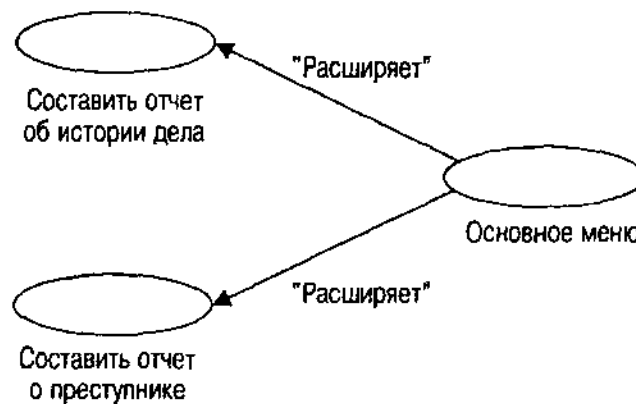


Рис. 4.6. Вариант использования основного меню и его отношение "расширения"

Другой типичный пример — расширение с применением вариантов использования меню и клавиатуры в системе. В пределах любого варианта использования можно получать доступ к общим действиям путем выбора элемента меню или нажатия клавиши. Таким образом, выбор меню и нажатие клавиши — есть варианты использования, относящиеся к основным вариантам использования через условный выбор меню и нажатия кнопки. Рис. 4.6 иллюстрирует диаграмму варианта использования с вариантом использования Основное меню. Этот вариант использования представляет доступное основное меню для Составления отчета истории дела и Составления отчета о преступнике, а также окна приложения каталога. Диаграмма предполагает, что эти два варианта использования — основные транзакции системы, обычно соответствующие "документам" или окнам в интерфейсе с несколькими документами.

Отношения "расширения" и "использования" в основном влияют двумя способами на модель вариантов использования. Во-первых, они упрощают модель благодаря удалению условных ответвлений и общего поведения. Во-вторых, они позволяют оценить систему более точно путем однократного представления одного типа поведения вместо того, чтобы делать это много раз.

Модель вариантов использования структурирует требования в простое представление системы, которую нужно создать. Следующий шаг — создать детальный вариант использования, подсказывающий, что нужно делать архитектору и конструктору.

Подготовка сцены

Цель описания варианта использования — довести до аудитории назначение и поведение системы, которую представляет вариант использования. Это может быть просто абзац текста или сложная полномасштабная модель системы. Нужно учитывать суждения разработчика об уровне детализации и формальности в варианте использования. В большинстве книг, посвященных вариантам использования [Jacobson et al., 1992; Harmon and Watson, 1998], дается простое описание процесса: рассказ о варианте использования. Автор

данной книги обычно идет чуть дальше и включает диаграмму активности, в основном потому, что в его аудиторию чаще всего входит тестирующий. Тестовая модель варианта использования является некоторым видом управления или моделью потока данных [Muller, 1996]. Диаграмма активности служит такой моделью, дающей представление о схеме передачи управления варианта использования.

Применение вариантов использования ориентируется на четыре аудитории.

Первой аудиторией является сам *разработчик*. Знание и исследование требований дает ему обширную картину. Семантический анализ по мере записи вариантов использования кристаллизует мысли и заставляет выстраивать связи, которые иначе не были бы созданы.

Вторая аудитория — *команда разработки*. В зависимости от размера системы можно разрабатывать варианты использования с помощью командных усилий. Другие члены команды должны знать, что делают все части системы. Это необходимо как для понимания их требований, так и для облегчения повторного использования общих мест в наборе вариантов использования. Варианты использования — хорошее место для построения связи фона и контекста, нужные членам команды при разработке и кодировании системы.

Третья аудитория — *клиенты*. Вариант использования — прекрасный способ заинтересовать клиента в разрабатываемой системе. Клиенты привносят свои ожидания, а разработчик изучает их неформальные требования. С множеством вариантов использования системы можно обратиться к клиентам, познакомить их со сценариями в предварительном процессе прототипирования. Это позволит клиентам понять, что разработчик думает о том, что они думают, высказать все замечания и вникнуть в понимание системы разработчиком.

Четвертой аудиторией является *организация*, прежде всего как рецензент. В главе 5 рассматриваются некоторые подробности тестирования вариантов использования. Основная часть этого процесса — критика вариантов использования человеком со свежим взглядом. Чем больше упростить рецензенту понимание вариантов использования, тем вероятнее обнаружение пропущенных проблем и недочетов.

Форма варианта использования может широко варьироваться в зависимости от ориентации разработчика на аудиторию. Желательно иметь резюме варианта использования, повествовательное и/или диаграммное описание деталей, а также ясную формулировку бизнес-правил и элементов данных, на которые ссылаются варианты использования.

Резюме

Резюме варианта использования — краткое описание того, что делает вариант использования. Параграф должен охватывать суть варианта использования, а не детали. Цель резюме — дать рецензенту полное понимание варианта использования. Это может быть очень полезно.

- Резюме фокусирует внимание рецензента на сути варианта использования, удаляя детали, которые могут запутать основное понимание. Оно делает то же самое, что делал разработчик при записи варианта использования, часто проясняя расплывчатое толкование основного назначения варианта использования.
- Резюме способствует быстрому пониманию варианта использования, который подробно рассматривает рецензент.
- Текст резюме можно использовать в тех ситуациях, когда требуется комментарий о варианте использования, краткий отчет или комментарий в коде, который ссылается на вариант использования.

Например, вариант использования "Составить отчет об истории дела" имеет следующий абзац резюме:

Вариант использования "Составить отчет об истории дела" создает отчет об одном или нескольких преступлениях для Частного консультирующего детектива, используя БД фактов и изображений. Для каждого случая отчет содержит краткое описание случая, список людей и организаций, связанных с этим случаем, любые объекты (фотографии, документы, аудио- или видеозаписи) и списки доказательств (включая анализ доказательств), относящиеся к случаю.

В резюме должны быть упомянуты актеры, взаимодействующие с вариантом использования. Например, в резюме варианта использования "Составить отчет об истории дела" упоминается, что отчет предназначен для Частного консультирующего детектива и он использует БД фактов и образов.

Для вариантов использования, выражающих общности или исключения с помощью отношений "расширений" и "использования", нужно резюмировать назначение и упомянуть общую природу варианта или условия (условий), которые могут инициировать вариант. Имеется две цели, которым нужно удовлетворить. Во-первых, резюме должно сказать рецензенту, что вариант использования — это не транзакция целиком сама по себе. Во-вторых, предполагаемые клиенты варианта использования (либо в данном проекте, либо в следующем) должны понять, что предоставляет вариант использования для повторного использования. Вот два случая, использующих предыдущие примеры отношений "расширения" и "использования":

Вариант использования "Идентифицировать преступника" предоставляет общее поведение для более специальных вариантов использования, таких, как "Идентифицировать с кличкой" или "Идентифицировать с доказательством". Вариант использования начинается со списка идентификаторов преступников, созданного с помощью соответствующего варианта использования. Затем он выводит на экран имя преступника, изображение, список кличек, список идентификационных документов и краткое описание истории преступления данного индивидуума.

Вариант использования "Невозможно идентифицировать подозреваемого" расширяет другие варианты использования с помощью исключения, означающего, что соответствующий вариант использования не

смог идентифицировать преступника. Этот вариант возникает, когда соответствующий вариант использования запрашивает идентификацию преступника или преступников из БД и возвращается ни с чем.

Дополнительно можно включить диаграмму вариантов использования с вариантом использования, актерами, его использующими, и любыми вариантами использования, связанными с данным через отношения "расширения" и "использования". Можно также дополнить текст резюме изображением. Некоторые рецензенты понимают отношения варианта использования лучше, если видят граф; другие предпочитают обычный текст. Эти различия приводят к разным способам представления деталей варианта использования — с помощью повествовательного изложения и/или диаграмм деятельности.

ВНИМАНИЕ

Можно также включить в резюме фоновую информацию, хотя бы в краткой и простой форме. Определите основные термины, идентифицируйте основных игроков, объясните главные концепции, необходимые для понимания любым читателем варианта использования, причину его появления. Не забудьте о краткости и простоте.

Описания

Описание варианта использования — последовательность текстов с изложением подробных шагов варианта использования. Обычно оно имеет форму пронумерованного списка шагов, предпринимаемых попеременно актером (актерами) и системой. "Система" в данном случае — программное обеспечение, реализованное в варианте использования.

Правила создания описаний немногочисленны. Главная цель — ясность. По мере составления описания думайте об аудитории варианта использования и придавайте тексту форму, соответствующую их уровню понимания событий. Если сложно описать происходящее в варианте использования на языке, понятном для аудитории, можно вернуться и переделать вариант использования в системе для лучшего отражения того, что должно произойти. Не усложняйте описание.

Кроме того, не следует описывать специфический пользовательский интерфейс или подробности системы, делая по возможности описание абстрактным. Этот прием (сущностный анализ варианта использования) переносит решения относительно пользовательского интерфейса на стадию разработки, в то время как требования имеют дело лишь с общим поведением приложения. Этот перенос решения проблем интерфейса снизит напряжение и беспокойство, когда начнется разработка пользовательского интерфейса. В действительности иногда имеются требования для пользовательского интерфейса, связанные с ранними архитектурными решениями или с явными пожеланиями клиента. Например, если менеджер верхнего звена в Holmes PLC вручит разработчику копию отчета по истории дела и скажет, что желательно, чтобы он в точности отвечал таким-то изменениям, нужно будет вставить специфические интерфейсные требования в вариант использования. Постарайтесь отстраниться от этих обстоятельств, чтобы

убедиться, что клиент в действительности хочет иметь этот формат. Предложите альтернативные варианты и внимательно выслушайте ответ. Однажды потребовалось разработать отчет в специальном формате, который позволял клиенту выводить отчет в форме размером с пол-листа, уже напечатанной компанией. После опроса выяснилось, что можно обойтись без бланков для печати, если создатель отчета сможет делать распечатки неплохого качества, что и было реализовано. При этом основной формат должен был остаться прежним; люди, использующие бланки, работали с ними годами и не хотели изучать новую структуру отчета.

В конце концов если клиент действительно хочет контролировать определенный экран или формат отчета, нужно сделать этот формат частью требований.

Если в варианте использования есть явная ссылка на базу данных, можно выразить данный шаг на языке доступа к базе данных. Для БД SQL это будет оператор SQL, для объектно-ориентированной БД — код C++ или команды OQL. Следует придавать SQL или OQL как можно более общий вид, работая только со стандартным языком ANSI/ODMG по той же причине, по какой мы избегаем специфических элементов управления пользовательским интерфейсом или форматами. Каждый оператор должен образовывать законченный шаг в описании и на диаграмме деятельности варианта использования.

Применение SQL или OQL на этом этапе разработки предполагает наличие схемы. Во время последовательных итераций создания случаев использования определяются классы устойчивых объектов, требуемых для вариантов использования. В главе 7 рассматриваются *статические структурные диаграммы UML* (диаграммы классов, объектов и ассоциаций), представляющие эти объекты. Эти диаграммы нужно разрабатывать параллельно или хотя бы итеративно по мере развития вариантов использования. Такой подход предоставляет основную схему с именами объектов или таблиц и именами атрибутов или столбцов, которые можно задействовать в шагах варианта использования SQL или OQL.

SQL или OQL в вариантах использования дают также четкую картину получения системой доступа к базе данных. Можно применять такие варианты использования на фазе разработки для прояснения понимания того, как система использует БД в терминах обработки транзакций и производительности.

Описание должно непосредственно обращаться к отношениям "использования" в тех шагах, которые основаны на общем поведении. Однако не следует ссылаться на отношения "расширения" где бы то ни было в описании. Ведь отношения "расширения" предназначены для удаления расширения из основного варианта использования для его упрощения. Помещение их обратно в описание, следовательно, снижает продуктивность. Обычно происходит добавление в конец раздела "Расширения", содержащего список расширений. В каждом элементе этого списка подробно описываются условия, при которых расширение раздвигает рамки варианта использования и ссылается на расширение по имени. Нужно идентифицировать в описании точку, где появляется условие расширения, а затем просмотреть расширение варианта использования, чтобы увидеть подробности.

Следующее далее описание представляет вариант использования "Идентифицировать преступника с кличкой", применяя в данном случае подход на основе SQL RDBMS/ORDBMS:

1. Вариант использования "Идентифицировать преступника с кличкой" начинается с того, что Частный консультирующий детектив запрашивает систему идентифицировать преступника, вводя кличку в виде строки символов.

2. Система ищет преступника, возвращая идентификацию преступника (PersonID, Name), с помощью оператора SQL:

```
SELECT PersonID, Name
FROM Person p, Alias a
WHERE p.PersonID = a.PersonID AND
      Alias = :Alias
```

3. Система с помощью варианта использования "Идентифицировать преступника" выводит на экран данные, относящиеся к идентификации преступника. Н этом транзакция завершается.

Первый шаг при создании описания — определение начала варианта использования. Для вариантов использования, относящихся к актерам, это обычно означает идентификацию актера и того, как актер взаимодействует с системой. Взаимодействия — это либо управление (запрашивает систему идентифицировать преступника), либо данные (вводя кличку в виде строки символов). Можно считать данные параметрами для варианта использования. Этот шаг начинает транзакцию.

Второй шаг в этом описании показывает, что делает система в ответ на запрос пользователя. В данном случае используется оператор SQL для описания ввода (:Alias, "переменная хоста"), обработки (объединение таблиц Person и Alias в PersonID и выборка на основе сравнения по равенству со строкой клички) и вывода (проекция PersonID и Name).

Третий шаг идентифицирует отношение "использования". Шаг описания этого типа идентифицирует другой вариант использования по имени. Он может также идентифицировать любые специальные вопросы по поводу отношения. Например, можно смешать шаги другого варианта использования с шагами этого. Это считается противоречащим интуиции, поэтому лучше избегать таких отношений. Этот шаг также явно завершает транзакцию. Нужно пытаться завершить транзакцию явно только в одном месте, на последнем шаге варианта использования. Многие разработчики вариантов использования сочтут, что это налагает сверхограничения, потому что они хотят завершить вариант использования в этой последовательности раньше, на основе определения некоторого условия. Логика варианта использования значительно понятнее, если его структурировать с единственной точкой окончания.

ВНИМАНИЕ

В этой части описания не упоминаются отношения "расширения". Цель такого отношения — получить другой вариант использования и условную логику, извлекаемую надлежащим образом из варианта использования.

На втором шаге можно использовать для определения запроса расширенные SQL или OQL ORDBMS. Например, используя ODBMS, можно было бы иметь логический метод на объекте Person, возвращающий true, если человек имеет кличку, совпадающую с введенной строкой клички. Метод может применять некоторый тип сложного алгоритма сравнения с шаблоном. OQL выглядит так:

```
SELECT p.PersonID, p.Name
FROM Person p
WHERE p.matchesAlias( :Alias)
```

Хорошо, когда шаги соответствуют одному оператору SQL, что делает назначение шага ясным и описание легко понимаемым. Исключение составляют вложенные операторы SQL в некотором виде итерации, где оператор SQL выполняется внутри цикла для каждой строки из внешнего оператора SQL.

Можно также продумать идентификацию и понять, что она является как расплывчатым процессом, так и средством возникновения неопределенности. В терминах БД с такой спецификацией нельзя гарантировать, что для этого запроса будет возвращена только одна строка или объект. Шаг должен быть описан так: "Система с помощью оператора OQL ищет множество преступников, имеющих введенную кличку, возвращая идентификацию преступников (PersonID, Name)".

Для варианта использования "Идентифицировать преступника" есть две возможности первого шага. Если вариант использования может существовать сам по себе (хорошая идея), допустимо начать вариант использования так же, как и предыдущий:

1. Случай использования "Идентифицировать преступника" начинается с того, что Частный консультирующий детектив просит систему идентифицировать преступника, вводя идентификатор человека (уникальное целое число для данного человека).

Если вариант использования всегда расширяет другой и никогда не взаимодействует с внешним актером, нужно заявить об этом в первом шаге.

1. Вариант использования "Идентифицировать преступника" начинается с *другого варианта использования*, его применяющего и передающего ему уникальный идентификатор человека (уникальное целое число для данного человека). Таким образом систем посылают запрос вывести на экран клички, историю преступника и идентификационные данные преступника.

Необходимо также отметить это в резюме варианта использования.

Последняя часть описания — раздел расширений. В этом разделе идентифицируются все "расширения" для варианта использования и условия, при которых они происходят. Например, возможно, что кличка, передаваемая для варианта использования "Идентифицировать преступника с кличкой", не приводит ни к какой идентификации. Можно составить расширяющий вариант

использования "Никакой преступник не идентифицирован", выводящий сообщение об ошибке. В разделе расширений добавляется такой параграф:

1. Вариант использования "Никакой преступник не идентифицирован" расширяет вариант использования "Идентифицировать преступника с кличкой", когда запрос на шаге 2 не может вернуть каких-либо строк или объектов.

Если вариант использования имеет условную логику, часто лучше всего выразить это с помощью подшагов. Например, шаг 5 в варианте использования "Идентифицировать преступника с кличкой" на основании определенного условия выводит на экран данные о роли человека в криминальной организации:

5. Вывести на экран роль человека в криминальной организации. Запросить набор ролей, которые выполняет преступник, с помощью данного оператора SQL:

```
SELECT p.NAME, o.OrganizationID, o.OrganizationName, rt.ShortType
      FROM Person p, Role r, RoleType rt, CriminalOrganization o
WHERE p.Person ID 5 r. Person ID AND
      r.RoleTypeID 5 rt.RoleTypeID AND
      r.OrganizationID 5 o.OrganizationID AND
      p.PersonID 5 :PersonID
```

- a. Если для этого человека нет ролей, выведение на экран текста "<Name> не имеет известной ассоциации с какой-либо криминальной организацией".
- b. Если имеются роли, для каждой из них вывести краткий тип роли и название организации в таком тексте: "<p.Name> играет <ShortType> роль в <OrganizationName> преступной организации".

На эти шаги можно сослаться как на шаги 5a и 5b в других частях описания и на диаграмме деятельности. Циклам соответствуют несколько шагов внутри большего шага.

Вариант использования является обычно частью более крупной системы моделей, включающей модель классов и диаграмму взаимодействия. В зависимости от требований уровня детализации можно абстрагировать сущности в описании варианта использования в модель класса. Можно связать с ней вариант использования при помощи диаграмм взаимодействия [Jacobson e. a., 1992]. В главе 7 рассматриваются подробности использования статических структурных моделей для разработки баз данных. В разделе "Резюме по элементам данных и бизнес-правилам" также предлагается несколько способов создания связей между элементами БД и элементами других диаграмм. Как и с другими частями ООП, эти модели постепенно и итеративно пересматриваются. По мере разработки модели классов можно многое узнать о вариантах использования: какие элементы пропущены, что допустимо организовать иначе, чтобы воспользоваться взаимосвязями классов и т. д. Затем, вернувшись к вариантам использования, улучшить их модели требований. Подобные возможности имеются при формализации описания с помощью диаграмм деятельности UML.

Диаграммы деятельности UML

Каждый программист знает *диаграммы деятельности* как блок-схемы, хотя имеются некоторые различия в нотации и семантике. Диаграммы деятельности используются для представления логики управления варианта использования в графическом формате. Эта модель в некотором роде формализует также поток управления варианта использования. Можно применять данный поток управления для структурирования системного тестирования вариантов использования (см. главу 5).

ВНИМАНИЕ

Диаграммы деятельности применимы для представления потока управления процедур. Они не обрабатывают ситуации с внешними или асинхронными событиями. Если вариант использования должен отвечать на внешний ввод иначе, чем через первоначальное состояние варианта использования, или если он должен отвечать на внешние асинхронные события, необходимо использовать схему состояний UML (иерархическая диаграмма перехода состояний). Обычно схемы состояний используются для моделирования поведения объектов, а схемы деятельности — для моделирования поведения отдельных методов или процессов, например вариантов использования. Оба подхода являются прекрасным способом создания тестовых моделей, так как они непосредственно представляют поток управления.

Элементы диаграммы деятельности

Диаграмма деятельности UML отражает две из основных управляющих структур: последовательность и условие. Последовательность — направленная связь между элементами; условие — эквивалент оператора IF. Рис. 4.7 является простой диаграммой деятельности. Можно также представить итерацию по последовательности, которая соединяет с элементом родовой последовательности (уже выполненным элементом).

Диаграммы деятельности — специальный случай схемы состояний [Rational Software, 1997a]. *Состояния действий*, прямоугольники со скругленными углами на рис. 4.7, — это состояния, представляющие действие без внутренних переходов, по крайней мере с одним переходом при завершении действия, и потенциально с несколькими переходами с защитой или условиями.

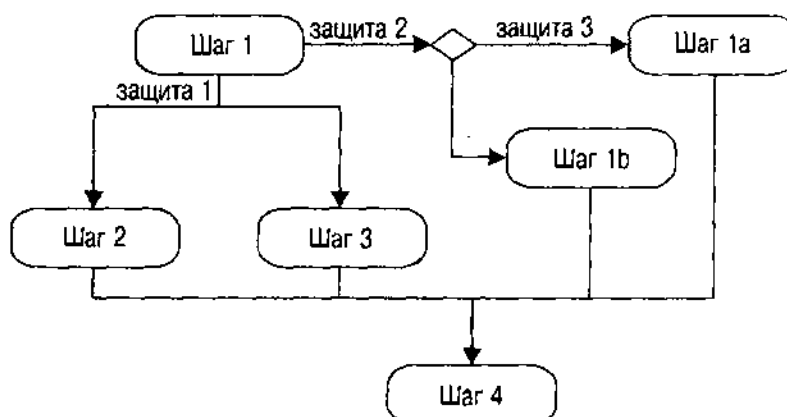


Рис. 4.7. Простая диаграмма деятельности UML

На рис. 4.7 переходы обозначены стрелками. Состояние действия соответствует процессу на блок-схеме, а переход — направлению управляющей стрелки. Можно показать защитные условия как метки на переходах, можно поместить ромб на переход и разделить поток, чтобы показать решение, как на рис. 4.7. Формат решения ближе к блок-схеме и лучше для понимания условного потока управления. Защитные условия должны быть булевы (истина или ложь) и не могут иметь ассоциированных событий.

ВНИМАНИЕ

Поскольку язык SQL, как и БД в целом, поддерживает значения null в трехзначной логике, имеет смысл разрешить трехзначную логику для этих защит в вариантах использования на основе обработки SQL. Если концепция трехзначной логики непонятна, прочтите книги по SQL [Groff and Weinberg, 1994] или статью Дейта на эту тему [Date, 1986, p. 313–334]. Поскольку большинство языков программирования не поддерживают трехзначную логику, это не очень поможет при переводе вариантов использования в код языка программирования, однако, позволит понять результат запроса SQL и поднять потенциальные проблемы с возвращением значений null. Если используется OQL или другой вид логики OODBMS на базе C++ или Smalltalk, вероятно следует выбрать двузначную логику, поскольку эти языки не поддерживают значений null.

На рис. 4.7 вариант использования начинается с шага 1. Шаг 1 имеет два возможных перехода — один для условия Защита 1, а другой для условия Защита 2. Переход, основанный на защите 2, приводит к последующему решению (предикат из нескольких частей, например), что требует совершения шага 1a, если Защита 3 истинна, и шаг 1b — в противном случае. Шаги 2 и 3 совершаются параллельно, как показывает разветвляющаяся линия от шага 1. Нужно завершить шаги 2 и 3 (и, возможно, 1a или 1b) прежде, чем начать шаг 4, окончательный шаг варианта использования.

Рис. 4.8 показывает логику примера из предыдущего раздела "Описания". Эта диаграмма деятельности не сложна, но хорошо отражает основную структуру варианта использования. Каждое состояние действия соответствует шагу в варианте использования. В этом варианте использования переходы последовательны и нет условной защиты или принятия решений. Можно поместить оператор SQL или OQL в состояние действия, как это показано на рис. 4.8, если он не слишком большой. SQL прямо и ясно продемонстрирует содержимое шага. Он также поможет понять требования тестирования варианта использования.

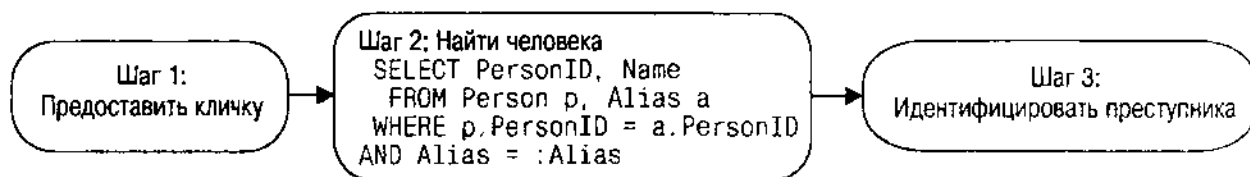


Рис. 4.8. Диаграмма деятельности для варианта использования "Идентифицировать преступника с кличкой"

Диаграмма деятельности как модель тестирования

Модель тестирования представляет элементы системы, интересующие тестировщика. Обычно это означает управление. Двумя основными формами модели тестирования являются диаграмма потока управления и диаграмма перехода состояний. Диаграмма потока управления представляет систему с помощью ее потока управления; диаграмма перехода состояний представляет состояния системы и возможные переходы между ними. Обе диаграммы помогают тестировщикам сосредоточиться на системных элементах, склонных к отказу, что является основной целью тестирования. Система, принимающая решение о том, куда передать управление или какой переход совершить, — это система с возможным отказом.

В главе 5 рассматриваются подробности системного тестирования и вариантов использования. Варианты использования помещаются в контекст системного качества, и становится видно, как получить преимущества от естественной формы варианта использования для структурирования системного тестирования.

Резюме по элементам данных и бизнес-правилам

Последний раздел, посвященный варианту использования, позволяет сформулировать детали требований к БД. Описание и диаграмма деятельности ссылались на элементы данных и ограничения, возможно, неформальным образом. Этот раздел позволит собрать все требования, чтобы продемонстрировать общее влияние варианта использования на БД. Можно также использовать этот раздел для подсчета функциональных баллов при оценке коэффициентов сложности [Garbus and Herron, 1996].

Выходя за пределы БД, можно также подвести итог множеству объектов и операций, применяемых вариантом использования. Как упоминалось в разделе "Описания", эти элементы находят свое выражение в связанных с разработкой диаграммах, например диаграммах классов и взаимодействий. Будут или нет варианты использования явно связаны с этими диаграммами, полезно суммировать их применение в самом варианте использования.

Любой элемент данных, на который ссылается оператор SQL или OQL, должен появиться в резюме. Вариант использования "Идентифицировать преступника с кличкой", в частности, указывает на эти элементы данных в таблице.

**Таблица элементов данных варианта использования
"Идентифицировать преступника с кличкой"**

Таблица	Столбец	Комментарии
Человек	ИдентификаторЧеловека	Уникальное целое число, идентифицирующее человека
	Имя	Составное имя человека (конкатенация имени и фамилии)
Кличка	ИдентификаторЧеловека	Внешний ключ Человек.ИдентификаторЧеловека
	Кличка	Кличка человека; вводится для использования при поиске человека

Бизнес-правила — это ограничения на данные или операции. Варианты использования содержат как простые, так и сложные ограничения, потому что использование данных — это контекст, в котором такие ограничения обычно применяются. Часто, однако, разработка перемещает эти ограничения в базу данных в качестве триггеров или ограничений схемы БД. В любом случае варианты использования определяют правила, чтобы гарантировать отражение точки зрения пользователя системы в проекте. Поскольку разнообразные варианты использования влияют на разные правила использования данных, разработка БД может отражать ввод нескольких вариантов использования, а не только одного. Иногда бизнес-правила даже конфликтуют в разных вариантах использования, что делает необходимым разработку уровней согласования для разрешения конфликтов. Источник этих бизнес-правил — описание варианта использования. Для простоты понимания правил пользователем и разработчиком размещение их в резюме также очень полезно.

Простое правило состоит в том, что человек должен иметь кличку, т.е. наблюдается ограничение по внешнему ключу, требующее, чтобы кличка имела ассоциированный PersonID для идентификации человека. PersonID, следовательно, является частью первичного ключа таблицы кличек, а также внешним ключом таблицы с данными о людях. А как насчет клички, которая известна, но которую нельзя связать с человеком? Например, Шерлок Холмс однажды имел дело с секретным информатором [VALL]:

— Порлок, Ватсон, — это псевдоним, простая идентификационная метка; но за ней стоит скользкая и верткая личность. В последнем письме он честно информировал меня, что имя — не его собственное, и отбил у меня охоту когда-либо проследживать его среди множества миллионов в этом огромном городе.

— ... И Вы никогда не пытались узнать, кто забрал [эти письма]?

— Нет.

Инспектор выглядел удивленным и слегка шокированным:

— Но, почему?

— Потому что я всегда держу слово. Я пообещал, когда он написал впервые, что я не буду следить за ним.

— Вы думаете за ним кто-то стоит?

— Я точно знаю, что да.

— Этот профессор, которого вы упомянули?"

Когда обстоятельства заданы, возможно, возникает бизнес-правило по поводу связи каждой клички с человеком. Это довольно распространенная проблема разработки. Можно обойтись без бизнес-правила и иметь клички, не соответствующие людям. К сожалению, это означало бы, что запросы данных о людях с помощью кличек просто пропустят Порлок, поскольку не будет ассоциированного с этой кличкой человека. Альтернатива состоит в том, что создается "ложная" строка в таблице данных о людях для еще неизвестной личности. Следуя объектно-ориентированному подходу, можно создать подкласс человека, называемый Неизвестный человек, который будет служить исключительно для этой цели. Объект может вести себя как

обычный человек, но имя будет "Неизвестный" или что-то вроде этого. Когда консультирующий детектив поймет, кто этот человек, Неизвестный человек превратится в человека с кличкой.

Другая проблема: что если Фред Порлок" — *псевдоним* нескольких людей, а не одного? Создание жесткой связи между человеком и кличкой сделает такую ситуацию невозможной. Нужно будет вводить кличку несколько раз, по одному для каждого человека, использующего ее. Лучшее проектировочное решение — создать отношение многие-ко-многим между кличкой и человеком с ограничением, т. е. должна быть по крайней мере одна строка в отношении для каждой строки клички. Это дополнительное бизнес-правило, которое надо внести в резюме бизнес-правила варианта пользования.

Вот резюме бизнес-правила:

У человека может быть несколько кличек. Несколько человек могут использовать одну кличку. Кличка должна ссылаться хотя бы на одного человека, даже если этот человек в данный момент неизвестен.

При желании можно выразить бизнес-правила на SQL, OQL или другом эквивалентном языке логики. Если логика не критична, то обычно лучше использовать формат простого естественного языка, который аудитория может легко понять. Формально выразить ограничения может модель классов.

Итоги

Перед самым началом разработки вариантов использования надо создать модель тех, кто ее использует: актеров. Актеры представляют контекст системы, тех, кто осуществляет мотивацию того, что делается. Актеры могут быть частью либо внутренней, либо внешней системы, такой, как БД, к которой осуществляется доступ.

Сами варианты использования представляют собой транзакции (логические единицы работы), выполняемые актерами. Допускается разбиение системы с помощью отношений "расширения" и "использования" для создания податомарного варианта использования, который можно применять совместно с транзакциями. Каждый вариант использования содержит резюме, повествовательное описание, необязательную диаграмму деятельности, таблицу элементов данных, задействованных в вариантах использования, и набор бизнес-правил, ограничивающих варианты использования.

Вариант использования — критически важная часть любого проекта разработки БД. Однако система программного обеспечения, как пьеса, не состоит только из сценариев. Актеры, реквизит, сцена, текст пьесы, аудитория — все это важно для работы. В следующей главе кратко рассматривается роль вариантов использования и требований при тестировании, что облегчит восприятие остальной части книги, посвященной разработке.

Глава 5

Тестирование системы

Есть поговорка, которую в особенности любит лорд Шефтсбери, о том, что насмешка — лучшая проверка истины.

Размышления и мудрость лорда Честерфилда. Эпиграммы

Качество вещи отражает ценность вложенной в нее работы. Хорошо прожаренный бифштекс — это ошибка, если, конечно, таково не было желание посетителя. Весь фокус получения высококачественной прикладной системы БД состоит не в том, чтобы загрузить большое количество данных в дикой надежде, что потом путем проб и ошибок можно будет определить, что в ней неправильно. Вместо этого нужно сравнить конечный результат усилий с потребностями и желаниями заказчиков. В этой краткой главе автор собирается рассмотреть некоторые созданные за многие годы приемы тестирования БД на соответствие требованиям.

Такое тестирование состоит из двух частей: тестирование БД и тестирование требований. Если вы рассчитываете, что проект БД и разработка будут соответствовать набору требований, нужно твердо убедиться в том, что этот набор правилен и полон. Это значит, нужно проверить требования таким же образом, каким проверялся проект и код. Как и любое тестирование, это не однократное тестирование раз и навек, которое надо завершить до перехода к разработке. Оно осуществляется вновь и вновь по мере проектирования и внедрения, всегда задавая вопросы, устанавливая границы требований. Начинают с представления вариантов использования клиентам и получения их отзывов о полноте и точности исполнения их требований в вариантах использования. По мере разработки и внедрения обнаруживаются части вариантов использования, требующие пересмотра, пропущенные варианты использования и различные способы представления проблемы (в отличие от решения). Эти изменения возвращают разработчика к оценке с помощью большего количества бесед с клиентами и наблюдений, всегда подчиненных нуждам проекта для завершения своих задач согласно графику в пределах разумного бюджета. Жизнь — это всегда компромисс.

В процессе оценки сравнивается конечный результат очередного шага проекта с требованиями. Тестирование соответствия системы базы данных требованиям — это критичный тест, который должен пройти проект, чтобы работающее приложение БД было поставлено заказчику. Раздел "Системы и точность соответствия" в этой главе показывает, как связать конечный результат с исходными требованиями при помощи тестирования.

Требования и точность соответствия

Основной риск, связанный с требованиями к проекту, состоит в том, что они неточно отражают реальные потребности заказчиков проекта. *При проверке* требований разработчик стремится снизить этот риск до минимального уровня. На практике это означает заключение соглашения с заказчиками о том, что варианты использования точно отражают то, что они хотят от системы. *Рецензирование требований* — это встреча с целью проверки системных требований, что сокращает риск неблагоприятного исхода.

Варианты использования имеют две аудитории: пользователи системы и ее разработчики. Пользователи — источник требований, тогда как разработчики — их "потребители". Обе стороны должны принимать участие в рецензировании требований через полномочных представителей. Нужно также включить экспертов предметной области, которые могут объективно прокомментировать правильность и полноту вариантов использования. Следует пригласить и других заказчиков (руководителей, правительственных представителей, профсоюзы и т. д.), которые примут участие в рецензировании на различных стадиях.

ОСТОРОЖНО

Проведение рецензирования требований только с людьми, участвующими в проекте, пустая трата времени. В такое рецензирование нужно включить заказчиков, которые будут работать с системой, иначе нельзя произвести истинную оценку вариантов использования. Вот пример: проект с 1000 функциональных точек, прошедший только внутренние рецензии вариантов использования командой программистов. Они обнаружили все случаи, где объекты никогда не удалялись, и все ситуации, для которых варианты использования не соответствовали друг другу. Пропущенных свойств не нашли. Беседа с заказчиками выявила потребность, по крайней мере, в пяти основных вариантах использования, которых никто из команды программистов даже не понимал, что они пропущены. Внутреннее рецензирование хорошо для проверки однородности и непротиворечивости, но не при оценке связи между вариантами использования и реальным миром, который и является истинным предметом такого рецензирования.

Цель рецензирования — обнаружение проблем с требованиями. Рецензирование требований должно рассмотреть все проблемы следующего вида [Freedman and Weinberg, 1990, p. 294–295]:

- *Полнота*: Все ли требования, которые должны быть, указаны? Не пропущены ли какие-либо элементы базы данных или бизнес-правила?
- *Правильность*: Имеются ли какие-либо ошибки в требованиях? Правильно ли определены все элементы данных?
- *Ясность*: Остается ли какая-либо неопределенность в требованиях?

- *Согласованность*: Существуют ли какие-либо противоречия в требованиях? Точно ли определены все допущения? Имеются ли какие-либо элементы данных, которые конфликтуют друг с другом? Есть ли бизнес-правила, противоречащие друг другу?
- *Уместность*: Все ли требования действительно относятся к проблеме и решению? Все ли элементы данных действительно необходимы? Относятся ли некоторые данные или бизнес-правила к вещам вне пределов текущего проекта?
- *Возможность тестирования*: Можно ли будет определить, удовлетворяет ли конечный продукт каждому требованию? Имея схему работающей базы данных, можно ли дать оценку разработке и конфигурации БД на соответствие требованиям? Будет ли база данных столь велика, что ее тестирование станет затруднительно с помощью имеющихся тестовых средств?
- *Трассируемость*: Можно ли будет проследить все функции системы до исходного требования? Имеются ли инструменты и процедуры для соединения комплекта поставки БД с требованиями?
- *Осуществимость*: Можно ли реализовать каждое требование в заданное время с имеющимися ресурсами и инструментами? В частности, существуют ли требования для графика работ и бюджета, противоречащие функциональным требованиям? Может ли выбранная технология БД влиять на производительность и требования к памяти?
- *Необходимость*: Содержат ли требования только необходимые детали, исключены ли ненужные спецификации деталей разработки? Имеются ли в требованиях элементы данных, которые не нужны для удовлетворения требований? Имеются ли элементы или правила в вариантах использования, существующие для совместимости, или они включены просто на случай возможного использования в будущем? Выбросьте их. Они только запутают тестирование.
- *Управляемость*: Подвержено ли множество вариантов использования контролируемому изменению? Можно ли модифицировать требование без каких бы то ни было последствий для других? Имеется ли процесс управления изменениями для схемы базы данных и/или данных в БД?

Систематическая проверка вариантов использования в этих категориях может выявить удивительное количество проблем. Степень их разрешения зависит от вклада проблемы в общую степень вероятности отказа и отношения разработчика к такому риску. Например, разработчик вправе оставить некоторые неопределенности в требованиях, решив исправить их позже, после того, как появится опыт разработки системы. Или, если значимость требования достаточно мала, можно смириться с тем, что система ему не удовлетворяет (желательный приоритет требования).

Системы и точность соответствия

Возможность тестирования кажется не очень важной, просто одним из компонентов в списке для проверки вариантов использования. Однако, этот

фактор — действительно один из наиболее существенных, потому что варианты использования образуют основу для тестирования системы. Большинство других факторов, подлежащих рецензированию, также влияют на возможность тестирования. Так, неверное требование — это требование, которое нельзя протестировать, в отличие, например, от неопределенного требования.

Системный тест — это тест системы в целом, а не по частям, ее образующим [Muller, 1996; Muller, Nygard and Crnkovic, 1997; Siegel and Muller, 1996]. При проверке объектов или отдельных программ тестируют отдельные блоки кода (классы и методы), а в случае проведения интеграционных тестов тестируют подсистемы, не являющиеся отдельными работающими системами. Эти тесты должны включать в себя большое количество проверок и могут обнаружить самые серьезные ошибки задолго до того, как они попадут в системный тест. При поставке рабочей версии системы тестирование приобретает другое значение.

Существует несколько разновидностей системного теста [Muller, Nygard and Crnkovic, 1997]:

- *Тест утверждения/приемки:* Тест, который проверяет систему на соответствие требованиям; приемочные испытания происходят, когда заказчик, а не разработчик системы планирует и выполняет тестирование.
- *Проверка производительности:* Тест, проверяющий специфические требования к производительности и определяющий параметры производительности для последующих итераций системы.
- *Проверка конфигурации:* Тест, проверяющий нормальное функционирование системы в разной аппаратной и программной среде.
- *Тест установки:* Конфигурационный тест, контролирующий процедуры установки в целом ряде различных сред.
- *Полевой тест:* Конфигурационный тест, проверяющий систему в полевых условиях, используя реальные данные заказчиков.
- *Тест на восстановление:* Тест, проверяющий надлежащее восстановление системных данных после блокирующего отказа.
- *Тест безопасности:* Тест, позволяющий определить способность системы предотвращать нежелательное проникновение или доступ к данным неавторизованных пользователей системы.
- *Тест под нагрузкой:* Тест, проверяющий способность системы работать при максимальной нагрузке (много пользователей, много транзакций, большие объемы данных и т. д.); цель — проследить, как система производит отказ, — аккуратно или с катастрофическими последствиями.

Тестовая модель — это системная модель, обобщающая характеристики системы, подлежащие тестированию, которые представляют интерес для тестировщика [Siegel and Muller, 1996; Muller, 1998]. Например, исходный код функции-члена на C++ не является тестовой моделью этой функции; его цель — представить работу функции в терминах, понятных программисту, и таких, которые компилятор сможет перевести в машинный код. Тестовая модель — это модель потока управления или перехода состояний, описывающая поток управления для функции-члена, интересующей тестировщика.

Реальный оператор программы не интересует тестировщика, ему важны только точки, в которых могут быть приняты неправильные решения, или циклы, которые могут выполняться бесконечно.

Варианты использования являются тестовой моделью для теста утверждения/приемки системы. Они могут содержать требования производительности и безопасности или другие требования к возможностям системы, но обычно они не составляют формальную модель этих характеристик системы. Если они образуют формальную модель, то нужно проверить систему на соответствие этим требованиям, как и всем другим требованиям. Иначе варианты использования не имеют никакого отношения к другим видам системного тестирования. Варианты использования представляют объекты интереса для тестировщика утверждения/приемки: тех шагов, которые осуществляет актер при использовании системы.

На практике применяются описания или схемы деятельности в вариантах использования для создания тестовых случаев. Тестовый случай — это отдельный экземпляр тестовой модели. Для варианта использования тестовый случай — это сценарий, специфический экземпляр варианта использования. Если используются схемы деятельности для моделирования варианта использования, можно достичь охвата ветвей варианта использования с помощью отдельного тестового случая для каждого конкретного пути в блок-схеме. Можно также создать дополнительные тестовые случаи с помощью разработки классов эквивалентностей данных, вводимых в вариант использования.

БД влияет на системное тестирование многими способами. Во-первых, база данных часто служит основой тестирования системы. *Основа тестирования* — конфигурация данных, используемых системой в тестовом случае или в *тестовом комплекте* (наборе тестовых случаев, относящихся друг к другу определенным образом). Разрабатываемая система тестирования должна давать возможность осуществить надлежащий уровень тестирования тестовой модели. Другими словами, тестовые данные должны позволить реализовать тщательную проверку варианта использования.

ВНИМАНИЕ

На рынке имеются несколько инструментов для генерации тестовых данных, но как правило, они неприемлемы. Особенно для стиля разработки, описываемого в данной книге, эти инструменты просто не способны создать данные, которые могут привлечь внимание все ассоциации, обобщения наследования и другие семантические аспекты данных. Например, если имеется таблица с внешним ключом для таблицы суперкласса, который в свою очередь имеет внешний ключ для таблицы класса ассоциации с внешними ключами на три другие таблицы, каждая из которых имеет агрегативные композиционные ассоциации с таблицами потомков, то такая ситуация безнадежна для этих инструментов. Они понимают связи не на семантическом уровне, а только на структурном уровне. Случайная генерация данных просто не подходит для такой системы тестирования. Не удалось сгенерировать эффективную систему тестирования в каждом отдельном случае при попытке использовать эти инструменты на БД любой сложности.

Во-вторых, БД часто содержит наряду с данными процедуры поведения, особенно для продуктов на базе OODBMS и ORDBMS и все чаще даже для продуктов на базе RDBMS. Хранимые процедуры, триггеры, методы и операции выполняют специфические элементы вариантов использования системы на сервере БД, а не на сервере приложения или клиенте. Затем начинается тестирование БД, что похоже на любой другой вид системного тестирования, а не просто тестирование данных.

В-третьих, БД может содержать данные, интегрируемые из различных источников. Данные сами по себе становятся частью системы, подлежащей тестированию. Например, Holmes PLC интегрирует в базе данных мультимедийные документы и объекты, данные полиции, записи об отпечатках пальцев и пробы ДНК. Эти объекты данных — такая же часть системы, как и код, выводящий их на экран. Если в данных есть ошибки, система не сможет реализовать свое предназначение. Таким образом, тестирование данных — существенная часть создания прикладной системы с центральной БД. Оно охватывает как тестирование данных на соответствие бизнес-правилам (подсказка: не отключайте ограничений целостности при загрузке данных, потому что потом их забудут снова ввести в действие), так и тестирование точности данных и их содержимого на правильность. Случайные выборки — часто лучший способ проверить содержимое.

В любом случае правильность вариантов использования критична для адекватного тестирования системы. Поскольку системное тестирование происходит после поставки системы, нужно также, чтобы варианты использования не были устаревшими. Если позволить им отклоняться от текущего понимания требований по мере итерации по проекту и конфигурации, может оказаться очень трудно разработать эффективные тестовые случаи проверки при системном тестировании. Нужно также вносить изменения в варианты использования в системе тестирования, чтобы она точно отражала требования к системе.

Итоги

Главный риск на этом этапе состоит в том, что требования могут не отражать реальных потребностей заказчиков. Для определения этого риска нужно проверить требования с помощью рецензирования вариантов использования. Такое рецензирование надо проводить как с помощью членов команды, так и с помощью внешних заказчиков.

Нужно также убедиться, что требования можно проверить. Подготовка к проверке системы завершённой системы на соответствие предъявляемым к ней требованиям, означает разработку как тестовых моделей, так и схемы тестирования, которую можно использовать для таких тестов. Как всегда, это итеративный процесс, требующий пересмотра тестовых моделей и схемы тестирования, поскольку требования меняются по ходу проекта.

Теперь, когда *предназначение* системы известно, пора рассмотреть *способы* ее реализации: разработку базы данных.

Глава 6

Создание моделей сущность — отношение

Превосходство любого искусства — в его интенсивности, способности сгустить все противоречия благодаря близкому родству с красотой и истиной.

Джон Китс. Посвящается Т. и Т. Китс, 13 января 1818 г.

В течение последних 20 лет искусство моделирования данных выросло и изменилось в соответствии с потребностями разработчиков. В современном мире разработки БД доминирующим методом моделирования данных стала диаграмма сущность — отношение (ER). Можно избежать этого метода или, наоборот, значительно расширить его возможности, но нельзя разрабатывать БД без знакомства с приемами построения ER-диаграмм. Даже при отсутствии других причин нужно хотя бы объясниться с теми, кто привержен только ER-методам. Источником многих методов, которые предлагает UML, является ER-моделирование. Понимание ER-моделей составляет половину пути к пониманию объектных моделей. В этой главе описаны некоторые основные концепции, используемые при моделировании данных любого типа.

Здесь приводится краткий обзор ER-диаграмм и моделей данных без подробного рассмотрения методов разработки. Следующие главы целиком посвящены методам моделирования с использованием UML. Обычно сравнение с ER-моделями сделать легко; в последних главах приводятся все значительные расхождения с тем, что делает ER-модель. Таким образом, эта глава вводит основные концепции моделирования данных с помощью ER-диаграмм, а следующие — показывают, как использовать UML для получения тех же или лучших результатов в контексте ООП.

Во-первых, немного истории. Питер Чен изобрел ER-диаграмму при появлении реляционных БД, чтобы моделировать их разработку более абстрактным образом [Chen, 1976]. В то время реляционные базы данных соперничали с сетевыми и другими типами БД, особенно с академическими разработками. Чен хотел найти способ представления семантики модели данных, которая могла бы объединить все эти модели БД, для чего и появились ER-модели.

Десятью годами позже ER-моделирование стало широко использоваться, но возникли многие конкурирующие формы, поскольку различные группы начали ощущать нехватку инструментов разработки БД. Основной прорыв произошел в 1986 г., когда Теорей и его соавторы предложили расширенную ER-модель [Teorey, Yang and Fry, 1986]. Поскольку появление этой работы совпало с появлением CASE-средств как следующего универсального средства, то она стала стандартом для них. Расширение ER-модели включило обобщение (наследование) как основное расширение исходной модели Чена. Расширения возникли на базе той же работы по семантическим сетям, которая привела к возникновению методов ООП.

Несколько других моделей стали использоваться в разных регионах мира. Европейцы сосредоточились на методе информационного анализа на базе естественных языков (NIAM) и методах моделирования данных на их основе, в то время как американцы продолжили развитие разновидностей ER-диаграмм. В 1990-е годы индустрия информационных систем начала широко использовать две разновидности: IDEFIX [Bruse, 1992] и диалекты информационной техники, часто называемые "диаграммы вороньих лапок" [Everest, 1986].

ВНИМАНИЕ

Эти ссылки могут показаться слишком старыми для быстро развивающейся индустрии ПО. Хотелось бы познакомить читателей с более современными учебными материалами по этим методам, но их пока нет.

Примерно в это же время появились первые предложения мира ООП по использованию объектно-ориентированных методов при создании БД. Разработчики языка моделирования объектов (OML) опубликовали исследование, описывающее, как можно использовать OML для разработки баз данных [Blaha, Premerlani and Rumbaugh, 1988], за которой последовала всеобъемлющая книга [Blaha and Premerlani, 1998]. Объектное моделирование OML само по себе происходит из ER-моделирования [Rumbaugh et al., 1992, p. 271] так же, как и подход Шлаера — Меллора [Shlaer and Mellor, 1988].

Поскольку многое в ООП имеет корни в моделировании данных, сходство с разработкой БД было достаточно очевидным. Началось экспериментирование с этими методами, но в связи с низким уровнем усвоения ООП, они не стали популярными. По мере ускорения принятия ООП все быстро меняется, о чем свидетельствует развитие методики, приведенной в данной книге.

Итак, что же на самом деле представляют собой диаграммы сущность — отношение и как их можно использовать?

Сущности и атрибуты

Сущность — это объект данных. Можно запутаться в терминологии, если осваивать это формально. Кратко, нужно различать конкретную вещь, набор вещей с подобными свойствами, определение этого набора и набор таких определений. Слово "сущность" в книге по разработке БД обычно относится к определению набора вещей. Оно потенциально может относиться и к конкретной вещи или даже к набору конкретных вещей. Другие названия для конкретных вещей включают экземпляр сущности, событие сущности, объект, запись или строка (для имеющих реляционную направленность). Иные названия наборов вещей — набор сущностей, результирующий набор, коллекция объектов, расширение, отношение или таблица. Другие названия определений — тип сущности, интенция, тип объекта, класс, отношение (чтобы запутать) или релвар (сокращение от реляционной переменной, формальный термин Дейта для этой концепции). Иные названия набора определений — схема, БД (очень неточное), модель данных и даже проект БД.

На практике термин "сущность" используется для ссылки на прямоугольник в диаграмме сущность — отношение [Teorey, 1999]. Формально — это модель специального определения набора объектов. Расширение сущности — набор объектов в базе данных, моделируемой сущностью. Рис. 6.1 демонстрирует ER-сущность Чена, прямоугольник с именем. Круги, связанные с прямоугольником, — атрибуты (свойства, столбцы) сущности.

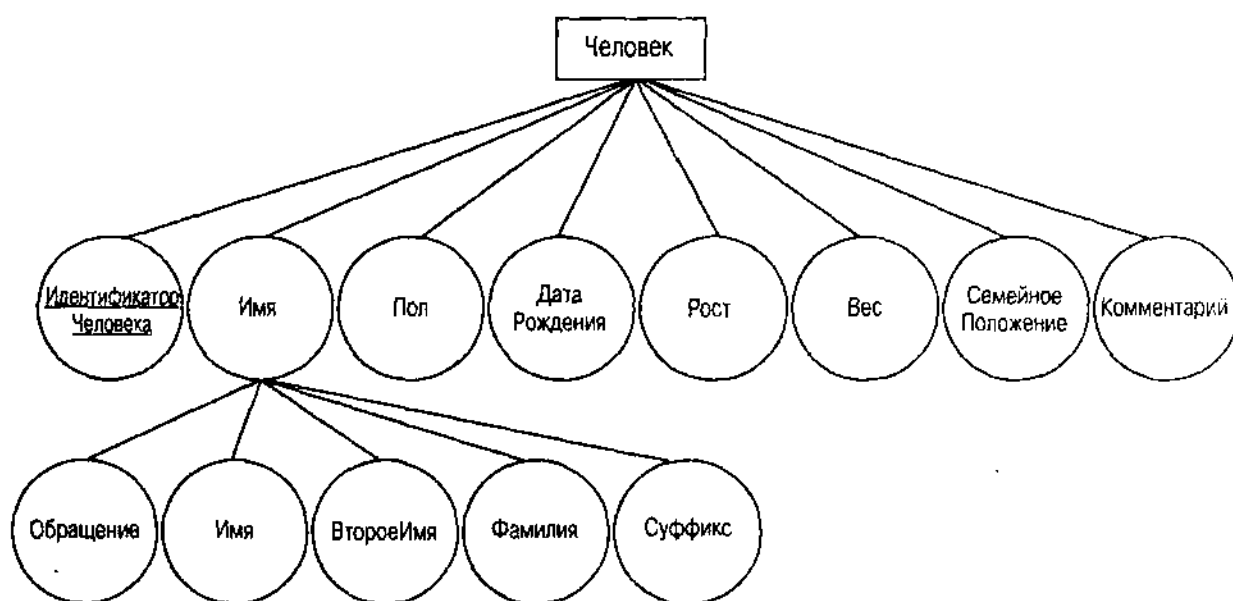


Рис. 6.1. Сущность Чена и ее атрибуты

Сущность Человек состоит из объектов, которые все имеют свойства ИдентификаторЧеловека, Имя, Пол, ДатаРождения, Рост, Вес, СемейноеПоложение и Комментарий. Имя — составной атрибут, состоящий из набора атрибутов (Обращение, Имя, ВтороеИмя, Фамилия и Суффикс). Все эти атрибуты имеют значения для каждого экземпляра сущности (каждого

человека). Подчеркнутый атрибут ИдентификаторЧеловека — идентификатор или первичный ключ для поиска человека, значение, уникальным образом определяющее любой объект.

На рис. 6.2 — IDEF1X-версия сущности Человек. Сущность — это прямоугольник с его именем наверху. Внутри прямоугольника находится тот же атрибут. Прямоугольник имеет две части. Верхняя часть содержит все атрибуты первичного ключа (идентифицирующие атрибуты). В нижней части содержатся все другие атрибуты. Этот формат требует гораздо меньше места, чем подход с атрибутами в кругах. Он также довольно похож на формат класса UML (см. главу 7).

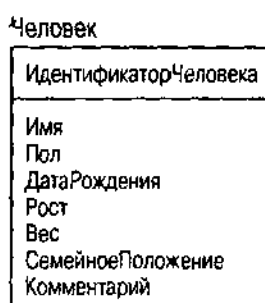


Рис. 6.2. Сущность IDEF1X и ее атрибуты

ВНИМАНИЕ

В подходе IDEF1X нет возможности показать сложные атрибуты. IDEF1X отражает атрибуты в первоначальной нормальной форме, что предполагает отсутствие внутренних компонентов в атрибуте IDEF1X. Можно смоделировать сложное имя как отдельную сущность, затем соотнести его с Человеком через отношение. Как будет показано в главе 7, UML имеет подобное требование, но исходит из другого источника: инкапсуляции внутренней структуры объектов. (В главе 11 см. дополнительный материал по нормальным формам.)

Отношения

Другая половина ER-диаграммы — отношение. *Отношение* — это модель связи между объектами одного типа сущностей или между объектами разных типов сущностей. Можно сделать такое же расширение формального языка для отношений, показанных в уже рассмотренном разделе "Сущности и атрибуты" для сущностей. Отношения показывают, как каждая сущность соединяется с другими сущностями. Они соответствуют отображениям между атрибутами различных сущностей. Эти отображения могут быть объектами сами по себе, имеющими свойства и первичные ключи (об этом рассказывается ниже в разделе "ER-бизнес-правила"). На рис. 6.3 показано две сущности — Человек и КриминальнаяОрганизация, отображенные одна в другой с помощью отношения "играет роль в".

ER-отношение всенаправленное; т.е. каждая сущность имеет отношение со всеми другими сущностями в отношении и может "видеть" эти сущности. Каждая сторона отношения (связь, соединяющая ромб с прямоугольником) —



Рис. 6.3. Отношение между двумя сущностями

это роль, функция сущности в отношении. Можно присвоить имя каждой роли для внесения ясности, хотя это и не обязательно. По договоренности для отношений и имен ролей используются глаголы, а для названий сущностей — существительные. Затем строятся предложения с надлежащим субъектом и объектом.

На рис. 6.4 показано отношение между сущностью Человек и сущностью Идентификация. С точки зрения человека, человек идентифицируется набором идентификационных документов, с точки зрения идентификации — она идентифицирует человека. Название роли со стороны человека — это, следовательно, "идентифицируется по", в то время как название роли со стороны идентификации — "идентифицирует". Этот активно-пассивный способ именования очень распространен, хотя определенные типы отношений не поддерживают такое присвоение имен. Можно назвать отношение одним или другим именем роли или отдельным названием, которое имеет смысл в контексте. В случае, представленном на рис. 6.4, отношение называется "идентифицирует", что имеет смысл в контексте.

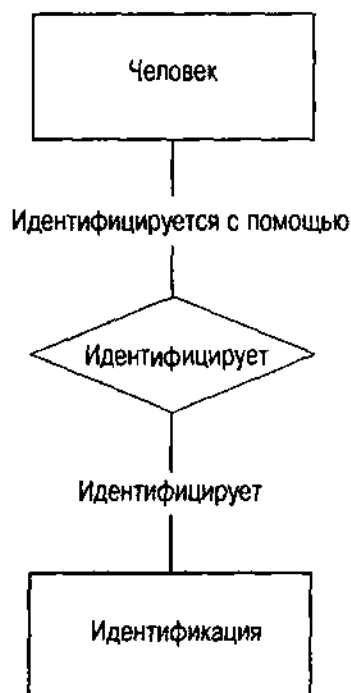


Рис. 6.4. Роли в отношении

ВНИМАНИЕ

Обычная ловушка — присваивать только уникальные имена ролям. В течение многих лет все имена в БД должны были быть уникальными. Не существовало концепции области действия или пространства имен. Все системы баз данных позволяют теперь определить область действия большинства имен на соответствующем уровне в схеме, поэтому на самом деле нет причины делать все имена атрибутов уникальными. Имена ролей (которые могут соответствовать именам атрибутов в реляционной таблице или классе) также могут не быть уникальными. Цель присвоения имен ролям — ясность: что отношение означает в контексте сущности. "Имеет" обычно хорошо подходит или можно включить объект существительного ("имеет адрес", например, в отношении между Человеком и Адресом). В ООП инкапсуляция усовершенствует эту функцию до принципа: имена в классе должны отражать точку зрения класса, а не уникальные имена всех других классов в системе. Как будет видно в главе 7, другой взгляд на имена ролей — считать имена собственностью класса, т. е. имеющими область действия в классе и, следовательно, уникальными только в пределах класса.

IDEF1X не использует ромбов и представляет отношения просто с помощью помеченной линии. Нотация "вороньих лапок" подобным образом представляет отношения. На рис. 6.5 показано отношение "идентифицирует" в этих обозначениях.



Рис. 6.5. Отношение "идентифицирует" в IDEF1X и нотации "вороньих лапок"

Рис. 6.6 иллюстрирует специальное отношение. Рекурсивное отношение соотносит объекты сущности с другими объектами того же типа. В данном примере хотят создать модель отношения между криминальными организациями. Криминальная организация может быть частью другой криминальной организации или нескольких таких организаций. На рис. 6.6 показано это отношение по Чену, IDEF1X и в нотации "вороньих лапок".

Отношение может иметь собственные свойства дополнительно к тем сущностям, к которым оно относится. Например, при желании иметь

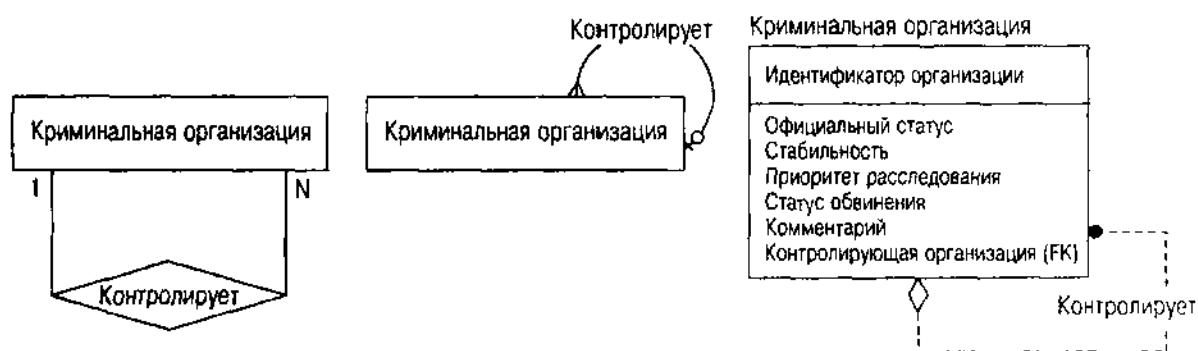


Рис. 6.6. Рекурсивное отношение в различных нотациях

атрибут отношения "играет роль в" на рис. 6.3, который присваивает имя роли (Босс, Его подчиненный, Лейтенант и т. д.). Это свойство ER-диаграммы расширяет семантику отношений для рассмотрения отношения как объекта со своими правами и свойствами. Классический пример этого — свадьба. Свадьба — это отношение между двумя людьми, у которого есть законные свойства как объекта. Можно смоделировать эти объекты как сущности, но они на самом деле являются отношениями со всей ассоциированной семантикой отношений.

Отношение имеет несколько свойств, представляющих интерес для разработчика баз данных:

- Количество объектов, которые *могут* участвовать в роли
- Количество объектов, которые *должны* участвовать в роли
- Количество сущностей, которые участвуют в отношении
- Сильное ли отношение (независимое) или слабое (зависимое)

Эти свойства — основной способ для ER-диаграммы определять бизнес-правила — предмет рассмотрения следующего раздела "ER-бизнес-правила". Каждый диалект имеет различные способы выражения этих ограничений. Однако имеются некоторые расширения ER-моделирования, которое представляет другой вид отношений: семантические.

Семантические отношения: порождение подтипа и агрегация

В 70-х годах XX века появились несколько направлений научного исследования ПО, которые взаимно обогащали друг друга идеями о семантических отношениях между структурами данных. Термин "семантический" означает или относится к значению объекта в отличие от его структуры (синтаксис) или его использования (прагматизм). ER-нотация Чена описывает часть этой фоновой теории данных с помощью интегрирования отношений и их ограничений как абстракций, это был значительный вклад в моделирование. Последние ER-нотации идут еще дальше, вводя концепцию порождения подтипа [Teorey, Yang and Fry, 1986] и агрегации [Bruce, 1992].

СОВЕТУЕМ

Обычно можно выполнить прекрасный проект БД, используя методы построения ER-диаграмм, не беспокоясь особенно о порождении подтипов и агрегации. Однако когда разработка ведется с помощью ОО на UML, эти проблемы оказываются очень важными.

Порождение подтипов также известно под несколькими другими именами: генерализация, наследование, является, является чем-то вроде, создание подклассов. Каждое из них происходит от основной используемой концепции генерализации в разных контекстах. Отношение *генерализации* определяет, что супертип обобщает свойства нескольких подтипов. Другими словами, супертип содержит свойства, которые разделяют подтипы, а все подтипы наследуют свойства супертипа. Часто супертип — это в чистом виде абстрактный тип, означающий, что нет экземпляров супертипа, а имеются только подтипы.

ВНИМАНИЕ

В главе 7 более подробно рассматривается наследование, являющееся фундаментальным свойством моделирования класса. В ней также обсуждается множественное наследование (наличие многих супертипов), чего никогда не встретишь в ER-моделях.

Рис. 6.7 иллюстрирует отношение подтипа, выраженное в нотации IDEF1X. Идентификация — это абстрактный тип объекта, идентифицирующий человека, и имеется много разновидностей идентификации. Иерархия подтипов на рис. 6.7 показывает дополнительный абстрактный подтип, идентификации, имеющие даты окончания срока действия (паспорт, водительское удостоверение и т. д.) в отличие от тех, которые ее не имеют (свидетельства о рождении).

Можно также добавить *взаимную исключительность* как ограничение этого отношения [Teorey, Yang and Fry, 1986; Teorey, 1999]. Если подтипы сущности взаимоисключаемые, это означает, что объект или экземпляр сущности должен быть только один среди подтипов. Иерархия идентификации представляет этот вид создания подтипов. Невозможно иметь идентификационный документ, который одновременно является, например, и паспортом и водительским удостоверением. Если подклассы не взаимоисключаемые, тогда подтипы перекрываются по значению. Можно, видимо, представить документ, который одновременно являлся бы и паспортом, и водительским удостоверением, и свидетельством о рождении. Лучшим примером была бы организация, являющаяся одновременно криминальной организацией и законной компанией, очень распространенная ситуация в реальном мире. Базовая реальность такого типа для поиска подтипов — перекрывающийся набор объектов. В большинстве случаев этого желают избежать, поскольку это делает объекты (строки в базе данных) взаимозависимыми, что затрудняет поддержку при изменениях. С другой стороны, если базовая реальность на самом деле перекрывается, моделирование должно это отражать.

И, наконец, можно ограничить отношение генерализации с помощью ограничения *полноты* [Teorey, Yang and Fry, 1986; Teorey, 1999]. Если подтипы сущности полны, это значит, что объект этого типа должен быть одним из подтипов и что нет дополнительных (неизвестных) подтипов. Это ограничение позволяет отразить ситуацию, при которой подтипы

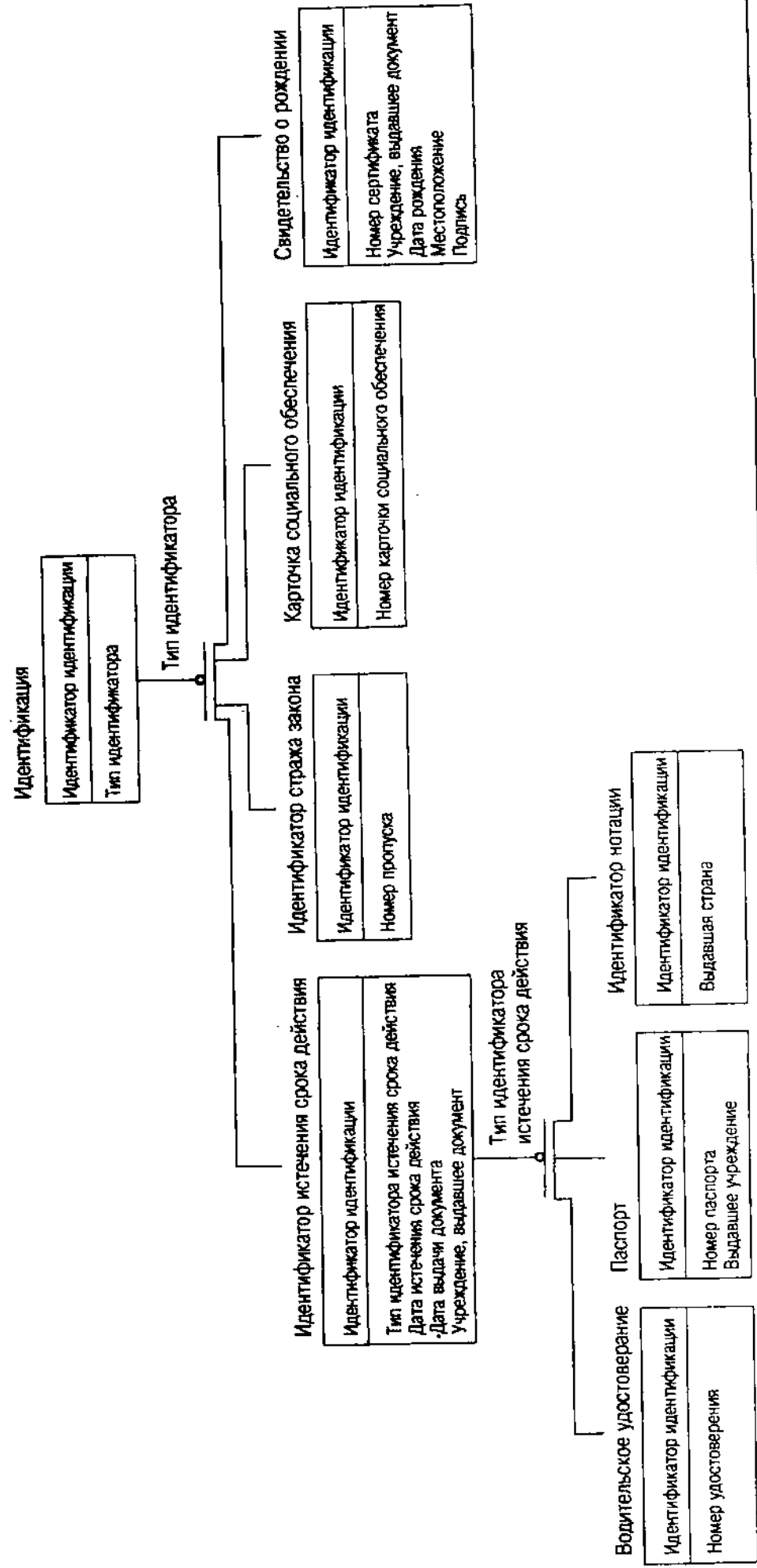


Рис. 6.7. Иерархия подтипа идентификации, использующая нотацию IDEF1X

что он держал в руках какое-то ружье со странным прикладом. Он открыл его сзади, что-то положил в него и защелкнул приклад. Затем, пятясь назад, он опустил конец ствола на край открытого окна, и я увидел его длинные усы, склоненные над ружьем, и его глаз, блестящий при попадании в поле зрения. Я услышал небольшой вздох удовлетворения, когда он прижал приклад к плечу, и увидел удивительную цель, чернокожего на желтой земле, взятого четко на мушку. В течение мгновения он был неподвижен и спокоен. Затем его палец потянулся к спусковому крючку. Послышался странный громкий свист и продолжительный серебряный звон разбитого стекла. В это мгновение Холмс бросился, как тигр, на спину меткого стрелка и сильно ударил его по лицу.

...Холмс поднял мощное духовое ружье с пола и стал изучать его механизм.

— Замечательное и уникальное оружие, — сказал он, — бесшумное и очень мощное: я знал фон Хердера, слепого немецкого механика, который создал его по приказу старого профессора Мориарти. В течение многих лет я знал о нем, хотя никогда раньше не имел возможности использовать его. Я рекомендую его вашему вниманию, Лестрейд, вместе с пулями, которые подходят к нему". [EMPT].

Рис. 6.8 иллюстрирует основы разделения на части в нотации IDEF1X. Каждый компонент ружья является частью другого, при этом есть части верхнего уровня (духовое ружье в вышеприведенной цитате, например), не имеющие никакого отношения (нет родителя). Эта структура представляет собой лес из механических деревьев, отношение один-ко-многим между механизмами и частями. Можно превратить это отношение в отношение многие-ко-многим, представляющее собой лес графов или сетей вместо иерархических деревьев. Агрегация представляется ромбом в конце линии отношения. Отношение сильное, или идентифицирующее, поэтому сущность получает атрибут внешнего ключа, не являющийся частью первичного ключа ("является частью"), и пунктирную линию (см. нижеследующий раздел "Сильные и слабые отношения" с подробностями о идентифицирующих отношениях).

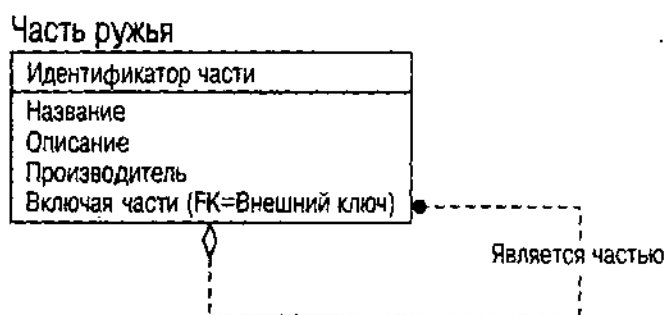


Рис. 6.8. Разделение на части духового ружья

Агрегация может быть интересна и полезна, но она, однако, редко влияет на основной проект БД. Главное влияние — в более сильном связывании проекта. При разработке системы она разбивается на подсистемы, разрывая связи между элементами как можно больше. Это позволяет повторно

использовать элементы в разных ситуациях. При идентификации агрегата обычно оговаривают, что нельзя разделить его с сущностью, которую он агрегирует. Значит, нельзя разделить две эти вещи и поместить их в разные подсистемы. Часто обобщение (шаблоны или родовые классы) допускает повторное использование при тесном одновременном связывании сущностей (как экземпляров шаблонов или родовых классов). ER-диаграммы не имеют концепции обобщения, однако эта концепция не слишком важна при ER-агрегации.

Кроме того, при ООП агрегация — это вид абстракции, точно так же, как и наследование. В этом случае происходит абстрагирование и инкапсуляция агрегированных объектов под агрегатом. Обычно агрегат будет иметь операции, управляющие инкапсулированными объектами, его образующими, пряча структурные подробности. Например, чтобы сделать поиск по дереву, создается операция, возвращающая итератор, который, в свою очередь, позволяет пройти по дереву в некотором хорошо определенном порядке. В терминах проектирования, если нежелательно иметь данный уровень инкапсуляции, не следует объявлять отношение агрегатом.

ВНИМАНИЕ

Можно представить почти любое составное отношение как агрегацию, если достаточно расширить семантику. Можно назвать атрибуты сущности агрегацией, например: сущность сделана из атрибутов. Это возможно, но не приносит пользы разработке. Следует ограничить использование агрегации такими ситуациями, как разделение на части или физическое включение, когда результат означает что-то специальное для разработки и кодирования. Это приемлемо, когда имеет важное значение для структур БД, как в случае дерева криминальной организации.

ER-бизнес-правила

Бизнес-правила — это любые логические условия, которые должны быть истинными при завершении транзакции. Более широко, бизнес-правила — любые правила, которые желательно ввести при управлении базой данных любой сложности. С точки зрения разработки БД бизнес-правила разбиваются на две категории: ограничения целостности и правила базы данных. Имеются и другие бизнес-правила, вводимые на уровне приложения, такие, как требования потока управления, и правила, влияющие на переменные данные. Эти правила не касаются разработчика БД, если они не влияют некоторым образом на транзакции. В объектной системе обычно имеется объектная модель или уровень домена, представляющий объекты системы, и этот уровень приложения управляет бизнес-правилами.

ВНИМАНИЕ

Конечный отчет по международному проекту бизнес-правил GUIDE проделал прекрасную работу по формальному определению концепции бизнес-правил [GUIDE, 1997]. Следует обращаться к этому документу для получения полных ссылок по всем вопросам, связанным с бизнес-правилами.

Ограничение целостности — это логические отношения между сущностями, накладывающие некоторые ограничения на эти сущности. Концепция непосредственно соответствует отношению из предыдущего раздела. *Правила БД* (такие, как безопасность или ограничения физической структуры) не зависят от ограничений конкретных сущностей и должны выполняться базой данных. В контексте ER-диаграммы имеются три основных ограничения: множественность, само отношение ("ограничение внешнего ключа") и первичный ключ. Можно также ограничить взаимодействие внешних и первичных ключей; ER-диаграмма делает это с помощью сильных и слабых отношений. Можно также определить ограничения домена на те данные, которые представляют атрибуты.

Множественность

Множественность — количество объектов, которые могут участвовать в роли отношения. Значит, множественность ограничивает область изменения количества объектов, относящихся к другим объектам через особую роль. В исходном подходе Чена [Chen, 1976], переменные буквы или постоянные целые присваиваются ролям, указывая значение множественности. "Играет роль в" отношении Чена, например, на рис. 6.9 показывает "М" на одной стороне и "N" на другой. Это отношение многие-ко-многим: человек может играть роль во многих (М) криминальных организациях, а криминальная организация может иметь несколько человек (N), играющих роль в организации. Это можно заменить числом, указывающим определенное количество экземпляров; обычно используется константа 1, означающая, что есть одна, а не много ролей.

Нотация "вороньих лапок" использует изображение линии, дающей название методу, для представления ролей многие-ко-многим. На рис. 6.9 показаны "вороньи лапки" на обеих сторонах отношения, что означает наличие отношения многие-ко-многим. Имеются разновидности символов, позволяющие показать, что роль может не иметь значения для данного объекта (множественность 0) или должна иметь в точности одно (множественность 1).

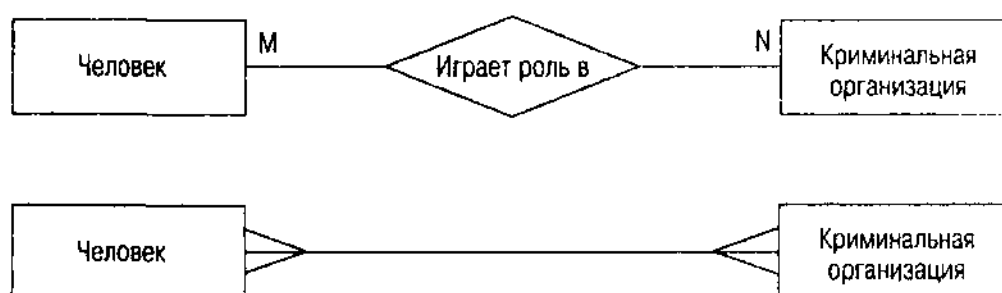


Рис. 6.9. Множественность отношения "играет роль в" в двух нотациях

IDEF1X представляет множественность с помощью специальных изображений ролей отношения, по крайней мере, в ее воплощении в инструменте ER-моделирования ERwin [Logic Work, 1996], который является продуктом компании Platinum Technology. "P" указывает ограничение

к-одному-или-нескольким, "Z" указывает ограничение к-нулю-или-одному и целое указывает ограничение по точному количеству. Однако, хотя имеется нотация для отношений многие-ко-многим, модели IDEF1X обычно создают ассоциативную сущность для отношения, затем помещают отношения один-ко-многим или один-к-одному из сущности в ассоциативную сущность. Рис 6.10 иллюстрирует оба формата для отношения "играет роль в". Во-первых, черные точки на обоих концах отношения указывают их характер многие-ко-многим. Во-вторых, это отношение становится ассоциативной сущностью с двумя отношениями: одно к человеку, а другое — к криминальной организации.

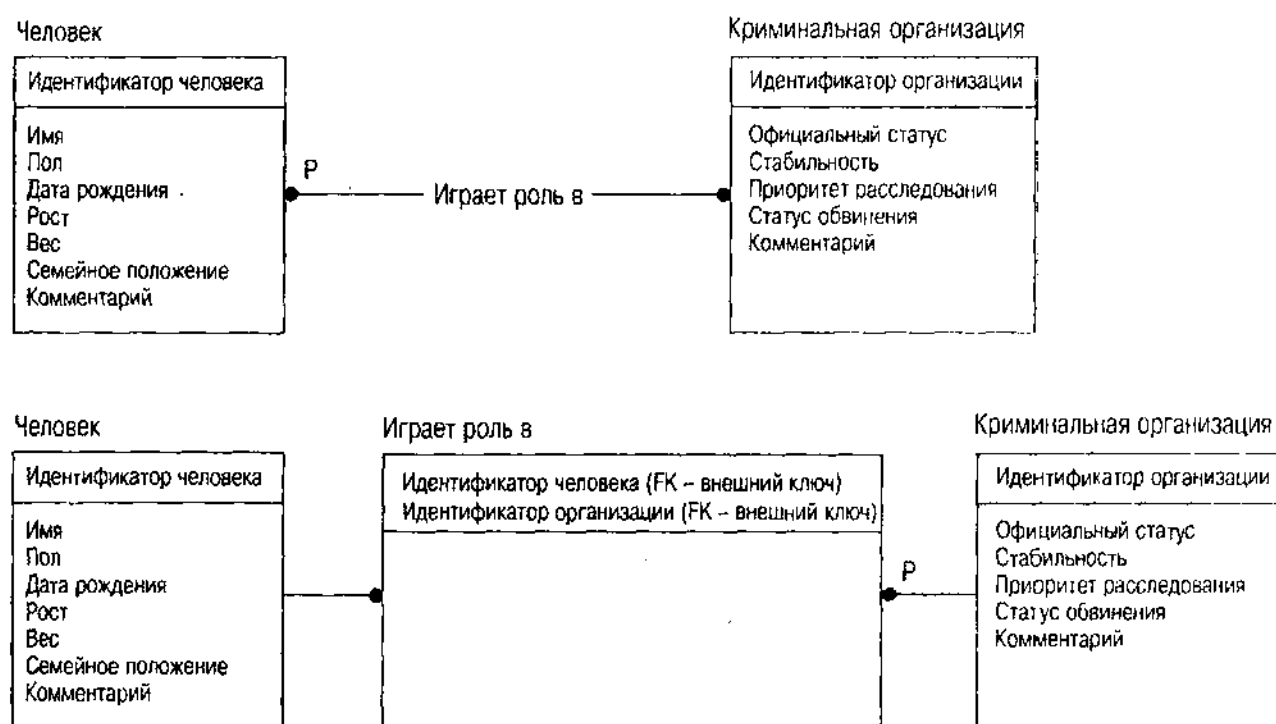


Рис. 6.10. Множественность отношения "играет роль в" в IDEF1X

Отношение между Человеком и Идентификацией является отношением один-ко-многим с названием "идентифицируется с помощью", показанным на рис. 6.11. Человек идентифицируется некоторым количеством идентификационных документов в то время, как идентификационный документ идентифицирует отдельного человека. В этом случае роль Человека имеет множественность 1, а роль Идентификации имеет множественность "много", делая ее отношением один-ко-многим.

Можно также иметь отношения один-к-одному, многие из которых на самом деле являются отношениями подтипа, а другие представляют ключи объектированного кандидата. На рис. 6.8 показано отношение подтипа в нотации IDEF1X. Свидетельство о рождении — это подтип идентификационного документа, как и паспорт, водительское удостоверение или идентификационная карточка. Все эти сущности разделены и связаны отношением. В нотации Чена, которая не предоставляет нотации генерализации или подтипа, это простое отношение один-к-одному. Один паспорт соответствует одному

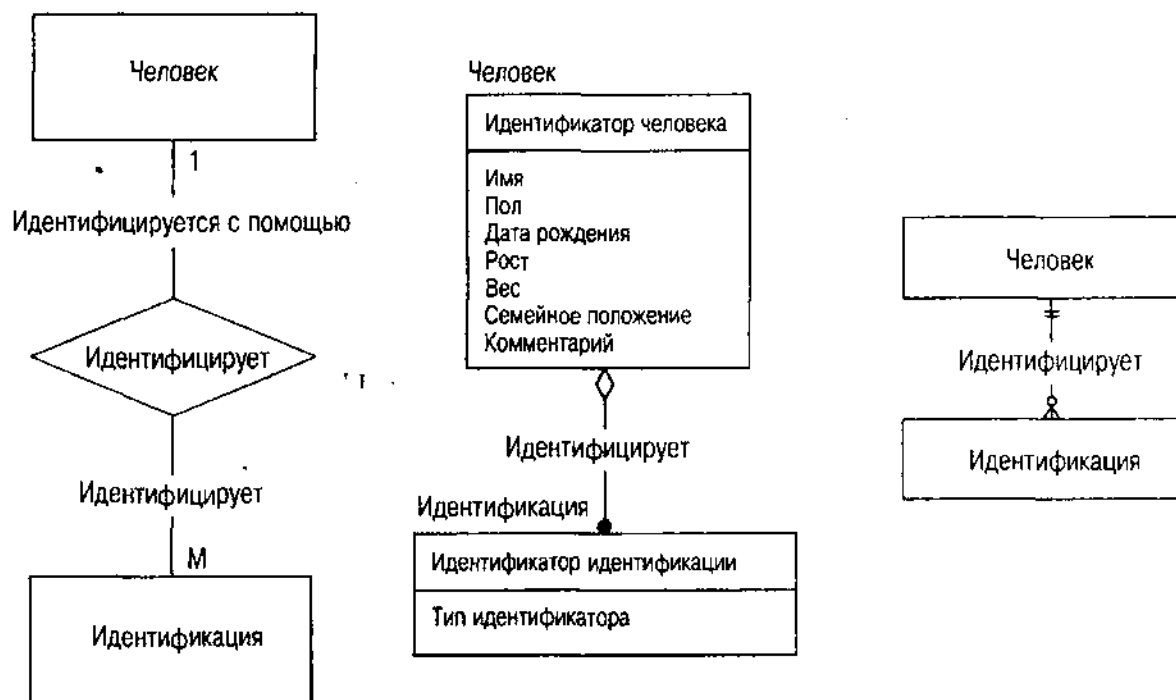


Рис. 6.11. Множественность отношения "идентифицируется с помощью" в различных нотациях

идентификационному документу. Нотации IDEF1X и "вороньих лапок" непосредственно показывают отношение генерализации, не указывая его природу один-к-одному, но методика подразумевает это из-за того способа, с помощью которого работают генерализация и порождение подтипов.

Другие отношения один-к-одному соотносят два объекта, являющиеся уникальными в их собственных коллекциях. Значит, одна сущность имеет уникальный атрибут, ссылающийся на уникальный ключ другой сущности (набор атрибутов, который уникальным образом идентифицирует объект, см. следующий раздел о "Ключах и отношениях"). Можно идентифицировать обе сущности, если известно значение атрибута. Эта ситуация относительно редкая. Пример: у каждого человека имеется уникальный набор отпечатков пальцев и уникальная ДНК. Отношение между сущностью Человек и сущностью Запись отпечатков пальцев является, таким образом, отношением один-к-одному, как и отношение ДНК к Человеку. Эти отношения обычно представляют простой связью с 1 для множественности обеих ролей.

ВНИМАНИЕ

Если используется ER-моделирование и отношение один-к-одному, проанализируйте его тщательно, чтобы увидеть, должна ли модель данных на самом деле выражать это как подтип. Если нет, рассмотрите, может ли одна сущность быть свернута с пользой в атрибут или атрибуты другой сущности. Если да, возьмите, например, Отпечатки пальцев как атрибут Человека, а не как отношение один-к-одному, если есть только одно значение отпечатков (фотография, например). С другой стороны, можно при желании разбить одну таблицу на две или более, относящихся друг к другу как один-к-одному на уровне физической модели для минимизации времени

и выборки данных, разделив данные, используемые все время, и данные, используемые довольно редко. Однако это не относится к проблемам логического проектирования.

Множественность может называться по-разному. Термин "множественность" — лучший, который можно было найти; он происходит из UML и первоначально из OMT [Rumbaugh e. a., 1992]. Различные методы разработки используют для названия этого понятия разные термины:

- Кардинальное число [Teorey, 1999; Logic Works, 1996]
- Степень кардинального числа [Fleming and von Halle, 1989]
- Связность [Teorey, Yang and Fry, 1986]
- Порождение [Date, 1986, p. 437–438]
- Частота осуществления [Halpin, 1995]
- Функциональность [Ullman, 1988]

ВНИМАНИЕ

В мире ER обычно используется термин "кардинальное число", возникший по аналогии с теорией множеств в математике, в которой он подобен размеру множества. Допустив, что это на самом деле ограничение на пределы изменения размера, а не на сам размер, автор предпочитает использовать термин "множественность", который не имеет противоречивых значений. Это также термин, используемый UML (и его предшественником OMT) для описания данной спецификации разработки.

Отношения и их множественности соединяются с различными сущностями в проекте схемы. Ограничения, которые они создают, предполагают некоторые понятия о структуре сущностей: первичные и внешние ключи.

Ключи и отношения

Ключ — это набор атрибутов, идентифицирующих объект. Первичный ключ сущности — один ключ, определяющий уникальным образом сущность, владеющую атрибутом. Альтернативный, или возможный, ключ — уникальный идентификатор, отличный от первичного ключа. Сущность может иметь любое количество ключей, хотя обладание несколькими ключами необычно. Внешний ключ — ключ, идентифицирующий объект в другой сущности; он ведет себя как указатель к другому виду сущности, имея отношение к двум сущностям. Составной ключ — такой ключ, для которого набор имеет более одного атрибута; видимо, не существует специального названия для ключа с набором из одного элемента, что встречается наиболее часто.

ВНИМАНИЕ

Как это происходит со всем, что касается информатики, кто-то может быть и дал этому название. Наиболее вероятно, что есть несколько названий этой концепции, но автору они просто не встретились. В обоих случаях — как с кардинальным числом, так и с множественностью — название не так важно по сравнению с самой концепцией.

На диаграммах Чена первичный ключ показывают либо звездочкой на имени/именах атрибута, либо выделяют подчеркиванием, как на рис 6.12. Нет способа показать альтернативные ключи. Нотация IDEF1X помещает атрибуты первичных ключей в верхнее подразделение сущности и помечает альтернативные ключи с помощью "АК<n>" в скобках после имени атрибута, где <n> – целое, идентифицирующее ключ, если у него есть несколько атрибутов. Нотация "вороньих лапок", не показывающая атрибуты, не имеет, следовательно, никакой возможности продемонстрировать первичные ключи. CASE-средства расширяют эту нотацию нотациями, подобными нотациям Чена или IDEF1X. Данные средства в зависимости от этой информации генерируют правильные реляционные таблицы, включая ограничения по ключам.

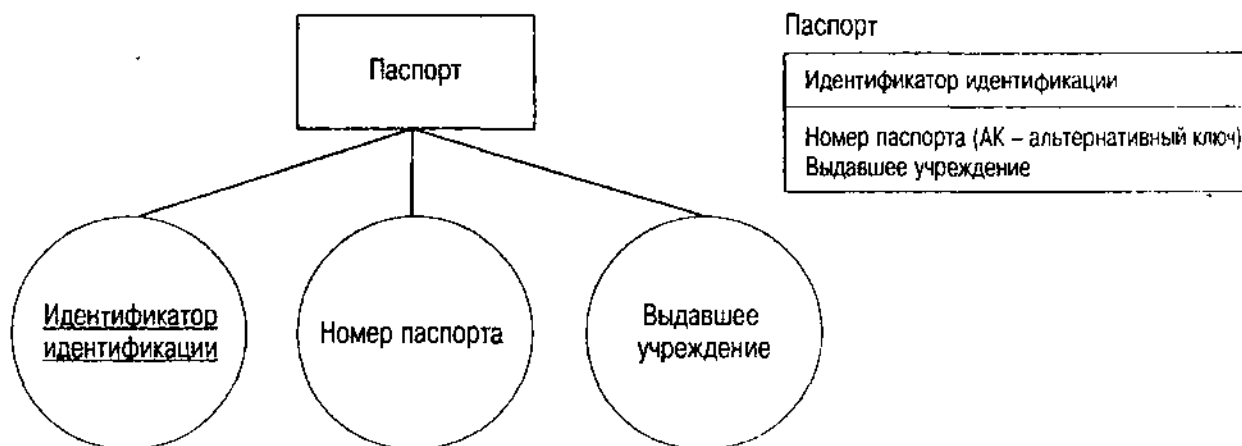


Рис. 6.12. Первичные и альтернативные ключи в ER-нотациях

Внешний ключ формально не существует на ER-диаграмме, поскольку он на самом деле является реляционным способом представления стороны многих в отношении один-ко-многим. При генерации реляционной схемы из ER-диаграммы большинство ER-CASE-средств берут эти отношения и включают атрибуты первичного ключа на стороне одного в таблицу на стороне многих. Внешний ключ является таким образом набором атрибутов, представляющих отношение для другой группы сущностей. IDEF1X представляет это явно, используя "FK<n>" как метку атрибутов внешних ключей, где <n> опять идентифицирует ключ со многими атрибутами.

Отношения многие-ко-многим создают внешние ключи для совершенно отдельного объекта, такого, который сам по себе представляет отношение. Отношение "играет роль в" на рис. 6.10, например, является отношением многие-ко-многим между человеком и криминальной организацией. Значит, один человек может играть роль в нескольких криминальных организациях, и одна криминальная организация имеет, по крайней мере, двух или более человек, играющих в ней роль. В таком случае помещение атрибутов первичных ключей в любую из двух сущностей не будет работать. Нужно поместить их первичные ключи в отношение, которое затем явно представляет связи между людьми и организациями. Первичный ключ отношения будет тогда комбинацией двух внешних ключей. При использовании нотации IDEF1X

два ключа появляются вместе в верхней части прямоугольника сущности, оба помеченные как (FK) для указания того, что это внешние ключи.

ВНИМАНИЕ

Эти разновидности ассоциативных сущностей также являются примерами слабых, или идентифицирующих, отношений (см. следующий раздел "Сильные и слабые отношения").

Можно иметь отношения, связывающие между собой более двух сущностей. Бинарное отношение — это двустороннее отношение с двумя ролями. Троичное отношение — это трехстороннее отношение с тремя ролями. Можно продолжить, но имеется очень небольшое количество отношений с четырьмя или более ролями. В качестве примера троичного отношения рассмотрим еще раз отношение "играет роль в". Рис. 6.13 показывает в нотации Чена расширенное отношение, включающее сущность Тип роли. Это может быть использовано при создании описания роли, которую играют несколько человек в разных организациях, а также при моделировании структуры ролей в организациях с графовой структурой (подчиненный босса докладывает ему, лейтенант докладывает подчиненному боссу и т. д.).

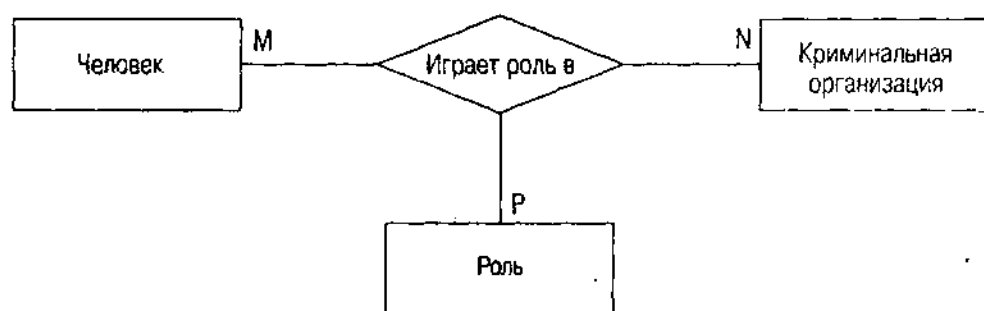


Рис. 6.13. Троичное отношение в нотации Чена

Отношение содержит три ключа — один для Человека, один — для Криминальной организации и один — для Роли. Оно представляет специфическую роль, которую играет специфический актер в специфической криминальной организации. Это особое троичное отношение выражает множественность многие-ко-многим-ко-многим, т.е. каждая из ролей отношения является ролью ко-многим. Часто имеются функциональные зависимости, ограничивающие троичные роли ролью к-одному. Например, можно потребовать, чтобы человек мог играть роль только в одной криминальной организации, а не во многих. Тогда роль криминальной организации (связь от отношения к криминальной организации) будет связью к-одному и отношение будет многие-ко-многим-к-одному. Это ограничение является функциональной зависимостью от человека и роли в криминальной организации. В главе 11 подробно обсуждаются функциональные зависимости и их влияние на разработку реляционных БД и нормальные формы. В данном контексте перевод отношения в пятую нормальную форму требует разбивания отношения на три таблицы, показывающие отношения между человеком и криминальной организацией, человеком и ролью, а также ролью и криминальной организацией. Объединение этих таблиц с помощью реляционного слияния позволяет выразить любой факт, касающийся отношения без введения

фиктивных фактов или потери какой-либо информации. ER-модель непосредственно показывает отношение и позволяет выразить реальную семантику ситуации вполне явным образом.

Внешний ключ как суррогат отношения находится в центре споров в мире моделирования данных. Более, чем что-либо другое, внешний ключ и отношение представляют различие между большинством нотаций разработки и методами, а также физическими моделями DBMS, реализующими отношения. Внешний ключ является существенной реляционной концепцией. Чен при создании отношения пытался абстрагировать нотацию от реляционной модели данных, чтобы удовлетворить потребности других моделей. Последующие ER-модели, такие как IDEF1X, умаляют этот аспект отношения, вынуждая разработчика создавать то, что важно в реляционном мире. Вместо отношений многие-ко-многим или троичных отношений, например, создается *ассоциативная сущность* [Logic Works, 1996] с отношениями один-ко-многим для образующих их сущностей. Рис. 6.14 демонстрирует расширенное троичное отношение "играет роль в" с использованием эквивалентной нотации IDEF1X.

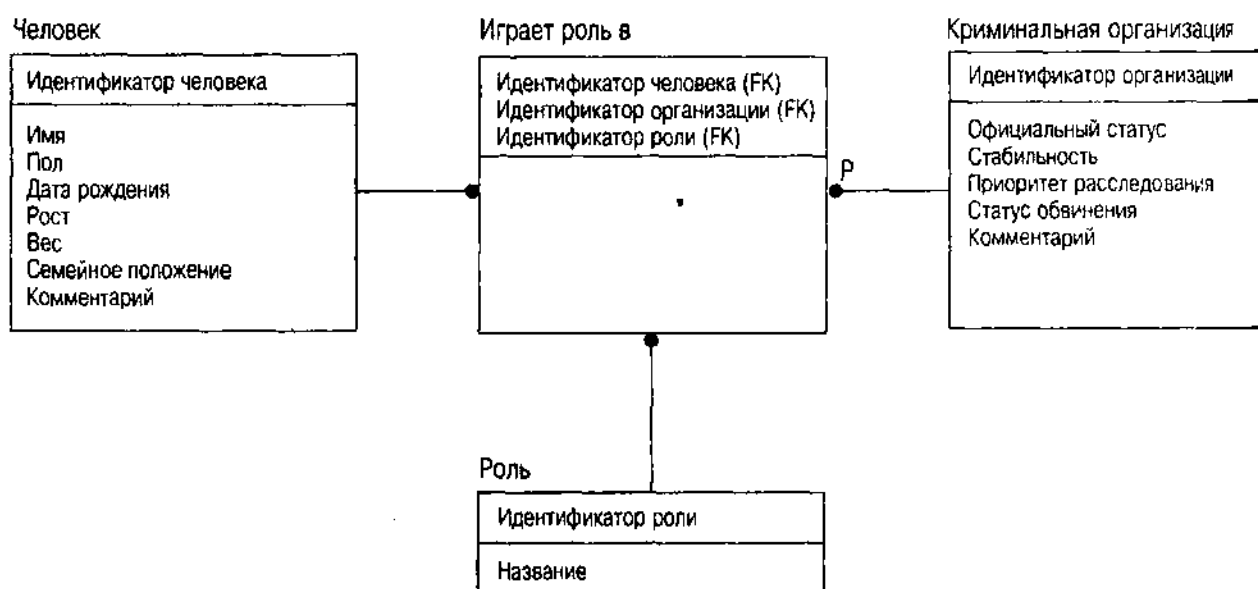


Рис. 6.14. Троичное отношение в нотации IDEF1X

Отношение "играет роль в" теперь стало сущностью с соответствующими связями отношения с тремя компонентами. Первичный ключ ассоциативной сущности состоит из трех первичных ключей трех относящихся к ним сущностей. Эти ключи теперь — внешние ключи, являющиеся частью первичного ключа и делающие ассоциативную сущность слабой или зависимой. Этот вид сущности и его связь с внешними ключами имеет специальное представление на ER-диаграмме.

Сильные и слабые отношения

Чен ввел идею *слабого отношения* для описания относительно общей ситуации, когда внешний ключ является частью первичного ключа [Chen, 1976]. Значит, *слабая сущность* — это сущность, чей первичный ключ включает первичный ключ относящейся сущности. Например, IDEF1X автоматически показывает элементы первичного ключа многих сторон как часть первичного ключа в верхней части прямоугольника сущности, если связать сущности с *идентифицирующим* отношением. Само отношение (см. рис. 6.14) — это сплошная линия в отличие от пунктирной линии для *неидентифицирующего* (сильного) отношения. Нотация Чена показывает отношение просто как ромб с двойной обводкой, а соответствующую слабую сущность — как прямоугольник с двойной обводкой.

В диаграмме Чена слабые сущности редки потому, что существует мало ограничений между реальными сущностями. Исходный пример Чена показывал сущность *Иждивенца*, относящуюся к сущности *Работника* с помощью слабого отношения. Первичный ключ *Иждивенца* объединялся с Идентификатором *Работника* и именем *Иждивенца*. Делая это, слабое отношение утверждало, что иждивенец не был интересен иначе, как только иждивенец работника. Но если это сопоставить с ситуациями реального мира, с которыми сталкивается такой человеческий ресурс, как несчастный клерк (много адресов для разведенных иждивенцев бывшего работника с продолжающимся покрытием COBRA и т. д.), понимаешь, что жизнь не часто предоставляет возможности использовать слабые отношения с реальными сущностями.

Почти все ассоциированные сущности, однако, являются слабыми сущностями потому, что их существование во многом зависит от существования сущностей, которые они ассоциируют. Они являются отношениями, а не отдельно стоящими сущностями. Поэтому слабое или идентифицирующее отношение гораздо более важно в методах построения ER-диаграмм, которые используют ассоциативные сущности, например IDEF1X.

Слабые сущности зависят от существования относящихся к ним сущностей. Если удалить относящуюся сущность, необходимо продолжить это удаление на слабые сущности, содержащие первичный ключ, который был удален.

ВНИМАНИЕ

Включения слабой сущности гораздо более интересны в реальном мире объектов. Такая ситуация при разработке объектов встречается, когда объект создает и владеет другим объектом, а не просто имеет ссылку на объект. Если уничтожить основной объект, то нужно уничтожить и все зависимые объекты, которыми он обладает. Если объекты независимы, один объект просто ссылается на другой объект. Когда уничтожается объект, на который происходит ссылка, отношение пропадает, но другой объект все еще существует.

Домены

Домен — это набор значений, в пределах которых атрибут отображает объект, т.е. каждое значение для атрибута в наборе сущностей должно происходить из домена атрибута. Это очень простая форма ограничения целостности. Она налагает ограничения на значения явным образом.

ВНИМАНИЕ

Значение домена не имеет никакого отношения к домену Microsoft Windows NT, сетевой концепции присвоения имен.

ER-диаграммы представляют непосредственно домены, но все CASE-средства, использованные автором, позволяют поместить данные домена в репозиторий для каждого атрибута. Поскольку критичной функцией CASE-средства является генерация языка определения данных или самой схемы, нужно иметь способ определения требуемых ограничений домена. Часто можно определить отдельный набор доменных требований для каждой цели DBMS, поскольку они все имеют разные функции, относящиеся к доменам.

Итоги

Сущности, отношения и ER-диаграммы, которые они порождают, являются основой разработки баз данных. Сущности — это прямоугольники, связанные с другими сущностями при помощи линий отношений со многими вариациями в различных средствах моделирования, начиная с нотации Чена к NIAM и до моделирования данных к IDEF1X.

Большинство нотаций ER-диаграмм поддерживают концепции:

- *Сущность*: набор объектов
- *Атрибут*: свойство сущности
- *Отношения (как сильные, так и слабые)*: связь между двумя или более сущностями, соответствующими внешним ключам в реляционных БД
- *Роль*: одна сторона отношения
- *Множественность*: количество объектов, участвующих в роли (один-к-одному, один-ко-многим, многие-ко-многим)
- *Первичные и потенциальные ключи*: атрибуты, которые уникальным образом идентифицируют объекты
- *Домены*: наборы значений, в пределах которых атрибут отображает объект

Теперь мы познакомились со всеми разнообразными элементами стандартной модели данных с использованием ER-диаграмм. В следующей главе все перевернуто с ног на голову и будут представлены концепции объектов моделирования.

Глава 7

Создание моделей классов в UML

Есть многое на свете, друг Горацио, что и не снилось нашим мудрецам.

Шекспир. "Гамлет"

Методика ER-моделирования из главы 6 пригодна для начала изучения методов, необходимых для эффективного моделирования данных. Данная глава расширяет эти основные методики, добавляя к ним мир объектной направленности. Вместо моделирования сущностей теперь мы будем учиться моделировать объекты. Гамлет был прав: есть многое на свете, что постигнуть сложно, но, по крайней мере, стоит попытаться с большей эффективностью смоделировать то, что можно постигнуть.

Универсальный язык моделирования (UML) — это нотация, объединяющая элементы из трех основных направлений ООП: ОМТ-моделирование Рамбо [Rumbaugh e. a., 1992], ОО-анализ и разработка Буча [Booch, 1994] и объектизация Якобсона [Jacobson's e. a., 1992]. Похоже, что UML превращается в стандарт. OMG перенял UML как стандартную нотацию для объектных методов. UML повсеместно присутствует на презентациях продавцов программного обеспечения. UML изложен более или менее подробно в нескольких профессиональных книгах, представляющих его методы анализа и разработки [Douglass, 1998; Eriksson and Penker, 1997; Fowler, Jacobson and Kendall, 1997; Harmon and Watson, 1998; Larman, 1997; Muller, Nygard and Grnkovic, 1997; Quatrani, 1997; Texel and Williams, 1997].

ВНИМАНИЕ

Таким образом, имеется множество способов изучить UML. Вместо обучения UML данная глава сосредоточивает внимание на обучении элементам UML, необходимым для моделирования данных. Это ни в коей мере не исчерпывает возможностей языка. OO-разработчик должен быть знаком с разнообразными сторонами UML. Ниболее прямой способ сделать это — загрузить спецификационные документы UML на свой компьютер с Web-сайтов Rational или OMG [Rational Software, 1997a, 1997b, 1997c]. Чтение этих документов и использование одного из многих учебников, упомянутых в предыдущем параграфе, введет вас во все OO-функции UML. Все последующее изложение основано на материале из трех документов по UML; следует обращаться к ним для получения сведений по любым индивидуальным элементам диаграмм классов. Дополнительное понимание может прийти при чтении предыдущего параграфа с его примерами. Книга Фаулера — хорошее введение [Fowler, Jacobson and Kendall, 1997], Ларман и Хармон удобны для среднего уровня [Larman, 1997; Harmon and Watson, 1998], а книга Эриксона — для продвинутых программистов [Eriksson and Penker, 1997].

В главе 4 рассмотрен один тип диаграмм UML — вариант использования. Эта глава охватывает диаграмму классов, моделирующую классы объектов. Объектно-ориентированный анализ и разработка с использованием диаграмм классов достаточно сложны; цель данной главы — рассмотреть только те элементы диаграмм классов, которые используются для моделирования данных. В первом разделе рассматривается структура объектов: пакеты, классы и атрибуты, во втором — структура поведения объекта: операции и методы. Третий раздел охватывает отношения и их представление в соответствующих бизнес-правилах. В последнем разделе бизнес-правила рассмотрены более широко в отношении тождества и уникальности объектов, ограничений домена и более общих ограничений.

Пакеты, классы и атрибуты

Класс UML и его структура соответствуют типу сущности и его структуре на ER-диаграмме. В UML:

Класс — это описание набора объектов, разделяющий одни и те же атрибуты, операции, методы, отношения и семантику. Класс может использовать набор интерфейсов для определения коллекций операций, которые он предоставляет в своей среде [Rational Software, 1997b, p. 20].

В этом определении встречается всего несколько слов со специфическим значением, касающихся разработчика БД:

- *Атрибут*: "Поименованный участок внутри классификатора [интерфейса, типа, класса, подсистемы, БД или компонента], описывающий интервал значений, которые могут содержать экземпляры классификатора" [Rational Software, 1997b, p. 149]. Это очень похоже на реляционное определение атрибута как поименованного отображения в домен [Codd, 1972].

- *Операция*: "Служба, которую можно запросить у объекта для того, чтобы повлиять на поведение. Операция имеет сигнатуру [имя и параметры, возможно, включающие возвращаемый параметр], которая может ограничивать возможные действительные параметры" [Rational Software, 1997b, p. 155].
- *Метод*: "Реализация операции. Он определяет алгоритм или процедуру, которая создает результаты операции" [Rational Software, 1997b, p. 154].
- *Отношение*: "Семантическая связь между элементами модели. Примеры отношений включают ассоциации и генерализации" [Rational Software, 1997b, p. 156].
- *Ассоциация*: "Семантическое отношение между двумя или более классификаторами, которое включает связи между их сущностями" [Rational Software, 1997b, p. 149].
- *Генерализация*: "Таксономическое отношение между общим элементом и специфическим элементом. Более специфический элемент полностью однороден с более общим элементом и содержит дополнительную информацию. Сущность более специфического элемента может быть использована там, где разрешен более общий элемент" [Rational Software, 1997b, p. 152].
- *Интерфейс*: "Описание набора операций, которые могут быть использованы для определения службы, предлагаемой сущностью" [Rational Software, 1997b, p. 153].

Классификатор — фундаментальная концепция в UML, способная запустить даже многих ОО-разработчиков. Это абстрактный класс UML, включающий различные подклассы нотаций UML, которые можно использовать для классификации объектов по структуре и поведению. Классы, типы, интерфейсы, подсистемы, компоненты и БД — это все виды классификаторов. Часто нотации диаграмм UML применяются к классификаторам в целом, а не просто к классам.

Термины объединяются, образуя семантику класса на диаграмме класса UML, т. е. чтобы придать смысл, нужно создать диаграмму класса с нотацией, соответствующей этим концепциям, так, как их определяет UML. Этот раздел рассматривает классы и атрибуты, а следующий — операции, отношения и другие бизнес-правила.

Диаграмма класса — это структурная или статическая диаграмма, с помощью которой осуществляется моделирование структуры системы классов. Во многом диаграммы классов сильно напоминают ER-диаграммы. Если сравнить определение класса, данное выше, с определением типа сущности для ER-диаграммы, можно увидеть, что они в основном одинаковы. Различия состоят в моделировании операций и отношений. Все это вряд ли может удивить, поскольку все ОО-нотации, на которых был основан UML, разработаны на базе ER-нотаций.

ВНИМАНИЕ

Диаграмма класса UML на самом деле шире, чем просто класс, поскольку она может моделировать интерфейсы, отношения и даже индивидуальные сущности класса или объекты. Альтернативным и иногда предпочтительным названием для диаграммы класса является "статическая структурная диаграмма", оно используется во многих книгах по UML вместо более общего "диаграмма класса". Поскольку внимание здесь обращается на классы и их отношения, то в этой книге будет использоваться термин "диаграмма класса".

Но перед тем как рассмотреть подробности построения диаграммы классов, нужно понять, как структурировать систему в целом, а для этого следует познакомиться с пакетами.

Пакеты

Большинство поддерживающих UML инструментов позволяют организовать классы независимо от структуры диаграммы UML, как предлагает спецификация UML, т. е. можно моделировать все классы в одной диаграмме, а можно разбить их на части в нескольких диаграммах. Некоторые инструменты допускают обе возможности, позволяя определить классы в репозитории или на отдельной диаграмме, из которой создаются другие диаграммы, ссылающиеся на репозиторий в отношении класса и структуры отношения. Таким образом, можно показать различные точки зрения на систему без дублирования действительных усилий по моделированию.

Во многом организация диаграмм зависит от инструментов, используемых для их изображения. С помощью простых изобразительных средств можно организовать классы как угодно, но для этого потребуются много работы. Используя развитые инструменты, например Rational Rose или одного из конкурентов, можно позволить такому средству сделать большую часть работы. Однако, кто платит, тот и заказывает музыку: часто приходится структурировать диаграммы таким образом, как этого требует выбранное средство.

Частью искусства управления ОО-проектами является структурирование проекта и его организация в соответствии с основными чертами архитектуры продукта [Hohmann, 1997; Muller, 1998]. Одним из аспектов проекта является модель данных и моделирование данных как предмет архитектурной структуризации, как любая другая часть проекта. Это может показаться трудным и не совсем ясным, но на самом деле все будет проще, если понять, как работают системы объектов в архитектурных терминах.

ОО *система* программного обеспечения — это работающий объект ПО, существующий отдельно, как, например, продукт программного обеспечения или библиотека интегрированной системы. Эти системы в свою очередь объединяют различные более мелкие системы, которые не существуют отдельно, а поставляются вместе, образуя систему в целом. Одни называют эти подсистемы кластерами [Muller, 1998]; другие — чем угодно: пакетами, подсистемами, слоями, разделами, модулями.

Пакет UML — это группировка элементов модели. *Подсистема UML* — это разновидность пакета, представляющая спецификацию и реализацию

набора поведений [Rational Software, 1997b, p. 130–137], что близко к концепции кластера автора. *Подсистема* — это также вид классификатора, и можно ссылаться на нее в любом месте, где допустимо использовать класс или интерфейс. *Спецификация* состоит из набора вариантов использования и их отношений, операций и интерфейсов. *Реализация* — это набор классов и других подсистем, которые обеспечивают определенное в спецификации поведение. Спецификация и реализация связываются через набор *взаимодействий*, отображений между вариантами использования или интерфейсами на диаграмме взаимодействия, которая не рассматривается в данной книге. Эти подсистемы являются строительными блоками или компонентами, образующими систему ПО. Именно они, а не классы или файлы, должны быть основой для конфигурационного менеджмента системы, если исходные инструменты управления достаточно интеллектуальны для понимания подсистем [Muller, 1998]. Обычно подсистемы группируют несколько классов вместе, хотя могут быть подсистемы с одним классом или даже подсистемы, которые образуют только внешний вид для других подсистем, совсем без классов, имея только интерфейсы. В терминах системной архитектуры подсистемы часто соответствуют библиотекам со статическими (LIB) или динамическими связями (DLL), представляющими внешние интерфейсы подсистемы для повторного использования другими библиотеками.

Пакеты в действительности представляют собой пространства имен, поименованные объединения элементов, имеющие уникальные имена внутри группы. Подсистемы гораздо более важны и образуют основу для разработки системы. Можно использовать пакеты как таковые для создания легковесных групп внутри подсистем без необходимости разработки отдельных вариантов использования и взаимодействия. Как решить, что нужно подсистеме и что хорошо оставить просто как пакет, зависит только от разработчика.

Модель UML — это пакет, содержащий полное представление системы с данной точки зрения на моделирование. Можно иметь различные модели системы, используя различные точки зрения или уровни. Например, можно иметь модель анализа, состоящую исключительно из вариантов использования, и модель разработки, состоящую из подсистем или классов.

Кроме моделей и подсистем имеется несколько видов пакетов, идентифицируемых только с помощью добавления *стереотипов* к имени пакета. Стереотип — это фраза в кавычках " ", которая присоединяется к идентификатору для представления "официального" расширения семантики UML.

- "*Система*": пакет, содержащий все модели системы.
- "*Внешний вид*": пакет, состоящий исключительно из ссылок на другие пакеты, представляющие характеристики этих пакетов при просмотре.
- "*Скелет*": пакет, представляющий собой расширяемый шаблон для использования в определенном домене, состоящий в основном из образцов.
- "*Пакет верхнего уровня*": пакет, содержащий все другие пакеты в модели, представляя все не входящие в рабочую среду части модели.

- "Пустышка": пакет содержащий интерфейс общего доступа и ничего больше, представляющий работу по проектированию, которая еще не завершена или отложена по какой-то причине.

ВНИМАНИЕ

Способ структурирования пакетов и подсистем является частью основного процесса ОО-разработки системной архитектуры [Rumbaugh et al., 1992, p. 198–226; Booch, 1994, p. 221–222; Booch, 1996, p. 108–115]. Системная архитектура включает в себя гораздо больше аспектов, чем те, которые относятся к разработке БД, поэтому, следует обращаться к указанным справочным материалам за подробностями. Спецификация семантики UML также является очень полезным справочником по концепциям системной архитектуры [Rational Software, 1997b].

Концепция системной архитектуры как связанных и вложенных пакетов помогает структурировать систему для оптимизации возможности повторного использования. Целью помещения элементов модели в пакеты является разъединение больших частей системы. Это в свою очередь позволяет инкапсулировать крупные части системы в пакеты, которые становятся компонентами повторного использования. Другими словами, пакеты — основа для создания компонентов повторного использования, а не класса. Пакеты обычно имеют интерфейс пакета или набор интерфейсов, моделирующий службы, которые предоставляет пакет. Другие пакеты получают доступ к элементам пакета через эти интерфейсы, но они ничего не знают о внутренней структуре пакета.

Необходимо представлять системную архитектуру как набор пакетов и их интерфейсов, а не как коллекцию связанных между собой сущностей БД. Этот упор на пакетирование является существенным различием между ER-подходом к разработке баз данных и ОО-методами, представленными в этой книге. Последствия такого способа мышления прослеживаются везде вплоть до базы, вернее до баз данных. Необходимо рассматривать различные схемы подсистем и, следовательно, различные БД, даже если на самом деле они создаются вместе в одной физической схеме. Сами БД становятся пакетами на некотором уровне, представляя общие данные, когда происходит детализация действительной структуры реализации системы. Разбиение системы на несколько БД дает возможность модуляризации системы для повторного использования. Можно создать пакет Человек, позволяющий моделировать людей и относящиеся к ним объекты (адреса, например). Можно встраивать этот пакет в несколько систем, повторно используя схему БД, вложенную в него вместе со всем прочим. Можно даже повторно задействовать реализацию БД, которая в конце концов является целью трехзвенной архитектуры ANSI/SPARC.

В основе большинства пакетов лежат два основных организующих принципа. Первый исключительно структурный: как классы в пакете относятся к классам вне его? Минимизация такого взаимодействия дает четкое представление о том, включать ли класс в тот или иной пакет. С точки зрения БД нужно рассмотреть другой аспект: транзакции. Минимизация количества пакетов, являющихся частью транзакции, — обычно замечательный способ. Идеально иметь транзакции в пределах одного пакета.

Можно подойти к созданию пакетов классов через варианты использования. Чтобы начать разработку классов, как это описано в следующем разделе, нужно просмотреть варианты использования, находя объекты и их атрибуты. По мере этого также следует обращать внимание на структуру транзакций. Часто можно упростить структуру пакетов, поняв, как сценарии вариантов использования влияют на объекты в БД. Начните создавать подсистемы путем группировки вариантов использования, которые влияют на подобные объекты сообща. Можно также обнаружить интересные способы модификации вариантов использования для упрощения транзакций. По мере создания пакетов подумайте о том, как варианты использования отображаются в транзакции, включающие классы в пакете. Если есть способ вновь определить требования для минимизации числа пакетов, то в итоге получится лучшая система. Поскольку варианты использования соответствуют транзакциям, этот метод гарантирует, что транзакции окажутся внутри пакетов, а не будут распространяться на всю систему.

Все это является частью итеративной природы ООП. Разработчик движется с уровня микроразработки обратно к уровню архитектуры и даже обратно к требованиям, затем снова движется вперед, имея улучшенную систему. Подсистемы растут от множества вариантов использования к пакету полной подсистемы с классами, интерфейсами и взаимодействиями, а также с вариантами использования. Варианты использования могут стать более подробными по мере расширения понимания того, что на самом деле происходит внутри них. Когда все это находится в поле внимания разработчика, такая итерация составляет сущность хорошей разработки. Достаточно абстракций. Как пакеты работают на практике? Приступая к созданию архитектуры, обычно проходят через создание множества вариантов использования. Разработчик получает общее представление о необходимых классах и довольно ясное представление о типах транзакций в системе. Нужно на этом этапе произвести мозговую атаку, проинспектировав основные пакеты, которые образуют базис для проектирования модели системной архитектуры. Разработчик представляет несколько иерархий классов и почти определенно идентифицирует некоторые главные предметные области системы.

Например, система каталога имеет несколько основных областей интереса. Повторим варианты использования из главы 4:

- *Аутентификация*: подключиться к системе как аутентифицированный пользователь.
- *Создать отчет по истории дела*: отчет строится как текст с мультимедийными отрывками.
- *Создать отчет о событиях*: отчет о ряде событий на основе критерия поиска в тексте и фактах.
- *Создать отчет о преступнике*: создать отчет о биографии преступника в виде текста с мультимедийными отрывками, включающими данные полиции и истории дел, относящиеся к преступнику.
- *Идентификация клички*: идентифицировать преступника по кличке.

- *Идентификация доказательств*: идентифицировать преступника на основе анализа доказательств, таких, как отпечатки пальцев, ДНК и т. д.
- *Создать отчет по теме*: создать энциклопедически полный отчет по теме.
- *Найти раздел криминальной хроники*: найти раздел криминальной хроники (объявление личного свойства в газете или в Интернете) по критерию текстового поиска.
- *Найти принадлежность*: найти данные о преступной организации на основе критерия поиска по фактам.
- *Исследовать*: исследовать отношения между людьми, организациями и фактами в базе данных.

Модель Системная архитектура содержит подсистему высокого уровня, называемую Каталог, включающую все другие подсистемы. Здесь можно обнаружить несколько интересных моментов. Аутентификация подразумевает, что нечто должно отслеживать пользователей системы и их атрибуты безопасности. Можно назвать эту подсистему пакетом безопасности или пакетом пользователя. Другие подсистемы сверяются с безопасностью прежде, чем разрешить доступ. Истории дел и событий имеют свои собственные подсистемы. Существуют варианты использования, относящиеся к преступникам, которые являются людьми, и варианты использования, ссылающиеся на другие виды людей, поэтому имеется подсистема Человек. Можно создать подсистему Преступник, расположенную поверх Человека, или генерализовать Человека, который имеет такие характеристики, как клички и роли в криминальной организации. Определенно необходима подсистема Криминальная Организация, чтобы объединять потребности вариантов использования Создать отчет об истории дела, Создать отчет о преступнике и Найти преступников. Два первых варианта использования являются частью подсистем Событие и Преступник, в то время, как Найти преступников становится частью подсистемы Криминальная организация. Необходимы также пакеты Энциклопедия и Отчет средств массовой информации для хранения данных об этих элементах вариантов использования. И, наконец, Поиск требует отдельной подсистемы, называемой Специальный запрос, которая предоставляет набор операций, позволяющих пользователям осуществлять поиск отношений в других подсистемах.

Подумав немного о потребностях отчетов по истории дел и отчетов о данных по преступникам, можно создать некоторые дополнительные пакеты для мультимедийных объектов, таких как отпечатки пальцев, ДНК и другие элементы данных каталога. Все это, возможно, слишком подробно для первого прохода. Выгоднее было бы разработку некоторых классов выполнять до создания для них пакетов, или, по крайней мере, обязательно осуществлять несколько итераций перед остановкой.

Можно также получить первый удар при определении отношений пакетов, которые отражены на диаграмме пакетов, представленной на рис. 7.1.

Диаграмма пакетов иллюстрирует всю структуру системы. Каждый пакет — это именованное изображение файловой папки с именем пакета и

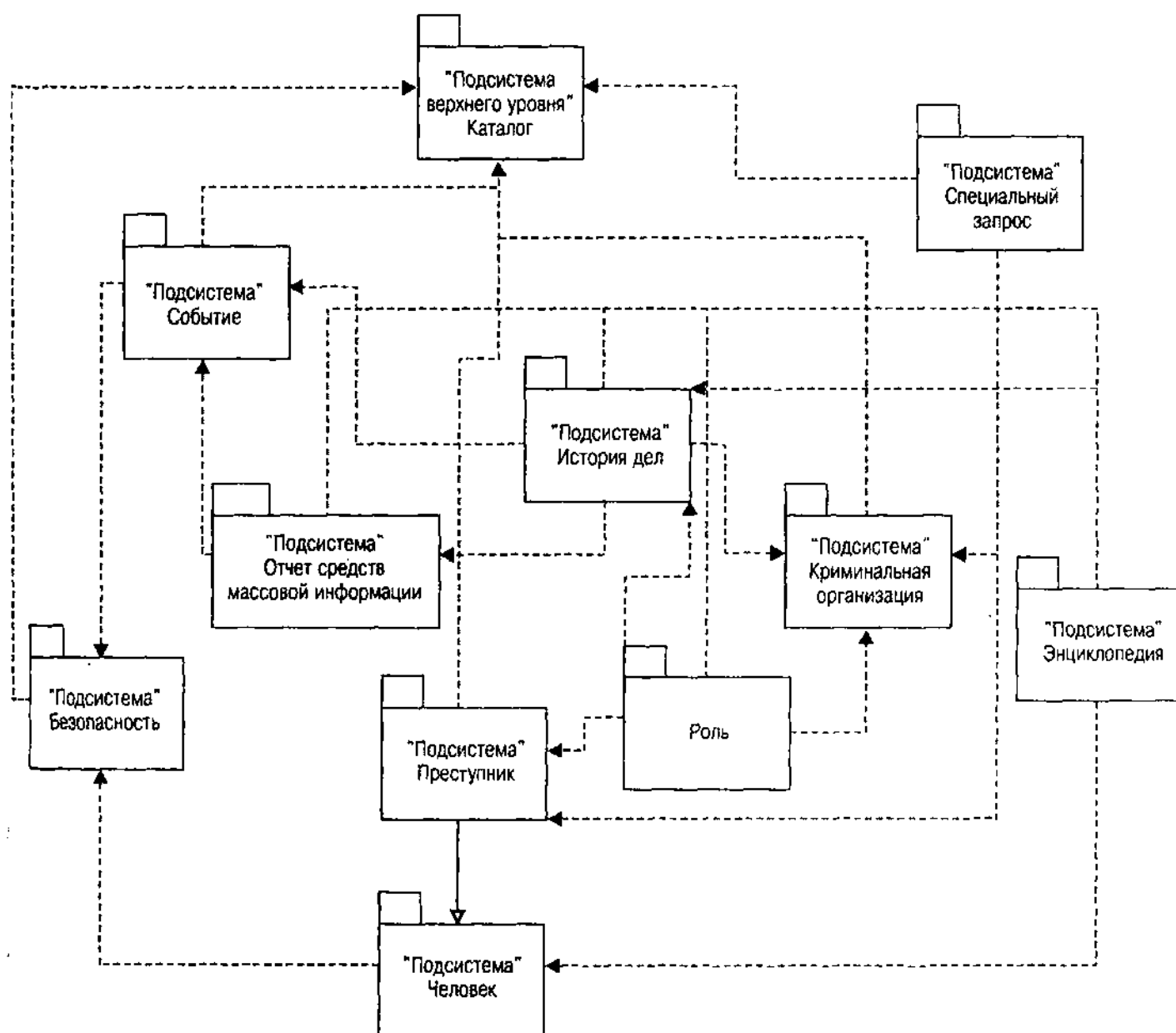


Рис. 7.1. Начальная диаграмма пакетов Каталога

соответствующим стереотипом. С помощью пунктирных направленных стрелок показаны зависимости между пакетами. На рис. 7.1, например, как подсистема Специального запроса, так и пакет Роль зависят от подсистемы Криминальная организация, а подсистема Истории дел зависит от Роли. Подсистема Каталог зависит от всех других подсистем, кроме Человека; она использует и пакет Роль. Безопасность также зависит от всех других подсистем, но на рисунке стрелки опущены для ясности. Всегда проблематично создание диаграммы, которая не напоминала бы тарелку со спагетти; использование прямых линий хорошо помогает в этом, а еще лучше спрятать второстепенные линии. Стрелка, идущая от Преступника к Человеку, — это генерализация, означающая, что подсистема Преступник наследует подсистему Человек (преступник — разновидность человека).

На самом деле можно все это пометить как "пустышка" до того времени, когда вы будете готовы перейти к разработке, но в этом примере целью было завершение создания подсистем на текущем шаге программирования. Если дать задания нескольким людям по проектированию различных частей на разных этапах разработки, можно исключить соответствующие подсистемы, пока не будет завершена первая стадия.

По мере углубления в детали системы пакеты приобретают такие внутренние подробности, как варианты использования, классы, интерфейсы и взаимодействия работы. По окончании работы должен получиться довольно солидный набор пакетов, в основном независимых друг от друга и взаимодействующих через хорошо определенные повторно используемые интерфейсы. Каждый пакет должен быть разбит на набор диаграмм классов, которые можно использовать для документирования и руководства разработкой реального кода системы, включая создание БД и кода обработки.

Опять же, хотя архитектурная разработка отлична от разработки БД, но в конце можно будет обнаружить, что подсистемы и пакеты в действительности имеют непосредственное влияние на архитектуру базы данных. Разработка БД производится в пределах структуры, которая создается при разработке классов с их атрибутами.

Во многом эта итеративная архитектура подобна тому, как Холмс расследует дело против Мориарти:

"Но профессор окружил себя охранниками, такими изобретательными плутами, что, если бы я сделал то, что должен, было бы невозможно получить доказательства, убедительные для правосудия. Вы знаете, на что я способен, мой дорогой Ватсон, но все же по истечении трех месяцев я вынужден был признать, что я встретил антагониста, равного мне по интеллекту. Мой страх по поводу его преступлений прошел вследствие восхищения его искусством. Но вот он допустил ошибку — маленькую-маленькую — и этого оказалось достаточно, когда я был столь близко от него. У меня был шанс, и начиная с того момента я опутал его своей сетью до сегодняшнего дня, когда все это близко к завершению. Через три дня — т. е. в следующий понедельник — профессор со всеми главными участниками его банды будет в руках полиции.

...Теперь, если бы я мог сделать это, не обладая знаниями о профессоре Мориарти, все было бы хорошо. Но он был слишком коварен. Он видел каждый шаг, который я осуществлял, чтобы ослабить петлю. Снова и снова он пытался вырваться, но я, как всегда, опережал его. Поверьте, мой друг, если бы была написана подробная хроника этого молчаливого поединка, она заняла бы свое место среди самых ярких примеров в истории расследований. Никогда я не поднимался на такую высоту и никогда не находился под таким давлением со стороны оппонента. Он бил наповал, а я просто отбивался. Сегодня утром были предприняты последние шаги, требовалось всего три дня для завершения этого дела". [FINA].

Как всегда, то, что занимает 10% времени проекта, труднее всего выполнить.

Классы и атрибуты

Теперь, возвращаясь к классу, нужно определить структуру, которая преобразуется в проект схемы БД. Конечно, за этим стоит гораздо больше. Разработка БД на самом деле является частью более крупного процесса, включающего разработку всех объектов системы, как хранящихся в оперативной памяти, так и устойчивых. Нужно предпринять некоторые специальные шаги для устойчивых данных. В этом разделе рассматривается часть вопросов, относящихся к созданию БД, остальные — в следующих разделах ("Отношения" и "Ограничения объектов и бизнес-правила").

Диаграмма классов может содержать целый ряд различных вещей — от классов до интерфейсов и пакетов и всех отношений между ними. Диаграмма пакетов из предыдущего раздела представляет собой диаграмму классов очень высокого уровня, которая содержит только пакеты. Каждая подсистема обычно имеет свою собственную диаграмму классов, описывающую классы и интерфейсы, реализованные в подсистеме. В этом разделе подробно рассматриваются классы и их атрибуты, а операции и отношения оставлены для последующих разделов.

Символ класса — прямоугольник, содержащий до трех разделов, как показано на рис. 7.2. Верхний раздел содержит имя класса и свойства, если они есть; средний раздел — список атрибутов и их свойства; нижний — список операций и их свойства. Можно спрятать или вывести на экран столько деталей, сколько нужно. Обычно прячут большинство деталей при импорте класса в другую диаграмму, показывая ее только как ссылку. Можно также сделать это для ясности при выводе на экран диаграммы классов как верхнего уровня модели классов, поскольку списки используются в больших подсистемах достаточно широко.

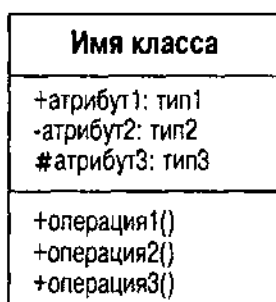


Рис. 7.2. Символ простого класса

Имя класса является уникальным в пакете, представляющем собой пространство имен, область действия имен элементов пакета. Можно иметь классы с одним и тем же именем в разных пакетах. Чтобы обратиться к объекту в другом пакете, к имени объекта добавляют префикс в виде имени пакета в формате "пакет::класс". Символ :: — это операция разрешения контекста, присоединяющая имя пространства имен к классу. Можно вкладывать пространства имени, чтобы таким образом иметь несколько префиксов для имени класса: пакет1::пакет1.1::пакет1.1.3::класс, например, означает, что пакет1.1.3 определяет класс в пакете пакета1.1, который, в свою очередь, вкладывается в пакет пакета1.

Верхний раздел символа класса на диаграмме содержит больше, чем просто имя класса. Классы, которые планируется сделать устойчивыми, имеют стереотип, указывающий на это: "устойчивый". Стереотип устойчивости говорит о том, что система не разрушает состояние сущности при разрушении сущности. "Устойчивый" класс преобразуется позже в нижележащую реализацию (например, реляционную таблицу, объектно-реляционный тип или объектно-ориентированный устойчивый класс). В главах с 11 по 13 более подробно рассматривается использование компонентной диаграммы для представления этих элементов в модели реализации системы.

Абстрактный класс — это класс, не имеющий экземпляров. Он существует прежде всего для представления общей абстракции, которую используют несколько подклассов. UML предлагает, чтобы символ класса показывал абстракцию с помощью выделения имени класса курсивом. Можно также выдать на экран свойство под именем класса в фигурных скобках: [абстрактный]. Превращение устойчивого класса в абстрактный не имеет отношения к данным в реляционной или объектно-реляционной БД, в отличие от его влияния на временные объекты и объектно-ориентированные БД. В связи со способом преобразования ОО-модели данных в схему реляционные БД действительно имеют данные для абстрактного класса. Эти данные представляют собой часть конкретных объектов, возникающих из абстрактного класса, а не экземпляр самого абстрактного класса.

На рис. 7.3 показан класс Человек как простой пример абстрактного устойчивого класса с различными атрибутами и операциями.

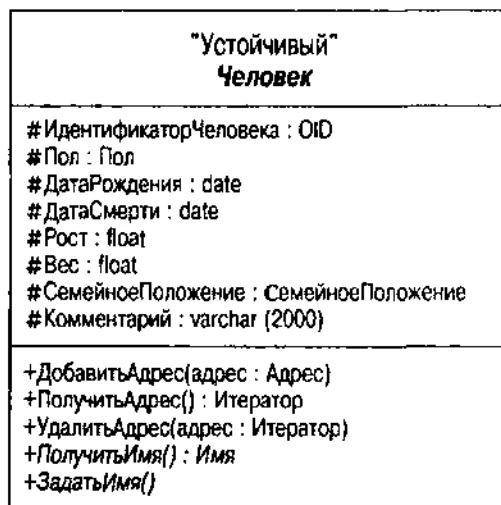


Рис. 7.3. Устойчивый класс Человек

Второй раздел на рис. 7.3 показывает также список атрибутов, а третий — операции на классе. Атрибуты, как и имя класса в верхнем разделе, имеют различные квалификаторы, которые можно при желании подавить: стереотип, видимость, тип данных, ограничения и свойства. Вот синтаксис выражения атрибута:

```
stereotype visibility name : type-expression =
    initial-value property-string
```

В случае на рис. 7.3 показаны видимость, имя и тип-выражение. Имеются три символа для видимости, получающие свои значения из концепций квалификатора доступа C++:

- + Общедоступная видимость (*public*), означающая, что любой другой класс может непосредственно проверить или изменить значение атрибута
- # Защищенная видимость (*protected*), когда только методы класса или общедоступного или защищенного подкласса могут непосредственно проверить либо изменить значение атрибута
- Скрытая видимость (*private*), означающая, что только методы класса (но не наследующих классов) могут непосредственно проверить или изменить значение атрибута

Стандартная нотация UML не определяет каких бы то ни было стереотипов атрибутов. Раздел "Ограничения объектов и бизнес-правила" предлагает некоторые расширения тегированных атрибутами значений.

Начальное значение — это значение, которое система автоматически присваивает атрибуту, когда создается экземпляр класса. Оно, например, непосредственно преобразуется в значение DEFAULT в операторе SQL CREATE TABLE.

Абстрактный устойчивый класс Человек на рис. 7.3 имеет несколько защищенных атрибутов и несколько общедоступных. Во время следующего цикла разработки можно будет добавить еще общедоступные операции (для получения и задания различных атрибутов), если потребуется. Можно, в частности, добавить конструктор, предоставляющий большинство данных при создании экземпляра объекта плюс ряд операций для предоставления доступа к значениям, плюс несколько операций для задания таких значений, как ДатаСмерти, которая может измениться во время жизненного цикла объекта Человек.

Можно принять некоторые упрощающие соглашения для получения и задания атрибутов. Если атрибут делают общедоступным, стандарты реализации могут преобразовать это в защищенную структуру данных вместе с операциями получения и задания. Можно добавить к атрибуту значение, помеченное только на чтение, чтобы определить создание только операций получения данных. Это именно та схема, которую использует, например, CORBA IDL для генерации классов.

СОВЕТУЕМ

При хорошей практической ОО-разработке все атрибуты делают либо защищенными, либо скрытыми, предоставляя непосредственный доступ только в пределах иерархии класса или в классе соответственно. Это менее очевидно с устойчивыми типами, поскольку реляционные и объектно-реляционные реализации могут оказаться не в состоянии представить видимость. Часто эти DBMS предоставляют только общедоступные атрибуты и, следовательно, инкапсуляция невозможна. Все же класс разрабатывается как для БД, так и для доменной модели системы, поэтому обычно атрибуты лучше инкапсулировать. Выбор между защищенными и скрытыми атрибутами зависит от того, желательно ли, чтобы подклассы могли

и использовать атрибуты. Автору больше нравится защищенная видимость для устойчивых классов с повторным использованием, в особенности для реляционных или объектно-реляционных реализаций. Однако при создании скрытых атрибутов классы отделяются от подклассов, снижая общую связность данных в системе.

Выражение типа данных зависит от языка. При определении устойчивых классов обычно используются типы данных SQL-92 или ODMG-2 ODL (см. подробнее в разделе "Доменные ограничения"). Это позволяет делать стандартный SQL или ODL, в то же время предоставляя достаточно данных для создания объявлений на других языках программирования. Как SQL-92, так и ODL имеют языковые привязки, преобразующие типы данных SQL/ODL в типы данных языков программирования. К сожалению, стандарт SQL-92 не имеет никаких привязок к языкам ООП, но можно для этого добавить стандартные привязки к стандартам разработки. Можно также определить типы данных C++, Java или других языков, а затем осуществить преобразование в типы SQL. ODL имеет стандартные привязки для этих языков.

ВНИМАНИЕ

Выражения типов данных являются частью более общей концепции ограничений домена. Дополнительная информация о выражениях типов данных и других ограничениях доменов — в разделе "Ограничения домена" последующего раздела "Ограничения объектов и бизнес-процесса". Можно использовать классификатор типа UML с целью определения типов данных для применения в других диаграммах, как показано в этом разделе.

Всегда возникает один вопрос: когда объявлять атрибут и когда объявлять ассоциацию? UML уравнивает атрибуты и ассоциации: тип данных атрибута — это имя класса или типа, показывающий, таким образом, ассоциацию к этому классу или типу [Rational Software, 1997a, p. 29]. В общем случае ООП рекомендуется определять атрибуты только для примитивных типов данных языков (например, INTEGER, VARRAY (FLOAT) или char[1000]). Для представления объектов, являющихся экземплярами классов, следует всегда пользоваться ассоциациями. Позвольте CASE-средствам или алгоритмам преобразования выполнить работу по созданию подходящих атрибутов в таблицах, типах объектов или объявлениях классов. Для специального случая разработки устойчивого класса необходимо определить атрибуты, преобразующиеся в стандартные примитивные типы данных целевой DBMS. На рис. 7.3 все типы являются стандартом SQL, кроме Пола и Семейного Положения. Это простые типы перечисления, действующие в некотором диапазоне кодированных значений (например, 'М' и 'Ж' для Пола и 'Холост' и 'Женат' для Семейного Положения). Большинство продуктов DBMS имеют некоторые способы непосредственного представления этих типов, хотя и не все. Если нет уверенности относительно типа, нужно сделать его классом и ассоциацией, а не атрибутом.

Некоторые атрибуты (и операции) в ОО-системах применяются к классам как целому, а не к каждому экземпляру. Например, если создается объект Преступник, он имеет все атрибуты от Человека на рис. 7.3 как

определенные значения для этого объекта. Можно также сделать атрибут атрибутом с областью действия класса, показывая его с подчеркиванием в списке атрибутов. Атрибуты с областью действия класса имеют единственное значение для всех объектов класса, т. е. все объекты используют одно и то же значение и все видят любое изменение этого значения. Реляционные таблицы не имеют атрибутов с областью действия класса, а ОР- и ОО-типы часто имеют такие атрибуты. В C++ атрибут с областью действия класса становится переменной статического класса памяти. Обычное использование этого в классах фабрики домена состоит в представлении статического оператора SQL, с помощью которого фабрика запрашивает коллекцию объектов. Это имеет меньше приложений в самих устойчивых классах.

Атрибуты устойчивых классов имеют ряд требований помимо стандартных свойств UML. В частности, нужно иметь возможность определения null, ключей, уникальности и ограничений домена. Последующий раздел "Объектные ограничения и бизнес-правила" рассматривает специальные свойства UML, соответствующие этим ограничениям, такие как OID, альтернативный OID и со свойством null.

В самом нижнем разделе прямоугольника класса содержатся операции. Можно определить операции с возвращаемыми типами данных, как на рис. 7.3. В следующем разделе "Операции" подробно рассматриваются операции и их синтаксис в диаграмме классов, а также их использование при моделировании данных.

Предоставляя информацию о классах и атрибутах, нотация класса позволяет определить почти полную модель данных. Расширение классов поведением через операции, методы и интерфейсы дает еще больше возможностей для спецификации проекта БД. Однако диаграмма классов становится гораздо более интересной, когда классы начинают соотноситься друг с другом при помощи отношений генерализации и ассоциации. В двух последующих разделах рассматриваются эти две темы.

Операции

Считается, что нет необходимости определять операции, если просто создается модель данных. Не верьте этому. С помощью хранимых процедур и функций в реляционной БД и методов в объектно-реляционной и объектно-ориентированной БД можно почти гарантировать присутствие операций на сервере базы данных.

Если не определено, что следует избегать кодирования поведения на сервере базы данных, нужно определить операции на классах. Позже можно решить, преобразуются ли методы, реализующие операции, в хранимые процедуры или другие подобные конструкции. В главах с 11 по 13 приводятся дополнительные рекомендации по операциям преобразования в расположенные на сервере методы для специфических видов концептуальных моделей данных.

Операции, методы и интерфейсы

Операция — это служба, которую можно запросить для экземпляра класса или объекта. Она запрашивает абстрактный интерфейс службы. *Метод* — это конкретная реализация интерфейса, выполняемый код, к которому обращаются при запросе службы. *Интерфейс* — это классификатор, представляющий абстрактную совокупность операций. Класс может иметь один или более интерфейсов, а интерфейсы могут группировать операции нескольких различных классов.

Операция имеет имя и список параметров, являющихся переменными, которые можно передать операции, чтобы повлиять на ее поведение, или которые операция может передать обратно для возврата данных. Операции появляются в третьем разделе прямоугольника класса при желании со своим списком параметров и другими разнообразными характеристиками. Вот синтаксис операций:

```
stereotype visibility name ( parameter-list ) : return-type-expression
( property-string )
```

Три стереотипа в UML для операций являются: "создать", "уничтожить" и "сигнализировать". Первые два стереотипа представляют конструкторы и деструкторы, операции, которые создают и уничтожают экземпляры класса. Они используются обычно для определения различных методов создания объектов и их уничтожения. Для моделирования данных они имеют другое назначение, особенно когда комбинируются с видимостью. Видимость принимает те же самые значения, что и атрибуты (общедоступные, защищенные или скрытые). Если конструктор объявляется защищенным, а не общедоступным, это означает, что другие объекты не могут создать объект без некоторых специальных свойств, позволяющих им осуществлять доступ. Для БД это означает, что приложения не могут добавлять строки в таблице объектов, такая же ситуация в ОО- и ОР-системах. Можно также использовать это для моделирования ситуации, когда база данных имеет одну или несколько строк, которые приложение не вправе добавить ни при каких обстоятельствах.

Стереотип "сигнализировать" используется специфическим образом при моделировании данных (см. раздел "Триггеры").

Список параметров (*parameter-list*) — это список переменных и типов данных в зависимом от языка синтаксисе, каким является выражение возвращаемого типа (*return-type-expression*). Комбинация имени, списка типов параметров и возвращаемого типа образуют *сигнатуру* операции, которая должна быть уникальной в пределах области действия класса. Если появляется вторая операция с той же самой сигнатурой, UML считает ее методом — реализацией операции, определенной при первом появлении сигнатуры. Список параметров определяется следующим синтаксисом:

```
kind name : type-expression = default-value
```

kind (разновидность) — это одна из нескольких возможностей, описывающая доступ операции к значению параметра:

- *in*: операция может читать значение, но не может изменять его (передача по значению).
- *out*: операция может изменять значение, но не может читать его или использовать.
- *inout*: операция может как читать значение, так и изменять его (передача по ссылке).

Default-value (значение по умолчанию) — это значение, принимаемое параметром, если параметр не передается, когда вызывается операция.

Можно пометить операцию свойством запрос. Определение операции как запрос означает, что у нее нет побочных эффектов — что она не изменяет состояние системы. Если операция преобразуется в устойчивые программные единицы языка программирования (хранимую процедуру или функцию), это указание бывает полезно при реализации. Например, Oracle позволяет расширить выражения SQL с помощью хранимых функций, не имеющих побочных эффектов. На рис. 7.3 операция ПолучитьАдрес — операция запроса. Обычно используется глагольный префикс "Получить" (Get) для указания запросов, возвращающих атрибуты или связанные объекты класса. Префикс "Задать" (Set) указывает на операции, не являющиеся запросами, которые обновляют атрибуты или связанные объекты.

ВНИМАНИЕ

Можно использовать ограничение (формальное булево выражение, которое оценивается как истинное или ложное в конце каждой транзакции и не имеет побочных эффектов) как спецификацию для операции запрос. Любой метод, реализующий операцию, должен удовлетворять ограничению. Подробности по ограничениям содержатся в разделе "Комплексные ограничения".

Можно также присвоить операции свойство абстрактная. Точно так же, как с именами классов в верхнем разделе прямоугольника класса, можно показать его, выделяя курсивом имя операции. Абстрактная операция не имеет соответствующего метода (реализации). В C++, например, это называется чистой функцией-членом. Абстрактные операции используются в абстрактных классах, чтобы показать интерфейсы, для которых подклассы предоставляют переопределяющую реализацию. Полиморфизм и динамическое связывание разрешают запрос соответствующим методом подкласса во время выполнения.

В классе Человек на рис. 7.3 операция ПолучитьИмя является абстрактной операцией запроса. Человек, будучи абстрактным классом, должен предоставить интерфейс, образующий поименованный объект для человека. Подклассы человека предоставят методы ПолучитьИмя, которые в действительности реализуют эту службу, но операция Человек является абстрактной и не имеет реализации. При создании экземпляра объекта Человек на самом деле создается экземпляр Преступника или человека какого-то другого типа. Допустим, что затем запрашивается операция ПолучитьИмя на объекте, воспринимаемом как объект класса Человек, который на самом деле является Преступником или каким-то другим подклассом. Система времени выполнения разрешает эту ситуацию с помощью динамического

связывания с методом, реализующим ПолучитьИмя для определенного подкласса. Можно, например, получить самую известную кличку преступника, включенную в имя: например, Сальватор "Счастливчик" Лучиано.

Интерфейс – это совокупность операций. Интерфейсы имеют специальную роль на диаграммах UML. Они являются чем-то вроде классификатора, это значит, что интерфейс можно определить везде, где можно определить класс. Такая абстракция позволяет определить пакеты с интерфейсами, которые абстрагируют внутреннюю структуру пакета как набор операций. Это дает возможность, кроме того, использовать один и тот же интерфейс в различных классах и пакетах. Язык Java имеет, например, ключевое слово "реализует" (implements), позволяющее включать в класс интерфейс, не требуя наследования его от суперкласса. Поскольку Java не поддерживает множественное наследование, это средство смешивает поведенческие абстракции, которые могут задействовать многие классы без усложнения модели наследования.

Для определения интерфейса создается прямоугольник класса, со стереотипом "интерфейс". На рис. 7.4 показан интерфейс Человек, как противоположность классу Человек. Обратите внимание, что там нет атрибутов, а только совокупность операций. Отметьте, что нет выделения курсивом; все в интерфейсе абстрактно, поэтому нет необходимости различать абстрактные элементы.

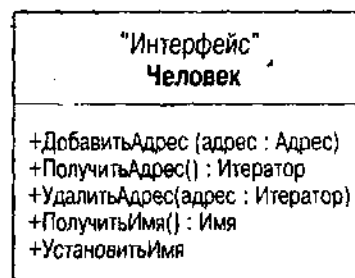


Рис. 7.4. Интерфейс Человек

Интерфейс присоединяется к пакету или классу с помощью помеченной линии, заканчивающейся кругом. Метка – это имя интерфейса. Эта стенограмма позволяет добавлять интерфейсы очень компактным образом, а также связывать классы или контейнеры, применяющие интерфейс, изображая пунктирную связь от использующего классификатора к интерфейсу.



Рис. 7.5. Использование интерфейса Человек

Это означает, что использующий классификатор применяет часть или весь интерфейс. Рис. 7.5 – диаграмма пакетов, показывающая подсистему Человек с двумя ее интерфейсами, интерфейсом Человек и интерфейсом Адрес.

На самом деле имеются два вида интерфейсов: интерфейсы пакетов и интерфейсы классов. Интерфейсы классов – небольшие совокупности операций, которые используют многие классы. UML использует пример интерфейса Comparable (Сравнимый), имеющего оператор `hash()` и оператор `isEqual()`. Его можно включить в любой класс, которому требуются операции сравнения вместо определения этих операций снова и снова в каждом классе. Интерфейсы пакетов, с другой стороны, предоставляют абстракции для пакетов, которые позволяют формализовать интерфейс пакета для использования и повторного использования другими пакетами. Это позволяет создавать подсистемы, которые повторно используются благодаря тщательной разработке интерфейса.

Когда операция превращается в действительный объект, поведение зависит от того, как и где реализуется операция. Можно реализовать операцию как поведение приложения, а можно как поведение сервера. Эти реализации имеют очень разные характеристики.

Поведение приложения

Операции превращаются в поведение приложения с помощью помещения методов в классы, являющиеся частью модели доменов приложения. *Поведение приложения* является таким образом поведением, ассоциируемым с устойчивыми объектами, которые выполняются как часть приложения.

До недавнего времени поведение большинства устойчивых объектов было поведением приложения, поскольку продукты DBMS не имели возможности выполнить поведение объекта на сервере БД. Весь исходный код надо было писать и компилировать как часть приложения, затем извлекать данные и выполнять над ними операции. Обычно имеется возможность выполнения служб либо в приложении, либо на БД. В приложении можно выбрать стандартную доменную модель, состоящую из устойчивых классов и различных вспомогательных средств и агрегатов (классов, агрегирующих экземпляры постоянных классов) или что-то более сложное.

Такое сложное поведение получает свой формат в зависимости от способа создания объектных подсистем. Например, можно создать классы, используя мультипоточную поддержку современных компиляторов и операционных систем, и можно сделать операции мультипоточными или мультизадачными. Можно определить некоторые операции как сигналы, отвечающие на события и прерывания вместо того, чтобы вызываться другими объектами. Мультипоточность позволяет действовать на объектах в приложении параллельно. Это означает также, что нужно создавать в объектах управление одновременностью, которое делает их сложнее. Это управление одновременностью полностью отделено от управления транзакциями, относящегося к одновременности использования БД; оно существует только для управления временными объектами.

При создании концептуальной схемы прикладные операции игнорируются. Таким образом, имеет смысл расширить свойство устойчивости UML на операции: (устойчивость=временный) означает, что операция является поведением приложения, а (устойчивость=устойчивость) означает, что операция имеет поведение сервера. Это расширенные свойства, а не стандартные свойства UML.

Поведение сервера базы данных

Поведение сервера — это поведение, которое ассоциируется с устойчивыми объектами, выполняющимися с помощью сервера БД. Где и как он на самом деле выполняется полностью зависит от сервера БД и его оптимизации. Допустим, что имеется DBMS с возможностью параллельной обработки на аппаратных средствах с несколькими процессорами. DBMS может осуществлять операции параллельно, не давая приложению почувствовать какую-либо разницу.

Большинством продуктов DBMS предлагаются три основные разновидности серверного поведения: методы, хранимые процедуры и триггеры. Это означает еще одно расширение UML — свойство формат, которое принимает одно из трех значений: метод, хранимая процедура или триггер, где метод является значением по умолчанию (в соответствии со стандартом UML). Например, можно определить, чтобы ПолучитьИмя на рис. 7.3 была бы серверной операцией, реализованной как хранимая процедура, присоединяя свойство формат=хранимая_процедура к операции в диаграмме классов.

ВНИМАНИЕ

Расширение UML — это очень высокий уровень, поскольку существует много параметров, реализацию которых можно определить в большинстве продуктов DBMS. Можно расширять свойства UML в соответствии со своими потребностями. Следует помнить о необходимости стандартизации расширений как части стандартов разработки так, чтобы любой, кто добавляет что-либо к проекту, пользовался теми же расширениями.

Методы

Метод — это конкретная реализация операции. В мире БД только объектно-реляционные и объектно-ориентированные БД имеют методы.

Продукты OODBMS имеют тенденцию рассматривать методы как методы приложений. Обычно ОО-схема создается с помощью кодирования классов с атрибутами и методами. Структура атрибутов включается в БД в то время, как методы остаются временными. При запросе объектов из БД создаются экземпляры временных объектов с помощью преобразования хранимых данных во временные данные, затем запрашиваются временные операции на временных данных в приложении. На самом деле никакое поведение не осуществляется на сервере БД.

Продукты ORDBMS определяют методы как серверные расширения. Некоторые продукты ORDBMS, например Oracle8, позволяют кодировать методы на языке программирования базы данных. В Oracle8 это означает PL/SQL (и в соответствии с маркетинговыми планами Oracle8, Java). Другие, в частности DB2 UDB, позволяют интегрировать внешние компилируемые

методы, написанные на "настоящих" языках программирования, таких как Си или C++. Есть и такие, как динамический сервер Informix, который создает модули расширения, определяющие методы для классов как части данного модуля (картриджи NCS для Oracle8 и DataBlades для Informix).

Продукты RDBMS не имеют методов потому, что таблицы не имеют поведения и типов. Если используется RDBMS, нужно определять методы как временные и выполнять их в приложении, а не на сервере. Однако можно схитрить с помощью хранимых процедур, как это описано в следующем разделе "Хранимые процедуры".

Не существует поддержки вызова метода со стороны стандартов ODBC или JDBC.

Если операции классов проектируются как устойчивые методы, то это ограничивает возможные варианты только несколькими доступными продуктами DBMS и по большей части продуктами ORDBMS. Лучше использовать стратегию разработки операций с открытой реализацией, которые преобразовать в различные реализации, если потребуется поддерживать другие БД.

Хранимые процедуры

Хранимая процедура — это операция, которую система не связывает явно с классом. Сравните это с глобальной функцией на C++, например. Все, кто видит хранимую процедуру, могут ее выполнить, передавая требуемые аргументы, не запрашивая наличия экземпляра объекта для квалификации запроса.

Хранимые процедуры возникли из попыток Sybase, Oracle, Informix и других продавцов RDBMS предоставить серверное поведение в их продуктах реляционной базы данных. Sybase определила язык, Transact-SQL, который был интегрирован с ее сервером; Oracle сделала то же самое с PL/SQL. Oracle сделала также PL/SQL средством прикладного программирования с помощью своего продукта Developer/2000, позволяя создавать библиотеки PL/SQL, которыми пользуется приложение, а не сервер.

Стандарт ANSI SQL-92 не предоставляет какой-либо поддержки хранимых процедур. ODBC и JDBC поддерживают запуск хранимых процедур с помощью специального управляющего синтаксиса [Jepson, 1996; Signore, Creamer and Stegman, 1995]:

```
{? = CALL <procedure name>(<parameter list>)}
```

Знак вопроса представляет значение, возвращаемое процедурой, если оно есть, и можно получить один или более результирующих наборов, создаваемых процедурой с помощью SQLFetch и его аналогов. Таким образом, можно кодировать реализацию операции как хранимую процедуру, затем вызывать ее при помощи специального синтаксиса во временном методе, который создается в объектной модели доменов.

Эта методика довольно полезна, когда целевая DBMS — реляционная. Используя стандартные механизмы безопасности, можно сделать реляционную БД по сути невидимой для всех приложений, кроме хорошо определенного набора хранимых процедур. Например, можно создать таблицу

Криминальная Организация и не предоставлять ее никаким пользователям. Никто из пользователей, кроме владельца таблицы, ее не увидит. Можно затем определить хранимые процедуры включения, обновления и уничтожения, которые выполняют эти обычные операции на таблице Криминальная Организация, предоставляя привилегии выполнения (нет, не того вида выполнения) процедурам заданных пользователей. Приложения затем смогут вызывать хранимые процедуры, но не смогут выполнять нижележащие операции над таблицами, эффективно инкапсулируя табличные данные точно так же, как класс делает это с его защищенными и скрытыми атрибутами.

Проект UML для данной ситуации — это стандартная разработка с защищенными атрибутами и хорошо определенными методами интерфейсов для создания, удаления, обновления атрибутов (операций Задать — Set) и запросов (операций Получить — Get и более широких операций "запросов"). Просто нужно определить свойство {формат=хранимая_процедура}, и на этом разработку завершить.

Триггеры

Триггер — это обработчик событий, т. е. код, выполняемый, когда происходит определенное событие на сервере БД. Стандарт ANSI SQL-92 ничего не говорит о триггерах или событиях, которые им сигнализируют. Большинство продуктов DBMS в действительности имеют триггеры в той или иной форме, но для них не существует стандартного формата.

Наиболее распространенным набором триггеров являются триггеры до и после события для трех стандартных событий манипуляции с реляционными данными: вставить, обновить и уничтожить. Это значит, что событие происходит на следующих этапах обработки в БД:

- Перед вставкой
- После вставки
- Перед обновлением
- После обновления
- Перед уничтожением
- После уничтожения

Ориентированные на строки триггеры выполняют проверку расширенных бизнес-правил и другие функции по реализации. Одно из применений — реализовать ограничения целостности ссылок (ограничения по внешним ключам) по всем БД с помощью триггеров. Например, когда строка включается в таблицу в одной базе данных, проверяется триггер Перед Включением, чтобы убедиться, что уже определенный внешний ключ существует в ссылке внешнего ключа на таблицу в другой базе данных. Это необходимо в средах с несколькими БД, потому что механизм ограничения целостности SQL требует, чтобы все таблицы были в одной БД, а это не очень удобно.

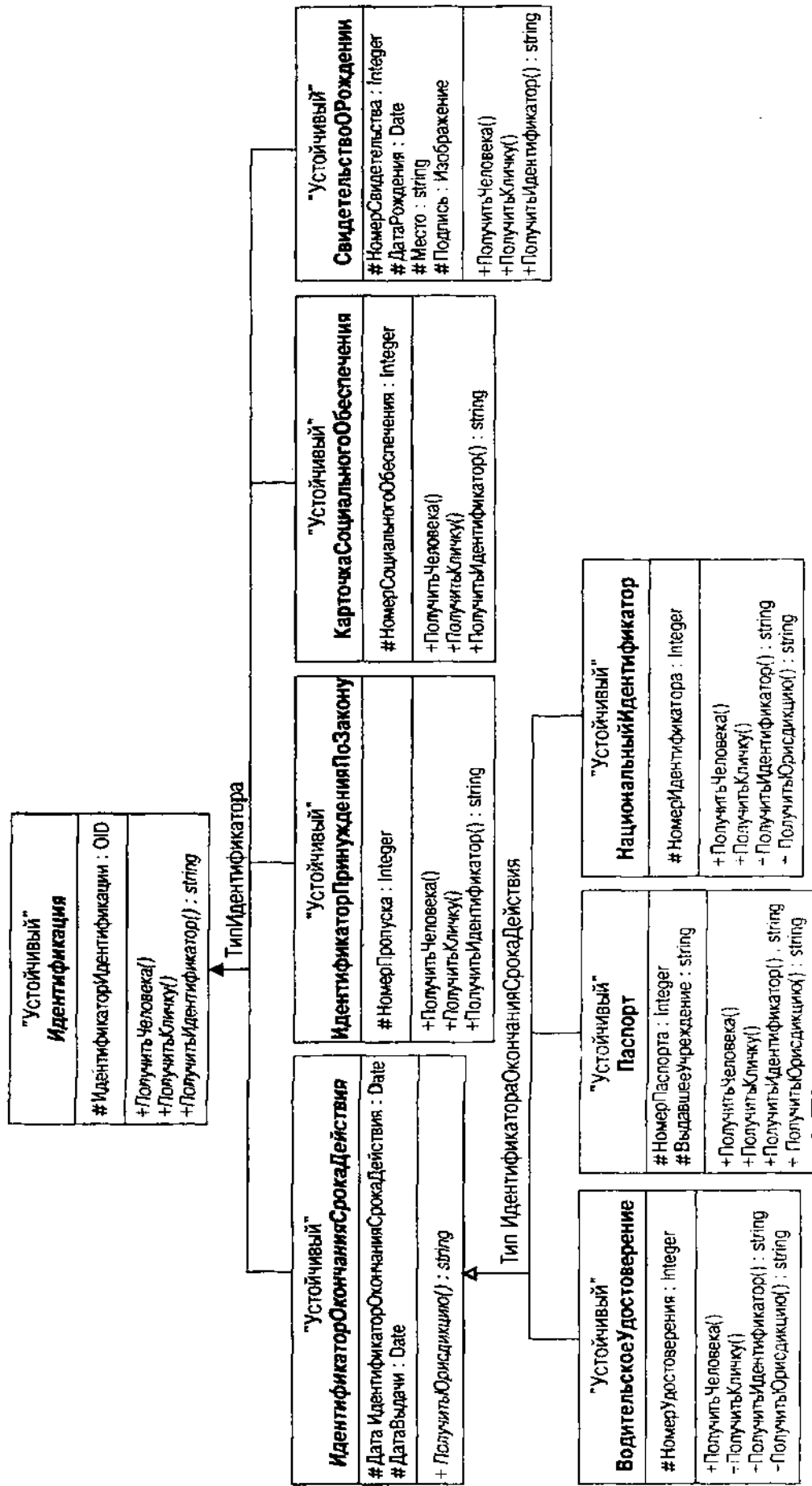


Рис. 7.6. Диаграмма классов Идентификация в UML

В главе 6 описан пример иерархии наследования, окружающей концепцию идентификации в разделе "Семантические отношения: создание подтипов и агрегация". Краткое повторение: Идентификация — абстрактная концепция, разбивающаяся на ИдентификаторОкончанияСрокаДействия, ИдентификаторПринужденияПоЗакону, КарточкаСоциальногоОбеспечения и СвидетельствоОРождении. ИдентификаторОкончанияСрокаДействия в свою очередь разбивается на несколько классов: ВодительскоеУдостоверение, Паспорт и НациональныйИдентификатор. Переработка этого примера на языке отношений генерализации UML: ВодительскоеУдостоверение, Паспорт и НациональныйИдентификатор являются более специфическими, чем ИдентификаторОкончанияСрокаДействия, который является более специфическим, чем абстрактный класс Идентификации. Каждый из этих трех элементов полностью однороден с ИдентификаторомОкончанияСрокаДействия и содержит больше сведений (например, номер удостоверения, выдавшая документ организация и страна). Можно использовать паспорт везде, где требуется идентификация окончания срока действия (или даже большую генерализацию). На рис. 7.6 показана диаграмма классов UML для иерархии классов Идентификации.

Стрелки с белыми наконечниками — отношения генерализации. Стрелки указывают на более общий класс, известный как *суперкласс*, или *базовый класс*. Стрелки исходят из более специализированных классов, известных как *подклассы*, или *производные классы*. *Листовой класс* — это класс, у которого больше нет специализации (нет входящего отношения генерализации). *Корневой класс* — это класс, не имеющий ничего более общего (нет исходящего отношения генерализации).

Суперклассы с несколькими подклассами, такими, как Идентификация или ИдентификаторОкончанияСрокаДействия, могут иметь именованный дискриминатор в UML. Дискриминатор становится атрибутом суперкласса, идентифицирующего подкласс, хотя атрибут в списке атрибутов не определяется, он является просто меткой для генерализации. Например, ИдентификаторОкончанияСрокаДействия имеет атрибут ТипИдентификатораОкончанияСрокаДействия с тремя возможными значениями: ВодительскоеУдостоверение, Паспорт и НациональныйИдентификатор. Поскольку класс ИдентификатораОкончанияСрокаДействия — это абстрактный объект, атрибут не может быть null; экземпляр объекта должен быть всегда одним из трех подклассов.

Наследование и генерализация — это наиболее сложные концепции ООП. Последующее обсуждение является обзором концепций и проблем без углубления в подробности, что требуется для того, чтобы стать экспертом при использовании генерализации в ОО-разработке. Вместо этого затрагиваются основные концепции с точки зрения моделирования данных. В других книгах по ООП эта тема рассматривается детально [Booch, 1994; Jacobson et al., 1992; Rumbaugh et al., 1992].

Генерализация

Человеческий разум пытался понять мир с помощью генерализации со времен Аристотеля и Платона, а может, и раньше. Вероятно, пирамиды или

клинопись и иероглифы представляют даже более древнюю абстракцию и генерализацию. Человеческий разум извлекает большую пользу из своей способности на основе структуры и поведения множества различных вещей создавать абстрактные классы вещей.

Например, на рис. 7.6 различные виды идентификации, использующие дату окончания срока действия, объединяются в абстрактный класс ИдентификаторОкончанияСрокаДействия, который содержит дату окончания срока действия. Абстракция, наряду с идентификационными классами без окончания срока действия, генерализуется в абстрактную концепцию идентификации. Это *генерализация* абстрактного объекта. Когда создается экземпляр паспорта или водительского удостоверения, этот объект подкласса наследует атрибут даты окончания срока действия из своего суперкласса — ИдентификатораОкончанияСрокаДействия. Это *наследование* из абстракции.

Двигаясь в другом направлении — *специализации*, — подкласс расширяет или ограничивает суперкласс. Расширение суперкласса означает добавление атрибутов, операций или ассоциаций к тем, которые присутствуют в суперклассе и унаследованы подклассом. Ограничение суперкласса означает переопределение поведения суперкласса полиморфными или виртуальными операциями.

Полиморфизм

Полиморфизм — это способность языка использовать одно и то же имя для различных операций. Используя терминологию C++, можно сказать, что существует два основных вида полиморфизма. *Перегрузка* — это способность предоставить операцию с одним и тем же именем, но с разными сигнатурами (типы параметров разные, хотя имя может быть одинаковое). *Переопределение* — это способность определить в подклассе операцию, ограничивающую или изменяющую поведение, но имеющую ту же самую сигнатуру.

Перегрузка предоставляет несколько разных возможностей творческому разработчику для упрощения (или усложнения) программирования. Так, можно использовать перегрузку для предоставления "единственной" операции, которую затем применить ко многим разновидностям объектов. На самом деле, конечно, нужно создать много различных перегруженных операций; однако программист, использующий их, не должен беспокоиться об этом. Другой пример — перегрузка имени, чтобы предоставить слегка отличное поведение в зависимости от типа ввода.

Переопределение — ключевое свойство в большинстве ОО-языков программирования. Все эти языки поддерживают *позднее* или *динамическое связывание*, способность системы времени выполнения определять, какой использовать метод в зависимости от типа объекта. Рис. 7.6 имеет расширенный пример переопределения. Абстрактный класс Идентификация имеет три операции: ПолучитьЧеловека, ПолучитьКличку и ПолучитьИдентификатор. Они сами по себе являются абстрактными операциями (интерфейс) без реализации методов. Подкласс ИдентификатораОкончанияСрокаДействия, являющийся также абстрактным классом, не определяет каких бы то ни было реализаций, но все другие конкретные классы переопределяют три операции для предоставления специфических реализаций. Рассмотрим человека,

имеющего несколько идентификационных документов. С его точки зрения это набор объектов Идентификации, а не объектов индивидуальных типов (Паспорт, Свидетельство О Рождении и т. д.). При создании сущности Паспорт, например, и запросе Идентификатора с использованием ПолучитьИдентификатор от объекта (как объекта Идентификации), система времени выполнения отвечает на запрос Идентификация::ПолучитьИдентификатор() как на запрос Паспорт::ПолучитьИдентификатор() с помощью позднего связывания. Метод, реализующий операцию, ищет атрибут номера паспорта и преобразует его в строку, а затем возвращает его. Это поведение ограничивает абстрактное поведение суперклассов.

ОСТОРОЖНО

Перегрузка и переопределение предоставляют возможность существенно изменять поведение подкласса. Как и для всех омонимов, если значение существенно различно, не получится ничего, кроме путаницы. Перегруженные и переопределенные операции должны иметь подобную семантику в различных классах.

Какое отношение имеют перегрузка и переопределение к моделированию данных? При использовании OODBMS или ORDBMS в качестве целевой реализации они обе поддерживают (в большей или меньшей степени) концепции полиморфизма. Даже некоторые реляционные БД (Oracle7 и Oracle8) поддерживают перегрузку: используя модель Ada, пакеты PL/SQL позволяют иметь несколько программных блоков с одним и тем же именем и различными сигнатурами.

Можно пойти довольно далеко с устойчивыми версиями операций. Поскольку модель классов используется как посредник для временной и устойчивой моделей, важно иметь ясное представление относительно операций перегрузки и переопределения, чтобы все было понятно с соотношением объектов в памяти и устойчивых объектов БД.

Абстрактные и конкретные классы и наследование

Вспомним, что абстрактный класс — это такой класс, который не имеет экземпляров, в то время как конкретный класс — это класс, который может иметь экземпляры. Интерфейс — это абстрактный класс без атрибутов, содержащий только абстрактные операции. В общем случае абстрактный класс может иметь как атрибуты, так и операции.

Абстрактные классы не имеют смысла без отношений генерализации. Что хорошего, например, в классе Идентификация, если это абстрактный класс без своих подклассов? Поскольку нельзя создать объект в классе, единственная цель его существования — установка общих свойств его конкретных подклассов наследования при создании их объектов.

Абстрактные классы позволяют выразить промежуточные концепции в семантической иерархии, такие, как Идентификатор Окончания Срока Действия. Абстрактные классы создаются, когда есть общие свойства в виде связанного кластера поведения. Иногда это соответствует легко понимаемому объекту, а иногда — нет. Если нет, проверьте еще и еще раз свою разработку, убедитесь, что не было введено слишком много абстракции. Чем проще и чем слабее связность наследования, тем лучше.

ВНИМАНИЕ

Интерфейс является абстрактным классом, но не все абстрактные классы являются интерфейсами. Хотя нельзя иметь методы, реализующие операции в интерфейсе, можно иметь их в абстрактном классе, не являющемся интерфейсом. Это позволяет использовать общее поведение в конкретных подклассах.

Каково место абстрактных классов в модели данных? Абстрактный класс предоставляет место для размещения атрибутов и операций, которые иначе надо было бы распределять по многим классам. Сокращается избыточность путем централизации этих объектов. Когда проект переводится в схему БД, при меньшей избыточности обычно получается более гибкая схема. Можно встретить больше сложностей в реляционных схемах из-за дополнительных таблиц, но там меньше проблем с нормализацией и легче управление таблицами и поддержка.

Множественное наследование

Классы могут иметь более одного общего родительского суперкласса. Если это входит в проект, нужно использовать множественное наследование. Мир на самом деле полон подобных вещей потому, что человеческий разум классифицирует мир, а не наоборот. Люди, как пикто, неоднородны и сложны и могут найти бесконечное количество способов классификации вещей. Наше мышление стремится к совпадениям — что хорошо для размышления, но отвратительно для разработки ПО.

Общая причина для использования множественного наследования — присутствие более одной размерности данных. Если размерности моделируются с помощью деревьев наследования, это приведет к нескольким под-деревьям из общего суперкласса. По мере разбиения вещей на части можно увидеть объекты, сочетающие массивы осмысленным образом, поэтому для объединения деревьев используется множественное наследование. Один из аспектов каталога — это прослеживание украденной собственности.

— Это гусь, мистер Холмс! Это гусь, сэр! — произнес он, задыхаясь.

— Да неужели? Что в нем такого? Он ожил и выпорхнул наружу через кухонное окно? — Холмс повернулся на диване, чтобы лучше рассмотреть возбужденное лицо человека.

— Посмотрите вот сюда, сэр! Посмотрите, что моя жена нашла в его зобу! — он протянул руку — на ладони у него искрился голубой камень, немного меньше боба по размеру, но такой чистоты и яркости, что он сиял, как электрическая точка в темной выемке его руки.

Шерлок Холмс встал, присвистнув.

— Клянусь Богом, Петерсон, — сказал он, — это настоящее присвоение чужой собственности! Я полагаю, вы знаете, чем владеете?

— Бриллиант, сэр? Драгоценный камень. Он режет стекло, как масло.

— Это не просто драгоценный камень. Это тот самый драгоценный камень.

— Не голубой ли карбункул графини де Моркар?! — воскликнул я.

— Вероятно, так. Я должен знать его размер и форму, поскольку читал объявление о нем в "Таймс" каждый день в последнее время. Он абсолютно уникален, и о его стоимости можно только догадываться, но предложенное вознаграждение в 1000 фунтов определенно не составляет одну двенадцатую часть его рыночной стоимости.

... Когда комиссионер ушел, Холмс взял камень и посмотрел его на свет.

— Это миленькая вещичка, — сказал он. — Ну, посмотрите, как он переливается и сверкает. Конечно, он ядро и центр преступления. Как и любой хороший камень. Это любимцы дьявола. В более крупных и старых ювелирных украшениях за каждым каратом может стоять кровавое дело. Этот камень не старше двадцати лет. Он был найден на берегу реки Амой в Южном Китае и замечателен тем, что имеет все свойства карбункула, сохранив голубой цвет вместо рубиново-красного. Несмотря на молодость, у него есть уже своя кровавая история. С ним связаны два убийства, отравление купоросом, самоубийство и несколько ограблений, совершенные из-за каких-то сорока карат кристаллизованного углерода. Кто бы мог подумать, что эта миленькая игрушка станет поставщиком для виселиц и тюрем? Я запрю ее теперь в моем крепком сундуке и чиркну пару строк графине о том, что она у нас". [BLUE]

При отслеживании таких объектов следует обратить внимание на две размерности: вид используемой собственности и статус страхования собственности. На рис. 7.7 — структура наследования, начиная с собственности. Одна ветвь иерархии разделяет собственность на типы по их природе: коллекционные вещи, ювелирные украшения, наличные деньги, ценные бумаги, личные и прочие (чтобы сделать классификацию полной). Другая ветвь иерархии разделяет собственность на незастрахованную, застрахованную частным образом или имеющую государственную страховку в зависимости от источника страхования от кражи. Все они являются абстрактными классами.

Сложность такой иерархии возникает, когда нужно создать настоящий объект: объединить два поддерева в объединенные подклассы, такие, как Незастрахованные ювелирные украшения или Ценные бумаги с государственной страховкой. В этом случае необходимо реально определить подкласс декартова произведения (все возможные комбинации абстрактных классов).

ВНИМАНИЕ

Автор не защищает данный способ разработки. В общем случае следует избегать этой разновидности проектирования как чумы. Можно кое-что использовать в зависимости от имеющихся потребностей. Главным оправданием такого вида разработки является возможность переопределения операций и добавления структур в различных размерностях. Если нет особой необходимости расширения или ограничения операций и структуры, не следует даже рассматривать подход с иерархией размерностей; просто используйте для размерностей перечислимые типы данных. Нужно пытаться ограничить наследование только необходимым, а не перегружать семантическими особенностями, которые с прагматической точки зрения похожи.

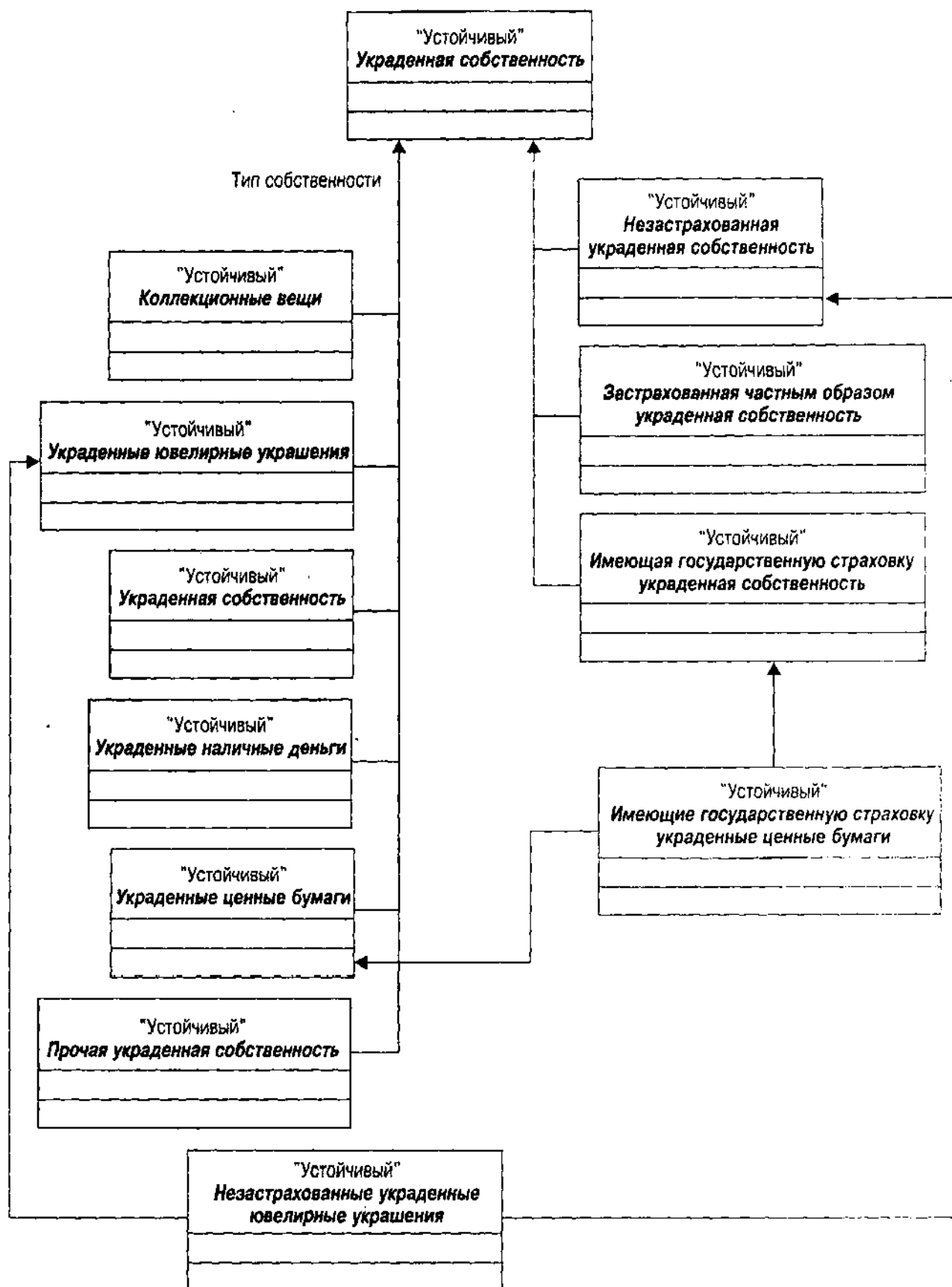


Рис. 7.7. Размерностное множественное наследование украденной собственности

Эта форма множественного наследования имеет непопулярную ромбовидную форму, где суперклассы в конечном счете сливаются в единственного предка. Эта структура приводит к некоторым сложным проблемам при разработке языков программирования. Если вызывается операция и эта операция существует как в непосредственно предшествующем суперклассе, так и в общем предке, то какой метод на самом деле выполняется? С++ заставляет точно определить операцию в операции разрешения контекста, идентифицируя владельца класса, что увеличивает связность. И еще хуже то, что из-за своей структурной логики С++ дает две копии общего предка — УкраденнойСобственности. Нужно объявить промежуточные классы виртуальными базовыми классами на С++, чтобы заставить компилятор использовать общего предка. В результате разработчик вынужден модифицировать несколько классов, если хочет добавить класс, используя множественное наследование, вновь увеличивая связность.

Подумайте о структуре, преобразованной в устойчивые структуры. Имеется таблица УкраденнаяСобственность, таблица для каждого типа собственности и таблица для каждого типа страховки. Кажется естественным добавить таблицу ЗастрахованныеУкраденныеЮвелирныеУкрашения, если нужно представить атрибуты или операции, которые имеют этот счет и не имеют — два его предка. Идентификатор объекта украденной собственности проходит через эти таблицы, представляя отношения генерализации. Строка в ЗастрахованныеУкраденныеЮвелирныеУкрашения имеет отношение один-к-одному со строками в ЗастрахованнаяУкраденнаяСобственность и УкраденныеЮвелирныеУкрашения. Это представляет тот факт, что объект ЗастрахованныеУкраденныеЮвелирныеУкрашения является также объектом ЗастрахованнаяУкраденнаяСобственность и УкраденныеЮвелирныеУкрашения.

Если затем восстановить это в обратном порядке в диаграмму UML, то естественно использовать ассоциации, а не генерализацию, делая объединенную таблицу отношением, а не сущностью. Это значит, вместо операций наследования и структуры нужно ссылаться на ЗастрахованныеУкраденныеЮвелирныеУкрашения как на ассоциацию между ЗастрахованнойУкраденнойСобственностью и УкраденнымиЮвелирнымиУкрашениями, а не как на подкласс их обоих. Один подход к данному виду множественного наследования — разложение размерностей на отдельные классы и использование ассоциаций для построения отношений между ними [Blaha and Premerlani, 1998, p. 60].

Размерностное множественное наследование — не единственная имеющаяся разновидность. Другая причина его использования — комбинация двух не соотносящихся классов (т. е. классов без общих предков). В большинстве случаев это отражает смешанный подход: смешиваются поведение или структура, необходимые подклассу. Например, на некотором этапе может потребоваться относиться к отпечаткам пальцев как к идентификации, но класс ОтпечатковПальцев — это вид изображения. На рис. 7.8 показано, что это выглядит как множественное наследование.

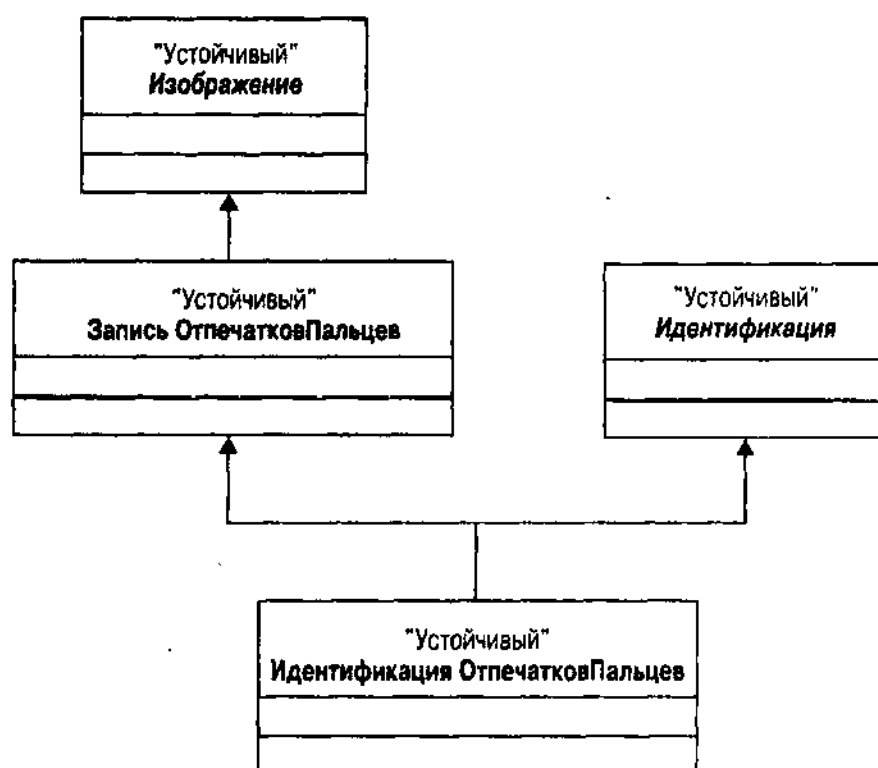


Рис. 7.8. ОтпечаткиПальцев как Идентификация с использованием смешанного множественного наследия

Эта разработка реализует интеграцию операций и структуры из класса Идентификации в класс ОтпечаткиПальцев с помощью выделения подкласса обоих, используя множественное наследование. Результирующий класс ИдентификацияОтпечатковПальцев действует как запись отпечатков пальцев (с изображением) и как вид идентификации без истечения срока действия.

ВНИМАНИЕ

Имеются способы это сделать лучше. Смешанное наследование удобнее делать с помощью интерфейсов. Создается интерфейс, необходимый для любого вида идентификации, затем он включается в класс ОтпечатковПальцев или в подкласс. См. следующий раздел "Подтипы: Сравнение типов с классами и с интерфейсами", который рассматривает данный пример.

Множественное наследование позволяет сказать, что вещь является разновидностью нескольких других вещей. В той мере, в какой это отражает действительное значение вещи, это хорошо. К сожалению, то, что происходит при разбиении на подклассы двух или более классов, экспоненциально увеличивает связность системы через наследование. Не всегда связывание по наследству плохо. Для любого данного класса общие атрибуты, операции и ассоциации, унаследованные из суперкласса, имеют смысл с точки зрения объекта. Но по мере роста числа суперклассов увеличивается взаимная зависимость все большего и большего количества частей системы.

Когда затем происходит изменение части системы, оно влияет на все ее подклассы определенным образом и необходимо их повторно протестировать. В модели данных с устойчивыми классами это изменение по

определению будет устойчивым: изменяется схема базы данных. Миграция схем при таких обстоятельствах является более трудной, чем поддержка запутанного, как спагетти, кода с плавающими в нем мясными шариками объектов. Чем больше взаимных связей, тем более сложной становится поддержка и расширение. Поэтому в интересах разработчика реже обращаться к множественному наследованию.

Обоюдоострый меч множественного наследования существует по одной причине: для моделирования объектов, которые желательно использовать как подтип двух или более типов. И это не так просто, как может показаться.

Подтипы: Сравнение типа с классами и с интерфейсами

Даже определение генерализации в UML включает в себя концепцию подтипа: "Экземпляр более специального элемента может использоваться там, где допустим более общий элемент". В строго типизированной системе поведение объекта зависит от его типа и нельзя вызывать операции или получать доступ к атрибутам в контекстах, требующих объектов другого типа. Создание *подтипа* позволяет расширить концепцию "того же типа" на иерархию связанных объектов, а не просто на объекты одного класса.

Специалисты по вычислительной технике тщательно разделяют понятия типа и подтипа от концепции наследования. Разработчики языков программирования обычно не так разборчивы, хотя некоторые ОО-языки действительно различаются. Такие языки, как Smalltalk, вообще не имеют типов, а пользуются только наследованием. В Smalltalk, например, можно посылать сообщение любому объекту, предоставляющему метод, с той сигнатурой, которая определяется в сообщении.

Основная причина строгого типизирования состоит в том, что этот тип гибкости приводит к неизбежным дефектам, по крайней мере, по трем причинам. Можно случайно настроить систему так, что во время работы будет вызван метод для несуществующего объекта, что повлечет прерывание (падение с вершины иерархии наследования). Можно вызвать метод, который делает что-то совершенно несоответствующее тому, что планировалось. Можно передать объекты, значительно отличающиеся по формату, вызвав ошибки операционной системы или ошибки при выполнении, такие, как деление на ноль или нарушение прав доступа.

C++ объединяет создание подтипов и наследование в единую структуру — иерархию наследования. Подкласс — это подтип в C++. Он использует объект подкласса C++ везде, где может быть использован один из его предков. Это в свою очередь представляет основную причину для множественного наследования. Поскольку строгое типизирование не позволяет использовать объект, не наследующий от определенного суперкласса, необходимо применять множественное наследование, чтобы наследовать этот суперкласс. Иначе следует отказаться от строгого типизирования, используя пустые указатели (void) или что-то подобное.

Интерфейс — альтернативный вариант для множественного интерфейса, вводя совершенно другое отношение подтипа. В системах, поддерживающих интерфейсы как типы, можно использовать объект, поддерживающий интерфейс везде, где используется данный интерфейс. Вместо объявления переменных

с типом класса применяются типы интерфейса. Затем это позволяет ссылаться на операции интерфейса на любом объекте, предоставляющем данные методы. Это все еще строгое типизирование, но можно получить многое от гибкости со слабо типизированных систем, таких, как Smalltalk, без сложности и сильного связывания множественного наследования. Язык программирования Java поддерживает такой вид контроля типов. Обратная сторона интерфейсов — их на самом деле нельзя использовать для наследования кода, а только для спецификации интерфейса. Необходимо реализовать все операции в интерфейсе с надлежащими методами при их применении к классу.

Интерфейсы поддерживают специальное отношение UML: отношение "осуществляет". Это вид отношения генерализации, представляющий не наследование, а *реализацию* типа. Когда интерфейс ассоциируется с классом, говорят, что класс реализует интерфейс. Таким образом, класс может действовать как подтип интерфейса. Можно реализовать объекты класса, затем присвоить их переменным, имеющим тип интерфейса, и использовать их через объявленный интерфейс. Это подобно тому, что делают с суперклассами, но имеется доступ только к интерфейсу, а не ко всем операциям на подклассе.

Реализация интерфейса позволяет использовать переопределение и позднее связывание вне иерархии класса, освобождая разработчика от искусственных ограничений, налагаемых наследованием в структуре дерева. Например, вместо создания экземпляров разных подклассов Идентификации и помещения их в контейнер, с типом класса Идентификация, можно присвоить контейнеру тип интерфейса Идентификации. Это предоставляет разработчику лучшее из обоих миров. Разработчик будет все еще иметь сильный тип и безопасность типа, но в то же время получит доступ к объекту и через множественные смешанные интерфейсы. Множественное наследование также предоставляет эту возможность, но с гораздо большей степенью системной связности между классами, что не является хорошей идеей.

На рис. 7.9 показана иерархия Идентификация, переделанная с использованием интерфейсов. Диаграмма становится проще и дает дополнительную гибкость благодаря использованию интерфейса Идентификация в других классах помимо иерархии Идентификация. Рассмотрите еще раз пример идентификации отпечатков пальцев из приведенного выше раздела "Множественное наследование". Простое добавление интерфейса Идентификации к ОтпечаткамПальцев и создание затем надлежащих методов для реализации трех операций интерфейса позволяет расширить ЗаписьОтпечатковПальцев и применять его везде, где используется интерфейс Идентификации.

На рис. 7.9 показаны также две формы отношения реализации. Пунктирные стрелки с белыми наконечниками — это формальное отношение реализации, показывающее связь Идентификации со СвидетельствомОРождении. В других классах, чтобы сэкономить пространство, показывают отношение реализации как маленький круг с меткой, связанный с классом. Метка — это имя интерфейса.

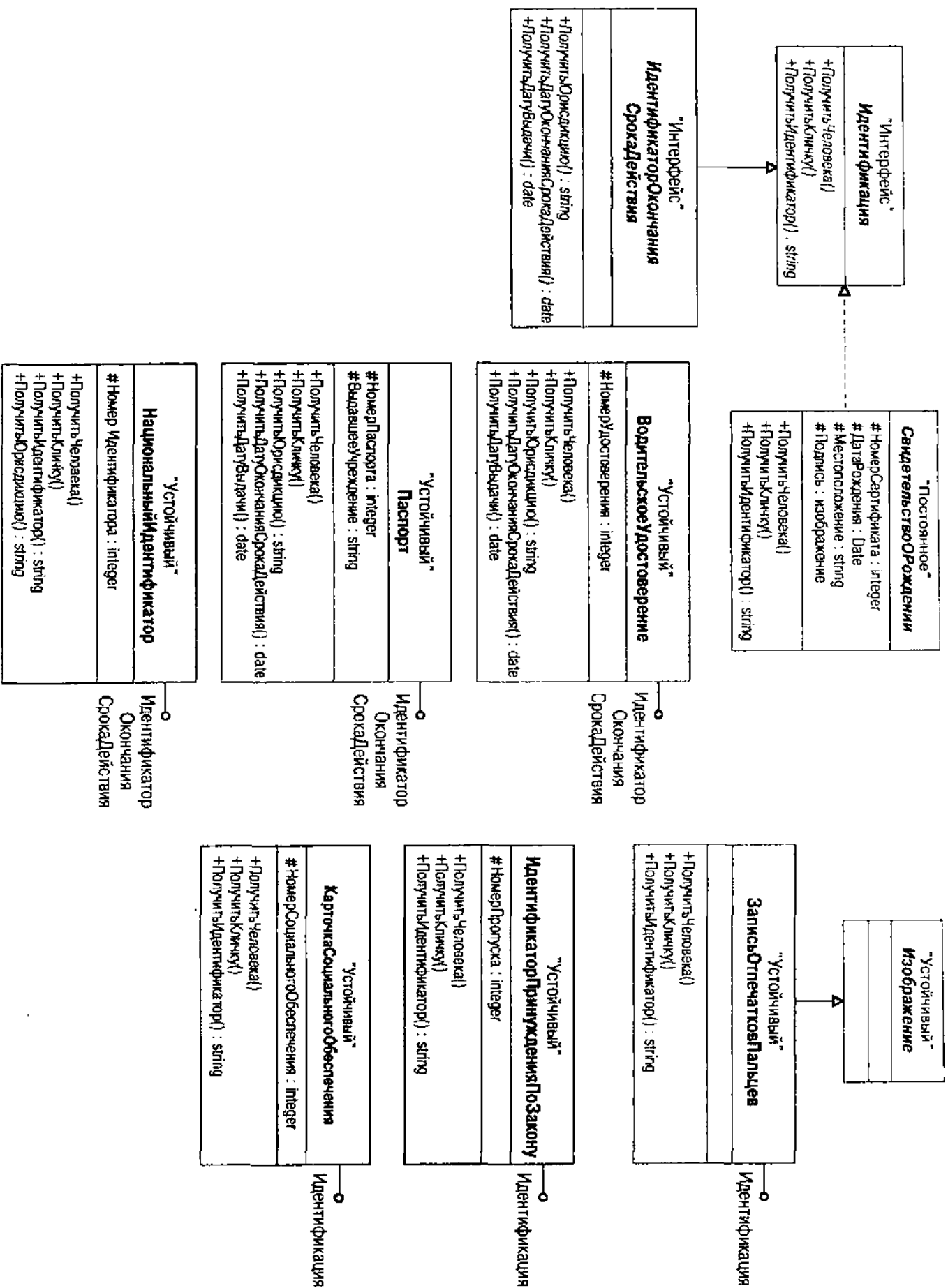


Рис. 7.9. Интерфейс Идентификации и реализация интерфейса

Рис. 7.9 менее сложен, но легко увидеть, почему интерфейсы не являются предпочтительным способом создания подтипов любого вида. Хотя наследование абстрактной операции `ПолучитьДатуОкончанияСрокаДействия` логически является таким же, как и наследование атрибута `ДатаОкончанияСрокаДействия`, последний имеет большее практическое значение. Это особенно верно в модели данных, предназначенной для реализации в логической схеме DBMS. Как правило, если имеются атрибуты с общими свойствами, нужно использовать генерализацию; если есть только операции, нужно использовать интерфейсы. Идентификация проходит эту проверку (игнорируя идентификатор объекта), в то время как `ИдентификаторОкончанияСрокаДействия` — нет. Класс `ЗаписьОтпечатковПальцев` — лучший пример хорошей реализации интерфейса. Если можно использовать фразу "используется как", как в "запись отпечатков пальцев используется как идентификация", ищут реализацию интерфейса. Если используется фраза "является", как в "паспорт является идентификатором со сроком истечения действия", это генерализация.

Наконец, рис. 7.9 показывает свойство интерфейсов: наследование интерфейса. Интерфейс `ИдентификаторОкончанияСрокаДействия` наследует интерфейс `Идентификация`, и можно использовать объект с интерфейсом `ИдентификаторОкончанияСрокаДействия` везде, где используется интерфейс `Идентификация`.

Имеется третий вид концепции, связанной с типом, в языках программирования: общность. Языки, поддерживающие шаблоны (C++) или общности (Ada, Eiffel), позволяют создавать множество версий реализации класса или функции, каждая из которых параметризуется набором типов и/или объектов. Когда создается экземпляр объекта с типом при помощи общего класса, предоставляются надлежащие типы и объекты как параметрические аргументы. Все операции и атрибуты получают соответствующие типы, и можно использовать объект в строго типизированной системе. Классическое применение общностей — базовые структуры данных, такие, как стеки и деревья, и общие алгоритмы, такие, как сортировки. Параметризуются те виды объектов, которые содержатся, например, в агрегированных структурах.

Присвоение типов хорошо подходит для моделирования данных. Большинство языков программирования баз данных имеют сильные типы, хотя некоторые из них весьма любопытны с этой точки зрения. Например, SQL не считает определение таблицы типом. При создании поименованных таблиц создается агрегатный объект (Дейтовская реляционная переменная `relvar`), а не тип, который можно использовать для определения типа атрибута. Курсоры предоставляют общий способ доступа к этим объектам. Можно найти самые последние реляционные CASE-средства DBMS для определенной последовательности моделирования данных. При определении схемы создается сущность. Когда развивается эта схема, сущность становится таблицей в базе данных. Если требуется определить несколько таблиц с одной и той же структурой, нужно скопировать имеющуюся сущность или создать новую, которая выглядит точно так же. Эти вещи не имеют типов. В последнем разделе "Ограничения доменов" рассматриваются некоторые из проблем типов атрибутов более подробно.

Продукты ORDBMS расширяют системы с типами при помощи типов, определенных пользователем (UDT на жаргоне продавцов и стандарта SQL3). Разработчик определяет тип, затем из него создаются таблицы. Можно получать доступ ко всем объектам данного типа с помощью определенных вариаций оператора SELECT или иметь доступ только к тем, которые находятся в единственном экземпляре таблицы [Stonebraker and Brown, 1999]. Это открывает мир интересных вариантов памяти, освобождая память строк/объектов от определения их типа. В RDBMS нужно определить физическую модель всего множества объектов данной сущности. В ORDBMS можно хранить несколько таблиц одного и того же типа в совершенно разных местах. Схема разбиения таблиц Oracle8 позволяет автоматизировать процесс, не создавая проблем логического местоположения экземпляра. Таблица — это одна таблица, хранимая в нескольких местах, определенных с помощью данных, а не многие таблицы одного и того же типа, хотя это тоже возможно.

Большинство продуктов OODBMS приняли модель C++ (небольшим исключением является Genstone с форматом Smalltalk). Хорошо или плохо, но это означает интеграцию типов и наследования в схемах OODBMS. Одно отличие продуктов OODBMS состоит в том, что создаются объекты определенного типа, которые хранятся в логических (а не физических) контейнерах.

Ни один из этих продуктов, насколько мне известно, непосредственно не поддерживает интерфейсы или общности. Интерфейсы полезны в модели данных как способ выражения отношений реализации, но преобразование логических и физических моделей требует определенной работы. Общности или шаблоны просто не существуют в мире баз данных.

В то время как генерализация во всем ее разнообразии является мощным средством моделирования данных, она бледнеет в свете другого основного типа отношений: ассоциации.

Ассоциации, включение и видимость

Отношение *ассоциации* — это "семантическое отношение между двумя или более классификаторами, включающее связи между их экземплярами" [Rational Software, 1997b, p. 149]. Другими словами, ассоциации показывают, как объект класса соотносится с другими объектами. Сеть ассоциаций определяет *иерархию включения* — граф, показывающий, какие типы объектов содержат другие типы объектов. Он также определяет *иерархию видимости* — граф, показывающий, какие типы объектов могут видеть и использовать другие типы объектов.

Включение важно потому, что обладание объектом является основополагающим бизнес-правилом. Обладание влияет на то, где создаются объекты, что происходит, когда объекты уничтожаются и как объекты сохраняются во всех различных типах DBMS. Основной выбор при создании ассоциации делается тогда, когда решают, представляет ассоциация ссылку на независимый объект или на связь владения.

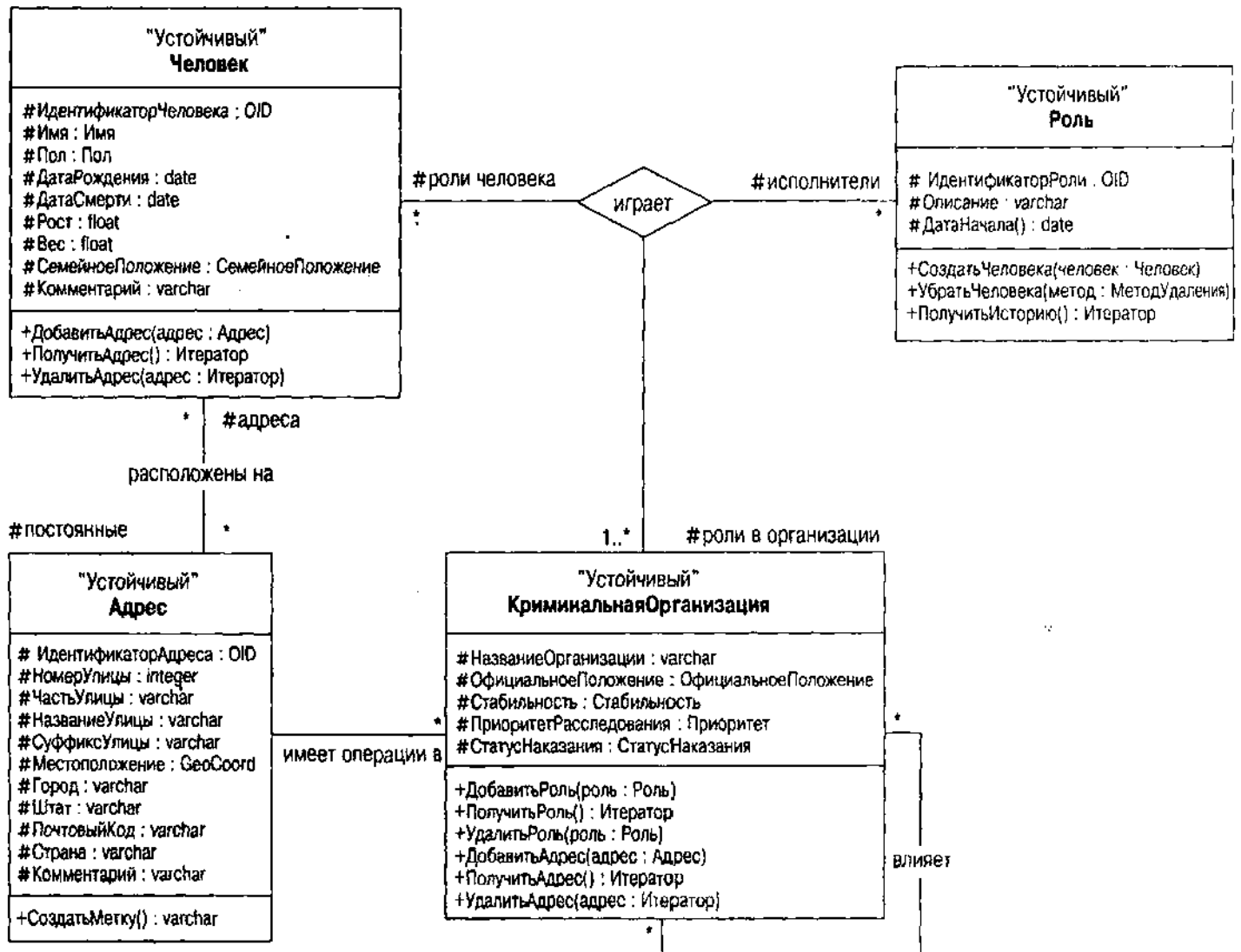


Рис. 7.10. Нотация ассоциации UML

Видимость важна потому, что важна инкапсуляция. При ООП желательно ограничить видимость объекта как можно больше. Ограничение видимости через инкапсуляцию — лучший способ уменьшить связность в системе, что ведет к созданию более простого в поддержке и надежного программного обеспечения. В базе данных видимость (и инкапсуляция) всегда были проблемой. Имея более новые продукты OODBMS, можно гораздо лучше контролировать видимость; у большинства продуктов RDBMS способность контролировать видимость в лучшем случае средняя.

Перед тем как перейти к подробностям, рассмотрим некоторую нотацию. На рис. 7.10 показана базовая нотация ассоциации из UML. Имеются два вида ассоциации: двоичная и прочая. Отношения между Человеком и Адресом, например, — это *двоичная ассоциация*, ассоциация между двумя классами. Ассоциация "влияет" между КриминальнойОрганизацией и ею самой — это *рекурсивная двоичная ассоциация*. Ассоциация "играет" между КриминальнойОрганизацией, Ролью и Человеком — *троичная ассоциация* (три класса, относящихся друг к другу, а не два). Двоичная ассоциация —

это линия, соединяющая два класса в то время, как троичная ассоциация и ассоциации более высокого порядка являются рядом линий, связывающих различные классы с помощью центрального ромба, представляющего ассоциацию. Можно иметь столько участников ассоциации, сколько нужно, хотя (как упоминалось в главе 6 для ER-диаграмм) в действительности имеется совсем немного ассоциаций выше порядком, чем троичная. Исключение — схема звезды для хранилищ данных, которые инвертируют обычную разработку в массивную ассоциацию между серией размерностных объектов с центральным объектом фактов, являющаяся ассоциацией с множеством размерностей.

Имена ассоциаций — это обычно глагольные фразы, в точности такие, как имена ER-отношений. Есть всего несколько ассоциаций, которые можно назвать как-то иначе, чем "имеет". Просто не заботьтесь об этих именах и считайте, что ассоциация без имени означает, что один класс "имеет" экземпляры другого класса.

Как и в случае ER-диаграмм, любая сторона ассоциации является ролью, играемой классом в отношении. Любая роль имеет необязательное имя, множественность и необязательную способность к навигации. Можно дополнительно построить отношение агрегации, показывающее владение, и включить квалификатор, чтобы показать несколько отношений.

Роли и названия ролей

Роль — это одна из сторон ассоциации. Двоичная ассоциация имеет две роли, третичная — три и т.д. Можно назвать роль для пояснения природы ассоциации, хотя это делать не обязательно. На рис. 7.10, например, показаны названия ролей в двоичной ассоциации между Человеком и Адресом и в третичной ассоциации между Ролью, КриминальнойОрганизацией и Человеком.

В отличие от ролей на ER-диаграмме, обычно являющихся глагольными фразами, большинство ролей в UML — словосочетания существительных, соответствующих субъектам и объектам того факта, который представляет ассоциация. Например, с точки зрения Человека, человек располагается в некотором множестве адресов. С точки зрения адреса — ряд постоянных жителей живет по определенному адресу. Глагольная фраза — название ассоциации, в то время как словосочетания из существительных для субъекта и объекта являются ролями.

Названия ролей переводятся непосредственно в имена переменных в реализации класса. Если название роли вызывает затруднение, подумайте об имени, которое можно было бы дать атрибуту или методу, реализующему роль в классе. Например, в классе Человек имеется ряд методов для работы с адресами. Метод ПолучитьАдрес получает итератор для коллекции адресов человека. ДобавитьАдрес добавляет адрес к набору адресов в то время как УдалитьАдрес избавляется от одного адреса, определенного итератором. Таким образом, "адреса" представляют то имя, которым назван набор адресов. Обычно оно становится именем внутренней переменной, представляющей набор (экземпляр STL `set<>` в C++, например, или отношение ODL `set<>`).

ВНИМАНИЕ

Рис. 7.10 не демонстрирует подобные методы для Адреса. Почему? В требованиях нет пункта, говорящего о необходимости обеспечить доступ к людям по их адресам, хотя это было бы разумно. Класс сам по себе может требовать итераций по набору постоянных жителей, но пользователи класса — нет. Таким образом, имеется название роли, но нет соответствующих операций. Вывод — операции и роли отличаются: операции упаковывают роли, а не реализуют их. Это различие является фундаментальным для инкапсуляции ассоциаций внутри класса и ограничивает видимость только необходимым.

Можно также определить видимость для роли (общедоступная, защищенная или скрытая, с помощью тех же символов, что и для атрибутов и операций). Это позволяет завершить спецификацию переменной ассоциации в реализации ОО-систем, которая некоторым образом поддерживают управление доступом.

Следует понимать, однако, что не все отношения равнозначны. Например, роль Человек в третичном отношении на рис. 7.10 довольно сложно назвать. Автор выбрал "исполнители" в качестве названия этой роли. Включенные сюда объекты — люди и организации, в которых они играют роли, очень абстрактны. С точки зрения роли имеется очень небольшой смысл. Она действительно поддерживает такие запросы, как "Предоставьте мне имена и адреса всех боссов в криминальной организации Лондона". Человек и организация критичны, в то время как роль просто квалифицирует отношение.

Имена ролей очень полезны при моделировании данных, потому что они транслируются в имена, которые логическая схема дает столбцам, коллекциям или другим логическим артефактам, представляющим ассоциации при создании схемы. Если выбраны значимые названия ролей, то схема также обретает смысл.

Множественности и упорядочение

Множественность — это свойство роли, представляющее бизнес-правило или ограничение количества объектов, участвующих в ассоциации с точки зрения роли. Повторение из главы 6 — ER-диаграммы видят множественность как квалификаторы один-к-одному, один-ко-многим или многие-ко-многим в ассоциации. "Один" и "многие" — множественности роли. UML идет дальше в этой области, чем ER-модели.

Множественность любой роли состоит из спецификации целых значений, обычно это диапазон таких значений. Вот некоторые варианты:

- 0..*: Ноль или больше объектов
- 0..1: Не более одного необязательного объекта
- 1..*: По крайней мере, один объект
- 1: Ровно один объект
- *: Ноль или больше объектов
- 2..6: По крайней мере, два, но не более шести объектов
- 1, 3, 5-7: По крайней мере, один объект, но возможно три, пять или семь объектов

Звездочки "*" представляют неограниченную верхнюю границу. Диапазон является включающим в себя; т. е. диапазон включает значения, разделенные при помощи индикатора диапазона "..". Многочисленные "*" и "1" наиболее часто встречаются с "1..*" для обязательных ко-многим и "0..1" для необязательных к-одному, идущим сразу после.

Можно также определить, что объекты, связанные с множественностью больше одного (ко-многим), являются в некотором смысле использующими свойство роли упорядоченно. Эта квалификация означает, что нужно использовать некоторый вид упорядоченного представления в логической схеме. В реляционной схеме это преобразуется в упорядоченный атрибут соответствующей таблицы. В схемах ОР и ОО это преобразуется в использование упорядоченного отношения коллекций, такого, как вектор, список, стек или очередь. Если бы он был не упорядочен, это могло бы быть множеством или неупорядоченной совокупностью. Множественность критична для модели данных, поскольку она преобразуется прямо в структуру бизнес-ограничений внешних ключей в логической схеме (см. главу 6 с подробностями о контексте ER-диаграммы). Если определена роль 0..*, это преобразуется в атрибут нулевого внешнего ключа ко-многим в реляционной базе данных, например, или в отношение set<> с необязательным членством в ODL.

Навигация и симметричная видимость

Свойство навигации для роли позволяет определить отношения видимости между классами. Способность к навигации — это способность перемещаться к классу, к которому относится роль, из других классов в ассоциации. Таким образом, ассоциация становится направленной ассоциацией. По умолчанию роли на диаграмме классов UML ненаправленные, т. е. имеется полный навигационный доступ к любому классу, участвующему в ассоциации. Добавление стрелки к ассоциации отменяет это, определяя навигацию только в указанном направлении.

Это вводит *асимметричную* видимость, способность одного класса в отношении видеть другие, которые в свою очередь не могут сделать обратное. Разумное использование способности к навигации может сократить связность в модели данных до необходимого для удовлетворения требованиям уровня. Однако при этом налагаются ограничения на возможность повторного использования модели, поскольку явно говорится о том, что нельзя видеть определенные вещи, даже если это имеет смысл в новой ситуации. Умеренность в применении способности к навигации может улучшить проект; чрезмерность — к чему-то, что очень трудно использовать.

При моделировании данных способность к навигации позволяет иметь односторонние отношения. При переводе этого в концептуальную схему способность к навигации очень полезна для ОР- и ОО-схем, поддерживающих одноподнаправленные отношения, как часто и бывает. Значит, создается отношение, существующее в одном классе, но другой класс не может видеть связанные объекты в первом классе. В ODL это соответствует, например, отношению, не имеющему обратного пересекающегося пути.

Агрегация, композиция и принадлежность

Агрегация — это отношение части и целого между двумя классами. UML позволяет определить два вида агрегации: слабую, или совместную, агрегацию и сильную, или составную, агрегацию. *Совместная агрегация* позволяет моделировать отношение части и целого, в котором объект обладает другим объектом, но в то же время другие объекты также могут обладать этим объектом. *Композиция* позволяет моделировать отношение части и целого, когда один объект обладает исключительным правом владеть другим объектом.

ВНИМАНИЕ

Агрегация композиции соответствует непосредственно концепции ЕЯ-моделирования слабого отношения (чтобы полностью запутаться в терминологии).

UML представляет агрегацию как ромб в конце линии ассоциации, которая заканчивается в классе владельца. Совместная агрегация — пустой ромб, в то время как роль составной агрегации — ромб, закрашенный черным. На рис. 7.11 показаны два отношения агрегации. Первое описывает совместное отношение агрегации между фотографическим образом и человеком, второе показывает роль составной агрегации между человеком и идентификацией.

Совместная агрегация описывает ситуацию, при которой вещь агрегирует некоторую другую вещь. В данном случае фотография агрегирует набор людей. Но, кроме того, каждый человек может появляться более чем на одной фотографии. Множественности "*" делают это явно. При заданной семантике не существует принадлежности: фотография не владеет человеком, а человек не исчезает (по крайней мере, из базы данных), когда уничтожается фотография.

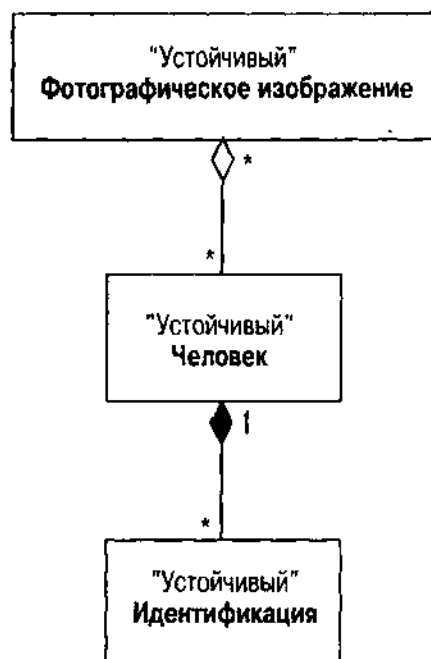


Рис. 7.11. Агрегации UML

Составная агрегация, с другой стороны, является довольно явной относительно принадлежности. Человек идентифицируется по одному или более идентификационным документам и владеет этими документами. Если человек исчезает, документы тоже исчезают. Таким образом, идентификационные документы идентифицируются с помощью их отношения к человеку.

Только одна роль в ассоциации может иметь маркер агрегации. По определению только один объект может "владеть" другим, n-ная ассоциация более чем с двумя ассоциированными классами не может иметь никакой агрегации [Rational Software, 1997b, p. 27]. Составная агрегация, по определению, должна иметь множественность 1, 1..1 или 0..1 для роли, соответствующей агрегированному владельцу.

Составная агрегация критически важна для моделирования данных потому, что она представляет собой слабое отношение. Переведенная в концептуальную схему реляционной или объектно-реляционной БД составная агрегатная ассоциация становится внешним ключом, являющимся частью первичного ключа зависимого объекта (в данном случае — идентификационного документа). Даже в объектно-ориентированных БД имеется импликация каскадных удалений: как только удаляется агрегация, все агрегированные части также исчезают. Это в свою очередь переводится в каскадное удаление спецификации или триггера в таблице концептуальной схемы.

Составная ассоциация

Составная ассоциация — это ассоциация с *квалификатором*: "атрибут или список атрибутов, чьи значения служат для разделения множества объектов, ассоциированных с объектом через ассоциацию. Квалификаторы являются атрибутами ассоциации" [Rational Software, 1997a, p. 58]. На рис. 7.12 показан квалификатор ассоциации между Человеком и Идентификацией. Квалификатор транслируется в ассоциативный массив или отображение, коллекцию, индексированную по некоторому атрибуту. В данном случае атрибут IDType (ТипИдентификатора) идентифицирует различные виды идентификации и множественности ассоциации указывают, что человек может иметь любое количество идентификаторов. Помещение квалификатора в эту ассоциацию означает, что можно иметь только один идентификатор каждого типа — в данном случае это разумное бизнес-правило.

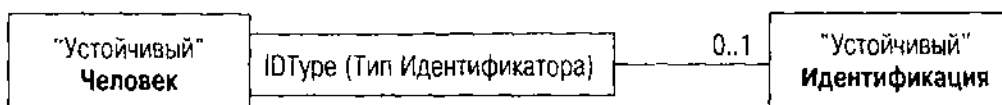


Рис. 7.12. Ассоциация UML с квалификатором

Квалификатор относительно редок в моделировании данных, но можно использовать его для получения преимущества, когда имеется этот особый вид ограничения. В реляционной модели данных концепция переводится в таблицу, содержащую атрибут квалификатора и остальные атрибуты объектов с квалификатором, являющимся частью первичного ключа таблицы. Атрибут IDType появится в таблице Идентификации, где он также служит

дискриминатором для различных подклассов Идентификации. В случае ОО-модели данных можно использовать отображение, словарь или подобные структуры данных для ассоциативного доступа, чтобы представить коллекцию. Человек будет иметь атрибут, определенный как `map<IDType, Identification>`, где первый параметр шаблона тип перечисления, содержащий все различные типы Идентификаторов, а второй — тип Идентификации.

Классы ассоциации

Ассоциации обычно настолько просты, насколько им позволяют быть роли, множественности, агрегация и квалификаторы. Некоторые ассоциации, однако, содержат сведения не только об относящихся классах, но также и о самой ассоциации. Чтобы добавить атрибуты к ассоциации, создается *класс ассоциации*, класс, который действует как ассоциация, но имеет свои собственные атрибуты и операции.

UML представляет класс ассоциации как прямоугольник класса, присоединенный к ассоциации при помощи пунктирной линии. Класс имеет имя ассоциации. На рис. 7.13 показан класс ассоциации, представляющий специальные свойства третичного отношения между Человеком, Ролью и КриминальнойОрганизацией. Этот класс имеет несколько атрибутов: пребывание

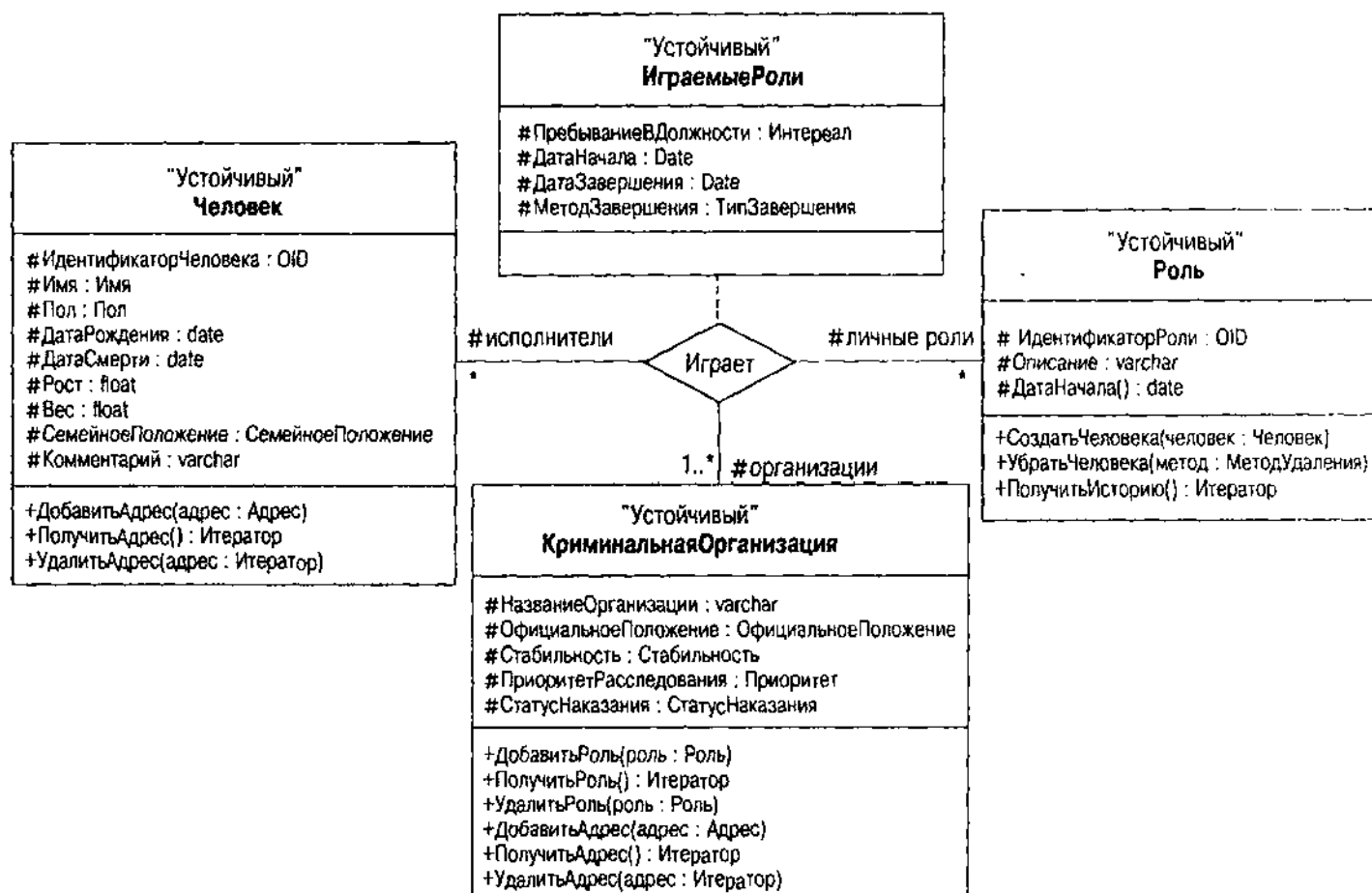


Рис. 7.13. Класс ассоциации UML

в должности (продолжительность пребывания в роли), дата начала, дата окончания и метод завершения (для тех, кто больше не играет роли — повышение, понижение, удаление различными ужасными методами, такими, какие применили к Порлоку в организации Мориарти).

При помощи тщательного определения классов ассоциации можно моделировать отношения как полноценные объекты в проекте концептуальной схемы. Классы ассоциаций преобразуются прямо в классы концептуальной схемы (таблицы в реляционной БД).

Генерализация и ассоциации предоставляют множество ограничений, которых требует модель данных для создания полной концептуальной схемы. Все же имеется несколько ограничений, добавляемых к модели, включая ключи, домены и более сложные ограничения.

Ограничения объектов и бизнес-правила

Ассоциации позволяют ограничить отношения между классами и объектами с помощью ссылочной целостности, и эти ограничения образуют основную часть концептуальной модели данных. Остальные ограничения — первичные ключи, домены и сложные ограничения — завершают модель данных. Если это еще не сделано, пожалуйста, вернитесь к разделу "ER-бизнес-правила" в главе 6. Он расширяет представление, налагая различные ограничения на контекст диаграмм классов UML.

Идентичность объектов и ограничения уникальности

Любая модель БД должна содержать способ идентификации каждого уникального объекта в базе данных. Идентификация Холмса для индивидуально оружия, использованная на мосту Тор, демонстрирует значение индивидуальности.

"Несомненно, она обвинила невинную леди во всех этих грубых сделках и недобрых словах, с которыми ее муж пытался отвергнуть ее слишком демонстративную привязанность. Ее первым решением было покончить с жизнью. Вторым — сделать это таким образом, чтобы ее жертву постигла участь, которая была бы гораздо хуже, чем может быть любая внезапная смерть.

Мы можем совершенно ясно проследить разнообразные шаги, и они показывают удивительную тонкость ума. Очень умно было привлечено внимание мисс Дунбар, что показывает, будто место преступления было выбрано не случайно. Горя желанием, чтобы оно было обнаружено, она некоторым образом переиграла, держа все нити у себя в руках до самого конца. Только одно это должно было бы возбудить мои подозрения раньше, чем это произошло.

Затем она взяла один из револьверов своего мужа — в доме был, как вы видели, целый арсенал — и сохранила его для себя. Подобный револьвер она подбросила тем утром в гардероб мисс Дунбар после разрядки одного ствола, что она легко могла сделать в лесу, не привлекая внимания. Затем она вернулась к мосту, где изобрела этот исключительно

остроумный способ отделаться от оружия. Когда появилась мисс Дунбар, она вылила на нее всю свою ненависть и затем, когда уже не могла ничего слышать, выполнила свое ужасное намерение. Все связи сейчас на месте и цепь замкнулась. Газеты могут спросить, почему истина не всплыла раньше, но легко быть мудрым после того, что произошло, и в любом случае просторы затянутого ряской озера не легко просмотреть, если нет ясного понимания того, что нужно искать и где". [THOR]

Большинство систем БД предоставляют уникальные, оригинальные и спрятанные идентификаторы строк и объектов (ROWID в Oracle, например). Менеджеры объектно-ориентированных БД предоставляют уникальность с помощью *идентификатора объекта*, или OID. Идентификация объекта с помощью такого суррогата является *неявной* или *основанной на наличии* идентификаций [Blaha and Premerlani, 1998, p. 193–194]. Альтернативой является *явная* или *основанная на значении* идентификация. С помощью явной идентификации можно идентифицировать объект при помощи значений одного или более атрибутов объекта. Идентификация объектов соответствует концепции первичного ключа в ER-моделировании. Автор использует термин "OID" для ссылки на любой тип ключа, который уникальным образом идентифицирует объект.

Повторим определения ключей в терминах объектов: *потенциальный ключ* — это набор неявных и явных атрибутов, которые уникальным образом определяют объект класса. Набор атрибутов должен быть *завершенным* в том смысле, чтобы нельзя было удалить любой атрибут и все еще сохранить уникальность объектов. Кроме того, никакой потенциальный ключ не может быть нулевым, поскольку это логически делает недействительным сравнение по равенству, которое нужно выполнить, чтобы убедиться в уникальности. *Первичный ключ* — это потенциальный ключ, который выбирается для идентификации класса, OID. *Альтернативный ключ* — потенциальный ключ, не являющийся первичным ключом. Альтернативный ключ, таким образом, представляет неявное или явное ограничение уникальности.

ВНИМАНИЕ

В UML объекты имеют идентификацию, а значения данных — нет. Значения данных — это значения примитивных типов, таких, как `int` или `VARCHAR`, которые не допускают идентификации базового объекта. Объекты — это сущности классов. См. раздел "Ограничения доменов", где полностью рассматриваются значения данных и значения объектов.

UML не предоставляет никаких стандартных средств выражения идентификации объектов, он предполагает, что идентификация объектов — основное, неявное и автоматическое свойство объекта. Чтобы включить эту концепцию в моделирование данных UML, нужно, следовательно, расширить UML некоторыми дополнительными свойствами атрибутов. Определяются явные ключи в диаграмме классов UML с двумя расширенными помеченными значениями OID и альтернативный OID. Здесь также не существует стандартной нотации UML; они являются специальными расширениями UML для моделирования данных.

Два эти свойства идентифицируют атрибуты, служащие для идентификации уникального экземпляра данного класса. С помощью присоединения помеченного значения `OID` к атрибуту определяют атрибут как часть первичного ключа или идентификатора объекта. С помощью присоединения помеченного значения {альтернативный `OID`} = `n` к атрибуту его определяют как часть альтернативного ключа `n`, где `n` — целое. Свойства {`OID`} и {альтернативный `OID`} ограничивают объекты класса вне зависимости от памяти. Они соответствуют ограничению SQL PRIMARY KEY и ограничению UNIQUE соответственно. Если, например, создается объектно-реляционный тип и две таблицы из этого типа, свойство {`OID`} на атрибутах заставляет все объекты в обеих таблицах иметь уникальные значения относительно друг друга.

ВНИМАНИЕ

Свойство {`OID`} определяется только на атрибутах первичных ключей регулярных создаваемых классов, а не на ассоциированных классах. `OID` класса ассоциации всегда неявный и состоит из комбинированных `OID`-атрибутов классов, которые участвуют в отношении.

На рис. 7.14 — `OID` классов Человека и Идентификации. Класс Человек — законченный объект и имеет сильную идентификацию объекта. Если нужно сделать идентификацию явной, обычно добавляют атрибут подходящего типа, который служит идентификатором объекта, в данном случае атрибут ИдентификаторЧеловека: `OID` {`OID`}.

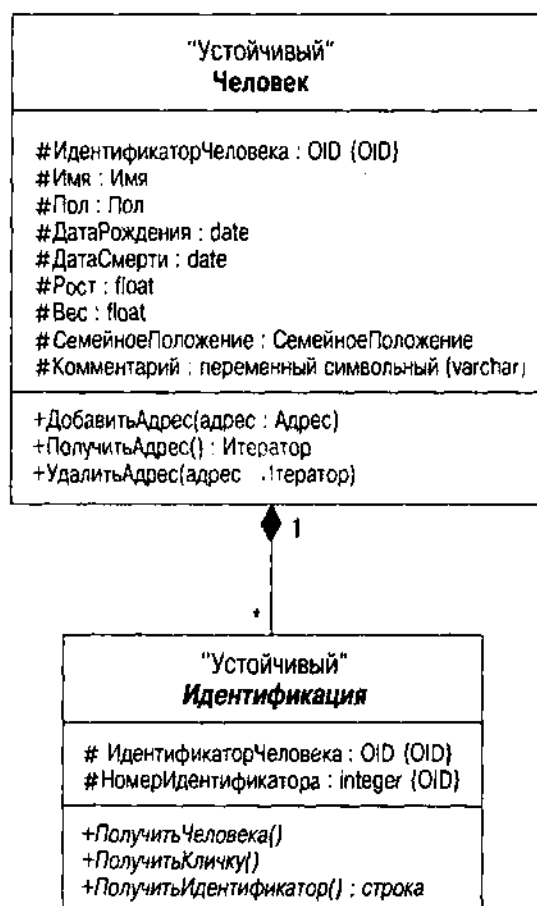


Рис. 7.14. Явное помеченное значение {`OID`}

В реляционных и объектно-реляционных системах это соответствует генерируемому идентификатору, такому, как последовательность в Oracle или IDENTITY в SQL Server. Однако будьте осторожны: если создается OID на базе таблицы, он действителен в пределах таблицы, а не класса. Значит, если определяются несколько таблиц из класса, OID не будет уникальным образом идентифицировать объекты обеих таблиц, а только внутри каждой таблицы. В большинстве систем OODBMS объекты имеют встроенные OID, к которым можно получать доступ и использовать для установления идентичности. Обычно целостность ссылок обеспечивается автоматически благодаря способу соотнесения объектов друг с другом и системы используют OID для представления отношений. Кодд описал эту проблему и ее требования для "завершенных" продуктов RDBMS. Он определил, что реляционные системы должны вести себя подобным же образом, рассматривая первичный ключ как потенциально используемый несколькими таблицами [Codd, 1990, р. 25–26, 36–37]. В частности, атрибуты используют общий составной домен, из которого все участвующие таблицы извлекают свои значения. Определение этого в терминах доменов позволяет сохранить строгое типизирование в операциях объединения, а также в выражениях сравнения [Codd, 1990, р. 49].

При построении иерархии генерализации определяют свойство {OID} на вершине иерархии. Подклассы корневого класса наследуют явный идентификатор объекта из этого класса. Можно всегда добавить спецификации {альтернативного OID} ниже в иерархии генерализации. Множественное наследование, конечно, в этой схеме напоминает обезьяньи гримасы, точно так же, как это происходит со структурой объекта. Если класс наследует атрибуты {OID} более чем от одного родителя, нужно переопределить свойство в наследующем классе, чтобы определить тот или иной набор атрибутов как {OID} нескольких наследующих классов. Можно также определить совершенно новое свойство {OID}, если это имеет смысл.

СОВЕТУЕМ

Трудно осмыслить множественное наследование. И это еще одна причина избегать его при моделировании данных. Так или иначе, это осложнит жизнь разработчика. Попробуйте обойтись без этого. Если не удастся, сделайте OID неявным и позвольте системе обрабатывать идентификацию через существование объектов, а не с помощью наследования. Иначе вам можно только посочувствовать.

Аналогично класс, связанный с другим через составную агрегацию (слабое отношение в терминах ER), получает OID от агрегирующего класса. Однако это не образует полного OID для агрегированного класса; нужно определить любые дополнительные атрибуты, требуемые для идентификации индивидуального элемента внутри агрегата. Например, на рис. 7.14 OID класса Идентификации – это комбинация OID Человека агрегирующего человека и идентификационного номера. Такое использование не совсем обычно для UML.

В ООП, а следовательно и в UML, каждый объект имеет неявную идентификацию. Во временных системах идентификацией часто служит адрес в памяти объекта. В устойчивых системах OODBMS каждый объект имеет

OID, который зачастую содержит данные физической или логической памяти. В реляционных продуктах и ORDBMS обычно имеется подобный способ идентифицировать каждую строку. Ни один из идентификаторов не являются явной частью проекта концептуальной схемы или класса, это происходит автоматически. Вот почему по большей части нет стандартного способа ссылки на OID в диаграммах UML; другие логические структуры, в частности отношения, классы и объекты, полностью соответствуют концепции.

Какой подход лучше — явный или неявный? С пуристской точки зрения ОО-разработки нужно оставить только явные ОО-атрибуты. На рис 7.15 показано, как рис. 7.14 выглядел бы с использованием этого подхода. Имея такую диаграмму при создании схемы реляционной БД, добавляют столбец, который уникальным образом идентифицирует каждую строку в таблице Человек, соответствующей объекту Человек. Отношение агрегации между двумя классами подразумевает, что имеется внешний ключ в Идентификации, указывающий на единственного человека. Агрегация подразумевает, что внешний ключ также является частью первичного ключа. Вместо создания уникального столбца OID для Идентификации, следовательно, создаются два столбца. Первый — OID Человека, который имеет обратную ссылку на таблицу человека как внешний ключ. Можно проследить это с помощью уникального числа для каждой строки с одинаковым OID Человека. Можно определить атрибут для использования как второй элемент OID с помощью другого помеченного значения {агрегативного OID}. Это помечает атрибут как элемент для добавления к OID агрегирующего объекта,

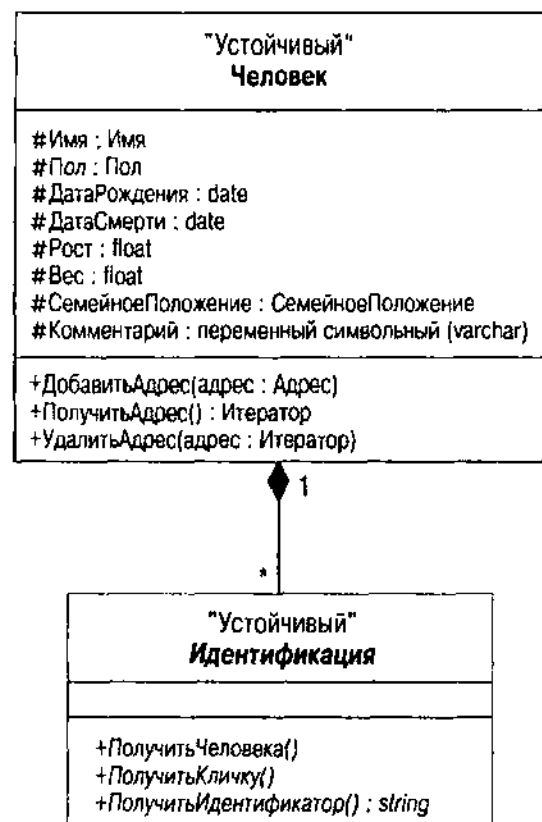


Рис. 7.15. Подход с неявным OID

чтобы создать OID нового объекта. При желании заменить неявный OID явным набором атрибутов нужно просто определить помеченное значение {OID} на этих атрибутах. В базовой концептуальной схеме не будет никакого неявного OID.

Второй подход делает все это явным на уровне проектирования, в основном так, как показано на рис. 7.14. Явный подход имеет то преимущество, что все атрибуты ясны и открыты в проекте. Это делает связь между моделью данных и концептуальной схемой в целевой модели данных более непосредственной. Поскольку объекты являются устойчивыми, обычно бывает нужно иметь методы классов, которые управляют OID по пути из устойчивой памяти во временную память. Продукты OODBMS управляют этим автоматически, а продукты RDBMS и ORDBMS — обычно нет. Превращение OID в явный идентификатор позволяет разработать OID, управляемый постоянными классами. Неявный подход, с другой стороны, прячет подробности и позволяет выполнить работу при преобразовании модели данных в концептуальную схему в целевой базе данных. Пока имеется стандартное управление OID, встроенное в иерархию постоянного класса некоторым образом, это прекрасно работает. В реляционных базах данных требуются значительные дополнительные усилия.

Можно иметь явный OID, охватывающий два или более атрибутов класса, соответствующий составному ER-ключу из главы 6. При объединении наборов объектов (таблиц или чего-то еще) по первичному ключу, если этот ключ имеет более одного атрибута, синтаксис для условия объединения становится слишком путаным, особенно для внешних объединений. Нужно одно выражение сравнения для каждого атрибута в объединении. Можно значительно упростить кодирование, заменив явные OID с множеством атрибутов (составные ключи) созданными разработчиком явными OID с одним атрибутом. Такое использование лежит где-то посередине между явным и неявным OID. Оно неявно потому, что зависит от существования. Оно явно потому, что должно быть действительным атрибутом класса, поскольку создавалось для явного использования.

И возникает еще одна проблема по мере роста размера модели данных. Для больших БД может оказаться, что однородность при выборе подхода идентификации увеличивает производительность. Однородность в таком случае означает неявный подход. При значительном числе таблиц программисты могут легко запутаться в изобилии атрибутов различных OID, распределенных по базе данных. С помощью неявных OID и стандартного соглашения об именах OID для устойчивых атрибутов в концептуальной схеме программисты смогут кодировать, не боясь перепутать, какие атрибуты что идентифицируют.

Ограничения доменов

В разделе "Домены" главы 6 домен атрибута определен как набор значений, на которые отображается атрибут. Имеется неограниченное количество потенциальных наборов возможных значений. Этот раздел классифицирует домены, чтобы предоставить структуру для обсуждения, не путаясь в деталях. Конечно, немного философии не повредит.

ВНИМАНИЕ

В книге Кодда по RM/V2 [Codd, 1990, p. 43–59] подробно рассматриваются домены в контексте расширенной реляционной модели, а в манифесте Дейта и Дарвена [Date and Darwen, 1998] — отношения между доменами и другими аспектами объектно-реляционной системы. Блага более подробно разбирает ОО-концепцию доменов с несколько другой точки зрения, чем приведенная здесь классификация [Blaha and Premerlani, 1998, p. 45–46].

В действительности имеется только два вида доменов: экстенциональные и интенциональные. Эти термины происходят из математической логики и денотационной семантики. Не путайте домен атрибута с совершенно отдельным использованием "домена" для ссылки на специфическую область бизнеса, к которой относится система программного обеспечения (или же к домену сети NT). Определение "доменная модель" в ОО-проекте ссылается на настройку классов для бизнес-области, не затрагивая основные типы данных системы. Не путайте определение типа с определением множества значений или типизированных объектов, т. е. не путайте определение целого с набором целых значений. Определение типа говорит обо всех возможных значениях данного типа, в то время как определение набора значений говорит о действительных значениях в конкретном контексте, таком, как база данных или таблица.

Интенция множества — это логический предикат (комбинация отношений и операций на этих отношениях таких, как AND, OR, NOT, FOR ALL и FOR ANY), определяющий содержание этого множества. Например, можно определить множество всех криминальных организаций как "множество всех организаций, созданных с целью вовлечения в криминальный бизнес". Это определение близко к тому, которое дано в Акте об организациях, находящихся под влиянием рэкетиоров, и коррумпированных организациях (RICO), определяющий подход федерального правительства США к организованной преступности. Если множество определяется по его интенции, ставится условие, которое можно применить к объекту, чтобы понять, является ли он членом множества.

Экстенция множества — коллекция объектов, принадлежащих множеству, *элементы* множества. Если множество определено с помощью интенции, объекты, удовлетворяющие интенционному определению, образуют экстенцию множества. Опять ссылаясь на криминальные организации, действительные криминальные организации, — это элементы множества криминальных организаций. Используя интенционное определение, только те организации, которые удовлетворяют условию, являются частью экстенции множества. Если множество определено через понятие экстенции, то оно определяется путем составления списка членов. Например, набор арабских цифр определяется как {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}.

Можно легко определить некоторые домены с помощью интенционального определения, например криминальная организация. "Простая" часть — определение, а не приложение. Например, правительство США судило Ангелов Ада в Калифорнийском федеральном суде как криминальную организацию по статье RICO. Но суд решил, что они были не криминальной организацией, а просто клубом мотоциклистов, и лишь отдельные его

члены продавали нелегально наркотики. Некоторые домены, такие, как набор целых, набор цветов или набор изображений JPEG, могут иметь только интенциональное определение. Буквально нет способа составить список элементов, поскольку их количество бесконечно. На практике, конечно, внутренняя схема представления чисел компьютера ограничивает количество целых. Цвет на самом деле — это все возможные комбинации трех 32-битовых целых. Количество изображений должно быть в действительности конечным, поскольку существует только конечное число атомов во вселенной, служащих битами. Все же реальность не должна слишком ограничивать разработчиков. В конце концов, кто сможет использовать более 64 Кб памяти?

Можно определить другие домены лучше всего с помощью экстенсионного определения, такого, как краткий список возможных значений. Например, возможные состояния официального статуса криминальной организации включают "официально определенный", "под судом", "утвержденный" или "неизвестный". Это *обычная шкала*, набор значений без внутреннего упорядочения или оператора сравнения, отличного от равенства [Fenton and Pfleeger, 1997]. С его помощью определяют тип перечисления в самых современных языках программирования.

ВНИМАНИЕ

Интересно, что тип перечисления C++ и ODL не полностью соответствуют определению. Тип обозначен как целый и, следовательно, имеет внутреннюю упорядоченность. Многие программисты C и C++ поступают таким образом, назначая целые и используя преобразования для получения настоящих целых значений, когда они им нужны, обычно для сравнений или сортировки. Этот вид программирования нарушает, к сожалению, математические предположения о номинальной шкале и может иметь вводящие в заблуждение результаты, особенно в статистических приложениях. Попробуйте избегать этого при программировании БД, в частности из-за неявных ограничений данных в домене. Это применимо по большей части только к ОО-системам баз данных, основанных на C++.

Домен может быть *простым* (множество целых чисел, или множество допустимых дат, или множество изображений JPEG), *структурированным* (тип записи struct или какой-то другой), *многозначным* (массив или коллекция объектов некоторого домена), *экстенсиональным* (список перечисляющий возможные значений) или *интенциональным* (логический предикат, определяющий набор возможных значений в качестве расширения).

ВНИМАНИЕ

В книге недостаточно места для углубления в теоретические подробности доменов. Если есть желание понять математику, лежащую в основе доменов, обратитесь к стандартным работам по теории измерений [Fenton and Pfleeger, 1997; Roberts, 1979]. В частности, обратите внимание на организацию данных, особенно сравнимость и другие операции. Если смешать или преобразовать данные между несовместимыми типами, невозможно определить значение результатов операций над ними.

Простые домены соответствуют стандартным типам данных с единственным значением. В SQL это означает точные или приблизительные числа, строки символов в различных формах, даты, время, метки времени и интервалы [ANSI, 1992]. Используя ограничения CHECK, можно определить ограничения на простых доменах для создания новых доменов. Нельзя определить их отдельно и специально как домены; нужно написать ограничение CHECK на каждом атрибуте. В некоторых продуктах ORDBMS, в частности DB2, можно определить подтипы, применяющие такие ограничения [Chamberlin, 1998].

В OQL больше вариантов для выбора: float, integer, character, string, Boolean, octet, date, time, timestamp и interval [Cattell and Barry, 1997]. Есть также тип "any" (любой), предоставляющий возможность ссылаться на любой объект. Продукты ORDBMS позволяют добавлять простые типы с помощью расширений системы типов.

Структурированные домены включают разнообразные формы записи, совокупность атрибутов, приходящих из разных доменов. Многозначные домены включают массивы и другие способы представления совокупности объектов, приходящих из одного и того же домена.

Экстенциональные домены определяют домен с помощью простого списка значений, что также называют типом перечисления. В SQL это делается с помощью ограничения CHECK, обычно используя выражение IN со списком элементов: CHECK ОфициальныйСтатус IN ("Официально Определенный", "Под Судом", "Утвержденный", "Неизвестный"). В SQL это будут ограничения целостности на отдельные атрибуты, а не на отдельные домены. Это означает, например, что нужно поместить ограничение CHECK на все атрибуты, которые должны соответствовать списку значений. В ODL явный тип перечисления enum позволяет определить домен и затем применить его к нескольким свойствам [Cattell and Barry, 1997].

UML не определяет никакой конкретной доменной модели. Наоборот — спецификация UML неявно делает тип данных "определенным реализацией", позволяя определить доменную модель на основе используемого языка программирования. UML определяет семантику типа данных следующим образом:

Тип данных — это тип, чьи значения не имеют идентификации, т. е. они являются просто значениями. Типы данных включают примитивные встроенные типы (такие, как целые и строки), а также определяемые перечислимые типы (такие, как предопределенный перечислимый тип boolean с литералами false и true). [Rational Software, 1997b, p. 22]. Имеются три подкласса классификатора ТипДанных в метамодели UML: Примитивный, Структурный и Перечисление. Это стереотипы, которые можно использовать для оценки классификатора (прямоугольник класса), задаваемого для представления типа и его операций.

Для моделирования данных нужно тщательно определить множество типов данных для моделей данных как отдельный пакет классификаторов. Рис. 7.16 иллюстрирует пакет типов данных на основе модели типа ODL (и OMG).

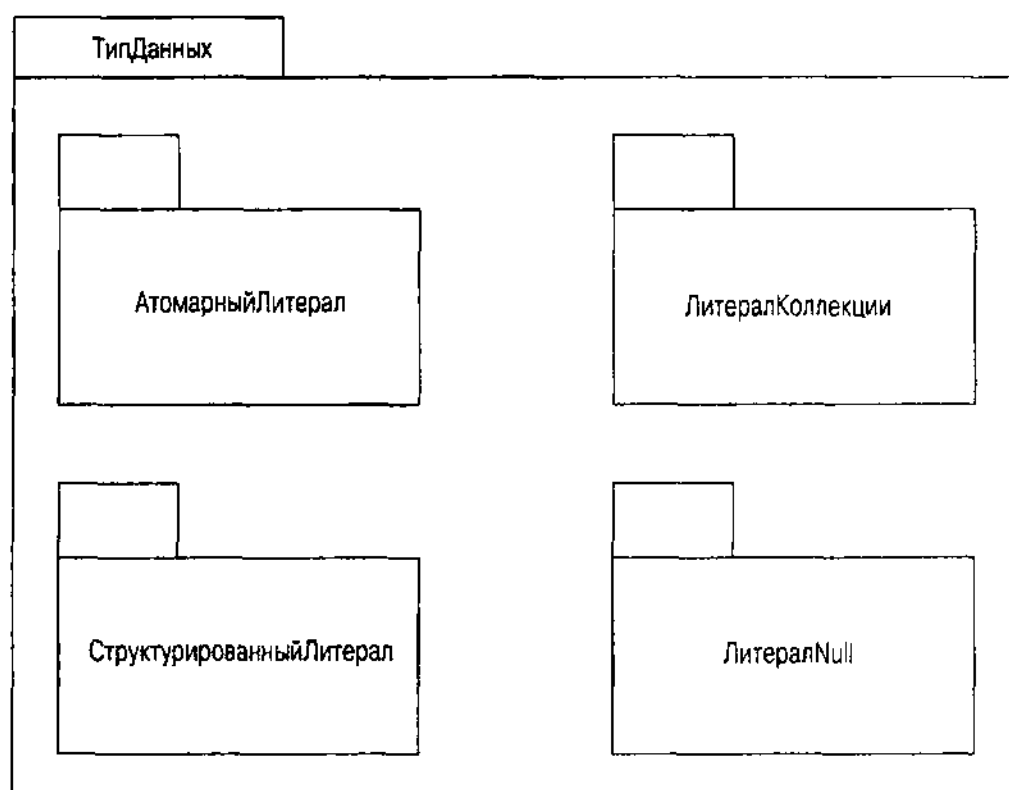


Рис. 7.16. Модель типа ODL как пакет типов данных UML

Рисунки с 7.17 по 7.19 демонстрируют детали отдельных пакетов (АтомарныйЛитерал, ЛитералКоллекции и СтруктурированныйЛитерал). Объектная модель OMG не определяет стандартных операций на этих типах, хотя модель типов данных SQL — определяет (например, для числовых типов операции $+$, $-$, $*$ и $/$). Можно добавлять операции, как это показано на рис. 7.17 для типа Элементарного Литерала. Когда этот пакет модели типов

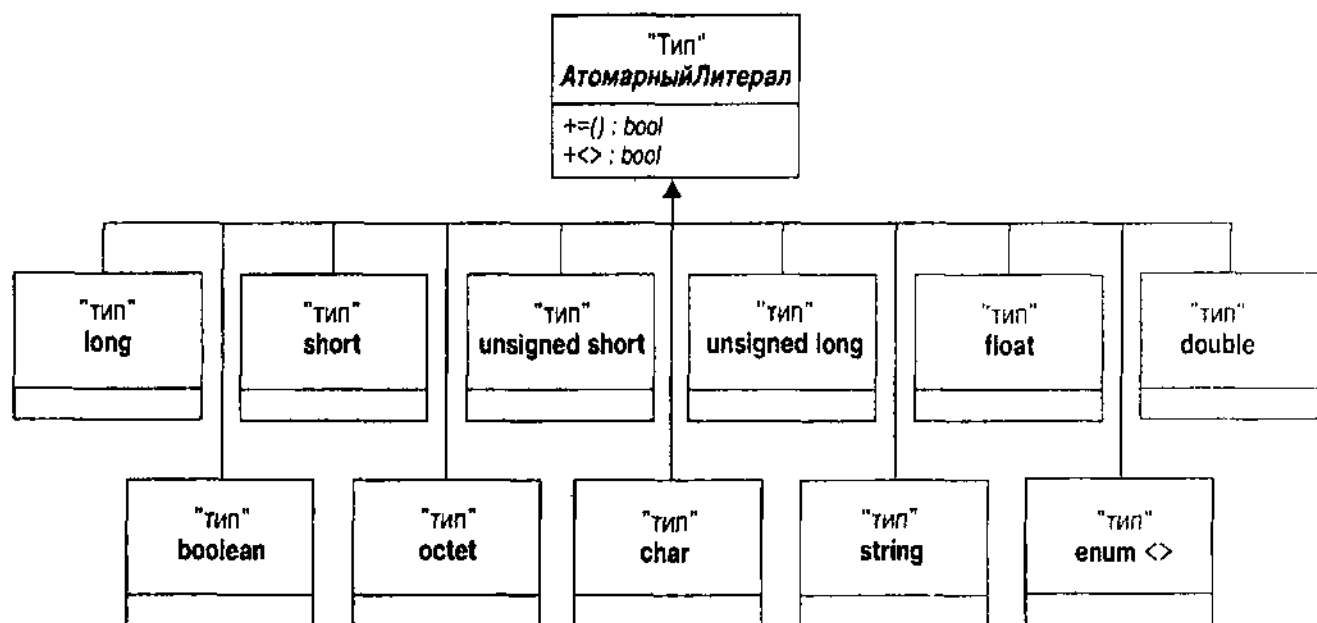


Рис. 7.17. Пакет АтомарныйЛитерал

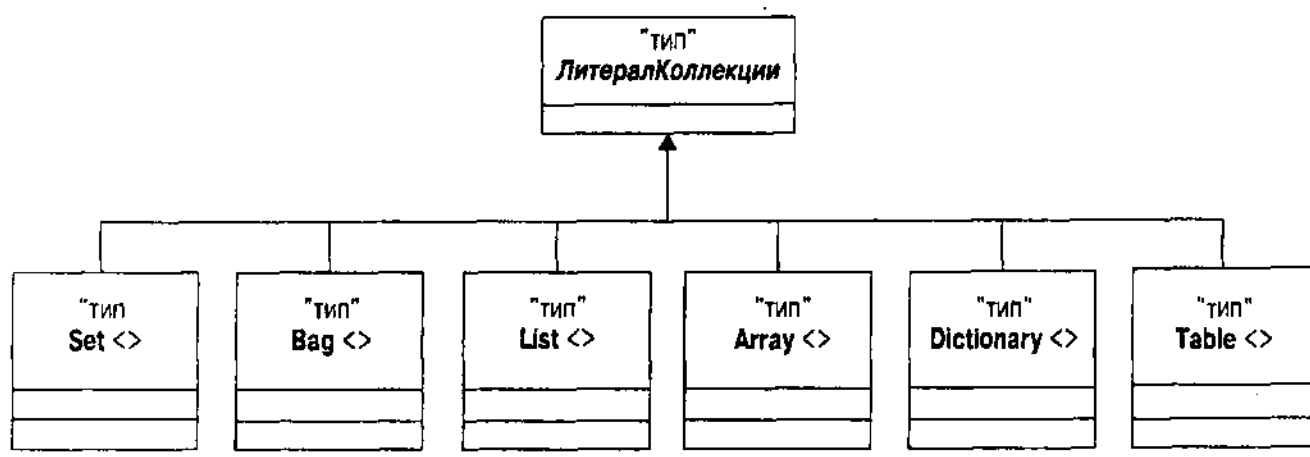


Рис. 7.18. Контейнер ЛитералКоллекции

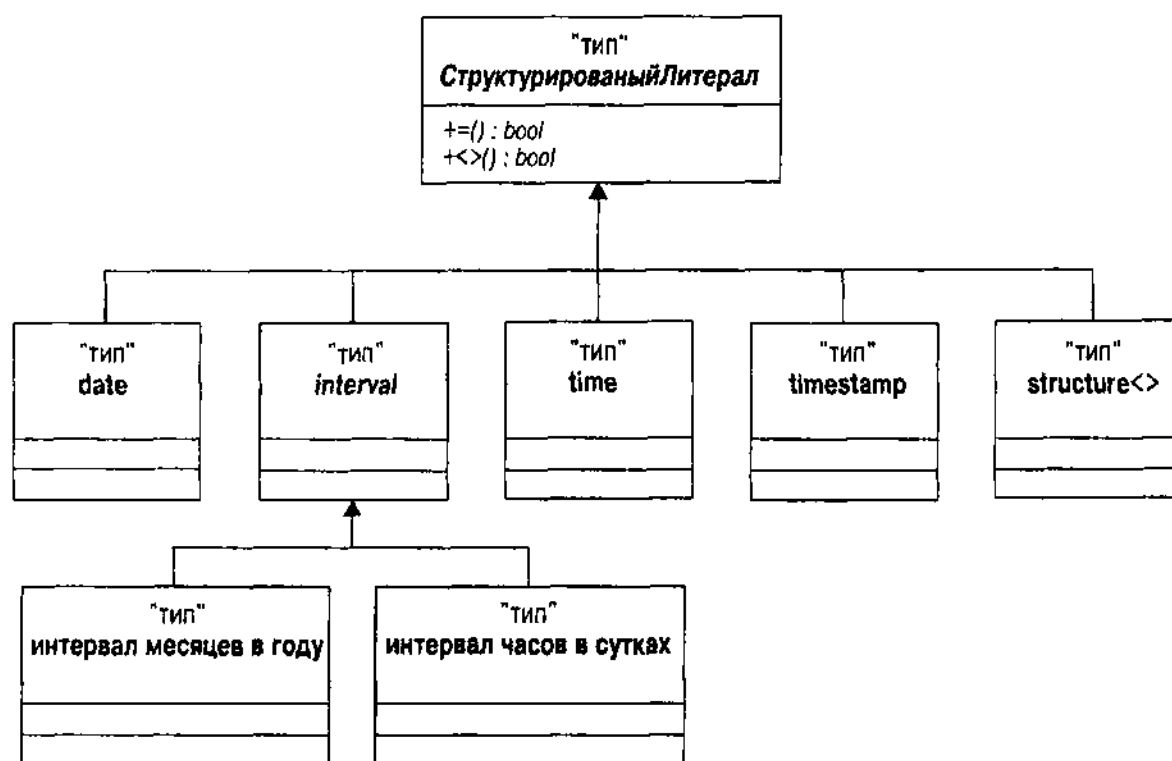


Рис. 7.19. Контейнер СтруктурированныйЛитерал

определен, можно без путаницы ссылаться на типы и операции, определенные на них в проекте. Можно также определить стандартные отношения между типами данных в модели типов и целевой моделью данных или типами данных DBMS, что позволяет затем создать корректные типы данных DBMS для различных целевых продуктов DBMS. Можно также определить подтипы как подклассы с ограничениями, такими, как ограничения диапазона.

Тип перечисления `enum <>` параметризуется с помощью набора литералов в объектной модели ODL. Таким образом, он создает экземпляры общего перечислимого типа. Затем можно с помощью этих экземпляров определять атрибуты или параметры. Этот подход – пример использования общности вместо наследования для создания разнообразия объектов с

различными качествами (в данном случае это перечислимые константы). Если есть возможность определить подтипы, можно поименовать специфический экземпляр его собственным именем и применять это имя. Например, чтобы определить ОфициальныйСтатус (LegalStatus) можно использовать это непосредственное объявление:

```
attribute enum<LegallyDefined, OnTrial, Alleged, Unknown> LegalStatus;
```

В качестве альтернативного варианта можно определить подтип и использовать объявление имени подтипа:

```
type enum<LegallyDefined, OnTrial, Alleged, Unknown> LegalStatusType;  
LegalStatusType LegalStatus;
```

Второй вариант лучше потому, что можно повторно использовать тип для нескольких различных атрибутов при одновременном определении типа как экземпляра общего типа перечисления. К сожалению, не все языки поддерживают перечисления, общность или создание подтипов, поэтому, чтобы сделать это, возможно, придется использовать другие способы.

Из-за трехзначной логики SQL (true, false и null) ODL определяет зеркальную иерархию типа для литералов null (nullable_float, nullable_set и т. д., что здесь не показано) [Cattell and Barry, 1997, p. 34]. Можно использовать эти типы для определения атрибутов, которые могут быть пустыми (null). Помните, однако, что null — это не значение, это отсутствие значения. SQL (и OQL) использует IS null для сравнений на null, а не синтаксис "=null", подразумевающий, что null — это значение.

СОВЕТУЕМ

Если будет использован другой подход, можно выразить отсутствие значения как расширенное свойство UML для атрибута: {nullable}.

Сложные ограничения

До этого момента UML выражал простые бизнес-правила, в частности ограничения целостности и доменные ограничения с помощью стандартной нотации. Множество бизнес-правил выходят за пределы этих простых механизмов выражения ограничений в UML. Следовательно, UML предоставляет формальный язык для выражения этих ограничений: язык объектных ограничений UML (OCL) [Rational Software, 1997c]. Из-за недостатка объема, мы не будем углубляться в подробности OCL; обратитесь к спецификации языка, если есть желание его использовать.

ВНИМАНИЕ

Можно выразить ограничения на любом подходящем языке, удовлетворяющем требованиям. Ограничение — это булево выражение, принимающее значение истина или ложь (не пусто) в конце каждой транзакции в любом контексте, в который она помещена. Выражение не может иметь побочных эффектов по состоянию этого контекстного объекта. Можно использовать выражение SQL или OQL, например, вместо OCL использовать Java, Smalltalk или вычисление предиката первого порядка, если это подходит. В некоторых системах, таких, как Oracle8, следует избегать языка, приводящего к возникновению побочных эффектов в состоянии объекта, что нельзя использовать в ограничениях.

Где на диаграммах UML можно использовать выражения OCL?

- *Классы и типы*: Ограничения на все экземпляры класса или типа (инварианты класса)
- *Операции*: Пред- и постусловия и другие ограничения на поведение операций
- *Переходы/потoki*: Граничные условия для диаграмм перехода состояния и деятельности

Можно использовать выражения инвариантов классов как расширение на индивидуальных атрибутах диаграммы классов с целью выражения правил для каждого атрибута. Как и для ограничений SQL CHECK, действительно не существует предела на то, что можно выразить с помощью таких ограничений, но зачастую удобно поместить правило вместе с атрибутом, к которому оно относится.

Нотация UML для ограничений — это заметка, присоединенная к ограничиваемому объекту. Рис. 7.20 демонстрирует простое ограничение инварианта класса для класса КриминальнаяОрганизация. Это ограничение, выраженное на SQL, говорит о том, что криминальная организация не может одновременно иметь ОфициальныйСтатус "ЗаконноУстановленный" и Статус Наказания, отличный от "История". Другими словами, если вы доказали в суде, что организация является криминальной организацией, то статус наказания должен быть "История", означающий, что вы выиграли дело.

Итоги

Очень трудно подытожить эту исключительно длинную главу, поскольку она сама по себе — резюме более крупного кодекса — спецификации UML. Глава фокусирует внимание на тех частях нотационной системы UML, которые непосредственно используются при моделировании данных и разработке БД.

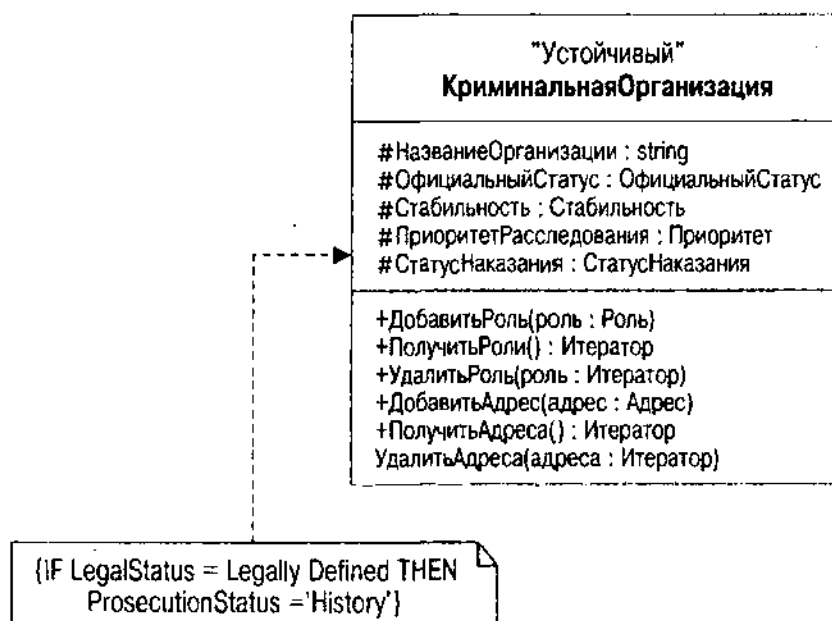


Рис. 7.20. Ограничение UML для КриминальнойОрганизации

Большинство классификаторов UML играют некоторую роль при разработке БД: пакеты, подсистемы, типы, интерфейсы, базы данных и классы — являются элементами такой разработки. Кроме того, отношения ассоциации, генерализации и реализации UML имеют важное значение для моделирования данных с помощью соотнесения классификаторов друг с другом.

Пакеты и подсистемы используются для создания архитектуры высокого уровня приложения базы данных. Эти классификаторы предоставляют поименованное пространство, организующее классы и другие подсистемы в четко разделяемые блоки. Они определяются с помощью вариантов использования транзакций, и варианты использования реализуются с помощью разнообразных классификаторов, которые работают совместно. Таким образом, создается модель данных, реализующая транзакции, определяемые требованиями.

Классы могут быть абстрактными (нет экземпляров) или конкретными. Каждый класс имеет набор атрибутов и операций, где атрибуты и их типы данных представляют состояние класса (данные), а операции являются поведением. Методы — реализация операций. Как атрибуты, так и операции имеют общедоступную, защищенную и скрытую видимость, функцию, позволяющую инкапсулировать данные и операции в пределах класса, снижая общую связность в имеющихся подсистемах.

Отношение генерализации позволяет соотнести классификаторы с иерархией наследования, либо с единственным, либо с множественным наследованием. Лучше избегать множественного наследования при разработке там, где это возможно. Можно использовать интерфейсы, чтобы избежать смешанного множественного наследования, сохраняя при этом строго типизированную разработку.

Отношение ассоциации позволяет соотнести классификаторы с иерархией видимости. Каждая сторона ассоциации — это роль, которую играет классификатор. Роль имеет множественность (1, *, 0..*, 2..6 и т. д.), что говорит о том, сколько объектов могут быть связаны с другими объектами при помощи ассоциации. Составная агрегация укрепляет ассоциацию до принадлежности вместо просто видимости: предок владеет объектом-потомком. Ассоциации сами могут становиться объектами с помощью классов ассоциаций, которые имеют собственные атрибуты и операции.

И, наконец, можно использовать UML для определения ограничений классификаторов и отношений. Идентификация объектов может быть с неявными метками (стандарт UML) или с явными метками (расширенный OID или альтернативный OID). Можно поместить доменные ограничения на атрибуты с помощью типов (любых типов значений, таких, как типы строки или объекта, которые определяются с помощью классификатора типа). Можно использовать расширенные метки для определения ограничений модели данных, в частности pullable. Можно определить произвольные сложные ограничения с помощью комментариев ограничения, прямоугольников, которые присоединяются к ограничивающей нотации. Используется любая разновидность языка ограничений от OCL UML до выражений SQL.

Теперь рассмотрена большая часть UML, относящаяся к моделированию данных. Следующая глава соединяет все вместе в процессе разработки модели данных.

Глава 8

Образцы моделирования данных

Воображение постоянно расшатывает традицию; именно в этом его предназначение.

Джон Пфайфер

Разработка в объектно-ориентированном мире за последние годы приобрела странное направление. Обычный процесс разработки в мире ПО представляет собой революцию за революцией. Каждое новое поколение разработчиков не принимает в расчет то, что было раньше, и создает новый способ того, что должно быть сделано. Эволюция языков программирования за последние 30 лет отражает революционный способ мышления к ужасу опытных разработчиков языков.

В мире ООП разработчики начали понимать, что имеются такие аспекты разработки, которые не следует менять или нужно менять совсем немного. Основная причина такого подхода проистекает из того факта, что ОО-разработчики делают упор на возможности повторного использования. Поскольку ОО-мышление вышло за рамки программирования, разработчики осознали, что некоторые из уроков мышления и философии в области архитектурного проектирования могут быть использованы также и при разработке ПО. Архитекторы программного обеспечения адаптировали работу Кристофера Александра, великого философа в области архитектуры, к разработке ПО с помощью концепции образца (*pattern*):

Образцы разработки вобрали в себя решения, которые были предложены и развивались в течение времени. Следовательно, они не являются попыткой программистов создать нечто новое. Они вызваны к жизни бесчисленными переработками проекта и перекодированием, стремлением разработчиков к возможности повторного использования и придания большей гибкости ПО. Образцы разработки впитали в себя эти решения в краткой и легко применимой форме [Gamma e. a., 1995, p. xi].

Сущность образца — традиция. Традиция — ядро решения. Забавная вещь, касающаяся традиций, состоит в том, что они постоянно противостоят воображению и наоборот. Сначала образцы разработки, как большинство традиций, помогают установить контекст для решения проблем разработки. Воображение затем начинает преобладать, двигаться за пределы традиционных способов разрешения проблемы. Иногда воображение выходит на первый план, предлагая новые образцы, т. е. преобладает новаторское мышление. Но если все делать заново, с самого начала, то невозможно продвинуться слишком далеко в работе.

В этой главе показано, как использовать образцы при моделировании данных. Она предоставляет примеры некоторых общих абстрактных и конкретных решений проблем моделирования данных и предлагает способы движения за пределы образцов для решения конкретных проблем.

Искусство создания систем с центральной БД определяется не используемой нотацией, а использованием нотации. По мере просмотра примеров образцов анализа и разработки подумайте о методах моделирования из главы 7 и как они применяются к образцам. Мышление само по себе и его выражение с помощью UML — сущность разработки БД. Такое моделирование данных, повторно использующее образцы и интегрирующее разные точки зрения на данные, предоставляет способ для более конкретных задач реализации концептуальных и физических схем.

Моделирование с помощью повторно используемых образцов разработки

Образец разработки является повторно используемой системой, которая описывает проблему разработки и ее решение. Можно варьировать структуру описаний образца. Однако базовая сущность образца состоит из имени, описания проблемы, описания решения и обсуждения проблем, компромиссов и последствий применения образца [Gamma et al., 1995].

Название образца — это буквально ключ к образцу. Когда говорят об образцах или любой традиции в этой области, на него ссылаются с помощью ключевой фразы, которая понятна всем людям, принадлежащим к этой сфере. Каждый человек может иметь личные варианты образца, но каждый понимает его основную природу при ссылке на него по имени. Можно поместить имя в комментарий, в код или диаграмму разработки, и рецензенты немедленно поймут, что они видят. Имя — абстракция, содержащая все усилия, вложенные в разработку традиции.

Проблема состоит в наборе обстоятельств, приводящих к необходимости решения. Часто наиболее полезная часть разработки — образец, проблема подсказывает, когда применить образец. Если проблема не соответствует полностью описанию проблемы, то проектное решение, возможно, неприемлемо. Небольшие вариации не важны при таком сравнении, главное — существенные элементы проблемы, поэтому при чтении описания проблемы, нужно сосредоточиться на существенных элементах, а не на деталях.

Как это происходит с большинством абстрактных дискуссий, описание проблемы часто кажется двусмысленным и неопределенным до тех пор, пока не наступит полного понимания. Образец почти всегда получает выигрыш от конкретного описания проблемы, проблемы реального мира, отображающейся на абстрактную проблему. Будьте осторожны, чтобы не попасть в ловушку конкретного примера, поскольку могут выявиться существенные различия между ним и проблемой, которую необходимо решить. Не путайте один реальный мир с другим, не являющимся точно таким же.

Решение состоит в такой разработке, которая снимает проблему. В данном случае это модель данных UML, представляющая абстрактный проект в общих терминах. Затем модель преобразуется путем присвоения имен разным элементам при помощи классов и отношений в той мере, в какой они применимы к данной специфической проблеме.

Обсуждение образца содержит все необходимые для разработчика вещи, которые он захочет рассмотреть при применении образца. Оно содержит один или два расширенных примера применения образца к реальным проблемам. Предоставление работающего кода обычно является хорошей идеей, если это можно сделать. Конечно, нужно упомянуть все возможные ограничения реализации на разных языках, проблемы переносимости или ситуации, которые могут привести к возникновению проблем тем или иным образом.

Последующие разделы представляют некоторые примеры того, что может сделать разработчик образцов. Из-за отсутствия достаточного места детализация примеров, к сожалению, не очень подробна.

СОВЕТУЕМ

Читателю, прежде всего архитектору ПО, готовому создавать собственные образцы, следует обратиться к более полным пособиям по образцам за идеями и подробностями (книгу "Банды четырех" [Gamma e. a., 1995] — необходимо прочитать). Кроме того, здесь не определен образец звезды, позволяющий создавать многомерное представление сложного хранилища данных. Данная книга не рассматривает моделирование хранилищ данных (см. введение, где говорится о схеме звезды и ее таблице факт-размерность). Это соответствует простому образцу UML с центральным n-ным отношением (ассоциация факта и его класса) и участвующими объектами (классами размерности).

Образцы разработки из других источников и созданные самостоятельно являются частью портфеля систем повторного использования. Необходимо создать каталог и проиндексировать их, чтобы разработчики в данной области могли найти и использовать их, как и любую систему повторного использования.

Имеется два вида образцов, представляющих непосредственный интерес для модельеров данных: абстрактные образцы и аналитические образцы. В следующих разделах приведены некоторые примеры, относящиеся к этим типам.

Абстрактные образцы

Абстрактный образец в контексте моделирования данных — это образец классов и отношений, которые описывают общее решение общей проблемы. Имена часто являются обобщающими (например, составными или опосредованными) и не отображаются непосредственно на объекты, рассматриваемые при решении проблемы разработки.

Каждый из последующих подразделов представляет один образец из книги "Банды четырех" или из собственной работы автора, преобразованный в образец для базы данных. Это образцы, для которых автор смог найти непосредственное применение в приложении БД.

Другие образцы относятся больше к разработке приложения, особенно к абстракциям или поведению в коде приложения, а не к серверу БД. Имеется большое перекрытие в методиках кодирования в современном программном обеспечении DBMS с прикладным ПО (триггеры, серверные расширения и включения, хранимые процедуры, устойчивые методы классов и т. д.). Однако при разработке представляют интерес и другие образцы.

Одноточечный образец (Singleton)

Одноточечный образец гарантирует, что класс имеет только один экземпляр в базе данных.

Проблема

Часто при моделировании данных обнаруживается, что есть абстракция, для которой может быть только один экземпляр объекта. Имеется несколько общих примеров этого в реляционных БД: строка, представляющая значение по умолчанию для таблицы, строка, представляющая указатель на одну строку в другой таблице, и строка, представляющая контрольное состояние системы. Так, можно иметь организационную таблицу, представляющую иерархию организаций, и одноточечную таблицу, содержащую первичный ключ организации, касающейся разработчика. Или можно иметь объект, представляющий собой интервал времени, после которого объекты в БД не могут изменяться.

Одноточечный образец используется, когда нужно иметь один объект и когда может потребоваться расширение этого объекта при помощи подклассов. Подклассы в одноточечном классе не создают разных экземпляров, они расширяют версию экземпляра, которая заменяет поведение оригинала полиморфным поведением в подклассе.

Решение

Рис. 8.1 представляет одноточечную модель данных.

Одноточечный класс на рис. 8.1 имеет единственный экземпляр. Отношение *УникальныйЭкземпляр* — это отношение один-к-одному, где оба объекта в отношении идентичны и *УникальныйЭкземпляр* — это атрибут класса, поэтому может быть только один экземпляр класса. Можно сделать все атрибуты атрибутами класса, поскольку имеется только один экземпляр, хотя это и не обязательно.

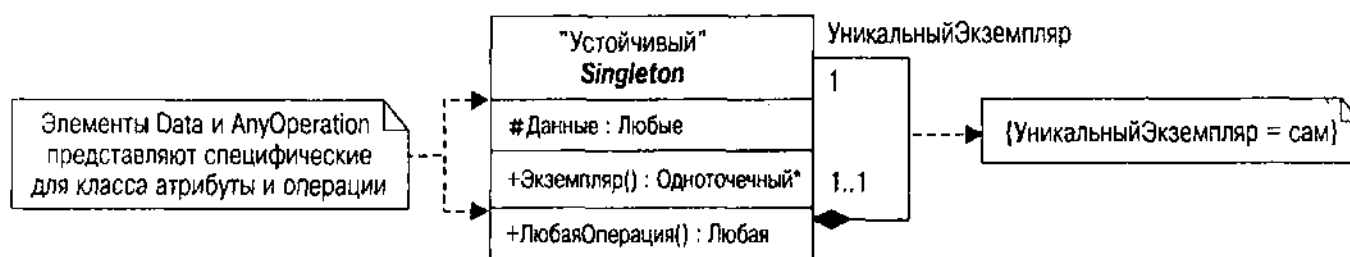


Рис. 8.1. Одноточечный образец (Singleton)

Операция Экземпляр используется, чтобы получить ссылку на единственный экземпляр. В реляционной БД это соответствует статическому запросу, который извлекает строку из базы данных. В ORDBMS она может также выполнять функцию конструктора для создания экземпляра, если он еще не существует. В объектно-ориентированной БД он извлекает объект экземпляра (или создает его, если нужно), используя статический идентификатор объекта или другое средство ссылки на экземпляр класса.

Обсуждение

Одноточечный образец (Singleton) на прикладном уровне, как правило, является средством задать единственный объект и получить к нему доступ. Основной цикл событий в системе соответственно — одноточечный. Если такие вещи делаются устойчивыми в OODBMS, то они обычно одноточечные. В реляционных БД одноточечный образец играет менее важную роль, ограничиваясь служебными таблицами, к которым присоединяются другие таблицы или которые используют для задания значений.

Обеспечение ограничения одноточечности требует, чтобы любые конструкторы были скрытыми или защищенными так, чтобы только операция Экземпляр могла создать экземпляр, если его еще нет. Атрибут/отношение УникальныйЭкземпляр также реализует ограничение одноточечности. Реляционная БД не имеет конструкторов или защиты доступа. Поэтому реализуется ограничение с триггером или при помощи процедурной инкапсуляции, скрывающее таблицу за привилегиями безопасности и разрешающее доступ только через хранимую процедуру Экземпляр.

Использование операции Экземпляр для создания экземпляра позволяет избежать необходимости создания единственного экземпляра, если это не нужно. В большинстве приложений базы данных это не проблема, поскольку экземпляр определяется и сохраняется для использования во многих приложениях с помощью извлечения. Возможно, найдется применение, которое требует создания экземпляра только при определенных обстоятельствах. Агентство Holmes PLC может внедрить систему с принудительным выключением при несанкционированном доступе, например, с помощью создания Одноточечного экземпляра, присутствие которого запретит доступ к любым чувствительным данным определенного пользователя или всех пользователей. Администратор БД затем расследует ситуацию и найдет решение, удаляя экземпляр. Этот вид экстремизма потребовал бы, конечно, много аудиторской работы и других данных о нарушении безопасности.

Составной образец (Composite)

Составной образец представляет структуру дерева, созданную из различных видов связанных объектов, в которых все используют один и тот же интерфейс. Составной образец служит для представления иерархии часть — целое в базе данных.

Составной образец очень распространен в прикладном моделировании. Разработчики БД, новички в области объектно-ориентированного моделирования и образцов могут считать его поначалу трудным из-за его чрезмерной сложности, но приобретаемая при этом гибкость стоит дополнительных усилий.

Проблема

Одной из проблем классического моделирования в теории реляционных БД является проблема взрыва частей. Несмотря на свое захватывающее название, это не относится к активным компонентам восточных баз данных и редко бывает опасным, кроме того случая, когда несчастный программист SQL вынужден иметь с ней дело. Взрыв частей — древовидная структура данных, моделирующая ряд частей продукта, отдельные из которых содержат другие части. Часть в данном случае — это не экземпляр или сам объект, а скорее *тип* части. Например, структура вводимых в записную книжку энциклопедических данных содержит разнообразные объекты. По множеству различных причин нужно рассмотреть эти разные объекты так, чтобы они были все в общем классе: в определенной части. Каждый тип части имеет номер части, ее идентифицирующий.

Рис. 8.2 иллюстрирует простой взрыв частей с использованием компонентов документа в качестве частей. Статья энциклопедии содержит другие статьи наряду с текстом, видео- и аудиоинформацией.

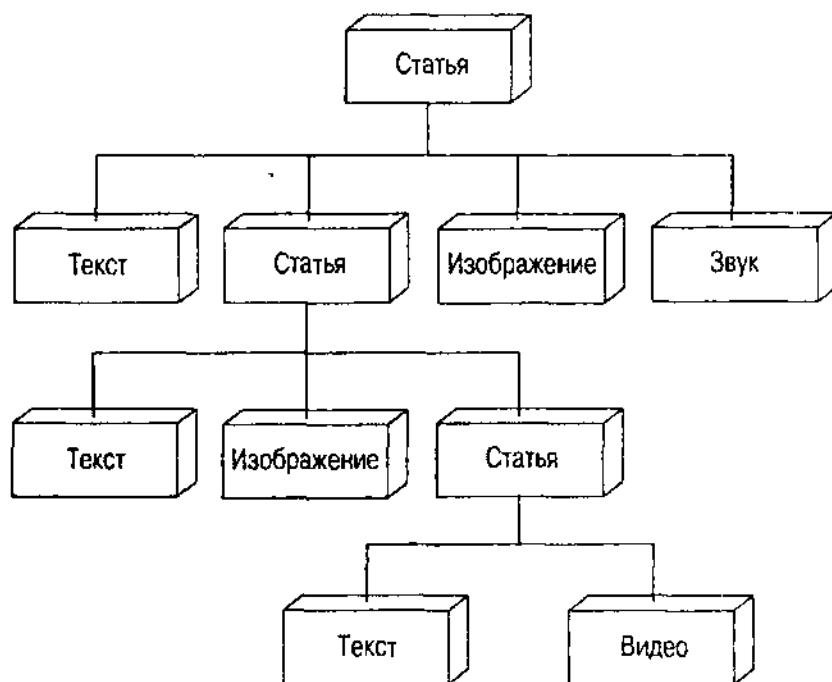


Рис. 8.2. Взрыв частей

Задача состоит в том, чтобы создать полный список всех частей данной части. Поскольку некоторые части содержат другие части, структурой является дерево, как показано на рис. 8.2, и задача состоит в проходе по дереву, чтобы найти все части. Математический термин для этой операции — *транзитивное замыкание*, набор всех частей в БД, которые можно достичь из корневой части.

Есть пара дополнительных проблем с этой структурой, вызывающих необходимость создания составного образца. Во-первых, части неоднородны; не все они являются членами одного класса. Во-вторых, все эти части должны использовать общий интерфейс, так как нужно получить доступ к ним одинаковым образом. В примере на рис. 8.2 требуется извлечь все компоненты статьи энциклопедии и вывести их на экран, не зная специфики вещи — классическое использование интерфейсного полиморфизма. В классической ОО-модели это последнее требование означает, что все различные классы должны наследовать от общего абстрактного суперкласса.

Решение

На рис. 8.3 показан составной образец (Composite). Компонент — это абстрактный суперкласс, имеющий два типа подклассов, в которых все являются объектами Компонент: листовой класс Лист, не имеющий потомков, и составной класс Композит, действительно имеющий потомков. Ассоциация потомков создает древовидную структуру, позволяя объектам Композит содержать другие объекты Композит или при необходимости объекты Лист.

Класс Компонент — это абстрактный интерфейс для всех объектов дерева. Он содержит как абстрактные операции, так и операции, предоставляющие поведение по умолчанию. Он может содержать атрибуты, используемые всеми компонентными объектами. Таким образом, Компонент предоставляет

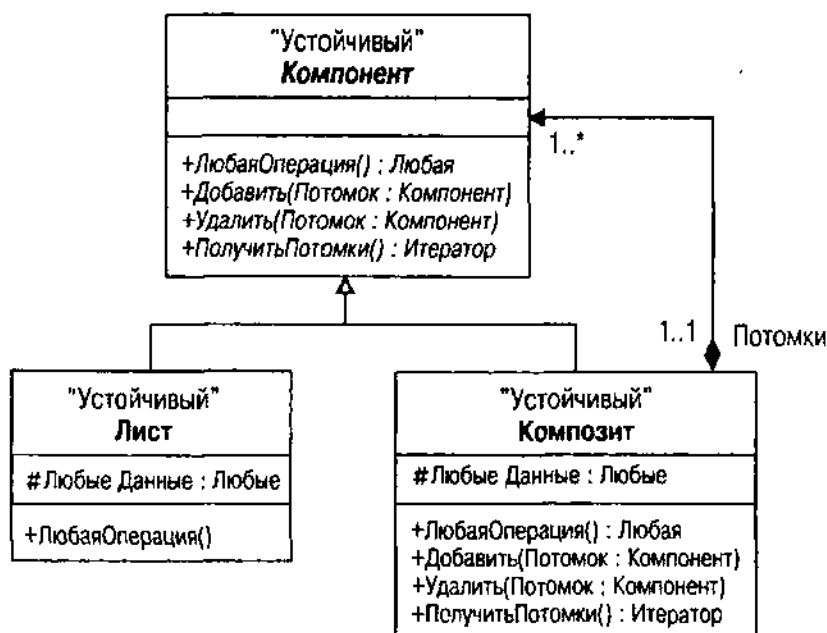


Рис. 8.3. Составной образец (Composite)

все операции, которые можно совершить с любым компонентным объектом, включая итерации по потомкам объекта. Операции и методы класса Лист предоставляют конечное поведение системы при реализации основных компонентных операций.

Класс Композит в действительности хранит потомков, на что указывает ромб составной агрегации в ассоциации потомков с классом. Он также предоставляет операции и методы, переопределяющие абстрактные операции Компонент для управления потомками, такие, как Добавить(), Удалить() и ПолучитьПотомки().

В случае взрыва частей на рис. 8.2 класс Статья является составным, а классы Текст, Видео, Изображение и Звук — все листовые подклассы (вместе со Статьей) компонентного класса. На рис. 8.4 — иерархия класса КомпонентыЭнциклопедии, которая содержит все эти классы.

Обсуждение

Большинство разработчиков, видящих составной образец (Composite) впервые, испытывают непреодолимое желание спросить: "А как быть с операциями над листовыми классами, которые бессмысленны на других листовых или составных классах?". Ответ: не делайте этого. Цель составного образца — считать все объекты дерева идентичными компонентными объектами, нелетовыми. Если листовые классы имеют семантику, требующую специальных операций, определите отдельные классы и инкапсулируйте их в пределах листовых классов. Затем можно поддерживать объекты в отдельной таблице и ссылаться на них через дерево по мере необходимости, сохраняя данные и операции индивидуального класса. Классы ФайлМРЕГ и ФайлWAV — примеры этого подхода на рис. 8.4. КомпонентЭнциклопедии просто выводит на экран компонент; два файловых класса имеют

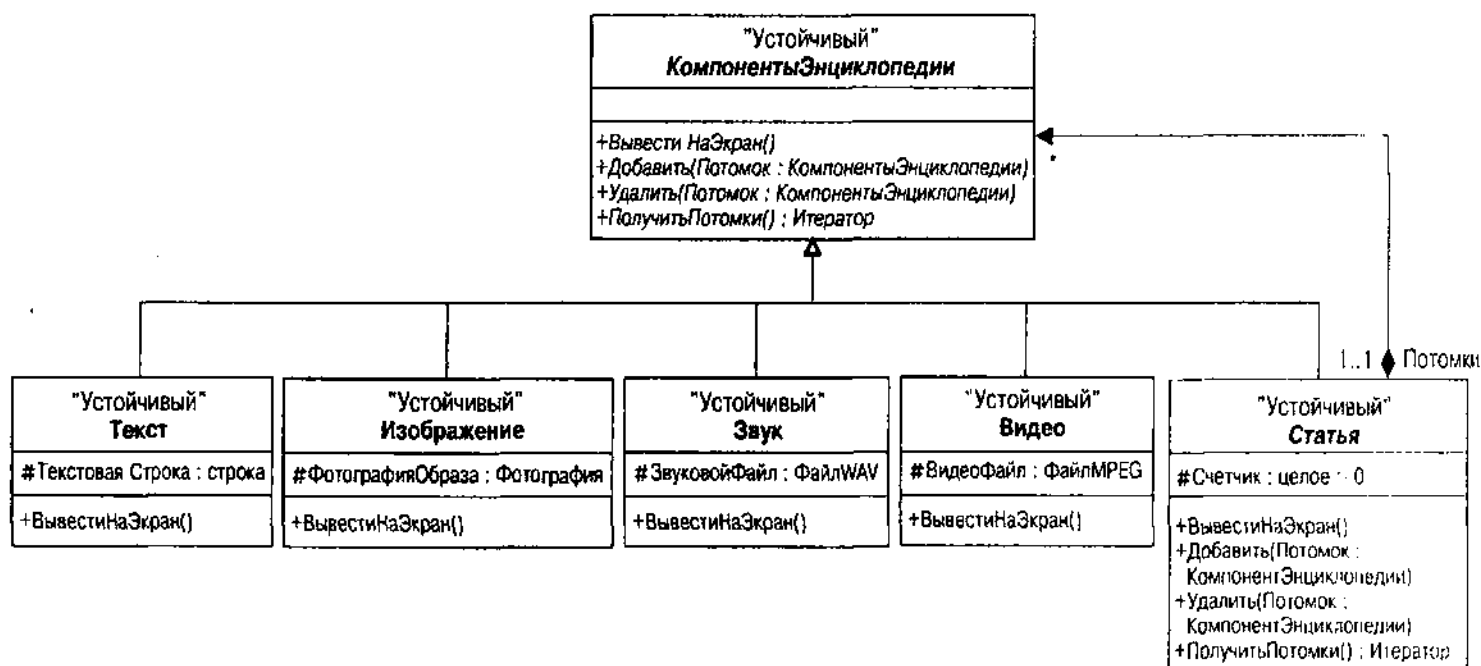


Рис. 8.4. Приложение составного образца КомпонентыЭнциклопедии

различные файловые операции, позволяющие манипулировать ими в пределах операции ВывестиНаЭкран. Если требуется особым образом управлять такими файлами, как MPEG и WAV за пределами дерева, извлеките их по отдельности и сделайте это.

Реализация составного образца может стать трудным, но интересным делом в базе данных. Например, реляционные БД усложняют реализацию потому, что они не имеют множеств для хранения потомков. Образец определяет хранящихся потомков в составном классе, поскольку листовые классы не наследуют отношение. В реляционной БД хранят OID предка в потомке. Например, если преобразовать рис. 8.4 в схему Oracle, будет создана таблица КомпонентЭнциклопедии. Затем добавляется атрибут для хранения идентификатора компонента энциклопедии, являющегося предком. Корневой компонент получает значение предка null.

В данном случае почти наверняка желательно создать отдельную таблицу для представления отношения, даже если это кажется более сложным. Создайте отдельную таблицу КомпонентПотомков, которая отображает идентификатор КомпонентЭнциклопедии для потомка в предке. Затем можно объединить все три таблицы (КомпонентЭнциклопедия, КомпонентПотомки и КомпонентЭнциклопедия), чтобы получить потомков данного компонента (или его предков). Этот подход на основе пятой нормальной формы также позволяет расширить образец, чтобы позволить компоненту иметь несколько предков (общее использование компонентов), что невозможно с помощью другой конструкции. Это изменяет составную агрегацию (черный ромб на Composite) до совместно используемой агрегации (белый ромб). См. раздел "Легковесный образец (Flyweight)", где рассматривается альтернативная конструкция.

С помощью продуктов ORDBMS и OODBMS можно непосредственно сохранять REF или множества, поэтому такой образец не представляет проблему.

Можно добавить метку {упорядоченный} к роли Потомки, чтобы определить, что потомки имеют специальное упорядочение. Это позволит обрабатывать статью энциклопедии каждый раз в одном и том же порядке, например для структурирования экрана.

Составная агрегация предполагает управление удалением с помощью составной схемы, т. е. когда удаляют составной класс, также удаляются все его потомки. Это предполагает наличие некоторого специального триггерного кода или определений ограничения ссылочной целостности в реляционной БД.

Легковесный образец (Flyweight)

Легковесный образец можно считать примером "вороньих лапок", возвращающихся домой на насест. Этот образец "использует общий доступ для эффективной поддержки большого количества тщательно структурированных объектов" [Gamma с. а., 1995, р. 195].

Проблема

Трансляция этой цели в контекст БД сводит ее к снижению избыточности с помощью разделения внутренних и внешних данных, т. е. к нормализации. *Внешние данные* — те, что варьируются в зависимости от контекста, а не от объекта; *внутренние данные* варьируются в зависимости от объекта. В терминах базы данных OID (первичный ключ) идентифицирует объект. Внутренние данные — такие данные, которые в контексте множества объектов варьируются вместе с OID. В терминах реляционной БД это преобразуется в функциональную зависимость от всего первичного ключа или четвертой нормальной формы (см. главу 11). Снижение избыточности означает создание объектов только с внутренними данными и помещение внешних данных в отдельное место. Разница в ситуации, приводящей к легковесному образцу, и концепции нормализации состоит в общем (совместном) использовании.

Если приложение использует большое количество объектов, дублирование внутренних данных среди всех объектов может быть значительным в зависимости от размера внутренних данных каждого объекта. Все эти дублирования приводят к громоздкой БД и громоздкому приложению в целом. С помощью общего доступа к данным можно значительно сократить совместную потребность в памяти, что всегда является преимуществом при проектировании БД (и приложения).

Рассмотрим, например, ситуацию с системой энциклопедии в каталоге. Многие изображения в БД — составные, сделанные из различных компонентов. Многие из этих компонентов одинаковы, просто по-другому расположены и раскрашены в каждом изображении. Если хранить изображения как композиции предок — потомок, где каждый компонент сопровождается местоположением и цветом, все компоненты каждого составного изображения будут храниться отдельно. Это означает, что имеется значительная избыточность БД. Делая общий доступ к компонентам, можно намного сократить потребность в памяти. Можно разделить изображения и хранить только данные о раскраске и местоположении составного объекта вместе с указателем на общедоступное изображение.

Это типичная ситуация с компьютеризацией вещей реального мира. Первый взгляд на любую ситуацию вызывает миллион вопросов; только при тщательном анализе объектов можно увидеть подобные вещи. Нужна теоретическая дисциплина, позволяющая пробиться сквозь нерегулярность с помощью систематической инновации, сокращающей хаос до ряда закономерностей.

Решение

Легковесный образец разбивает объекты на внутренние и внешние свойства, чтобы позволить системе представить как можно меньше объектов с помощью их общего использования (см. рис. 8.5).

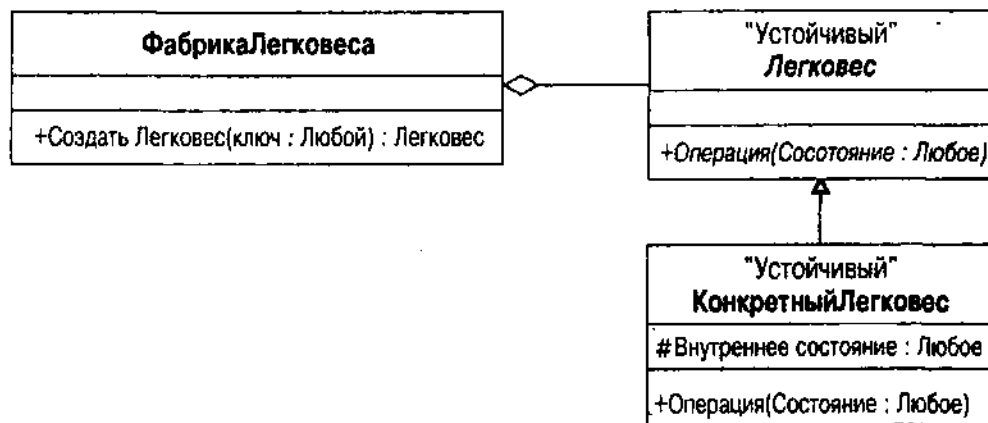


Рис. 8.5. Легковесный образец

Класс Легковес — это абстрактный класс, представляющий потребности подклассов во внешней информации. Различные порции внешних данных являются элементами различных интерфейсных операций. Класс ФабрикаЛегковеса — это класс, управляющий совокупностью объектов Легковес, обеспечивая общий доступ для различных клиентских объектов. Он имеет методы-построители, которые либо создают новый общий объект, либо просто возвращают его, если он уже существует. Он может также иметь методы-построители для подклассов класса Легковес, которые просто создают неразделяемые объекты всякий раз, когда вызывается метод. ФабрикаЛегковеса владеет также временными структурами агрегированных данных, хранящими объекты и код запроса, считывающий их из БД. Класс КонкретныйЛегковес представляет действительное внутреннее состояние объекта Легковес, используемое различными классами.

На рис. 8.6 приведен пример изображения как легковесного образца.

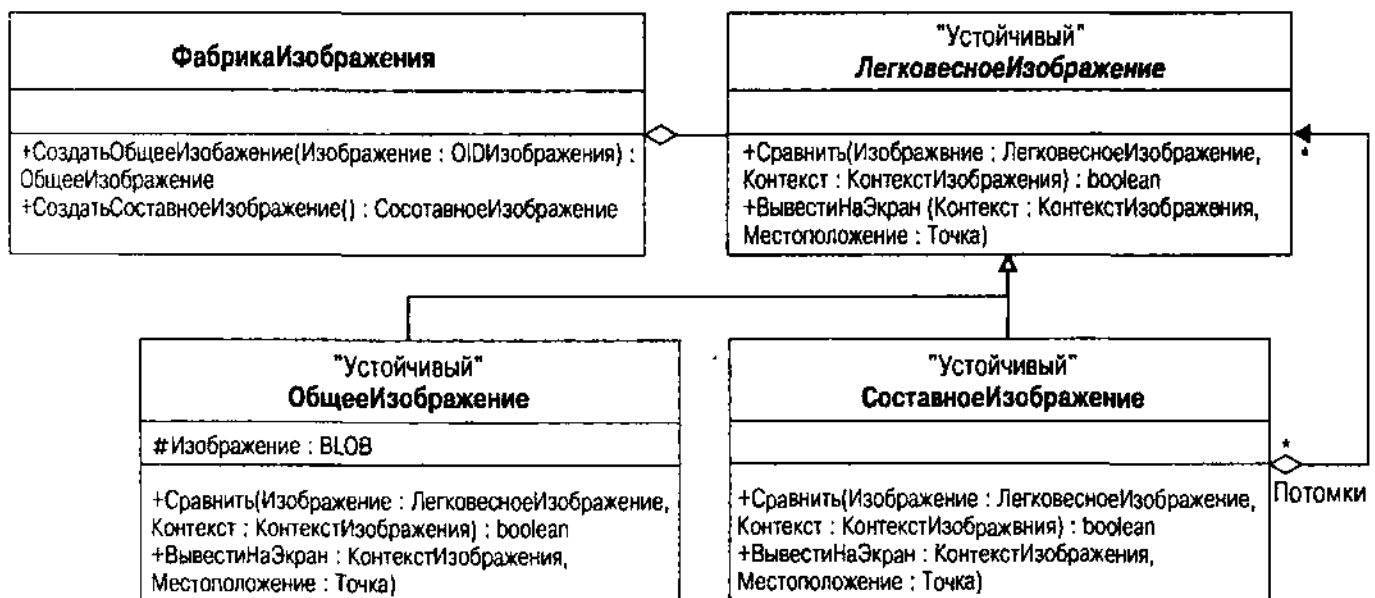


Рис. 8.6. Компонентная иерархия изображения как легковесного образца

Этот пример упрощает интерфейс. ФабрикаИзображений производит либо изображение для общего доступа, либо неразделенное составное изображение в зависимости от того, что хочет клиент. Она хранит совокупность изображений общего доступа, запрашивая их из БД по мере необходимости (или сразу все, если нужно). Интерфейс в ИзображенииЛегковеса импортирует сведения о контексте и местоположении по мере необходимости. Битовая матрица изображения (или любой другой тип данных Изображения) — это внутреннее состояние в общем конкретном классе Легковеса — ОбщееИзображение. Другой конкретный класс СоставноеИзображение хранит набор потомков как составной образец — общий доступ отсутствует.

В БД центральная таблица или кластер объектов хранит общие изображения. Другая таблица или класс, которые не показаны на рис. 8.6, хранят данные о местоположении и раскраске изображений, отображая их в общее изображение с помощью внешних ключей или ассоциации некоторого рода. Когда приложению требуется общее изображение, клиент запрашивает его у Фабрики, предоставляя идентификатор OID Изображения в качестве ключа. Фабрика управляет запросами ранее созданных изображений или созданием новых.

ВНИМАНИЕ

Чтобы не усложнять схему, пример, приведенный на рис. 8.6, не включает операций для реализации составного образца, таких, как Первый и Следующий, а также ПолучитьИтератор.

Обсуждение

Устойчивые объекты Легковеса отличаются от обычных устойчивых объектов в основном в области отношений, имеющих с другими объектами. В реляционной БД количество связей с легковесным образцом должно быть весьма значительным, чтобы оправдать сохранение его нормализованным, вместо денормализации для сокращения в приложении излишних объединений.

Большая часть легковесного образца посвящена тому, как представить объекты в памяти, а не тому, как хранить их в БД. Например, в OODBMS имеется возможность превращения Фабрики в устойчивый класс и сохранения связей с общими легковесными объектами как устойчивых связи при задании ассоциации. Затем просто извлекается Фабрика и используется интерфейс, чтобы извлечь легковесные объекты в память. Для реляционной БД это в действительности не очень полезно, поскольку объект Фабрика не имеет состояния для сохранения в реляционном отображении. Также просто можно создать Фабрику и запросить OID общих объектов во внутреннюю структуру отображения, затем запросить объекты так, как клиенты запрашивают их через интерфейс Фабрики.

Критический факт жизни для легковесного образца состоит в том, что все подклассы должны использовать одни и те же типы внешних данных. Если это затруднительно (имеются подклассы, использующие различные типы данных), нужно расширить легковесный объект с помощью образца агрегации. Этот образец позволяет представить различные виды интерфейсов в иерархии одного класса. В легковесном объекте данные являются либо внешними, либо внутренними, при этом все внешние данные передаются через легковесный интерфейс.

Общее использование, как отмечено в главе 7, — другой вид агрегации. При общем использовании объектов можно больше не полагаться на образец, создающий объект и управляющий им. Как любое совместное предприятие, легковесный объект общего пользования нуждается в отдельном управлении, чтобы выжить. Это означает, что Фабрика или какой-то другой управляющий объект контролирует множество легковесных объектов и реализует общий доступ к этому множеству. По большей части Фабрика является временным объектом, ответственным за создание и уничтожение легковесных объектов из устойчивых данных. Потенциально можно сделать Фабрику устойчивой, если она содержит данные о легковесном объекте, такие, как специальный способ ссылки на объект (например, схема индексации для элементов изображения на основе строк).

Исходный образец [Gamma et al., 1995] утверждает, что эффективность образца возрастает по мере увеличения количества состояний объекта, которые можно сделать внешними. Фактически эффективность возрастает по мере увеличения количества состояний объектов, которые можно сделать *внутренними*. Чем больше их будет, тем лучше.

Если внешнее состояние сложно и требует дополнительного нормализационного структурирования, то можно обнаружить, что количество объектов приближается к тому числу, которое имелось в первоначальной проблеме. Это классический случай *денормализации*, перестройки элементов нормализованной БД для сокращения общей сложности и увеличения производительности. Денормализация сокращает общее количество объектов (строк в реляционной БД) и количество операций, которые нужно выполнить для навигации между объектами. Обратитесь к главе 11 для полного рассмотрения нормальных форм, нормализации и денормализации. Если по мере внедрения легковесного образца создается огромное количество классов, то лучше от него отказаться и выполнить денормализацию базы данных.

Реляционная БД реализует легковесный образец как простую последовательность таблиц. Легковесный суперкласс (ЛегковесноеИзображение на рис. 8.6) показывает абстрактный внешний интерфейс для различных компонентов изображения. Поскольку у него нет состояния, таблица содержит только первичный ключ для таблиц подклассов. Можно альтернативно определить легковесный класс как интерфейс UML вместо наследования из него как из суперкласса и удалить его полностью из реляционной БД. Это создает классический сценарий нормализации. В обоих случаях ссылки являются классическим отношением ссылочной целостности по внешнему ключу.

В ORDBMS легковесный образец использует типы и ссылки, а не вложение таблиц. Вложение таблиц — классический пример составной агрегации, где таблицы-предки владеют вложенной таблицей. Вместо этого легковесный класс изымается из внешних классов, которые будут на него ссылаться. Нужно все-таки поддерживать ссылочную целостность в приложении или в триггерах, поскольку ссылки не имеют автоматических механизмов поддержания целостности ссылок (см. главу 12).

В OODBMS создается легковесный суперкласс и его подклассы, а расширение легковесного объекта извлекается через этот класс и индивидуальные расширения через подклассы. Можно сделать Фабрику устойчивой, и тогда множество легковесных объектов, используя OODBMS, тоже будут устойчивой ассоциацией. Таким образом, доступ к легковесному объекту получают через Фабрику так же, как это делается в чисто временной организации. Для всех этих ассоциаций используются двунаправленные отношения с автоматической поддержкой ссылочной целостности (см. главу 13).

Образец метамодели

Образец метамодели моделирует структуру объекта как последовательность объектов. Это позволяет создавать очень общие модели сложных произвольных отношений данных.

ВНИМАНИЕ

Метамодель — это не образец Банды Четырех, а скорее оригинальный образец на базе работы, выполненной автором в прошлом. Другие авторы имеют подобные структуры [Нау, 1996, р. 61–65].

Проблема

Временами реальность устраивает проверку реальности. Это обычно происходит, когда сложность структуры объектов так высока, что модель взрывается и распадается на подклассы и альтернативы. Это может также случиться, если модель создается простой, а затем, когда заказчик начинает ее использовать, встречается с реальным миром. Заказчик выдвигает одно требование за другим, каждое из которых добавляет большое количество классов и атрибутов к БД.

С этой ситуацией связаны две проблемы. Во-первых, чем больше сложность модели данных, тем труднее со временем ее поддерживать, понимать и изменять. Во-вторых, изменения нередко влияют на очень небольшую часть всех пользователей. Разработчик вынужден приспособливать систему для каждого пользователя и затем передавать эти изменения всем пользователям.

Решение

Образец метамодели создает структуры, используя строительные блоки из структурной парадигмы (см. рис. 8.7). Класс Клиент представляет класс объектов, к которым нужно добавить атрибут. Класс Атрибут представляет атрибут данных, добавляемый к классу Клиент. Класс ассоциации Значение моделирует отношение многие-ко-многим между атрибутом и целевым клиентом. Имеется один подкласс КонкретноеЗначение для каждого типа данных, которые нужно моделировать.

Образец метамодели решает проблему усложнения функций за счет мелких улучшений, позволяя "модифицировать" структуру БД без ее реальной модификации. Вместо этого данные добавляют к структурам метамодели, и приложение представляет эти данные, как будто они фактически являются частью Клиента. Программа может представлять фиксированный набор атрибутов, а также предоставить интерфейс для пользователя, позволяющий добавлять атрибуты через приложение.

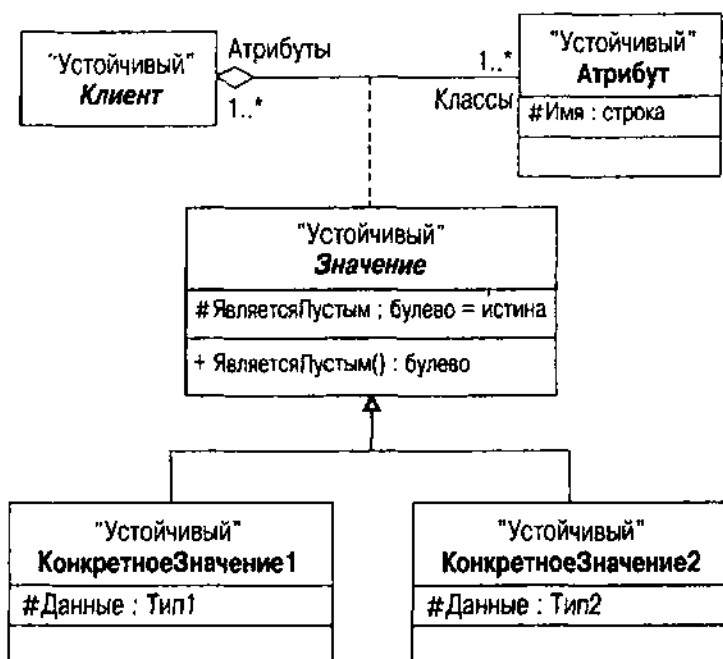


Рис. 8.7. Образец метамодели

Обсуждение

Если известна конкретная разновидность БД, которая будет использоваться в системе, можно модифицировать образец, чтобы он представлял структуры концептуальной модели, а не использовал классы и атрибуты. Например, в реляционной метамодели можно моделировать столбцы, а не атрибуты.

Самая большая проблема с образцом метамодели — ограничение на выборку данных. Атрибуты и значения не являются реальными структурами БД, и нельзя использовать SQL или OQL, чтобы выполнить их непосредственную выборку. Запрещается, например, использовать атрибут в предложении WHERE со значимым литералом для выборки подмножества строк из таблицы. Один выход из этого ограничения — предоставить системе способ создавать реальные структуры в БД на ходу, что могут делать почти все базы данных. Это означает размещение немного большего количества данных в классы Атрибут и Значение. Дополнительная информация позволяет вводить операторы SQL или ODL, которые создают правильные структуры и заполняют их данными, полученными из объектов Значение. Приложение может затем создать надлежащие запросы SQL или OQL на новых объектах.

ВНИМАНИЕ

За последние несколько лет CASE и другие средства разработки ПО начали использовать репозитории, базы данных с метаданными о программном обеспечении. Эти БД служат примером образца метамодели.

Аналитические образцы

Аналитический образец [Fowler, 1997] — это модель конкретных определяемых доменом объектов, генерализованных для повторного использования в различных ситуациях. Это — образцы, которые возникли в результате многих лет разработок БД. Программисты рассматривали одни и те же таблицы или объекты снова и снова до тех пор, пока не понимали, что нужно делать, когда ситуация возникает в новом контексте. Однако генерализация не соответствует уровню образцов разработки в разделе "Абстрактные схемы". Аналитические модели имеют специфические семантические привязки к определенным доменам.

Имеется огромное количество доменов для моделей данных детально разработанных, например, система предоставления патентов, и очень широких, как модель людей и организаций. Существуют некоторые базовые определяемые доменом модели, которые имеют промежуточную степень детализации, такие описаны в данном разделе, и эти модели могут иметь высокую степень повторного использования в доменах с большей степенью детализации.

ВНИМАНИЕ

Данный раздел описывает пионерские разработки разных людей [Blaha and Premerlani, 1998, p. 81–88; Fowler, 1997; Hay 1996; Silverston, Inmon and Graziano, 1997]. Представленные автором модели являются модификациями этих работ, перерисованные и переработанные с помощью методики моделирования UML из главы 7, они используют сжатый стиль абстрактного образца. Аналитические модели, представленные в данном разделе, — это только некоторые из имеющихся. Кроме того, из-за недостатка места в этом разделе не рассмотрены все детали, имеющиеся у авторов первоисточников; пожалуйста, обратитесь к этим работам за подробностями об образцах, определяемых доменом. Где можно было что-то улучшить, автор делал это, упоминая при обсуждении об улучшении исходного образца. Это будет отражено в будущей книге об образцах БД.

Образец партнерства (Party)

Образец партнерства моделирует людей и организации и их основное отношение: работу.

Проблема

Большинство бизнес-доменов в той или иной мере включают людей и организации: людей потому, что они составляют организацию, а организации потому, что деловая деятельность без организации обычно не нуждается в БД. Организация, однако, нечто большее, чем просто агрегация людей. В определенной степени это что-то вроде юридического лица со своими собственными правами, особенно, если это корпорация или компания с ограниченной ответственностью. У обеих есть названия и адреса, а у нескольких людей и организаций может быть общий адрес.

Работа — это отношение между человеком и организацией. Люди и организации со временем меняют производственные отношения. Люди, работающие в одной и той же организации в течение некоторого времени, занимают разные позиции с разными названиями должностей.

И, наконец, люди и организации вступают в определенные иерархические отношения друг с другом. Одни люди подчиняются другим, а организации подчиняются другим организациям. Точная форма таких взаимоотношений часто носит более общий характер, чем просто иерархия, поскольку возможно сложное матричное подчинение людей и организаций, связанных друг с другом сетевой структурой.

Решение

Партнер — это генерализация человека или организации. Рис. 8.8 показывает образец партнерства, моделирующий людей и организации, их адреса (местоположение), производственные отношения и отношения подчинения.

Партнер относится к другим партнерам с помощью отношения подчинения многие-ко-многим между множеством Начальников и множеством Подчиненных. Это совершенно общее отношение между партнерами, которое управляет любой разновидностью отношений подчинения. Специальное производственное отношение отделено, однако, как ассоциация между людьми и организациями. Партнер "размещается" в одном или более местах, каждое из которых имеет свое назначение (Место жительства, Офис, Производство и т. д.) и текстовый адрес (возможно, с шаблоном строк для почтовых пометок). В каждом месте может находиться любое число партнеров (людей и организаций).

Человек, имеющий какие-то данные представляет интерес для определенного домена, как и Организация. Эти стороны соотносятся как Сотрудники и Работодатели, а класс Работа, является классом ассоциации, обладающим временным периодом, в течение которого длится работа. Этот класс, в свою очередь, ассоциируется с классом Распределение должностей, определяющим ряд должностей на какой-то рабочий период. Каждая должность относится к данной организации (составная агрегация), которая отвечает за определение должности.

Обсуждение

Основная проблема этого образца — способ, с помощью которого он управляет контактной информацией. Телефонных номеров нет, и структура Места для адресов оставляет желать чего-то еще. Если нужно запросить всех людей, находящихся, например, в Сан-Франциско, то класс Место не поможет. Телефонной проблемой могут быть просто атрибуты объекта Человек или Организация или даже Сторона; к сожалению, в этом задействовано больше структур. Каждая сторона может иметь несколько контактных точек, включая несколько телефонов (домашний, рабочий 1, рабочий 2, сотовый, автомобильный), несколько адресов электронной почты, факсов и различных почтовых адресов наряду с их физическим местоположением. Обычно такие сведения имеют приоритеты — звоните сначала по такому-то номеру, затем используйте электронную почту и т. д. Образец партнерства их не контролирует.

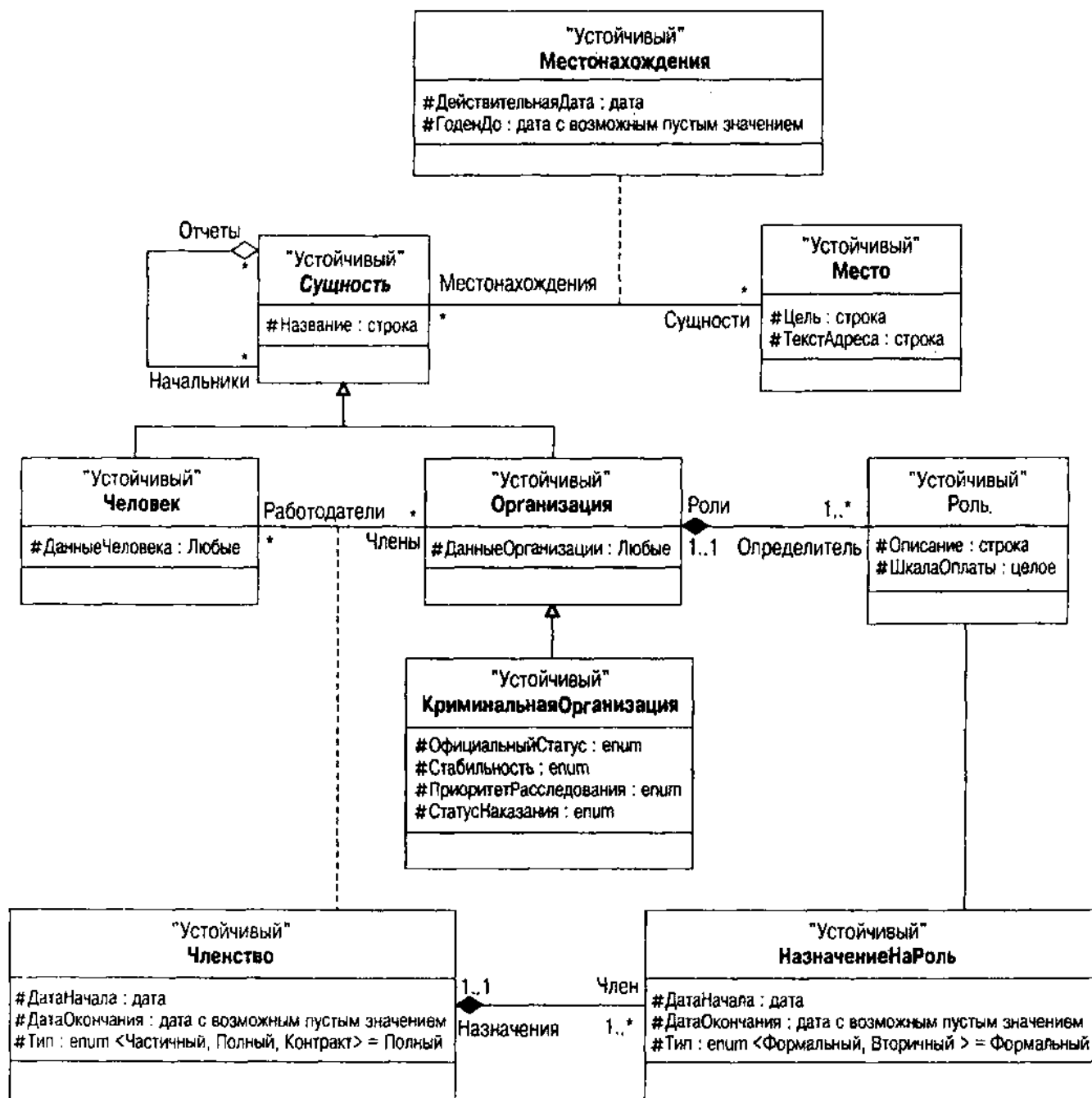


Рис. 8.9. Криминальная Организация как Партнер

Образец географического местоположения

Образец географического местоположения моделирует связи в географических областях.

Проблема

Многие из таких сущностей, как различные виды бизнеса, организации и правительства, требуют моделирования географического местоположения. Им нужно иметь возможность отслеживать все виды различных географических зон, часто по разным причинам. Географическое местоположение

предоставляет данные о странах, городах, штатах, графствах и других зонах с официальным статусом. Нужно отслеживать данные по опрашиваемым зонам, таким, как поселки и кварталы, обычно установленные налоговым инспектором графства, для взимания налогов или ведения записей о владениях. Им нужно отслеживать специфические типы зон, такие, как водные бассейны, нефтяные месторождения или другие залежи полезных ископаемых, зоны естественного обитания различных животных, а также данные об административных регионах. Все это выходит далеко за рамки местного управления, которое нуждается только в адресе. Большинство приложений требуется понять на достаточно глубоком уровне, как все эти места взаимосвязаны друг с другом.

Географические информационные системы (GIS) предоставляют прикладной интерфейс и структуру базы данных для этих видов приложений. Какой потребуется вид структуры БД, если разрабатывается свое собственное ПО?

Решение

Образец географического местоположения предоставляет базовую разбивку видов местоположений и способ взаимосвязи их друг с другом. Рис. 8.10 иллюстрирует этот образец. Частная версия образца отражает некоторые требования правительственного агентства землеустройства на третьем уровне, но категории второго уровня носят довольно общий характер.

Отношение многие-ко-многим между местоположениями отражают тот факт, что любое конкретное место может перекрываться любым другим, пространственно и политически. Например, правительственное агентство по делам коренных жителей Америки потребует сведения о географических зонах обитания индейских племен. Эти зоны перекрываются со штатами, графствами и другими геополитическими образованиями, а также с национальными парками и другими областями управления.

Обсуждение

Представление адресов с использованием географического местоположения выходит за рамки поставленной задачи. Можно представить местоположение как топографическую зону, город, штат, почтовый код и страну — т. е. географические места, имеющие связь друг с другом. Однако эта модель лучше представляет демографические данные или составные зональные данные управления, такие, как данные для правительственного агентства или горнодобывающей компании.

Другая проблема, связанная с этим образцом, — его относительно произвольная природа. Не слишком много размышляя, можно изменить классификацию географических областей несколькими различными способами, увеличив спектр множественного наследования и бесконечных усложнений. Это может быть экзemplар, где меньшее больше: возможно, следует моделировать местоположение как концепцию и дать модели данных подробную классификацию. Другими словами, используйте образец классификации для моделирования классификации и используйте образец местоположения для моделирования местоположения, затем объедините их для создания классификаций местоположений.

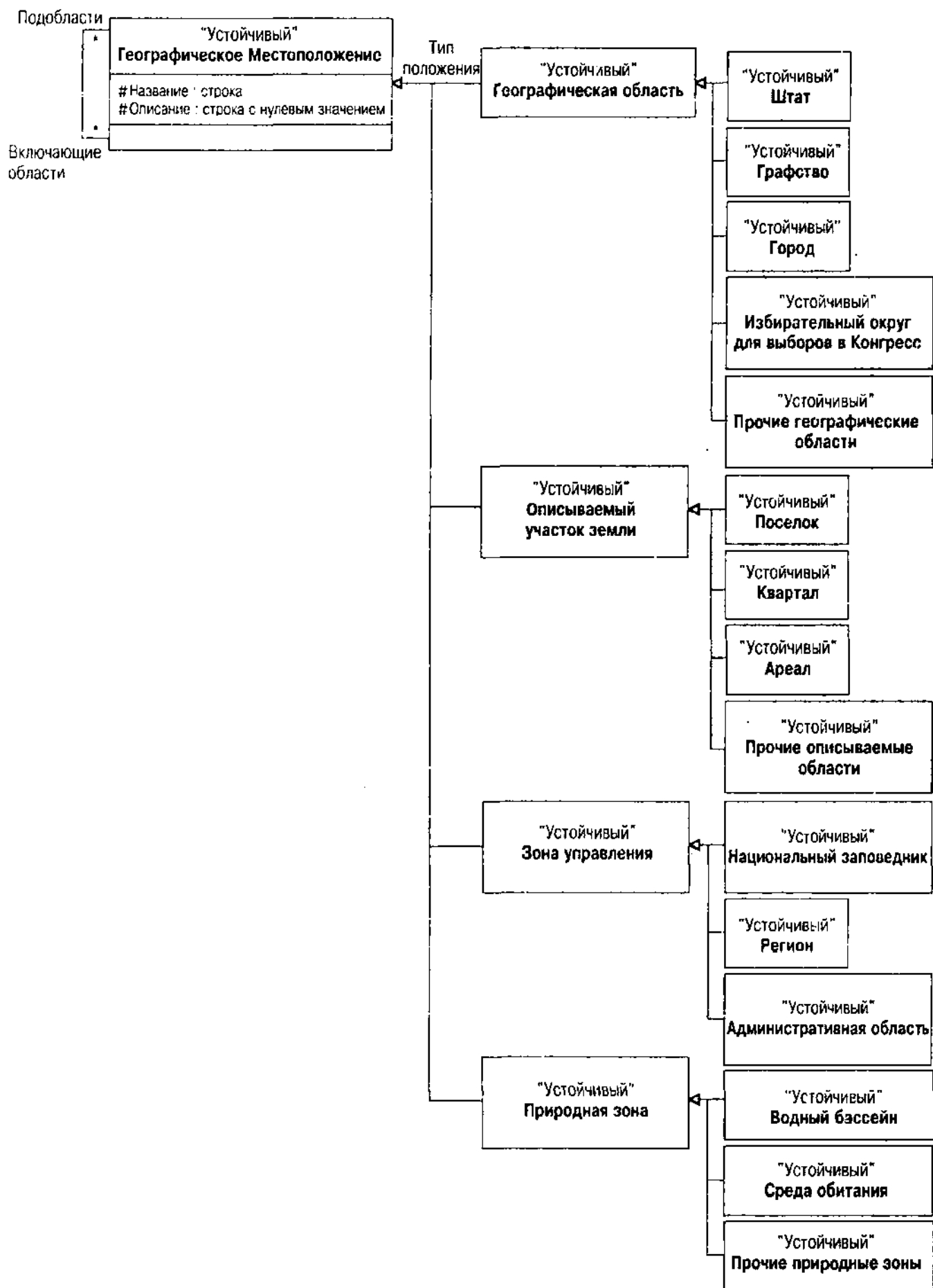


Рис. 8.10. Образец географического местоположения

Образец процесса

Образец процесса моделирует непрерывный производственный процесс. Большая часть производства подпадает под дискретные модели процесса, которые можно обычно найти в системах технологических процессов, когда входные объекты движутся вдоль траектории через обрабатывающие машины, превращаясь в новые выходные состояния. Непрерывное производство требует других видов моделей, потому что оно включает потоки материалов, таких, как нефть или химикаты, а не дискретные предметы.

Проблема

Бизнес-модели, моделирующие активы и взрыв частей, пригодны для моделирования производства автомобилей или других производственных процессов, производящих большие части из более мелких частей. При сравнении с непрерывными процессами на химическом производственном предприятии эти модели проигрывают. Они годятся для моделирования производства машин и труб, но не для моделирования процессов, в результате которых получается продукция. Для таких процессов конденция потока материала, в частности его скорость, — наиболее важна при моделировании. Кроме того, чтобы гарантировать качество с помощью планирования и оценки процесса, нужно различать действительные потоки и потенциальные. Следует оценивать то, что должно произойти, а также то, что происходит на самом деле.

Решение

Рис. 8.11 — довольно сложное представление образца процесса. Он моделирует целый ряд различных аспектов данной проблемы:

- Модели процесса (условия и потенциальные входы и выходы процессов)
- Требования к материалам процесса (продукты, потоки и составляющие)
- Структуры средств (средства, учет продукта, пути перемещения)
- Операции процессов (потенциальные и действительные потоки продукта через экземпляры процесса)

ВНИМАНИЕ

Исходный образец [Нау, 1996, р. 187—196] существенно отличается от представленного на рис. 8.11. По мере преобразования исходного образца автор столкнулся со многими проблемами, двигаясь от диаграммного стиля CASE-метода к ОО-стилю. В частности, моделирование отношений как сущностей очень затруднило эффективное преобразование образца Хэя. Поэтому автор был вынужден преобразовать образец в проект, более совместимый с ОО-концепциями.

Этот образец является примером подробного образца конкретного домена среднего уровня. Детали образца явно выходят за пределы того, что представляет абстрактный образец. Все же он носит общий характер и может быть применен к любому количеству отраслей, а не только к фирмам в определенной области. Можно видеть, как уровни детализации изменяются в зависимости от конкретной проблемы, проблем компании, проблем отрасли, типа проблем отрасли и общих проблем.

Данный процесс является также примером транзитивных свойств ассоциаций в программной системе. Рис. 8.11 имеет несколько ассоциативных классов, представляющих реализованные отношения между классами:

- *УчетПродукта*: Отношение между Продуктом и СредствомПроизводства; средство содержит учет продукта, который является смесью различных материалов.
- *Составляющие*: Отношение между ТипомМатериала и Продуктом, композиция смеси Продукта.
- *ПутьПеремещения*: Отношение между двумя Средствами производства; это буквально труба, соединяющая два средства, что позволяет продукту перемещаться от процесса к процессу.
- *Условие*: Отношение между МодельюПроцесса и набором Переменных; переменные управляют условиями, при которых могут происходить моделируемые процессы.
- *Действительный Поток*: Отношение между УчетомПродукта и ПутемПеремещения; действительный поток смеси материалов (продуктов) через трубы (пути перемещения).

Действительный поток — пример отношения на отношении. Действительный поток материалов от учета к процессу следует за отношениями между производственными средствами. Хотя абстрактная форма кажется сложной, можно легко видеть, как все это работает в реальном мире. Имеются смеси продукта в цистернах. Продукт перемещается из цистерны через производственные средства, которые используют процессы, управляемые моделями для преобразования исходного продукта в конечные продукты. Эти конечные продукты в свою очередь перемещаются в другие цистерны и потенциально становятся исходными для других процессов.

Обсуждение

Когда такие отношения начинают встречаться в ассоциациях, нужно подумать о возможности применения продуктов и схем OODBMS вместо RDBMS или даже схем ORDBMS. Приложение вполне определенно будет заниматься прежде всего навигацией по путям и потокам, а не запросами расширенных классов по типам сущностей. Навигационные качества множеств OODBMS вероятнее всего предоставят средства, необходимые для приложения. В этих системах обычно хотят следовать по трубам или потокам, а не извлекать о них табличные данные.

Ограничения по схеме такой сложности трудно определить правильно. Глядя на рис. 8.11, как можно, например, автоматически применить модель процесса к реальным процессам? Модель показывает, какие входы ожидает процесс, и какие выходы он создает, и условия, налагаемые на переменные, при которых он производит продукт. Соотнесение этого с производственными средствами означает знание, где находятся материально-производственные запасы, какие продукты могут перемещаться по каким путям и в какие емкости и т. д. Образец содержит базовые данные, но многие детали все же пропущены, такие, как расписание потоков, контроль за образцами и

оценка процесса, а также средства управления запасами. Образец процесса является началом в оценке отношений процесса, но он не предоставляет всего, что нужно для непрерывного производственного домена даже при самом богатом воображении.

Образец документа

Образец документа позволяет моделировать документы, включая структурированные документы и их пользователей.

Проблема

Проблема библиотекарей — бесконечная, как отметил один из самых знаменитых библиотекарей, мечтая (возможно) о бесконечном количестве шестиугольных комнат, слабо освещенных, с книгами, содержащими все возможные комбинации алфавита [Borges, 1962]. Проблема состоит в том, чтобы найти документ, содержащий необходимые данные, среди огромного количества имеющихся документов. По мере того как в издательском мире начинают использоваться для хранения новые носители, работа библиотекаря становится еще сложнее.

Решение

Образец документа позволяет описать вселенную документов, которые нужно хранить в библиотеке. На рис. 8.12 показан образец документа.

Первый аспект этого образца — моделирование типов документов. Вместо использования генерализации образец моделирует типы документов как отдельный класс с помощью образца метамодели. Так как имеется потенциально бесконечное количество видов документов с очень незначительными отличиями по структуре и поведению, это предоставляет способ классификации документов без усложнения модели разработки. В данном случае документ имеет единственный тип для целей классификации.

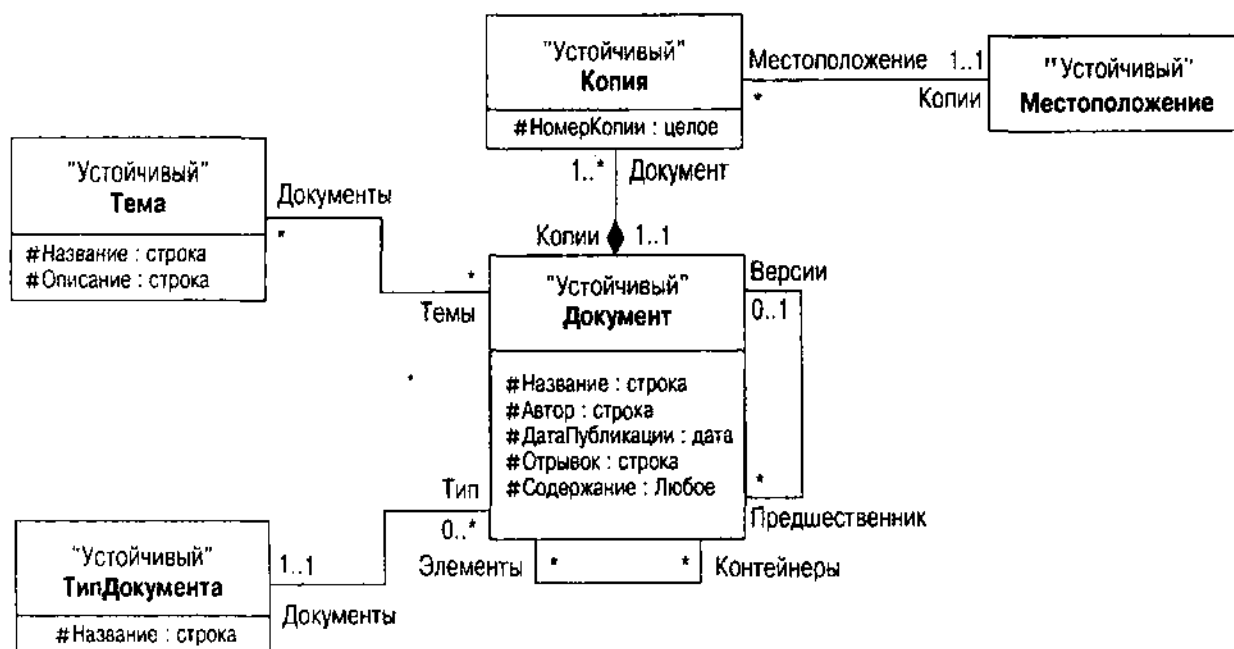


Рис. 8.12. Образец документа

Затем необходимо представить копии и версии документов. Документ — это абстрактная концепция документа; копия — реальный экземпляр: копия абстракции. Например, "Приключения Гекльберри Финна" Марка Твена — это документ; копия с загнутыми уголками страниц на третьей полке слева в комнате 23 — это копия документа. Ключевые элементы копии — это способ ее идентификации и определения местоположения.

Версия документа — абстракция, но она лучше самого документа. Имеется два вида версий: поправка и издание. Поправка добавляет второй документ, расширяющий первый и становящийся (обычно на законном основании) его частью. Издание заменяет документ исправленной версией. Продолжая пример с Марком Твеном, некто может опубликовать оригинальное издание с дополнительным вступлением, содержащим биографические и исторические материалы, образующие содержание. Другой пример — контрактный документ и последующая поправка к контракту, которая изменяет его условия после подписания контракта. В качестве примера издания — издатель решил выпустить исправленную версию "Приключений Гекльберри Финна", основанную на широких сверках и добавлениях к материалам первоначального издания книги. Схема предполагает, что можно создать множество разновидностей версий, но каждая разновидность основана на единственном исходном документе.

Образец документа позволяет как угодно ассоциировать темы с документами. Можно отыскать конкретную тему в любом количестве документов, а конкретный документ может содержать любое число тем. Структура предполагает, что документ не ссылается ни на какие темы. Это позволяет создать документ без его немедленной индексации; это также позволяет хранить документы, к которым нельзя получить доступ с помощью индекса тем.

Обсуждение

Очевидные приложения образца документа относятся к таким областям, как управление знаниями или смысловой поиск в Интернете и прочая индексная функциональность. Выходя за рамки такого индексирования и поиска, следующий шаг в управлении документами — управление потоком работ. Можно сделать добавление в образец партнера и соединить его через роли для демонстрации, как документы перемещаются от роли к роли в процессе выполнения работ. Одна такая роль может заменить атрибут Автор с помощью отношения с Партнером. Другие могут соединить Человека как текущего обладателя документа, человека, подписывающего документ, или любую другую подходящую роль. Можно также записать историю процесса как ряд записей принадлежности роли.

Можно расширить иерархию класса Документ наследованием, все еще продолжая использовать объекты ТипДокумента. Это предоставит две возможности для классификации документов, одну — относящуюся к структуре и поведению, а другую — являющуюся совершенно абстрактной категоризацией. Обе могут быть полезны в разное время для разных целей. Предположим, есть структурированная информация для документа, которую хотят использовать для поиска или для обработки данных. Вместо добавления расширенного мета-моделирования подчас более эффективно создание подклассов в БД для разных видов документов. Например, иерархия ИдентификационныйДокумент

может содержать данные, которые удобно использовать для целей выборки, или это могут быть документы с отпечатками пальцев с элементами специальной категории, полезными при быстром поиске записей.

Итоги

Образец разработки — это система для повторного использования, описывающая проектную проблему и ее решение. Общий образец содержит целый ряд элементов:

- **Имя**
- **Описание проблемы**
- **Описание решения**
- **Описание нерешенных проблем, компромиссов и последствий использования образца**

Абстрактный образец — образец классов и отношений, описывающий общее решение общей проблемы. В этой главе приведено несколько примеров абстрактных образцов моделирования данных:

- *Одноточечный*: гарантирует, что класс имеет только один экземпляр в базе данных
- *Составной*: представляет собой древовидную структуру, созданную из различных видов подходящих объектов, которые все используют один и тот же интерфейс; иерархия часть — целое или взрыв частей в БД
- *Легковесный*: использует совместное использование для снижения избыточности системы (нормализация)
- *Метамодел*: представляет сведения о данных, чтобы обеспечить гибкое и расширяемое моделирование данных

Аналитический образец является моделью конкретных, связанных с определенным доменом объектов, генерализованных для повторного использования в различных ситуациях. В этой главе приведено несколько примеров аналитических образцов:

- *Партнер*: объединенное представление людей и организаций (сущности)
- *Географическое. Местоположение*: представляет расширенное разнообразие физических мест
- *Процесс*: представляет поточно-ориентированный производственный процесс
- *Документ*: представляет широкое разнообразие документов и средств поиска

Абстрактные образцы и аналитические образцы улучшают моделирование данных. В следующей главе показано, как получить представление о том, насколько велика та лошадь, которую вы пытаетесь оседлать.

Глава 9

Критерии достижения успеха

О, могущественный Цезарь! Ты пал так низко? Разве поблекли все твои завоевания, слава, триумф, достижения, уменьшились они до этой малой меры?

Шекспир. "Юлий Цезарь"

Автор хотел назвать эту главу "Критерии успеха", но предпочел название "Критерии достижения успеха". Оценивание не является пассивным. Это активный и даже агрессивный подход для обеспечения успеха в конкретной ситуации. По своей сути оценивание – это обратная связь, процесс получения информации с целью придания работе определенной направленности для ее успешного завершения.

Во многих случаях определения критериев, в которых автор принимал участие, создавались пассивные критерии. Пассивные критерии собирают данные потому, что так поступают профессионалы, или потому, что это посоветовал консультант, или потому, что кто-то прочел какую-то книгу. Это превращает критерии в магический талисман, в амулет, который следует носить для защиты от злых духов. Но разработка ПО – это нечто другое.

Для создания агрессивных критериев нужно, во-первых, знать, для чего это делается. Какую обратную связь хотят получить? Нужно просто решить, не является ли подход слишком сложным? Надеются ли улучшить качество продукта путем оценки количества дефектов? Надеются ли поднять производительность? Пытаются ли понять эффективность внедренных систем для получения определенных результатов? Если невозможно указать определенную причину для создания критериев, то не следует этого делать, поскольку нельзя будет воспользоваться результатами.

Цели, показатели и шкалы

Оценить некоторые эмпирические аспекты реального мира можно с помощью выработанных показателей. В данном случае показатель — это средство оценки достижения провозглашенной цели, насколько хорошо разработанная система соответствует ее предназначению.

Трудно, но обязательно нужно разработать действительный показатель для определенной цели:

- Уметь оценивать цели
- Использовать правильный тип шкалы для цели данного типа
- Использовать систему оценок, которая позволяет делать значащие утверждения о состоянии объекта

Некоторые цели нельзя оценить даже с помощью хитроумных приемов. Неясные или плохо структурированные цели не позволяют измерить результаты. Если нельзя получить хороших эмпирических данных с помощью оценок и связать эти данные непосредственно с целью, нужно еще раз позаботиться о цели.

Шкала — отображение эмпирической системы в цифровую. *Проблема представления* возникает всякий раз, когда отношения между эмпирическими элементами — те же, что и отношения между цифровыми элементами [Roberts, 1979, р. 54–55]. *Проблема уникальности* состоит в том, насколько уникален этот гомоморфизм (отображение между элементами), т.е. какой вид преобразований можно применить к данным, не уничтожая представления. Теория измерений определяет стандартный набор типов шкал, которые идентифицированы по виду преобразования. В таблице 9.1 представлены стандартные шкалы и их характеристики.

Например, целью может быть определение истинности или ложности цели "представить криминальные организации". Будет использоваться номинальная оценка (1 или 0 для истины или лжи соответственно). Если цель связана с упорядочением, таким, как "упорядочить криминальную организацию по приоритету наказания", используйте порядковую оценку (1, 2 и 3 для высокого, среднего и низкого приоритетов соответственно). Цель с ненулевым значением, в частности "предоставить данные обвинителям в течение двух дней", должна использовать интервальную оценку, например дату получения данных. Цель определения количества ("достичь управляемой сложности схемы") нуждается в пропорциональной оценке (измеряемая оценка сложности схемы — число отношений, деленное на число классов). И, наконец, абсолютная цель некоторого рода ("подсчитать количество ограблений в Западном Суссексе") требует абсолютной оценки (общее число ограблений).

Совсем неправильно оценивать достижение цели, не имеющей измеримых характеристик. Например, продолжается спор об оценке интеллекта и, в частности, ее приложения к определению интеллектуального успеха отдельных людей. В предыдущем предложении нет слов, значимых самих по себе, и еще меньше слов, которые можно оценить. Может ли IQ быть

мерой интеллекта, а если да, то как? Эффективно оцениваемые цели должны быть тесно связаны с относительно общими вещами в реальном мире, такими, как усилие, время и деньги. Нельзя применять оценки, использующие определенный вид шкалы, к целям, требующим другой шкалы. Например, рассмотрите цель — доставить объект, затратив не более 10% предварительного оцененного усилия. Решено оценить эту цель с помощью метрики, которая оценивает усилие, затрачиваемое на объект, как "слишком маленькое, такое, как нужно, или слишком большое". Вы столкнетесь с проблемами, придавая цели какое-то реальное значение.

Таблица 9.1. Типы шкал измерений из теории измерений

Шкала	Характеристики	Пример
Номинальная	Возможно любое преобразование один-к-одному. Нет упорядочения чисел и нет возможности добавить, вычесть или связать два любых числа иначе, как только с помощью простого сравнения на равенство. Использует числа как метки или имена	Булевы значения (истина/ложь), категоризации (хорошо, плохо, все равно), альтернативные коды, такие, как WBS-числа или идентификаторы объектов
Порядковая	Возможно любое преобразование с сохранением упорядочения. Имеет значение только упорядочение, а не относительное расстояние между числами или их цифровое значение	Ранжирование (хорошо, плохо, лучше всего), оценки, твердость минералов
Интервальная	Линейные преобразования типа $f(x) = ax + b$ для $a > 0$. Упорядоченный интервал с произвольным начальным значением b и единицей измерения a , что позволит сравнивать интервалы между значениями шкалы	Температура (по Цельсию или по Фаренгейту), время
Пропорциональная	Линейные преобразования типа $f(x) = ax$ для $a > 0$. Значения шкалы уникальны относительно произвольной единицы измерения a , позволяя сравнивать сами числа. Шкала, однако, имеет фиксированное исходное значение (ноль)	Длина, масса, абсолютная температура и подобные физические величины; временной интервал; усилие; большинство разновидностей оценок, где имеется концепция отсутствующего (нулевого) значения
Абсолютная	Невозможно никакое преобразование	Счетчик объектов

Таблица 9.2 устанавливает несколько характеристик, которые можно использовать для оценки метрики [Osgood, Suci and Tannenbaum, 1957, p. 11; Henderson-Sellers, 1996, p. 33–38]. Чем большему числу критериев удовлетворяет метрика, тем она лучше.

И, наконец, оценка измерения. Следите за влиянием оценочных действий и улучшайте их, как и любую другую систему.

Ключевые области, требующие измерения, — размер модели, ее сложность, насколько она взаимосвязана (связанность) и насколько хорошо

разделены компоненты (соединение). В конечном итоге основное значение модели состоит в возможности повторного использования как самой модели, так и системы, построенной на ее основе. Последующие разделы обсуждают некоторые метрики, которые можно использовать для измерения этих аспектов модели данных.

Таблица 9.2. Критерии оценки метрик

Критерий	Определение
Объективность	Содержит ли метрика воспроизводимые данные? Могут ли различные собиратели данных оценить один и тот же объект с одними и теми же результатами?
Надежность	Содержит ли метрика одни и те же результаты при одних и тех же условиях в допустимых пределах измерения ошибки?
Действительность	Можно ли оценить данные метрики с помощью подобных данных, которые оценивают цель подобного же типа? Имеется ли другая оценка, которую можно использовать для предоставления гарантии того, что получатся действительные данные?
Чувствительность	Содержит ли метрика данные, способные различить отличия на уровне принятой цели?
Сравнимость	Можно ли применить метрику ко всем объектам, имеющим ту же самую или подобную цель?
Полезность	Можно ли собрать значащие данные по разумной цене и с разумными усилиями?

Оценки размера

По любым стандартам критической оценкой является оценка размера системы. Наиболее общая мера — количество строк кода — почти бесполезна для оценки систем с централизованной БД, поскольку код на самом деле — не наша забота. Для систем такого типа цель оценки — выявить, насколько велика база данных для предварительной оценки ресурсов памяти и насколько велика модель данных для предварительной оценки объемов работ.

Размер базы данных

Определение размеров БД — вещь сложная, даже когда оценивается реальная база данных. На этапе моделирования данных ресурсы еще более ограничены. Это особенно справедливо, если базовая технология DBMS еще не определена или если имеется несколько целевых DBMS для разрабатываемой системы. На этапе моделирования данных лучше всего получить преувеличенную оценку, основанную на количестве классов и среднем предварительно оцененном количестве устойчивых объектов в каждом классе. Можно получить это значение, рассматривая существующие системы или логическое определение системы для оценки, сколько объектов сможет создать система в определенный интервал времени.

Размер схемы

Оценка размера схемы — простая задача, не правда ли? Нужно лишь посчитать классы, вот и все! Возможно, но если собираются использовать метрику размера для предварительной оценки объема работ, то обнаружится, что не все классы созданы равными, по крайней мере, для этой цели.

Наилучшей общей метрикой размера схемы, которую смог найти автор, является балльная оценка функциональности. Функциональная оценка — это метрика порядковой шкалы (с некоторыми тенденциями к интервальным и пропорциональным шкалам), которая измеряет функциональность системы ПО. Здесь нет места для широкого описания функциональной оценки, поэтому за подробностями лучше обратиться к учебникам [Dreger, 1989; Garmus and Herron, 1996; IFPUG, 1994; Jones, 1991].

Поскольку имеются варианты использования, можно посчитать функциональную сложность системы в целом. Общее число заданных функциональных точек дает точную оценку размера ПО, включая свойства централизованной базы данных. При подсчете функциональных точек нужно разделить функциональность системы на пять подсчитываемых отдельно типов функций:

- *Внешние вводы (EI)*: Транзакции, которые оканчиваются вводом через границу приложения
- *Внешние выводы (EO)*: Транзакции, которые оканчиваются выводом за границу приложения
- *Внешние запросы (EQ)*: Транзакции, посылающие вводимые данные в приложение и получающие обратно вывод без изменения чего-либо в БД
- *Внутренние логические файлы (ILF)*: Данные, которыми управляет приложение
- *Внешние интерфейсные файлы (EIF)*: Данные, которые приложение использует, но которыми не управляет

Для определения размера схемы наиболее важны два последние счетчика. ILF — по большей части устойчивые классы в модели данных. Чтобы считать ILF, устойчивый класс должен находиться в пределах приложения; т. е. приложение должно создавать устойчивые объекты класса как часть его существенной функции. ELF, с другой стороны, — это классы, на которые ссылаются, но не поддерживают, т. е. приложение не создает и не обновляет устойчивые объекты, оно просто использует их после выборки из БД.

Чтобы подсчитать файлы ILF и ELF, нужна модель данных и варианты использования. Во-первых, разделите классы в модели в соответствии с тем, управляет ли ими приложение или просто запрашивает их. Например, в системе каталога большое количество классов представляет данные, которые приложение запрашивает для оперативников агентства Holmes PLC, не позволяя им что-либо менять. Сюда относятся такие ELF, как Человек и Криминальная Организация. Имеются некоторые классы, представляющие ввод оперативников в систему, например ПримечаниеПоДелу, в котором оперативники записывают свои отчеты по текущим делам. Это ILF. Большинство систем имеют много маленьких таблиц, которые поддерживает

администратор, в частности списки городов и штатов, коды типов данных и данные по безопасности. Это обычно ELF в большинстве приложений. Таблицы, критичные для данного приложения, — это обычно ILF, но не всегда, как в каталоге. Такие системы являются по большей части системами только для запросов.

СОВЕТУЕМ

Если модель данных согласованным образом помечает операции классов меткой {запрос}, следует добавить метку {запрос} к классу, чтобы указать, что он является в разрабатываемом приложении ELF, а не ILF. Это облегчит оценку функции.

Вторая часть функциональной оценки — определение сложности счетчика для данного элемента. Чтобы сделать это для двух файловых счетчиков, нужно определить типы элементов записей (RET) и типы элементов данных (DET) для данного файла. С точки зрения функциональной оценки сложность — весовой коэффициент, на который умножаются значения счетчиков для получения "неприведенной" функциональной оценки (приведем эти счетчики позже).

DET являются "уникальными распознаваемыми пользователем нерекursивными полями/атрибутами, включая атрибуты внешних ключей, поддерживаемые ILF и ELF" [Garmus and Herron, 1996, p. 46–47]. Они соответствуют индивидуальным атрибутам и отношениям в классах UML. Подсчитывают по одному DET для каждого атрибута/отношения; это значение не изменяется в зависимости от количества объектов класса или связанных объектов. Подсчитываются только атрибуты и отношения. Если в классе имеется отношение с явным OID, состоящее более чем из одного атрибута, считают по одному DET для каждого из атрибутов, а не просто по одному DET для отношения. Явный OID означает, что используется распознаваемый пользователем атрибут для идентификации объекта и, следовательно, каждый усложняет систему. Можно проигнорировать атрибуты, которые появляются только в связи с методикой реализации. Также следует свернуть повторяющиеся атрибуты, идентичные по формату, что является общим приемом реализации. Обычно это массивы или структуры (или встроенные классы); иногда для ясности понимания их разбивают на отдельные атрибуты (последовательности битовых флагов или группу полей месячного объема). Игнорируйте эти дополнительные атрибуты и включите в расчет единственный DET.

RET являются "опознаваемыми пользователем подгруппами (необязательными и обязательными) элементов данных, содержащихся в ILF или EIF. Подгруппы обычно представлены в диаграмме отношений сущностей как подтипы сущностей или атрибутивные сущности, обычно называемые отношениями предок — потомок" [Garmus and Herron, 1996, p. 47]. RET, таким образом, соответствует либо отношению генерализации, либо составной агрегативной ассоциации, либо рекурсивному отношению предок — потомок на диаграммах устойчивых объектов UML. Идея состоит в том, чтобы учесть один RET для одного объекта, при этом объект состоит из своего класса и своих суперклассов, а также объектов, которыми он обладает через агрегацию. В расчет принимается один RET объекта без генерализации или ассоциации агрегации либо учитывается один RET для каждого такого отношения.

В качестве примера из реального мира рассмотрим криминальную организацию на рис. 8.9, воспроизведенную здесь и помеченную для удобства читателей как рис. 9.1. Каталог предоставляет оперативные запрошенные данные о криминальной организации; другие части системы Holmes PLC осуществляют информационную поддержку. Следовательно, все классы, представленные на рис. 9.1, являются классами EIF или RET. В приложении ввода данных криминальной организации это будут ILF и RET. Определение RET/DET, однако, точно такое же. EIF появляются со стереотипом "EIF", а RET появляются со стереотипом "RET" просто для того, чтобы быть полностью совместимыми с миром нотаций UML. Счетчики DET показаны как помеченные значения DET на классах.

На рис. 9.1 имеется пять EIF с пятью RET и 29 DET. Однако не все так просто, потому что нужна ассоциация RET и DET с EIF, частью которых они являются. Это позволяет разложить сложный комплекс на составляющие EIF или ILF при подсчете функциональной оценки. Пример показывает тот вид решения, который следует принять.

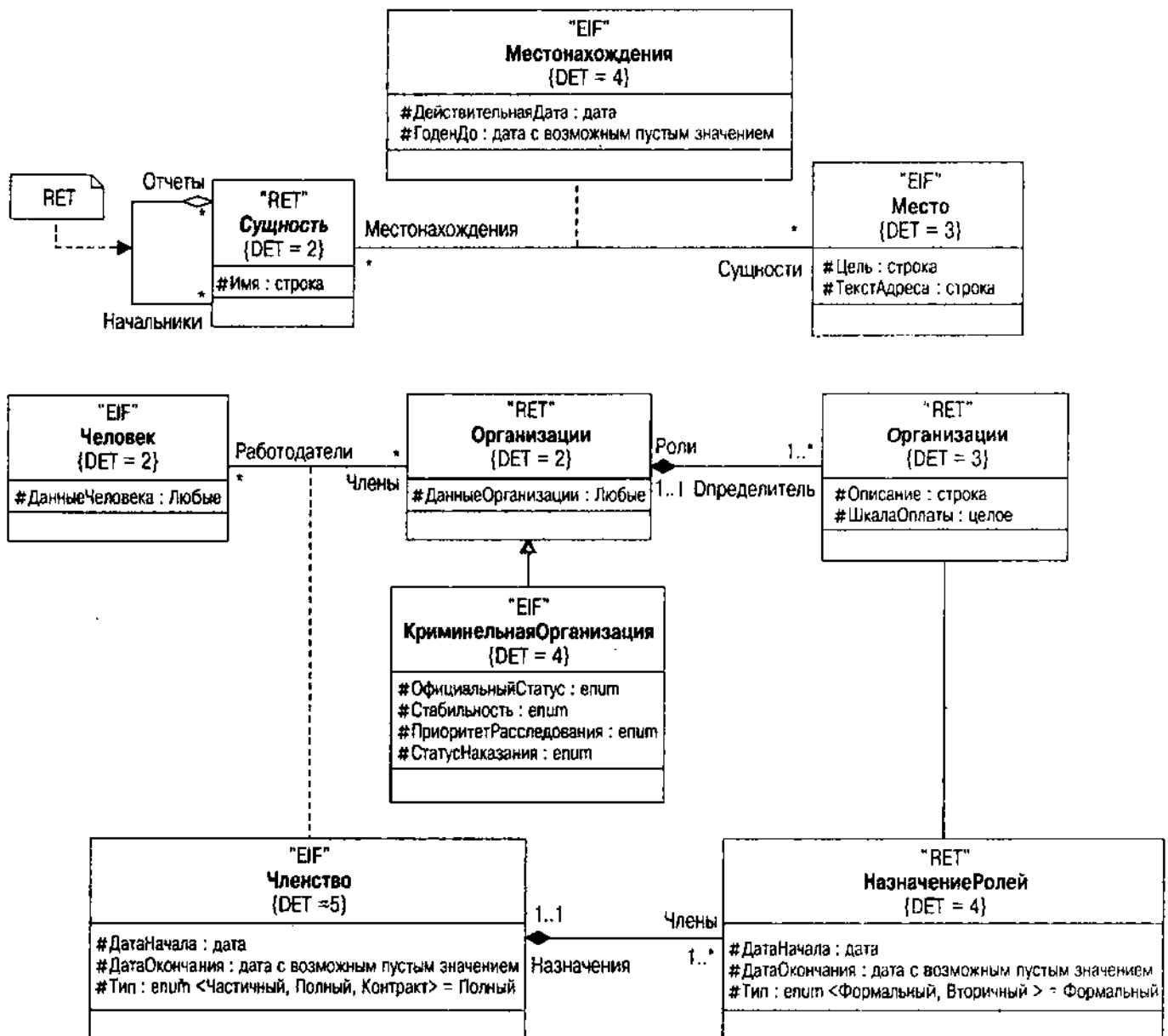


Рис. 9.1. Оценивание Криминальной Организации

ВНИМАНИЕ

Рис. 9.1 поясняет, как можно использовать нотацию UML совершенно другим образом, а данном случае для более простого расчета функциональной оценки. Если используются средства построения диаграмм, позволяющие реализовывать разные представления о модели, можно таким образом очень эффективно применить разнообразие стереотипы и метки. Однако такие средства трудно найти; автору не известно ни одного, которое бы в данный момент поддерживало этот вид моделирования.

Первое, что необходимо сделать, — решить, какой класс в иерархии классов использовать как EIF. В данном случае наше приложение сосредоточивается на Криминальной Организации, поэтому она становится первым EIF. Если расчет производится в отдельно разрабатываемой БД, начинают с корневого класса как EIF. Затем создаются подклассы RET. В данном случае создается класс Криминальная Организация EIF, а остальные будут RET.

Класс Сущность представляет собой второе решение. Он имеет рекурсивное отношение предок — потомок для демонстрации того, как сущности связаны друг с другом. Эта ассоциация становится RET.

Класс Роль представляет третье решение. Поскольку он связан с классом Организация как составной агрегат, он становится RET, а не EIF. Роль считается частью объекта Организация вместо того, чтобы представлять их по отдельности. Роль — это часть Организации.

И, наконец, классу Членство принадлежит класс НазначениеРолей так, что он в свою очередь является RET, а не EIF. И снова НазначениеРолей — это часть Членства (которое в свою очередь является отношением между Человеком и Организацией просто с целью все запутать).

Как происходит преобразование первичной функциональной оценки (в данном случае пять EIF) в неприведенную функциональную оценку? Используйте таблицу 9.3 для определения порядкового значения сложности для заданного числа RET и числа DET для заданного EIF.

Таблица 9.3. Порядковые значения сложности файлов

RET	1–19 DET	20–50 DET	51+ DET
1	Низкая	Низкая	Средняя
2–5	Низкая	Средняя	Высокая
6+	Средняя	Высокая	Высокая

Рассмотрим EIF по очереди. Первый — это Криминальная Организация, центр паутинки. Она имеет две генерализации RET, Организация и Сущность, ее суперклассы. Организация в свою очередь имеет RET Роли, создавая три RET. Оставьте НазначениеРоли ее владельцу, Членству. Сущность имеет RET для рекурсивного отношения предок — потомок. Таким образом, EIF Криминальной Организации имеет четыре RET и суммарно восемь DET из различных классов, что приводит к упрощению.

У Человека есть единственный RET, его суперкласс Сущность. Он сам имеет два DET и два для Сущности. Это снижает сложность. Не нужно

снова учитывать рекурсивное отношение предок — потомок, так как оно уже вошло в счетчик Организации.

Класс Место не имеет RET, что означает то, что один RET уже был учтен для самого класса. Он имеет три DET. Это снижает сложность.

Местоположение не имеет RET(1) и имеет четыре DET, два для своих атрибутов и два для внешних ключей класса ассоциации, что приводит к упрощению.

У Членства один RET (НазначениеРолей) и пять DET, три для атрибутов и два для внешних ключей класса ассоциации, что приводит к упрощению.

Для данных порядковых значений, подсчитывается неприведенная функциональная оценка при помощи таблицы 9.4.

Поскольку все порядковые значения сложности — низкие, число EIF умножается на пять, что даст 25 для неприведенной функциональной оценки (пять EIF, умноженные на коэффициент сложности пять). Приложение КриминальнаяОрганизация, которое использует эти классы, таким образом, имеет функциональную оценку 25, относящуюся к базе данных. Возможны, конечно, и другие функциональные оценки при расчете EI, EO и EQ для временных систем программного обеспечения; они вносят вклад в предварительную оценку трудовых затрат просто как вычисление данных.

Последний шаг в расчете функциональной оценки — корректировка итога "общими системными характеристиками". Начиная с этого момента, функциональные баллы становятся плохой метрикой. Приложение оценивается в соответствии со шкалой порядковых номеров для разных характеристик, таких, как связь данных, производительность, скорость выполнения транзакций, ввод данных в реальном масштабе времени, повторная использование, простота установки, множество рабочих мест и обновление в режиме реального времени. Каждому элементу присваивается оценка на шкале порядковых номеров от 1 до 5, затем они суммируются, чтобы получить суммарную степень влияния. После этого подсчитывается коэффициент корректировки значения путем умножения этого числа на 0,01 и добавления к нему 0,65. Этот расчет, к сожалению, совершенно сводит на нет действительность оценки, преобразуя ее из порядковой шкалы в шкалу пропорций, что приводит к бессмысленному дробному числу - 4,356.

К счастью, для большинства БД, использующих модель клиент/сервер, и предназначенных для реальной коммерческой среды со значительным количеством пользователей, коэффициент коррекции значения подходит удивительно близко к единице. Именно по этой причине автор рекомендовал бы использовать неприведенное значение функциональной оценки в качестве оценки размера.

Таблица 9.4. Множители для расчета функциональной оценки

Тип	Низкий	Средний	Высокий
ILF	x7	x10	x15
EIF	x5	x7	x10

ВНИМАНИЕ

Следует обратиться к документации IFPUG для выяснения подробностей подсчета приведенных функциональных оценок. Если ситуация, возникшая при разработке, существенно отличается от случая обычного приложения с коммерческой базой данных, может потребоваться корректировка счетчика функциональных баллов. Однако не вкладывайте слишком много смысла в точное значение числа, поскольку отменяется математическое значение шкалы из-за перехода к масштабу пропорции.

Оценка степени сложности

Функциональные баллы дают оценку размера информационной системы, но они немного говорят о том, насколько она сложна. Хотя и происходит корректировка функций по сложности, результат все же оценивается по размеру.

Большинство работ, посвященных сложности, относится к сложности программирования, но даже при этом проблема с термином — в его неясном смысле [Fenton and Pfleeger, 1997; Henderson-Sellers, 1996]. В контексте модели данных определенно непонятно, что имеется в виду, когда говорят, что хотят оценить сложность. Словарь определяет "сложный" как "сделанный из составных частей" или "запутанный", что в свою очередь означает "трудный, непонятный, замысловатый или комплексный". Сложность модели представляет интерес из-за имеющейся связи между этой сложностью и трудностью создания системы.

Первое, что нужно четко представлять, — отличие между сложностью проблемы и сложностью решения. При моделировании данных обычно моделируется проблема и поэтому сложность проблемы представляет больший интерес, чем сложность решения. Однако модель данных — первый шаг при разработке системы, и сложность разработки — это сложность решения. Потратив какое-то время на создание БД, очень быстро понимаешь, что имеются разные способы моделирования проблемы: одни более сложные, а другие — более простые. Что лучше, обычно зависит от используемой модели.

Сложность решения можно разбить с помощью ряда различных способов. Одно разбиение разделяет сложность на алгоритмическую (эффективность системы), структурную (архитектурные элементы и отношения) и познавательную (усилие, требуемое для понимания системы) [Fenton and Pfleeger, 1997, p. 245]. При другом подходе сложность разбивается на вычислительную (алгоритмическую), психологическую (познавательную, структурную и проблемную) и сложность представления [Henderson-Sellers, 1996, p. 51–52]. Как правило, исследования сосредотачиваются на структурной сложности, лишь некоторые посвящены сложности понимания. К сожалению, большинство этих работ относится к процедурной разработке, а не разработке БД.

В качестве примера рассмотрим наиболее часто используемую оценку сложности — цикломатическую сложность, или метрику Мак-Кейба [McCabe and Butler, 1989]. Эта метрика рассматривает модель процедуры как связный граф (граф, в котором ко всем узлам можно получить доступ из любого другого узла):

$$V(G) = e - n + 2$$

где e — количество ребер в графе; n — количество узлов в графе; 2 — связано с количеством путей в сильносвязанном графе. $V(G)$ оценивает число линейно независимых путей по графу G [Fenton and Pfleeger, 1997, p. 293]. К сожалению, имеются существенные ограничения в метрике как мере сложности и она применима к процедурной структуре, а не к структуре данных.

ВНИМАНИЕ

Если требуются средства, позволяющие применить оценку Мак-Кейба к ряду языка программирования, обратитесь к Web-странице McCabe Associates www.mccabe.com/vqt/.

Нет общепринятой оценки сложности схемы БД. Тем не менее разработчики баз данных интуитивно понимают, когда они усложняют жизнь своими разработками. Лучшее, что здесь можно сделать, — это использовать лезвие языка программирования параллельных процессов ОССАМ, превращающее сложность в простоту: как можно больше упрощайте модель данных. Это не означает наличие меньшего количества классов; можно объединить множество классов в один класс и получить гораздо более сложную систему, поскольку результат сложнее использовать и поддерживать. Другой тест — "проверка дураком": объясните модель любому "чайнику" (понимающему UML) посмотрите, где он не сможет понять, что происходит. Проблема "проверки дураком" состоит в предположении, что автор модели ее понимает достаточно хорошо, чтобы сформулировать вопросы, восполняющие недостаток знаний в тех частях, которые делали другие люди, что по меньшей мере является довольно субъективным предположением. Конечно, это грубоватый способ.

Оценка степени связности

Имеются два типа связности, интересующие разработчиков БД: абстрактная связность классификатора и структурная связность множества связанных классификаторов. Важно осознавать, что связность — центральный аспект повторного использования. Чем более связны классы, тем вероятнее, что можно их повторно использовать при разных обстоятельствах. В разделе "Потенциал повторного использования" подробно рассматривается эта проблема.

Абстрактная связность

Связность индивидуального классификатора (класса, интерфейса, пакета, подсистемы) — это "степень, в какой его индивидуальные компоненты необходимы для выполнения определенной задачи" [Fenton and Pfleeger, 1997, p. 312]. Классификатору модели данных требуется некоторая модификация этого определения: степень, в которой ему необходимы индивидуальные компоненты для поддержания своей *абстракции*.

Модель данных как статическая структурная модель UML — это набор абстрактных классификаторов и отношений между ними. Следовательно, нужно иметь возможность использовать стандартные оценки абстрактной связности для оценки связности разрабатываемой модели. Одна такая

оценка — это *оценка отсутствия связности* [Chidamber and Kemerer, 1993]. Эта оценка связывает попарно методы класса с его атрибутами. Нужно систематически сравнить все пары методов и классифицировать их как хорошие или плохие: хорошая пара функции члена использует по крайней мере один атрибут или отношение класса. Метод не сравнивается сам с собой, а также не различаются пары, основанные на порядке операций в паре.

Самая большая проблема с такой оценкой связности состоит в том, что она требует хотя бы частичной реализации методов системы. Можно это оценить на этапе моделирования данных, составив список атрибутов или отношений, которые, видимо, будет использовать метод, реализующий операцию. Это оценка до тех пор, пока не начнется кодирование. Большинство примеров в последних двух главах даже не имеют определенных операций, поскольку они не критичны для моделирования данных.

Другая проблема с этой оценкой — подчеркивание абстракции как концепции данных, а не как семантической концепции. На самом деле нужно оценить, насколько хорошо класс связан концептуально, насколько взаимосвязаны различные компоненты по отношению к значению класса. До сих пор не существует непосредственного способа оценить это. Использование интуиции разработчика может быть лучшим способом оценки связности класса. Являются ли атрибуты и отношения значащими аспектами некоторой базовой моделируемой вещи? Если кажется, что можно опознать кластеры атрибутов, относящиеся к разным вещам, то можно попытаться разбить класс, чтобы посмотреть, могут ли разные компоненты работать как отдельные классы. Часто это приносит успех при создании иерархий наследования, поскольку разделяются индивидуальные элементы, применяемые только при определенных обстоятельствах. Классический пример этого — неопределяемое пустое значение, когда атрибут является null по определению при некотором логическом условии, принимающем значение true. Создание подкласса и перемещение в него атрибута зачастую ликвидирует необходимость делать атрибут пустым. Это хороший пример абстрактной связности.

Структурная связность

Если считать набор связанных классов системой, включающей классы и отношения между ними, можно подумать о непосредственной связности всей этой системы, а не просто о связности индивидуальных классов. Такая структурная связность является степенью, с какой объектная структура приносит значащие результаты при запросах.

Зависимость — это импликационное ограничение на набор атрибутов. *Функциональная зависимость* — это импликация с единственным атрибутом: если известно значение атрибута А, тогда известно и значение атрибута В. *Многозначная зависимость* — это ограничение по двум атрибутам: если известно значение А и В, тогда известно значение С. В общем, *объединенная зависимость* — это ограничение на n значений. Зависимости являются основой процесса нормализации в реляционных БД (см. главу 11).

Нормализация гарантирует следующее:

1. Можно обновлять или удалять значение в таблице с помощью единственной команды
2. Можно объединять любые таблицы без потери данных

Эти гарантии также являются хорошим определением структурной связности, по крайней мере до той степени, до какой они применимы к лежащим в их основе структурам. Проблема с использованием строгой нормализации как меры ОО-связности набора классов состоит в том, что ее формальные требования являются слишком строгими для очень гибкого объектно-ориентированного мира. Например, первая нормальная форма требует, чтобы все атрибуты имели единственное значение, в то время как многозначные атрибуты, в частности массивы, структуры и вложенные классы, возможны в ОР-и ОО-системах. Операция объединения не имеет смысла на таких атрибутах, поскольку она зависит от оператора сравнения с единственным значением. То есть нельзя объединить два набора объектов на атрибуте массива; все представление бессмысленно.

Однако можно выдвинуть довольно веский аргумент о том, что БД с такими конструкциями трудна для использования, если язык запросов содержит оператор объединения. Если используются массивы, например как атрибуты постоянного класса, и проектируется реляционная БД, то, возможно, возникнут проблемы с использованием ANSI SQL для формирования запросов этих данных. Поскольку реляционные БД не имеют концепции массива, то, вероятно, возникнут проблемы с возможностью отображения разработки UML в реляционную схему.

С учетом этого факта ясно, что если только целевая модель данных схемы не является ОР- или ОО-моделью, не следует использовать структурированные данные в создаваемых классах. В противном случае сократится структурная связность. Вместо этого нужно разбить структурированные данные на отдельные классы и использовать отношения для представления многозначной структуры. Это решение, конечно, непосредственно отображается в нормализацию.

Таким образом, можно оценить структурную связность системы по пятибалльной порядковой шкале:

- *Первая нормальная форма:* нет классов, содержащих атрибут структурированного типа данных.
- *Вторая нормальная форма:* нет классов, содержащих атрибут, не полностью зависящий от OID класса.
- *Третья нормальная форма:* нет классов, содержащих функциональную зависимость, не принадлежащую полному OID класса (это фактически нормальная форма Бойса-Кодда).
- *Четвертая нормальная форма:* нет классов, содержащих многозначную зависимость, не принадлежащую полному OID класса.
- *Пятая нормальная форма:* нет классов, содержащих объединенную зависимость, не принадлежащую полному OID класса; другими словами, объединение любых таблиц не приводит к потере информации.

Использование статуса нормализации в качестве посредника связности обнадеживает, но это определенно ограничивает интересные способы разработки. Под "интересными" автор понимает способы, склонные вызывать споры среди знающих людей. Можно встретить тех, кто непреклонно верит, что "хорошая" разработка БД не имеет ничего общего с нормализацией, и тех, кто верит, что без преобразования данных, по крайней мере в третью нормальную форму, разработка является злом и исключительно дегенеративна. Также встречаются и те, кто рассматривает нормализацию независимо от ОО-разработки [Blaħa and Premerlani, 1998, p. 273]. Автор предпочитает думать о нормализации в основном как об оценке связности. Автор старается использовать связность как логическое обоснование для принятия решений, но это не ограничивает его применение здравого смысла к разработке моделей данных. Автор полагает, что используя методы на базе сущностей или классов, таких, как UML, разработчик имеет тенденцию получить полностью нормализованные базы данных в пятой нормальной форме потому, что они имеют тенденцию отражать семантически значимые объекты и отношения.

Оценка степени соединения

Соединение — это "степень взаимозависимости между модулями" [Fenton and Pfleeger, 1997, p. 310; Yourdon and Constantine, 1979]. Теория разработки говорит, что легче модифицировать и повторно использовать системы, минимизирующие соединение. Йордон и Константин определяют шесть видов соединения между двумя модулями на порядковой шкале. Эта оценка ссылается на способ, в котором модуль использует данные другого модуля, что (в обычной программе) либо происходит через непосредственную ссылку, либо с помощью передачи данных в стек.

1. *Нет соединения*: два модуля не зависят один от другого.
2. *Соединение данных*: два модуля передают данные (не управление) через параметры в методах.
3. *Матричное соединение*: два модуля передают данные одного и того же структурного типа через параметры методов (они оба зависят от третьей структуры).
4. *Соединение по управлению*: один модуль передает данные, управляющие точкой принятия решения в другом модуле (управляющие данные, или флаговые параметры, проверяемые в условном операторе перед тем, как что-то сделать).
5. *Общее соединение*: оба модуля ссылаются на общие глобальные данные.
6. *Соединение через содержимое*: один модуль непосредственно ссылается на данные внутри другого.

Преобразование этих порядковых значений в мир БД довольно прямолинейно. Ведь база данных предоставляет больше связей. Можно сослаться на данные через ассоциации, через генерализацию и наследование или с помощью ссылки в оперативном коде (SQL, OQL или процедурный код в хранимых процедурах или методах).

Нет соединения — в базе данных означает, что данный класс не имеет никаких ассоциаций с другими классами и не наследует от другого класса. Методы класса никоим образом не ссылаются на данные в других классах. К ним можно обратиться только с помощью вызова методов других классов, выполняя хранимые глобальные процедуры, которые ссылаются на данные в других классах или непосредственно через SQL или ссылки переменных.

Соединение данных ограничивает доступ к значениям устойчивых данных (не объектов, а просто примитивных типов данных) с помощью инкапсуляции их всех в методы. Класс, который обращается к таким данным через запросы метода, а не с помощью ассоциации, наследования, непосредственной ссылки и любых других объектных данных, демонстрирует соединение данных с другим классом.

Класс, получающий доступ к структурированным данным (типам записей, массивам или объектам) через методы проявляет *матричное соединение*. В реляционной БД это означает доступ к данным через хранимые процедуры, возвращающие курсоры или структурированные данные (типы записей или некоторые другие структурированные объекты). В ОР базе данных матричное соединение означает наличие методов, передающих или возвращающих объекты, например просмотр объектов в Oracle8. В объектно-ориентированной БД матричное соединение может дополнительно означать использование логических структур OQL, которые ссылаются на объект. Так, можно иметь выражения, возвращающие набор или неупорядоченную совокупность объектов, или выражения, которые имеют функции, использующие аргументы типов класса.

Соединение по управлению по определению является процедурным, а не ориентированным на данные. Соединение по управлению в языковых операторах имеется в той степени, в какой язык БД поддерживает процедурное управление. Например, в реляционной базе данных коррелированный подзапрос демонстрирует соединение по управлению. Именно здесь вложенный SELECT обращается за свои пределы к значению во внешнем SELECT (параметр), чтобы ограничить его возврат. И еще более непосредственно это проявляется в создании параметризованного оператора SQL, использующего параметр в разделе WHERE, или при создании оператора SQL с применением данных, передаваемых параметром во временном методе, — оба примера демонстрируют соединение по управлению. Булевы флаги в методах все еще являются самыми общими случаями применения соединения по управлению в ОР- и ОО-базах данных.

Итак, если создаются булевы флаги в таблицах или классах, будут ли они использованы для принятия решений по управлению или они просто представляют данные истина/ложь? Если справедливо последнее, нужно рассмотреть подкласс для представления различных вариантов управления. Например, возьмите Криминальную Организацию в каталоге. Можно

добавить булево значение, чтобы указать, активна или нет эта организация. Если целью является представление данных для выполнения запросов, это подходит. Если нужно поддерживать метод, возвращающий другое значение (т. е. тот, кто вызывает его, устанавливает флаг), то это соединение по управлению. Та же самая логика применяется к неверным пустым значениям — пустым значениям, представляющим не пропущенные данные, а неверные атрибуты. Разбиение атрибутов в отдельной таблице ликвидирует пустые значения и соединение по управлению, которое для них требуется.

Иной способ взглянуть на это — решить, является ли флаг частью состояния объекта или он представляет другой тип объекта. Лучший способ принять решение — подумать, что случится, если поменяется значение флага. Нужно ли объекту превращаться в другой объект? Если да, это точно указывает на то, что флаг представляет состояние. Когда создан объект, обычно не следует преобразовывать его в другой объект в связи с некоторой задачей управления в системе. Разработка будет лучше, если изменить внутреннее состояние объекта с помощью внутреннего флага.

Общее соединение относится к использованию глобальных данных. В некотором смысле использование БД вообще означает, что берутся глобальные данные и, следовательно, уровень соединения поднимается до общего для любого класса, который ссылается на данные в БД. Глобальные данные — это данные, которые видны всем частям системы. Чтобы получить доступ к данным в БД, нужно подключиться к серверу БД, затем использовать SQL, OQL или некоторый другой механизм доступа для манипуляции данными. Наличие небольшого контроля видимостью — часть механизма безопасности БД, а не программного механизма видимости.

ВНИМАНИЕ

Было бы интересно разработать средство в RDBMS или ORDBMS для обеспечения управления видимостью только по этой причине. Приложение могло бы объявить видимость определенной схемы, снизив взаимодействие до взаимодействия двта/метка. Автор часто использует механизмы безопасности для достижения этого эффекта с помощью разбиения интегрированной схемы на подсистемы, сохраняемые в различных областях (пользователи, роли, базы данных, авторизация или все, что может создать система безопасности в определенной DBMS). Затем автор использует механизмы безопасного доступа, обычно либо привилегированные разрешения, либо связи БД, чтобы обеспечить выборочную видимость. Это ограничивает глобальное влияние, но это не очень переносимо и требует некоторого участия администратора БД со стороны сервера. Другой подход, применяемый автором, — инкапсуляция таблицы в хранимых процедурах или в случае PL/SQL — в пакетах. Автор прячет таблицы у пользователя, которому принадлежит все, но предоставляет доступ только к процедурам, а не к базовым данным.

Для реляционной схемы нет особенного выбора, кроме доступа к данным глобально. К сожалению, ОР-схемы по большей части не инкапсулируют свои данные, поэтому доступ к данным также осуществляется глобально. ОО-схемы предоставляют полностью инкапсулированный доступ к данным, поэтому в объектно-ориентированной БД имеется полный контроль видимости.

Аналогично, с помощью соединения через содержимое происходит непосредственная ссылка на данные, а не ссылка только через методы. Реляционная и объектно-реляционная БД не инкапсулируют данные, и SQL ссылается на них непосредственно. Можно использовать представления и хранимые процедуры для эмуляции инкапсуляции в реляционных и ОР-системах. Представление — это способ инкапсуляции доступа к схеме с помощью создания хранимого оператора SQL, который выглядит как таблица, по крайней мере для целей выборки данных. Аналогично можно создать хранимые процедуры (в Oracle — хранимые пакеты), которые эмулируют классы и методы с некоторыми ограничениями, уже обсужденными в главе 7. При работе с объектно-ориентированными БД никогда не следует делать базу данных непосредственно видимой, а нужно создать операции доступа и мутации, предоставляющие доступ, ограничивая таким образом соединение соединением дата/метка.

Соединение непосредственным образом связано с возможностью поддержки и повторным использованием. Чем ниже уровень соединения, тем легче поддерживать систему и тем вероятнее, что можно будет ее повторно использовать при разных сценариях. Чем менее сильно соединение классов, тем меньше влияние изменений любого данного класса на остальную часть системы.

При ОО-разработке имеется три специфических способа ограничить соединение.

Во-первых, обратитесь к инкапсуляции. Если атрибуты общедоступны, то другим классам разрешено любое соединение по содержанию, это самый худший вариант. Если класс изменяется, все классы, использующие его, также должны измениться. Чем больше можно скрыть, тем лучше. Например, не просто создавайте методы получить/ задать для всех атрибутов подряд. Строго говоря, это лучше, чем соединение по содержанию (это соединение по дате или метке), но также обнаружится, что изменение внутренней структуры без изменения интерфейса произвести довольно сложно. Создайте интерфейс, который имеет значение, а не просто раскрывайте внутреннюю структуру. Все это легко запомнить с помощью закона Деметра: любой объект должен иметь непосредственный доступ только к объектам, непосредственно связанным с ним через ассоциацию или генерализацию [Lieberherr, Holland and Riel, 1988].

В терминах устойчивости это означает, что следует инкапсулировать атрибуты данных как можно больше. Если при кодировании используется SQL или OQL, инкапсулируйте их в соответствующие классы. SQL, в основном по определению, — программный язык глобальных переменных совсем без инкапсуляции. Это естественно приводит к программному стилю, использующему общее соединение и соединение по содержанию. Как только что упоминалось, имеются обходные способы применения хранимых процедур и функций для инкапсуляции таблиц и также можно использовать представления для ограничения видимости. Расширенный OR SQL лучше, а OQL еще лучше, в нем дается более непосредственная методика инкапсуляции с помощью интеграции операций и методов в язык программирования.

Во-вторых, используйте наследование умеренно. Если подкласс непосредственно ссылается на член суперкласса, это соединение по содержанию. Если изменяется суперкласс, нужно изменить все подклассы, использующие его. Если можно, сделайте члены данных скрытыми и используйте функции членов суперкласса для доступа к средствам суперкласса.

Устойчивое наследование сложнее контролировать. Где возможно, избегайте объединений супер- и подклассов; вместо этого разбейте доступ к таблицам на логические части и поместите эти части в супер- и подклассы во временном коде. Передавайте данные из одного результата запроса в запрос супер- или подкласса. Это сокращает общее соединение до соединения данных. В ОР- и объектно-ориентированных БД используйте объявление скрытого доступа для максимального ограничения доступа.

В-третьих, избегайте соединения по управлению, используя, где возможно, полиморфизм. Соединение по управлению — это когда передаются данные для осуществления некоторого выбора передачи управления в коде. Передача булева флага — наиболее распространенный пример. Обычно можно избежать этого вида управления с помощью создания подклассов и определения виртуальных или полиморфных функций в качестве альтернативного варианта. Однако имеются некоторые обстоятельства, когда все это вызывает трудности.

Например, рассмотрите систему преобразования валют. Можно создать иерархию классов с одним классом для каждой поддерживаемой валюты, добавляя методы преобразования для каждой новой валюты и общий набор виртуальных методов для сложения, вычитания и т. п. С помощью двойной диспетчеризации можно создавать произвольные выражения валют, которые всегда возвращают правильный результат вне зависимости от того, какая и где валюта используется. В операции и в концепции это выглядит прекрасно. На практике один вид соединения заменяется другим.

Альтернативный вариант — наличие единственного класса преобразования валют, имеющего таблицу преобразований. В него передается количество и флаг валюты (управляющие данные), а обратно получают количество в долларах или в другой общей валюте. Можно воспользоваться методом, при котором передаются доллары и другая валюта, а обратно получают количество в интересующей валюте.

Второй подход определенно находится на уровне соединения по управлению. Функция использует флаг валюты, чтобы решить, где нужно искать множители преобразования во внутренних таблицах (или в БД). Первый подход избегает этого, распределяя таблицу по иерархии класса. Однако теперь каждый класс знает о другом классе, что минимально является матричным соединением, а максимально — соединением по содержанию, если используется дружественный доступ или какой-то другой непосредственный механизм получения внутренних данных. Матричное соединение лучше, чем соединение по управлению; однако в этом случае создан монстр путем образования дюжины классов, взаимодействующих друг с другом с помощью матричного соединения. Добавьте новую валюту, и потребуется совершенно изменить все классы в системе валют. Второй подход, соединение по управлению, определенно предпочтительнее.

В БД это преобразуется в подобные структуры. Таблица преобразования валют предпочтительна для распространения данных о преобразовании среди многих отдельных таблиц, по одной для каждой валюты. Разработчик реляционных БД никогда не стал бы думать, лучше это или хуже.

Потенциал повторного использования

Повторное использование — это возможность встроить БД за пределами задачи или прикладной системы, для которых она создавалась и в которых она работала. Потенциал повторного использования — это вероятность, с которой БД будет повторно использоваться. Он зависит от трех факторов [Goldberg and Rubin, 1995; Karlsson, 1996]:

- *Собственная способность к повторному использованию*: Вероятность повторного использования, обусловленная внутренними свойствами, такими, как завершенность БД и ее программного интерфейса или доменной модели с учетом вариантов использования или отсутствия соединения элементов БД с другими элементами (степень инкапсуляции), а особенно с элементами в других БД или во временных классах, образующих прикладную систему.
- *Доменное повторное использование*: Вероятность повторного использования БД благодаря тому, что ее содержимое и структура будут удовлетворять *требованиям* новых систем; другими словами, это вероятность того, что подобные требования (домены) возникнут в будущем.
- *Организационная повторная используемость*: вероятность повторного использования, обусловленная *организационными* факторами, например наличием средств репозитория, связи и правил повторного использования.

Относительный вклад каждой вероятности потенциала повторного использования зависит от типа системы. Если система сильно доменно зависима, как, например, специализированная производственная БД, нужно присвоить больший вес возможности повторного использования домена. Если система меньше зависит от определенного домена и больше от собственной структуры, как ориентированные на схемы БД общих организаций и людей, нужно присвоить больший вес возможности собственного повторного использования. Вес повторной используемости организации зависит от общего влияния организационных факторов на повторное использование; если хотят сфокусировать усилия на организационных аспектах, присвойте этому фактору больший вес. Можно подсчитать оценку взвешенного потенциала повторного использования по формуле:

$$\text{Потенциал} = w_i P_i + w_d P_d + w_o P_o$$

где $w_i P_i$ — взвешенная вероятность собственного повторного использования; $w_d P_d$ — взвешенная вероятность доменного повторного использования; $w_o P_o$ — взвешенная вероятность организационного повторного использования. По умолчанию веса равны $1/3$, поэтому вероятности вносят одинаковый вклад, но можно варьировать веса, чтобы выделить один или другой тип вероятности, если это имеет смысл для данной системы, но в сумме веса должны составлять единицу.

Определенные характеристики методик ОО-разработки делают более общие виды повторного использования более простыми. Например, ударение на инкапсуляции придает особое значение внутренним свойствам, которые создают возможность собственного повторного использования. Наличие наследования делает более вероятным удовлетворение, по крайней мере, некоторых требований в будущем, даже если понадобится заменить одни характеристики и добавить другие. Сосредоточенность ОО-разработки на моделирующих доменах приводит к высокой степени доменной повторной используемости в БД. Со связностью, вводимой при ОО-разработке, легче взаимодействовать и ее легче документировать, что приводит к более высокой организационной повторной используемости. Даже глобальной БД, такой, как реляционная база, разработанной с помощью методов ОО, легче придать смысл для организаций, которым требуются ее свойства.

При разработке повторно используемых БД следует рассмотреть потенциальных повторных пользователей и их требования. Как и для любого проекта разработки, можно никогда не идентифицировать всех требований или заказчиков, поскольку всегда возникает что-то новое. В такой ситуации можно применить общие принципы ОО разработки и схемы, делающие систему расширяемой и адаптируемой:

- Параметризируйте систему.
- Позвольте пользователям задавать варианты с помощью устойчивых хранилищ данных.
- Сделайте классы такими, чтобы их можно было легче повторно использовать через наследование.
- Не вводите слишком много допущений о том, что люди собираются делать.

Если вы создадите предпосылки для повторного использования, то они захотят повторно использовать вашу базу данных и ее конструкцию.

Сертификация повторного использования

Сертификация базы данных как повторно используемой требует правил сертификации для создания доверия системе. Доверие существенно для повторного использования; если системе не доверяют, ее не используют.

Во-первых, нужно установить уровень риска при использовании системы. Это означает сертификацию того, что система может выполнить свою задачу со степенью риска, находящейся в определенных пределах (допуск риска в системах повторного использования). Для этого используются соответствующие подходы управления риском, такие, как рецензирование разработок и тестирование БД с целью снижения риска до сертифицированного уровня. Повторный пользователь задает допуск риска, который в свою очередь определяет, можно ли повторно использовать систему с указанной степенью риска.

Во-вторых, нужно предоставить четкое описание целей и структуры системы. Обычно это требует публикации представления, миссии и рабочих целей компонентов БД. Эти атрибуты системы должны быть полезны для потенциальных повторных пользователей при определении, можно ли доверять системе в конкретной ситуации. Помимо миссии повторные пользователи также захотят знать, что делает система и для какого домена или доменов. Рабочие цели показывают повторным пользователям, как система достигает своей цели и как она оценивает успех, но могут быть также дополнительные компоненты системы, предоставляющие полезные сведения. В частности, опубликование модели данных и проекта схемы позволяет потенциальным повторным пользователям иметь полное описание БД. Кроме того, предоставление тестовой базы БД, если она есть, значительно увеличит уровень комфорта повторных пользователей.

В-третьих, нужно предоставить четкое определение ответственности за БД и ее компоненты: письменно обозначив систему как повторно используемую, предлагают что-то вроде "контракта" для повторных пользователей. Частью любого официального контракта является гарантия качества, количества, производительности или официальное название системы. Эта гарантия может варьироваться от обычной гарантии "как есть" (нет выраженной или имеющейся в виду гарантии, включая подразумеваемые гарантии торгового спроса или пригодности для определенной цели) до полной, явно выраженной гарантии выполнения некоторого обещания. Гарантия означает, что разработчик несет ответственность за проблемы с системой. Уровень гарантии часто определяет уровень доверия к системе со стороны пользователя. Например, если предоставляется повторно используемая схема БД "как есть", разработчик не несет ответственности за ее проблемы. Это почти гарантирует то, что никто не будет повторно использовать схему. Можно дать определенные гарантии некоторого уровня поддержки или предоставить повторному пользователю достаточно подробные сведения о том, как они сами смогут осуществлять поддержку, если захотят. Это обычно означает предоставление модели данных и всех сценариев или программ, которые нужны повторному пользователю для воссоздания БД. В таблице 9.5 перечислены некоторые стандартные гарантии ответственности.

ОСТОРОЖНО

Все это выглядит как бездушный официальный капкан, и на самом деле так оно и есть. Можно взять на себя определенные официальную ответственность и обязательства, если повторно используемое ПО станет доступно для других, даже в той же компании. Нужно обязательно посоветоваться с адвокатом о смысле гарантий и даже расширенной сертификации повторного использования, просто для того, чтобы знать об имеющихся рисках.

Сертификация должна определять для повторного пользователя точно, какую ответственность несет создатель системы и какие действия он примет и в каких временных рамках, если возникнет проблема. Повторный пользователь информируется о состоятельности разработчика и таким образом устанавливается третья планка доверия к системе. Нужно сказать пользователю, например, кто является экспертом по доменам и может ответить на вопросы о том, насколько система соответствует предполагаемым доменам. Нужно предоставить экспертов и специалистов по поддержке, чтобы система была работоспособной.

Сертификация БД повторного использования устанавливает доверие, но только для тех, кто сведущ. Если потенциальный повторный пользователь не знает о существовании БД и ее разработке, то на самом деле не имеет значения, насколько хорошо она сертифицирована. Разделяющий этап связывает наличие системы с ее потенциальными повторными пользователями. В центре этого этапа — репозиторий повторного использования, средство памяти, в котором хранится разработка. Обычно это определенный вид БД с набором ассоциированного браузера и средств поиска. Нужно классифицировать и индексировать повторно используемые системы для поддержки этих средств.

Но репозиторий — в лучшем случае пассивное средство коммуникации. Если хочется сделать ударение на повторном использовании как подходе для увеличения производительности, нужно активно продавать БД. Можно это сделать на конференциях по обмену информацией, на которых люди, знакомые с тем, что имеется, находят потенциальных повторных пользователей, интересующихся системами повторного использования. Можно проводить учебные занятия и обучать людей работе с системами, которые они хотят использовать. Обычно менеджеры проектов и архитекторы получают наилучшие результаты такого обучения. Можно также создать организационные структуры для передачи знаний. Например, обучить людей работе с тем, что имеется для повторного использования, затем назначить этих специалистов экспертами повторного использования в рабочие группы проектов. Сотрудники, знающие, что есть в наличии в репозитории повторного использования, будут перекрестно "опылять" рабочие группы.

Таблица 9.5. Системные гарантии для сертификации повторного использования системы

Гарантия	Описание
Производительность	База данных будет функционировать с определенной степенью риска и будет делать то, что заявлено в доменах, определенных для этого.
Титул и нарушение авторских прав	Создатель базы данных имеет право позволить повторным пользователям ее использовать, обычно с учетом прав интеллектуальной собственности или договоры о предоставлении права исключительного использования. Нужно дать гарантии пользователям, что они не нарушают авторских прав, например, используя планы тестирования.
Повреждение	Свойств системы, которые бы вывели БД из строя, не существует.
Совместимость	База данных будет функционировать в конкретной среде, определенной в описании системы. Например, можно предназначить проект или схему для использования только с определенной DBMS или сертифицировать ее для использования с любой DBMS, для которой существует ODBC-драйвер третьего уровня со специфическими расширенными свойствами. Нужно также оговорить ПО третьей стороны, требуемое для работы системы, например Microsoft Access, управляющие элементы OLE или любое другое ПО, необходимое системе для повторного использования.

Этап поддержки — это последняя задача, решаемая для систем повторного использования. Поддержка обеспечивает гарантии сертификации системы с помощью действующих специалистов. Нужно предоставить соответствующие услуги по поддержке для выполнения обязанностей. Обычно это включает некоторую проверку повторно используемой БД. Но на этом не стоит останавливаться; нужно доводить изменения до сведения повторных пользователей БД. Необходимо поддерживать список систем, повторно использующих разработанную БД, чтобы можно было информировать эти системы об изменениях. Модель систем повторного использования использует отношение класса системы для других систем с целью представления этого списка.

ВНИМАНИЕ

Может оказаться проще поддерживать БД, имеющую единственный экземпляр, повторно используемый многими приложениями. Именно это обещают промышленные базы данных. Однако разработчики могут захотеть иметь больше контроля над БД, чем у них есть, поскольку зачастую промышленные базы данных одевают настоящую смирительную рубашку на новое ПО. Сертификация всегда чревата потерями.

Итоги

Глава началась с обзора того, как преобразовать цели оценки в настоящие метрики или критерии, которые можно использовать для осуществления оценок. Чтобы сделать это эффективно, нужно понимать концепцию шкалы из теории измерений и знать, как оценить результат, используя несколько разных областей критерия оценки.

Специфические метрики оценивают разные аспекты БД и моделей данных:

- *Размер*: размер базы данных (количество классов и объектов) и размер схемы (функциональные баллы)
- *Сложность*: оценка сложности проблемы и решения, включая метрику Мак-Кейба и проверку дураком
- *Связность*: оценка того, насколько хорошо связана система; абстрактная связность и структурная связность
- *Соединение*: оценка степени взаимозависимости между модулями, включая тип соединения и логический горизонт
- *Потенциал повторного использования*: оценка того, насколько просто будет повторно использовать БД или модель данных

В этой главе был представлен ряд методов для оценки БД и модели данных. Теперь пора начать разработку настоящей схемы БД на концептуальном уровне, что будет предметом рассмотрения следующих четырех глав.

Глава 10

Выбор предшественников

Если бы люди могли усваивать уроки истории, какой это мог бы быть урок! Но страсть и веселье ослепляют наши глаза, и свет, который дает опыт — это фонарь на корме, освещающий только волны позади нас!

Сэмюэл Тейлор Колридж, 18 декабря 1831 г.

Большинство баз данных, созданных автором за 15 лет, не вышли целиком из его головы, как Афина из головы Зевса. Напротив, они появились из существующих систем, работ других программистов и на базе различных корпоративных культур, в которых он работал.

При движении от моделирования данных к разработке БД нужно прежде всего оценить влияние предыдущих проектировочных решений на проектируемую систему. Эти решения могли быть хорошими или плохими в своем контексте и могут быть одинаково плохими или хорошими в новом контексте, который предоставляется для их роста. Все это происходит в организационной культуре, сильно влияющей на принимаемые решения. Разработчик должен на основе имеющейся схемы (и, возможно, имеющихся данных) создать новую схему и базу данных. Раздел "Культура разработки программного обеспечения и унаследованная система" поможет преодолеть политические, культурные и технические опасности, с которыми вы встретитесь. Она рассматривает проблемы с заменой или повторным использованием унаследованных схем.

Когда начинают с нуля, то считают это невероятной удачей или катастрофической неудачей, в зависимости от ситуации. Зачастую легче начать с готовой схемы, чем пытаться представить (или того хуже — определить) природу реальности, которую необходимо смоделировать. Раздел "Новая разработка" рассматривает проблемы, возникающие при создании совершенно новой схемы, включая выбор технологии базы данных.

С особыми трудностями сталкивается разработчик, когда ему нужно начать с предшествующей БД, которая "неправильна" в каком-либо смысле. Возникают проблемы доверия и легитимности, они могут разрушить политическую поддержку, требующуюся для того, чтобы проект состоялся. Раздел "Черновая разработка" помогает преодолеть эти проблемы и дает некоторые подсказки для продвижения вперед.

Следующий раздел "Структура разработки схем" суммирует всю информацию о структуре, которая будет использована в следующих главах для представления специфических преобразований модели данных при разработке реляционной, объектно-реляционной или объектно-ориентированной схемы. Эта структура устанавливает рамки разработки схемы, которые могут быть использованы независимо от технологии целевой БД.

Последний раздел "Объединение представлений моделей данных" рассматривает интеграцию всех моделей в цельную модель системы. Когда это сделано, можно приступать к разработке схемы.

Культура разработки программного обеспечения и унаследованная система

Каждое ПО создается в контексте определенной культуры разработки. Природа культуры структурирует направление, в котором идет разработка и создание базы данных. Эта культура имеет большое влияние на ход работы с унаследованными системами.

События культуры

Культура разработки, вероятно, является системой с наиболее частым повторным использованием, которую можно встретить на рабочем месте. Она есть, ее нельзя проигнорировать, она не уйдет прочь, как и большая часть повторно используемого кода. Система культуры — это коллекция передаваемых социальных систем, помогающих в выполнении работы. Помимо простого процесса изучения технологии, важной задачей при вступлении в организацию является изучение того, как надо работать, чтобы стать частью ее культуры.

Во-первых, вот резюме из Антропологии 1А: имеется пять основных систем, образующих культуру организации, разрабатывающей ПО [Simons 1997; Hohmann 1997]:

- Нормы
- Ценности, отношения и мнения
- Ритуалы
- Фольклор (истории и символы)
- Общий язык

Нормы

Нормы — это общие правила и стандарты, которые использует организация для установления границ того, какое поведение допустимо, а какое — нет. Ограничения существуют, чтобы избежать или снизить риски, внешние или внутренние. Одни нормы формальны, установлены явно руководителями, другие являются неформальными, установленными влиятельными общественными лидерами в организации. Вот примеры норм, находящихся в диапазоне от очень формальных (установленных законом) до очень неформальных (совсем не установленных):

- Правила отдела кадров о дискриминации и сексуальном преследовании
- Нормы расходов на поездки
- Стандарты кодирования SQL
- Обязательные методы и инструментарий разработки, например UML или ER
- Мандатный уровень зрелости SEI
- Модели процессов и жизненный цикл разработки ПО
- График поставки системы
- "Обычная" продолжительность рабочей недели
- Антиэтикет одежды (т. е. никто не может носить галстук из-за боязни подвергнуться общественному ostracism или, по крайней мере, осмеянию)
- Начинать совещания вовремя
- Не играть в компьютерные игры в рабочее время (культура костюма)
- Играть в компьютерные игры в рабочее время (культура футболки)

Все нормы связаны с некоторыми видами риска. Культура полезна в большей или меньшей степени в зависимости от того, насколько она соответствует бизнес-миссии. Например, можно тратить весь рабочий день, пытаясь обеспечить своевременную поставку товара, а менеджеры могут слишком много времени уделять попыткам разместить больше людей в кабинетах. Если бизнес-миссия фокусируется на качестве, а не на бюджете и сроках, то такая культура не поддерживает миссию. Это означает, что имеется высокий риск невыполнения данной миссии. Нормы помогают организации сосредоточить внимание на управлении такими рисками.

Как то, что всех заставляют принимать участие в совместных компьютерных играх, связано с риском? По большей части, это управляет риском социальной направленности и придумано для объединения людей в слаженную команду, даже если они и убивают друг друга с помощью симуляторов. Проблема не в том, чтобы совершить виртуальное убийство, а в том, чтобы совершить это в *отношении кого-то другого*. Это снижает риск того, что люди, не являющиеся частью групповой культуры, не введут бессмыслицу Дилберта в процесс разработки.

Кроме того, важно осознавать, что нормы устанавливают доверие. Если рынок ожидает поставки высококачественных продуктов в определенный срок, но в компании нет внутренних норм, касающихся качества и дисциплины, заказчики не будут доверять такой организации. Автор участвовал во многих презентациях, которые преследовали лишь одну цель — убедить заказчиков, что организация и ее культура заслуживают доверия. При наличии стандартов и правил нужно пройти длинный путь, чтобы заслужить внешнюю поддержку. А еще лучше иметь хорошую историю поставки продуктов.

Фирма, не имеющая внутренних стандартов, часто ставит разработчиков в неловкое положение. С совершенно анархичным заведением трудно иметь дело на регулярной основе. И все же часто разработчики предпочитают стабилизировать процесс с помощью авторитарных неформальных решений и правил. Выбор стандартной БД, например, в большой разнообразной организации значительно упрощает жизнь разработчиков и инженеров. Она позволяет осуществлять более точное планирование и выполнение планов. Иногда слишком активное творчество является такой же проблемой, как и слишком пассивное, особенно, когда оно превращается в борьбу с решениями и людьми, не будучи продуктивным. Это поле деятельности для руководителей, которые должны балансировать между формальными и неформальными нормами и поощрять продуктивную, целенаправленную культуру.

Ценности, отношения и мнения

Ценности — это социальные принципы организации, то, о чем люди заботятся. *Отношения* выражают мнения людей об организационных структурах. *Мнения* — это то, что считают правильным (или неправильным) в отношении организации или мира, в котором она находится.

Вот некоторые общепризнанные ценности, отношения и мнения в организации, занимающейся программным обеспечением:

- Умение сформулировать точку зрения.
- Понимание миссии.
- Формирование ценностей.
- Слишком трудно что-либо сделать здесь во время рабочего дня. Поэтому я прихожу домой в 10 часов вечера.
- Как эта свинья поживает сегодня? (Это относится к системе финансовой базы данных, которая почти готова к поставке 10 000 жаждущих клиентов).
- Этим должен заниматься отдел маркетинга, а не мы (ответ кому-то, запрашивающему модель требований для нового продукта).
- На рынке не купят ничего, кроме ПО, работающего под Microsoft Windows (UNIX, Mac и что угодно).
- Рынок примет ПО под любую операционную систему (включая RSX-11M).

- Мой менеджер настаивает на том, чтобы мы все работали по выходным.
- Мы преуспели сверх всяких ожиданий с нашей последней системой!
- Мы можем сделать гораздо лучшую систему, чем ту [бранные выражения опущены], которую вы купили!

Подумайте над этими примерами. Какова их основная идея, цель? Без ценностей, отношений и мнений члены команды будут находиться в полной неопределенности насчет целей организации и того, чего они могут достичь. Формирование представлений, миссии и ценностей является жизненно важным для утверждения культуры организации, согласующейся с ее целями в целом.

У большинства людей, занимающихся БД, очень развиты системы ценностей, сформированные за время долгого пути к принятию своего сана. Это, конечно, метафорическое расширение; тем не менее, программисты и разработчики баз данных имеют набор твердых убеждений, которые позволяют им заниматься своей профессией. Одни из них являются приверженцами основной реляционной религии, другие — принадлежат к более мелким культурам, таким, как объектно-ориентированные БД или dBase, почти вымершая религия, сторонники которой время от времени вдруг появляются где-то. Любой, кто сталкивался с Oracle DBA с целью денормализации схемы БД, понимает религиозную природу этих ценностей и убеждений. Движение к разработке с использованием UML является до некоторой степени вызовом этим ценностям и убеждениям, поэтому автор пытается с максимальной возможной убедительностью сказать: "На самом деле это то же самое, просто в другой нотации".

Ритуалы

Ритуалы — это повторяющаяся модель социального поведения. Относясь к нормам, ритуалы устанавливают заведенный порядок, которому все следуют. Иногда ритуалы могут нарушать нормы, например привычка плохо отзываться о начальнике. Формальность ритуала происходит из его источника (верхнее звено управления, рабочая группа или другой уровень руководства). Вот некоторые примеры ритуалов в организации, занимающейся программным обеспечением:

- Рождественские вечеринки
- Совместные обеды рабочей группы
- Собrania для обмена мнениями
- Проведение дней открытых дверей
- Общие завтраки по пятницам
- Дни свободной формы одежды
- Системный подход к разработке
- Тестирование качества продукта
- Структурированный сквозной контроль
- Спаивание босса во время карнавала

В современных американских компаниях, занимающихся созданием ПО, ритуалы можно разделить на три типа: выпускающие пар, служащие для определенной цели и магические. *Выпускающие пар* — это ритуалы, позволяющие снизить напряжение в группе и объединить весьма неординарных индивидуумов, например во время вечеринки. *Ритуалы, служащие для определенной цели*, — это такие, которые обусловлены определенной причиной, например необходимостью структурированного сквозного контроля или системного тестирования. *Магические ритуалы* — служат защитным и эмпирическим целям, от них не ждут получения очевидной сиюминутной пользы (это может быть проверка менеджмента или проведение совещания по обмену мнениями). Независимо от типа основным пунктом ритуала является повторение поведения, обычно в надежде достичь какой-то цели, реальной или магической.

Ритуалы баз данных встречаются повсюду. При разработке процесс нормализации является таким ритуалом. Как подробно рассматривается в главе 11, очень легко привести БД к пятой нормальной форме с помощью простого, непосредственного проектного решения (связанного с ОО-методами разработки). Но поскольку обычный ритуал такого подхода не предусматривает, автору было очень трудно убедить некоторых администраторов баз данных сделать это. Как правило, они настаивают на прохождении всех стадий нормализации (первой, второй, третьей, BCNF, четвертой, пятой), четко следуя всем догмам. Шутки в сторону, нормализация — это ритуал, зачастую не очень полезный. Тем не менее это один из тех ритуалов, с каким приходится иметь дело в большой организации.

Другой ритуал — оптимизация. По мере приобретения опыта (проходя через ритуальные пытки), администраторы и разработчики БД создают небольшие ритуалы, существующие для умиротворения богов производительности баз данных. Создание индекса битовой карты здесь, разрешение грязных считываний там, использование уникальных имен столбцов, хотя SQL ограничивает область действия имен таблицей, а не базой данных, и т. д. — все это ритуалы с целью разработки защиты от "неправильного выполнения работы". Обычно эмпирическая польза от всего этого небольшая или никакая. Иногда администратор БД может просто сказать, что это "правильно сделано" и все так и оставить. По мере продвижения организации от ритуальной, магической культуры к культуре обратной связи начнут появляться оценки. Как антропологический наблюдатель, разработчик баз данных может различить эти стадии развития культуры по поведению: проводит ли администратор баз данных эталонное тестирование и вносит ли в них изменения на основе полученных результатов? Или просто утверждается, что использование VARCHAR медленнее, чем CHAR, и этого делать не следует? Магические ритуалы действуют настолько долго, насколько не будет времени привязать систему к эмпирической реальности, но в конечном счете проблемы и масштаб БД выйдут за рамки ритуала такого типа.

Разумеется, есть и полезные ритуалы. По мере приобретения опыта мы учимся; по мере обучения создаем новые ритуалы. Появляется понимание, как можно надежно установить Oracle путем прохождения последовательности очень специфических шагов, повторяемых много раз (к счастью, не

всегда в выходные дни). Учимся тому, что однородное преобразование модели данных в схему требует специфических трансформаций в определенной последовательности. Учимся тому, что использование суррогатных ключей единственного атрибута часто предпочтительней, чем применение мультиатрибутных первичных ключей, поэтому разработка становится ритуалом при создании большинства таблиц. С накоплением опыта ритуалы отстаиваются и превращаются в процессы или действия, которые становятся второй натурой — до тех пор, пока кто-то не подвергнет их сомнению, и тогда придется их снова расширять. По мере ритуализации создается фольклор для поддержки ритуалов, это тот фольклор, который появляется, когда ритуалы подвергаются сомнению.

Фольклор

Фольклор — это набор символов и историй об организации. Фольклор зачастую передает ценности и нормы компании или личности и является частью социальной структуры организации. Основные цели фольклора — продемонстрировать статус, показать достижения и представить некую ценность, которая высоко почитается в организации. Вот некоторые символы и истории, общие для фирм, занимающихся программным обеспечением:

- Футболка или кофейная кружка с логотипом продукта
- Истории о зарождении компании
- Истории о других частях организации (иногда это слухи)
- Боевые истории о прошлых проектах компании (как предупреждение о чем-то)
- Басни, в которых определенное поведение обозначается как хорошее
- Лозунги и девизы ("Скорее лучше, чем позднее")
- Пятиминутки
- Байки, представляющие фирму в выгодном свете (истории могут быть и вымышленные)
- Командные трофеи
- "Настенная агитация" — развешенные по стенам проекты компании
- Офисы с окнами
- Офисы с окном в углу

Фольклор целиком посвящен передаче аспектов культуры. Без фольклора нет возможности связаться с культурой, не подвергаясь зависимости от сухих академических описаний антропологии организации (таких, как этот раздел).

Работа с БД полна фольклора, как хорошего, так и плохого. Любой, кто имел дело с базами данных не менее года, знает боевое прошлое Oracle, Informix, Sybase или любой другой использованной системы. У многих разработчиков есть набор аналитических схем (см. главу 8) с сопроводительными историями о том, как они применялись на практике. У большинства есть кофейные кружки с надписью DBMS, выставяемые напоказ, или футболки с

последних пользовательских конференций. Некоторые из нас даже работали в DBMS-компаниях и помнят множество разнообразных историй и баг-сен, объясняющих кое-какие причуды продуктов, с которыми пришлось работать или которые пришлось создавать.

Большинство людей не осознают, например, что PL/SQL (встроенный в Oracle язык программирования) происходит от Ada, выбранный военными инженерами по программному обеспечению в 80-х годах XX века. Архитектором PL/SQL был специалист по Ada, принятый на работу в Oracle для руководства проектами и выросший до роли менеджера по языкам программирования всей компании. История Ada говорит людям, почему в том или ином месте присутствуют определенные вещи (пакеты, например, с перегружаемыми процедурами и исключениями), благодаря которым язык вписывается в более широкий контекст, чем просто язык программирования БД. Фолклор вносит значительный вклад в общий язык, предоставляя символы и истории его пользователям.

Общий язык

Общий язык — это специальный язык, который развивается в организации для описания предметов, интересующих ее членов. Вот некоторые примеры языка совместного использования в организациях, занимающихся программным обеспечением:

- Использование жаргонных названий инструментария
- Манера говорить
- Каламбуры или изобретение нового имени компании ("слова на номерном знаке автомобиля")
- Клички для продуктов, групп или людей
- Создание культурной или географической связи с названием продуктов, групп или людей
- Введение аббревиатур для общих задач или понятий
- Легко запоминающиеся фразы, привлекающие внимание (но не слоганы)

Общий язык устанавливает лингвистические границы вокруг организации или ее частей. Язык, возможно, является самым важным аспектом социальной группы, поскольку с его помощью люди общаются. Значительная доля социального содержания организации, занимающейся программированием, заключена в программном обеспечении, которое она создает; в сущности все, кроме кофейных кружек, — это язык.

Мир баз данных имеет свой собственный технический жаргон, от нормализующих отношений до закрепляющихся объектов и оптимизации на основе стоимости. Различные приемы (индексы битовых карт, внешние объединения, коктейль указателей и т. д.) гораздо быстрее производят "лингвистическое окультуривание" рабочей группы, их необходимо освоить, чтобы стать эффективными проектировщиками и разработчиками БД.

По мере разработки моделей данных, особенно на глобальном уровне предприятия, также создается общий язык для описания бизнес-объектов.

Нужно принимать бесчисленные решения о том, как назвать вещи, особенно абстрактные, например отношения. Появляются стандартные сокращения и начинается их использование в повседневной речи по отношению к объектам. Все это — часть лингвистической культуры организации.

Представляет интерес влияние общего языка на ценности и убеждения организации. Если разрабатываемый продукт называют, например, "свинья, мусор" и т. д., то все непременно будут думать, что продукт на самом деле никуда не годится, и это плохо скажется на мотивации. Если повсюду слышны профессиональные выражения из области высоких технологий, это значит, что в организации ценится технология. Если все, о чем говорят, — язык мотивации ("гребь сильнее", "работай умнее, а не тяжелее"), обратите внимание: 1) на убеждения этих людей, так как они, а не технология позволяют им выполнять свою работу; 2) на отсутствие мотивации и причины, стоящие за этим; 3) на существовании в группе твердой уверенности в том, что людей нужно подгонять, иначе они не справятся с работой. Можно многое рассказать об организации, изучая язык, используемый ее менеджерами.

Культуру компании формируют люди, взаимодействующие друг с другом. Культура разработки сосредоточивается на системах ПО, являющихся центром этого взаимодействия. Культура наиболее важна при создании нового программного обеспечения на основе старого: т. е. создании унаследованной системы.

Замена или использование унаследованной системы

Унаследованная система — это система, созданная достаточно давно и уже не представляющая лучшие технологические или проектные достижения, но все еще ценная для бизнеса. Она может включать более старую технологию базы данных, такую, как базы данных IMS для большой машины или системы dBase для персональных компьютеров. Имея новую модель данных, можно приступить к замене унаследованной схемы новой с использованием современных технологий. Этот путь может быть как правильным, так и нет.

Жизнеспособность технологии — это главный фактор для принятия решения о системе наследования. Не следует использовать новую технологию просто потому, что она есть. Имеющаяся технология превращается в проблему при различных обстоятельствах:

- Когда поставщик отказывается от части или всей технологии, не желая ее поддерживать или далее совершенствовать из-за отсутствия рыночного спроса
- Когда базовая технология (платформы аппаратных средств или операционные системы) существенно изменяется, превращая технологию в бесполезную или устаревшую
- Когда природа бизнес-проблем настолько меняется, что исходная технология не в состоянии оценить изменившиеся проблемы или не имеет возможностей для их решения
- Когда появляется новая технология с улучшенными характеристиками по многим показателям (производительности, возможности оценки, доступу, безопасности, восстанавливаемости и др.)

- Когда появляются новые методы, которые сосредотачивают внимание на решениях, отличных от исходной технологии (адаптация ОО-разработки ко всем приложениям, например, может сделать старые технологии РС или большой машины ненужными и даже вредными)
- Когда велика вероятность того, что старая технология не будет работать эффективно после внесения необходимых изменений

Все эти ситуации приводят к необходимости замены унаследованной системы. Определенно при наличии кризисной ситуации, когда просто нельзя продолжать пользоваться старой технологией, решение предопределено. Однако зачастую ситуация не настолько очевидна:

- Поставщик не собирается совершенствовать технологию, но все еще подписывает контракты на поддержку и отладку
- Унаследованная версия операционной системы может продолжать работать с помощью добавления совместимого ПО или других изменений, адаптирующих старые программы к нуждам разработки
- При наличии массовых усовершенствований на самом деле может и не потребоваться разрабатываемое приложение

В качестве метафоры рассмотрим разницу между двумя популярными шоу застройщиков на радиостанции PBS "Этот старый дом" и "Домашнее времяпрепровождение" [O'Day 1998]. Посылками "Этого старого дома", как следует из заголовка, являются: 1) старые дома сами по себе представляют ценность; 2) старый дом можно реставрировать, улучшив его с помощью современных материалов и технологий. Посылки "Домашнего времяпрепровождения" состоят в том, что: 1) новые дома изумительны; 2) строительство нового дома дает возможность использовать лучшие современные материалы и технологии. Эти подходы не конфликтуют; просто они применяются в разных ситуациях. Если имеется старый дом в хорошем состоянии, его реставрируют. Когда состояние старого дома не очень хорошее, его полностью обновляют. Если есть хороший перспективный участок или старый разрушенный дом, строят новый дом.

Рассмотрев метафору с точки зрения культуры, можно сказать, что два шоу выражают совершенно разную культуру. Культура "Этого старого дома" придает огромную ценность методике реставрации и способам интеграции современных материалов и технологий в имеющийся контекст. Культура "Домашнего времяпрепровождения" делает ударение на современных материалах и технологии и сосредотачивает внимание на приемах и процессах, с помощью которых они используются, дает множество советов по общению с подрядчиками и архитекторами и подсказок для выгодных покупок. Подчеркнем, что это не конфликтующие культуры. "Домашнее времяпрепровождение", например, иногда советует произвести реставрацию, но обычно делает ударение на качество в таких ситуациях. "Этот старый дом" иногда вдается в инновации, если этого хочет владелец, или надо удовлетворить требования комиссии по планированию, или речь идет о действительно новой популярной технологии. Звучит знакомо?

Если проводится анализ прибыли от инвестиций (ROI) в случае замены унаследованной системы по сравнению с ее модернизацией, можно обнаружить, что последний вариант существенно обгоняет первый. Оцените трудозатраты на реализацию нового решения, не забывая стоимости поддержки в течение определенного периода времени, например пяти лет. Затем оцените стоимость поддержки в течение того же периода унаследованной системы. Подсчитайте дополнительную прибыль, которую получают от унаследованной системы. Включите этот приток наличных в стандартный ROI-алгоритм, используя методику скидок на поток наличных, и тогда можно увидеть внутреннюю скорость возврата или чистую стоимость обоих решений. Просто, не правда ли?

К сожалению, хотя ROI-анализ и дает хороший, ясный набор цифр, с помощью которого можно произвести впечатление на друзей и босса, все же эти цифры не очень реалистичны. Отбросив слабость предварительной оценки, ROI игнорирует большое влияние реального мира на принятие решения. Ценится ли в данной организации передовая технология? Часто ли упоминают выражение "резать под корень" при обсуждении новых методов? Возможно, сотрудники не забыли предыдущие проекты, когда менеджеры приняли неверное решение (то или иное). Не выставляет ли кто постоянно напоказ кофейную кружку с Oracle8 или надевает футболку с надписью CORBA? Технология важна для принятия решений, но мнения о технологии, ценностях и символах, хранимых компанией, зачастую более важны. Другими словами, нужно принимать во внимание культуру при решении по поводу унаследованных систем.

Вся разработка направлена на удовлетворение требований заказчиков проекта. Если они инвестировали значительные суммы в культуру своей организации, разработчик подвергает себя опасности, игнорируя ее. Добавьте сюда истории, фольклор, ценности, нормы и другие аспекты культуры. Если рассказывают историю о том, как хорошо работала унаследованная система, или наоборот о том, насколько ужасна ситуация, то можно понять, в каком направлении хочет развиваться организация и, очень возможно, куда ей следует идти. Помогите ей попасть в цель.

На самом деле попасть туда — это только полпути. Поскольку разработчик БД углубился в тайны организационной культуры, то ему следует определить, как наилучшим образом представить модель базы данных, зная унаследованные требования.

ВНИМАНИЕ

Для крупных унаследованных систем, вероятно, стоит использовать оценки надежности и стабильности системы. Например, среднее время отказа поможет определить, насколько работоспособна система в руках ее пользователей, а оценка стабильности [Schneidewind, 1998] подскажет, успешны ли усилия по поддержке унаследованной системы или эта система деградирует.

Новая разработка

Вы только что убедили заместителя директора компании по финансам в том, что унаследованная IMS-база данных 60-х годов готова превратиться в грудку хлама. Вы заключили контракт на демонтаж и готовы начать разработку новой системы. О чем в первую очередь надо беспокоиться при замене унаследованной системы?

Использование унаследованной системы

Даже если идет разработка с нуля, нельзя просто проигнорировать то, что было раньше. Прежде всего и главным образом старая система образует интеллектуальный актив данной компании. Она представляет собой годы трудовых усилий по извлечению сущности и материализации знаний о клиентах, разработчиках и менеджменте компании. Потребности могли измениться, но унаследованная система, безусловно, некоторым образом связана с этими потребностями, иначе она была бы не унаследованной, а просто мусором. Это подобно материалам или свойствам, которые можно сохранить в старом доме, подлежащем сносу. А кроме того, унаследованная система может быть больше, чем интеллектуальным активом; она может быть интеллектуальной собственностью.

В то время как *интеллектуальный актив* представляет собой знание, реализованное в некоторой постоянной форме, например в форме ПО, интеллектуальная собственность — защищенная законом форма, имеющая специфическое значение для компании. Патенты, авторские права, торговые марки и торговые секреты — все это является интеллектуальной собственностью. Нельзя просто отбросить ее, нужно сделать все возможное для ее сохранения. Повторно используйте эту собственность как можно больше. Так, даже если сносится дом, обычно пытаются сохранить ландшафт, в частности старые, но жизнеспособные деревья. Некоторые элементы (озера, реки) составляют главную ценность земли. Возможно, нельзя применить старое ПО, но можно использовать запатентованные процессы, торговые марки или торговые секреты, представляющие собой знание, содержащееся в программном обеспечении.

Другой вид интеллектуального актива, представляемого унаследованной системой, — ноу-хау, накопленное в компании в процессе работы над старой системой. Большая часть этого ноу-хау может существовать в неосознанной форме, в головах людей, которые создавали систему. По мере разработки вариантов использования, модели данных и проекта схемы советуйтесь с людьми, связанными с унаследованной системой.

Системы баз данных в частности имеют ценность, выходящую за рамки интеллектуальной собственности. Данные в БД часто сами по себе представляют ценность — не как биты, образующие данные, но как информация, которую они представляют. Минимально бизнес-БД содержит устоявшееся знание организации о бизнесе. Оно представляет собой историю транзакций, людей, события, процессы и действительно бесконечный массив других систем, на которых держится бизнес, а также организационные

параметры бизнеса (спецификации процессов, производственные модели и др.), историю создания разных баз данных (БД продукта, БД для связи с менеджментом, бухгалтерские БД и т. д.).

Вне зависимости от того, намереваются ли повторно использовать программное обеспечение унаследованной системы, безусловно нельзя отказываться от данных. Если решено не использовать БД повторно, то это означает, что нужно переместить данные в формат новой базы данных. Если повезет, новый менеджер БД предоставит средства, способные переместить данные с минимальной информацией о конфигурации. Возможно, удастся найти средство независимого поставщика, чтобы это сделать; имеется несколько таких средств, находящихся в диапазоне от общих средств считывания файлов (Data Junction или Cambio, www.datajunction.com) до переносных ориентированных на DBMS средств миграции данных (Powermart, www.informatica.com; или Data Navigator, www.platinum.com). В самом худшем случае нужно будет написать программу для экспорта данных из старой системы в формат, загружаемый в новую систему. Это хорошая работа для независимого консультанта, потому что он может ее сделать при очень небольшом взаимодействии или обращении к другим частям проекта.

Основная проблема при повторном использовании унаследованных данных — это качество данных. Насколько можно доверять унаследованным данным? От ответа на этот вопрос зависит, сколько труда нужно будет вложить в повторное использование данных. Вот в этом и состоит вся сертификация повторного использования. Если унаследованная БД не сертифицирована для повторного использования, формально или неформально нужно выполнить определенную работу просто для выяснения того, насколько можно доверять данным. Часть процесса передачи данных в новую схему использует стандартные средства ограничения такие, как ограничения целостности ссылок, триггеры и методы оценок. Нужно быть уверенным в том, что все данные, которые приходят в систему через процесс импорта, имеют те же ограничения, что и данные, которые пользователи будут вводить с помощью проектируемой прикладной системы.

Следует также провести аудит или тестирование данных с помощью случайных образцов или методов ценза, чтобы определить их качество в соответствии с критериями, устанавливаемыми разработчиком. Это относительная оценка качества, а не абсолютная. Превосходные данные не требуются в этом лучшем из миров; необходимы данные, удовлетворяющие нуждам пользователей приложения. Требования разработчика должны учитывать все эти касающиеся данных запросы.

ОСТОРОЖНО

Качество данных является почти наибольшей проблемой, которая возникнет в проекте, связанном с наследованием. Если в расписании не отведено достаточного времени на анализ, отладку и тестирование данных унаследованной системы, такое расписание никуда не годится.

Обсудив проблемы с данными, давайте посмотрим, а как обстоят дела с функциональностью? Именно сюда проникает культура. Нужно извлечь ритуалы, фольклор и прочие аспекты культуры, годами формировавшиеся

вокруг старой системы. Спросите клиентов, как они использовали старую систему и что они хотели бы видеть в новой. Что стоит оставить, а что не использовать? Что совершенно неприемлемо в старой системе? Что они любят в ней? Какие новые идеи созрели при работе, которые стоит учесть? Послушайте их байки о системе.

Спросите разработчиков старой системы — тех, кто не вознесся на корпоративное небо, о сложных алгоритмах и отношениях. Зачастую совсем не очевидно, что привело к тем или иным решениям в старой системе. Нужно отличать решение проблем программирования — временных решений проблемы — от внутренней сложности проблемы. Кто-то мог разработать большое количество кода, чтобы обнаружить ошибку в версии 3 используемой DBMS. Поскольку она заменяется OODBMS, не нужно об этом беспокоиться. А может, разработчик просто не осознает, как сложны некоторые финансовые транзакции. Исследуйте ритуальные решения, чтобы точно выяснить, какую функцию они выполняют.

СОВЕТУЕМ

Можно считать допустимым использование деассемблеров, декомпиляторов или подобных средств для реорганизации данных. Например, возьмите средства, предложенные компанией McCabe and Associates (www.mccabe.com). Хотя эти средства и помогают понять существующий унаследованный код, они не непогрешимы. Не ставьте себя в зависимость от любого средства для перевода унаследованного кода в современную систему, потому что это почти невыполнимая задача.

Можно также повторно использовать унаследованную схему до некоторой степени. Если кто-то работал с клиентами для решения широкого круга сложных проблем с особенно витиеватым набором таблиц и отношений, его знания пригодятся для структуризации модели классов более эффективным образом, вероятно, улучшая проект с помощью снижения сложности, уменьшения связности и увеличения взаимодействия. И опять схема может отражать решение проблем программирования, а не внутреннюю сложность. Исследуйте фольклор и нормы системы, ее культуру, чтобы понять, почему она разработана именно так.

Даже если все начать заново, можно широко использовать то, что было раньше.

Определение границ системы и культуры

При создании нового приложения базы данных с нуля имеется возможность правильно оценить масштаб, что, вполне вероятно, не было сделано, когда расширялась существующая система. Извлеките из этого максимум пользы.

Системные требования

В главах 3 и 4 подробно рассматривался сбор требований и создание модели требований с вариантами использования. Когда имеется набор вариантов использования, описывающий систему, происходит оценка масштаба

проекта. Методика из главы 3 для сбора требований является существенной частью в этом процессе, потому что заставляет разработчика исследовать масштаб системы. Если выбрана совершенно новая технология, нужно также начать все заново с требований.

Не следует предполагать, что унаследованная система выражает требования пользователей. Она может только отражать культуру разработки, которая создала ее. Исследуйте нормы и ценности этой культуры, если вы еще не являетесь ее частью. Если будет обнаружена система настоящих ценностей, удовлетворяющая нуждам клиентов, хорошо. Это значит, что можно рассматривать унаследованную систему как основу удовлетворения клиентов. Культура разработки, создавшая унаследованную систему, может быть культурой, которая считает технологию ценностью, или, что хуже, придает ценность чему угодно, кроме выполнения работы. Если это так, нельзя использовать унаследованную систему как точку отсчета из-за возможных проблем с качеством данных. Потребуется сначала произвести оценку и аудит данных. Кроме того, нужно будет оценить требования к тому, что унаследованная система выражает, для оценки их с точки зрения текущих запросов.

Нужно также взглянуть за пределы требований приложения. Возможно, планируется добавить другие приложения к системе, использующей БД. Хотя не следует принимать во внимание все требования, которые к тому же могут быть нечетко сформулированы, но все же лучше попытаться что-нибудь сделать. В этом случае для получения широкого представления надо воспользоваться генерализацией и сертификацией повторного использования (см. главу 9). Поработайте над потенциалом повторного использования требований и разработки. Проведите тестирование и анализ рисков для сертификации на повторное использование при определенных условиях.

Системная культура

Может обнаружиться, что масштаб задач новой системы шире просто технических проблем, включающих программное обеспечение и данные. Может оказаться, что среда значительно изменилась и необходимы изменения культуры разработки. Для предварительной оценки взгляните на три вида статистики: размер системы, удовлетворение клиентов и исполнение графика поставок [Muller 1998, p. 532–536]. Если система, которую предлагается создать, на порядок больше по размеру, чем старая, то скорее всего нынешняя культура разработчика не способна справиться с задачей. Если несколько последних проектов, предпринятых в культуре разработчика, завершились с относительно низким уровнем удовлетворения клиентов (например, много жалоб и отчетов о дефектах, требуется дополнительная поддержка БД и есть проблемы с доступностью системы), или с большим опозданием, или с превышением бюджета, или проект вообще был отменен, значит, у проектировщика — проблемы с культурой.

Способность данной культуры к выполнению миссии проекта зависит от уровня риска и ожиданий заказчиков относительно проекта, осуществляемого организацией. По мере увеличения риска, роста сложности требований, культура должна адаптироваться и совершенствоваться. Свыше

определенного уровня риска и ожиданий культура может утратить способность дать то, что обещано. Она должна измениться, чтобы вобрать в себя лучшие процессы, обратную связь и предвидение проблем [Weinberg 1992].

При работе с БД большие перемены наступают в проектах, включающих более 100 классов. Это число указывает на то, что система, видимо, будет сложной в связи с природой наследования бизнес-проблем, которые она собирается решить. Увеличение риска в системах БД возникает вследствие технических трудностей или неподходящих средств. Подчеркнем, что здесь рассматриваются системы, создающиеся с нуля, а не унаследованные системы, которые поддерживаются или совершенствуются.

Технические трудности в области БД возникают вследствие использования передовой технологии, которая с большой вероятностью отказывает после загрузки. Поэтому разработчики стремятся использовать такую технологию, только когда появляются бизнес-проблемы, требующие ее применения. Можно легко справиться с риском, связанным с ненужной технологией, просто не используя ее. Нельзя легко управлять риском, возникающим из-за серьезных проблем. По мере роста решимости преодолеть их, проектировщик начинает расширять границы существующей технологии и принимает на себя гораздо больший риск. Терабайты данных, тысячи одновременных транзакций, огромные интерактивные запросы — все это бизнес-проблемы, возникшие с расширением безопасности технологии, использовавшейся в прошлом. Именно эти проблемы могут стать причиной отказа от унаследованной системы и перехода к совершенно новой.

Проблема возникает и в случае применения технологий, не подходящей для достижения данной цели. Возможно, используется менеджер персональной БД для создания сложного приложения с многими пользователями, или задействована сложная система разработки приложения с распределенными объектами для разработки адресной БД. Главной проблемой является выбор типа менеджера базы данных. Многие проекты по базам данных выбирают, например, технологию RDBMS вместо движения к технологии ORDBMS или OODBMS. Разработчик может это делать, следуя корпоративным стандартам, или из-за ограниченности в средствах, или потому, что это ему хорошо известно. Если проблема включает объекты с массой требований к поведению или необходимы расширенные средства навигации, создание деревьев или что-то подобное, выбор реляционной технологии, вероятно, значительно увеличит риски. Программные средства не смогут обеспечить то, что требуется.

В такой ситуации придется переместить культуру в среду ценностей, норм и ритуалов, требующих улучшения. Введение процедур обратной связи с помощью оценок, совокупности данных и оценки данных обычно является большим культурным продвижением многих компаний, занимающихся ПО. Когда установлены ритуалы, приступайте к расширению потенциала возможностей организации для более крупных и сложных приложений баз данных.

Черновая разработка

Каждый разработчик БД может рассказать свою "страшную" историю об обнаружении всей схемы в первой нормальной форме вместо третьей или о том, как половина атрибутов в схеме оказалась ключевыми словами в новой системе БД, на установку которой ушло три недели, или о скитаниях в лабиринте загадочных сокращений имен столбцов вроде YRTREBUT, B_423_SPD или FRED. Эти байки — прекрасные темы для разговоров перед совещаниями у босса. Но в реальности столкнуться с такими проблемами означает в лучшем случае — провести несколько кошмарных бессонных ночей, но в результате добиться порядка, а в худшем — создать слепого на скорую руку монстра.

Сейчас мы находимся в культуре "Этого старого дома". Мы подразумеваем, что приложение создается как усовершенствование имеющейся системы, которая может быть или не быть унаследованной, т. е. расширяют то, что уже работает. При обычном положении дел это то же, что разработка новой системы, только нужно определить, куда вставить новые данные, основанные на новых требованиях. Здесь также предполагается, что расширяемая система неидеальна: она далека от совершенства, и в ней обнаружена огромная проблема. Пол провисает или крыша протекает, а верхний этаж так поврежден, что не подлежит восстановлению. Однако нельзя игнорировать проблему, поскольку по той или иной причине организация приняла решение, что существующая система чрезвычайно важна. Другими словами, данная культура предполагает, что старый дом представляет ценность сам по себе. Он объявлен национальной достопримечательностью, и теперь не отделаться от необходимости привести его в надлежащий вид.

Боевые истории и общий язык

Во-первых, обратите внимание на отдельные боевые истории и язык культуры. Обнаружить момент, в который, по мнению людей, дела пошли не так, — отличный способ определить рискованные места в системе. Например, автор работал с одной прикладной системой, использовавшей пакетную обработку по ночам для массовых обновлений акций из портфеля некоего финансового института. Требования к данным были огромны, и в случае интерактивного процесса, работа системы сильно замедлялась. Примечательна аббревиатура этой программы — PIG (Principal and Interest Generation) — что значит "свинья". Автор так и не узнал, было это случайно или преднамеренно, но "большая свинья" там была подложена. Когда был проанализирован проект базы данных, обнаружилось несколько проблем, касающихся неверных предположений о потребностях клиентов, плохо организованных данных и отвратительно разработанного кода приложения. По одному из предложенных проектов код приложения был переписан, с использованием структурированных методик для улучшения производительности за счет памяти, также были реструктурированы требования и база данных для улучшения производительности машины БД. Это все еще была "свинья", но ей уже не требовалось 36 часов, чтобы выполнить ежедневную пакетную обработку.

Перевод на другой язык также может привести к проблемам. Вы хорошо знакомы с ОО теорией разработки, а данная культура знакома только с реляционным SQL? Это несоответствие языков отражает лежащее в его основе концептуальное несоответствие между потребностями системы и имеющейся БД. База данных может отражать реляционную культуру, но она, вероятно, не будет отражать те предложения, которые высказаны в главе 11. Там можно найти табличные структуры, не совсем оптимальные для повторного использования.

Например, работая с БД, сделанной людьми, знакомыми только с реляционными концепциями, новый менеджер представил объектно-ориентированную методику разработки, подобную той, которая описана в данной книге. Автор разработал иерархию классов для некоторых географических концепций в системе, чтобы расширить ее новыми требованиями, добавив новые географические классы. Перевод иерархии этих классов во множество таблиц отношений оказалось слишком сложным для системы. Проблемы возникли не из-за каких-то технических трудностей с технологией БД, они появились из-за языка, с помощью которого разработчики говорили о системе. Ввод отношений генерализации запутал их тем, что в систему был введен новый язык и новый набор структурных требований. Они, например, постоянно забывали о необходимости обновлять таблицу суперклассов и о добавлении триггеров и/или кода приложения, требуемых для этого.

Нормы, ценности и убеждения

Во-вторых, рассмотрите нормы, ценности и убеждения используемой культуры. Люди зачастую принимают неправильные технические решения потому, что они убеждены в чем-то, что неверно. Существующая система зачастую отражает такие убеждения и неправильные решения. Они проявляются в двух формах – программистский трюк и ошибки.

Программистский трюк – решение специфической проблемы, имеющее некоторый неблагоприятный побочный эффект. Это классический "Этот старый дом". У каждого плотника имеются в запасе хитроумные уловки, помогающие годами скрывать повреждения. Код базы данных может быть переносимым, как при использовании специальных средств SQL Server для решения очень специфической проблемы, может приводить к снижению производительности при разных условиях и использовать средства языка программирования или SQL, который трудно понять.

Например, многие приложения Oracle содержат соответствующий устаревшей версии 3 код, обращающийся к встроенной функции DECODE. Она позволяет использовать условную логику в выражениях SELECT или предложении WHERE для достижения эффектов, которые иначе потребовали бы программного кода. Этот программистский трюк (особенно когда есть выражения DECODE с вложениями до трех или более уровней) происходит из уверенности, что лучше выразить решение отдельным оператором SQL, чем многими, входящими в программный код. Зачастую это происходит из убеждения, что такой программный код трудно написать, он дорог и его трудно отлаживать, в то время, как SQL – нет. Любой, кто пытался поддерживать оператор DECODE с шестью уровнями вложения, не согласится

с этим, и никто, кроме наиболее яростных защитников SQL, не поверит в это. При необходимости расширить это решение предпочтительнее вообще отказаться от него и переписать заново. Например, DECODE сегодня лучше описывается как функция PL/SQL с условной логикой, выраженной, как следует быть, с операторами IF в ясной логике передачи управления.

Ошибка — это решение, которое когда-то работало, а теперь не работает. Опять "Этот старый дом". Там тыкают отверткой балку, поддерживающую первый этаж, и она полностью отваливается, потревожив термитов. Если все не исправить, дом обвалится. Классический пример — система, не имеющая определенных ограничений целостности из-за того, что разработчики БД не сделали новую версию сценариев схемы для использования средств целостности ссылок, когда они появились. Ошибки происходят из двух основных культурных областей: убеждений и ценностей.

Убеждения зачастую обладают свойством устойчивости сверх той области, где они полезны, и на самом деле они становятся вредны из-за изменяющихся бизнес-требований. Один разработчик баз данных полностью отказывался нормализовать свой проект, потому что знал, что пострадает производительность. Когда вышла новая версия DBMS с оптимизатором по стоимости, устранившим проблему, он проигнорировал это и продолжал разрабатывать ненормализованные БД, порождавшие разнообразные проблемы с кодом обновления. Обычно такие проблемы решают с помощью обучения.

С ценностными ошибками сложнее. Разработчик, любящий кодирование, но полагающий, что создание документации — пустая трата времени, обычно является источником основных ошибок в системе. Они ценят сам процесс кодирования настолько высоко по отношению ко всему остальному, что уже не знают, что именно они внесли в систему. Они могут, например, скопировать код без тестирования, они помещают изменения схемы в промышленную БД, что требует перекомпиляции и новой установки всего кода. Вопиющие примеры, подобные этим, к счастью, редки, потому что их последствия ужасны и заставляют менеджеров, даже старающихся не вмешиваться в процесс, все-таки вмешаться. Как правило, самые неумовимые ошибки разработки схемы и памяти создаются этими людьми, вызывая проблемы при обновлении унаследованной системы.

Специальный случай ценностного влияния на унаследованную систему возникает из допущений, лежащих в основе повторного использования. Первоначальный архитектор дома обычно обладает основанным на ценностях видением, как дом адаптируется к людям, живущим в нем. Это источник движения моделирования, возглавляемого знаменитым архитектором Кристофером Александром, которое теперь адаптировано как модель объектно-ориентированной разработки (см. главу 8). Если эти допущения несправедливы или изменяющаяся среда делает их менее ценными, возможно, потребуется снести дом и заменить его, а не реставрировать и не производить капитальный ремонт.

Некий разработчик структур, например, предположил, что каждое приложение, работающее с общей структурой графического интерфейса, которую он предложил, будет использовать стандартный диалог открытия

файла. Автор создал приложение, открывавшее базы данных, а не файлы. Оказалось, что невозможно заменить другое диалоговое окно стандартным диалогом открытия. Разработчик структур связал классы, задействованные в других частях системы, до такой степени, что замена стандартного диалога привела к катастрофическому отказу системы с ошибкой указателя. Пришлось "открывать" файл БД вместо того, чтобы использовать стандартное окно установления связи при входе в систему.

Ритуалы

В-третьих, рассмотрим ритуалы в культуре разработки. Только процессы сами по себе содержат огромный потенциал, способный заблокировать возможность модификации существующей системы. Можно обнаружить, например, что малейшее изменение унаследованной системы требует огромного количества сопроводительной документации и прохождения бюрократических процедур для осуществления изменений. Если встретились проблемы по проекту унаследованной БД или с архитектурой унаследованного приложения, основанной на этом проекте, вполне вероятно, что это следствие действия культурных ритуалов по сохранению имеющейся системы вне зависимости от ее адекватности и качества.

Еще сложнее, чем простой процесс блокировки, ситуация, при которой необходимо работать с "духовенством". Священником при разработке БД является член команды, осуществляющий отношение медитации между Вами и Богами, когда схема и содержимое базы данных объявляются физическим манифестом Богов на земле. По определению, священнослужители, создавшие его (конечно, с помощью Богов), контролируют унаследованную систему. Чтобы модифицировать БД, следует добиться понимания священником того, что нужно, затем священник скажет разработчику, согласны ли Боги. Как со всеми Oracle (оракулами), все это бывает очень неопределенно.

Метафора может зайти слишком далеко. В любом случае обычно имеется сотрудник, связанный с унаследованной системой, который должен провозгласить то, что совершает разработчик, законным. Уровень ритуала варьируется в зависимости от культуры. Главное — убедиться, чтобы дело не дошло до человеческих жертв.

Автор работал с очень крупными бюрократическими компаниями, со многими ритуалами вокруг унаследованного приложения. Например, одна большая контрактная административная система, с которой имел дело автор, нуждалась в миграции на язык четвертого поколения. Разработчики, которые поддерживали базу данных на Oracle, находились в другом здании, не в том, где размещалась группа, отвечавшая за создание новой прикладной системы. Чтобы произвести какие-либо изменения в БД, прикладная группа должна была встретиться с группой БД, затем занести в систему запросы на изменение файлов и получить подтверждение. Это применялось к новым таблицам, а также к изменениям существующих таблиц. Как можно себе представить, разработка ПО шла очень медленно, чему способствовала не только организация процессов, но и корпоративная политика. Группа, отвечающая за повторное использование (хотя и в другом здании),

должна была гарантировать возможность повторного использования или, по крайней мере, внести какой-то вклад в концепцию повторного использования. Глава этой группы не любил руководителя прикладников, поэтому им ничего не удалось сделать.

Производственная культура особенно сказывается при разработке проблемной унаследованной базы данных. Многие из возникающих при этом проблем могут быть решены, если разработчик приобретет поддержку носителей прежней культуры для внесения необходимых изменений. Получение поддержки возможно только путем переговоров и приобретения разработчиком легитимности в данной культуре. Часто требуется поддержка и для того, чтобы сделать изменения легитимными с помощью внедрения новых позитивных организационных правил и утверждения сильного технического лидерства. Может проще, например, убедить приверженцев наследства принять изменения, если разработчик продемонстрирует преимущества ОО-разработки на небольших опытных проектах. Имея критерии успеха, можно создать легитимность и начать убеждать сторонников текущей культуры в пользу изменений.

Какой бы выбор ни пришлось сделать с учетом необходимости разработки новой системы или адаптации унаследованных систем, переход от созданной модели данных к схеме — критический процесс разработки БД.

Структура разработки схем

На определенном этапе разработки наступает важный момент: необходимо выбрать подход к управлению базой данных, который будет использован. Последние главы книги (с 11 по 13) демонстрируют процесс преобразования модели данных в схему с помощью одного из трех подходов. Остальная часть раздела обрисовывает модель этой трансформации. Завершается глава дискуссией по интеграции представлений и систем.

Структуры

Любая схема БД содержит ряд структур, представляющих некоторым образом классы модели данных. Структура группирует атрибуты данных, как позволяет архитектор данных: простые структуры — в реляционной архитектуре, а более сложные в ОО- или ОР-архитектурах.

Структурный раздел каждой главы показывает, как преобразовать основные структуры модели данных UML в структуры схемы:

- Пакеты
- Подсистемы
- Классы
- Интерфейсы
- Атрибуты (включая видимость)
- Домены и типы данных, включая возможность пустого значения и инициализацию

- Операции и методы (включая полиморфные операции, сигналы и видимость)
- Идентификация объектов

После преобразования структур модели данных в структуры схемы начинается разработка схемы.

Отношения

Модель данных UML содержит два вида отношений — ассоциацию и генерализацию. Преобразование отношений UML является областью, которая больше всего зависит от компетентности программиста, особенно при преобразовании генерализаций.

Существуют определенные взаимодействия между отношениям и структурами схемы в различных архитектурах. Создание отношения не всегда является просто созданием отношения: оно может включать создание структуры вместо или в дополнение к отношению. Определенные отношения также имеют характеристики структур; например, иногда в роли отношения выступает атрибут.

Каждый из этих элементов UML имеет какое-то преобразование для каждой архитектуры данных:

- Генерализации (абстрактные и конкретные классы, множественное и единичное наследование)
- Двоичные ассоциации
- Роли
- Общие и композитные агрегации
- Множественности
- Атрибуты ассоциаций
- Ассоциации триадные и с большим количеством элементов
- Уточненные ассоциации
- Упорядоченные ассоциации

Бизнес-правила

В предыдущем разделе описаны многие классические бизнес-правила. Первичные ключи, внешние ключи, множественности — все это части структур и отношений. Однако имеется и ряд ограничений, существующих вне этих структурных элементов.

Когда выражают явное ограничение UML, то задают бизнес-правило, выходящее за предел возможностей стандартного языка моделирования классификаторов и отношений. Большинство систем управления БД способны выразить эти ограничения. Трансформация модели данных в схему должна также трансформировать эти ограничения. Есть две возможности выбора.

Во-первых, можно преобразовать ограничения в набор временного поведения, т. е. построить ограничения в приложении (или на прикладном сервере), а не в базе данных (или на сервере базы данных). В крайнем случае, ограничение создается на стороне клиента. К сожалению, клиентские ограничения редко можно повторно использовать. Нужно реализовывать ограничение заново в каждом приложении и, возможно, в каждой подсистеме разрабатываемого приложения.

Во-вторых, можно преобразовать ограничения в набор постоянного поведения. В реляционной БД это означает введение триггеров и хранимых процедур. В объектно-реляционной БД это означает то же самое и, возможно, введение методов типов. В объектно-ориентированной БД это означает добавление методов или утверждений в методах для установления ограничений.

Общее направление разработки

Какова бы ни была целевая архитектура данных, требуется общее направление разработки. Любая архитектура имеет разный набор неформальных и формальных моделей, таких, как нормализация для реляционных БД. Для ОР-систем нужно сделать выбор между различными способами проведения одних и тех же операций. Для ОО-систем имеются такие специфические направления, как наследование, инкапсуляция и другие аспекты объектно-ориентированной разработки, относящиеся к устойчивости.

Языки определения данных

Каждая архитектура БД имеет свой собственный язык определения данных. В следующих главах дано краткое введение в язык архитектуры (SQL-92, SQL3 и ODL).

Интеграция представлений моделей данных

Моделирование данных вряд ли когда-нибудь происходило в вакууме. Большинство крупных БД поддерживают несколько представлений о данных, различные способы использования данных в одной БД. С помощью использования методик, приведенных в книге, можно создать модели данных, обладающие высокой повторной используемостью. Это делается потому, что в домене реального мира обычно имеется несколько способов использования любых информационных подсистем.

В главе 2 введена одна из ранних логических архитектур систем БД — трехуровневая архитектура ANSI/SPARC. Самый верхний уровень — представление приложений, использующих данные. Функция концептуальной модели данных — отобразить различные представления пользователей на единственную концептуальную модель данных, которая служит для всех представлений приложения. По мере увеличения сложности модели лежащая в ее основе структура концептуальной и физической схем также усложняется. Многочисленные повторно используемые подсистемы заменяют цельные предпринимательские модели данных и многочисленные высоко

специализированные оптимизации заменяют монолитную модель пути доступа. Можно даже объединить гетерогенные коллекции баз данных в полностью распределенную объектную архитектуру.

На самом верхнем уровне логической архитектуры находятся приложения. Они описываются набором вариантов использования и соответствующей моделью данных объектов, на которые ссылаются варианты использования. Каждый вариант использования представляет точку зрения на устойчивые данные. Отображение между вариантом использования и моделью устойчивых данных является базой данных представлений, схемой, представляющей только данные, необходимые вариантам использования.

При движении по архитектуре вниз варианты использования становятся частью подсистемы и разрабатываемые подсистемы в конце концов объединяются в рабочую систему или приложение. Каждая подсистема отображается в модель данных, как и полная система. Отображение моделирует БД для этой системы. По мере продвижения вниз к концептуальной схеме рабочие системы объединяются в глобальную систему приложений, работающих на глобальной или производственной БД. Модель данных на этом уровне интегрирует все представления предприятия в единую модель. Следующий шаг — перевод этой модели данных в концептуальную схему. В нескольких последующих главах описывается движение от модели данных к концептуальной схеме и ее вывод из модели данных. В конце этого процесса интегрируют несовместимые и конкурирующие представления пользователей в единую глобальную БД, поддерживающую все приложения. Так, по крайней мере, обстоит дело в теории.

Может оказаться исключительно трудным примирить все конкурирующие представления о системе. Когда определены все "за" и "против" относительно структуры логических данных, нужно решить вопрос с конкурирующими запросами физической памяти и производительности. БД, требующие для своей работы наличия огромных ресурсов, часто порождают множество проблем конкуренции и надежности для поддержки только одного представления пользователя. Другие пользователи базы данных могут считать, что их приложение более важно. Может оказаться проще установить несколько отдельных БД на глобальном уровне для поддержки этих пользователей и внедрить ограничения для этих баз данных. Процесс интеграции направлен на достижение следующих целей:

- Точное представление функциональных требований с помощью структур и ограничений, включая те, которые находятся между системами высокого уровня или приложениями
- Управление неопределенностью
- Ликвидация неоднородности

Кажется, что легко достичь точности, но это требует очень ясного понимания всех моделируемых доменов. Точность, однородность и неопределенность часто являются аспектами одного и того же процесса — взаимодействия ограничений данных. При создании вариантов использования консультируются с имеющимися и потенциальными пользователями системы, доменными экспертами и всеми, кто может предоставить информацию о домене.

Как аналитик, разработчик интегрирует все эти сведения в вариантах использования и создает модель классов для поддержки требований к данным в этих вариантах использования. В итоге нужно полагаться на собственное понимание при разработке правильных структур и ограничений. Пользователь может сказать, описывает ли вариант использования то, что ему нужно. Но он не всегда может оценить структуру БД и ограничения, особенно когда разработчик пытается объединить различные представления пользователя и приложения.

Например, агентство Holmes PLC имеет несколько разных пользователей, заинтересованных в различных аспектах системы каталога. Скажем, одна группа в Швейцарии поддерживает связь с Интерполом. Они хотят убедиться, что могут получать информацию, которая нужна Интерполу, и тогда, когда она им потребуется. Для этого (пример не имеет отношения к тому, как на самом деле работает Интерпол) нужны данные о криминальных происшествиях и подозреваемых с определенной специфической международной информацией об этом. Определенная структура нескольких классов противоречит обобщенной модели людей, организаций и происшествий, которую будут использовать большинство пользователей каталога. Противоречия заключаются в разных ограничениях множественности на отношения и разных ограничениях атрибутов на классы. Прорабатывая детали, программист понимает разницу и то, как можно сделать два набора требований совместимыми друг с другом. Переговоры и взаимодействие (итогом которых станет, например, появление отдельных подклассов или объединение результатов в модель классов) позволяют совместить два представления в базу данных, которая удовлетворит всех пользователей. Можно обнаружить пару вещей (или больше, если это сложное приложение), которые не имеют смысла, или в дальнейшем понять, что некоторые требования были неверно интерпретированы. Так, требование Интерпола, первоначально выглядящее, как отношение многие-ко-многим, при дальнейшем исследовании может оказаться отношением один-ко-многим.

Допустимо сделать интеграционный подход настолько формальным, насколько нужно. Если разработка происходит в такой среде, где требуется возможность устранять некоторые из произведенных изменений или отслеживать изменения индивидуальных схем, то здесь годится подход формального преобразования [Blaha and Premerlani 1998, p. 210–222]. Преобразование — это специфическое изменение, которое можно выполнить с моделью данных, такое, как вычитание элемента, создание экземпляра класса и факторизация множественного наследования. По мере определения проблем интеграции исходные схемы преобразуются в схему слияния с использованием этих преобразований. В любое время можно точно видеть, как перейти от исходной схемы к схеме слияния. Это позволяет понять основную цель схемы слияния и селективно осуществить обратные преобразования, чтобы вернуться назад к предыдущему состоянию схемы, если будут совершены неверные действия.

Большинство проектов баз данных, однако, не нуждаются в таком уровне формализации и могут вестись с помощью нескольких ключевых стратегий (при этом важно помнить об основных целях):

- Слияние классов с подобными структурами
- Использование генерализации, когда это возможно, для интеграции структур и ограничений
- Использование специализации, когда это подходит, для разделения структур и ограничений с общими характеристиками
- Использование генерализации и ассоциации для устранения ненужной избыточности в структуре и отношениях между классами; замена унаследованных атрибутов, созданных путем слияния с операциями, которые порождают значения
- Реализация всех ограничений, особенно сложных, согласованных друг с другом
- Поиск и добавление всех пропущенных отношений
- Размещение классов в слабосвязанных подсистемах с минимальным количеством отношений
- Оценка общей картины с позиции разработки и перераспределение всех компонентов для улучшения связности и соединения

Структурная интеграция

Первым шагом при интеграции двух разных моделей данных является их сравнение на наличие общих мест (перекрытий). Можно применять для этого два подхода. Во-первых, находится структурное сходство — подобные имена классов или атрибутов и подобные структуры классов и отношений. Сравните идентификаторы объектов тех классов, которые, вероятно, одинаковы, чтобы убедиться, что основные идентификационные требования верны. Во-вторых, используйте свое теперь расширенное знание о доменах для сравнения двух моделей на перекрытие по значению. Это означает поиск подобных концепций, затем поиск подобной структуры в классах и отношениях. Нужно иметь возможность устранить внешние различия, добавляя или убирая атрибуты или реорганизуя отношения.

Например, система каталога использует Человека как центральную таблицу для описания всех людей, представляющих интерес для оперативников. Агентство Holmes PLC также имеет приложение контактов, которое нужно интегрировать в ту же глобальную базу данных, что и каталог. На рис. 10.1 показаны исходные схемы Контакта и Человека.

При сравнении Человека и Контакта можно видеть некоторое имеющееся структурное сходство. Человек связан с Именем и Адресом, Контакт связан с Именем и Способом Контакта, а ключом является стандартный идентификатор объекта. Семантически сравнение еще более подобно. Контакт — вид Человека, и на самом деле между ними нет различий, кроме предназначения данных. Контакт в приложении контактов менеджмента — это Человек вместе с разными способами взаимодействия с этим человеком. Человек в записной книжке — биографический ввод, прослеживающий личную историю и различные личные характеристики требований идентификации и расследования. Произвести слияние в этом случае нетрудно (см. рис. 10.2).

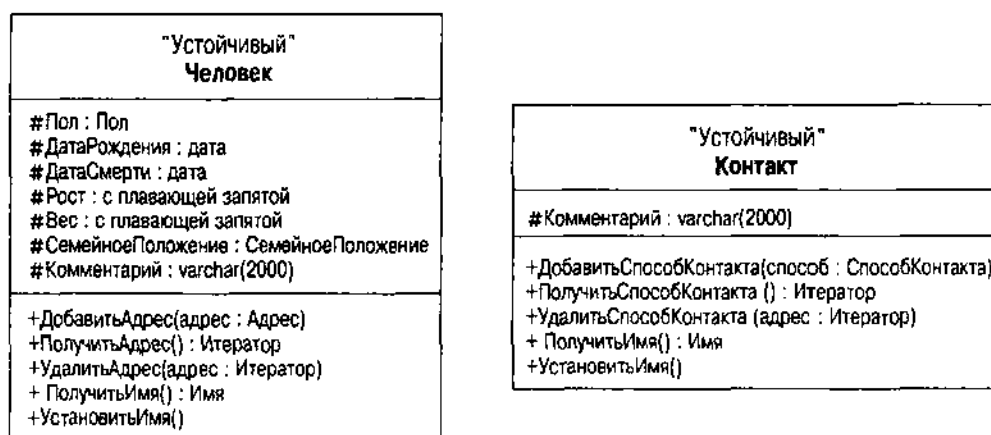


Рис. 10.1. Люди и Контакты в исходной форме

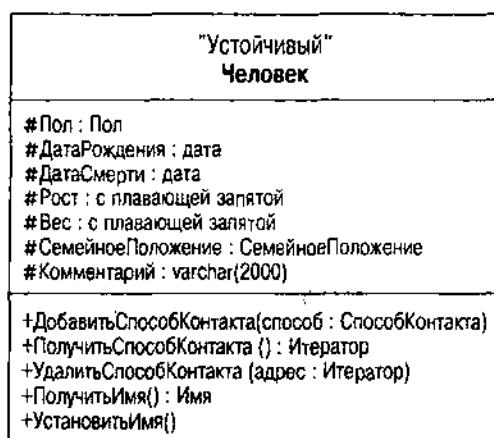


Рис. 10.2. Объединенные классы Человек и Контакт

Чтобы объединить два класса в один класс Человек, нужно установить связь Человека с классом Способ Контакта, заменив связь с Адресом. Адрес становится подклассом в Способе Контакта (см. ниже в разделе "Интеграции генерализации"). Нужно перекодировать три метода Адреса (Добавить, Получить и Удалить) как более распространенные методы управления контактной информацией. Можно добавить специфический метод ПолучитьАдрес для упрощения программирования в классах записной книжки, если это имеет смысл.

СОВЕТУЕМ

В стандартном проекте БД структурная интеграция уделяет внимание прежде всего первичным ключам и атрибутам. В ОО-модели наиболее вероятно найти сходство, основанное на отношениях и операциях, хотя атрибуты тоже играют определенную роль. Использовать первичные ключи для обнаружения сходства нежелательно из-за допущений по поводу неявных первичных ключей. Если можно определить, что первичный ключ (объектный идентификатор) разделяет домен с другим классом в другом приложении, это подходящий кандидат для слияния двух классов.

Интеграция генерализаций

Два класса или кластера могут лишь слегка отличаться по структурным требованиям, имея гораздо больше сходства, чем отличий. Это классическая ситуация использования генерализации. Генерализуйте общие атрибуты в суперкласс и сохраняйте разные в подклассах. Более того, можно обнаружить концепции, полностью подходящие для иерархий генерализации в других системах; система, в которой они появляются, просто не нуждается в остальной части иерархии.

Например, одно из приложений Holmes PLC реализует систему хода работы детектива. Каждый тип процесса в ходе работы — это отдельный подкласс в иерархии классов процесса. Зачастую два процесса или более разделяют набор атрибутов потому, что они концептуально подобны. Общие атрибуты из разных процессов выделяют в отдельный суперкласс. Это генерализует концепцию, но предоставляет конкретные подклассы для специфических производственных процессов.

Другая система моделирует людей и организации для представления преступников, преступных организаций, а также других людей и организаций, представляющих интерес для оперативников. И снова из-за структурных характеристик, которые их объединяют, модель данных приложения выделяет людей и организации в общий суперкласс — Сущность.

Интеграция бизнес-правил

В глобальной модели данных крупного бизнеса имеется множество бизнес-правил. И почти всегда можно гарантировать, что любые два приложения, сливающиеся в одной модели, будут иметь противоречивую логику. У отношений — разные множественности, некоторые атрибуты будут иметь пустые значения в одном приложении и непустые — в другом, или сложные ограничения просто логически невозможно совместить друг с другом.

Например, Holmes PLC имеет приложение, которое управляет безопасностью текущих счетов в банках. Федеральное регулирование требует, чтобы банк проверял документы клиента с фотографией при открытии счета. Класс Банковский Счет связывает класс Человек с отношением, включающим класс ДокументСФотографией, используемый для создания счета. Когда эта модель данных интегрируется с каталогом, имеющим гораздо более обширную идентификационную модель, Банковский Счет связать с уже существующим классом Человек без ДокументаСФотографией уже сложнее. На рис. 10.3 показано это противоречие.

В этом случае связь ДокументаСФотографией немного более сложна, чем просто определение новой связи генерализации или ассоциации. Все представление документа с фотографией совершенно не согласуется с моделью каталога, которая моделирует специфическое идентификационное разнообразие. Перемещение ДокументаСФотографией в иерархию класса Идентификационного документа не сработает потому, что объекты ДокументаСФотографией (водительское удостоверение, национальный идентификационный документ или что-либо еще) перекроются с объектами другого класса. Имеются некоторые альтернативы:

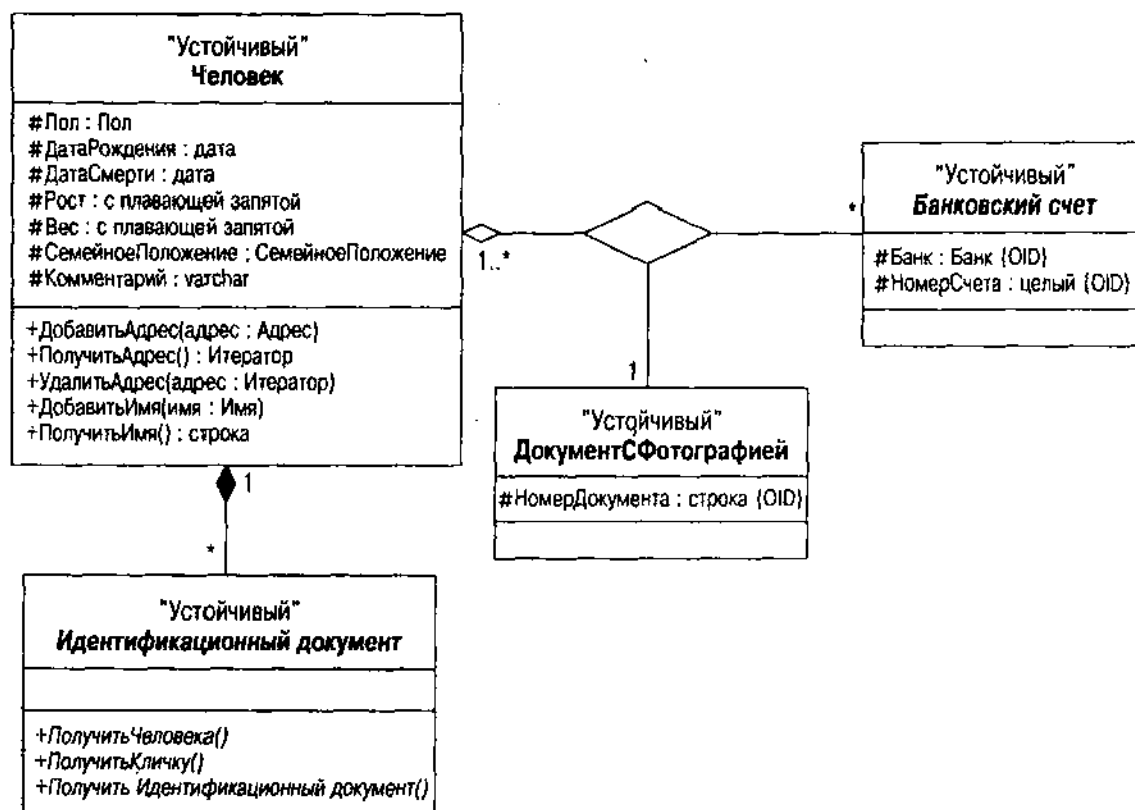


Рис. 10.3. Противоречие между ДокументомСФотографией и Идентификационным Документом

- Свяжите ДокументСФотографией с определенными (противоположность абстрактному) подклассами, которые по сути являются ДокументамиСФотографией. Это вводит множественное наследование в схему и наверняка все усложняет.
- Создайте интерфейс ДокументСФотографией и добавьте его к определенным подклассам, которые на самом деле являются ДокументамиСФотографией. Это хорошее решение на короткое время. Оно позволяет объединять две схемы без какого бы то ни было изменения лежащей в основе структуры схемы, отличной от добавления единственного интерфейса и реализации нескольких методов (не показанных на рис. 10.3) в определенных подклассах, поддерживающих ее.
- Подумайте об иерархии класса Идентификационного документа, чтобы включить абстрактный класс ДокументСФотографией. Реорганизуйте его специфические классы. Это в значительной мере разрушит систему каталога в зависимости от того, насколько много работы программисты уже выполнили, используя текущую иерархию.
- Подумайте еще о классе ДокументаСФотографией и замените его ассоциацией с Идентификационным документом. Добавьте ограничения, требующиеся Идентификационному документу, для его существования, когда имеется Банковский счет для человека, требующий Идентификационного документа с фотографией. Добавьте это полиморфное свойство к

соответствующим интерфейсам Идентификационного документа. Это прекрасно генерализует отношение, снижая связность, с помощью использования полиморфизма, но при этом также увеличивается число дополнительных ограничений, требуемых для выражения сложных требований Банковских счетов к Идентификационным документам.

Выбор за разработчиком. Любой способ имеет свои преимущества и недостатки. Для обеспечения максимального повторного использования схемы последний вариант генерализующего ДокументаСФотографией, возможно, наилучший, несмотря на добавленную сложность. Его можно инкапсулировать. Интерфейсное решение прекрасно подходит на короткое время, если необходимо объединить две системы так, чтобы они быстро заработали. Однако в следующий раз возникнут проблемы из-за несколько аварийной природы отношения. Если кажется, что ДокументыСФотографией более важны, чем исходное разбиение Идентификационных документов со сроком действия и т. д., тогда третий вариант может быть наилучшим. Все зависит от целей проекта в данный момент или от долговременных целей.

Интеграция общей картины

Помимо простого рассмотрения краткосрочных и долговременных целей, разработчику на самом деле нужно отойти от проектирования и взглянуть на общую картину прежде, чем осуществить необходимый выбор. Пройдитесь по сценариям из вариантов использования и из вариантов использования в системах, объединяемых со схемой, чтобы посмотреть, как схема в действительности их поддерживает. В процессе этой работы будет все более четко проявляться общая картина объединенной схемы в целом, а не коллекция частей. Взаимоусиление системы в целом и ее взаимоотношений — ядро окончательного набора стратегий для объединяющихся схем.

Прежде всего нужно проверить, нет ли пропущенных отношений. Если это желательно делать систематически, можно создать матрицу электронной таблицы для каждой подсистемы со всеми ее классами сверху донизу. Заполните ячейку значением, если есть отношение. Само значение может быть "Истина", или может использоваться тип отношения (такой, как генерализация или ассоциация), или можно использовать множественность. Не забудьте о рекурсивных отношениях по диагонали матрицы, где классы относятся сами к себе. Можно оставить ячейки выше диагонали (или ниже — как захотите) пустыми, если отношения считаются симметричными. Если используются направленные стрелки, с другой стороны, нужно представить различные направления отношения в разных ячейках. Кроме того, включите транзитивные генерализации туда, где класс является подклассом другого не напрямую на более низком уровне. Ниже этой матрицы можно составить список отношений классов вне подсистемы, который будет использован позже.

Теперь взгляните на пустые ячейки. Во-первых, рассмотрите каждую ассоциацию, которая отсутствует там. Будет ли это иметь смысл в любом потенциальном приложении для ассоциации классов? Имеет ли ассоциация значение? Полезно ли внести некоторую сложность в модель данных? Если можно ответить "да" на все эти вопросы, создайте новую ассоциацию.

Во-вторых, рассмотрите каждую отсутствующую генерализацию. Имело ли бы смысл связать классы как суперкласс-подкласс? И опять — стоит ли увеличивать сложность? Комментарии к различным вариантам в предыдущем разделе "Интеграция бизнес-правил" показывает некоторые имеющиеся здесь недостатки. Если в этом есть смысл, реорганизуйте иерархию классов, чтобы она включала новый класс в соответствующем месте иерархии.

После такого упражнения для каждой подсистемы перейдите на другой уровень и рассмотрите отношения между подсистемами. Если это помогает, нарисуйте контейнерную диаграмму и аннотируйте контейнерные зависимости с помощью точных отношений, которые включены в список внизу различных электронных таблиц.

Во-первых, найдите любые циклы. Цикл на диаграмме пакетов — это когда зависимость от пакета ведет к пакету, от которого зависит первый пакет, прямо или косвенно.

Во-вторых, найдите любые отношения, являющиеся композитной агрегацией (черный ромб, устанавливающий класс, содержащийся внутри другого класса). Без действительно веской причины не следует иметь таких отношений между пакетами. Композиты сильно связаны, обычно с перекрытием объектной идентификации. Следовательно, они принадлежат одному и тому же блоку связности в системе.

В-третьих, найдите любые отношения генерализации между пакетами. И снова подклассы сильно связаны с их суперклассами поскольку объект содержит экземпляр обоих классов. Однако имеются определенные ситуации, которые оправдывают помещение суперкласса в различные подсистемы.

Например, можно повторно использовать пакетный компонент, такой, как структура с помощью повторного использования "черного ящика", требующего создания подклассов в классе компонент, чтобы сделать его домен специфическим. Это по определению значит, что суперкласс находится в другом пакете, поскольку природа ситуации "черного ящика" означает, что нельзя модифицировать общий пакет компонентов.

Другой пример: можно иметь иерархии классов с разрозненными поддеревьями, которые относятся к совершенно другим частям системы. Объединенный суперкласс может существовать только для реализации или наследования общих свойств. Это именно тот случай, скажем, когда в системах имеется единственный корневой объект (CObject, ooObject и т. д.). Иерархическая природа UML является своеобразным препятствием с этой точки зрения. Может будет лучше считать пакеты перекрытием, когда некоторые классы являются частью пакета дерева генерализации, связанного с ассоциацией пакета. Он представляет иерархию целого класса, начиная с произвольного корня в то время, как другие пакеты представляют использование этих классов. Однако чем больше имеется перекрытий, тем более трудно понять все, что происходит из-за возрастающей сложности отношений между пакетами.

И последнее — взгляните на количество зависимостей между каждым пакетом и другими частями системы. Цель декомпозиции системы на подсистемы — максимально устранить связность подсистем с остальной частью системы. Чем меньше имеется зависимостей, тем лучше. Однако каждая зависимость представляет собой форму повторного использования. Если система обладает высокой степенью повторного использования, это означает, что в будущем многие системы окажутся в зависимости от свойств нынешней системы.

Чтобы понять недостатки, нужно оценить зависимости с помощью метрик связности и потенциала повторного использования (см. главу 9). Если у системы низкий потенциал повторного использования, следует иметь относительно низкую оценку связности для каждой зависимости и небольшое количество зависимостей. Чем выше потенциал повторного использования, тем больше зависимостей может поддерживать пакет. С другой стороны, пакет, зависящий от одного и более других пакетов, не должен иметь особенно высокий потенциал повторного использования из-за своей природы сильной связности. Если такая комбинация обнаружена, вернитесь назад и постарайтесь понять, почему потенциал повторного использования оценен так высоко. Должен ли весь набор подсистем быть частью более крупной системы повторного использования ("структуры")?

ВНИМАНИЕ

Другой способ оценки связности подсистем — использование логического горизонта классов. Логический горизонт класса — транзитивное замыкание отношений ассоциации и генерализации из класса, который завершается на множественности 1..1 или 0..1 [Blaha and Premerlani, 1998, p. 62–64; Feldman and Muller, 1986]. Пройдите по всем путям от класса к другим классам. Если ассоциация других классов имеет множественность 1..1 или 0..1, этот класс является логическим горизонтом, но путь здесь заканчивается. Можно пройти на верх отношения генерализации к суперклассу класса, но не вниз к его составляющим. Классы с самыми большими горизонтами обычно расположены в центре подсистемы, включающей большую часть логического горизонта этого класса. Однако, как и с большинством механических методов, использование подхода логического горизонта может увести в неправильном направлении и не гарантирует высоко связанных подсистем, особенно если ассоциации не закончены или неверны (такого никогда не бывает, не правда ли?).

В связи с происходящими изменениями отношений подсистем неплохо еще раз проанализировать систему, чтобы увидеть, изменилось ли что-либо из-за перемещения классов между подсистемами.

Итоги

Разработка моделей данных унаследованных систем ставит в центр внимания культуру организации, создавшей эти системы. Эта культура имеет огромное влияние на развитие унаследованной системы и работу по поддержке с помощью нескольких механизмов:

- Нормы
- Ценности, отношение и убеждения
- Ритуалы
- Фольклор
- Общий язык

Имея эти элементы культуры организации, можно будет усилить или ослабить существующие данные и технологию в унаследованной системе для обеспечения базиса развития системы. Иногда лучше начать заново (провести обновление старого дома). Частью работы создателя БД является оценка масштаба новой системы на основе требований обеих систем и системной культуры. Но при обновлении нужно уделять гораздо больше внимания культуре, чтобы понять как масштаб старой системы, так и масштаб ее изменений.

Следующие три главы (остальная часть книги) показывает, как преобразовать модель данных в одну из трех схем: реляционную, объектно-реляционную или объектно-ориентированную. Каждая глава имеет похожую структуру, описывая преобразование следующих элементов:

- Структур
- Отношений
- Бизнес-правил
- Основных направлений разработки
- Языков определения данных

Последний этап создания модели данных – интеграция всех разнообразных представлений в одну общую концептуальную модель. Различные структуры интегрируются в единое и хорошо структурированное целое. Осуществляется генерализация концепций, объединенных в иерархии классов. Производят интеграцию бизнес-правил и разрешают все логические конфликты между ними. И в конце разработчик отходит в сторону и смотрит на общую картину, чтобы оптимизировать суммарное представление о системе.

Теперь разработчик находится в точке, где колеса встречаются с дорогой. К сожалению, это компьютерная игра, где дорога изменяет свою природу в зависимости от того, где находится игрок и как он экипирован. Переход от модели к схеме – сложный процесс. Нужно пробиться сквозь шарады культуры, понять имеющийся инструментарий и усилить то, что разработчик может привнести в схему.

Глава 11

Разработка схемы реляционной базы данных

Вы приготовите для меня стол рядом с теми, кто следует за мной:
вы смажете мою голову маслом, и моя чаша будет полна.
Но возлюбленная доброта и милосердие будут следовать за мной
всю мою жизнь: и я поселюсь в доме Господа Бога навсегда.

Молитвенник англиканской церкви, 23:4

Преобразование модели данных в схему БД — непростая задача, однако это несравнимо с высадкой еще одного человека на Луну. Методы превращения ER-модели в R-модель хорошо известны [Teorey, 1999, Ch. 4]. Преобразование модели данных UML в реляционную БД использует их и добавляет другие, чтобы взять на вооружение расширенные возможности таких моделей. Первые три раздела этой главы подробно рассматривают структуры, отношения и ограничения модели данных UML и их преобразование в реляционную схему.

В следующем разделе определяется понятие нормализации данных. На самом деле нельзя говорить о реляционных БД, не упомянув о нормализации. Однако если к ней подойти с точки зрения ООП, можно существенно ограничить сложность и детальность. Все еще важно понимать, насколько структура вашей БД отвечает "нормальным" требованиям реляционного мира, но гораздо проще получить ее из ОО-модели данных.

Последний раздел суммирует последовательность шагов, предпринятых для преобразования ОО-модели данных в схему SQL-92. Он также демонстрирует некоторые специализированные вещи, которые можно сделать, если использовать нестандартные элементы менеджера БД, в данном случае Oracle7.

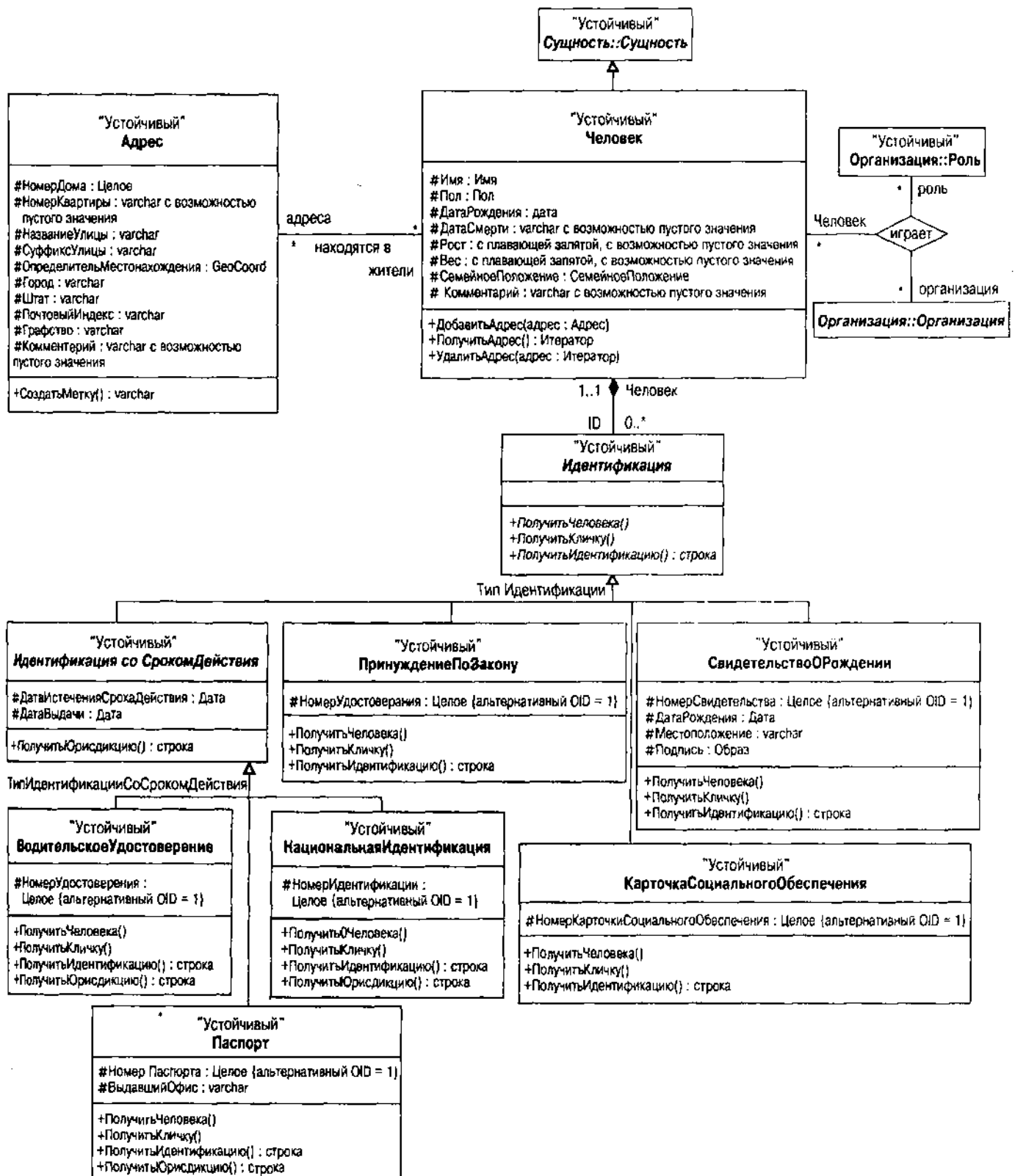


Рис. 11.1. Представление подсистемы Человек на UML

Преобразования таблиц

Структуры реляционной БД очень просты. Перед вами таблицы и только. Эта простота имеет одно негативное последствие: усложняется процесс представления сложных объектов. При переходе от модели данных UML к R-схеме весь фокус состоит в том, чтобы заставить простоту R-таблицы работать на разработчика, а не против него.

Для иллюстрации этого процесса на рис. 11.1 представлена модель UML для подсистемы Человек в каталоге, а на рис. 11.2 – модель UML для

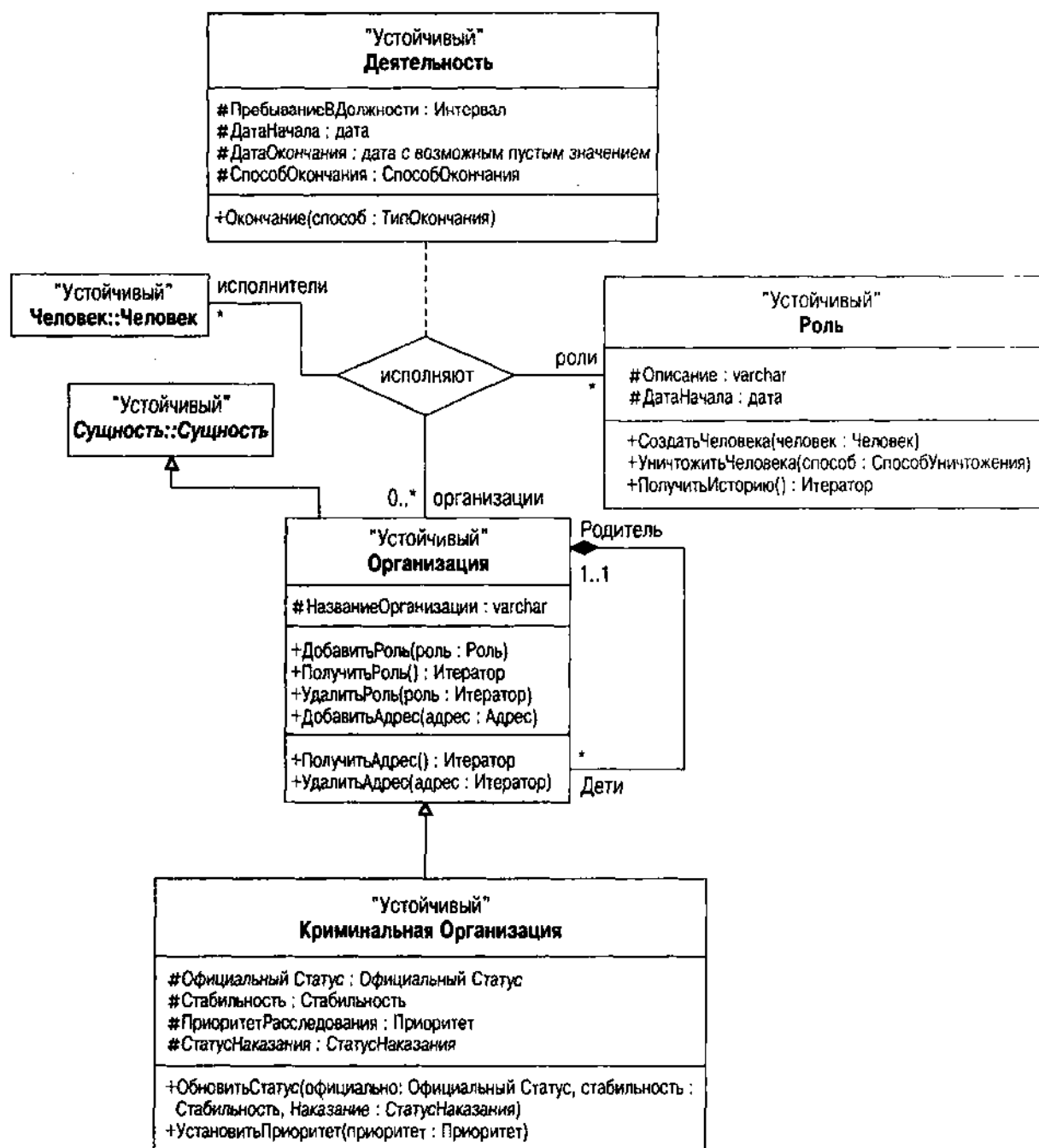


Рис. 11.2. Представление подсистемы Оргвнизвция на UM

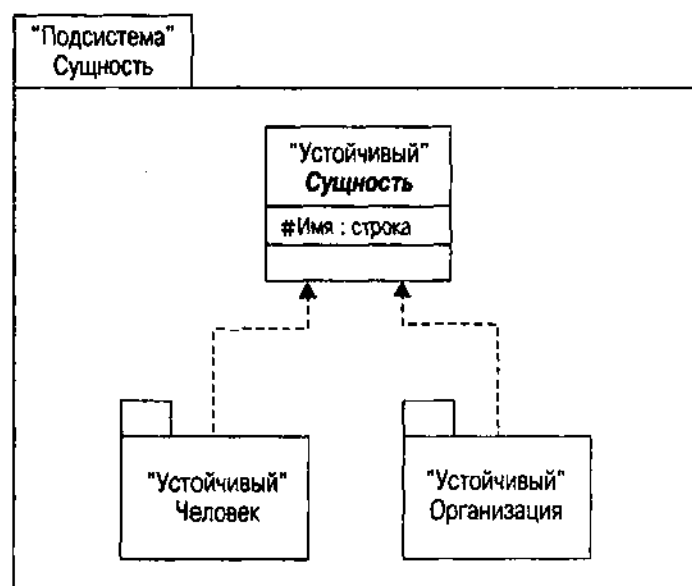


Рис. 11.3. Представление подсистемы Сущность на UML

подсистемы Организация. Обе эти подсистемы являются частью третьей подсистемы – Сущность. На рис. 11.3 показана архитектура этого пакета.

Подсистема Человек содержит класс Человек и иерархию Идентификации, ему принадлежащую. Автор выбрал версию наследования Идентификации, а не интерфейсную. Люди связаны с организацией через отношение с тремя путями доступа к классу Роль в контейнере Организация. Обозначение области действия (Организация::Роль) идентифицирует классы, не являющиеся частью подсистемы Человек.

Подсистема Организация содержит иерархию Организации, включающую класс КриминальнаяОрганизация. Она также предусматривает класс Роль и отношения между Ролью и Организацией. Организация связана с людьми через отношение с тремя путями доступа к Роли.

Подсистема Сущность содержит три элемента: две подсистемы – Человек и Организация – плюс абстрактный класс Сущность. От него наследуют Классы Человек и Организация в соответствующих пакетах. В будущем Сущность будет вступать в отношения с другими подсистемами, такими как СпособКонтакта, ГеографическоеМестоположение и Образ.

Пакеты, подсистемы и пространства имен

Пакет и его специализация, подсистема (см. главу 7), предоставляют организованные пространства имен для системной архитектуры. Цель – разработать систему, состоящую из повторно используемых частей, абсолютно независимых друг от друга.

Пакеты – это новая концепция для реляционного мира. Большинство разработчиков БД думают об R-схемах как виде глобального репозитория данных или даже как о едином пространстве имен. Этот подход проистекает из технологических ограничений в прошлом и из культур, созданных в этих ограничениях. Например, можно все еще встретить администраторов БД,

которые продолжают настаивать на глобальных именах столбцов или на именах, включающих все виды идентифицирующей информации. Например, CMN_ID_ID_NO_NUM может представлять такое излишнее соглашение об именах: столбец Идентификационного номера (ID_NO) идентификационной таблицы (ID) каталога (CMN), являющийся цифровым типом данных (NUM). Автор же предпочитает ИдентификационныйНомер в таблице Идентификация, получая все преимущества поименованного пространства, установленные схемой и таблицей.

ВНИМАНИЕ

Не путайте пакет UML, рассмотренный здесь и в главе 7 (и далее в этой главе, которая ставит разработку БД на промышленные рельсы), с пакетом PL/SQL в Oracle. PL/SQL берет идею пакета из языка программирования Ada, где пакет — это блок компиляции, собирающий ряд различных элементов в единый объект. Он скорее является модулем или классом, чем подсистемой, хотя концепции связаны с концепцией поименованного пространства.

Проблема схем как множеств глобальных имен является как раз той, для которой предназначены пакеты — повторного использования с помощью инкапсуляции и разрыва связности. Большинство разработчиков БД на определенном этапе сталкиваются с необходимостью интегрировать более одной БД в единое целое. На самом деле нужно понимать проблему как интеграцию нескольких пакетов в систему. Каждый пакет вносит свой вклад инкапсулированным несвязным образом. Следуя ОО-методике, не пользуйтесь подходом наименьшего общего знаменателя, поднимая все имена на вершину как глобальные, — применяйте пространства имен, чтобы тщательно организовать доступ.

Стандартный SQL предоставляет базовые возможности пространства имен с помощью имени схемы [Melton and Simon, 1993; ANSI, 1992]. Схема ANSI — это дескриптор, включающий имя, идентификатор авторизации, имя набора символов и все описатели набора компонентов (таблицы, представления, домены, условия, привилегии, наборы символов, сравнения или преобразования). Ни одна из известных автору основных DBMS точно не реализует возможности этой схемы. Системы SQL/DS и DB2 компании IBM де-факто устанавливают стандарт, задавая систему пользователей с единой БД. Oracle и Informix следуют этому подходу, как и большинство более мелких поставщиков DBMS. Концепция пользователя соответствует концепции ANSI идентификатора авторизации. В более раннем стандарте SQL имя схемы и авторизация были одним и тем же — стандарт 1992 г. их разъединил.

ВНИМАНИЕ

Здесь автор должен выразить свое собственное мнение (повторю, мнение) в небольшой притче о неудачном подходе, в котором основные реляционные поставщики проигнорировали стандарт ANSI. Ни один из основных поставщиков даже и не приблизился к реализации всех характеристик стандарта, несмотря на его значительные улучшения на языке SQL в областях, находящихся в диапазоне от обработки дата-времени до множеств символов и интернационализации [Melton and Simon, 1993]. Возможности пространства имен

схемы еще один пример неспособности основных поставщиков двигаться вперед. Разработчикам надо оказать на них давление, чтобы они внедряли полный стандарт как можно скорее. Поставщики, жалующимся на трудности реализации стандарта, разработчики могут напоминать, что люди платят огромные деньги за их навыки в разработке программного обеспечения и его реализации. Кроме того, обладание полным ANSI SQL — исключительно важное предложение для клиентов. Поставщик, его предоставляющий, будет занимать очень сильные конкурентоспособные позиции на рынке RDBMS.

Пользователь соответствует идентификатору аутентификации и имени схемы. Эти БД не имеют объекта "схема", хотя и реализуют оператор CREATE SCHEMA из стандарта. Вместо установки пространства имен схемы эти системы устанавливают пространство имен пользователя. Каждый пользователь может выполнить несколько операторов CREATE SCHEMA, не предполагая никаких отношений между наборами объектов, создаваемых различными операторами, помимо того, что все объекты принадлежат поименованному пользователю.

На самом деле пользователи являются частью стандартного механизма безопасности SQL, который теоретики безопасности называют дискретным управлением доступа. Этот вид системы безопасности устанавливает владельца для каждого объекта типа таблицы и набора привилегий, предоставленных этому объекту другими пользователями. Пользователи должны ссылаться на объекты, принадлежащие другим пользователям, предваряя объект именем владеющего им пользователя. И снова это соответствует идентификатору аутентификации стандарта SQL и имени схемы.

ВНИМАНИЕ

Фирма Microsoft интегрировала свою систему безопасности SQL сервера с системой безопасности базовой операционной системы Windows NT. Это упрощает администрирование БД при помощи пользователя и пароля NT для аутентификации доступа также и к БД. Не требуется определять отдельных пользователей БД и операционной системы. Пространство имен БД, таким образом, становится частью более крупного пространства имен, установленного работающей системой безопасности.

Различные поставщики DBMS расширяют эту систему дополнительным понятием — синонимом или альтернативным именем. Можно создавать альтернативные имена объекта, принадлежащего другому пользователю. Затем можно ссылаться на них, не указывая перед ними имени владеющего пользователя. При помощи синонимов можно создавать имена в пользовательском пространстве имен, которые заставляют БД с множеством пространств выглядеть как единое глобальное пространство имен. Общедоступный синоним делает имя глобальным для всех пользователей, предоставляя тем самым настоящее глобальное имя.

А как насчет составной БД? Стандарт SQL умалчивает об этом виде БД: его единственная забота — установить пространство имен схемы. Сей провал привел к появлению целого ряда различных подходов к составной БД. Oracle и Informix добавили концепцию связи базы данных. Это по сути

синоним, который относится к внешней БД в базе данных, где создается связь. Затем имя объекта во внешней БД предваряется не только именем владеющего пользователя, но также и именем связи БД.

Sybase и его кузен SQL Server компании Microsoft приняли более сложный подход. Эти системы имеют составные БД как объекты, зарегистрированные на сервере. БД и пользователи совершенно автономны. Так же имеются регистрации входа пользователя в систему, имя пользователя и пароль, с помощью которых пользователи себя аутентифицируют, и они также совершенно автономны. Можно предоставить доступ пользователю к любому объекту в любой доступной базе данных. Затем можно сделать ссылку на этот объект, предварив его именем базы данных и именем владеющего пользователя, которое может быть или не быть регистрационным именем. В данный момент пространство имен расширяется только до границ сервера, где находятся БД, благодаря способу, с помощью которого эти БД идентифицируют сервер. Имеется административная регистрация (sa — системный администратор) с правом полного доступа к чему угодно. Каждой БД соответствует принадлежащий всем набор таблиц (стандартный пользовательский dbo, владелец БД, обладает этими таблицами), который может видеть любой пользователь, имеющий доступ к базе данных.

ВНИМАНИЕ

Повторим, что SQL Server версии 7 интегрирует систему безопасности с безопасностью операционной системы.

Интересно связать все это разнообразие с проблемой реализации устойчивых пакетов UML. Во-первых, забудьте о стандартном подходе: такого не существует. Во-вторых, потратьте время на то, чтобы установить, как создаваемые приложения будут ссылаться на различные пространства имен. В идеальном случае подсистема или пакет должны ссылаться на четко определенное множество других пакетов и проигнорировать все отношения между этими пакетами.

Пример реализации подхода в отношении пространств имен в Oracle7 дан ниже, в разделе "Пакетные подсистемы".

В заключение можно установить отдельные пространства имен пакетов и подсистем и инкапсуляцию с использованием комбинации разных аспектов менеджера реляционной БД. Полного представления пакета еще не получится, но степень инкапсуляции станет больше, чем при обычной реализации системы.

Типы и домены

Установив пространства имен, определитесь, какие виды данных необходимо поместить в БД. Перед преобразованием классов UML в таблицы нужно установить типы, которые будут использоваться для объявления столбцов таблиц.

Как указывалось в главе 7, можно легко создать набор основных типов данных в отдельном пакете классификаторов, определенном с помощью стереотипа "Тип". Есть, по крайней мере, четыре альтернативы для структуризации этого пакета:

- Набор типов, соответствующих типам предоставляемым выбранной RDBMS; например, если используется Oracle7, можно определить типы NUMBER, VARCHAR2, CHAR, DATE, LONG, RAW и LONGRAW.
- Набор типов, соответствующих стандартным типам ANSI: CHARACTER, CHARACTER VARYING, BIT, BIT VARYING, NUMERIC, DECIMAL, INTEGER, SMALLINT, FLOAT, REAL, DOUBLE PRECISION, DATE, TIME, TIMESTAMP и INTERVAL.
- Набор типов, основанный на типах ООП, таких как C++: int, string, char, array of type, struct, enum и т. д.
- Набор типов, основанный на стандартах объектно-реляционной и объектно-ориентированной баз данных для максимизации переносимости между разными видами менеджеров БД. Примерами этого являются типы ODMG, определенные в главе 7, раздел "Ограничения доменов".

Существуют два вида преобразования этих типов в схему реляционной БД — создание стандартных доменов ANSI и установление таблицы преобразования.

Домены ANSI

Стандарт ANSI SQL-92 добавил понятие домена к стандарту SQL. *Домен SQL* — это "набор размещенных данных... Цель домена — ограничить набор действительных значений, которые могут быть сохранены в данных SQL с помощью различных операций" [ANSI, 1992]. Мелтон и Симон дают более толковое

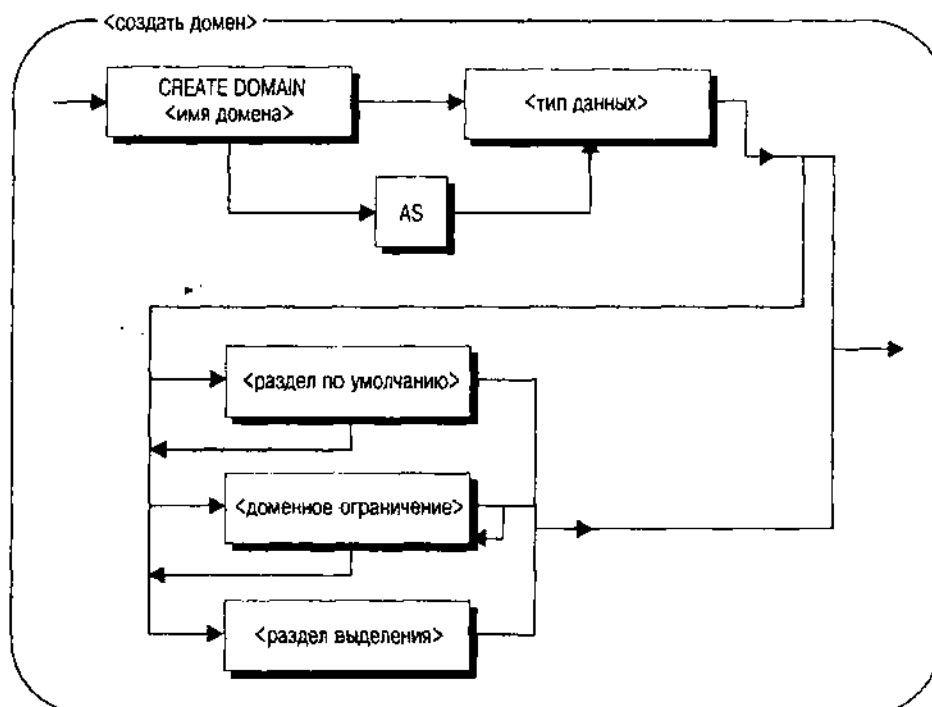


Рис. 11.4. Оператор SQL-92 <создать определение домена>

определение: "Домен – это вид макроса, который можно определить, чтобы работать вместе с определенным типом данных (и, если применимо, размером) наряду с некоторыми характеристиками [включая] значения по умолчанию, ограничения и сравнения. Затем можно использовать имя домена для определения столбцов, наследующих тип данных (и другие характеристики) домена" [Melton and Simon, 1993].

На рис. 11.4 показан синтаксис оператора ANSI CREATE DOMAIN.

ВНИМАНИЕ

Эти синтаксические диаграммы являются "железнодорожными диаграммами", потому что идут по путям, прослеживая связи от начала до конца. Они начинаются со стрелки, ничего не имея в начальной точке, и заканчиваются на стрелке, которая указывает в никуда, следуя по стрелкам между синтаксическими элементами. Если можно проследить линию, проходящую через прямоугольник, значит это необязательный элемент, наподобие <раздел по умолчанию> и другим вариантам на рис. 11.4. Такое разделение, как между <тип данных> и <имя домена> на рис. 11.6 (раздел <определение столбца>), являются альтернативами: можно идти по одному пути или другому. Если линия идет назад к прямоугольнику, значит это повторение или цикл, как в <доменное ограничение> на рис. 11.4. Допустимо не иметь ни одного или более доменных ограничений, следующих одно за другим в операторе CREATE DOMAIN. При создании оператора можно идти прямо от типа данных в конец выражения или вставить раздел по умолчанию, ограничения столбца или раздел выделения. Элементы в угловых скобках < > указывают на элементы с определениями в других диаграммах. Элементы в других прямоугольниках отделяют, по крайней мере, одним пробелом. Некоторые прямоугольники объединяют несколько элементов для удобства чтения, но можно поместить любое количество пробелов между внутренними элементами прямоугольника, если там есть хотя бы один пробел.

<Раздел по умолчанию> соответствует спецификации начального значения в определении атрибута UML (см. последующий раздел "Атрибуты"). На рис. 11.5 показан синтаксис раздела по умолчанию.

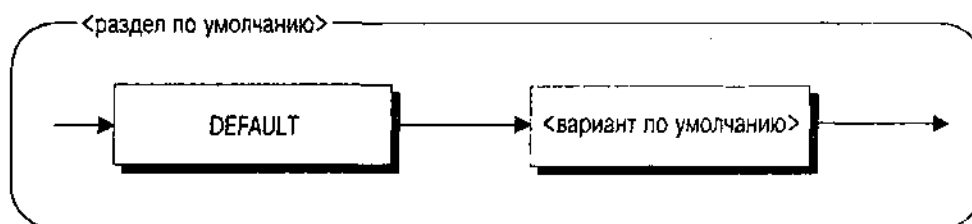


Рис. 11.5. Синтаксис <раздела по умолчанию> в SQL-92

Стандарт ANSI определяет несколько возможных значений для <варианта по умолчанию>, не реализованных в большинстве продуктов RDBMS, предоставляющих средства, которых нет в стандарте. Можно использовать стандартные значения в создаваемом UML, но их нужно преобразовать в системнозависимые значения, предоставляемые целевой RDBMS.

По аналогии можно определить свой собственный набор возможностей для использования в UML, если предоставляются таблицы преобразования для целевой RDBMS.

<Раздел выделения> на рис. 11.4 определяет последовательности выделения, если <тип данных> — символьный тип. Поскольку никто из главных поставщиков не реализует этот синтаксис, не беспокойтесь об этом.

Теперь, прочитав все, забудьте об этом. Позволим себе повторить вам в назидание кое-что о поставщиках БД и стандарте ANSI: никто из основных поставщиков не реализует оператор `CREATE DOMAIN` в своих системах. Более новые их продукты ORDBMS добавляют возможности, которые подозрительно похожи, такие как `CREATE TYPE`, но даже они не реализуют `CREATE DOMAIN` из стандарта. Следовательно, хотя разработчик мечтает о преобразовании типов в домены, на практике это безнадежно.

Таблица преобразования

Даже если создать домены в схемах RDBMS, все же необходим способ соотнести типы UML с доменами и типами данных. До преобразования классов нужно иметь таблицу в определенной форме, показывающую, как отображать выражения типов UML в SQL целевой DBMS. Если у вас несколько целей, должно быть много таблиц. С целью оптимальной повторной используемости поместите вместе диаграмму типов пакетов UML и таблицу в репозиторий повторного использования для применения в разработках составных моделей данных и схем.

Преобразование может быть простым (например, "если тип UML — строка, то тип SQL — `VARCHAR(254)`"), но обычно встречаются различные "если", "и" или "но", которые усложняют работу. Например, преобразуется ли строка в `VARCHAR(254)` или можно ли определить меньшую размерность? Или: не захочется ли иногда использовать `CHAR` вместо `VARCHAR`?

Типы перечисления создают настоящую проблему. Если есть способ определения доменов, можно закодировать ограничение `CHECK`, соответствующее списку значений в операторе `CREATE DOMAIN`:

```
CREATE DOMAIN Boolean AS CHAR (1) NOT NULL DEFAULT 'T'  
CONSTRAINT Bool_Constraint CHECK (Boolean IN ('T', 'F'));
```

При отсутствии доменов, как это обычно бывает, нужно вставить ограничение `CHECK` (и, возможно, также разделы `DEFAULT` и `NOT NULL`) в каждый атрибут, использующий этот тип. Одновременно можно создать таблицу типов и поместить в нее значения, затем создать ограничение внешнего ключа для этой таблицы. Разработчик наверняка это сделает, если полагает, что можно будет добавить новые значения к перечислимому типу через некоторое время. Для булевого типа это не так, но для других перечислимых типов зачастую справедливо.

Можно также определить некоторые значения для вывода на экран, чтобы использовать их в программах. Например, если пользовательский интерфейс предоставляет поле, получающее значение перечислимого типа, можно сделать его выпадающим списком. Затем он выведет на экран

слова живого языка, такие как "Истина" и "Ложь", для булева типа. Можно закодировать это в таблице типов, по одной таблице для каждого типа со следующей структурой:

```
CREATE TABLE <Name> Type (  
    Value <Data Type> PRIMARY KEY,  
    DisplayString VARCHAR2(50) NOT NULL,  
    Description VARCHAR(2000))
```

В этом параметрическом операторе CREATE TABLE <Name> — имя типа UML, а <Data Type> — тип данных SQL, соответствующий перечислимому значению. Тип — это обычно CHAR (n) для кода символов или NUMBER для представлений цифрового кода. Для каждого перечисляемого значения вставляют по одной строке. Так, возможные состояния официального статуса LegalStatus криминальной организации включают LegallyDefined (официально определенный), On Trial (под судом), Alleged (подозреваемый) и Unknown (неизвестен). Следующий код Oracle7 SQL создаст таблицу типов для типа UML LegalStatus:

```
CREATE TABLE LegalStatus Type (  
    Value CHAR (1) PRIMARY KEY,  
    DisplayString VARCHAR2(50) NOT NULL,  
    Description VARCHAR2(2000));  
INSERT INTO LegalStatus Type (Value, DisplayString)  
VALUES ('L', 'Legally Defined');  
INSERT INTO LegalStatus Type (Value, DisplayString)  
VALUES ('T', 'On Trial');  
INSERT INTO LegalStatus Type (Value, DisplayString)  
VALUES ('A', 'Alleged');  
INSERT INTO LegalStatus Type (Value, DisplayString)  
VALUES ('U', 'Unknown');  
CREATE TABLE LegalStatusDefault(  
    Value CHAR (1) PRIMARY KEY);  
INSERT INTO LegalStatusTypeDefault (Value)  
VALUES ('U');
```

Эта последовательность операторов SQL создает таблицу, вставляет по одной строке для каждого перечислимого значения с алфавитным кодом и задает одноэлементную таблицу, содержащую код по умолчанию. Преимущество данного подхода состоит в том, что можно добавлять или модифицировать множество значений, вставляя или обновляя строки в таблице типов. Разрешается использовать таблицу значений по умолчанию для модификации значения по умолчанию. Альтернативный способ указания значений по умолчанию — с помощью столбца с булевым дескриптором в таблице типов, IsDefault или что-то вроде того. Одноэлементная таблица более понятна в большинстве случаев, и ее легче поддерживать из приложения.

ВНИМАНИЕ

Иногда в проектах реляционных БД встречаются таблицы типов, которые объединяют несколько типов в одной таблице. Это плохая проектная идея по двум причинам. Во-первых, столбец Значение должен иметь тип значения, подходящий для перечисляемого типа. Это может быть NUMBER, CHAR или DATE. Нужно либо использовать неверный тип, взяв один из этих, или добавить столбец для каждого типа и присвоить ему пустое значение для перечисляемых типов, не принадлежащих к этому типу. Ни то, ни другое не годится для разработки. Первый подход плох потому, что он не добивается цели. Второй — потому, что в таблицу вводятся сложные ограничения. Вторая причина — ввод связности управления в БД, что нежелательно. Добавляя столбец-дискриминатор, обычно имени типа, просто для различения этих столбцов, которые относятся к данному типу, приложения заставляют добавить логику управления к выборке SQL.

Классы

Класс UML содержит атрибуты и операции. Преобразование несложно, по крайней мере для атрибутов: каждый класс становится таблицей в реляционной БД, а каждый атрибут — столбцом таблицы. Например, в подсистеме Человек с рис. 11.1 таблица создается для каждого класса диаграммы: Человек, Адрес, Идентификация, Идентификатор Срока Действия, Идентификатор Принуждения По Закону и т.д. Вот преобразование класса Человек и атрибутов в стандартный оператор CREATE TABLE:

```
CREATE TABLE Person (  
    PersonID INTEGER PRIMARY KEY,  
    Sex CHARACTER(2) NOT NULL CHECK (Sex IN ('M', 'F')),  
    BirthDate DATE NOT NULL,  
    Height FLOAT,  
    Weight FLOAT,  
    MaritalStatus CHARACTER(1) NULL  
        CHECK (MaritalStatus IN ('S', 'M', 'D', 'W')),  
    Comment VARCHAR(200))
```

Прием создания операторов CREATE TABLE состоит в преобразовании типов атрибутов в типы данных SQL и ограничения с единственным атрибутом. В следующих разделах "Атрибуты" и "Домены и типы данных" подробно рассматриваются эти решения. Существует также проблема с атрибутами первичного ключа (см. раздел "Ограничения и идентификация объектов" с подробностями о создании ограничения PRIMARY KEY).

ОСТОРОЖНО

Некоторое беспокойство может представлять длина имени класса. Стандарт SQL-92 ограничивает длину имен таблиц до максимум 128 символов, включая квалификатор имени схемы, но повезет, если система позволит сделать нечто подобное. Все системы, с которыми работал автор, имеют ограничения до 31 или 32 символов (Oracle, Informix, Sybase и сервер SQL) или даже 18 символов (DB2).

А что насчет операций? Их нет в операторе CREATE TABLE. К счастью, в большинстве реляционных БД сегодня имеются хоть какие-то возможности для предоставления поведения так же, как и структуры. См. следующий раздел "Операции и методы", содержащий некоторые предложения.

Атрибуты

Каждый атрибут класса UML становится столбцом в таблице. Повторим еще раз, что нужно позаботиться о длине имени: это 18–32 символа, в зависимости от целевой DBMS. Большинство преобразований атрибутов, однако, связаны с созданием подходящего типа данных и объявлениями ограничений.

Повторим синтаксис UML определения атрибутов из главы 7:

```
stereotype visibility name : type - expression =  
  initial - value { property - string }
```

В реляционных БД не существует стереотипов атрибутов и видимость всегда общедоступна (+), поэтому можно проигнорировать эти части определения атрибутов. См. раздел "Операции и методы", где рассмотрен способ сделать вид, что атрибуты скрытые.

На рис. 11.6 представлен синтаксис SQL-92 для определения столбцов.

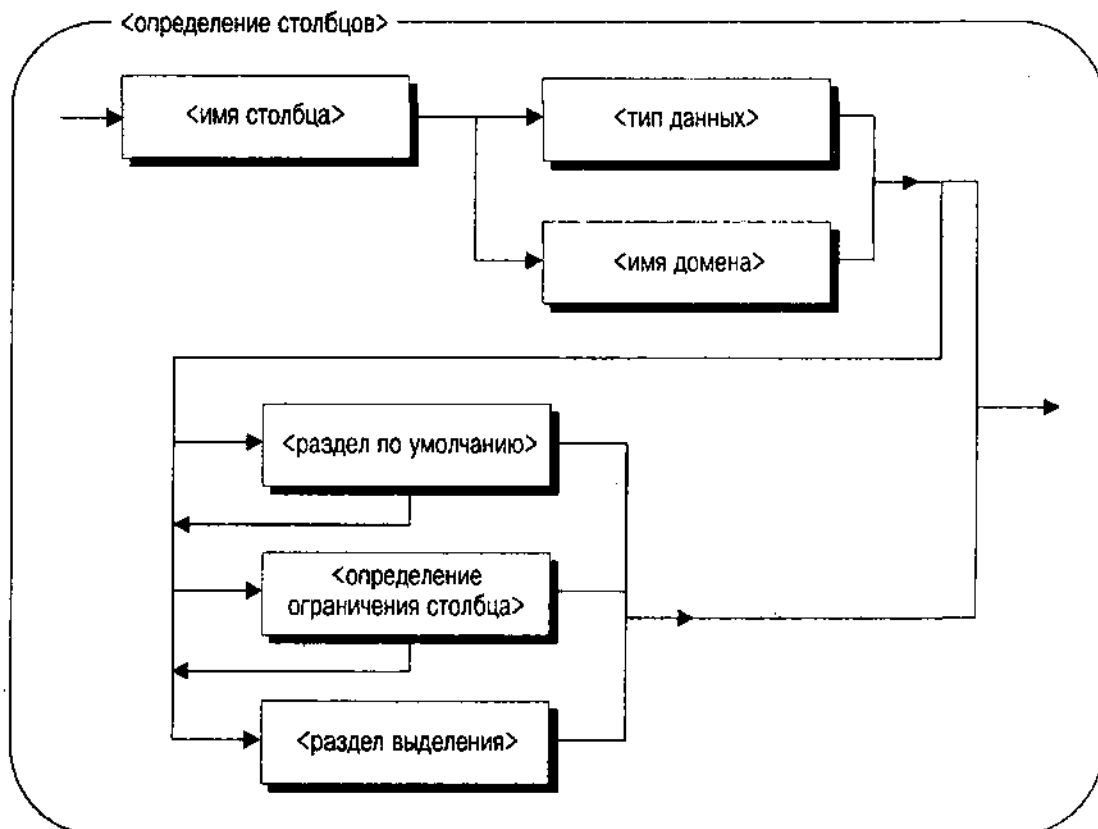


Рис. 11.6. Синтаксис SQL-92 <определения столбцов>

<Тип данных> или <имя домена> соответствует выражению типа, ассоциированному с атрибутом в UML. См. следующий раздел "Домены и типы данных" с подробностями. <Определение ограничения столбца> — один из нескольких определяемых вами типов ограничения целостности, или ограничение NOT NULL. Это соответствует дескрипторным значениям в строке принадлежности. См. следующий раздел "Ограничения и идентификация объекта".

Домены и типы данных Выражение типа может включать два вида типов: тип, непосредственно соответствующий базовому типу данных SQL, или более сложный домен, основанный на таком типе (см. предыдущий раздел "Типы и домены"). Таблица преобразования типов используется для определения подходящего типа данных SQL атрибута. Помимо этого, все усложняется.

Для перечисляемых типов нужно создать таблицу типов (см. "Типы и домены"). Затем определить ограничения внешнего ключа для отображения столбца в этой таблице:

```
LegalStatus CHAR(1) NOT NULL DEFAULT 'U'  
REFERENCES LegalStatusType(Value),
```

Это определение столбца соединяется со столбцом Value таблицы LegalStatusType для получения множества возможных значений столбца LegalStatus в таблице Криминальная Организация. Используется тот же тип данных, который определяет таблица типа (CHAR(1)). NOT NULL необязателен, как и раздел DEFAULT. Разработчик может воспользоваться подходом с таблицей по умолчанию, в которой приложение запрашивает значение по умолчанию, а предложение DEFAULT не применяется.

Для вида подтипов, которые имеются в SQL, можно определить символичный размер, цифровую точность и масштаб, а также любые другие ограничения такого типа, имеющиеся в целевой RDBMS.

Для диапазонов и других подтипов такого сорта нужно использовать ограничение CHECK:

```
IDNumber NUMBER(10,0) CHECK (IDNumber > 999999999),
```

Это определение столбца обозначает 10-значное целое число, число между 1,000,000,000 и 9,999,999,999. Обратите внимание на то, как цифровая точность сочетается с проверкой ограничения в установленном диапазоне. Можно также непосредственно обозначить диапазон:

```
IDNumber INTEGER CHECK (IDNumber BETWEEN 1000000000 AND 9999999999),
```

Чтобы исключить не целые значения, наше определение использует тип данных INTEGER. Это не присущий Oracle7 тип данных, а внутренний синоним NUMBER со шкалой 0.

Подводя итоги по приведенному примеру, можно продвинуться довольно далеко в удовлетворении большинства ограничений подтипов с использованием встроенных типов SQL и механизма ограничения CHECK. К сожалению, нужно определять все это каждый раз, когда применяется тип UML потому, что современные продукты RDBMS еще не реализуют домена

ANSI SQL-92. Если такая реализация у поставщика есть, обязательно используйте домен для представления подтипов, поскольку это значительно улучшает повторную используемость и однородность схемы.

ВНИМАНИЕ

Реляционные схемы не могут представлять структурированные или многозначные типы данных, например записи или массивы. Если разработка UML включает атрибуты с этими видами типов, нужно обращаться с ними так, как будто тип является классом. Создавайте отдельную таблицу для данных, затем свяжите ее обратной связью с таблицей принадлежности с помощью отношения внешнего ключа.

Ограничения и идентификация объектов Последовательность строк принадлежности в фигурных скобках в определении атрибутов UML включает следующие расширенные значения с дескриптором:

- *{OID}* — явный первичный ключ
- *{альтернативный OID}* — явный потенциальный ключ
- *{с возможным пустым значением}* — спецификация, определяющая, что атрибут может быть пустым вместо того, чтобы иметь значение

Раздел главы 7 "Идентификация объектов и ограничения уникальности" подробно рассматривает подходы UML к явной и неявной идентификации объектов. Реляционные БД полагаются исключительно на явную идентификацию. Реляционные теоретики поднимают это свойство до принципа, утверждая, что "вся информация в БД в любой данный момент времени должна быть абсолютно явной в терминах значений и никак иначе" [Date and Darwen, 1998, p. 145–146]. Следствие этого принципа — не может быть такой вещи, как идентификатор объекта, не являющийся значением в столбце таблицы, как не может быть и указателей на значения или строки (см. главу 12, где содержатся, например, сведения об OR концепции "ссылки").

Большинство продуктов DBMS поддерживают некоторый вид "скрытой" идентификации. Например, Oracle7 предоставляет ROWID, который уникальным образом идентифицирует строку в таблице. Однако данный ROWID может изменяться, если DBMS переместит строку в другое место на диске. Это физический идентификатор, а не логический, и разработчик не должен рассматривать его в качестве идентификатора объекта. Таким образом, нельзя хранить значение ROWID в другой таблице и использовать его позже для получения исходной строки, так как внутреннее значение может измениться. Кроме того, Oracle оставляет за собой право изменять структуру ROWID от версии к версии, как это произошло в Oracle8. Таким образом, с точки зрения и теории, и практики, в схемах RDBMS следует ограничиваться явной идентификацией.

ВНИМАНИЕ

SQL Server версии 7 предоставляет тип данных OID для явного представления идентификации объектов. Это продвигает SQL Server на шаг вперед в направлении объектно-реляционных БД, рассмотренных в главе 12. Он также интегрирует концепцию идентификации объекта непосредственно в R-модель. Нельзя сказать, чтобы это образовывало "значение" в соответствии с определением Дейта.

Выбрав явный подход, нужно определить, как преобразовать явную и неявную идентификацию UML в явную R-идентификацию. Начинать с явных атрибутов {OID} гораздо проще: достаточно создать столбцы из атрибутов и ввести для них ограничение PRIMARY KEY. Если есть всего один столбец, можно просто добавить слова PRIMARY KEY к определению столбца как <определение ограничения столбца>. Если имеется несколько атрибутов с дескрипторами {OID}, нужно добавить <определение ограничения таблицы>, чтобы обозначить PRIMARY KEY.

Преобразование неявного идентификатора UML-объекта в явный R-идентификатор немного сложнее потому, что инструментарий широко варьируется от системы к системе. Самый простой подход — добавить к таблице целочисленный столбец. Разница в том, как задаются значения этого столбца. Oracle7, например, имеет объект SEQUENCE, который можно использовать для создания монотонно возрастающих целых значений. Это непосредственно соответствует домену первичного ключа, способному использовать любое количество таблиц. SQL Server и Sybase, с другой стороны, позволяют добавить ключевое слово IDENTITY к атрибуту, который затем генерирует монотонно увеличивающееся число как часть операции INSERT. Это не соответствует домену потому, что нельзя использовать определенное значение в разных таблицах.

СОВЕТУЕМ

Следует думать о домене идентификатора с точки зрения требуемого целого значения. Компьютеры ограничивают целые значения архитектурой памяти машины. Многие продукты DBMS ограничивают целые 4 млрд или чем-то вроде (2^{32}), или половиной того, что имеется для целых со знаком (от -2 млрд до 2 млрд). Если будет менее 2 млрд (4 млрд) объектов, то все хорошо, но подумайте об этом. Прикладной код должен также понимать это ограничение и постепенно сокращаться, чтобы создавать больше объектов, когда приближается предел.

Альтернативный вариант — создать хранимую процедуру либо добавить код в приложение, чтобы генерировать эти числа. Автор насчитал для этого три способа.

Один путь — создать таблицу, имеющую одну строку для каждого домена; число увеличивают и сохраняют его как "текущее", эмулируя SEQUENCE. Единственная проблема с этим подходом — блокирование строки (или, на SQL Server и Sybase, еще хуже — страницы). Блокирование может существенно повлиять на параллелизм, если очень быстро создается много строк из множества разных соединений (сценарий OLTP).

Второй путь — запросить максимальное значение столбца таблицы, увеличить его на единицу и использовать как новое значение в операторе INSERT. Этот подход может не сработать или сработать очень хорошо, в зависимости от имеющейся в DBMS оптимизации: в худшем случае она может просканировать все строки в таблице. Он также по-разному воздействует на блокировку строк таблицы (и блокировку ее индексных страниц, если они используются в запросе).

Третий путь — каким-то образом сгенерировать уникальное число. Возможен вариант хеширования (использование простого преобразования MOD текущей даты и времени, например, или некоторого генератора случайных чисел) или генерация GUID (глобальный уникальный идентификатор). GUID также известен как uuid (универсальный уникальный идентификатор) [Leach and Salz, 1997]. Это значение генерируется с помощью стандартного алгоритма, основанного на идентификации аппаратных средств и маркере времени. Хеширование применяется в 90% случаев (реальное количество на самом деле зависит от той хеш-функции, которая используется), поэтому необходимо проверить число и добавить код, чтобы сгенерировать новое число, если INSERT не сработает из-за повторяющихся значений первичного ключа. Подход GUID дает хороший результат, если все ключи можно получить, применяя 36 байт символьного пространства фиксированной длины в таблице и в индексе первичного ключа. Для очень больших таблиц это пространство фактически может быть больше по сравнению с одним целочисленным значением.

СОВЕТУЕМ

GUID являются предпочтительным методом для доменов идентификации, которые способны управлять фактически любым количеством объектов, как в случае Holmes PLC. Если разработчика волнует, что целые значения могут быть слишком ограниченными, он должен определенно обратить внимание на GUID [Leach and Salz, 1997]. Интерфейс прикладного программирования Win32 содержит функции, которые генерируют GUID как часть библиотеки удаленного процедурного управления (rpc.h).

Потенциальные ключи требуют некоторого дополнительного синтаксиса SQL. Повторим, что каждый дескриптор {альтернативный OID} определяет число, идентифицирующее определенный потенциальный ключ. Это позволяет иметь несколько потенциальных ключей, которые содержат несколько столбцов. Для каждого уникального числа в этом дескрипторе нужно генерировать ограничение UNIQUE либо как <определение ограничения столбца>, либо как <определение ограничения таблицы>. Поскольку эти потенциальные ключи всегда явные, атрибуты создаются просто как столбцы, затем туда помещаются ограничения.

Тег UML {возможное пустое значение} соответствует ограничению NOT NULL, по крайней мере при его отсутствии, т. е. {возможное пустое значение} присоединяется к атрибуту UML, если нежелательно иметь ограничение NOT NULL в реляционной БД. Если у атрибута нет дескриптора, нужно добавить ключевые слова NOT NULL к определению столбца. Большинство продуктов RDBMS позволяют добавлять пустое ограничение, противоположное ограничению NOT NULL, хотя это и не часть стандарта ANSI для определения столбцов.

Операции и методы

Создание столбцов — только часть пути к получению законченного приложения. Столбцы таблицы включают статическую часть, а как насчет динамической? Наряду с атрибутами классы UML имеют операции. Как можно усовершенствовать эти операции в среде реляционных БД?

Основные реляционные БД ограничивают разработчика хранимыми процедурами и функциями в качестве объектов в схеме. Например, Oracle7 позволяет определить хранимые функции или процедуры. Если функции не имеют побочных эффектов, их можно использовать в стандартных выражениях SQL везде, где можно использовать скалярную величину. Если немного пофантазировать, зачастую можно представить поведение класса в виде хранимых процедур или функций, которые можно вызывать либо в выражениях SQL, либо как часть более крупной хранимой процедуры. Их также вызывают из прикладных программ при помощи различных нестандартных API. Имеет ли это смысл — зависит от природы поведения, более крупной системной архитектуры, чьей частью является БД, и требований к повторному использованию среди приложений.

Однако, во-первых, важно понимать различие между сервером БД и кодом приложения. Оно произвольно: одни вещи имеют значение (серверное поведение БД), другие нет. Это и является искусством разделения приложения.

Разделение Имеются целые классы операций, которые нежелательно иметь на сервере. *Разделение* — это процесс решения, разместить ли поведение на сервере БД, на сервере приложений или на клиенте.

ВНИМАНИЕ

Мир полон делений. Что касается компьютеров, в различных контекстах можно разделять любое количество разных вещей. Вследствие этого читатель нашей книги может запутаться в определениях одного и того же слова для обозначения разных вещей. В частности, дифференцируйте разделение приложения, разделение приложения между клиентом и сервером или сервером и сервером при помощи разделения данных, разделение таблицы или другой коллекции данных на разные физические куски для оптимизации физического ввода/вывода.

Во-первых, не нужны операции, управляющие временными проблемами. Любые операции, требующие ссылки на адрес, находящийся в оперативной памяти, или на находящийся там объект, не годятся для сервера БД, поскольку этот сервер не имеет доступа к таким вещам.

Во-вторых, не нужны вызываемые клиентом операции, которые получают доступ к атрибуту данных с помощью атрибута. Напомним из главы 7, что можно создать операции с помощью свойства {запрос}, называемого здесь аксессорами или наблюдателями, которые просто возвращают значение атрибута (обычно как постоянную ссылку в C++, например). Серверы БД действуют другим образом: они возвращают записи, а не значения. Представьте себе очень большой туннель, в котором находится огромный грузовой поезд, перевозящий за каждый рейс по зернышку кукурузы. Потребуется большое количество ресурсов, чтобы доставить на рынок хотя бы мешок кукурузы. Передача данных по одному значению по сети — не самая лучшая идея. Данное соображение справедливо для любых операций на одиночных атрибутах. Например, простые мутаторы (операций задания значений) часто изменяют значение единственного атрибута. Нужно сделать это на стороне клиента или прикладного сервера, затем отправить

изменения как целые строки данных, когда транзакция готова к завершению. Если можно сгруппировать несколько атрибутов для единственного обновления, это лучше, но все еще имеет ограничения.

ВНИМАНИЕ

Если операции предназначаются только для использования на сервере БД (т.е. другими хранимыми процедурами), тогда, вероятно, можно выйти из положения с помощью аксессора или мутатора в качестве хранимых функций. Если таков ваш выбор, не предоставляйте функции в распоряжение приложений.

В-третьих, нежелательно размещать на сервере операции, которым в любом случае нужно ссылаться на временного клиента. Только в плохих разработках серверу требуется поддерживать состояние поведения между запросами. Достаточно поработать над управлением состоянием транзакции. Не нужно говорить, что этого сделать нельзя: пакеты PL/SQL, например, позволяют установить значения данных и хранить их в специальном общем буфере для такого количества вызовов, какое захотите. Это приводит к странным последствиям, поскольку может вырасти количество клиентов, получающих доступ к данным в одно и то же время. Обычно требуется, чтобы серверный код был *реентерабельным*, т.е. сессия могла иметь доступ и выполнять поведение, не беспокоясь ни о чем из того, что произошло раньше. А значит, независимо от любого состояния, оставшегося от предыдущих вызовов операции (статических данных или данных из таблиц). Оставьте эти виды операций на сервере.

Кроме того, некоторые операции выполняют интенсивную проверку ошибок в памяти, прерывая операцию, если что-то сделано неправильно. Проводите эту проверку на клиентской стороне или на стороне прикладного сервера, а не на стороне сервера БД. На стороне сервера надо будет только управлять ошибками, касающимся структур БД (проверка бизнес-правил, ошибок физической памяти с данными, ошибок DBMS и т.д.). Это в первую очередь применимо к коду, который, видимо, находится в триггере. Из-за того что триггеры работают, когда DBMS создает событие, их деятельность фактически не контролируется. Проявляйте благоразумие при выборе кода, помещаемого в триггер. Нужно кодировать триггерные операции, имеющие смысл для события, которое запускает триггер (AFTER INSERT и т.д.). Если разработчик создает триггерный код для решения некоторых проблем с помощью хранимой процедуры, рассмотрите другие варианты. Если он создает триггерный код для работы с другим триггерным кодом, скорее откажитесь от этого решения! Для триггеров подходят только простые решения.

И наконец, разработчику желательно оценить производительность оставшихся операций. Возможно, дешевле выполнять операцию на компилируемом языке на сервере приложений, а не на интерпретируемом языке, замедляя хранимую процедуру PL/SQL.

Итак, какие же операции остаются?

- *Запрос множества результатов*: возвращает клиенту множество записей

- *Вставить/обновить/удалить*: модифицирует данные отдельной строки (а не единственное значение)
- *Проверка правил*: обеспечивает выполнения бизнес-правил с помощью триггеров или процедур (например, можно управлять специальными ограничениями внешних ключей, используя триггеры или добавляя код, который оценивает сложные процедурные ограничения на введенную строку или строки)
- *Производные значения*: вычисляет значения на основе данных из БД с помощью алгоритма, который нельзя определить как выражение SQL в версии SQL целевой DBMS
- *Инкапсуляция операций*: вызывает другие операции БД, такие как хранимые процедуры, предоставляемые поставщиком, или доступны к словарию данных, относящемуся к данным в таблице (например, хранимая процедура для получения определения столбца первичного ключа таблицы)
- *Проверка ошибок*: проверяет различные ошибочные состояния, и порождает исключение (или любой метод, с помощью которого ошибки возвращаются клиенту)

Стимулы и ответ: поведение и побочные эффекты Многое в мире реляционных БД зависит от природы поведения, которое вы намереваетесь поместить в устойчивые классы. Oracle7, Informix и Sybase предоставляют средства программирования на серверной стороне, но имеются различные тонкости в отношении структуры и использования, зависящие непосредственно от того, что происходит внутри черного ящика операции.

Oracle7.3 предоставляет как хранимые процедуры, так и хранимые функции. Можно использовать эти функции в операторе SQL, но только если у них нет побочных эффектов. В терминах UML операция должна иметь дескриптор {запрос}. Это позволяет синтаксическому анализатору Oracle7 вызывать функцию и возвращать значение, зная, что функция не нарушит внутреннюю обработку DBMS.

В зеркальном отображении, если операция не имеет дескриптора {запрос}, нельзя использовать ее как функцию SQL. Однако можно добавить ее к набору объектов БД как хранимый программный блок, применяемый другими программными блоками или клиентом через некоторую разновидность API для вызова хранимых блоков из программ.

Система безопасности SQL вводит другой фактор — привилегии. Один из часто задаваемых вопросов на телеконференциях Usenet по Oracle: "Я написал хранимую процедуру и имею доступ ко всем данным, которые там используются, но не могу выполнить ее из моей программы". Это происходит обычно потому, что пользователь регистрируется под именем, отличным от имени владельца хранимой процедуры и ему не предоставлена привилегия EXECUTE для процедуры того пользователя. Различные продукты RDBMS имеют много разных тонкостей в отношении привилегий работы с хранимыми программными блоками.

Третий фактор — внешний доступ, требуемый данной операцией. Многие программисты желают открыть или записать файлы, что-то напечатать или вывести на экран из хранимого программного блока. RDBMS позволяет все это выполнить, но понять ее действия непросто в контексте клиент/сервер. Обычно, если что-то запускается вне DBMS, код выполняется на сервере. Таким образом, печать выводится на принтеры, подсоединенные к серверу, а не на те, что подсоединены к клиенту. Файлы записываются на диски, доступные серверу, а не на жесткий диск клиента (если, конечно, он недоступен серверу, идентифицирован и запись на него производится с использованием параметров). Отправка почты, управление внешними устройствами, синхронизация с другими приложениями — все эти операции гораздо больше осложняют новые операции, если выполняются из хранимых процедур.

Другая форма доступа — доступ к данным, находящимся в оперативной памяти. Зачастую программисты хотят ссылаться в хранимом программном блоке на данные в своих работающих программах. А так как последние выполняются в разных процессах и, потенциально, на разных машинах, то это невозможно. Все такие ссылки должны быть параметрами хранимой процедуры. В своей предельной форме это приводит к *программированию без данных* о состоянии — гарантия того, что хранимый программный блок получает все, что ему нужно, через свои параметры и не делает никаких предположений о предыдущих запросах или действиях в БД. Часто можно будет встретить процедуры со встроенным завершением или отменой транзакций, поскольку отсутствие данных о состоянии подразумевает отсутствие переноса транзакций через вызовы доступа к серверу.

Это приводит к последнему компоненту поведения — требованиям транзакций. Если транзакции сложны, нужно будет решить на определенном этапе, где они совершаются. Обычно в приложении, а не на сервере БД. Другими словами, обычно команды транзакций (COMMIT, ROLLBACK, SAVEPOINT) не помещают в хранимые программные блоки. Вместо этого сервер приложения (или клиент в двухзвенной системе) выдает команды отдельно, потому что это ядро семантики транзакций. Какое бы подходящее решение ни было выбрано, затем нужно обратить внимание на то, что смешивание запрещено. Если некоторые хранимые процедуры имеют логику транзакций, а другие нет, программисты окончательно запутываются в том, должны ли они кодировать завершение и отмену. И удивляются, если (в Oracle7) пытаются отменить данные, обновление которых хранимая процедура уже завершила. Программисты недоумевают, когда (в SQL Server или Sybase) получают исключительное состояние или ошибку базы данных при завершении, поскольку нет активной транзакции из-за того, что хранимая процедура завершилась или была отменена ранее.

Проблема с логикой транзакций приводит нас к следующей теме — системная архитектура.

Язык образцов: архитектура и видимость Системная архитектура определяет многое в подходе разработчика. Если UML-проект объектно-ориентированный, в отличие от реляционных БД, нужно сблизить архитектуры друг с другом. Лучше, если R-система будет выглядеть более

объектно-ориентированной, чем наводить беспорядок в объектно-ориентированную системную архитектуру, чтобы она выглядела реляционной. Автор осуществлял оба варианта и считает, что последний подход определенно плох для восприятия.

К сожалению, продукты RDBMS не переполнены ОО-возможностями. Это отсутствие отчасти привело к первоначальному утверждению продуктов OODBMS — "полное несовпадение" ОО-и R-структуры. Существует целый ряд различных способов преодолеть это несовпадение, но ни один из них не является полностью удовлетворительным.

На стороне БД можно использовать хранимые процедуры и пакеты для инкапсуляции объектов. Например, в Oracle7 создавать пакеты, соответствующие классам в модели данных UML, в других системах просто использовать совокупность хранимых процедур. Операции, имеющие смысл для сервера БД, реализуются как заключенные в пакеты программные блоки. Базовые таблицы и пакеты создаются в схеме, принадлежащей конкретному пользователю — "владельцу" прикладной схемы. Разработчик не дает права доступа к таблицам — вместо этого он гарантирует соответствующие привилегии, такие как EXECUTE, пользователям, которым нужен доступ к объектам класса. Процедуры предоставляют интерфейс сервера БД (API) для класса. Когда приложение регистрируется как разрешенный пользователь, код запрашивает процедуры, а не использует SQL для запроса или работы с базовыми таблицами.

Этот подход снижает несовпадение двух подходов. Во-первых, он упаковывает базовые таблицы как классы, а не как таблицы. Это позволяет использовать ОО-структуру, а не заставляет осуществлять программирование модели простых таблиц. Во-вторых, данный подход исключает программный интерфейс SQL, сводя любой доступ к БД к процедурным запросам. Такие запросы гораздо ближе к ОО-способу мышления, хотя и не совпадают с ним полностью. Все еще требуется передавать идентификатор объекта (т. е. первичный ключ или идентификатор строки) большинству процедур как параметр, поскольку пакет не позволяет создавать экземпляр каждого объекта в программе. Значит, процедуры независимы от состояния любой индивидуальной строки или строк в нижележащих таблицах, поэтому нужно передавать первичные ключи или другие идентификаторы, чтобы работать с правильными данными.

Кроме того, этот подход предоставляет полную инкапсуляцию данных. Никакое приложение не может осуществлять непосредственный доступ к данным в таблицах. Таким образом, можно защитить данные от случайного или преднамеренного повреждения через неавторизованный доступ к табличным данным. Все осуществляется через пакетные процедуры. Это предоставляет такой же контроль видимости, какой дает ОО-язык при помощи ограничений контроля доступа и рабочей видимости. Однако мы имеем дело с подходом типа "все или ничего": необходимо прятать все табличные данные, а не только некоторые из них.

Ограничения этого подхода — сама гипотеза несовпадения: отсутствие SQL. SQL — декларативный язык. Используя его, определяют, что требуется, а не как это получить. Хранимые процедуры, с другой стороны

(и ОО-языки), процедурны: компьютеру точно говорят, как перемещаться по данным с помощью запросов процедур. SQL существует исключительно потому, что гораздо легче использовать декларативный язык для определения сложных требований к данным. Исключая SQL, процедурный подход исключает все преимущества, которые SQL предоставляет разработчику. Можно полностью вести разработку только ОО-средствами, но при этом не достичь эффекта в создании кода для управления данными. Для простого доступа к данным это обычно не проблема. Но если надо добавить больше, чем несколько строк кода к пакетной процедуре, следует пересмотреть свой подход.

Переработанные приложения: переносимость и повторное использование "Серая вода" — это вода, переработанная из потребляемой в домашнем хозяйстве, такая как вода из ванной, для повторного использования — орошения или других нужд, где не нужна "чистая" вода. Многие юридические документы несправедливо запрещают применение серой воды, полагая, что грязная вода — это зло. Серая вода — полезная метафора для операций реляционной БД. Каждое переработанное приложение использует свою технологию, что затрудняет разработку стандартного подхода. Следовательно, люди используют чистую воду в тех же целях, в каких они применяли бы и серую, и серую воду сливают, не используя повторно. В итоге много воды расходуется напрасно.

Подобным образом различные продукты RDBMS имеют совершенно разные подходы к поведению БД. Стандарт SQL-92 умалчивает об этом, оставляя простор для творчества поставщиков. Многие компании ставят вне закона использование хранимых процедур и другие поведенческие конструкции, такие как триггеры, на основании того, что это делает БД менее переносимой. У автора имеется своя собственная позиция: это неверно. Правильно потому, что код переносим, и неверно потому, что произвольно лишает инструмента, который можно эффективно применить для достижения повторного использования на сервере БД. Существенная доля потенциала повторного использования сливается в канализацию.

Святой Грааль портативности состоит в том, чтобы сократить количество кода, требующего работы по перемещению на другую систему, до нуля. Получив чашу Грааля, можно перемещать систему с одной платформы на другую без всяких усилий -- основа в повторном использовании кода.

Фокус с чашами Грааля в том, что очень немногие люди могут увидеть или получить их, и за этим стоит огромное количество работы, даже если есть проблемы с сердцем, телом и душой. Обычно под полной переносимостью БД имеется в виду как уровень производительности, так и уровень продуктивности кодирования, что близко к неприемлемому. Многие менеджеры полагают, что, настаивая на полной переносимости, они сэкономят миллионы на поддержке и затратах на перенос программ. Это не что иное, как магический способ мышления в шаманской культуре.

Например, разные продукты DBMS имеют совершенно разные архитектуры обработки транзакций. Некоторые поддерживают блокировку на уровне страницы; другие поддерживают блокировку на уровне строки. Некоторые поддерживают однородность по чтению (Oracle7); другие — нет.

Некоторые продукты RDBMS автоматически поддерживают временные таблицы (Sybase, SQL Server, Informix); другие — нет (Oracle7). Некоторые продукты RDBMS поддерживают UNICODE в качестве стандартной международной языковой поддержки (Oracle7); другие — нет (SQL Server). Даже не просите автора начать рассмотрение оптимизаторов SQL. Переносимость? Только если отсутствуют транзакции, не используются временные таблицы и нет потребности в срочной выборке данных.

Видимо, глупо не ввести ограничения на диапазон целевых платформ, которые имеются в виду. Глупо также не воспользоваться преимуществами основных свойств продуктивности и производительности целевой DBMS. В самом лучшем из миров примут стандарт, которого будут придерживаться все поставщики. Пока такого стандарта не существует и не будет существовать для реляционных БД, особенно в отношении оптимизации и транзакций, а также глобализации и символьных строк. Экономика не допустит этого, установленная база и Microsoft/IBM/Oracle тоже. Автор не будет возражать, но разработчики иногда должны подчиняться реальности.

Наилучший выход из этой ситуации — рационально оценить, какие части приложения могут получить преимущества от использования переносимого подхода, повторного использования, а какие извлекут максимум пользы от кодирования хранимых процедур на сервере БД. Это не будет на 100% один или другой подход. Как и в случае с серой водой, выяснится, что использование чего-то грязного вполне годится, если совершается в подходящем месте.

Специальные темы

После рассмотрения основных тем, касающихся операций, нужно также обсудить некоторые специфические вещи: полиморфизм, сигналы и интерфейсы.

Полиморфные операции Напомним из главы 7, что некоторые операции полиморфны: они используют одно и то же имя для разного поведения. *Перегрузка* использует то же имя операции, но другую сигнатуру (параметры), в то время как *переопределение* применяет одну и ту же сигнатуру в подклассе. Перегрузка в основном носит косметический характер, позволяя командам выглядеть одинаково, даже если они отличаются, основываясь на объектах. Переопределение работает с динамическими связыванием, что дает настоящие преимущества: возможность вызывать операцию, не зная, объект какого типа вызывается. На рис. 11.7 представлена часть рис. 11.1 из подсистемы Человек, которая иллюстрирует операции переопределения.

Абстрактные классы *Идентификация* и *ИдентификаторСрокаДействия* определяют абстрактные операции. Листовые классы реализуют эти операции, заменяя абстрактные операции конкретными. Это позволяет создавать листовый объект, но ссылаться на него, как *Идентификация* и *ИдентификаторСрокаДействия*, затем вызывать *ПолучитьЮрисдикцию* или что угодно, не требуя знания, каким специфическим, конкретным классом на самом деле является объект.

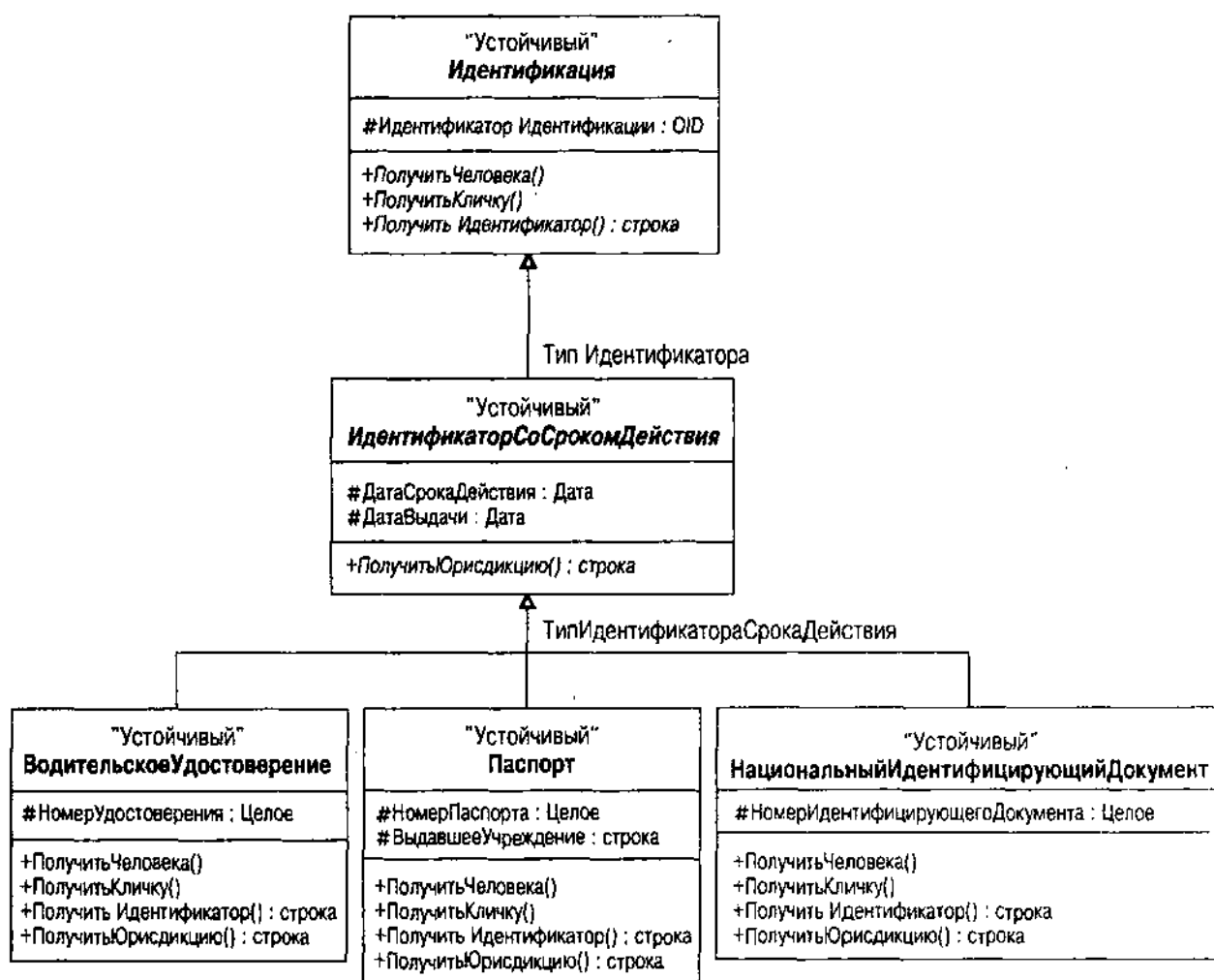


Рис. 11.7. Операции переопределения в иерархии идентификации

Большинство реляционных БД не поддерживают никакого полиморфизма, но автору известно одно исключение. Oracle7 PL/SQL поддерживает перегрузку в процедурах пакетов. Можно предоставить несколько процедур с одним и тем же именем, если типы объектов, передаваемые с помощью параметров, отличаются по типу или порядку (т.е. разный тип *сигнатуры* операции). Первопричина разработки таких видов перегружаемых программных блоков — обеспечить альтернативный список параметров для несколько другого поведения при поддержке одного и того же имени. Используя перегружаемые процедуры, можно пропустить "необязательные" параметры или переставить параметры так, чтобы программисту не нужно было помнить порядок. Повторим, что это косметические изменения, которые не приводят к огромному росту продуктивности, но это хорошее средство.

Например, пусть подсистема обработки изображений в каталоге имеет возможность сравнивать отпечатки пальцев, почерки и изображения лиц с библиотеками отпечатков пальцев, записями почерков и фотографий. Эта услуга позволяет консультирующим детективам использовать доказательства собранные на месте преступления, для идентификации людей, которые могут помочь следствию.

"Однажды опознав своего человека, было легко получить подтверждение. Я знал фирму, с которой работал этот человек. Взяв напечатанное описание, я исключил из него все, что могло быть следствием маскировки — усы, очки, голос, — и послал его в ту фирму с просьбой проинформировать меня, соответствует ли это описанию какого-либо из их путешественников. Я уже обратил внимание на сходство шрифта пишущей машинки и написал самому человеку по его рабочему адресу, попросив его прийти сюда. Как я и ожидал, его ответ был напечатан на пишущей машинке и демонстрировал те же простые, но характерные дефекты. С той же почтой ко мне пришло письмо от Вестхауз&Марбанк с улицы Фенчерч, в котором сообщалось, что описание совершенно совпадало с описанием их сотрудника Джеймса Виндибенка. *Вот так-то!*" [IDEN]

Параметры операции Сравнения этих классов будут совершенно другими. Например, операция Сравнения отпечатков пальцев может определять, что отпечаток — это отпечаток большого пальца или мизинца, или что изображение содержит два или более соединенных отпечатков. Изображение шрифта печатной машинки может содержать параметры, которые идентифицируют текст изображения или производителя пишущей машинки. Изображение лица может содержать список черт, игнорируемый при сравнении, чтобы исключить маскировку. Все классы имеют несколько операций Сравнения, ни одна из которых не имеет одинаковых типов параметров или сигнатур. Когда вызываемая операция подразумевает вызов Сравнения, компилятор решает, какой метод запросить, основываясь на параметрической сигнатуре.

Истинное переопределение, динамическое связывание или виртуальные операции просто не существуют в реляционном мире. Что нужно сделать в R-программном блоке для реализации модели, если модель UML зависит от этого?

Проблема с переопределяемым поведением состоит в том, что система времени выполнения нуждается в таблице просмотра, чтобы решать, какую процедуру выполнить. В C++ каждый класс с виртуальными методами имеет *table* — таблицу функциональных указателей, которая позволяет работающей системе C++ вызвать переопределяющий метод, а не единственный метод, определяемый компилятором при создании системы. Среди R-систем существует простой выбор: или создать возможность динамической привязки в процедурном языке, или просто проигнорировать ее. Совет автора: чем проще, тем лучше. Например, он видел, сколько рабочих часов было потрачено на системы для эмуляции динамического ОО-связывания и наследования в контексте программирования на С. Обычно это более-менее действует, но всегда за определенную плату — стоимость поддержки исключительно сложного кода, затраты на объяснение принципа работы кода, и ваши усилия, чтобы сказать боссу, почему это на самом деле полезно, хотя и снижает производительность.

Таким образом, к чему же в итоге мы приходим со своей разработкой? Нужно переименовать переопределяемые операции, используя только уникальные имена и вызывая их в подходящих местах. Значит, нужен более

замысловатый условный код наподобие операторов выбора или условий если-то-иначе, проверяющих типы участвующих объектов и решающих, какую процедуру вызвать.

Возьмем, например, операцию ПолучитьЮрисдикцию. Она возвращает разные данные для каждого класса, поскольку тип юрисдикции широко варьируется в зависимости от классов. Графства выдают свидетельства о рождении, национальные органы власти — паспорта и национальные идентификационные документы, а штаты — водительские удостоверения. В ОО-системе все эти операции переопределяют одного и того же абстрактного предка. В Oracle7 это невозможно, поэтому нужно предоставить другие хранимые процедуры для каждой операции. Пакет ВодительскоеУдостоверение имеет свою операцию ПолучитьЮрисдикцию, а пакет Паспорт — свою. В таком случае используется пространство имен пакетов для определения различия между двумя функциями. В коде PL/SQL для этого в качестве префикса применяется имя пакета, например: ВодительскоеУдостоверение.ПолучитьЮрисдикцию().

В других процедурных языках, не имеющих пакетов, нужно использовать соглашение об именах для различения процедур. Можно, скажем, добавить к имени операции имя класса — ВодительскоеУдостоверение_ПолучитьЮрисдикцию(). Часто рекомендуется использовать уникальное сокращение имени класса для сокращения всего имени, чтобы получить его в пределах ограничений длины идентификатора SQL для RDBMS — ВодУд_ПолучитьЮрисдикцию(), например.

ВНИМАНИЕ

Следует понимать, что несмотря на создание этих методов, разработчик не получает никаких преимуществ полиморфизма. Код SQL или PL/SQL должен точно знать, с объектом какого вида вы имеете дело, чтобы вызвать метод, поэтому разработчик не получает преимуществ динамического связывания через наследование.

Сигналы Если разрабатывается операция UML со стереотипом "сигнал", значит операция отвечает на событие некоторого рода. Это соответствует триггеру, когда события ограничивают определенным набором триггерных событий. Большинство продуктов RDBMS сейчас поддерживают триггеры, хотя стандарта синтаксиса ANSI для них в SQL-92 нет. Стандарт SQL3 определяет оператор CREATE TRIGGER (см. рис. 11.8).

Событие, включающее триггер, — некая точка в работе системы БД. Рис. 11.8 определяет стандартные события для "сигнальных" операций в БД как разнообразные комбинации <время действия триггера> и <событие триггера>:

- BEFORE INSERT
- AFTER INSERT
- BEFORE DELETE
- AFTER DELETE
- BEFORE UPDATE
- AFTER UPDATE

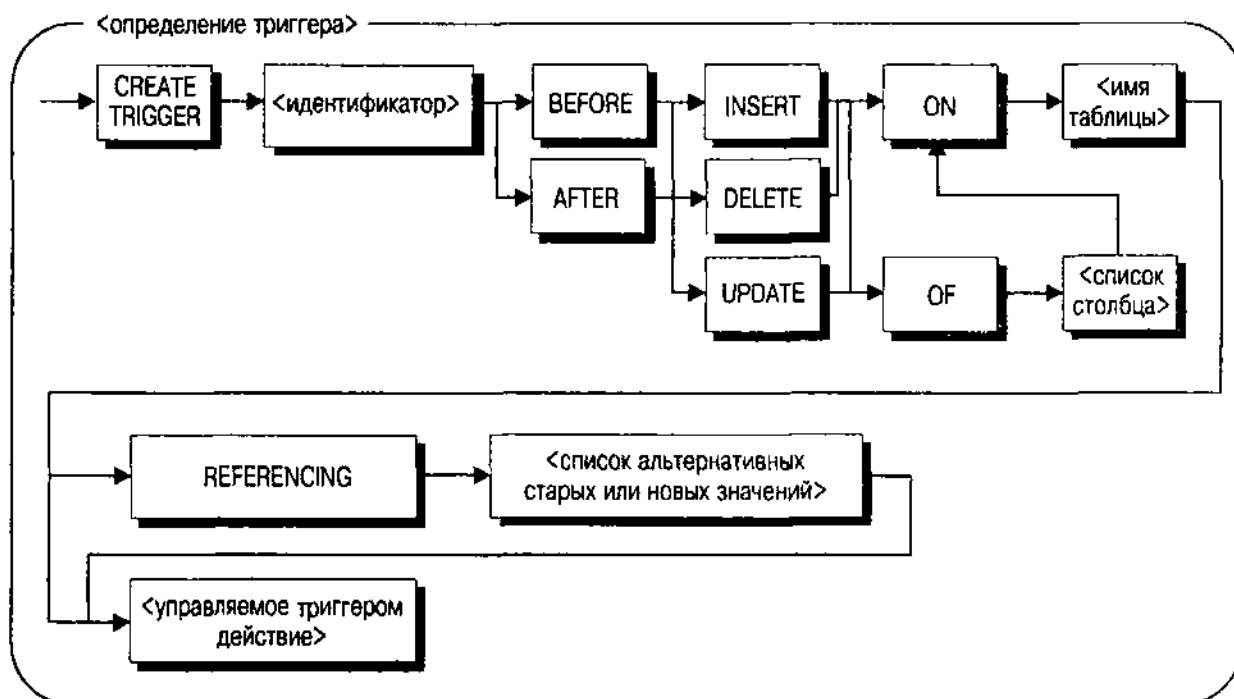


Рис. 11.8. Синтаксис SQL3 <определение триггера>

Следовательно, можно определить "сигнальные" операции в устойчивых классах с помощью этих имен, преобразованных подходящим образом в любое используемое соглашение об именах, например "BeforeInsert". Операции "before" выполняются перед соответствующими операциями БД (вставить, удалить или обновить), в то время как "после"-операции — после них.

ВНИМАНИЕ

При преобразовании "сигнальных" операций в триггер БД нужно использовать процедурный язык, имеющийся в системе, такой как Transact/SQL для SQL Server и Sybase, PL/SQL для Oracle7 и Oracle8, или язык программирования C для DB2. Каждая система имеет свои собственные рекомендации и приемы, касающиеся триггеров, поэтому используйте документацию своей системы для определения лучшего способа перемещения кода из модели данных в схему БД. Как и любую операцию, можно использовать процедурный язык из целевой DBMS или некоторый нейтральный псевдокод, или же язык программирования для представления триггерной реализации в модели данных.

Другие части синтаксиса определения триггера варьируются от продукта к продукту. Стандарт SQL3 содержит предложение REFERENCING, позволяющее ссылаться как на старые, так и новые значения в действии UPDATE. Само по себе действие — это последовательность процедур SQL. В Oracle7, например, эквивалентом является блок PL/SQL.

Интерфейсы Интерфейсы являются специальным случаем класса — класс без атрибутов, только операции. Таким образом, устойчивый интерфейс — что-то вроде противоречия в терминологии, особенно для реляционной БД. Следует считать интерфейс простой нотацией полиморфизма

(см. предыдущий раздел "Полиморфные операции"), т.е., если устойчивый класс реализует интерфейс (см. главу 7 о нотации), нужно реализовать хранимые процедуры, соответствующие интерфейсным операциям, если они годятся для выполнения на сервере DBMS.

Таким образом, если разрабатывается иерархия Идентификации как иерархия интерфейсов, как в главе 7 (рис. 7.9), нужно реализовать операцию ПолучитьИдентификацию для каждой таблицы, генерируемой из классов, реализующих Идентификационный интерфейс. Повторим: нет никаких преимуществ полного динамического связывания с применением реализации интерфейса, но в действительности необходимо реализовать проект, в полной мере используя имеющиеся R-средства.

Внешние связи

В реляционной БД нет отношений. Все является таблицей. Это означает, что отношения нужно создавать в таблицах, используя специальные столбцы. Способ, применяемый для проекта на UML, отличается от того, что используется при стандартном проекте сущность-отношение, но общего у них больше, чем отличий. Основные отличия связаны с генерализацией (наследованием) и со специальными свойствами UML, такими как агрегации и специализированные ассоциации.

Двоичные ассоциации

Двоичные ассоциации — это отношение между двумя классами, когда каждая сторона ассоциации является ролью и имеет множественность (глава 7 "Ассоциации, содержание и видимость"). Роль — это то, что класс играет на данном конце ассоциации в отношении. Множественность — это ограничение количества объектов, которые могут играть роль в отдельной связи. Связь — экземпляр ассоциации, отношение между объектами, противоположное тому, что имеется между классами.

Преобразование двоичной ассоциации в R-схему довольно прямолинейно и напоминает преобразование ER-отношения. Для его осуществления нужно прежде всего понять идею ограничения внешнего ключа в реляционной БД.

Внешние ключи

Внешние ключи — один или более столбцов таблицы, использующих свой объединенный домен с помощью первичного ключа другой таблицы. Это способ, с помощью которого R-модель данных представляет ассоциацию и семантику ее данных, утверждая, что одна строка (объект) относится к другой строке (объекту) хорошо определенным, простым в использовании способом. Реляционные БД позволяют соединять таблицы по этим внешним ключам как первичный способ выборки данных, обладающих смыслом для заданных отношений между объектами. Теория реляционных БД называет всю область первичных и внешних ключей *ссылочной целостностью* — идея, которая состоит в том, что внешние ключи всегда должны ссылаться на действительные первичные ключи, чтобы быть согласованными.

SQL обеспечивает ограничения REFERENCES и FOREIGN KEY как способ поддержания ссылочной целостности на сервере БД. Предложение REFERENCES (рис. 11.9 иллюстрирует <предложение REFERENCES>) – ограничение строки, позволяющее связать отдельный столбец с первичным ключом единственного столбца в другой таблице. Раздел FOREIGN KEY (рис. 11.10 иллюстрирует <ограничение внешнего ключа>) – ограничение таблицы, позволяющее связать более одного столбца с первичным ключом в другой таблице.

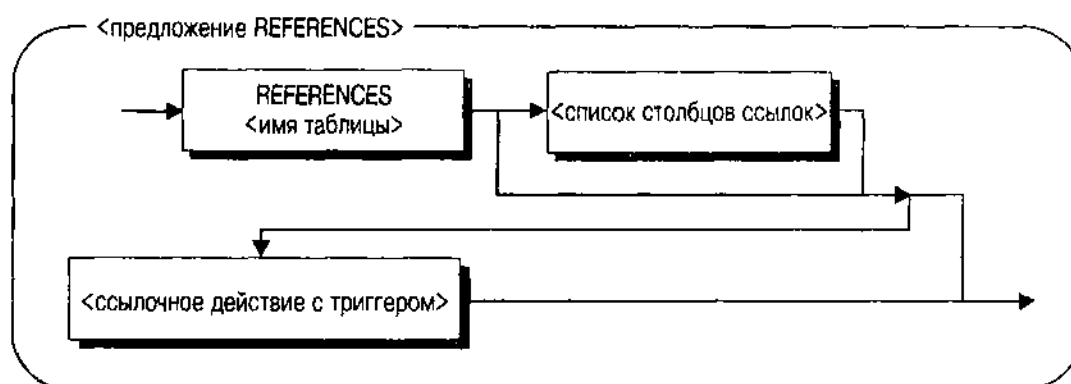


Рис. 11.9. Синтаксис SQL <предложение REFERENCES>

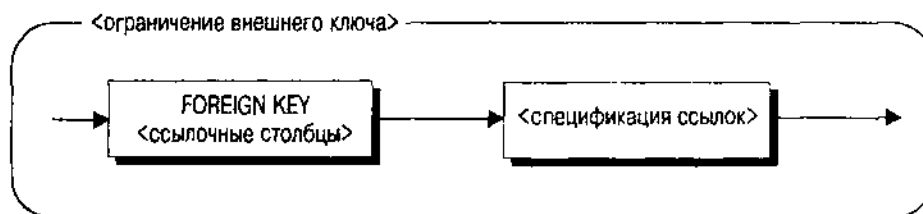


Рис. 11.10. Синтаксис SQL <ограничение внешнего ключа>

Синтаксис табличных ограничений использует <предложение REFERENCES>. В контексте ограничения столбца <предложение REFERENCES> может иметь только один столбец в своем <списке столбцов ссылок>.

Когда это ограничение помещают в таблицу, предполагается, что, если вставить строку со столбцом внешних ключей с непустыми значениями, значения должны соответствовать значениям первичных ключей другой таблицы. Если значения столбца внешних ключей обновляются, новые значения должны соответствовать значениям первичных ключей другой таблицы. Можно также иметь ссылочные триггерные действия (см. нижеследующий раздел "Общие и композитные агрегации" с подробностями о данном синтаксисе и области его применения).

Получив все эти возможности для выражения внешних ключей, перейдем к объяснению того, как преобразовать двоичные ассоциации в подобные выражения.

Роли, множественность и столбцы внешних ключей

Двоичная ассоциация — это отношение между двумя классами, предполагающее, что один или оба класса видимы другим. Это в точности соответствует понятию внешнего ключа в теории реляционных БД. Как можно ожидать, преобразование прямолинейно: каждая двоичная ассоциация преобразуется в ограничение REFERENCES или FOREIGN KEY в определениях таблиц. Интересно создание не ограничений, а столбцов внешних ключей, в которые эти ограничения помещаются.

Разработка UML не определяет атрибутов внешних ключей. Она представляет их как двоичную ассоциацию. Эта ассоциация, в свою очередь, имеет две роли (по одной для каждого класса, участвующего в ассоциации) и две множественности (также). Роли и множественности управляют преобразованием для добавления строк внешних ключей.

Для определения структуры преобразования нужно учесть как индивидуальные множественности, так и их комбинацию между двумя ролями.

Индивидуальные множественности могут иметь целый ряд различных конфигураций:

- 0..* — ноль или более объектов
- 0..1 — не более одного необязательного объекта
- 1..* — по крайней мере, один объект
- 1 — ровно один объект
- * — ноль или более объектов
- 2..6 — по крайней мере, два, но не более шести объектов
- 1, 3, 5-7 — по крайней мере, один объект, возможно три, пять, шесть или семь объектов

Характеристики этих множественностей разбиваются на два компонента, представляющие интерес при R-преобразовании: столбцы для создания в определенной таблице и как создать ограничение NULL/NOT NULL в столбце/столбцах.

Если максимальная множественность равна 1 (0..1 или 1 из вышеуказанного списка возможных множественностей), тогда эта роль соответствует столбцу/столбцам внешних ключей. Во-первых, определите идентификацию объекта в классе, присоединенном к другой роли. Если класс подразумевает неявную идентификацию объекта, то обычно имеет единственный столбец первичного ключа, целочисленную сгенерированную последовательность некоторого рода. Существует несколько вариаций, таких как таблица с первичным ключом, зависящим от первичного ключа другого класса. Это приводит к наличию двух столбцов (исходного первичного ключа и дополнительного, идентифицирующего специфическую строку в зависимой таблице).

Например, в подсистеме Человек на рис. 11.1 класс Человек имеет двоичную ассоциацию с классом Идентификация. Эта двоичная ассоциация — составная агрегация, так как человек владеет идентификациями, которые

его определяют. Класс Человек имеет неявную идентификацию объекта и столбец ИдентификаторЧеловека в таблице Человек, являющийся уникальным числом для каждого человека. Класс Идентификация имеет неявную идентификацию, но является агрегацией. Он владеет как столбцом ИдентификаторЧеловека, который идентифицирует все эти Идентификации, применяемые к отдельному человеку, так и столбцом ИдентификаторИдентификации, который идентифицирует уникальную Идентификацию внутри множества Идентификаций человека. Столбец ИдентификаторЧеловека в Идентификации — внешний ключ для доступа к таблице Человек.

```
CREATE TABLE Identification (  
    PersonID Integer REFERENCES Person,  
    IdentificationID INTEGER,  
    CONSTRAINT Identification_PK PRIMARY KEY (PersonID, IdentificationID))
```

Если бы объект Идентификация не зависел от Человека, то столбец ИдентификаторЧеловека был бы ни к чему: достаточно единственного уникального идентификатора Идентификатора Идентификации. Если бы любая идентификация принадлежала в один момент времени одному человеку, была бы множественность 0..1 роли Человек в ассоциации, поэтому Идентификация получила бы столбец ИдентификаторЧеловека, не являющийся частью первичного ключа.

```
CREATE TABLE Identification (  
    PersonID Integer REFERENCES Person,  
    IdentificationID INTEGER PRIMARY KEY )
```

Если множественность содержит 0 или подразумевает (0..*, *, 0..1), то можно иметь пустые значения в качестве значений внешних ключей.

Роль — имя, присваиваемое одной стороне ассоциации. Это та роль, которую играет ассоциированный класс в ассоциации. В ООП она становится именем переменной, хранящей элемент или коллекцию элементов другого класса. В таблицах роли, сопровождаемые множественностью с максимальным значением 1, могут стать именем внешнего ключа. Здесь следует выбрать: использовать либо имя роли, либо имя атрибутов идентификации в другом классе. Некоторые примеры могут продемонстрировать эти возможности более подробно.

Классический пример использования имени роли — рекурсивная ассоциация. Рис. 11. 2 иллюстрирует класс Организация с его представлением иерархии организации — рекурсивной составной агрегированной ассоциации, связывающей организацию с ее предком и порождающую организацию с ее потомками. При преобразовании этой ассоциации в схему реляционной БД вы столкнетесь с основной проблемой — присвоением имен. Очевидный путь представления этой ассоциации создает следующую таблицу Организации:

```
CREATE TABLE Organization (  
    OrganizationID INTEGER PRIMARY KEY,
```

```
OrganizationName VARCHAR2(100) NOT NULL,  
OrganizationID INTEGER REFERENCES Organization)
```

Сложность спецификации этой таблицы — дублирование имени столбцов для внешнего ключа, указывающего на порождающую организацию. Самая большая проблема — пространство имен: в одной и той же таблице нельзя иметь дубликаты имен столбцов. Следующая серьезнейшая проблема состоит в том, что "ИдентификаторОрганизации" в действительности не выражает назначение столбца для пользователя. Переименование этого столбца с помощью имени роли — хорошее решение:

```
CREATE TABLE Organization (  
  OrganizationID INTEGER PRIMARY KEY,  
  OrganizationName VARCHAR2(100) NOT NULL,  
  Parent INTEGER REFERENCES Organization)
```

Проблемы с пространством имен не существует для нерекursивных ассоциаций, но имя первичного ключа может оказаться совсем непонятным во внешней таблице. В этом случае разработчик должен принять решение: использовать ролевое имя или имя столбца первичного ключа в качестве имени столбца внешнего ключа. Один из случаев реализации такой возможности — наличие большой иерархии классов, каждый из которых использует единственное имя первичного ключа. Например, архив мультимедийных документов в системе каталога стандартизует иерархию класса абстрактных документов со столбцом ИдентификаторДокумента в качестве первичного ключа. Видеоклипы являются документами с ИдентификаторомДокумента для звуковых клипов, цифровых фотографий и образов, а также изображений отсканированных документов. Класс ВодительскоеУдостоверение может ссылаться на документ цифровой фотографии как внешний ключ. Использование "ИдентификатораДокумента" в качестве имени столбца не способствует тому, что пользователь класса ВодительскогоУдостоверения понимает происходящее. Вместо этого используйте ролевое имя "Фотография" в таблице ВодительскогоУдостоверения:

```
CREATE TABLE DriverLicense (  
  PersonID INTEGER NOT NULL REFERENCES Person,  
  IdentificationID INTEGER NOT NULL,  
  LicenseNumber INTEGER NOT NULL,  
  Photograph INTEGER REFERENCES DigitalPhotograph,  
  CONSTRAINT DriverLicense_PK PRIMARY KEY (PersonID, IdentificationID))
```

Несомненно, есть множество других ситуаций, когда использование ролевого имени более понятно, чем использование имени столбца первичного ключа, но генерализация — лучший из известных примеров.

Генерализации

В главе 7 представлены некоторые подробности отношений генерализации и их влияние на атрибуты и операции. В данном разделе разберемся, как преобразовывать генерализации в R-таблицы. Специфические ситуации, которые нужно рассмотреть, включают простое наследование, наследование из абстрактных классов и множественное наследование.

Простое наследование

Простое наследование имеет место, когда класс связан с единственным предком через генерализацию. Более специфический класс наследует атрибуты и поведение из более общего класса.

Для преобразования этой ситуации в R-таблицу можно использовать один из двух подходов: прямое отображение классов или отображение их с помощью распространения атрибутов и поведения на подклассы.

Прямое отображение Чтобы осуществить прямое отображение классов, создается одна таблица для каждого класса в иерархии наследования, а затем — отношение внешнего ключа для каждого отношения генерализации. Рис. 11.11 повторяет рис. 7.6 — иерархию наследования Идентификации.

Для преобразования иерархии, представленной на рис. 11.11, в таблицы создается одна таблица для каждого из классов на диаграмме, включая абстрактные классы. Следуя основным положениям из предыдущего раздела "Классы", создается первичный ключ из явных или неявных идентификаторов объектов. Идентификация имеет неявную идентификацию объекта, поэтому создается столбец первичного ключа для таблицы Идентификация, обычно целочисленный, получающий значение от автоматического генератора последовательных чисел. Назовем его ИдентификаторИдентификации: он идентифицирует объект Идентификация. Из-за того что класс Идентификация имеет также ассоциацию составной агрегации для класса Человек, первичный ключ содержит также первичный ключ таблицы Человек — ИдентификаторЧеловека.

```
CREATE TABLE Identification (
    PersonID INTEGER NOT NULL REFERENCES Person,
    IdentificationID INTEGER NOT NULL,
    Constraint Identification_PK PRIMARY KEY (PersonID, IdentificationID))
```

Для каждого подкласса корневого предка создается класс, а затем — один и тот же столбец для каждого подкласса в качестве первичного ключа суперкласса. Если по какой-то причине сформировано несколько столбцов первичных ключей в суперклассе, то те же самые столбцы создаются и в подклассе. Например, с помощью ИдентификатораПринужденияПоЗакону — те же самые столбцы первичных ключей ИдентификаторЧеловека и ИдентификаторИдентификации.

```
CREATE TABLE LawEnforcementID (
    PersonID INTEGER NOT NULL REFERENCES Identification,
```

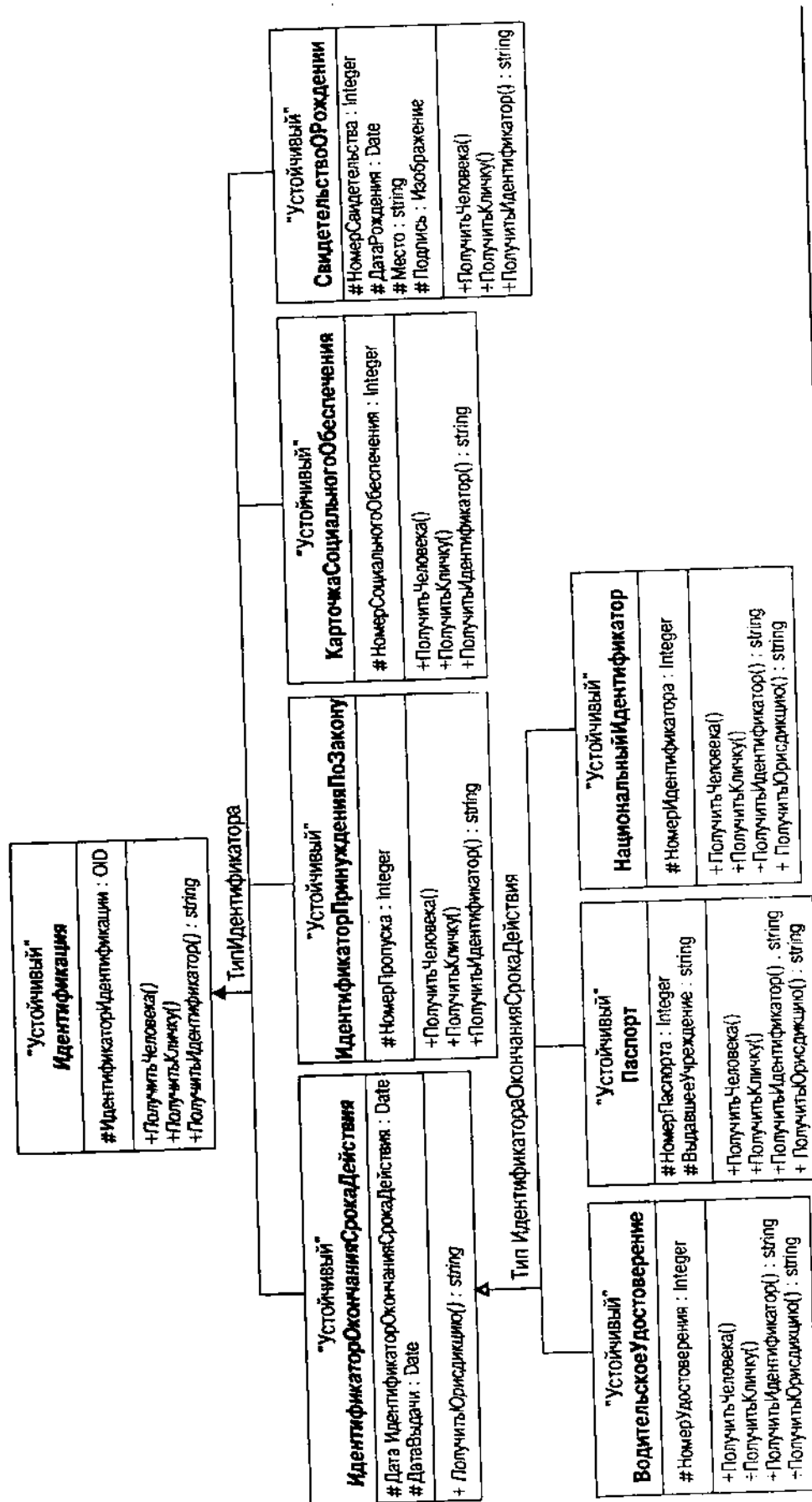


Рис. 11.11. Иерархия наследования Идентификация


```
IdentificationID INTEGER NOT NULL REFERENCES Identification,  
BadgeNumber INTEGER NOT NULL UNIQUE, - candidate key  
Constraint LawEnforcementID_PK  
PRIMARY KEY (PersonID, IdentificationID))
```

В дополнение к превращению столбца в первичный ключ, используя ограничение столбца или таблицы PRIMARY KEY, нужно также сделать его внешним ключом с ограничением столбца REFERENCES или ограничением столбца FOREIGN KEY. В приведенном примере ИдентификаторЧеловека ссылается не на таблицу Человек, а на таблицу Идентификация. Вернитесь к тому, о чем говорилось ранее в этом разделе "Внешние отношения", для знакомства с некоторыми точками зрения на данные ограничения. Ограничение внешнего ключа частично представляет отношение генерализации, но не обязательно полностью. Если строка включается в ИдентификаторПринужденияПоЗакону, тогда ИдентификаторЧеловека и ИдентификаторИдентификации этой строки должны соответствовать Идентификации строки. Ограничение внешнего ключа вводит это ограничение. Однако, если включается строка Идентификации, не существует ничего, связывающего ИдентификаторИдентификации указанием строки с таблицей подкласса. Даже концептуально это не имеет смысла, потому что SQL не способен выразить, какой тип таблицы планируется создать, поскольку он не понимает генерализации. Другое ограничение внешнего ключа на Идентификацию положить нельзя, так как это означало бы циклическую зависимость.

ВНИМАНИЕ

Рассмотренная ситуация возникает, когда порождающий класс абстрактный. Если класс конкретный, то объект этого класса создавайте, не заботясь о подклассах. Этот анализ предполагает, что абстрактный класс нуждается в триггере, который обеспечивает его абстрактность. Триггер должен гарантировать, что если строка включается в таблицу, соответствующую абстрактному классу, то также включается строка в один из нескольких подклассов абстрактного класса. При таком подходе вернитесь назад и оцените сложность системы в целом перед тем, как добавить триггер. Очевидно, что это создает меньше проблем при поддержании целостности на прикладном уровне, особенно для маленьких БД. Однако, когда количество приложений возрастет до приличного размера, захочется ввести на сервере как можно больше целостности, поскольку следующее приложение может все запутать.

Распространение Основная идея генерализации — наследование, когда каждый подкласс наследует атрибуты и операции суперклассов. Можно представить классы непосредственно, как в предыдущем разделе "Прямое отображение", затем использовать соединения и союзы для представления наследования. В качестве альтернативы можно сразу создать наследование с помощью атрибутов и операций в подклассах. Такую процедуру автор называет *распространением*.

Главное преимущество распространения — больше не нужно ссылаться на суперклассы для получения значений данных или существования. Это ликвидирует многие соединения и союзы, а также триггеры и, таким

образом, упрощает схему и код приложения. Почему бы тогда это не сделать? Обратная сторона этого преимущества — денормализация проекта схемы БД. Получающаяся значительная и ненужная избыточность данных приводит к аномалиям включения, обновления и удаления (см. ниже, раздел "Нормализующие отношения").

Рассмотрим класс Паспорт на рис. 11.11. Используя прямой подход, получаем такую конструкцию схемы:

```
CREATE TABLE Passport (
    PersonID INTEGER NOT NULL REFERENCES Identification
        ON DELETE CASCADE,
    IdentificationID INTEGER NOT NULL REFERENCES Identification
        ON DELETE CASCADE,
    PassportNumber INTEGER NOT NULL UNIQUE, - candidate key
    IssuingOffice VARCHAR2(100) NOT NULL,
    Constraint Passport_PK
        PRIMARY KEY (PersonID, IdentificationID))
```

а используя подход распространения — такую:

```
CREATE TABLE Passport (
    PersonID INTEGER NOT NULL REFERENCES Identification,
    IdentificationID INTEGER NOT NULL REFERENCES Identification,
    ExpireDate DATE NOT NULL CHECK (ExpireDate > IssueDate),
    IssueDate DATE NOT NULL,
    PassportNumber INTEGER NOT NULL UNIQUE, - candidate key
    IssuingOffice VARCHAR2(100) NOT NULL,
    Constraint Passport_PK
        PRIMARY KEY (PersonID, IdentificationID))
```

Вместо ссылки на таблицу ИдентификаторИстеченияСрокаДействия для ДатыИстеченияСрокаДействия и Даты Выдачи это решение копирует столбцы вниз по иерархии в таблицу Паспорт (и также во все другие подклассы ИдентификатораИстеченияСрокаДействия). Паспорт стоит отдельно как полезная таблица со всеми своими данными для пользователей. Вариант применения, которому требуются паспортные данные, должен осуществить доступ только к одной таблице. Однако, когда бы ни обновлялись ДатаИстеченияСрокаДействия и ДатаВыдачи Паспорта, нужно также обновлять те же столбцы в строке суперкласса, которая соответствует Паспорту. Это означает введение в приложение дополнительного триггера или кода.

Как и в принятии любого проектного решения, нужно рассмотреть, насколько оно повлияет на качество системы в целом. Хорошо ли сбалансирована возросшая из-за триггера сложность с уменьшением сложности прикладного кода, получающего доступ к паспортным данным? Например, если иерархия довольно глубока, можно принять решение получать доступ к нескольким классам с помощью соединений, а не одной таблицы,

предоставляющей все данные. Однако триггерная обработка вносит незначительный вклад в поддержку системы потому, что, сделав это один раз, можно больше к этому не возвращаться (в данном случае, при одном уровне в иерархии, — скорее всего, не стоит).

Также нетрудно совместить два подхода в одной системе. Можно производить распространение определенных глубоких наследований, оставляя другие, более мелкие наследования для использования прямого подхода. Всякий раз, когда вы смешиваете различные подходы к разработке, вы рискуете запутать тех, кто будет работать позже. Поступая так и не иначе, нужно создать подробную документацию о том, что делается, предпочтительно в самой БД, используя табличные комментарии или некоторый тип репозиторийной документации.

Множественное наследование

Множественное наследование возникает, когда класс имеет отношения генерализации с более чем одним суперклассом. Подкласс наследует все атрибуты и операции своих суперклассов.

На самом деле нет хорошего способа представить множественное наследование в схеме реляционной БД. Обычно лучше реструктурировать разработку, чтобы избежать множественного наследования, либо с помощью использования типов интерфейсов, либо с помощью факторизации элементов в разные, но концептуально избыточные классы. "Элегантность" множественного наследования заставляет платить слишком высокую цену в семантике R-таблицы и ее внешних ключей [Date and Darwen, 1998, p. 299–315].

В качестве примера плохого преобразования рассмотрим подход к созданию таблицы подкласса, включающей первичные ключи двух суперклассов. Что будет, если, например, разработчик решит, что Цифровая Фотография будет как Документом, так и Изображением (см. рис. 11.12)?

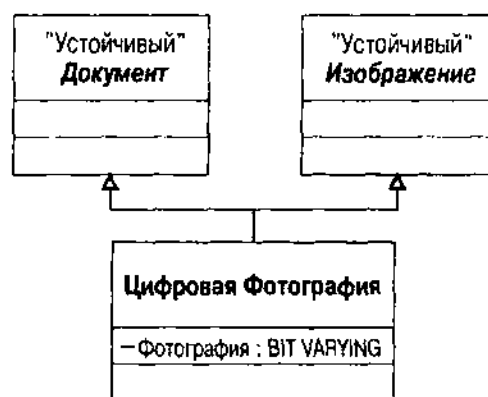


Рис. 11.12. Множественная генерализация

Можно создать таблицу Цифровая Фотография следующим образом:

```
CREATE TABLE DigitalPhotograph (
  DocumentID INTEGER NOT NULL REFERENCES Document,
```

```
ImageID INTEGER NOT NULL REFERENCES Image,
Photograph BIT VARYING (65536))
```

Теперь выберем первичный ключ. Будет ли это ИдентификаторДокумента или ИдентификаторИзображения? Оба являются уникальными идентификаторами, но в качестве внешнего ключа в других таблицах может быть только один первичный ключ, как в примере с ВодительскимУдостоверением из раздела "Роли, множественность и столбцы внешних ключей". Обычно разработчику требуется, чтобы первичный ключ представлял отношение генерализации, но так как допустимо иметь только что-то одно, этого сделать нельзя.

Когда встречаются стандартные головоломки представления многократного наследования (такие, как ромбовидное наследование, при котором два суперкласса ссылаются по очереди на общий суперкласс), разработчик попадает в еще большую беду (рис. 11.13). Потенциально можно иметь два отдельных объекта в общем суперклассе с помощью разных первичных ключей. Какой первичный ключ использовать в строке подкласса? Если применяется подход распространения, создаются ли два атрибута для каждого ключа в общем суперклассе? Положение все усложняется.

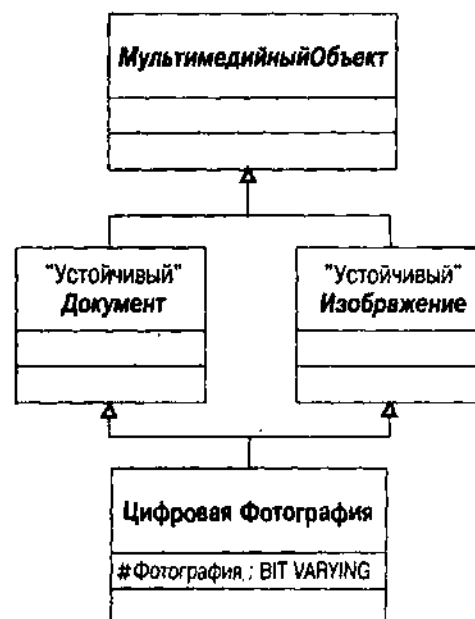


Рис. 11.13. Множественная генерализация с общим суперклассом

В C++ и других языках программирования разработчики языков создали разнообразные решения этой проблемы, например классы с виртуальной базой в C++, чего нет в R-схеме и ее стандартном языке SQL-92.

Цифровую Фотографию лучше реализовать через интерфейсы:

```
CREATE TABLE DigitalPhotograph (
  DigitalPhotographID INTEGER PRIMARY KEY,
  DocumentID INTEGER NOT NULL REFERENCES Document,
```

```
ImageID INTEGER NOT NULL REFERENCES Image,  
Photograph BIT VARYING (65536))
```

Объект (строка) стоит отдельно с первичным ключом, уникальным для таблицы Цифровая Фотография. Имеются ссылки на документ и изображение, — и таблица концептуально наследует поведение интерфейса и ведет себя как реализованный документ и изображение. Семантически это малопонятно в контексте R-схемы.

В качестве альтернативы можно переделать отношение между классами в ассоциацию вместо генерализации или реализации. С точки зрения ОО-разработки, основное отличие — отказ от возможности считать саму фотографию документом или изображением. Вместо этого "получены" документ или изображение, связанные с фотографией. Классы Документ и Изображение таким образом инкапсулируют фотографию (где возможно), а класс Цифровая Фотография, в свою очередь, — документ и изображение. Поскольку все эти классы непосредственно связаны, нужно создать третичную ассоциацию, а не двоичную (см. раздел "Ассоциации с количеством элементов более двух", содержащий подробности по преобразованию третичных ассоциаций). Результаты в данном случае таковы:

```
CREATE TABLE DigitalPhotograph (  
    DigitalPhotographID INTEGER PRIMARY KEY,  
    Photograph BIT VARYING (65536))  
  
CREATE TABLE PhotoUsedAs (  
    DigitalPhotographID INTEGER,  
    DocumentID INTEGER NOT NULL REFERENCES Document,  
    ImageID INTEGER NOT NULL REFERENCES Image,  
    CONSTRAINT PhotoUsedAs_PK (DigitalPhotographID, DocumentID,  
        ImageID))
```

Очевидная разница в том, что теперь вместо одной мы имеем две таблицы, где таблица PhotoUsedAs соответствует третичной ассоциации. В качестве первичного ключа таблица имеет три первичных ключа участвующих таблиц. Это ограничение подразумевает такое ограничение, когда может быть только одна комбинация любой отдельной фотографии, документа и изображения в БД. Может также потребоваться ограничение множественности, если может быть только один документ и изображение, связанные с фотографией. Это означает ограничение UNIQUE на Идентификатор Цифровой Фотографии. Поскольку это подразумевает наличие функциональной зависимости, не являющейся первичным ключом, разработчик может захотеть произвести обратную нормализацию к единой таблице. Сделайте Идентификатор Цифровой Фотографии первичным ключом, чтобы получить ее в четвертой нормальной форме.

ВНИМАНИЕ

Кажется достаточно редким случай, когда по стандартным ОО-методам разработки создается схема не в четвертой нормальной форме. Такие ситуации легко обнаружить, и справляться с ними надо при помощи стандартных методов нормализации, которыми не стоит пренебрегать (см. раздел "Нормализация отношений"). Одновременно это еще одно доказательство того, что следует избегать множественного наследования.

Особые ситуации

Ассоциации и генерализации породили ряд специальных ситуаций, которые нужно рассмотреть при преобразовании модели данных в R-схему: третичные ассоциации, ассоциации классов, составные агрегации, уточненные и упорядоченные ассоциации. В предыдущих разделах упоминались некоторые из этих ситуаций в их обычном контексте, сейчас они — главная тема обсуждения.

Ассоциации с количеством элементов более двух

Двоичная ассоциация — наиболее общий вид ассоциации. Количество элементов ассоциации (сколько классов участвует в ассоциации) может быть 3 или больше, и в таком случае создается отдельный класс ассоциации.

На рис. 11.2 ассоциация Играет — ромб, соединяющий Роль, Человека и Организацию. Человек играет Роль в Организации. Это преобразуется непосредственно в таблицу Играет.

```
CREATE TABLE Plays (
    PersonID INTEGER REFERENCES Person,
    OrganizationID INTEGER REFERENCES Organization,
    Role INTEGER REFERENCES Role,
    CONSTRAINT Plays_PK PRIMARY KEY (PersonID, OrganizationID, Role ID))
```

Таблицу выстраивают исключительно столбцы первичных ключей из ассоциированных таблиц. Если первичный ключ имеет несколько столбцов, все эти столбцы мигрируют в таблицу ассоциаций.

Классы ассоциаций и атрибуты

Любая ассоциация может иметь класс ассоциации и атрибуты, свойства самой ассоциации. Любой такой класс ассоциации получает свою собственную таблицу.

Снова обратившись к рис. 11.2, видим класс Деятельности, связанный с ромбом ассоциации Играет. Этот класс ассоциации содержит атрибуты ассоциации: время ее существования, даты начала и окончания, и способ, каким организация завершает отношения.

ВНИМАНИЕ

Когда попадают в подкласс КриминальнаяОрганизация класса Организация, эти атрибуты становятся более интересными, особенно метод завершения.

Таким образом, таблица расширяется атрибутами ассоциации и ее ограничениями:

```
CREATE TABLE Plays (  
    PersonID INTEGER REFERENCES Person,  
    OrganizationID INTEGER REFERENCES Organization,  
    Role INTEGER REFERENCES Role,  
    Tenure INTERVAL DAY NOT NULL, - your DBMS probably won't have this type  
    StartDate DATE NOT NULL,  
    EndDate DATE,  
    TerminationMethod INTEGER REFERENCES TerminationMethod  
    CHECK (TerminationMethod IS NOT NULL OR  
           (TerminationMethod IS NULL AND EndDate IS NULL))  
    CONSTRAINT Plays_PK PRIMARY KEY (PersonID, OrganizationID, Role ID))
```

Общие и составные агрегации

Агрегация — это ассоциация между двумя классами, представляющая отношение часть-целое между классами (см. в разделе "Агрегация, композиция и принадлежность" главы 7). Общая агрегация нечувствительна к структуре R-схемы, но составная агрегация имеет интересный побочный эффект.

Композиция говорит, что объект, ее образующий, владеет другим объектом и никакой другой объект не может быть связан с ним. Эта сильная форма агрегации непосредственно соответствует внешнему ключу с операциями обновления и уничтожения. На рис. 11.2 демонстрируется ассоциация композиции между Человеком и Идентификацией. Как показано в предыдущем разделе, класс Идентификация получает первичный ключ ИдентификаторЧеловека класса Человек как часть своего первичного ключа. Это придает мультиатрибутному первичному ключу — композитному — значение уникального имени. Отношение композиции применимо также ко всем подклассам в иерархии классов, хотя композиция неявна. Например, класс Паспорт наследует как первичный ключ таблицы Человек, так и столбец дополнительного ключа ИдентификаторИдентификации. Но внешний ключ ИдентификаторЧеловека ссылается на таблицу ИдентификаторОграниченияСрокаДействия, а не на таблицу Человек. Однако он все еще представляет собой композицию.

Специальный синтаксис в разделе <ссылочное триггерное действие> в ограничении целостности ссылок позволяет управлять некоторыми действиями при обновлении или уничтожении строки с первичным ключом в таблице принадлежности [ANSI, 1992; Melton and Simon, 1993, p. 221–227]. Вместе с обновлением или уничтожением первичных значений изменяются внешние ключи в зависимых таблицах. Предложение ON сообщает DBMS, что конкретно нужно сделать. На рис. 11.14 показан синтаксис этого предложения.

Операция CASCADE обновляет значение в зависимой таблице при обновлении значения первичного ключа в таблице принадлежности. Или она уничтожает строку в зависимой таблице при уничтожении строки с

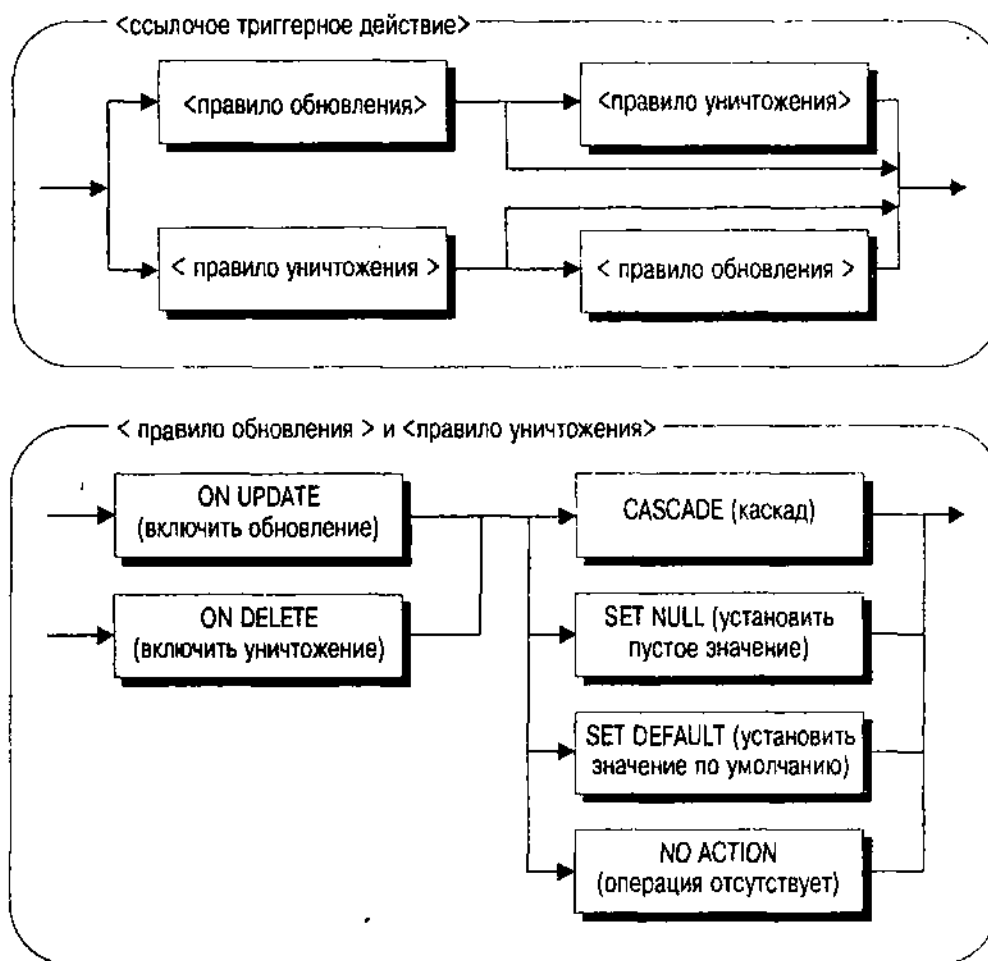


Рис. 11.14. Синтаксис SQL <ссылочное триггерное действие>

первичным ключом, на которую ссылается. Поскольку внешний ключ является также частью первичного ключа зависимой таблицы, CASCADE может последовательно включать другие таблицы, имеющие внешние ключи для этого первичного ключа.

Таблица Паспорт служит примером этой операции:

```

CREATE TABLE Passport (
  PersonID INTEGER NOT NULL REFERENCES Identification
    ON DELETE CASCADE ON UPDATE CASCADE,
  IdentificationID INTEGER NOT NULL REFERENCES Identification
    ON DELETE CASCADE ON UPDATE CASCADE,
  PassportNumber INTEGER NOT NULL UNIQUE, - candidate key
  IssuingOffice VARCHAR2(100) NOT NULL,
  Constraint Passport_PK
    PRIMARY KEY (PersonID, IdentificationID))
  
```

Если уничтожается или обновляется ИдентификаторЧеловека, DBMS также уничтожит или обновит соответствующий ИдентификаторуЧеловека Паспорт.

SET NULL (установить пустое значение) устанавливает пустые значения внешних ключей вместо обновления или уничтожения их, когда обновляют или уничтожают строку первичного ключа. Операция SET DEFAULT (установить значение по умолчанию) устанавливает значения внешних ключей равными значению раздела DEFAULT столбца.

NO ACTION (действие отсутствует) — действие по умолчанию. Этот вариант "отсутствия действия" говорит, чтобы DBMS ничего не сделала с внешним ключом. Если этим не управляют другие операторы SQL в транзакции, будет получена ошибка, и все обновление или уничтожение возвращается в исходное состояние.

Вместо использования этого синтаксиса можно также создать код поддержания целостности с помощью триггеров, связанных с событиями UPDATE и DELETE. См. раздел выше "Сигналы" с подробностями о разработке и использовании триггеров.

Теперь забудьте о том, что мы изложили. Основное предположение, стоящее за всеми этими операциями: изменяется значение первичного ключа. В большинстве случаев за некоторым исключением это нехорошо. Идентификация объектов обычно должна быть неизменной: с изменением значений ключей связано больше последствий, чем просто последовательные обновления или уничтожения. Много околичных жизней оборвались в борьбе за поддержание БД, позволявших изменять первичные ключи. Это неудачная стратегия.

Если при разработке используется неявная идентификация объектов, вообще беспричинно изменять значения первичных ключей. Например, ИдентификаторЧеловека не должен изменяться в приложениях. И нет никакой причины разрешать это делать. Допустимо изменить имя, но не идентификатор. Некоторые разработчики даже не показывают этот идентификатор через свой пользовательский интерфейс: он остается инкапсулированным во внутренних объектах. Генерируемые любым способом целочисленные ключи могут никогда не меняться — до тех пор, пока не закончатся все целые числа. Разработчик действительно должен позаботиться о долговременном количестве объектов, которые он собирается создать, если нет возможности изменить ключи.

Но к двум ситуациям данное ограничение не относится. Прежде всего и главным образом, когда уничтожают строку в таблице, владеющей строками других таблиц, нужно производить уничтожение последовательно. Такая принадлежность соответствует составной агрегации в проекте UML. Выходит из употребления прием, когда упорядочивают ассоциацию в таблице-потомке, как следствие появляется первичный ключ из двух частей для той таблицы, которая включает целое порядковое число. Если разрешено изменять порядок элементов в ассоциации, нужно изменять порядковое число в первичном ключе. Это вырождается, когда другие таблицы зависят в свою очередь от этого первичного ключа.

Во-вторых, если используется явная идентификация, разработчику придется разрешить изменять значения первичных ключей, поскольку они соответствуют данным, которые могут измениться в реальном мире. В идеале выбирают атрибуты элементов реального мира, неменяющиеся, но не

всегда возможна такая роскошь. Хорошей генеральной стратегией является переход к неявной идентификации и целочисленным сгенерированным ключам, когда имеется потенциальный ключ, который может измениться.

Уточненные ассоциации и дискриминаторы

Уточненная ассоциация — это что-то вроде краткой формы комбинации перечислимого типа данных и ассоциации. Она говорит, что уточненный объект имеет одну связь для значения каждого типа (см. в главе 7 "Уточненная ассоциация"). Так обычно преобразуют то, что было ассоциацией ко-многим, в ассоциацию к-одному, уточненную уточняющим атрибутом. На примере, представленном на рис. 11.15 (воспроизводящем рис. 7.12), показана ассоциация ИдентификацияЧеловек как уточненная ассоциация — значит, каждый человек имеет несколько различных видов документов Идентификаторов, структурированных по уточненному атрибуту, например свидетельство о рождении, водительское удостоверение и паспорт.

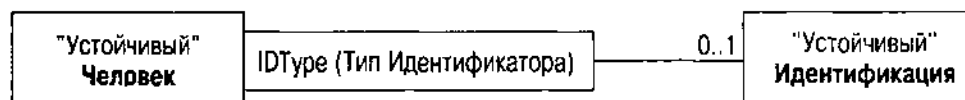


Рис. 11.15. Уточненная ассоциация

Механика создания R-схемы из уточненной ассоциации требует комбинации внешнего ключа и уточняющего столбца, который определяет тип объекта.

```

CREATE TABLE Identification (
  PersonID INTEGER NOT NULL REFERENCES Person,
  IdentificationID INTEGER NOT NULL,
  IDType INTEGER NOT NULL REFERENCES IDType,
  CONSTRAINT Identification_PK PRIMARY KEY (PersonID, IdentificationID))
  
```

Если есть специфические требования к уточняющему классу, то нужно добавить триггер или другое ограничение, которое их вводит. Например, если человек должен иметь свидетельство о рождении и паспорт, то нужен триггер, который проверяет, что имеются два объекта Идентификация с типом Идентификатора СвидетельстваОРождении и типом Идентификатора Паспорт.

СОВЕТУЕМ

Автору никогда не нравились уточненные ассоциации как средство моделирования. Они, кажется, еще больше все запутывают (это мнение, не суждение): могут упростить сложное отношение, но обычно этого не происходит. Автор их не использует.

Можно также преобразовать уточненную ассоциацию в параллельные колонки в уточняющей таблице. Например, можно добавить два столбца к Человеку, Свидетельству о Рождении и Паспорту, которые, в свою очередь, являются внешними ключами таблиц Свидетельства о Рождении и

Паспорта. Создатели моделей называют эту структуру *параллельными атрибутами*. Общий пример — создание таблицы с семью столбцами, по одному на каждый день недели, или с тремя столбцами адресов для поддержки вплоть до трех адресов на человека. Ограничения этой разработки вполне очевидны: если имеется потенциал создания неизвестного количества объектов, нужно добавить столбцы. Это в действительности проблема не в днях недели. Проблема в том, что нельзя просто использовать SQL для сбора информации из многих параллельных атрибутов. Скажем, при желании усреднить некоторое число для каждого дня недели очень трудно сделать это с помощью параллельных столбцов дней. Было бы просто, если бы данные находились в отдельной таблице с одним столбцом дней для идентификации дня [Blaha and Premerlani, 1998, p. 287 — 288].

ВНИМАНИЕ

Параллельные столбцы не "повторяющиеся группы" (в следующем разделе "Нормализующие отношения" рассматривается первая нормальная форма и повторяющиеся группы). С их помощью можно получить полную нормализацию, но они не являются хорошим способом разработки R-таблиц.

Дискриминатор — атрибут типа, ассоциируемый с отношением генерализации. Тип Идентификатора в примере Идентификации уточненной ассоциации может также служить дискриминатором, ассоциированным с генерализацией класса Идентификации из его подклассов. Тип не имеет потенциала для ограничений уточненной ассоциации, как и не предусматривает множественных связей.

Столбец этого типа нужен, когда через суперкласс перемещаются в подклассы. Например, во временной доменной модели человек имеет набор идентификаций. Запрос, используемый для создания этого набора, извлекает строки из таблицы Идентификация. Без атрибута дискриминатора трудно записать запросы, извлекающие информацию, специфическую для объекта, из таблиц подклассов. Столбец дискриминатора позволяет достать строку Идентификации со значением столбца типа Идентификатора Паспорта, затем запросить соответствующую ему строку паспорта. Без столбца дискриминатора недостаточно сведений для запроса таблицы Паспорт — необходимо проверять все таблицы подклассов по очереди. В терминах схем дискриминатор поддерживает Абстрактную Фабрику или схему Метода Фабрики для создания объектов подклассов в иерархии классов [Gamma et al., 1995].

Однако нужно уравновесить эту потребность с помощью дополнительного усложнения, которую поддерживающие дискриминаторы добавляют к коду. Например, если суперкласс конкретен (можно иметь сущности), тогда дискриминатор должен включать значение, идентифицирующее объект из этого класса без подкласса. Например, Водительское Удостоверение имеет подкласс Удостоверение На Вождение Грузовика (разновидность водительского удостоверения, позволяющая управлять грузовиком в дополнение к управлению другими средствами). Можно иметь водительское удостоверение и удостоверение на вождение грузовика. Дискриминатор в Водительском Удостоверении должен иметь два возможных значения —

"Водитель" и "Водитель Грузовика". Первое идентифицирует объект типа Водительское Удостоверение, а второе — объект подкласса Удостоверение На Вождение Грузовика. Использование этого дает среднюю степень сложности. Что произойдет, если перевести удостоверения из общего в удостоверение водителя грузовика? Что произойдет при добавлении другого подкласса? Нужно ли проверять или использовать дискриминатор при выборке данных водительского удостоверения? Нужно ли осуществлять выборку объектов подкласса, если интересуют только сведения водительских удостоверений? Может, проще иметь дело непосредственно с таблицами, чем использовать для навигации дискриминатор.

Упорядоченные ассоциации

Другой особый случай — *упорядоченная ассоциация* — ассоциация с множественной ролью ко-многим, которая имеет дескриптор {упорядоченный}. При ОО-разработке дескриптор {упорядоченный} означает, что для представления ассоциации нужна некоторая разновидность упорядоченной коллекции такая, как список или вектор. При преобразовании этого вида ассоциации в реляционную БД нужно преобразовать структурное представление (список) в представление данных (атрибут порядка).

Типичный атрибут порядка — это целое значение, изменяющееся от 1 до $\langle n \rangle$, где $\langle n \rangle$ — количество связей с ассоциированным объектом. Два основных способа применения упорядочения этого типа включают порядок вывода на экран элементов пользовательского интерфейса и порядок деталей в отношении мастер-деталь. Пользовательские интерфейсы часто выводят на экран списки объектов. Если надо, чтобы они появлялись в одном и том же порядке или чтобы пользователь управлял порядком элементов программы, нужно хранить порядок в таблице. Столбец порядка в таблице соответствует высвечиваемым на экране данным и осуществляет это. По аналогии, если имеются заказы на покупки и списки элементов для каждого заказа, обычно хочется, чтобы они были упорядочены определенным образом.

В последнем случае порядок несет на себе более одной функции в таблице деталей: он является также частью первичного ключа. Это происходит из-за того, что комбинация мастер-деталь — ассоциация составной агрегации (см. выше, раздел "Общие и составные агрегации"). Если добавляется дескриптор {упорядоченный} к составной ассоциации, то генерируется счетчик для второй части составного ключа. Например, если по какой-то причине упорядочили ассоциацию Идентификация Человека, допустим, для вывода на экран в пользовательском интерфейсе, тогда таблица реализуется по-другому:

```
CREATE TABLE Identification (  
    PersonID INTEGER NOT NULL REFERENCES Person,  
    OrderNumber INTEGER NOT NULL,  
    IDType INTEGER NOT NULL REFERENCES IDType,  
    CONSTRAINT Identification_PK PRIMARY KEY (PersonID, IdentificationID))
```

Вместо Идентификатора Идентификации как второй части первичного ключа теперь стоит столбец ПорядковыйНомер. Приложение или методы конструктора на таком сервере БД должны присваивать числу правильное числовое значение порядка — обычно индекс элемента в списке оперативной памяти. Это требует некоторого дополнительного кода для обновления, поскольку нужно обновлять весь набор объектов новыми номерами порядка при изменении порядка.

Жизнь по правилам

С правилами в реляционной БД можно встретиться везде, просто их трудно использовать. При создании столбца с определенным типом данных вводится доменное ограничение. Когда добавляется предложение PRIMARY KEY, вводится ограничение идеитификации объекта. Когда столбец начинает ссылаться на другой столбец в предложении REFERENCES, вводится ограничение по внешнему ключу. При добавлении триггера AFTER INSERT добавляют комплексное ограничение целостности или бизнес-правило.

В первой части этой главы рассматривались многие из основных ограничений, а именно:

- *Доменные ограничения:* типы, домены, ограничения CHECK и NOT NULL
- *Ограничения первичных ключей:* неявная и явная идентификация объектов
- *Ограничения уникальности:* потенциальные ключи и альтернативные OID
- *Ограничения внешних ключей:* ассоциации, множественность и агрегация

Имеются другие бизнес-правила, которые нужно ввести в БД, помимо структурных бизнес-правил. Эти ограничения появляются в модели данных UML в качестве прямоугольников ограничений, ассоциированных с соответствующими вещами, которые хотят ограничить. См. раздел "Комплексные ограничения" главы 7, содержащий подробности относительно выражения ограничений в UML OCL или в SQL.

Инварианты классов

Когда есть ограничение класса, оно обычно выражает инвариант класса над атрибутами класса. *Инвариант класса* — логическое выражение, которому всегда должен соответствовать объект класса (истинность выражения не изменяется вместе с состоянием объекта).

Преобразовав класс в таблицу, следует выразить инварианты классов в схеме. Есть два варианта — ограничения CHECK и триггеры.

CHECK позволяет присоединить выражение SQL, оценивающее на истинность, ложь или пустое значение столбца таблицы. Основной синтаксис очень прост — условие поиска, которое оценивает значение истинности. В языке SQL имеется три значения истинности: истина, ложь и неизвестное (пустое) [ANSI, 1992, p. 188–189]. Стандартная логика — логика с двумя значениями: истина и ложь. Большая часть ограничений, закладываемых в модели данных, отражает этот стандарт.

При использовании SQL для выражения ограничений вы переменщаетесь в мир квазимодальной логики — трехзначной, добавляющей новый фактор истинности, *неизвестное* значение истинности. Булево условие поиска должно принять во внимание ситуацию, при которой значения имеют пустые значения, что требует создания выражения для проверки пустого значения. Синтаксис стандарта ANSI позволяет проверить выражение на значение истинности: <условие поиска> IS [NOT] <значение истинности>. Никто из поставщиков RDBMS не реализует это выражение, следовательно условие поиска не должно оцениваться как ложное, чтобы соответствовать ограничению. Прочтите еще раз последнее предложение: что произойдет, если условие поиска оценивает пустое значение? Ответ: таблица соответствует ограничению CHECK. Следовательно, нужно обеспечить правильное управление пустыми значениями в выражении, обычно через их преобразование в функцию значения (функция NVL() в Oracle7, например, делает это) или проверку с использованием IS NOT NULL как части условия поиска. Таким образом, использование трехзначной логики значительно затрудняет проверку ограничений, и SQL отнюдь не корректен в способе использования значений null [Date, 1986, p. 313 – 334]. Разберитесь, какими ограничениями обладают используемые средства и будьте внимательны.

Существуют некоторые ограничения на SQL, который можно использовать в условии поиска. Нельзя применять агрегатную функцию, например AVG или SUM, если только она не находится в подзапросе, вложенном в выражение оператора SELECT. Таким образом, нельзя выразить ограничение, которое непосредственно сравнивает сумму или среднее со значением столбца. Нельзя ссылаться на специфические данные сеанса или динамически изменяющуюся информацию, такую как CURRENT_USER (ANSI) или SYSDATE (Oracle7), например. Это также означает, что нельзя вызвать определенную пользователем функцию в разделе CHECK. Можно изменить функцию, и ограничение CHECK обязательно будет оцениваться как true для данных, которые уже находятся в БД.

Полный синтаксис ANSI позволяет использовать в выражении подзапросы, но промежуточная форма стандарта это исключает, что отражает отраслевую норму. Никто из поставщиков RDBMS не реализует полный синтаксис выражений для ограничений CHECK. Oracle7, например, не разрешает подзапросы. Таким образом, нельзя ввести какое-либо ограничение, ссылающееся на данные вне таблицы или в строках, отличных от текущей строки, в которой проверяется это ограничение.

И наконец, имеются две формы ограничения CHECK в его использовании — как ограничение столбца или домена и как ограничение таблицы. Первое обычно проверяет один столбец, второе — больше одного, чтобы представить сложные инварианты классов над несколькими атрибутами. В любом случае RDBMS оценивает ограничение, когда операция SQL, например INSERT, завершается. Не существует действительного различия между этими двумя формами; можно ссылаться на несколько столбцов в ограничении столбцов или на один столбец в табличном ограничении. Некоторые системы, такие как SQL Server, предупреждают, когда происходит ссылка на столбец, отличный от столбца, к которому присоединено ограничение

столбца CHECK. Однако это просто предупреждение, а не фатальная ошибка при создании таблицы.

В связи с проблемами ограничения CHECK в теории на практике придется, вероятно, создать много инвариантных для класса ограничений в виде триггеров, а не ограничений CHECK. Триггер — это хранимая процедура, которую RDBMS выполняет, когда происходит определенное событие (см. раздел "Сигналы").

Системные инварианты

Имеется другой класс ограничений, выходящих за пределы одного класса. Системный инвариант — это логическое выражение, которое всегда должно быть истинным для системы в целом. В этом случае под "системой в целом" автор понимает коллекцию схем БД и данных, содержащихся в таблицах, определенных этими схемами. Если система определена с помощью многих схем, иногда встречаются ограничения, выходящие за границы одной схемы. Наиболее общая разновидность такого рода — ограничение внешнего ключа, которое делает одну систему зависимой от другой. См. выше раздел "Пакеты, подсистемы и пространства имен", где полностью представлена разработка нескольких подсистем для схем БД. Можно создать триггеры, которые обеспечивают эти ограничения.

ВНИМАНИЕ

Современные системы не позволяют использовать ограничение REFERENCES или FOREIGN KEY между схемами. Некоторые ограничения на распределенные БД с несколькими серверами часто ограничивают разработчика в том, что можно делать одновременно на серверах. Тщательно проверьте свою целевую систему с этой точки зрения, если имеются системные инварианты такого рода.

Вторая разновидность системного инварианта — ограничение, ссылающееся на две или более таблиц в одном логическом выражении. Нельзя выразить этот вид ограничения как ограничение CHECK потому, что это не инвариант класса в одной таблице.

Стандарт ANSI определил этот вид системного ограничения как *утверждение* и дал ему новый оператор DDL — CREATE ASSERTION. Он является по сути предложением CHECK для схемы, а не для таблицы. Хотя стандарт позволяет использовать подзапросы в таблице или ограничения столбцов CHECK, на самом деле желательно, чтобы RDBMS оценила системные инварианты на более высоком уровне. Изменения любых задействованных таблиц должны привести к проверке ограничения.

СОВЕТУЕМ

Ограничение таблицы или столбца на пустой таблице всегда удовлетворяется (подумайте об этом). Если необходимо удостовериться, что в таблице есть данные, нужно использовать утверждение. Оно удовлетворяется тогда, и только тогда, когда условие возвращает истину, независимо от наличия данных в любой из задействованных таблиц.

Основная проблема с утверждением состоит в том, что никто из поставщиков RDBMS не реализует его. В современных системах нужно создать

хранимые процедуры или даже код приложения, который прикладная система выполняет в соответствующем месте каждой транзакции. Это требует дополнительного кода и сложности в системе, но в настоящее время другого выбора нет.

Отношения нормализации

Нормализация – это процесс структурирования таблиц и столбцов в схеме во избежание определенных ограничений зависимости (функциональных зависимостей, многозначных зависимостей и объединенных зависимостей).

Можно написать целую главу или книгу по нормализации [Dutka and Hanson, 1989], хотя вряд ли она будет интересной. Странность нормализации в том, что на самом деле это состояние, а не процесс. Продвижение от первой ко второй и третье и т.д. нормальной форме – иллюзорное упорядочение более прямого подхода: найти и устранить структурные зависимости, вызывающие проблемы. Не получается никакого выигрыша при переходе через несколько нормальных форм. Вместо этого следует понять математику зависимостей, а затем сфокусироваться на практических методах разработки, которые полностью их устраняют.

Перед тем как рассмотреть эти зависимости, отметим, что ОО-разработка, как отражено в модели данных UML, почти автоматически удаляет большинство зависимостей. Вся идея идентификации объектов и создания связанных иерархий классов направлена на структуризацию данных в связанные таблицы, а не перепутанные отдельные таблицы со множеством зависимостей между столбцами. Начинать с ОО-разработки (или даже ER-разработки), значит скорее создать нормализованную базу данных, чем не создать ее. Этот факт настолько впечатляет большинство разработчиков, что они полагают, будто можно совершенно обойтись без теории нормализации [Blaha and Premerlani, 1998, p. 273 – 274]. Автор не стал бы заходить так далеко, однако справедливо будет сказать, что нормализация на самом деле не должна вызывать беспокойство, если начинать с ОО-разработки.

Элементарные значения

Далее хотелось бы затронуть достаточно распространенную ошибочную концепцию первой нормальной формы – элементарные значения. Первая нормальная форма говорит, что каждое значение должно быть элементарным. Рассмотрим в качестве примера таблицу:

```
CREATE TABLE Atomic (  
    NuclearProgramID INTEGER PRIMARY KEY,  
    Reactor1 VARCHAR2(50),  
    Reactor2 VARCHAR2(50),  
    Reactor3 VARCHAR2(50))
```

Три столбца Реактор представляют значения для трех реакторов в ядерной программе.

Вот загадка для практикующего администратора БД: какой нормальной форме соответствует эта таблица?

Большинство администраторов, очевидно, полагают, что эта таблица *не* в первой нормальной форме. Скажу прямо: *они ошибаются*. Это совершенно очевидное, полностью нормализованное отношение в пятой нормальной форме. Каждое значение элементарно, и в этой таблице отсутствуют зависимости любого рода. Спор начинается со значения слова "элементарное", и частью проблемы является путаница с концепцией "повторяющиеся группы". Эта старая концепция программирования позволяет повторять структуру внутри другой структуры. Она не имеет ничего общего ни с элементарностью, ни с первой нормальной формой.

Элементарность очень проста. Все, что она утверждает: каждый столбец может иметь только одно значение для каждой строки в таблице. В примере таблицы Atomic каждый столбец Реактор имеет единственное значение для каждой строки таблицы, следовательно, он элементарен и находится в первой нормальной форме. А как будет выглядеть таблица, не находящаяся в первой нормальной форме? Это не так-то легко представить, поскольку в стандартном SQL ее нельзя определить. Однако, заглянув на главу вперед, рассмотрим такое определение из синтаксиса Oracle8:

```
CREATE TYPE ReactorArray AS VARRAY(3) OF VARCHAR2(50);  
CREATE TABLE NotAtomic (  
    NuclearProgramID INTEGER PRIMARY KEY,  
    Reactors ReactorArray);
```

VARRAY (массив переменных) — многозначный тип данных, имеющий определенное количество объектов однозначного типа данных. Таблица NotAtomic не находится в первой нормальной форме потому, что столбец Reactors содержит более одного значения для каждой строки таблицы.

Учтите, что *невозможно* создать неэлементарные таблицы, используя стандартный реляционный SQL. Этот язык не имеет многозначных типов данных, подобных VARRAY. Следовательно, когда бы ни разрабатывались таблицы в стандартной R-схеме, таблицы будут находиться в первой нормальной форме. И точка.

ВНИМАНИЕ

Разработка элементарных таблиц не обходится без проблем, но это не проблемы нормализации. Из-за того, что решение подобно некоей декомпозиции, имеющейся в нормализации, многие администраторы БД сравнивают ее с процессом нормализации, но это совершенно другое. Основное ограничение в разработке — отсутствие гибкости в отношении количества реакторов, которое может иметь каждая программа. Другое основное ограничение — когда хотят осуществить поиск или рвсчитать агрегации с разными значениями: гораздо проще, если каждое значение находится в отдельной строке. Кроме того, весь этот анализ по поводу первой нормальной формы противоречив, если имеется более одного элемента повторяющихся данных (реактор плюс текущая дата плюс время работы, например, вместо просто имени реактора), тогда вводятся функциональные или многозначные зависимости — значит это не пятая нормальная форма, а вторая. Но это проблема, отличная от требования элементарности первой нормальной формы.

Зависимости и нормализация

Теперь о самом сокровенном в нормализации – зависимости. Лучше всего объяснять, что такое зависимости, на примере, поэтому мы переделаем некоторые примеры из этой книги в примеры зависимостей. Те, что были представлены прежде, не имеют нежелательных зависимостей.

Кодд разработал первоначальный подход к нормализации в некоторых ранних статьях по R-теории [Codd, 1970, 1972]. Основная идея этих статей – предоставить основные направления R-разработки при наличии определенных аномалий манипулирования данными, возникающими из-за плохой структуры.

ВНИМАНИЕ

Нормализация как подход начинается с законченного проекта БД. Нужно иметь все атрибуты, где-то определенные, чтобы иметь возможность производить нормализацию таблиц, где они появляются в анализе зависимостей. Хотя это и очень формальный, однородный и математический подход, он мало что дает разработчику при создании БД. UML и методы, использующие его, с другой стороны, ориентируют разработчика постоянно, предоставляя методы разработки, рекомендации, схемы и оценки (одна из которых – степень нормализации).

Когда определяется зависимость между имеющимися элементами данных, зависимые данные разбивают на отдельные таблицы, ликвидируя ее. Кодд назвал этот процесс *декомпозицией* и структурировал его как последовательность нормальных форм. По мере того как специалисты в области вычислительной техники и математики начали разработку этих простых идей, они переросли в еще более общую форму. В итоге наступил конец истории: доказали, что "объединенная" зависимость – наиболее общая форма зависимости и, следовательно, пятая нормальная форма является высшей формой нормализации [Fagin, 1979]. Что также хорошо, поскольку те, кто может найти довольно адекватное объяснение четвертой нормальной форме, испытывают настоящие затруднения с пятой нормальной формой. Приняв эти допущения, продолжим.

Функциональная зависимость имеет место, когда значение столбца определяет значение другого столбца в той же самой таблице. Мы уже видели наиболее общую функциональную зависимость – ключ. Первичный ключ определяет значения других столбцов таблицы, как и каждый потенциальный ключ. Поэтому каждая таблица, создаваемая с помощью идентификации объектов, явной или неявной, представляет собой функциональную зависимость.

Немного более тонкая, но очень распространенная функциональная зависимость наступает, когда отношение ко-многим представляют одной таблицей вместе со всеми относящимися к ней данными. Например, возьмем отношение между Человеком и Идентификацией на рис. 11.1. Нам уже известно, что стандартный способ представления этого отношения – создание двух таблиц, Человек и Идентификация (и всех подклассов так же, как отдельных таблиц). Вместо этого рассмотрим, как может выглядеть отдельная таблица Человек, если в нее включены данные таблиц Водительское Удостоверение и Паспорт.

```
CREATE TABLE Person (  
    PersonID INTEGER PRIMARY KEY,  
    Name VARCHAR2(100) NOT NULL,  
    Sex CHAR (1) NOT NULL CHECK (Sex IN ('M', 'F')),  
    DateOfBirth DATE NOT NULL,  
    DateOfDeath DATE,  
    Height NUMBER,  
    Weight NUMBER,  
    MaritalStatus CHAR(1), CHECK (MaritalStatus in ('S', 'M', 'D', 'W'))  
    Comment VARCHAR2(2000),  
    DriverLicenseNumber NUMBER,  
    LicenseIssueDate DATE,  
    LicenseExpirationDate DATE,  
    PassportNumber NUMBER,  
    PassportIssueDate DATE,  
    PassportExpirationDate DATE,  
    IssuingOffice VARCHAR2(100))
```

Интуитивно понимаем, что происходит: три объекта объединены в одну таблицу, человек, водительское удостоверение и паспорт. С математической точки зрения, введены пять функциональных зависимостей: ДатаВыдачи Удостоверения и ДатаИстеченияСрокаУдостоверения – оба зависимы от НомераВодительскогоУдостоверения. Три связанных с паспортом столбца зависят от НомераПаспорта, предполагая, что два эти номера уникальны.

Это интересный пример функциональной зависимости, поскольку структура предполагает, что имеется отношение один-к-одному между человеком и водительским удостоверением и между человеком и паспортом. Следовательно, ИдентификаторЧеловека – это все еще ключ таблицы Человек, и нет особых проблем с обновлением или удалением данных. Если человек получает новый паспорт, просто нужно, например, обновить эти столбцы. Но мы рассмотрим более общую модель, представленную на рис. 11.1, и последствия нашей попытки представить ее непосредственно. Поскольку человек может иметь изменяющееся количество нескольких ИдентификационныхДокументов, наилучший способ сочетания всей этой информации в одной таблице – отвести одну строку каждой комбинации человека и ИдентификационногоДокумента:

```
CREATE TABLE Person (  
    PersonID INTEGER NOT NULL,  
    Name VARCHAR2(100) NOT NULL,  
    Sex CHAR (1) NOT NULL CHECK (Sex IN ('M', 'F')),  
    DateOfBirth DATE NOT NULL,  
    DateOfDeath DATE,  
    Height NUMBER,  
    Weight NUMBER,  
    MaritalStatus CHAR(1), CHECK (MaritalStatus in ('S', 'M', 'D', 'W'))
```

```
Comment VARCHAR2(2000),
IDNumber NUMBER,
LicenseIssueDate DATE,
LicenseExpirationDate DATE,
PassportIssueDate DATE,
PassportExpirationDate DATE,
IssuingOffice VARCHAR2(100)
CONSTRAINT Person_PK (PersonID, IDNumber))
```

В этом месте надо остановиться потому, что мы ушли так далеко, что прошла охота путешествовать. Эта таблица слишком запутанна. Все виды данных, имеющие разную множественность, смешаны вместе. Это ужасный способ представления мира, который пытаются моделировать. Он приводит к тому, например, что появляются две записи Человек для каждого человека с Паспортом и Водительским Удостоверением. В строке, связанной с Паспортом, все столбцы Водительского Удостоверения пусты. В строке, связанной с Водительским Удостоверением, все столбцы Паспорта пусты. Более важно то, что дублируются все данные Человек в каждой из двух строк. Когда добавляются идентификации разных видов вместе с их специфическими столбцами, становится еще хуже. Теперь, когда обновляется Вес, например, нужно обновить его в двух разных строках. Перед нами Коддова аномалия обновления.

Эта проблема решается с помощью представления объектов напрямую в отдельных таблицах, что является третьей нормальной формой нормальной формы Бойса-Кодда (BCNF), зависящей от конкретной структуры. Теория нормализации берет одну огромную таблицу и разбивает ее на части, используя функциональные зависимости, представляющие группы фактов, связанных с идентификацией объекта, и отношениями между объектами. Теперь можно понять, почему OO- и ER-разработка, представляющая объекты и отношения непосредственно, имеет тенденцию к тому, что ей не требуется существенная нормализация. Функциональные зависимости представляются непосредственно, и модель структурируется с самого начала в декомпозированные отношения потому, что ассоциации представляются непосредственно.

Но положение ухудшается.

Многозначная зависимость имеет место, когда таблица является объединением двух ее проекций, имеющих разделяемый поднабор столбцов [Fagin, 1981, p. 390]. Функциональная зависимость — специальный случай многозначной зависимости. Повторим, давайте исследовать концепцию на примерах, а не будем сосредотачиваться на R-математике. Рассмотрим Криминальную Организацию на рис. 11.2 и то, что может произойти при моделировании отдельной криминальной организации, которой управляли люди в этой организации (их "бизнес"). Следующий код добавляет таблицу ТипБизнеса, содержащую все виды бизнеса, которыми может заниматься криминальная организация, и таблицу КриминальныйБизнес, перечисляющую людей и виды бизнеса, которыми они руководят.

```

CREATE TABLE CriminalOrganization (
  OrganizationID INTEGER PRIMARY KEY REFERENCES Organization,
  LegalStatus CHAR(1) NOT NULL,
  Stability CHAR(1) NOT NULL,
  InvestigativePriority CHAR(1) NOT NULL,
  ProsecutionStatus CHAR(1) NOT NULL);
CREATE TABLE BusinessType (
  BusinessTypeID INTEGER PRIMARY KEY,
  TypeName VARCHAR2(100));
CREATE TABLE CriminalBusiness (
  OrganizationID INTEGER REFERENCES CriminalOrganization, - the mob
  PersonID INTEGER REFERENCES Person, - the mob member
  BusinessType INTEGER REFERENCES BusinessType, - the criminal business
  PRIMARY KEY (OrganizationID, PersonID, BusinessID));

```

В этой модели нет ничего очевидно неправильного, пока не становится понятной базовая семантика данных. Отрасли бизнеса управляются *организациями*, а не индивидуалами внутри организации. Таблица Криминального Бизнеса представляет это, но она также включает и человека. В таблице 11.1 представлены данные для таблицы Криминального Бизнеса (неявные идентифицирующие объект целые числа заменены соответствующими именами для ясности: имена относятся к некоторым историям о Холмсе).

Таблица 11.1. Криминальный Бизнес

Название Организации	Имя Человека	Название Отрасли Бизнеса
Организация Мориарти	Фред Порлок	Вымогательство
Организация Мориарти	Фред Порлок	Грабеж
Организация Мориарти	Полковник Себастьян Моран	Вымогательство
Организация Мориарти	Полковник Себастьян Моран	Грабеж
Банда Клэя	Джон Клэй	Ограбления Банков
Банда Клэя	Джон Клэй	Грабеж
Банда Клэя	Арчи	Ограбления Банков
Банда Клэя	Арчи	Ограбления Банков

Многозначная зависимость здесь проистекает из того, что ограничение первичного ключа PRIMARY KEY для таблицы Криминального Бизнеса — это полный набор первичных ключей из трех связанных с ней таблиц Человек, Криминальная Организация и Тип Бизнеса. Однако, если известна организация, то известны и осуществляемые ею отрасли бизнеса, а также люди в этой организации. Но люди и отрасли бизнеса независимы друг от друга. Таким образом, имеются два столбца, зависящие от третьего, но независимые между собой — многозначная зависимость, не являющаяся первичным ключом таблицы. Таблица Типа Бизнеса — это объединение двух проекций — Организация-Человек и Организация-Тип Бизнеса.

Таблица 11.2 представляет таблицу Партнер, содержащую проекцию ОрганизацияЧеловек наряду с ролью человека в организации. В этом случае, хотя три столбца образуют первичный ключ, каждый человек играет только одну роль (сделано только для иллюстрации).

Таблица 11.2. Партнер

Название Организации	Имя Человека	Название Роли в Организации
Организация Мориарти	Фред Порлок	Руководитель Организации
Организация Мориарти	Полковник Себастьян Моран	Второй из наиболее опасных людей в Лондоне
Банда Клэя	Джон Клэй	Четвертый из самых умных людей Лондона
Банда Клэя	Арчи	Приятель

Таблица 11.3 — это исправленная таблица Криминального Бизнеса, включающая только организации и отрасли бизнеса, что лучше представляет базовое значение данных.

Таблица 11.3. Исправленная Таблица Криминальный Бизнес

Название Организации	Название Отрасли Бизнеса
Организация Мориарти	Вымогательство
Организация Мориарти	Грабеж
Банда Клэя	Грабеж
Банда Клэя	Ограбления Банков

Мы видим, что присоединение к новой таблице Криминального Бизнеса таблицы Партнер в результате приводит к первой таблице Криминального Бизнеса — таблице 11.1. Сохранение данных в форме таблицы 11.1 чревато опасностями. Вторым из самых опасных людей Лондона становится угрозой целостности БД потому, что нужно обновлять эти излишние отрасли бизнеса в четырех отдельных строках при изменении бизнеса организации. Это аномалия обновления для многозначной зависимости. Декомпозиция на проекции ликвидирует избыточности и аномалии.

Объединенная зависимость на основе многозначной зависимости создает ее расширение до общего случая объединения трех или более проекций, разделяющих поднабор столбцов. Если бы существовал другой набор столбцов в отдельной таблице, агрегирующей объекты в организацию, объединение создало бы избыточные последовательности строк. Декомпозиция объединенной зависимости называется пятой нормальной формой, и ее очень трудно определить. Обычно отыскивается набор таблиц проекций (декомпозиций), которые, объединенные вместе, ликвидируют все ненужные строки, созданные вследствие объединения любых двух таблиц. Это приводит к созданию трех таблиц декомпозиций. Обычно довольно трудно обнаружить указанные ограничения до тех пор, пока не столкнешься с проблемой конкретного объединения [Date, 1998b].

ВНИМАНИЕ

Чтобы разобраться в теме объединенных заеисимостей, познакомьтесь с двумя лучшими статьями Фагинв [Fagin, 1979, 1981]. У Дейта есть хорошая статья в двух частях об этом предмете [Date, 1998a, 1998b] — модифицированный материал его фундаментальной книги по теории БД [Date, 1995]. Классическая статья Уильяма Кента также толково излагает теорию объединенных зависимостей [Kent, 1983].

Основное правило, таким образом, состоит в декомпозиции таблиц в БД без потери информации об отношениях данных до тех пор, пока не получено отсутствие зависимостей по полному первичному ключу каждой таблицы. Пятая нормальная форма также требует достаточного количества таблиц для представления всех первичных ключей, чтобы получить правильные данные при повторном объединении этих таблиц (фактически части определения без потерь).

Если следовать основному правилу и уничтожать любую из этих зависимостей, не являющуюся зависимостью от ключа в таблице, вы всегда будете получать базу данных в пятой нормальной форме.

Денормализация созданной схемы

Повращавшись некоторое время в кругу администраторов реляционных БД, можно услышать горячие споры о "денормализации" данных. Некоторые свято верят в то, что данные необходимо хранить в пятой нормальной форме; другие, так же свято, — в то, что пятая нормальная форма — удел фанатиков и не служит полезной цели.

Обе эти позиции — крайности, которых надо избегать. Из примеров рассмотренного раздела "Зависимости и нормализация" становится ясно, что устранение многозначных и объединенных зависимостей — это хорошо. Если для устранения объединенных зависимостей декомпозиция осуществляется неправильно, объединения приводят к неверным результатам из-за потери информации. Аналогичным образом, сложность данных из-за многозначной зависимости и простота нормализованной формы (пятой нормальной формы) вполне очевидны.

Почему администраторы БД рассматривают денормализованные данные? Одним словом, из-за производительности. Самой дорогостоящей операцией в реляционной БД является объединение (если не считать, прежде всего, ее покупки, конечно). С помощью предварительного объединения проекций в одну таблицу администраторы БД устраняют необходимость объединения таблиц в прикладном коде SQL, что существенно повышает производительность. Но так ли это на самом деле? Лучший ответ, который может дать честный администратор: "Как сказать..." Из-за отделения физической схемы от логической любое изменение логической схемы для поддержки физических операций является по необходимости косвенным. Например, в зависимости от требований к памяти предварительно объединенная таблица может занимать более одной физической страницы. Это приводит к большему объему операций ввода/вывода на запрос, что может повлечь объединение двух отдельных таблиц, хранимых на отдельных дисках [Date, 1997b, p. 24].

Настройка производительности реляционной БД требует хорошего анализа данных, глубокого понимания целевой DBMS и ее физических операций, а также четкого представления потерь в денормализованных данных. Следует рассматривать денормализованные данные, только когда возникают проблемы производительности. Обязательно сначала взгляните на выражения SQL и альтернативные физические структуры, поскольку они не имеют вредных побочных эффектов денормализации. Это должно быть последним средством, особенно если имеется множество приложений, использующих интересные структуры данных.

Важно понимать, что "денормализация" больше не является процессом, как нормализация. Если нет третьей нормальной формы, то нет и пятой нормальной формы. Однако вы способны устранить все нетривиальные объединения и многозначные зависимости в БД и все же иметь несколько функциональных зависимостей в предварительно объединенных денормализованных таблицах. Целью должна быть ликвидация как можно большего количества неключевых зависимостей и переход как можно ближе к моделированию реальных объектов.

И наконец, многие администраторы БД полагают, что производят денормализацию, когда на самом деле они просто вводят избыточность [Date, 1997b].

Во-первых, вернитесь назад и прочтите раздел "Атомарные значения". Многие администраторы полагают, что введение "повторяющихся групп" в структуру таблиц таких, как столбцы Адрес1, Адрес2 и Адрес3, или понедельник, вторник, среда, четверг, пятница, суббота и воскресенье — это денормализация. Если только не вводится зависимость в полный первичный ключ, таблица остается полностью нормализованной. Обычно администратор спорит, что это "повторяющиеся группы" и, следовательно, таблица уже не будет находиться в первой нормальной форме. Как показано в предыдущем разделе, это чепуха. Первая нормальная форма касается многозначных столбцов, а не последовательности столбцов с единственными значениями. При вводе последовательностей нескольких столбцов, подразумевающих функциональную зависимость (от которой лучше бы избавиться с помощью сохранения проекции этих столбцов со своим собственным первичным ключом в отдельной таблице), можно получить зависимость. Например, вы хотите добавить столбец ЯнвДоход, ФевДоход, МарДоход и т.д. к таблице Партнер. Таблица все еще находится в пятой нормальной форме. Однако, если добавить два столбца, оба зависящих от месяца, будет введена функциональная зависимость — ЯнвДоход, ЯнвЗарплата, ФевДоход, ФевЗарплата и т.д.

Во-вторых, сохранение агрегатов не денормализует БД. Например, можно сохранить общее количество человек в организации в таблице Организация. Вводится ограничение, означающее, что это число является счетчиком числа различных людей, связанных с организацией через таблицу Партнер. Если ограничение сложно, оно скорее всего является не функциональной, многозначной или объединенной зависимостью. Следовательно, сохранение счетчика не переводит таблицу Организация в пятую нормальную форму.

В-третьих, часто встречается ситуация, когда таблицы содержат пустые значения для ситуаций, которые логически не могут существовать. Это результат комбинирования суперкласса и подкласса. Например, если объединены таблицы Организация и Криминальная Организация, столбец ОфициальныйСтатус будет иметь пустые значения для любой строки, соответствующей организации, не являющейся криминальной. Хотя случай, возможно, неидеальный, это также не денормализация, потому что отсутствуют функциональные зависимости или любой другой вид зависимости. Денормализация возникает из объединения проекций, а не соединения их: объединяют объекты двух исключających подклассов в таблицу суперкласса с помощью операции объединения, а не соединения. Нормализация же зависит от соединения (join) проекций, а не от их объединения (union).

В-четвертых, создание схемы звезды для хранилища данных (см. Предисловие, содержащее определение) не денормализует БД. В схеме звезды создается очень сложная таблица n-арных отношений (таблица фактов) со многими относящимися к ней таблицами (таблицами размерностей). Не вводятся зависимости или даже избыточности — просто представляют многомерные характеристики данных с использованием образца Схема Звезды (см. раздел "Моделирование с помощью повторно используемых схем разработки" главы 8).

Вероятно, можно привести много других случаев декомпозиции и композиции, не влияющих на нормализацию. Главное — рассматривать зависимости, чтобы не быть одураченным процессом композиции. Повторим еще раз, что нормализация на самом деле — не процесс, а состояние.

Язык мира

SQL-92 предоставляет понятный язык определения схем. Раздел "Правила согласования" указывает, какие шаги надо осуществить для преобразования ОО модели в этот язык, используя методы, описанные в предыдущих разделах данной главы. Следующий раздел "Опасности несоответствия" начинается со стандартного языка, чтобы показать некоторые методы в Oracle7 как пример, предоставляющий альтернативы стандартному процессу.

Процесс преобразования использует представленную в примерах на рис. 11.1, 11.2 и 11.3 модель данных.

Правила согласования

Есть два основных шага для создания БД, соответствующей SQL-92, из модели данных UML: создание таблиц из устойчивых типов и создание таблиц из ассоциаций многие-ко-многим и третичных ассоциаций. Каждый шаг предполагает варианты выбора. Выше рассмотрена каждая из этих возможностей в контексте структур UML — этот раздел проведет читателей через полное преобразование, связывающее все.

Устойчивые классы

В качестве первого преобразования создайте таблицу для каждого класса в модели данных UML, которая имеет стереотип "устойчивый". Он включает классы ассоциаций и классы, наследующие от других классов. Присвойте таблице имя класса. На основе рис. 11.1, подсистемы Человек, создается 10 таблиц: Человек, Адрес, Идентификация, ИдентификаторОкончанияСрокаДействия, ВодительскоеУдостоверение, Паспорт, НациональныйИдентификационныйДокумент, ИдентификаторНаказанияПоЗакону, КарточкаСоциальногоОбеспечения и СвидетельствоОРождении.

ВНИМАНИЕ

Позже можно объединить класс ассоциации с другой таблицей, если это окажется возможным. А пока оставьте его отдельной таблицей в этой создаваемой схеме.

Теперь в каждой таблице сформируйте столбец для каждого атрибута в разделе атрибутов класса. В качестве примера для Адреса создайте Номер дома, Номер Квартиры, Название Улицы, СуффиксУлицы, Определитель-Местоположения, Город, Штат, ПочтовыйИндекс, Страна и Комментарий.

В этом месте следует показать, как преобразовывается каждый тип данных из модели данных в тип данных ANSI SQL. Система каталога использует основные типы ANSI, расширенные для включения последовательностей типов "с возможным пустым значением" для каждого типа ANSI, поэтому наше отображение тривиально. Существуют определенные специальные типы, соответствующие идентификации объектов (OID) или перечислимым типам (Пол, например). Такие типы, как VARCHAR, нуждаются в усовершенствовании с помощью добавления спецификатора размера. Если требуется максимальное количество символов для размера, используйте подход наименьшего общего знаменателя. Взгляните на свои целевые DBMS и возьмите максимально возможное из той, которая поддерживает минимальное количество. Например, SQL Server поддерживает только 254 символа в символьном поле переменной длины. Пока другие типы соответствуют ссылкам на другие таблицы с помощью внешних ключей, не трогайте эти столбцы: они будут добавлены в процесс позже. Конечный результат этого этапа для таблицы Адрес выглядит так:

```
CREATE TABLE Address (
    AddressID INTEGER PRIMARY KEY, - OID type
    StreetNumber NUMBER,
    StreetFraction VARCHAR(5),
    StreetName VARCHAR(100),
    StreetSuffix VARCHAR(25),
    City VARCHAR(100),
    State CHAR(2),
    PostalCode CHAR(20),
    Country VARCHAR(100),
    Comment VARCHAR(250)); - size for worst case, SQL Server
```

Если имеется дескриптор {с возможным пустым значением}, добавьте ограничение NULL либо NOT NULL. В каталоге модель данных использует типы данных с возможными пустыми значениями для представления ограничений null, и все столбцы в Адресе, кроме НомераКвартиры и Комментария — не пустые:

```
CREATE TABLE Address (  
  AddressID INTEGER PRIMARY KEY, - OID type  
  StreetNumber NUMBER NOT NULL,  
  StreetFraction VARCHAR(5) NULL,  
  StreetName VARCHAR(100) NOT NULL,  
  StreetSuffix VARCHAR(25) NOT NULL,  
  City VARCHAR(100) NOT NULL,  
  State CHAR(2) NOT NULL,  
  PostalCode CHAR(20) NOT NULL,  
  Country VARCHAR(100) NOT NULL,  
  Comment VARCHAR(250)); - size for worst case, SQL Server
```

На рис. 11.2 представлен пример дескриптора {с возможным пустым значением}: класс ассоциации ВидыДеятельности содержит столбец МетодЗавершения, имеющий перечислимый тип. Вместо создания отдельного типа МетодЗавершения С Возможным Пустым Значением эта диаграмма использует дескриптор {с возможным пустым значением} для определения возможности присвоения пустого значения.

Кроме того, на этом этапе следует создать все служебные таблицы перечислимых типов. Например, требуется таблица Типа Завершения:

```
CREATE TABLE TerminationType (  
  TerminationType CHAR(1) PRIMARY KEY, - character code  
  DisplayString VARCHAR(25)); - string to display in UI and reports
```

Если имеется первоначальное значение столбца, добавьте выражение DEFAULT к определению столбца.

Теперь пора создать первичный ключ и то, что от него зависит. Для начала найдите корневой класс в каждой иерархии генерализации (классы, на которые указывают стрелки генерализации, но из которых не исходит ни одной стрелки генерализации). Также рассмотрите все классы без отношений генерализации.

Если у класса неявная идентификация объектов, добавьте столбец к таблице. Присвойте столбцу первичных ключей имя класса с суффиксом _ID, например Person_ID, и добавьте PRIMARY KEY в качестве ограничения. PRIMARY KEY подразумевает NOT NULL, поэтому такое ограничение не нужно в столбце первичных ключей. Обычно следует присваивать столбцу тип данных NUMBER и использовать последовательность или эквивалент для генерации для него чисел.

ВНИМАНИЕ

Если класс имеет составную агрегатную ассоциацию с другим классом, а другой класс является агрегирующим, воздержитесь от создания первичного ключа, пока не сформируете внешние ключи. Составные агрегатные ключи — это комбинация первичного ключа связанного класса и другого атрибута.

Все классы, представленные на рис. 11.1 и 11.2, имеют неявную идентификацию объектов. Рассмотрим класс Идентификация — корень с неявной идентификацией объектов:

```
CREATE TABLE Identification (
  IdentificationID INTEGER PRIMARY KEY);
```

Если у класса явная идентификация объектов (дескриптор {OID}), добавьте предложение ограничения PRIMARY KEY для таблицы и поместите все столбцы с дескриптором {OID} в ограничение. Если имеется только один столбец в явном идентификаторе, просто добавьте к нему ограничение столбца PRIMARY KEY.

Если у класса есть подклассы, добавьте столбец для представления дискриминатора и создайте ограничение CHECK с соответствующими перечислимыми значениями для представления подклассов.

Теперь перейдем к подклассам. Если есть единственный суперкласс класса, добавьте столбец к таблице, поименованной либо именем первичного ключа в суперклассе, либо новым именем, более подходящим для подкласса. Добавьте PRIMARY KEY в качестве ограничения и добавьте предложение REFERENCES, чтобы связать столбец с первичным ключом таблицы суперкласса в качестве внешнего ключа. Используйте ограничения таблиц PRIMARY KEY и FOREIGN KEY вместо ограничений столбцов, если в суперклассе есть явный многостолбцовый идентификатор.

Класс ИдентификаторПринужденияПоЗакону добавляет столбец Номер Удостоверения к атрибутам Идентификации. Он получает первичный ключ своего суперкласса:

```
CREATE TABLE LawEnforcement ID (
  IdentificationID INTEGER PRIMARY KEY REFERENCES Identification,
  BadgeNumber INTEGER NOT NULL);
```

Если суперклассов несколько (множественное наследование), добавьте столбец к таблице для каждого столбца в каждом первичном ключе каждого суперкласса. Добавьте предложение PRIMARY KEY ко всем столбцам, а также предложение FOREIGN KEY ко всем столбцам для каждого первичного ключа суперкласса. Примеры см. выше в разделе "Множественное наследование".

Классы ассоциаций имеют свои особенности в отношении первичных ключей. Вместо создания нового ключа для представления идентификации объекта присвойте классу ассоциации первичные ключи всех связанных с ним классов. Например, класс ВидыДеятельности на рис. 11.2 представляет собой третичную ассоциацию между тремя классами. Следовательно, ВидыДеятельности получает первичный ключ, содержащий в себе три первичных ключа своих ассоциированных классов. Каждый ключ, в свою очередь,

является внешним ключом таблицы исполняемых ролей, поэтому он получает ограничение REFERENCES для ключей с единственным значением и ограничение таблицы FOREIGN KEY для многозначных ключей.

```
CREATE TABLE Plays (  
    PersonID INTEGER REFERENCES Person,  
    OrganizationID INTEGER REFERENCES Organization,  
    RoleID INTEGER REFERENCES Role,  
    Tenure INTERVAL YEAR TO MONTH,  
    StartDate DATE NOT NULL,  
    EndDate DATE,  
    TerminationMethod CHAR(1),  
    PRIMARY KEY (PersonID, OrganizationID, RoleID));
```

Далее найдите все потенциальные ключи. Если есть какие-либо дескрипторы {альтернативный OID = <n>}, добавьте ограничение UNIQUE к таблице для каждого уникального идентификатора <n> и поместите в него все столбцы с одним и тем же идентификатором <n>. Повторим, что Идентификатор-ПринужденияПоЗакону служит примером, поскольку НомерУдостоверения имеет дескриптор {альтернативный OID = <1>}:

```
CREATE TABLE LawEnforcementID (  
    IdentificationID INTEGER PRIMARY KEY REFERENCES Identification,  
    BadgeNumber INTEGER NOT NULL UNIQUE);
```

Следующий шаг: добавьте любые простые табличные ограничения. При наличии прямоугольников ограничений в диаграмме преобразуйте их в предложения CHECK, если можно выразить ограничение в выражении SQL, содержащем только ссылки на столбцы в определяемой таблице. Если имеются ограничения типов данных, перечисления или диапазоны, добавьте предложение CHECK к столбцу для представления ограничения. Для перечислений с помощью вспомогательных таблиц, содержащих список возможных значений типа дождитесь создания внешних ключей, что является следующим шагом.

Теперь добавьте внешние ключи. На этом этапе все таблицы должны иметь первичный ключ. Если присутствуют какие-либо двоичные ассоциации класса, определяемые с множественностью 0..1 или 1..1 на роль, присоединенную к другому классу, и ассоциация не имеет никакого ассоциированного класса, создайте столбец для представления ассоциации. Конечно, по мере перемещения по таблицам будет достигнут другой конец двоичной ассоциации. Если это ассоциация 0..1 или 1..1 и уже создан столбец для нее в другой таблице, не создавайте внешний ключ в обеих таблицах. Формирование двух внешних ключей отношения не только циклично и трудно поддерживается, но и способно всерьез запутать разработчиков, использующих таблицы. Можно оптимизировать это решение, подумав о том, как приложения станут использовать таблицы. Если вам представляется, что одна сторона ассоциации будет применена более естественно и вероятно, поместите столбец в таблицу. Если множественность роли 1..1, а не 0..1, добавьте ограничение NOT NULL к столбцу/столбцам внешнего ключа.

Внешние ключи для суперклассов уже созданы с помощью отношений генерализации на предыдущем шаге при создании первичных ключей.

Созданию внешних ключей противостоит пара альтернатив, в зависимости от количества столбцов первичного ключа, на который они ссылаются. Если первичный ключ ассоциированной таблицы имеет единственный столбец, создайте единственный столбец с тем же самым типом данных и названием роли в текущей таблице. Добавьте предложение REFERENCES как ограничение столбца, чтобы связать столбец с первичным ключом ассоциированной таблицы. Например, атрибут ОпределительМестоположения в таблице Адреса ссылается на другую систему, называемую ГеоКоорд (ГеографическиеКоординаты) с неявным идентификатором объекта Идентификатор ГеоКоорд:

```
Locator INTEGER NOT NULL REFERENCES GeoCoord(GeoCoordID),
```

Если у первичного ключа ассоциированной таблицы несколько столбцов, создайте то же количество столбцов с теми же типами данных в текущей таблице. Присвойте столбцам имена, используя и название роли, и имена столбцов в ассоциированной таблице (преобразуя имена, как положено или требуется ограничениями длины имени столбца). Добавьте ограничение таблицы FOREIGN KEY, чтобы соотнести столбцы с первичным ключом ассоциированной таблицы.

Если внешний ключ добавлен из составной агрегированной ассоциации с множественностью к-одному с другим классом, добавьте столбец/столбцы внешних ключей к первичному ключу (пара классов предок-потомок). Добавьте соответствующее ограничение CASCADE в предложение FOREIGN KEY в другой таблице. Затем создайте второй столбец для первичного ключа, который уникальным образом определяет объект. Во многих случаях это будет либо столбец с явным {OID}, либо последовательность, если ассоциация является {упорядоченной}. Если нет, тогда создайте целое значение, чтобы уникальным образом определить потомков агрегации.

И наконец, оптимизируйте классы ассоциаций. Если присутствуют какие-либо двоичные ассоциации с ролевой множественностью больше чем 1 для роли, исполняемой определяемым классом, и ассоциация имеет класс ассоциации, добавьте атрибуты из класса ассоциации к таблице с типами данных и ограничениями, как это сделано выше. Удалите из схемы таблицу, созданную для класса ассоциации.

Ассоциации многие-ко-многим и третичные ассоциации

Возможны ассоциации многие-ко-многим или третичные ассоциации, не имеющие классов ассоциаций. Если они есть, значит класс для ассоциации уже был создан. Теперь нужно создать такой класс, если он еще не существует.

Ассоциация многие-ко-многим имеет две множественности ко-многим (0..*, 1..*, *, 1..5, 10 и т. д.). Третичная — три класса, принимающих участие в ассоциации, связанной с помощью ромба. Конечно, можно иметь также четвертичные ассоциации или ассоциации большей кратности.

Найдите в своей модели все ассоциации многие-ко-многим или третичные. Создайте таблицу для каждой из них, которая не имеет класса ассоциации для своего представления. Добавьте столбцы к таблице для первичных ключей каждой из участвующих таблиц точно так же, как это делалось для класса ассоциации.

Если первичный ключ ассоциированной таблицы — единственный столбец, добавьте столбец с названием роли или столбец первичного ключа (в зависимости от семантики). Используйте тип данных из столбца первичного ключа и добавьте ограничение REFERENCES, чтобы связать столбец с первичным ключом.

Если у первичного ключа ассоциированной таблицы несколько столбцов, добавьте аналогичное количество столбцов к ассоциированной таблице, применяя название роли и имя столбца. Добавьте ограничение FOREIGN KEY, чтобы связать столбец с первичным ключом.

Добавьте ограничение PRIMARY KEY к таблице и все столбцы, представляющие первичные ключи участвующих таблиц, к этому ограничению.

Например, ассоциация между Адресом и Человеком — это *, где * указывает на ассоциацию многие-ко-многим. Создадим для нее таблицу ассоциации:

```
CREATE TABLE PersonalAddress (  
    AddressID INTEGER REFERENCES Address,  
    ResidentID INTEGER REFERENCES Person,  
    PRIMARY KEY (AddressID, PersonID));
```

Отметьте адаптацию имени роли "резиденты" к имени столбца ИдентификаторРезидента (ResidentID). Это точнее соответствует значению столбца, чем мог бы сделать PersonID (ИдентификаторЧеловека).

Если множественность роли имеет минимальное значение больше 0, добавьте хранимую процедуру, проверяющую, что строка в таблице ассоциации существует, если есть строка в ассоциированной таблице. Нужно выполнить эту хранимую процедуру в качестве последнего шага в транзакции, включающей INSERT в ассоциированную таблицу. Можно сделать это как триггер AFTER INSERT, но необязательно, если более одной ассоциированной таблицы имеет такие множественности.

Если провести этот процесс несколько раз в разработках средней сложности, то он станет второй натурой разработчика. Повторим еще раз, что в результате создается схема в стандарте ANSI SQL. Теперь посмотрим, как используются отдельные свойства целевой RDBMS, выходящие за рамки стандарта.

Опасности несоответствия

Решение "сойти с колеи" зависит от многих факторов, связанных, главным образом, с культурой. Если клиенты используют только определенные DBMS или организация неособенно следит за соответствием стандартам, разработчик волен применить инновационные способы преобразования своей модели в определенную реляционную БД. Особенно если он намерен приблизиться к ОО-модели и, следовательно, оценить ее преимущества, а

именно: облегчение поддержки, сниженное количество ошибок и высокий потенциал повторного использования. К сожалению, стандартный SQL имеет много преимуществ, но они не включают новаторской архитектуры и ОО-свойств.

Однако, как было сказано, реляционные БД никоим образом не относятся к ОО-системам. Можно приблизиться к ОО-модели, если использовать кое-какие стандартные средства и расширенные средства RDBMS, но это не значит, что мы имеем дело с ООБД. Так, следующее преобразование в качестве примера добавляет средства Oracle7 к стандартной модели из раздела "Правила соответствия". Цель — получить более высокий уровень инкапсуляции и с помощью стандартного SQL смоделировать как можно больше поведения на сервере, а не в приложении. Этого нельзя сделать, используя стандартный SQL, потому что у него нет поведенческих средств, имеющихся у большинства продуктов RDBMS.

Чтобы продемонстрировать некоторые интересные способы отказа от соответствия, мы рассмотрим два примера: как Oracle7 реализует пакетированные подсистемы модели данных и как он выполняет операции классов.

ВНИМАНИЕ

Как говорилось в предисловии, автор, по большей части, имеет опыт работы с Oracle. У каждой RDBMS есть свой собственный набор расширений, которые можно использовать подобным же образом. Поскольку автор больше знаком с Oracle, его примеры с ней.

Пакетированные подсистемы

Подсистемы, также известные как пространства имен, имеют представление на стандартном SQL: концепция схемы ANSI SQL-92. В сущности, ни одна RDBMS не реализует непосредственно эту концепцию, а все предоставляют средства, выходящие за рамки стандарта в этой области, особенно в сфере поддержки распределенных баз данных (БД с несколькими серверами в разных местах).

На Oracle7 можно использовать один из двух подходов к решению этой проблемы. Можно реализовать пакет подсистемы UML как другое пространство имен в одной базе данных либо другие БД, т.е. создается схема для пакета либо у отдельного пользователя, либо в отдельной БД.

Вот так задают пользователи, если хотят создать пространства имен в одной БД Oracle7:

```
CREATE USER Entity IDENTIFIED BY xxxxx;  
CREATE USER Person IDENTIFIED BY xxxxx;  
CREATE USER Organization IDENTIFIED BY xxxxx;
```

Пользователи определяют три схемы, соответствующие устойчивым пакетам в модели данных каталога (см. рис. с 11.1 по 11.3). Затем создают объекты с помощью CREATE TABLE, CREATE VIEW или любых других подходящих средств. Потом, если хотят спрятать факт разделения БД на три части, создают синоним для каждого объекта — иначе нужно предпослать именам объектов их имя в схеме кода SQL.

Чтобы поработать со схемами, лучше определить пользователей, отличных от пакетных пользователей. Именно они нуждаются в синонимах. Можно создать общедоступные синонимы, но это опять же не лучший вариант для инкапсуляции пространств имен. Когда создан зарегистрированный пользователь для приложения, нужно предоставить соответствующие привилегии (SELECT, INSERT, UPDATE и/или DELETE) для объектов, которыми будет пользоваться приложение. Поддержание привилегий на минимуме помогает гарантировать относительно высокий уровень инкапсуляции. Можно также создать представление для дальнейшей инкапсуляции нижележащих объектов.

Этот подход сочетает элементы безопасности, независимости данных и определения схемы Oracle7 SQL для достижения, по крайней мере, частичной инкапсуляции. Он не идеален, но разработчик получает гарантию, что никакое приложение не может получить доступ к базовым данным помимо того, что явно предоставлено.

Альтернатива пользовательскому подходу — создание отдельных БД. Здесь многое зависит от позиции администратора БД. Он должен идентифицировать файлы БД, оценить ее размер и позаботиться обо всех необходимых вещах физической БД, таких как журналы повторов и сегменты отката. Когда БД созданы с помощью команды CREATE DATABASE или Enterprise Manager, затем создаются те же самые системные и прикладные пользователи, что и в предыдущем подходе. Также нужно предоставить привилегии для всех таблиц, но вместо того чтобы непосредственно сделать это, следует сначала установить связи с БД:

```
CREATE DATABASE LINK PersonLink;  
CREATE DATABASE LINK IdentificationLink;  
CREATE DATABASE LINK OrganizationLink;
```

Теперь можно предоставить доступ к объектам каждой схемы путем добавления имени связи к имени объекта:

```
GRANT SELECT ON Person.Person@PersonLink TO AppUser;
```

Единственное преимущество данного подхода состоит в том, что здесь можно повторно использовать элементы системы, совершенно не заботясь о базовых физических связях между схемами. О них надо думать, только когда они все находятся в одной БД. Здесь у нас на самом деле не так уж много преимуществ по сравнению с подходом с единственной БД, если нет некоторой физической оптимизации отдельных БД, которая имеет смысл. Обычно это характерно для полностью распределенных систем в территориально-распределенной сети.

Операции

Простейший способ представления операций в Oracle7 — создание хранимой процедуры или функции для каждой операции, определенной в модели данных. Используя этот подход, разработчик может сам принять решение, какие операции должны вызываться приложениями. В качестве

примера рассмотрите класс КриминальнаяОрганизация на рис. 11.2. Он экспортирует две операции: ОбновитьСтатус и УстановитьПриоритет, обе — "мутаторы", изменяющие состояние определенного объекта криминальной организации.

Язык программирования PL/SQL, предоставляемый Oracle7, — это вычислительно полный язык программирования, содержащий в себе подязык данных SQL для доступа к БД. Технически можно использовать команды SELECT, INSERT, UPDATE и DELETE, используя встроенный SQL, но можно вызвать любой SQL с помощью специальных программных блоков, поставляемых Oracle (пакет DBMS_SQL) для создания или изменения таблиц, предоставления привилегий и т. д. Oracle стремится принять в свое семейство второй язык программирования Java, вскакивая на ходу в грохочущий ОО-вагон. Возможно, так и случится к тому времени, когда выйдет второе издание этой книги. Но автор пока уклоняется от применения таких средств.

Можно преобразовать операцию над классом в хранимую процедуру PL/SQL. Метод (напомним, что метод — реализация операции в UML) имеет, по крайней мере, один вход — нечто, идентифицирующее строку таблицы (объект), на которой собираются работать.

Не превратив этот маленький раздел в длинный трактат о PL/SQL и его возможностях, автор не сможет рассмотреть детали многих имеющихся вариантов для представления специфических концепций. PL/SQL кое-что предполагает для любого разработчика и иногда способен помочь в управлении данной нотацией UML. В данном случае мы рассматриваем два требования: перечислимые типы данных и обновление строки.

Первое требование — пример трудновыполнимого решения в PL/SQL. Системы типа PL/SQL близко моделируют системы типа SQL, не имеющие концепции перечислимого типа. По крайней мере, определение столбца SQL может содержать ограничение CHECK — что, к сожалению, несправедливо в отношении определения переменной PL/SQL. Поэтому нужно перепрыгнуть через пару обручей, чтобы представить перечислимый тип.

Классический подход состоит в создании таблицы просмотра перечислимых значений. Раздел "Таблица преобразования" (см. выше) продемонстрировал, как это сделать для типа ОфициальныйСтатус:

```
CREATE TABLE LegalStatusType (  
  Value CHAR (1) PRIMARY KEY,  
  DisplayString VARCHAR 2(50) NOT NULL,  
  Description VARCHAR2(2000));  
INSERT INTO LegalStatusType (Value, DisplayString)  
VALUES ('D', 'Legally Defined');  
INSERT INTO LegalStatusType (Value, DisplayString)  
VALUES ('T', 'On Trial');  
INSERT INTO LegalStatusType (Value, DisplayString)  
VALUES ('A', 'Alleged');  
INSERT INTO LegalStatusType (Value, DisplayString)  
VALUES ('U', 'Unknown');
```

Чтобы использовать данный тип, код значения передает процедуре, а затем применяют данную таблицу для проверки этого значения:

```
CREATE OR REPLACE PROCEDURE SetStatus (p_Self IN INTEGER,
                                       p_Status IN CHAR) IS
    v_TypeOK INTEGER := 0;
BEGIN
    SELECT 1
    INTO v_TypeOK
    FROM LegalStatusType
    WHERE Value = p_Status;
    IF v_TypeOK = 1 THEN
        UPDATE CriminalOrganization
        SET LegalStatus = p_Status
        WHERE OrganizationID = p_Self;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN-Couldn't find the status
        DBMS_OUTPUT.Enable(100000);
        DBMS_OUTPUT.Put_Line(to_char(sysdate, 'dd-Mon-yy hh:mm'))||
        ' SetStatus: '||p_Status||
        ' is not a valid legal status for a criminal
        organization. ');
END SetStatus;
```

Такая процедура PL/SQL сравнивает перечислимый тип с таблицей значений и, если он там, обновляет определенную разработчиком криминальную организацию (p_Self) новым статусом. Если его там нет, PL/SQL порождает ошибку NO_DATA_FOUND и передает управление обработчику прерываний, который записывает сообщение об ошибке в журнал вывода сервера. При наличии этой процедуры мутатора разработчик больше не должен кодировать операторы обновления в программе. Вместо этого вызывается хранимая процедура, возможно из Pro*C – встроенной в SQL C++ программы:

```
exec sql execute begin SetStatus(:self, :legalStatus); end; end-exec;
```

Можно инкапсулировать этот код, например, в функцию-член Установить Статус класса КриминальнаяОрганизация.

В этом случае код обновления строки – простой оператор SQL, но код для проверки типа перечислимого значения довольно велик. Перед нами типичный случай кодирования в среде R-(и OR-) системы управления данными, которая нещедра в средствах программирования для управления ошибками. Ошибки в PL/SQL на самом деле – значительное улучшение по сравнению с подходом на основе кода возврата, который превалировал в предыдущей среде. Однако, как и в любом другом случае управления ошибками, нужно быть осторожным и изолировать управление ошибками на

сервере. Распространение ошибок на клиента нежелательно: это приводит к фатальному сбою серверного процесса.

Более систематический подход к операциям добавляет еще один шаг к алгоритму в разделе "Правила соответствия".

Создайте пакет PL/SQL для каждого класса с "постоянным" стереотипом. Присвойте ему имя класса, добавив суффикс `_pg`, чтобы различать пакет и таблицу, определенную для этого класса (единое пространство имен). Добавьте тип `RECORD` к базовой таблице и ее столбцам, которые используются для определения позже курсора `SELECT`. Теперь тип `RECORD` — к столбцам первичного ключа. Затем подпрограммы для операций строк `INSERT`, `UPDATE`, `DELETE` и `LOCK` — к базовой таблице. Добавьте подпрограмму для возврата курсора ко всем строкам в таблице (общий запрос таблицы) и подпрограммы для каждой операции, определенной в разделе поведения прямоугольника класса, присваивая методам имена операций.

Для класса *Криминальная Организация* получаем следующий пакет:

```
CREATE OR REPLACE PACKAGE CriminalOrganization_pg IS
  TYPE tCriminalOrganization IS RECORD (
    rOrganizationID CriminalOrganization.OrganizationID%TYPE,
    rLegalStatusID CriminalOrganization. LegalStatus%TYPE,
    rStability CriminalOrganization. LegalStatus%TYPE,
    rInvestigativePriority
      CriminalOrganization. InvestigativePriority%TYPE,
    rProsecutionStatus CriminalOrganization. ProsecutionStatus%TYPE);
  TYPE tCriminalOrganizationKey IS RECORD (
    rOrganizationID CriminalOrganization.OrganizationID%TYPE);
  TYPE tCriminalOrganizationCursor IS REF CURSOR
    RETURN tCriminalOrganization;
  TYPE tCriminalOrganizationTable IS TABLE OF tCriminalOrganization
    INDEX BY BINARY_INTEGER;
  TYPE tCriminalOrganizationKeyTable IS TABLE
    OF tCriminalOrganizationKey INDEX BY BINARY_INTEGER,
  --Операции
  PROCEDURE SelectCursor (pCursor IS OUT tCriminalOrganizationCursor);
  PROCEDURE SelectRows(pTable IN OUT tCriminalOrganizationTable);
  PROCEDURE InsertRows(pTable IN tCriminalOrganizationTable);
  PROCEDURE UpdateRows(pTable IN tCriminalOrganizationTable);
  PROCEDURE DeleteRows(pKeyTable IN tCriminalOrganizationKeyTable),
  PROCEDURE LockRows(pKeyTable IN tCriminalOrganizationKeyTable);
END CriminalOrganization_pg;
```

Такой подход позволяет полностью инкапсулировать базовую таблицу в пакет распространенным способом. Первый оператор `TYPE` определяет столбцы таблицы как тип записи. Второй позволяет представить значение первичного ключа в одной записи. Третий `TYPE` предоставляет "ссылочный

курсор", участвующий в запросе всех данных в таблице. Четвертый — "множество результатов" таблицы PL/SQL, позволяющее сохранить несколько криминальных организаций в одной переменной. Пятый TYPE дает подобную возможность сохранять несколько первичных ключей. Затем процедуры используют эти типы для предоставления стандартных возможностей выполнить выборку, включение, обновление и уничтожение. Процедура SelectCursor возвращает курсор, который можно использовать для итераций по таблице. Можно добавить параметры к этой процедуре, чтобы передать квалификаторы предложения WHERE для возврата подмножеств. Процедура SelectRows возвращает все множество результатов для одного запроса, а не возвращает курсор. Процедуры InsertRows и UpdateRows вставляют и обновляют строки из множества записей в таблице. Процедура DeleteRows уничтожает все строки, идентифицированные в таблице первичных ключей. Процедура LockRows позволяет заблокировать набор строк, определенных их первичными ключами при обновлении. Эти общие процедуры предоставляют все возможности, необходимые для поддержания БД. Можно, конечно, добавить такие специфические операции, как установка статуса:

```
PROCEDURE SetStatus(pKey IN tCriminalOrganizationKey,  
                   pStatus IN CHAR);
```

Повторим, что объект определяется для обновления с помощью ключа, а статус для его обновления как параметр. Можно также предоставить процедуру группового обновления, используя таблицу ключей:

```
PROCEDURE SetStatus(pKeyTable IN tCriminalOrganizationKeyTable,  
                   pStatus IN CHAR);
```

Если теперь прикладным пользователям предоставляется привилегия EXECUTE для пакета, но не без привилегий для базовой таблицы: приложение нужно программировать для интерфейса пакета, а не выбирать данные непосредственно из таблицы. Использование этой комбинации безопасности SQL и пакетов PL/SQL позволяет разработчику полностью инкапсулировать таблицу БД в рамках программирования API.

ВНИМАНИЕ

Developer/2000, система генерации приложений Oracle, использует этот вид пакета как альтернативу непосредственного доступа к таблицам. Таким образом, можно обеспечить гораздо больший контроль за таблицами с помощью приложения Developer/2000, потому что никакое другое приложение любого вида не сможет иметь доступ к таблицам непосредственно без авторизации. Это предоставляет разработчику средство быстрой разработки приложений с высокоинкапсулированным доступом к данным RDBMS.

Итоги

К этой длинной главе невозможно сделать заключение в виде пары параграфов. То, что действительно требуется для преобразования модели UML данных в реляционную БД,— это быть очень и очень последовательным и

своем подходе. Если вы поняли, как концепция UML преобразуется в реляционную, то легко представить большинство аспектов модели. Прежде всего надо решить, соответствовать ли стандарту ANSI SQL для достижения переносимости или воспользоваться возможностями целевой DBMS, выходящими за рамки стандарта. Второй вариант позволяет значительно приблизиться к ОО-реализации системы.

Для удобства итоги подводит таблица 11.4. FOREIGN KEY и PRIMARY KEY означают либо табличное ограничение, либо ограничение столбца REFERENCES, в зависимости от того, многозначен ли ключ.

Таблица 11.4. Пошаговое преобразование R-схемы

Шаг	Преобразование
1	Класс UML становится таблицей.
2	Атрибут UML в классе становится столбцом в таблице.
3	Тип атрибута UML в классе становится типом столбца в таблице с помощью таблицы преобразования типов.
4	Если есть дескриптор атрибута UML {с возможным пустым значением}, атрибут имеет ограничение NULL, иначе — ограничение NOT NULL.
5	Если у атрибута UML есть первоначальное значение, добавьте к столбцу раздел DEFAULT.
6	Для классов без генерализации (корневых или независимых) и неявной идентификации создайте целочисленный первичный ключ: для {OID} добавьте столбец с дескриптором {OID} к ограничению PRIMARY KEY, проигнорируйте классы составных агрегаций и ассоциаций.
7	Для подклассов добавьте ключ каждого порождающего класса к ограничениям PRIMARY KEY и FOREIGN KEY.
8	Для ассоциированных классов добавьте первичный ключ из каждой таблицы исполняемых ролей к ограничениям PRIMARY KEY и FOREIGN KEY.
9	Если {альтернативный OID = <n>}, добавьте столбец к ограничению UNIQUE.
10	Добавьте CHECK для каждого явного ограничения.
11	Создайте столбцы FOREIGN KEY в таблице ссылок для каждой роли 0..1, 1..1 в ассоциации.
12	Создайте PRIMARY KEY для составной агрегации с FOREIGN KEY для агрегирующей таблицы (с помощью варианта CASCADE); добавьте дополнительный столбец для PRIMARY KEY.
13	Оптимизируйте классы двоичных ассоциаций путем перемещения к таблице стороны ко-многим, где возможно.
14	Создайте таблицы для третичных ассоциаций и ассоциаций многие-ко-многим без классов ассоциаций.
15	Создайте ограничения PRIMARY KEY, FOREIGN KEY из ключей таблиц исполняемых ролей в ассоциациях многие-ко-многим и третичных.

Таким образом, стандартный подход очень ограничен, но безопасен в терминах взаимодействия с реляционной БД. Получив две разновидности систем управления БД, разработчик больше не будет испытывать проблем. В то время как возникают стандарты для ООБД и ОРБД, на практике необходимо адаптироваться к реализациям целевых DBMS, которые во многом не соответствуют никаким стандартам.

Глава 12

Разработка схемы объектно-реляционной базы данных

*Ex umbris et imaginibus in veritatem. — От теней и символов
к реальности.*

Девиз кардинала Ньюмена

Объектно-реляционные базы данных расширяют возможности реляционной БД в представлении объектов. Хотя пока еще не существует стандартов в этой области, из предпринимаемых усилий по созданию стандартов SQL3 выстраивается достаточно ясная картина.

Так что же нового?

Продукты ORDBMS также известны как "универсальные" серверы или менеджеры БД, видимо, потому, что они могут управлять любыми видами данных или потому, что поставщики хотят, чтобы их клиенты адаптировали новую технологию универсальным образом (весьма слабая шутка). Продукты ORDBMS снабжают R-систему такими основными свойствами, как:

- Типы данных больших объектов
- Расширенные типы данных
- Многозначные типы данных
- Идентификация объектов
- Расширения, помогающие при использовании вышеперечисленных средств

Свойства

Во-первых, продукты ORDBMS добавляют дополнительные типы к системе типов для представления объектов произвольной сложности, именно объекты BLOB, CLOB, FLOB и т.д. LOB — это сокращение от "large object" (большой объект), и все эти типы данных представляют объекты, давая вам доступ к компьютерному представлению объекта как данных SQL. Хотя, подумав, можно сказать, что смысл всего этого несколько преувеличен. На самом деле типы данных LOB позволяют помещать в БД отдельные произвольные наборы битов, вот и все. Наиболее развитые БД позволяют получать биты или символы обратно, но только разработчик может придать им смысл. Стандарт ANSI SQL3 определяет три разновидности больших объектов: CHARACTER LARGE OBJECT (CLOB), NATIONAL CHARACTER LARGE OBJECT (NCLOB) и BINARY LARGE OBJECT (BLOB). Поскольку эти типы не имеют встроенных средств, продукты ORDBMS также предоставляют способ добавления поведения в систему, чтобы оживить эти объекты. К сожалению, это наименее стандартизированная часть расширения. Oracle8, Informix Dynamic Server и DB2 Universal Data Base (UDB) — все поддерживают эти виды типов данных, хотя с разным синтаксисом и свойствами.

Во-вторых, продукты ORDBMS расширяют систему типов, чтобы позволить SQL иметь дело с определяемыми пользователем типами (UDT), абстрактными типами данных (ADT) или объектными. Новый стандарт SQL3 имеет оператор CREATE TYPE, который создает расширенные типы, например отраженные в Oracle8 как CREATE TYPE и в DB2 IBM как UDB. SQL3 включает генерализацию типов в структуру наследования. Oracle8 и UDB не поддерживают наследования, в отличие от Informix. Многие продукты ORDBMS также предоставляют средства расширения представлений типов, подобные картриджам Oracle8 NCA, Informix DataBlades и непрозрачные типы или отдельные типы UDB и хранимые процедуры. Главное отличие между этими расширениями и подобными средствами в R-системах — тесная интеграция расширений в систему типов SQL. Это делает данные доступными в контексте как SQL, так и прикладных программ. Старая технология реляционных БД просто хранила данные в неструктурированных объектах BLOB и нужно было переправлять их в программы для того, чтобы что-то с ними сделать.

В-третьих, многие (но не все) продукты ORDBMS ослабляют ограничения первой нормальной формы, что позволяет вкладывать таблицы в таблицы, чтобы массивы выглядели как значения столбцов, и связывать таблицы через указатели или ссылки, а не через данные. У стандарта ANSI SQL3 есть массивы, но нет вложенных таблиц. UDB предоставляет табличные типы, внешние программы, материализующие таблицы, которые затем можно использовать в разделе FROM для дальнейшей обработки с помощью SQL. Oracle8 имеет вложенные таблицы и массивы переменных. Informix — три типа данных коллекций: множество, мультимножество и список

ожидать, что кто-то сможет в ближайшем будущем разработать переносимые OR-схемы. Эквивалентов ODBC, JDBC или даже ADO просто не существует в БД такого типа, и абсолютная сложность проблемы в том, что они вряд ли скоро появятся. Будем надеяться, что поставщики ORDBMS ввели автора в заблуждение по этому поводу, но сомнения остаются.

Следовательно, в этой главе мы не будем сильно вдаваться в подробности специфических технологий. Основное внимание здесь уделяется основным принципам преобразования, которые можно применить к модели данных в различных видах средств, которые предоставляют продукты ORDBMS.

Процесс преобразования для продуктов ORDBMS

В этой главе рассматривается проект стандартного языка схем SQL3 для OR-систем [ISO, 1997]. Он содержит примеры вариантов схем в среде менеджера БД Oracle8 с использованием Oracle SQL и PL/SQL [Koch, 1998]. Там, где это подходит, предоставляется обзор синтаксиса, который обеспечивают Informix Dynamic Server [McNally, 1997] и DB2 UDB, использующей DB2 SQL [Chamberlin, 1998]. Хотя язык SQL3 разъясняющий, его отличия от новой технологии и трудности, связанные с его принятием в качестве формального стандарта ISO [Melton, 1998], столь серьезны, что не следует подробно разбирать данный вопрос.

Для иллюстрации процесса в этой главе используется разработка из главы 11: так удобнее. На рис. 12.1 представлена модель UML для подсистемы Человек из каталога, а на рис. 12.2 — модель UML для подсистемы Организация. Обе подсистемы являются частью третьей — подсистемы Сущность. На рис. 12.3 представлена архитектура этого пакета.

Подсистема Человек содержит класс Человек и иерархию Идентификация, которая ей принадлежит. Автор предпочел использовать версию Идентификации с наследованием, а не интерфейсную версию. Люди связаны с Организацией через трехстороннее отношение с классом Роль в пакете Организация. Нотация с оператором разрешения области видимости (Организация::Роль) идентифицирует классы, не являющиеся частью подсистемы Человек.

Подсистема Организация содержит иерархию Организации, включающую класс КриминальнаяОрганизация, а также класс Роль и отношения между Ролью и Организацией. Организации связаны с людьми через трехстороннее отношение с классом Роль.

Подсистема Сущность состоит из трех элементов: двух подсистем — Люди и Организация — плюс абстрактный класс Сущность. Классы Человек и Организация в соответствующих пакетах наследуют из этого класса.

Однако, рассматривая только три пакета, нельзя понять, насколько полезен переход к ORDBMS. Поэтому добавим в этот состав два пакета — для управления изображениями и для управления географическим местоположением.

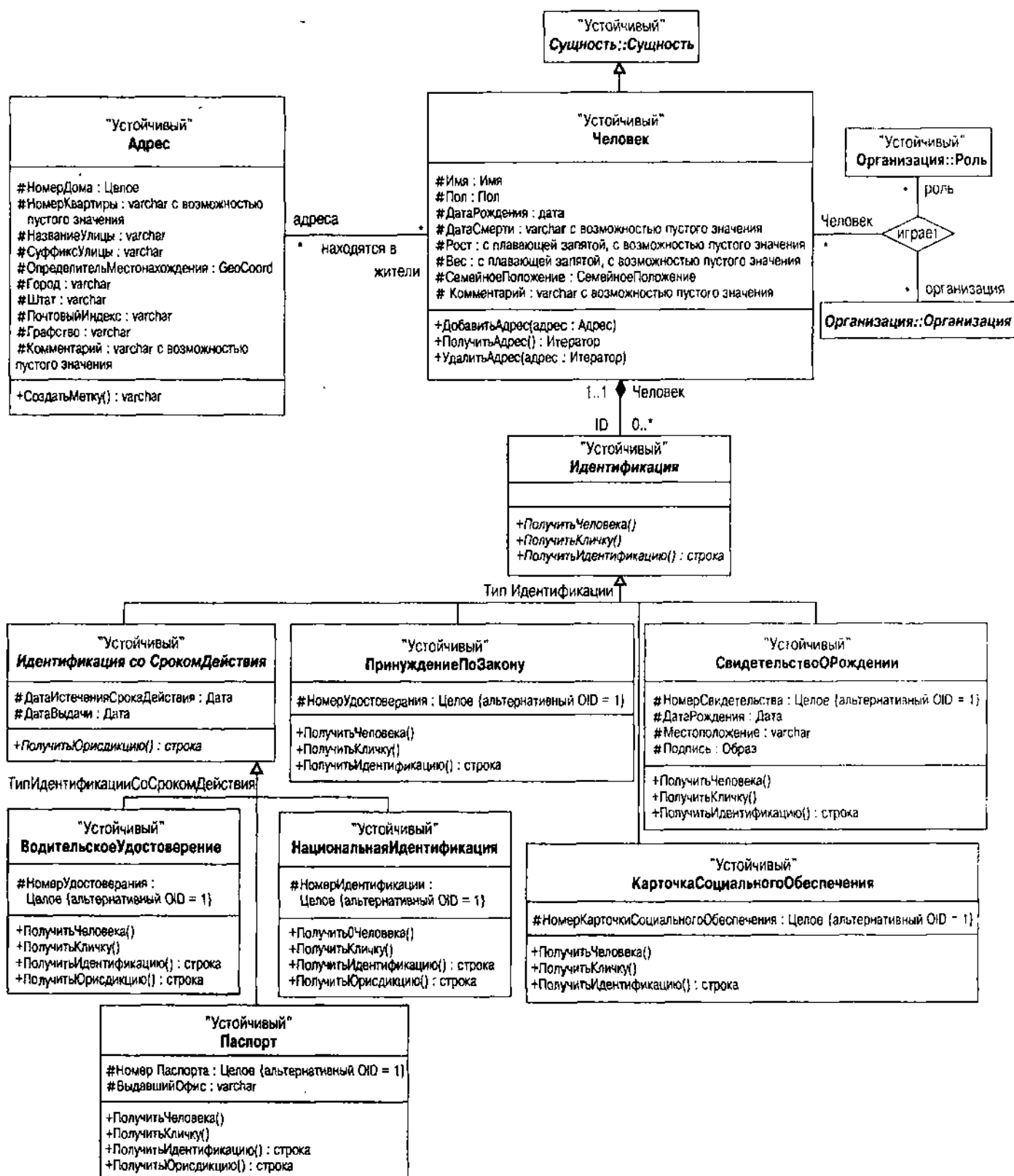


Рис. 12.1. Представление подсистемы Человек на UML

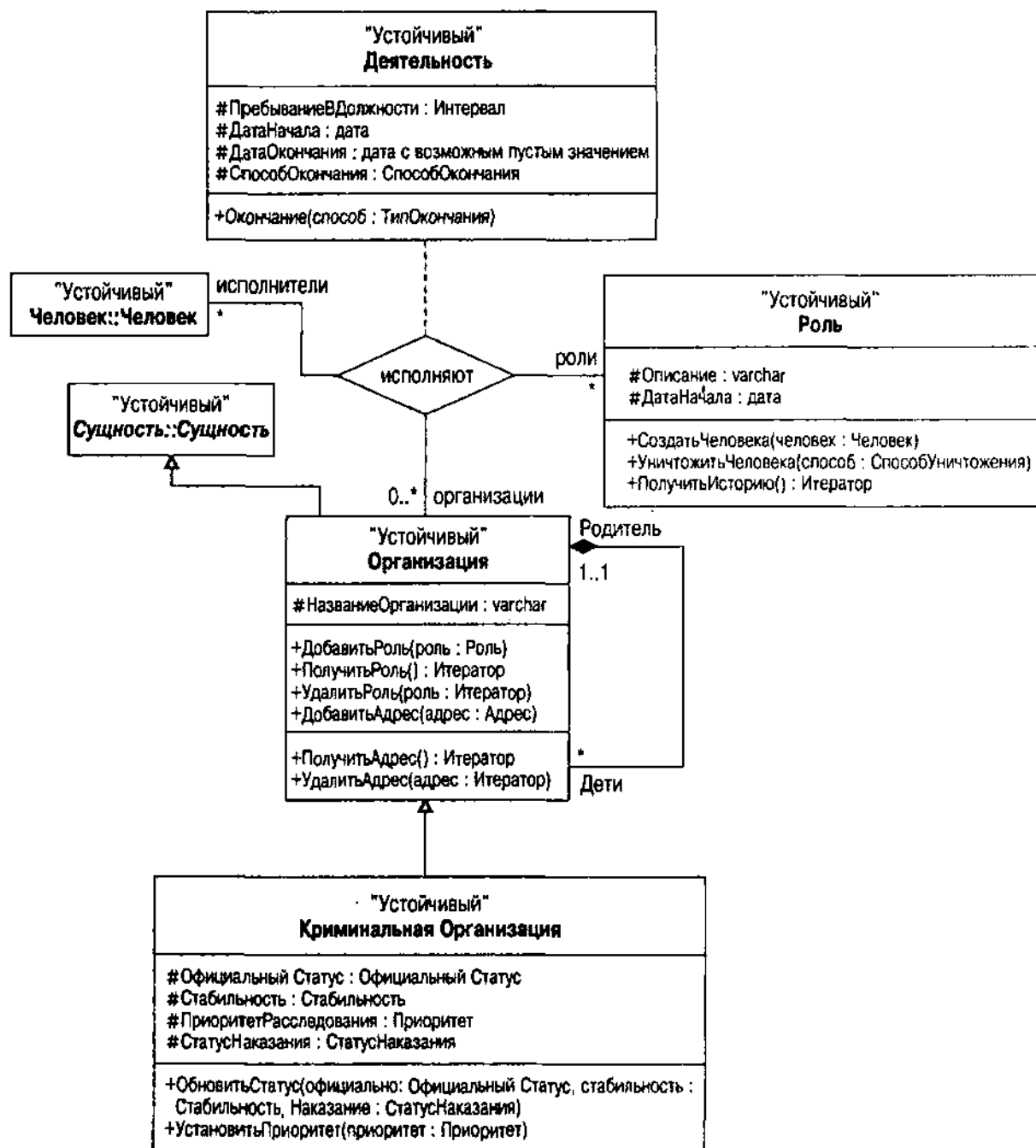


Рис. 12.2. Представление подсистемы Организация на UML

На рис. 12.4 показана простая иерархия изображений. Идея разработки основана непосредственно на принципах картриджа данных для выборки визуальной информации Oracle8 [Oracle, 1997b]. Подкласс отражает некоторые опции, какие Холмс захотел бы видеть в каталоге: изображения пепла сигары, изображения лиц, а также изображения собранных вещественных доказательств. Можно связать их с другими классами, такими как отношение между подсистемой Человек класса Человек и классом ИзображениеЛица.

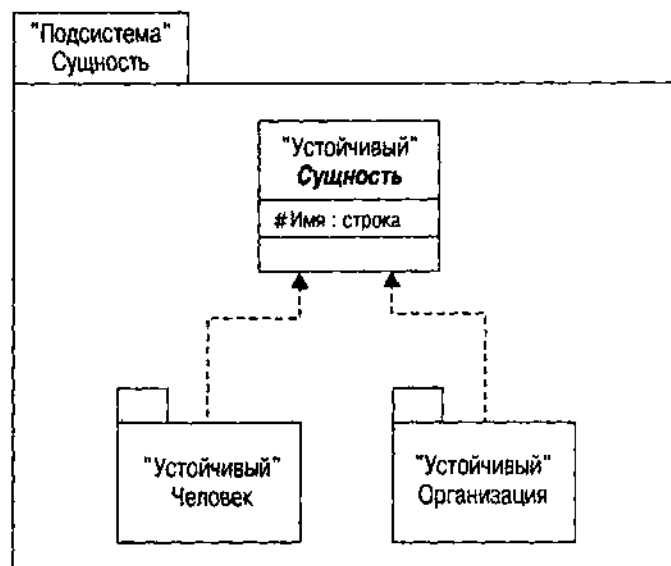


Рис. 12.3. Представление подсистемы Сущность на UML

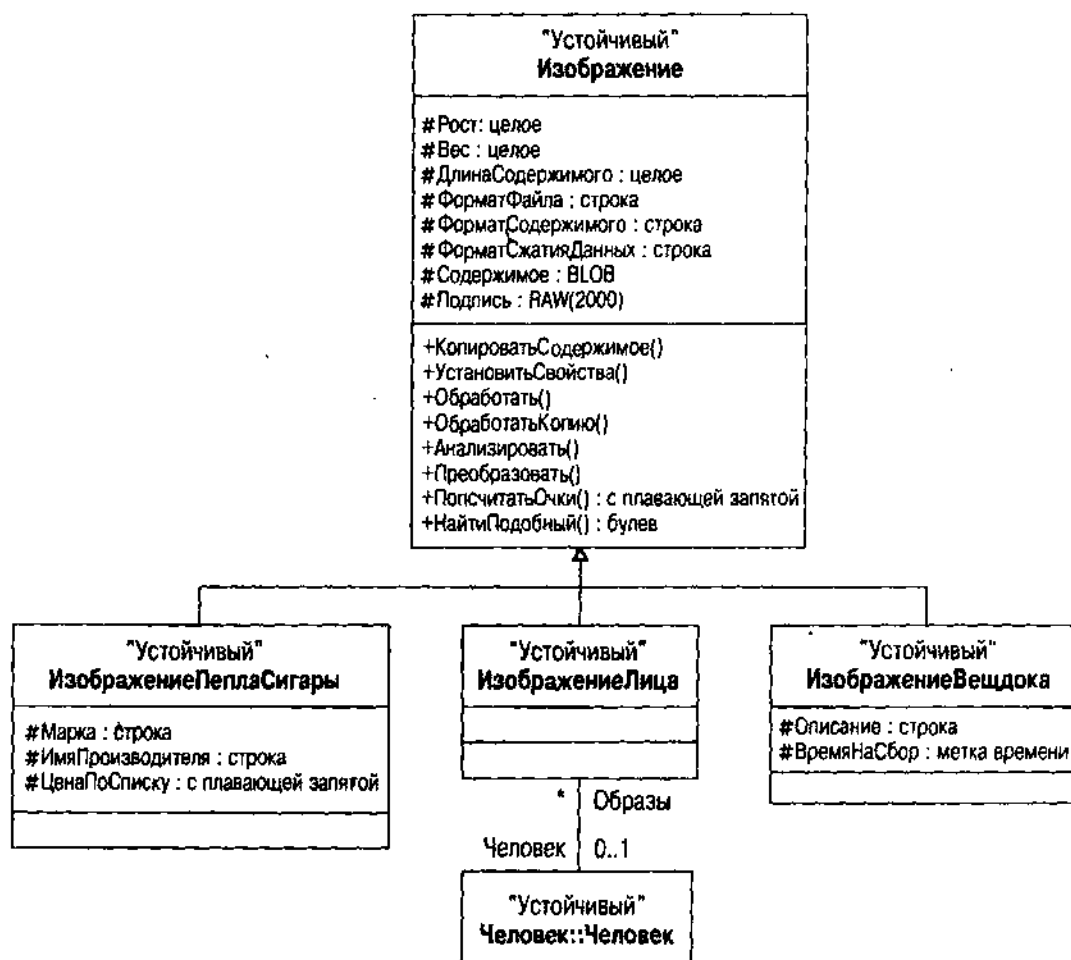


Рис. 12.4. Подсистема Изображение на UML

Для обоснования иерархии Изображения рассмотрим некоторые ситуации, характерные для Холмса. В одном из ранних дел возникла проблема идентификации сигарного пепла.

"Я собрал немного пепла, просыпавшегося на пол. Он был темного цвета, похож на хлопья: такой пепел мог получиться только от Трихнополиии. Я специально изучал сигарный пепел — фактически написал монографию на эту тему. Я себе льщу, полагая, что могу отличить с первого взгляда пепел любой известной марки либо от сигары, либо от табака. Именно в таких мелочах и заключается разница между маститым детективом из Грегсона и детективом типа Лестрейда." [STUD]

С помощью изображения сигарного пепла, которое можно отыскать в БД, закодировав текстуру и цвет, любой детектив из Скотленд-Ярда мог бы сделать работу, используя фотографию образца пепла.

В одном из расследований, что велось гораздо позже, когда Холмс отошел от дел, он использовал фотографию:

— И наконец, имеется вопрос об инструменте, с помощью которого были нанесены эти раны.

— Что же это могло быть, как не плетель или какой-то кнут?

— Вы изучили следы? — спросил я.

— Я видел их. И доктор тоже.

— А я их исследовал очень тщательно с помощью линзы. У них есть особенности.

— Какие, мистер Холмс?

Я отошел к своему бюро и принес увеличенную фотографию:

— Вот мой метод в таких случаях, — объяснил я.

— Вы определенно делаете все очень тщательно, мистер Холмс.

— Я вряд ли бы был тем, кто я есть, если бы этого не делал. Теперь давайте рассмотрим этот рубец, проходящий через правое плечо. Вы не наблюдаете ничего примечательного?

— Да вроде нет.

— Совершенно очевидно, что он неоднороден по интенсивности. Здесь точечный кровоподтек, а там другой. Подобные метки есть на этом другом шраме ниже, вот здесь. Что это значит?" [LION]

Сравнивая фотографии с медицинскими историями дел, детектив мог бы потенциально идентифицировать характеристики шрамов Льюиса Гривы. Определенно, использование цифровых фотографий упростило бы работу консультирующего судебного эксперта, который, вполне возможно, заметил бы странные характеристики ран на спине жертвы.

Холмс может быть драматичным, когда делает выбор:

"Он вставил ключ в замок, и все мы очень тихо вошли в келью. Снявший повернулся в пол-оборота, а затем опять заснул в глубоком покое. Холмс наклонился к кувшину с водой, намочил губку, а затем провел ей дважды энергично вдоль и поперек лица заключенного.

— Позвольте вам представить, — крикнул он. — Мистер Невиль Сейнт Клер из Ли, что в графстве Кент.

Никогда в жизни не видел я такого зрелища. Лицо человека отпало под губкой, как кора с дерева. Грубый коричневый тон исчез! Также исчез и ужасный шрам, пересекавший лицо поперек вместе с искривленной губой, придававшей лицу отталкивающее презрительное выражение!

— ...Боже мой! — вскричал инспектор, — Да это же и вправду тот, кого мы ищем. Он мне известен по фотографии". [TWIS]

Если бы Скотленд-Ярд использовал систему идентификации лиц — компьютер, возможно, выявил мистера Невилля Сейнт Клера на основании фотографии субъекта — человека с искривленной губой. Вероятно, сходство черт лица могло бы быть значительным.

Последний пример поможет выявить тесную связь между технологией и требованиями: она возникает, когда начинают использовать ОР-подхода. Рассмотрим проблему монастырской школы:

— Это дело зрест во мне, Уатсон, — сказал он. — Несомненно, есть несколько зацепок. На этой начальной стадии я хочу, чтобы вы знали те географические особенности, которые могут иметь непосредственное отношение к нашему расследованию.

— Взгляните на карту. Этот темный квадрат — монастырская школа. Я воткну в нее булавку. А вот эта линия — главная дорога. Вы видите, что она простирается на восток и запад от школы, а также видите, что в пределах мили в любую сторону боковая дорога отсутствует. Если эти два приятеля ушли по дороге, то это была вот эта дорога.

— Совершенно верно.

— Благодаря единственной счастливой возможности мы в силах отчасти проверить, кто прошел по этой дороге интересующей нас ночью. В том месте, где сейчас лежит моя трубка, с двенадцати до шести на посту находился констебль. Это, как вы понимаете, первый перекресток с восточной стороны. Этот человек утверждает, что не уходил с поста ни на минуту, и он уверен, что ни мальчик, ни мужчина не могли бы пройти мимо него незамеченными. Я поговорил с полицейским сегодня вечером, и он показался мне человеком, которому можно полностью доверять. Вопрос с этой стороны закрыт. Теперь посмотрим, что с другой. Здесь постоялый двор "Красный Бык", хозяйка которого была больна. Она послала в Маклтон за доктором, но он не пришел до утра, поскольку был занят с другим случаем. Люди на постоялом дворе ночью не ложились спать, ожидая его прихода, и то один, то другой постоянно бросал взгляд на дорогу. Они утверждают, что никто не проходил. Если это так, то нам повезло и мы можем закрыть вопрос и с запада, а заодно можно сказать, что беглецы вообще не пользовались дорогой.

— Но велосипед? — возразил я.

— Вот именно. Сейчас мы перейдем к велосипеду. Продолжим наши рассуждения: если эти люди не шли по дороге, то они должны были пойти либо на север от дома, либо на юг. Это точно. Давайте сравним эти варианты. К югу от дома, как вы видите, находится большой участок

пахотной земли, нарезанный на небольшие поля, с каменными степами между ними. Полагаю, что здесь невозможно воспользоваться велосипедом. Это предположение отпадает. Повернем на север. Вот здесь находится рощица под названием "Рваный Шоу", а там, вдалеке, на десять миль простирается великий плавающий торфяник Нижний Джил Мур и хлюпающий постепенно выше. Здесь, с этой стороны этого дикого места, находится Холдернес Холл, десять миль по дороге, но только шесть через торфяник. Это особенная необитаемая равнина. Несколько фермеров с торфяников имеют небольшие владения, где они изредка пасут овец и коров. Кроме них, ржанка и кроншнеп являются здесь единственным обитателями, пока вы не попадете на Честерфильдский тракт. Там есть церковь, смотрите, несколько коттеджей и постоянный двор. Вдали находятся отвесные горы. Именно здесь, к северу, должна находиться отгадка".

— Но велосипед? — настаивал я.

— Ну, ну! — нетерпеливо сказал Холмс. — Хороший велосипедист не нуждается в шоссе. Торфяник пересекают тропинки, и было полнолуние".
[PRIO]

В середине дела, включающего географию такого рода, детективу очень пригодилась бы БД географических местоположений, как справочный материал и поисковое средство. Подсистема Геометрия на рис. 12.5 иллюстрирует одну такую разработку.

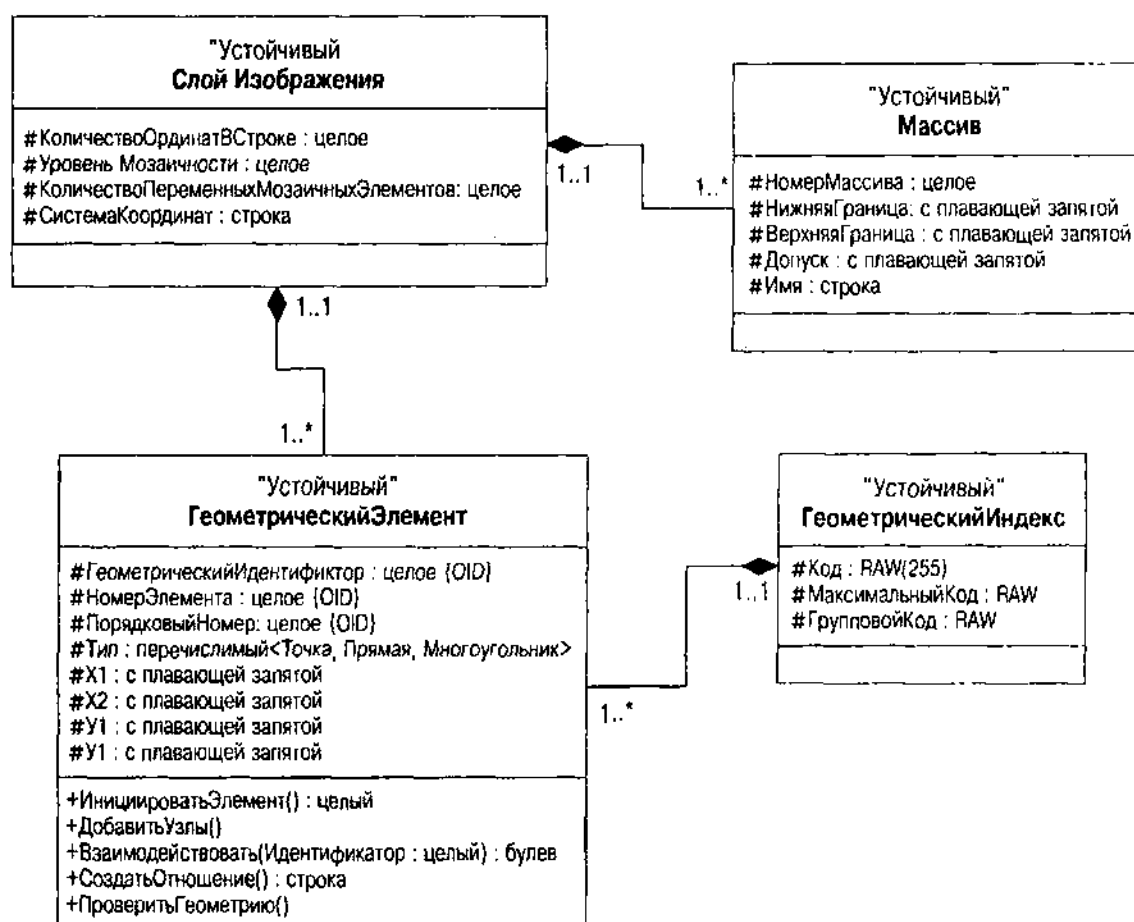


Рис. 12.5. Представление подсистемы Геометрия на UML

Элементами являются точка (широта и долгота), линия (две точки) или многоугольник. Геометрия — это набор элементов, образующих географический объект (торфяник, например, или два постоянных двора). Слой — это совокупность связанных геометрий. Можно иметь слой характеристик земель, слой дорог и слой зданий для описания всей карты, с помощью которой Холмс нашел решение проблемы. Класс ГеометрическийЭлемент поддерживает все поведение системы, включая создание элементов (ИнициализироватьЭлемент и ДобавитьУзлы), проверку внутренних характеристик элемента (ПроверитьГеометрию) и поиск (Взаимодействовать и СоздатьОтношение для проверки, перекрываются элементы или связаны друг с другом по включению либо через другие отношения).

Это разнообразие проекта, с которым сталкиваются в реальной жизни. Применение специальной методики означает, что проект БД должен использовать схему, которую предоставляет данная технология. Если вы собираетесь создать свою собственную географическую информационную систему (GIS), можно задействовать любой проект по желанию, если он подходит. В данном случае на рис. 12.5 использован проект из картриджа пространственных данных Oracle8 [Oracle 1997a] — технология, разработанная третьей стороной и интегрированная с Oracle, которая применяет вычислительную геометрию линий и многоугольников для представления и обработки географической информации. Этот картридж — абсолютно оригинальный и уникальный подход к разработке географических информационных систем, не совместимых с другими подобными системами. Он индексирует географические элементы, используя подход на основе клеточного разбиения, называемый мозаичным представлением. Мозаика разбивает пространство на элементы, содержащие части географической карты до получения хорошего разрешения. Использование картриджа данных означает установку ряда таблиц, необходимых системе для хранения необходимой информации, в частности таблиц Слой и Геометрия. Затем они становятся основой модели данных. Таким образом, технологические требования доминируют над функциональными и определяют выбор решения при разработке. Эта неплохая ситуация в проекте Изображение на рис. 12.4.

В данном случае проект не является таким, каким его хотели бы видеть ОО разработчики. Во-первых, картридж пространственных данных не использует типы объектов или любые другие более интересные ОО-дополнения Oracle8. Это простая передача R-реализации "Пространственных данных" в Oracle7. Кроме того, видимо для оптимизации производительности, разработчики картриджа пространственных данных нарушили большинство правил проектирования.

Во-первых, каждый слой в системе получает уникальный набор таблиц, идентифицированных именем. Следовательно, если имеется слой дорог, для него будет четыре таблицы с именами: Road_SDOLayer (слой), Road_SDODim (массив), Road_SDOGeom (геом) и Road_SDOIndex (индекс). Это превращает таблицу Слой в единственную в своем роде таблицу, с одной строкой, описывающей этот слой. На самом деле единственная таблица требуется только для того, чтобы избежать объединения, и поэтому накладывает слишком большие ограничения. Вместо этого нужно иметь последовательности

слоев, идентифицированных с помощью Идентификатора и Имени. Таблица Размерность существует в виде дуплета, если так можно сказать, с двумя строками, соответствующими двум массивам, поддерживаемым картриджем пространственных данных. Ни одна из двух таблиц не содержит первичного ключа, поскольку использован подход на основе единственности (singleton).

Таблица Geom, как можно было бы подумать, не является геометрической таблицей — напротив, она представляет индивидуальные элементы, образующие геометрию. Хуже того, разработчики использовали схему "повторяющихся групп", позволяющую по-другому построить конфигурацию таблицы для каждого слоя. Одни слой может иметь строки Geom с двумя координатами, когда другой слой — шесть. Еще хуже, индивидуальный элемент может объединять объекты с помощью одного настоящего геометрического элемента (многоугольника), распространяясь на несколько строк с многими парами координат. Стандартные методы разработки добавили бы настоящую таблицу Geometry (геометрия) для хранения геометрических данных, таблицу Элемент для хранения сведений об элементе и таблицу Координаты для хранения пар координат. Немного расширив это, можно поддерживать более двух массивов класса Координаты, который хранит векторы координат постоянной размерности. Это будет очень трудно сделать с помощью настоящей схемы.

Таблица Индекс содержит элементы геометрии, которую вычисляет схема мозаики. Таблица связана с Геометрией. К сожалению, нет такого объекта в модели данных, поскольку таблицы представляют его не непосредственно, а только в агрегатной форме через таблицу элементов с несколькими строками.

Такой проект неприятен ОО разработчику. Однако он существует. И представляет ценность для прикладного разработчика Oracle8 потому, что позволяет ему выполнить географическое представление и поиск, хотя и негибко, без написания соответствующего кода. Считается, что на арене повторно используемого программного обеспечения сам факт существования на 9/10 является доказательством законности. А закон временами бывает жесток.

Таким образом, у вас есть выбор между разработкой и повторным использованием. Обычный ответ в таком случае: образование слоев — создание пакетов для пакетов. Чтобы сделать подход картриджа пространственных данных более общим, можно инкапсулировать устойчивые классы в более мягкий слой абстракции, представляющий структуру более непосредственно, без слоев, геометрий и элементов. Можно упаковать общие запросы. (Например, какие дороги в данной ограниченной области обеспечивают выход? Или сколько тропинок существует между двумя фиксированными точками в слое дороги?) Однако, если не написать полномасштабную географическую систему запросов, тогда, вероятно, гибкость базовой технологии из-за введения таких слоев значительно уменьшится. Предоставляя консультирующим технологам доступ к пространственной БД и обучая их работе с пространственными запросами с помощью фильтров первого и второго порядка, вы гораздо лучше докажете, когда требуется более гибкий стиль

запросов. Это обычно означает демонстрацию базовой технологии и, следовательно, делает проект более привязанным к этой технологии.

Обычно, приложив некоторые усилия, можно расширить информацию, входящую в такие проекты. В отличие от скомпилированных повторно используемых компонентов, базирующиеся на сервере расширенные типы позволяют добавить свои собственные столбцы в таблицы или свои собственные таблицы, которые отображаются в таблицы, поддерживающие тип. В случае с картриджем пространственных данных, например, можно добавить таблицу Геометрия, имеющую ИдентификаторГеометрии в качестве первичного ключа. Стандартные таблицы Элемент и Индекс соединяются с ней с помощью внешних ключей. Можно было бы также создать таблицу История, которая содержит записи об инцидентах в данном месте. Это эквивалент того случая, когда берут стандартную карту и втыкают булавки в те места, где произошел инцидент. Компоненты повторного использования предоставляют карту, а разработчик — булавки.

Проблема в этом случае возникает из-за отсутствия настоящих ОО-средств в ORDBMS. Базовый тип данных подлежит использованию только в качестве черного ящика, т.е. его нельзя поменять — можно только сделать подклассом или добавить к нему дополнительные классы. Поскольку в Oracle8 нет подклассов, значит допустимо только добавлять классы, а исходный тип и его поведения ничего о них не знают. Нужно добавить свои собственные поведения (хранимые процедуры или методы) к новым классам. На самом деле нельзя использовать поведения в расширенном типе — используйте их только по ссылке.

Этот вариант становится более общим в OR-схемах, чем в R-схемах. OR-подход поощряет применение повторно используемых компонентов, несмотря на наличие расширенных типов данных, таких как пространственные и изобразительные типы, предоставляемые Oracle8.

Разнообразие объектов: типы

Основное преимущество универсальных серверов — их способность представлять широкое разнообразие данных. Как принято на рынке программного обеспечения, продукты ORDBMS тоже способны представлять данные самыми разнообразными способами. Настоящий секрет разработки объектно-реляционной БД состоит в извлечении преимуществ правильных OR-структур для решения конкретной задачи в многообразии возможных структур.

Многие инновации в мире ORDBMS сосредотачиваются на типе. В ОО-разработке тип играет ключевую роль. В главе 7 рассмотрены подробности разных аспектов пакетов, классов, типов и интерфейсов, каждый из которых в UML считается классификатором, основной концепцией, являющейся ядром подхода разработки и нотации.

В реляционной БД тип — это один из основных типов данных, ассоциированный со строками, например NUMERIC, FLOAT, VARCHAR или TIMESTAMP. Не существует каких-либо структурированных типов.

ВНИМАНИЕ

Программисты любят считать таблицы "эквивалентом" типов записей на своих языках программирования. К сожалению, SQL как язык программирования не обращается с таблицами или строками как с типами — по крайней мере, этого не делает SQL-92 и не будет делать [Date and Darwen, 1998]. В главе 11 показано, как преобразовывать определения типа и класса в таблицы схем, но на самом деле это не одно и то же. Именно гибкость позволяет ОБЪЕДИНИТЬ ВСЕ во многих таблицах, беспокоясь, например, не о базовых типах данных, а только о структуре столбцов. И именно гибкость в первую очередь критикуется в отношении SQL [Codd, 1990; Date, 1986; Date and Darwen, 1998]. SQL-92 является основой языков программирования БД; SQL3 принимает форму C++ среди таких языков, неважно, к лучшему это или к худшему.

В объектно-реляционных БД ситуация совершенно другая. Стандарт SQL3, например, определяет несколько новых типов данных, в диапазоне от определяемого пользователем (UDT) до определяемого записью:

- *Определяемый пользователем тип данных* — это тип, определяемый с применением оператора CREATE TYPE, который можно использовать с типами ссылок, в генерализации (создании подтипов) отношений, в возвращаемых значениях после применения определяемых пользователем функций, в операциях кастинга (преобразования типов) и, возможно, в более важном случае как основу для определения таблицы: отдельный UDT ссылается на встроенный тип, когда структурированный UDT позволяет определить сложную структуру.
- *Тип строки* — тип, представляющий структуру строки данных.
- *Ссылочный тип* — "указатель" на определяемый пользователем тип, используемый для ссылки на структурированный тип из столбца с одним значением.
- *Тип коллекции* — многозначный тип, в настоящее время ограниченный типом ARRAY, коллекцией элементов одного и того же типа, организованных в последовательность.
- *(CLOB)* — определяемая реализацией область памяти для символов, которая может быть очень большой.
- *(NCLOB)* — CLOB, который хранит мультимбайтные символы, как определяет один из наборов символов SQL3.
- *(BLOB)* — определяемая реализацией область памяти для двоичных данных, которая может быть очень большой.

Имеются две разновидности UDT — отдельный и структурированный. *Отдельный тип* — это UDT, который ссылается на предварительно определенный или встроенный тип как на свой источник. Например, можно определить MONEY UDT, являющийся отдельным типом типа NUMERIC. *Структурированный тип* — это UDT, содержащий определяемый пользователем набор атрибутов данных вместо того, чтобы ссылаться на встроенный тип с единственным значением.

SQL3 UDT включает OO средства инкапсуляции и наследования с помощью определения методов доступа к данным. Например, когда вы ссылаетесь на имя столбца в операторе SQL3 SELECT, то на самом деле вызываете функцию, возвращающую значение столбца, не ссылаясь непосредственно на это значение. UDT также поддерживает создание подтипов, что является основной причиной для этой явной фикции, поскольку позволяет подтипам ссылаться на значения столбцов супертипов через хорошо определенный набор функций на типе. Описанные свойства все еще считаются спорными в среде стандартов SQL, и подробности могут измениться [Melton, 1998].

Поставщики ORDBMS реализуют части этих стандартов, но никто из поставщиков не подходит и близко к предложению полного массива типов SQL3. Динамический сервер Informix — единственный поставщик, предлагающий в настоящее время наследование любого вида: никто из поставщиков не поддерживает полную инкапсуляцию атрибутов таблиц или типов. Большинство имеет сегодня полный комплект LOB типов, поскольку они могут легко создать их и у них нет семантического влияния на их языки БД или методы доступа (индексы, кластеры и т.д.).

Если бы объектно-реляционный мир был бы так же хорошо организован, как мир UML, то проблем создания OR-схем из модели данных UML не существовало бы. К сожалению, не в этом дело. Поставщики ORDBMS и стандартизация SQL3 породили сложный массив конкурирующих структур. Однако есть некоторые общие темы, проходящие через все это: объекты, типы, типы больших объектов, коллекции, ссылки и поведение.

Определенные пользователем и объектные типы

Определенный пользователем тип возник как более теоретический абстрактный тип данных (ADT). Oracle8 изменил имя UDT на "объектный тип", более благозвучное название, которое мы в этой главе с радостью позаимствуем. Переименование фокусирует внимание на вкладе UDT в OO представление. Главное нововведение в OR-системах — отделение понятия типа от понятия таблиц вместо попыток переопределить таблицу как вид типа.

ОСТОРОЖНО

Очень опасно придавать значение таким терминам, как, например, "ADT" или "объектный тип", в присутствии теоретика баз данных [Date and Darwen, 1998]. В результате не избежать трагедии: либо теоретик получит апоплексический удар, либо разработчик взорвется. Относитесь к этим терминам как к рабочим названиям продуктов, их использующих, а не как к теоретической концепции.

В мире SQL3 тип — это описание данных. Он объясняет значение данных, например, то, что символьная строка — последовательность символов, определенных набором символов с порядком, определенным сравнением набора символов, или то, что метка времени с часовым поясом имеет характеристики, определенные стандартами хранения времени. Тип также имеет набор операций (например, арифметические, строковые, операции с датами). И наконец, тип объясняет, какой вид значений можно сравнивать или преобразовывать в какие виды значений. С объектным типом выбор

семантик и операций полностью остается за разработчиком, сравнение же требует, чтобы типы были связаны. Можно сравнить два объекта только в том случае, если у них есть одни и тот же базовый тип (одни и тот же суперкласс на некотором уровне в иерархии класса). Преобразование типов, которое не удовлетворяет этому строгому соглашению о контроле типов, обычно определяется как операторы на объектном типе.

В разделе главы 7 "Ограничения доменов" рассматривается использование классификатора типов UML. Язык универсального моделирования определяет семантику типа данных следующим образом.

Тип данных является типом, значения которого не идентифицированы, т.е. это чистые значения. Типы данных включают примитивные встроенные (такие, как целочисленный или строчный) и определяемые перечислимые (такие, как предопределенный перечислимый тип `boolean`, литералами которого являются значения `false` и `true`) [Rational Software, 1997b, p. 22].

Имеются три подкласса классификатора ТипДанных в метамодели UML: Примитив, Структура и Перечисление. Эти стереотипы можно использовать для квалификации классификатора (прямоугольник класса), с которым работают для представления типа и его операций. Если классификаторы определены с помощью стереотипов "тип", "примитив", "структура" или "перечисление", то эти классификаторы преобразуются в отдельные типы в ODBMS.

Классификаторы класса и интерфейса на диаграмме UML статической структуры соответствуют структурированному типу. В R-системах (глава 11) классы преобразуются в таблицы, а интерфейсы — в хранимые процедуры некоторого вида (если вообще преобразуются). В OR-системах можно преобразовать все классы и интерфейсы в структурированные типы с помощью оператора `CREATE TYPE`.

ВНИМАНИЕ

На самом деле не нужно переводить все классы в типы, а только те, которые вы намереваетесь использовать как типы в операторах SQL. Класс используется как тип, когда он является целью ассоциации (внешний ключ, ссылка на объект или вложенная таблица в ORDBMS) или когда она превращается в предка другого класса, или же когда определяются несколько таблиц, основанные на типе. В большинстве систем это охватывает почти все классы. Можно иметь классы, которые просто используются для просмотра или по какой-либо причине абсолютно изолированы в системе: единичные объекты — пример этого. Однако возникновение сложности от присутствия некоторых таблиц без типа может затруднить разработку, особенно по мере расширения системы.

Для иллюстрации разновидностей объектно-реляционных объектных типов рассмотрим составные части данного понятия, а именно:

- Определяемый пользователем тип `SQL3`.
- Объектный тип `Oracle8`.

- Особый тип DB2 UDB (аналогичен особому типу SQL3 и Informix Dynamic Server).

SQL3 UDT

Определяемый пользователем тип из незаконченного стандарта SQL3 — наиболее общее определение типа объекта. Он также не существует ни в одном из основных продуктов ORDBMS. На рис. 12.6 представлен синтаксис оператора SQL3 CREATE TYPE, который создает структурированный UDT.

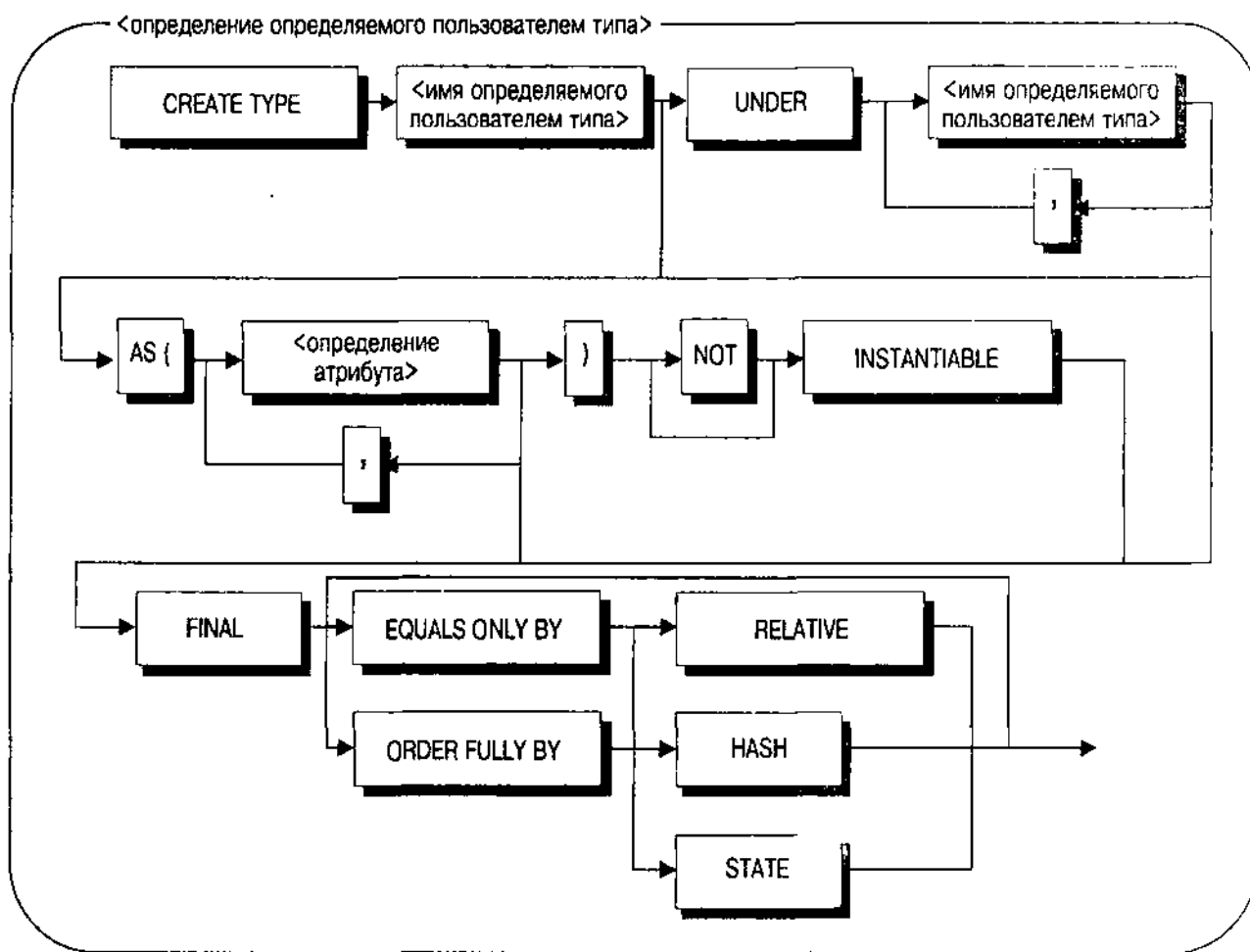


Рис. 12.6. Синтаксис SQL3 CREATE TYPE

ВНИМАНИЕ

Полный синтаксис оператора SQL3 CREATE TYPE содержит несколько дополнительных вариантов, особенно тех, которые позволяют определить особый тип, включая операторы приведения типов.

Предложение UNDER идентифицирует суперклассы данного типа, определяемые в разделенном запятыми списке имен типов. Таким образом, преобразуют генерализации UML в другие классы в имена типов в списке предложения UNDER.

Предложение AS определяет все атрибуты класса. Если нет предложения AS, нужно иметь предложение UNDER, которое определяет атрибуты из суперкласса. Они преобразуются из класса UML непосредственно в определения атрибутов в предложении AS, используя те же самые приемы, которые использовались бы для создания определений столбцов в R-таблице.

ВНИМАНИЕ

То есть это исключение ограничений. Тип определяет атрибуты, но он не определяет их связи с другими типами, а также их первичного ключа или бизнес-правил (ограничений CHECK). Следует применять эти ограничения непосредственно к таблицам, создаваемым с использованием данного типа с оператором CREATE TABLE. Это означает, что в OR-модели нельзя полностью реализовать разработку до тех пор, пока не будут созданы как типы, так и таблицы, реализующие эти типы. К сожалению, синтаксис SQL3 неясен в отношении ограничений, применяемым к таблицам, создаваемым из структурированного типа.

Если класс UML абстрактный, определите NOT INSTANTIABLE. В противном случае — INSTANTIABLE. В БД SQL3 можно создавать строки в таблицах типа только если тип — INSTANTIABLE. Раздел NOT FINAL, требуемый стандартом для структурированных UDT, не имеет никакой семантики в данном стандарте, поэтому не понятно, для чего он существует.

Последнее предложение определяет метод упорядочивания для объектов данного типа. Предложение EQUALS ONLY ограничивает упорядочивающие сравнения сравнением на равенство в то время, как ORDER FULL позволяет определять полный порядок для расширения данного типа. Методы упорядочивания RELATIVE и HASH используют реляционное сравнение (больше, чем или меньше, чем) или хеширование для упорядочивания элементов с помощью так называемых хранимых функций, которые предоставляет разработчик. Вариант STATE действителен только для сравнений EQUALS ONLY: он сравнивает на равенство на основе атрибутов данного типа. Разработчик должен иметь возможность определить эквиваленты в списке операций UML для класса и преобразовать их в хранимые функции, если требуется.

ВНИМАНИЕ

Синтаксис и семантика оператора CREATE TYPE — одни из наиболее спорных проблем, которые все еще должны быть разрешены в процессе стандартизации SQL3 [Melton, 1998]. Здесь, вероятно, можно ожидать изменений.

Объектный тип Oracle8

Объектный тип в Oracle8 находится в центре расширений объектов в реляционной БД Oracle. Типы объектов служат основой вложенных таблиц, позволяют ссылаться на объекты в других таблицах и предоставляют способы интеграции поведения со строками таблицы. Они действительно имеют

два основных ограничения с точки зрения ООП — полное отсутствие любой формы наследования и полное отсутствие инкапсуляции. Эти ограничения значительно затрудняют их использование с точки зрения UML, но они, по крайней мере, предоставляют структурированный тип и объектную идентификацию, которые можно использовать творчески в приложениях Oracle8. На рис. 12.7 представлен синтаксис оператора Oracle8 CREATE TYPE, который создает объектный тип.

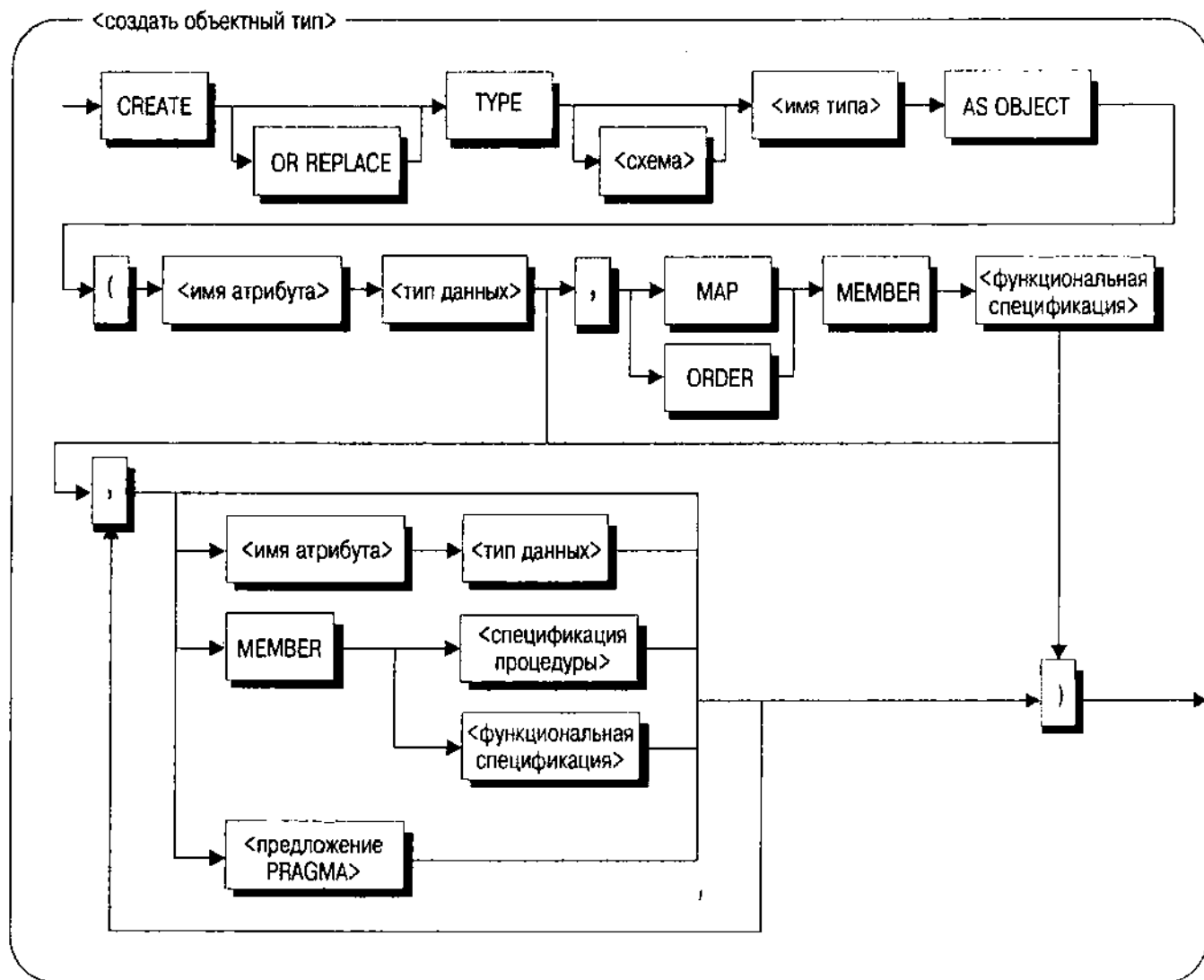


Рис. 12.7. Синтаксис Oracle8 CREATE TYPE

Оператор Oracle8 CREATE TYPE содержит в себе имя типа, но, помимо этого, мало чем похож на оператор SQL3. Этот синтаксис — один из трех вариаций в Oracle8, который также включает создание типа VARRAY (см. далее раздел "Коллекции") и тип вложенных таблиц. Синтаксис на рис. 12.7 создает объектный тип Oracle8, который можно затем использовать для создания таблиц этих объектов. Оператор CREATE TYPE содержит список членов. Можно предложить либо MAP, либо член ORDER для поддержки

индексирования и сортировки. Если ни тот, ни другой не предоставлены, то можно сравнивать только на равенство. Это соответствует синтаксису `ORDER FULL` и `EQUALS ONLY` в SQL3. Необходимо предоставить хотя бы один атрибут данных с типом и, при желании, дополнительные атрибуты данных и типы. Можно также предоставить ряд объявлений подпрограмм-членов (функций или процедур), которые представляют операции объектного типа. Имеется отдельный оператор `CREATE TYPE BODY`, отделяющий реализацию операции от ее объявления в операторе `CREATE TYPE`. Предложение `PRAGMA` позволяет ограничить побочное влияние на БД и состояние пакета PL/SQL в методах, требующие использовать эти методы в операторах SQL.

Отсюда вывод: тип Oracle8 поддерживает простое преобразование нескольких логических структур UML такими способами, которых нет в SQL3. Каждый класс на статической структурной диаграмме UML преобразуется в объектный тип, а каждый атрибут и тип данных — в атрибут объектного типа и тип данных.

Хотя нельзя непосредственно ссылаться на объектный тип для атрибута, можно определить REF для такого типа (см. далее раздел "Ссылки"). Значит, можно определить ассоциации к-одному как атрибуты объектного типа с множественностью 0..1, преобразующейся в тип REF, который допускает пустые значения и 1..1 или просто 1, будучи NOT NULL. Поскольку одной разновидностью объектного типа является VARRAY, можно также определить атрибут с VARRAY из REF на другой объектный тип для управления множественностью ко-многим. Аналогичным образом, можно определить объектный тип для вложенной таблицы. Если имеется ассоциация составных агрегаций, вы способны определить объектный тип вложенной таблицы для класса потомка, затем включить REF в этот объектный тип, чтобы вложить данные в каждую строку таблиц, основанных на объектном типе предка.

ОСТОРОЖНО

По ряду причин автор не рекомендует использовать вложенные таблицы или любые VARRAY-объекты, отличные от REF из другой тип. Во-первых, применение этих средств приводит к проблемам нормализации, о которых уже известно в течение почти 30 лет. Не существует веских оснований для того, чтобы забыть об основах разработки БД и вернуться к временам иерархического управления БД со всей его сложностью. Во-вторых, ОО-разработка сокращает сложность с помощью инкапсуляции. Вложенные таблицы Oracle8 и VARRAY ведут себя с точностью до наоборот, разорвав все структуры и усложнив разработчику задачу новым и более сложным синтаксисом SQL (функция THE, если хотите на нее взглянуть).

Операции отображаются в подпрограммы-члены, и реализации методов операций для подпрограмм-членов воплощены в операторе `CREATE TYPE BODY`. Если операция имеет дескриптор {только чтение} или {запрос}, можно добавить предложение `PRAGMA`, которое ограничивает ссылки, используя уровни чистоты `WNDS`, `WNPS` и `RNPS` (не записывать состояния БД, не записывать состояния пакета и не считывать состояния пакета соответственно). Определение `WNDS` позволяет использовать функцию как функцию SQL.

ВНИМАНИЕ

Реальное значение и использование уровней чистоты Oracle сложно. Если есть желание широко использовать уровни чистоты в схеме, либо с объектными типами, либо с пакетами PL/SQL, следует обратиться к приложению Руководства разработчика Oracle8 [Oracle, 1997с], где рассматриваются причины и следствия ограничений побочных эффектов в PL/SQL.

В качестве примера создания объектного типа и его использования рассмотрим пример Изображения на рис. 12.4. Oracle8 представляет этот класс посредством своего картриджа визуальной информации. Однако, если нужно разработать свои собственные объектные типы, можно создать объектный тип Изображение для облегчения представления и разработки:

```
CREATE OR REPLACE TYPE Image AS OBJECT (
    ImageID NUMBER,
    Height NUMBER,
    Width NUMBER,
    ContentLength NUMBER,
    FileFormat VARCHAR2(64),
    ContentFormat VARCHAR2(64),
    CompressionFormat VARCHAR2(64),
    Content BLOB,
    ImageSignature RAW(2000)
    MEMBER PROCEDURE copyContent (destination IN OUT Image),
    PRAGMA RESTRICT_REFERENCES (copyContent, WNDS),
    MEMBER PROCEDURE setProperties (SELF IN OUT Image),
    PRAGMA RESTRICT_REFERENCES (setProperties, WNDS),
    MEMBER PROCEDURE process (SELF IN OUT Image,
        command IN VARCHAR2),
    PRAGMA RESTRICT_REFERENCES (process, WNDS),
    MEMBER PROCEDURE processCopy (command IN VARCHAR2,
        destination IN OUT BLOB),
    PRAGMA RESTRICT_REFERENCES (processCopy, WNDS)
);
```

Аргумент SELF — явное объявление текущего объекта как первого аргумента каждого метода. Если разработчик не предоставляет его, Oracle8 полагает, что тип параметра — IN (только чтение). Поэтому, например, первая процедура-член copyContent дает доступ на чтение к текущему объекту Изображение, пока вторая процедура setProperties дает доступ к нему как по чтению, так и по записи.

Теперь задайте таблицы изображений, на которые сможете ссылаться из других таблиц:

```
CREATE TABLE CrimeSceneImages OF Image (
    ImageID PRIMARY KEY);
CREATE TYPE ImageArray AS VARYING ARRAY (50) OF REF Image;
```

```
CREATE TABLE CrimeScene (
  SceneID NUMER PRIMARY KEY,
  Description VARCHAR 2(2000) NOT NULL,
  Images ImageArray
);
```

ВНИМАНИЕ

Оператор CREATE TABLE, основанный на типе Изображение, содержит объявление первичного ключа. Можно добавить ограничение для каждого атрибута, определенного в типе, а также табличные ограничения, включающие более одного атрибута.

В зависимости от того, как будет организован доступ к изображениям, может оказаться лучше создать набор ссылок на изображения в отдельной таблице, что позволит производить индексирование и соединение. Изменяющийся массив позволяет только осуществлять навигацию к определенному изображению с помощью программных циклов или (очень витиевато) операторов SQL. Можно также рассмотреть структуру вложенных таблиц с индексами или без них. Здесь вероятно значительное усложнение поддержки и доступа операторов SQL, особенно если решено хранить изображения более чем в одной вложенной таблице.

Этот пример иллюстрирует основное положение в Oracle8 и его объектно-реляционных друзьях: действительно имеется огромный выбор возможностей при использовании этих систем. Реально создавать структуры поразительного разнообразия форм, каждая из которых имеет преимущества и недостатки. Если целевой системой является ORDBMS как репозиторий объектов, нужно рассчитывать на крутой поворот в ходе познания того, что работает, а что нет. Конечно, это сложно из-за отсутствия стандартов.

Легко заметить большую разницу между оператором SQL3 CREATE TYPE и оператором создания объектного типа Oracle8: последний нельзя использовать для создания особого типа — можно только для структурированного. Повторим еще раз, что особый тип — это тип, получаемый из одного из встроенных типов. Затем SQL вводит сильный контроль типов, который гарантирует, что в операторах SQL используются правильные типы данных. Последствия преобразования статической структурной диаграммы UML, состоят в том, что, если целью является Oracle8, нельзя перевести классификаторы "типа" непосредственно в особые типы. Вместо этого нужно использовать таблицу преобразования так же, как в R-системе (см. главу 11). Но особые типы способны на гораздо больше, как показывает их использование в продукте DB2 UDB.

Особый тип UDB

DB2 UDB не предоставляет структурированных типов, но включает концепцию особых типов с синтаксисом, представленном на рис. 12.8.

Таким образом, оператор UDB CREATE TYPE гораздо проще, чем SQL3 или Oracle8. Все, что от него требуется — присвоить имя новому типу, соотносить это имя со встроенным типом-источником и, необязательно, определить включение операторов сравнения по умолчанию (=, <, <=, >, >= и <>).

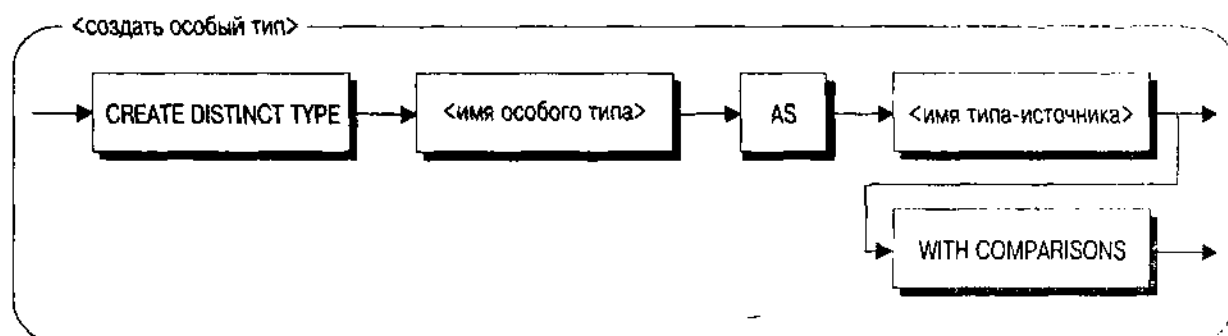


Рис. 12.8. Синтаксис UDB CREATE DISTINCT TYPE

Если WITH COMPARISONS не используется, нельзя сравнивать значения типа.

Основное использование особых типов в UDB – инкапсуляция объектов LOB как отдельных типов [Chamberlin, 1998]. Например, чтобы представить класс Изображение с рис. 12.4 в R-таблице, нужно создать столбец BLOB:

```
CREATE TABLE Image (
  ImageID INTEGER PRIMARY KEY,
  Height INTEGER,
  Width INTEGER,
  ContentLength INTEGER,
  FileFormat VARCHAR(64),
  ContentFormat VARCHAR(64),
  CompressionFormat VARCHAR(64),
  Content BLOB(2M),
  ImageSignature BLOB(2000))
```

Это хорошо, пока работает, но нет способа сказать оператору SQL, что столбец Content содержит изображение или столбец Signature содержит подпись. Для того чтобы сделать это, можно использовать особые типы:

```
CREATE DISTINCT TYPE Image AS BLOB(2M);
CREATE DISTINCT TYPE Signature AS BLOB(2000);
CREATE TABLE Image (
  ImageID INTEGER PRIMARY KEY,
  Height INTEGER,
  Width INTEGER,
  ContentLength INTEGER,
  FileFormat VARCHAR(64),
  ContentFormat VARCHAR(64),
  CompressionFormat VARCHAR(64),
  Content Image,
  ImageSignature Signature)
```

Два особых типа определяют специфические множества объектов BLOB как типы Image (Изображение) и Signature (Подпись), а оператор CREATE TABLE использует затем эти типы для объявления столбцов. Теперь при ссылке на эти столбцы UDB SQL будет обращаться с ними, как со специальными типами, и не позволит присвоить им или вызвать на них функции. Затем можно определить внешние функции на языке программирования, которые принимают аргументы особого типа. Например, определить функцию КопироватьСодержание, Анализировать и Подобный как функции на C и объявить их функциями UDB, принимающими аргументы Image и/или Signature. Затем вызовем их для любого данного Изображения или Подписи с уверенностью, что используются данные правильного типа.

Ассоциации

Создавать ассоциации в OR-системах трудно не из-за отсутствия технологии, а потому, что есть слишком много средств. Можно увязнуть в проверенных и правильных методиках из главы 11 с использованием внешних ключей или пуститься на риск в объектно-реляционном мире объектных типов, ссылок и коллекций. Автор предпочитает использовать OR-средства только там, где другие не помогут, а также непосредственно и просто реализует объектную структуру в разработке UML.

ОСТОРОЖНО

Как всегда, то, что очевидно одному, может быть покрыто мраком для другого: автор выражает мнения, а не факты.

Во-первых, нужно изучить понятия объектного атрибута, ссылки и типов коллекций, чтобы воспользоваться этими новыми способами ассоциации объектов. Можно также все еще иметь внешние ключи, но объявить их в таблицах, а не в объектных типах.

Объектные атрибуты

Объектный атрибут — атрибут объектного типа, который сам по себе является объектом, а не значением со встроенным типом данных, подобным NUMBER или VARCHAR. Если это предложение непонятно, то для автора это не удивительно. Попробуем разобрать по шагам. В R-таблицах есть столбцы, которые объявляются с помощью встроенных типов данных, таких как NUMBER, CHARACTER VARYING или LONG. Значения в столбцах — это действительно значения: у них нет отдельной идентификации вне строки, в какой они находятся. Объекты всегда имеют идентификацию как отдельные, индивидуальные вещи. Когда в OR-системе создается атрибут с определенным пользователем или объектным типом, возникает возможность внедрения объекта в таблицу, каждая строка которой образует еще один объект с идентификацией. На диаграмме UML — это ассоциация с множественностью 1..1 или 0..1 с классом, представляющим внедренный объект.

Например, на рис. 12.1 класс Идентификация ассоциируется с классом Человек с множественностью 1..1: каждая идентификация идентифицирует только одного человека. В Oracle8, например, можно создать объектный

тип для Идентификации, внедряющий единственный объект Человек. Когда вы ищете объект Идентификация, вам также известен объект Человек, идентифицируемый Идентификацией. Но это не очень хороший пример внедренного объекта, поскольку Человек имеет более одной идентификации. Внедренный объект, или объектный атрибут, является ярким примером составной агрегации, при которой объекту принадлежит внедренный объект. Но в случае Идентификации принадлежность — это другой путь: Человеку принадлежит несколько Идентификаций.

Следуя этой логике, нужно использовать объектные атрибуты или внедренные объекты только тогда, когда присутствует ассоциация — составная агрегация с множественностью 1..1 со стороны агрегированного объекта, а не агрегирующего объекта. Это довольно редкий случай в практике, поскольку большинство составных агрегаций являются отношениями мастер-деталь с несколькими деталями для каждого мастера. Он встречается только тогда, когда есть отображение один-к-одному или даже реже один-ко-многим с агрегацией на одной стороне (одна деталь для многих мастеров, когда мастеру принадлежит каждая деталь, и если можно понять, что это означает семантически, — пожалуйста).

Ссылки

Ссылка — это логический указатель на объект, хранимый в каком-либо месте, отличном от текущей таблицы. "Указатель" в этом случае — идентификатор объекта, или OID. ORDBMS создает его для каждого объекта создаваемого структурированного типа. Затем можно ссылаться на объект, поместив OID в значение столбца в таблице, на которую ссылаются. Ссылка — это способ разбить модель сильной принадлежности: значит, атрибут больше не заключает в себе ассоциацию составной агрегации, а содержит простую ассоциацию с единственным объектом. Такой порядок шире применяется на практике, чем внедренные объекты.

Таким образом, в ORDBMS имеется возможность представления ассоциаций либо как внешних ключей, либо как ссылок. Если используются внешние ключи, нужно соединять таблицы в запросах. Если используются ссылки, можно перемещаться к объекту, на который ссылаются, с помощью выражений SQL, избегая соединения. Со ссылками возникают другие проблемы: ограниченная область их действия и висящие ссылки.

Как и стандарт SQL3, Oracle8 предоставляет механизм, определяющий область действия ссылки определенной таблицей или таблицами. Каждый объектный тип может быть источником происхождения нескольких таблиц. Если ссылаются на объект данного типа, то по умолчанию нет ограничения на то, какая таблица может хранить объект, на который ссылаются. Добавление квалификатора области действия к ссылке позволяет определить список таблиц. Когда включают новую строку с помощью ссылки или обновляют ссылку, ссылка должна быть направлена на объект соответствующего типа в одной из таблиц, определенной как область действия ссылки.

Но со ссылками есть одна небольшая проблема. Как предусмотрено в различных продуктах ORDBMS и определено в стандарте SQL3, ссылки не вводят ссылочной целостности. Если объект удаляется, любые ссылки на него

не меняются. Кроме того, нельзя ввести ограничение, что такие операции, как DELETE, завершатся отказом, если присутствуют неразрешенные ссылки на объект. Если используются ссылки, следует инкапсулировать объекты, на которые они ссылаются в пределах ряда операций, поддерживаемых ими. С этой целью задействуйте триггеры на таблицах, на какие указывают ссылки, или схему инкапсуляции хранимых процедур, чтобы запретить непосредственный доступ к таблицам, как показано в главе 11.

Если возникают проблемы целостности ссылок, следует использовать ссылки только для оптимизации производительности. Обычно при работе с ассоциациями устанавливают внешние ключи. Если суммарная производительность таблиц по какой-то причине невелика, рассмотрите вместо этого вариант использования ссылок, но нужно помнить, что затем потребуется дополнительный код для поддержки ссылочной целостности.

ВНИМАНИЕ

Деят. тщательно рассматривает понятия, лежащие в основе ссылок (он называет их "указатели") и решительно отвергает их как несовместимые с реляционной моделью [Date and Darwen, 1998].

Коллекции

До сих пор внимание было сосредоточено на стороне ассоциаций к-одному. А как насчет многих?

Коллекция — это многозначный атрибут некоторого рода, такой как массив, множество или таблица. SQL3 предоставляет ARRAY как единственный вид коллекции. Oracle8 — VARYING ARRAY и вложенные таблицы. Informix — SET, MULTiset и LIST. Цель всех этих типов — собрать несколько объектов в значение одного столбца. Именно то, что нужно для представления *стороны множественности в ассоциации. Не правда ли?

Возьмем, к примеру, ассоциацию Человек-Идентификация на рис. 12.1. Каждый человек имеет свои номера идентификационных документов. Это ассоциация составной агрегации, поэтому нужно иметь возможность сохранять объекты Идентификации как часть объекта Человек, которому они принадлежат, поскольку они не используют объекты совместно.

Как изменяющийся массив в Oracle8, идентификационные документы являются частью столбца индексируемого типа таблицы Человек. И сразу возникает одно ограничение: нужно определить ограничение количества элементов в массиве при создании индексируемого типа. В исходной диаграмме UML множественность — 0..*, означающая, что ограничения нет. Можно заменить *, скажем, числом 20 для адаптации ограничения реализации. Тип будет выглядеть так:

```
CREATE TYPE IdentificationDocuments_t
  AS VARYING ARRAY (20) OF Identification;
CREATE TYPE Person_t AS OBJECT (
  PersonID INTEGER,
  . . . . .
  IDs IdentificationDocuments);
```


Если бы множественность была 1...20, нужно было бы определить атрибут Идентификатора при создании таблицы с ограничением NOT NULL и, возможно, ограничение CHECK в отношении размера массива: он должен быть ≥ 1 . Отметим, что имеется разница между массивом с пустыми значениями и массивом, не содержащим элементов (пустой массив). В первом случае значение столбца абсолютно пусто (null); во втором случае строка имеет память для VARRAY, но в массиве нет элементов. Это различие может немного усложнить программирование.

Для ассоциаций, не являющихся составными, изменяющиеся массивы ссылок — лучший выбор. Например, каждый человек может иметь некое количество адресов (рис. 12.1). Со стороны человека представим это как изменяющийся массив ссылок на объекты Адрес. Повторим: ссылки не вводят ссылочной целостности, поэтому, если дом сгорел дотла или кто-то сел в тюрьму, нужно убрать соответствующие ссылки в прикладном коде или в триггерах.

Можно также заметить в последнем примере, что, хотя коллекция представляет сторону человека в ассоциации, она ничего не делает для стороны адреса. Чтобы позаботиться об этом, нужно создать другую коллекцию (на этот раз как атрибут Адреса), хранящую ссылки на объекты Человек. Прикладной код или триггеры должны гарантировать, что наши две коллекции останутся синхронными.

Имея ограничения ассоциации как коллекции, нужно использовать этот метод только для односторонних ассоциаций (ассоциаций без стрелок, демонстрирующих одностороннюю видимость без поддержки обеих сторон ассоциации). Это делает навигацию немного менее гибкой, но значительно улучшает поддержку кода.

И наконец, рассмотрите третичную ассоциацию многие-ко-многим, такую как ВидыДеятельности на рис. 12.1. Человек исполняет Роль в Криминальной Организации. Можно ли использовать коллекцию для представления этой ассоциации?

Одним словом, нет. Каждый элемент изменяющегося массива должен быть единственными объектом другого типа. Допустимо иметь массив людей в криминальной организации, но нельзя использовать их роли. Допустимо создать тип для ВидовДеятельности, позволяя ему ссылаться на три других вида объектов, затем создать массив для объектов ВидыДеятельности в каждом из трех других типов. Изза повисших ссылок и, возможно, других внутренних проблем Oracle8 (для первого случая) не позволяет создавать первичный ключ на ссылке, поэтому таблицы ссылок не смогут ввести ограничения уникальности на связи ассоциации — основная головная боль.

Итак, можно сделать вывод, что если имеется составная агрегатная ассоциация, можно считать коллекцию ее представлением. Если есть однонаправленная ассоциация ко-многим, в качестве ее представления можно рассматривать массив ссылок. Однако для сохранения ссылочной целостности рекомендуем этот вариант только для повышения производительности, если оно требуется. И наконец, в случаях с другими ассоциациями коллекции лучше не использовать.

Поведение

В реляционных БД операции преобразуются в соответствующие хранимые процедуры или триггеры. Конечно, эти элементы совершенно не стандартны, поэтому нет простого способа представления поведения в R-системах.

Объектно-реляционные БД только слегка разнообразят эту тему, предоставляя некоторые дополнительные способы присоединить поведение к объектам. Нестандартные элементы поведения предлагают разработчику неудовлетворительный способ выбора между отсутствием переносимости и невозможностью получить поведение там, где следует, на сервере БД. При переходе к ORDBMS приходится пользоваться нестандартными решениями в отношении поведения. Например, Oracle8 добавляет методы к объектному типу, а стандарт SQL3 — нет. Он в действительности определяет операции на структурированном типе в терминах функций, генерируемых системой, таких как получить или задать (обозреватели и мутаторы на языке стандарта), соответствующие атрибутам данного типа, конструкторам и функциям приведения типов.

ВНИМАНИЕ

Для ознакомления с другой логической моделью в этой области обратитесь к книге Дейта [Date and Darwen, 1998, pp. 104-122]. Модель Дейта помещает конструкторы, обозреватели и мутаторы в совершенно другое положение относительно базовых типов. Конструктор, который Дейт называет "селектором", выбирает имеющиеся данные, а не занимается выделением памяти. Дейт определяет операторы THE, которые действуют как псевдопеременные для обеспечения как выборки, так и обновления возможных представлений вложенных Дейтом типов. Можно использовать вложенные запросы операторов для навигации по сложным представлениям вместо, например, уточняющей нотации через точку на именах атрибутов. Все эти операторы имеют глобальную область применения, не будучи связаны непосредственно с типом. У такого подхода есть некоторые преимущества, как логические, так и практические, но нужно целиком заимствовать RM-модель для их реализации, а автор сомневается в том, что основные поставщики сделают это в ближайшем будущем.

Если используется Oracle8, можно создать все операции на классах UML как методы на объектном типе. Ранее упоминалось, что каждая операция имеет неявный параметр только для чтения SELF, который можно использовать как ссылку на текущий объект. Если нужно изменить текущий объект, объявите SELF как IN OUT в списке параметров метода. Следует создавать такие методы только для операций, которые имеют смысл как операции на сервере БД. Операции, работающие на объектах в оперативной памяти, не годятся для преобразования на сторону сервера.

Кто диктует правила?

Что касается бизнес-правил, то имеется очень небольшая разница между R- и OR-системами. Используются те же самые ограничения, те же самые условия и тот же вид кода процедур для введения правил.

Единственное отличие — объектный или структурированный тип вместо таблиц, создаваемых с помощью таких типов. Типы не имеют ограничений: они есть только у таблиц. Следовательно, если создается несколько таблиц данного типа, нужно каждую из них ограничивать отдельно: иметь отдельные объявления внешних ключей, отдельные условия проверки и т.д.

ВНИМАНИЕ

Логика Применения ограничений к данным в SQL3 несколько запутанна. Как подчеркивает Дейт, в нем нет способа добавить ограничение типа — ограничение, применяемое к определению типа, а не к базовым отношениям или значениям, использующим его [Date and Darwen, 1998, pp. 159-168, 389-390]. Это обрекает программистов, применяющих SQL3, на многолетнюю поддержку многочисленных ограничений CHECK и таблиц для просмотра значений, которые должны исчезнуть в системе с хорошим контролем типов.

Язык войны

Было бы неплохо иметь уже готовый несложный способ создания эффективной OR-схемы. До появления стандарта он вряд ли появится и, как упоминалось выше, этот стандарт имеет серьезные проблемы [Melton, 1998]. Хуже того, они сосредоточены в таких частях стандарта, которых в основном касается эта книга, — в свойствах объектов.

Тем не менее этот раздел пытается обобщить наилучшие способы создания OR-схемы, фокусируя внимание не на относительно неприменимом на практике стандарте SQL3, а на очень широко используемом на практике наборе средств Oracle8. В структуре и языке этого раздела отражены структура и язык раздела "Язык мира" главы 11. Итак, здесь мы рассмотрим R-методы преобразования схемы, модифицированные OR-возможностями, которые обсуждались в предыдущих разделах данной главы. Обратитесь к главе 11 за примерами и подробностями по R-преобразованиям: этот раздел фокусирует внимание на ОО дополнениях к процессу преобразования. Процесс преобразования использует модель данных из примера, представленного на рис. 12.1-12.5. Вы увидите OR-синтаксис Oracle8, а не SQL3 потому, что в данный момент не существует продукта, реализующего стандарт SQL3.

Точно так же, как и в случае с R-схемой, OR-схему создают в два основных этапа — создание типов и таблиц из устойчивых типов и создание таблиц для ассоциаций многие-ко-многим и третичных ассоциаций. Этот раздел определяет основное направление полного преобразования, связывая все ранее рассмотренное в единое целое. Соответствие в данном случае будет очевидно для читателя. Можно обдумать требования SQL-92, с которым совместим SQL3, при решении вопроса о том, какие OR-расширения использовать с наибольшей выгодой. Соответствие этим свойствам невозможны в данный момент, но следует уделить некоторое внимание выходящему стандарту SQL3 и его способу интерпретации этих свойств.

Устойчивые классы

Перед началом преобразования у вас уже будут идентифицированные классы, соответствующие OR-расширениям в целевой ORDBMS. Рис. 12.4 и 12.5, на которых соответственно демонстрируются подсистемы Изображение и География для системы каталога, непосредственно относятся к картриджам Oracle8, расширяющим возможности Oracle8 RDBMS. Прежде всего нужно создать объектные типы, таблицы и другие элементы, требуемые для того, чтобы расширение могло работать.

ВНИМАНИЕ

Повторим еще раз, что автор использует пример Oracle8 потому, что он в основном знаком с этой системой. Типы таблиц Informix DataBlades и DB2 UDB с их внешними C-программами предоставляют подобные возможности и имеют подобные проблемы с разработкой и повторным использованием. Автор не хотел бы увидеть преподавателей, использующих любую из этих технологий в качестве примера разработки современных систем программного обеспечения, чтобы объединить теоретиков в области вычислительной техники с инженерами по программному обеспечению.

Что касается класса Изображение, основной тип изображений уже существует в Oracle8 как объектный тип ORDSYS.ORDVIRB. Все, что нужно сделать в этом случае — использовать этот тип как атрибутный в классах, инкапсулирующих данные изображений. Здесь тип больше является интерфейсом, хотя действительно включает атрибуты данных, представляющие изображение. Вместо того чтобы работать с типом ORDSYS.ORDVIRB как с корневым классом в иерархии классов изображений, возможно, лучше создать новый класс Изображение с внутренним атрибутом типа ORDSYS.ORDVIRB:

```
CREATE TYPE Image_t AS OBJECT (  
    ImageData ORDSYS.ORDVIRB;  
);
```

Можно добавить методы, чтобы реализовать поведение на классе Изображение на рис. 12.4. Как альтернативу можно разработать интерфейс для объектного типа Изображение, что более полезно для практических нужд разработчика. Затем реализовать операции с базовыми методами ORDSYS.ORDVIRB.

Класс Геометрия более сложный, что затрудняет разработку. Oracle8 предоставляет пару сценариев SQL, выполняемых с помощью SQL*Plus или другого интерпретатора команд SQL. Сценарий crlayer.sql создает таблицы разных слоев для какого-то определенного слоя, присваивая таблицам имена с его именем. Эти таблицы будут соответствовать подклассам базовых типов на рис. 12.5. Из-за того что в Oracle8 отсутствует автоматическое обновление суперклассов, вероятно, не очень продуктивно создавать таблицы для абстрактных суперклассов на рис. 12.5. Таблицы подклассов, создаваемые crlayer.sql, не имеют возможностей первичных и внешних ключей, сильно затрудняя их интеграцию в ОО разработку. Пример с Геометрией — случай унаследованной системы, которую хотят

использовать. Как для всех унаследованных систем, имеется выбор расширения унаследованной разработки или ее инкапсуляции в собственные классы разработчика. В этом случае, если выбрана инкапсуляция, следует добавить внешний слой, представляющий интерфейс Геометрии, который соответствует требованиям разработчика. Затем этот интерфейс реализуется с помощью таблиц пространственных картриджей предпочтительно таким образом, какой делает их совершенно невидимыми для прикладных программистов, осуществляющих разработку в созданной схеме.

В качестве первого преобразования создайте объектный или структурированный тип для каждого класса в модели данных UML с "устойчивым" стереотипом, включающим классы ассоциаций и классы, которые наследуют от других классов. Присвойте имя типу, используя имя класса, добавив суффикс "_t", указывающий, что объект — это тип. Теперь можно создавать таблицу, основанную на типе с именем класса, поскольку большинство продуктов ORDBMS помещают типы и таблицы в одно и то же пространство имен, требуя, таким образом, уникальных имен. На базе подсистемы Человек, рис. 12.1, создаются 10 объектных типов: Person_t, Address_t, Identification_t, Expiring_ID_t, Driver_License_t, Passport_t, National_ID_t, Law_Enforcement_ID_t, Social_Security_Card_t и Birth_Certificate_t.

ВНИМАНИЕ

Можно объединить класс ассоциаций с другим типом позже, если удастся. А пока оставьте его как отдельный тип в создаваемой схеме.

Теперь внутри каждого типа добавьте объектный атрибут для каждого атрибута в атрибутном разделе класса. В качестве примера для Address_t добавьте НомерДома, НомерКвартиры, НазваниеУлицы, СуффиксУлицы, ОпределительМестоположения, Город, ПочтовыйИндекс, Страну и Комментарий.

На этом этапе следует иметь отображение преобразования, показывающее, как преобразовывать каждый тип данных в модели данных в тип данных SQL3. Эти типы данных состоят из встроенных типов, включая BLOB, CLOB и т.д.; особых типов, получаемых из этих встроенных типов; и определяемых пользователем типов или объектных типов, включая ссылки на эти типы.

Теперь мы подошли к первой OR-возможности выбора, отличной от использования типа вместо создания таблицы для каждого класса. Если идентификация объекта неявная, то теперь нужно решить, использовать ли идентификацию встроенного объекта ORDBMS для представления идентификации или создать атрибут первичного ключа, как в R-схеме. Атрибут первичного ключа позволяет определять ограничения ссылочной целостности. Если хотите иметь внешние ключи, указывающие на данный тип, нужен атрибут первичного ключа. Если вместо внешних ключей намерены использовать объектные ссылки, без него можно обойтись. Конечно, сделав это, вы лишитесь также возможности соединяться с таблицей на основе совпадения с первичным ключом, что является критичным R-свойством. Для равновесия, как полагает автор, следует всегда добавлять атрибут первичного ключа для сохранения гибкости в разработке схемы и облегчения адаптации реляционной ссылочной целостности, где это рекомендовано.

Присвойте имя столбцу первичного ключа, используя имя класса с суффиксом ID, как, например, AddressID. Обычно присваивают столбцу тип данных NUMBER и используют последовательность или эквивалент для генерации чисел для столбца.

Конечный результат этого этапа для типа Address_t выглядит так:

```
CREATE TYPE Address_t AS OBJECT (  
    AddressID INTEGER,  
    StreetNumber NUMBER,  
    StreetFraction VARCHAR(5),  
    StreetName VARCHAR(100),  
    StreetSuffix VARCHAR(25),  
    City VARCHAR(100),  
    State CHAR(2),  
    PostalCode CHAR(20),  
    Country VARCHAR(100),  
    Comment VARCHAR(250));
```

ВНИМАНИЕ

Ограничений на атрибуты не существует. В ORDBMS ограничивают таблицы, а не типы. В SQL3 можно иметь раздел DEFAULT, соответствующий начальному значению UML, и все. В Oracle8 в основном все точно так же. Подробнее см. раздел "Кто диктует правила?" в настоящей главе.

У нас есть набор типов, относящихся только к использованию типов в объявлении атрибутов, если таковые имеются. Теперь нужно создать таблицы на основе этих типов и добавить ограничения. Для каждого типа нужна, по крайней мере, одна таблица. Если есть основание для разделения объектов данного типа на две или более коллекции, создайте дополнительные таблицы с другими именами. По мнению автора, это больше относится к разработке физической схемы, чем логической.

Если имеется дескриптор {с возможным пустым значением} на атрибуте, добавьте ограничение NULL, в противном случае — NOT NULL. В каталоге модель данных использует типы с пустым значением для представления ограничений NULL и все столбцы в Address_t, кроме StreetFraction и Comment, для NOT NULL:

```
CREATE TYPE Address OF Address_t (  
    AddressID PRIMARY KEY, — OID type  
    StreetNumber NOT NULL,  
    StreetFraction NOT NULL,  
    StreetName NOT NULL,  
    StreetSuffix NOT NULL,  
    City NOT NULL,  
    State NOT NULL,  
    PostalCode NOT NULL,  
    Country NOT NULL;
```

Все-таки создайте служебную таблицу проверки типов для перечислимых типов в ORDBMS, поскольку система преобразования типов все еще не имеет этого вида типов (см. предыдущий раздел "Кто диктует правила?" этой главы).

Теперь пора создать ограничение первичного ключа и все, что от него зависит. Для начала найдите корневой класс каждой иерархии генерализации (классы, на которые указывают стрелки генерализации, но из которых никакие стрелки генерализации не выходят). Кроме того, рассмотрите все классы без отношений генерализации.

Если класс имеет неявную объектную идентификацию, следует определиться, надо включать атрибут первичного ключа. Если решите его добавить, добавьте PRIMARY KEY как ограничение к этому столбцу в операторе CREATE TABLE. PRIMARY KEY подразумевает NOT NULL, поэтому не надо вводить это ограничение на столбец первичного ключа.

ВНИМАНИЕ

Если класс имеет составную агрегированную ассоциацию с другим классом, а другой класс агрегирующий, воздержитесь от создания первичного ключа до тех пор, пока не будут созданы внешние ключи. Составные агрегированные ключи — это комбинация первичного ключа связанного с ним класса и другого атрибута. Также можно решить представить составную агрегированную ассоциацию с помощью изменяющегося массива или другой коллекции вместо внешнего ключа.

Если класс имеет явную объектную идентификацию (дескриптор {OID}), добавьте предложение ограничения PRIMARY KEY к таблицам, созданным из типа, и поместите все столбцы с дескриптором {OID} в это ограничение. Если в явном идентификаторе только один столбец, просто добавьте к нему ограничение столбца PRIMARY KEY.

Если у класса есть подклассы, добавьте атрибут к типу для представления дискриминатора. Создайте ограничение CHECK в таблицах, создаваемых из типа с соответствующими перечислимыми значениями для представления подклассов. Если ORDBMS поддерживает наследование (Informix Universal Server, например, наследует либо через свои иерархии ROW TYPE, либо через таблицы наследования для таблиц, основанных на этих типах строк, а SQL3 — непосредственно с помощью раздела UNDER в CREATE TYPE), используйте подходящий синтаксис для связи типов. Иначе потребуется применить один и тот же метод первичного ключа, который рекомендован в главе 11.

ВНИМАНИЕ

Автор согласен с Дейтом в том, что наследование — это свойство типа, а не таблицы [Date and Darwen, 1998, pp. 373-378]. Следует избегать использования свойств наследования таблиц Informix, поскольку они не внесут ничего, кроме путаницы, в отношения модель-схема.

В SQL3, например, можно создать класс *ПрипуждениеПоЗакону* как тип, добавив атрибут *НомерУдостоверения* к атрибутам и поведение абстрактного класса *Идентификация*:

```
CREATE TYPE LawEnforcementID_t UNDER Identification_t (
    BadgeNumber INTEGER);
```

Если присутствуют составные суперклассы (множественное наследования), нужно вернуться к схемам главы 11, где они рассматривались. SQL3 позволяет определять составные суперклассы в разделе UNDER, но семантика стандарта очень слаба и не разрешает даже основных проблем, таких как дублирование имен атрибутов в различных суперклассах. Это должно либо предотвращать определение двусмысленного подкласса, либо включать некий сложный механизм разрешения неопределенности, реализованный в таких языках, как C++ [Stonebraker and Brown, 1999]. Лучше всего избегать множественного наследования, если можно. Отсутствие его поддержки в выбранной DBMS определит ваш выбор.

Классы ассоциаций представляют ассоциации с атрибутами, ассоциации многие-многим и n-арные ассоциации, связывающие два или более классов вместе. Класс ассоциаций преобразуется в таблицу, содержащую первичные ключи всех относящихся к ней классов. Можно создать объектный тип, содержащий ключ и другие атрибуты, затем определить таблицу, основанную на нем, хотя это не дает большого выигрыша. Сравните объектный тип Plays_t с таблицей Plays (ВидыДеятельности) в главе 11:

```
CREATE TYPE Plays_t AS OBJECT (
    PersonID INTEGER,
    OrganizationID INTEGER,
    RoleID INTEGER,
    Tenure INTEGER, - Oracle8 не имеет интервальных типов
    StartDate DATE,
    EndDate DATE,
    TerminationMethod CHAR(1));
CREATE TYPE Plays OF Plays_t (
    PRIMARY KEY (PersonID, OrganizationID, RoleID),
    PersonID REFERENCES Person,
    OrganizationID REFERENCES Organization,
    RoleID REFERENCES Role,
    Tenure NOT NULL,
    StartDate NOT NULL);
```

- Вот реляционная таблица для сравнения

```
CREATE TABLE Plays (
    PersonID INTEGER REFERENCES Person,
    OrganizationID INTEGER REFERENCES Organization,
    RoleID INTEGER REFERENCES Role,
    Tenure INTEGER NOT NULL, - Oracle8 не имеет интервальных типов
    StartDate DATE NOT NULL,
    EndDate DATE,
    TerminationMethod CHAR(1),
    PRIMARY KEY (PersonID, OrganizationID, RoleID));
```


Если выбран подход на основе типов, разработчик получает возможность создать более одной таблицы для хранения связанных объектов, которые могут представлять или не представлять ценности. Большинство систем рассматривают эти ассоциации как уникальные: если первичный ключ разделен между таблицами, то он не сможет ввести такую уникальность. Следует выбрать подход типов, только если действительно необходимо иметь две или более таблиц связей с одной и той же структурой и одними и теми же связанными классами.

Далее найдите любые потенциальные ключи. Если имеются любые дескрипторы {альтернативный OID=<n>}, добавьте ограничение UNIQUE к таблицам, созданным из типов для каждого уникального идентификатора <n>, и поместите все столбцы с одинаковыми идентификаторами <n> в ограничение. Повторим, что LawEnforcementID служит примером, поскольку дескриптор BadgeNumber имеет дескриптор {альтернативный OID=1}:

```
CREATE TABLE LawEnforcementID OF LawEnforcementID_t (  
    BadgeNumber NOT NULL UNIQUE);
```

Следующий шаг: добавьте любое простое табличное ограничение к таблицам, созданным из типов. Если на диаграмме есть прямоугольники ограничений, преобразуйте их в раздел CHECK, если можно выразить ограничение в выражении SQL, содержащем только ссылки на столбцы в определяемой таблице. Если есть такие ограничения типов данных, как перечисления или диапазоны, добавьте раздел CHECK к столбцу для представления ограничения. Все эти ограничения находятся в таблицах, а не в типах. Для перечислений с использованием служебных таблиц, которые имеют список возможных значений в типе, подождите создания ассоциаций на следующем шаге.

В этом месте все таблицы должны иметь первичный ключ. Найдите двоичные ассоциации класса, определяемого с помощью множественности 0..1 или 1..1 на роли, присоединенной к другому классу. Если ассоциация имеет класс ассоциации, проигнорируйте пока эту ассоциацию. Для представления направления ассоциации кодному либо создается атрибут для представления ассоциации как внешний ключ, либо создается ссылка на другой класс. Если создается внешний ключ, нужно создать соответствующие ограничения (REFERENCES или FOREIGN KEY) в таблицах, основанных на типе. Если ссылка — добавить код для поддержки ссылки в случае уничтожения объекта или других проблем ссылочной целостности.

По мере прохождения по таблицам разработчик, конечно, достигнет другого конца каждой двоичной ассоциации. Если это ассоциация 1..1 или 0..1 и уже создан атрибут для нее в другой таблице, не создавайте внешний ключ или ссылку в обоих типах. Присутствие двух внешних ключей для отношения не только порождает заикливание и трудно поддерживается, но и способно совсем запутать разработчиков, использующих типы. Можно оптимизировать принятое решение, подумав, как приложения будут использовать таблицы, основанные на данном типе. Если одна сторона ассоциации кажется наиболее естественной или подходящей для использования, поместите атрибут в этот тип. Это соответствует ассоциации со стрелкой,

указывающей на другой тип. Если множественность на роль 1..1, а не 0..1, добавьте ограничение NOT NULL к столбцу/столбцам внешних ключей в данной таблице.

Уже созданы все внешние ключи к суперклассам через отношения генерализации на предыдущем шаге создания первичных ключей.

Для составной агрегации только с одним агрегированным объектом нужно рассмотреть внедрение объекта в тип как альтернативу создания внешних ключей или ссылок. Если составной объект агрегирует несколько объектов, можно представить это как коллекцию объектов-потомков. Если был выбран вариант, не связывающий потомков слишком близко с типом родителя, добавьте атрибуты первичных ключей к объектному типу потомков. Затем создайте второй атрибут для первичного ключа, уникально определяющий данный объект. Чаще это либо явный столбец {OID}, либо последовательность, если ассоциация {упорядоченная}. Если нет, тогда создайте целое значение, чтобы уникальным образом определить потомков агрегата. Сделайте атрибуты из таблицы предка внешним ключом к таблице предка в табличных ограничениях на таблицы, основанные на типе потомков. Добавьте также соответствующее ограничение CASCADE к разделу FOREIGN KEY в таблицах.

Теперь оптимизируйте классы ассоциаций один-к-одному. Найдите все двоичные ассоциации с множественностью ролей больше 1 на роль, исполняемую определяемым классом. Если ассоциация имеет класс ассоциации, добавьте атрибуты из класса ассоциации к типу вместо того чтобы создавать отдельный тип для класса ассоциаций.

Создайте ассоциации многие-ко-многим и третичные ассоциации, используя внешние ключи как при R-преобразовании. OR-коллекции и ссылочные методики неприменимы к этим видам ассоциаций.

Операции

Простейший способ представления операций в Oracle8 – создать процедуру или функцию-член для каждой операции, определенной в модели данных. В качестве примера рассмотрим класс КриминальнаяОрганизация на рис. 12.2. Он экспортирует две операции: ОбновитьСтатус и УстановитьПриоритет – "мутаторы", изменяющие состояние определенного объекта КриминальнаяОрганизация. В этом случае требуются перечислимые типы данных и обновление текущего объекта.

Первое требование, перечислимые типы данных, все еще трудно удовлетворить в PL/SQL. Система типа PL/SQL не имеет понятия перечислимого типа. По крайней мере определение столбца SQL может иметь ограничение CHECK: это неверно, к сожалению, для определения переменных PL/SQL. Нужно создать такую же таблицу поиска, которая была составлена для R-системы (см. главу 11). Затем код в процедуре-члене ОбновитьСтатус находит соответствующее значение для использования в обновляющемся объекте.

Таблица 12.1. Преобразование ОР-схемы

Шаг	Преобразование
1	Создайте все типы, таблицы или вспомогательные объекты, которые необходимы для того, чтобы воспользоваться расширениями повторного использования ORDBMS, такими как картриджи Oracle8, Informix DataBlades или расширения DB2 UDB.
2	Класс UML становится объектным (структурированным) типом обычно с помощью соответствующей таблицы этого типа, хранящей строки объектов, но и, возможно, с помощью более чем одной такой таблицы.
3	Тип UML становится особым типом, если он основан на встроенном типе, или объектным типом, если у него есть атрибуты.
4	Атрибут UML в классе становится атрибутом в объектном типе.
5	Тип атрибута UML в классе становится типом атрибута в объектном типе с помощью таблицы преобразования типов и/или объектных и особых типов.
6	Если присутствует UML-дескриптор {с возможным пустым значением}, атрибут имеет ограничение NULL; в противном случае — ограничение NOT NULL.
7	Если атрибут UML имеет инициализатор, добавьте к столбцу раздел DEFAULT.
8	Для классов без генерализации (корневых или независимых) и с неявной идентификацией создайте целочисленный первичный ключ; для {OID} добавьте столбцы с дескриптором {OID} к ограничению PRIMARY KEY; проигнорируйте классы составных агрегаций и ассоциаций.
9	Для подклассов добавьте ключ каждого класса родителя к ограничениям PRIMARY KEY и FOREIGN KEY; для продуктов ORDBMS, полностью совместимых с SQL3, можно вместо этого использовать раздел UNDER для представления отношения, но для всех остальных следует поместить ограничение внешних ключей в таблицы, определяемые с помощью объектных типов.
10	Для классов ассоциаций создайте объектный тип и добавьте первичный ключ из каждой таблицы исполнения ролей к ограничениям PRIMARY KEY и FOREIGN KEY.
11	Если имеется дескриптор {альтернативный OID= <n>}, добавьте столбцы к ограничению UNIQUE.
12	Добавьте CHECK для каждого явного ограничения.
13	Создайте столбец FOREIGN KEY в таблице ссылок для каждой роли 0..1, 1..1 в ассоциации либо используйте ссылку на объектный тип для объявления объекта как атрибута со своими собственными правами для одиночных объектов или коллекции, подобных массиву ссылок для многократно связанных объектов.
14	Создайте PRIMARY KEY для составной агрегации с FOREIGN KEY на агрегирующую таблицу (с вариантом CASCADE), добавьте дополнительный столбец для PRIMARY KEY; как альтернативу используйте объектный тип для хранения агрегата в самой таблице либо через объектный атрибут (для одиночного объекта), либо коллекцию типа массива или вложенной таблицы (не рекомендуется без проведения интенсивных опытов для определения целесообразности).
15	Оптимизируйте классы двоичных ассоциаций с помощью перемещения на сторону таблицы ко-многим, где это возможно.

Таблица 12.1. Преобразование ОР-схемы (продолжение)

Шаг	Преобразование
16	Создайте таблицы для многих-ко-многим, третичных ассоциаций без ассоциированных классов, используя внешние ключи.
17	Создайте ограничения PRIMARY KEY, FOREIGN KEY из ключей таблиц исполняемых ролей во многих-ко-многим и третичных ассоциациях.
18	Создайте методы на объектных типах для операций на соответствующих классах UML; используйте соответствующие форматы "уровней чистоты" для операций с дескрипторами {только для чтения} или {запрос}.

Итоги

Для удобства преобразование объектно-реляционной БД резюмировано в таблице 12.1. FOREIGN KEY и PRIMARY KEY означают либо табличное ограничение, либо ограничение столбца REFERENCES, в зависимости от того, является ли ключ многозначным. Эта таблица очень напоминает таблицу 11.4, описывающую R-преобразование.

Таким образом, объектно-реляционные БД предлагают широкий спектр альтернативных представлений для разработки моделей данных, возможно даже слишком широкий. Следующая глава переводит вас в полностью объектно-ориентированный мир, в котором меньше возможностей выбора, но они лучше накладываются на концепцию UML.

Глава 13

Разработка схемы объектно-ориентированной базы данных

Работа без надежды протекает, как вода сквозь сито, а надежда без объекта не может существовать.

Сэмюэл Тейлор Колзфридж, "Работа без надежды"

Последняя остановка в этом путешествии по миру разработки баз данных — ОО-схема. Модель данных UML преобразуется в такую схему легко, хотя, как всегда, есть несколько проблем, которые стоит рассмотреть. Одни возникают из-за того, что поставщики OODBMS не придерживаются никакого стандарта в определении схем. Другие связаны с наследственной природой разработки устойчивых объектов, появляющихся во всех продуктах OODBMS и также в стандарте ODMG.

Процесс преобразования для продуктов OODMBS

В этой главе рассматривается язык определения объектов (ODL) ODMG 2.0 для ОО-систем [Cattell and Barry, 1997]. Приводятся примеры выбора схем в менеджере БД POET с использованием предкомпилятора POET ODL и привязки к языку C++ [POET, 1997a, 1997b].

Стандарт ODMG 2.0 — это всеобъемлющий испытательный полигон возможностей OODBMS. К сожалению, специалисты OODBMS еще не представили средств определения схем, согласующихся с данным стандартом. Одни (POET и Versant) предлагают вариации стандарта, другие (ObjectDesign и Objectivity) — совершенно иной способ определения своих схем.

Автор использует ODL, потому что он предполагает обобщенный набор средств, которые хорошо отображаются в ПО различных поставщиков. А POET — потому что он предоставляет полнофункциональное, но не законченное средство определения схем ODL, основанное на ODMG 1.5 ODL. ODL, в свою очередь, основан на OMG — языке определения интерфейсов (IDL) CORBA, языке для определения интерфейсов распределенных объектов для использования с системами распределенных объектов CORBA.

ВНИМАНИЕ

Интересно, что POET адаптировал интерфейс ODL дополнительно к привязкам языков C++ и Java для поддержки независимых от языка интерфейсов для использования вместе с технологией распределенных объектов, подобной COM и Visual Basic. Независимость ODL от языка, таким образом, придает ценность миру распределенных объектов, иллюстрируя один из софизмов о логическом обосновании "импедансного несоответствия" для объектных БД.

Почти все продукты OODBMS вначале использовали языки ООП в качестве своих языков определения схем. Большинство отдавало предпочтение C++, но, по крайней мере, один (Gemstone) работал на Smalltalk. После добавления элементов языка или библиотек классов для поддержки устойчивости эти продукты превратили интерфейсные определения C++ в объявления схем. Большинство также добавило поддержку схем Java. Примеры этой главы используют привязку языка C++ системы POET [POET, 1997a] для иллюстрации специфического языкового подхода к определению схем.

В итоге процесс определения ОО-схемы БД с применением UML приходит к тому же типу преобразования, который делается между разработкой UML и ODL или реализацией этой разработки на C++. Это простая часть преобразования, отображающая из ОО-концепции UML в концепции C++ или других языков ООП.

Сложности начинают возникать, когда язык поставщика OODBMS входит в игру через расширения языков ODL или ООП, которые позволяют использовать устойчивые объекты. Каждый язык имеет различные средства и различные подходы к моделированию ОО-концепций. Никакая OODBMS не предоставляет непосредственной поддержки для ассоциаций многие-ко-многим или n-арных ассоциаций, например, а также не предлагает классов ассоциаций, поэтому необходимо немного преобразовать эти концепции UML. Кроме того, большинство языков ООП не предусматривают никакого способа представления пустых значений, поэтому представление атрибутов с возможными пустыми значениями становится сложной задачей.

Чтобы нагляднее проиллюстрировать процесс, в этой главе используется та же разработка, что и в главе 11. На рис. 13.1 показана модель UML для подсистемы Человек из каталога, а на рис. 13.2 — модель UML для подсистемы Организация. Обе они являются частью третьей — подсистемы Сущность. На рис. 13.3 представлена архитектура данного пакета.

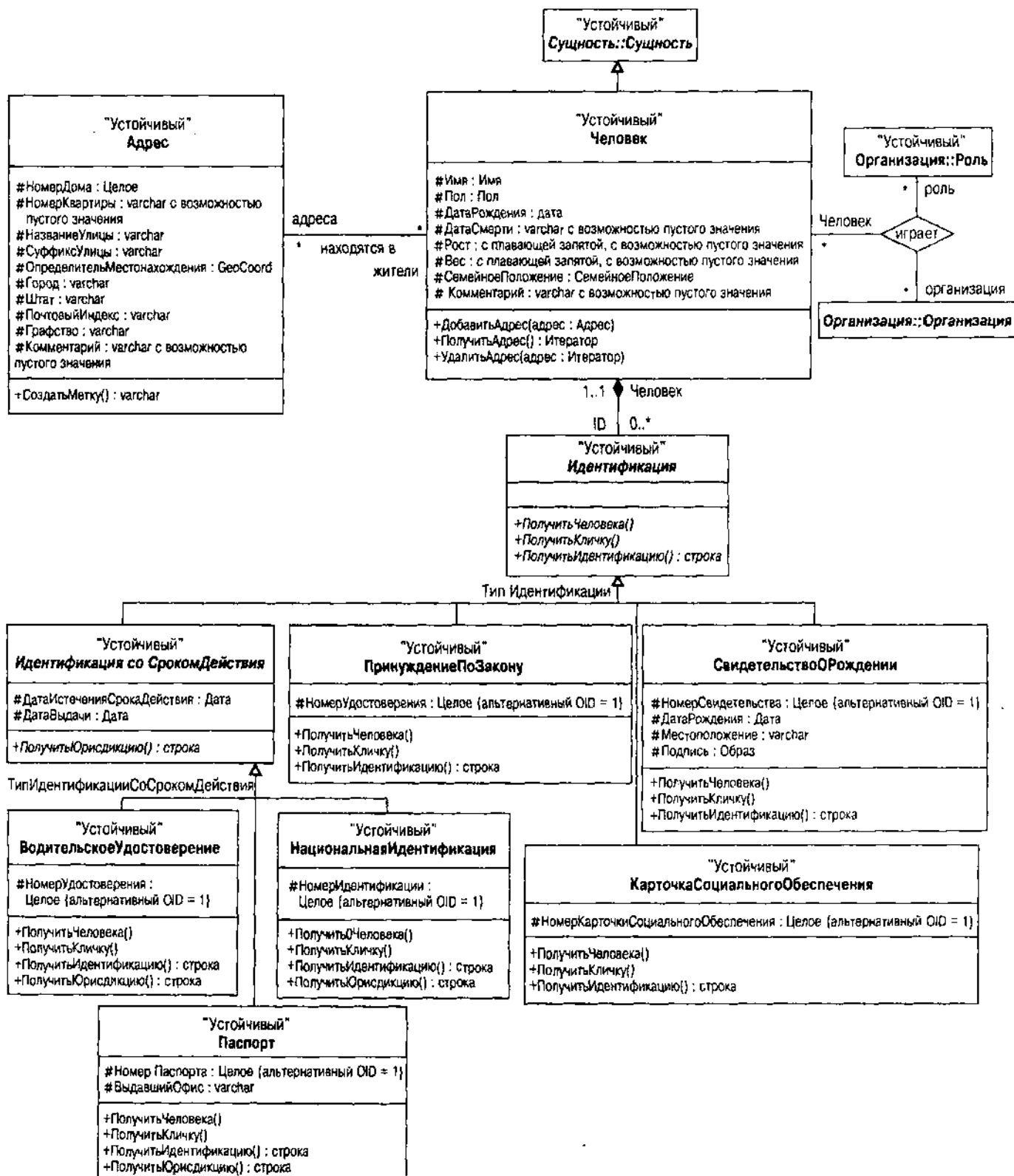


Рис. 13.1. Подсистема Человек на UML

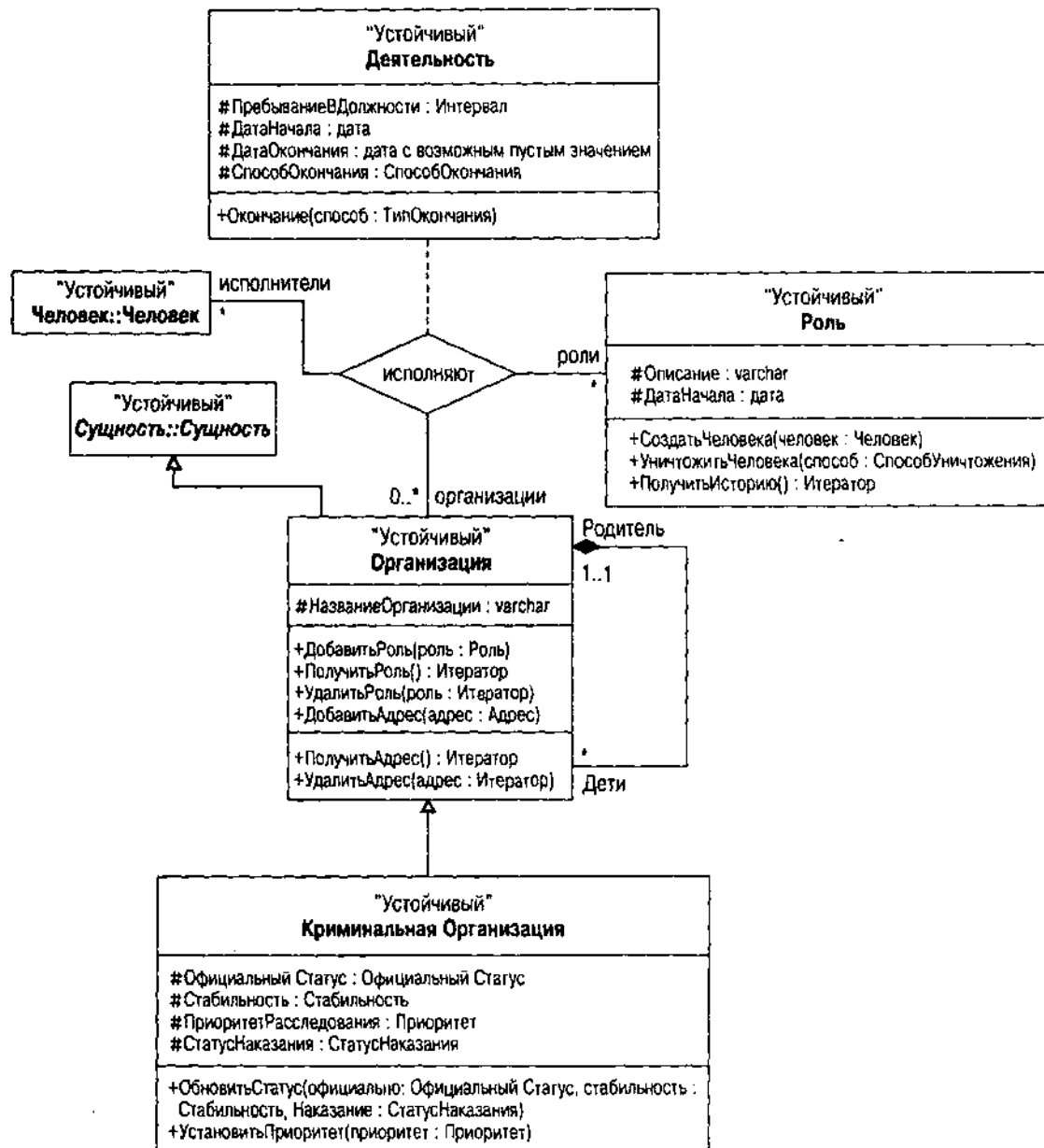


Рис. 13.2. Подсистема Организация на UML

Подсистема Человек содержит класс Человек и иерархию Идентификация, ему принадлежащую. Автор решил использовать версию наследования, а не версию интерфейса. Человек связан с Организацией через трехстороннее отношение с классом Роль в пакете Организация. Нотация разрешения области действия (Организация::Роль) идентифицирует классы, не являющиеся частью подсистемы Человек.

Подсистема Организация содержит иерархию Организация, которая включает в себя класс КриминальнаяОрганизация, а также класс Роль и отношения между Ролью и Организацией. Организации связаны с людьми через трехстороннее отношение к Роли.

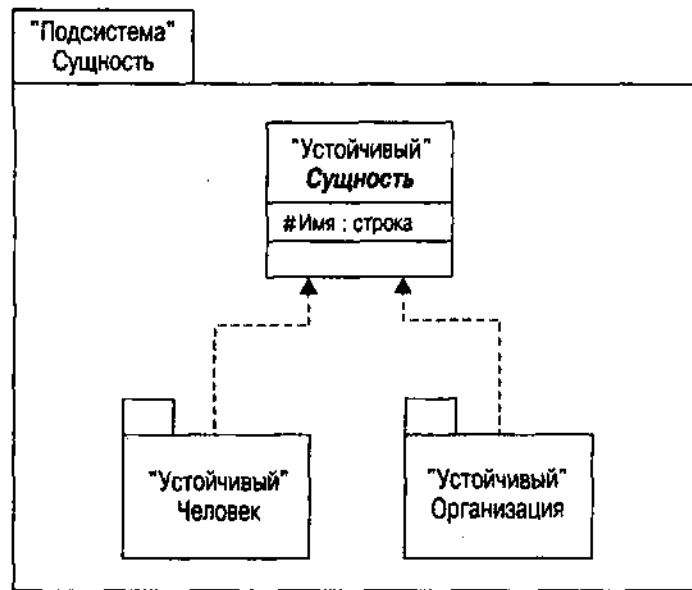


Рис. 13.3. Подсистема Сущность на UML

Подсистема Сущность содержит три элемента: две подсистемы — Человек и Организация — плюс абстрактный класс Сущность. Классы Человек и Организация в соответствующих пакетах наследуют от этого класса.

Действительно простота?

Повторим из главы 2, что первопричиной появления ООБД стало сокращение импедансного несоответствия между языком программирования и БД. Вместо того чтобы иметь отображение между представлениями приложения и концептуальной схемой с помощью другого языка (SQL) разработчик просто делает свои классы устойчивыми и выполняет приложение.

Несмотря на заманчивые обещания ОО-прозрачности и простоты приложений, OODBMS фактически этого не предоставляет, если взглянуть на формирование схем. В целом процесс отображения проще, чем для реляционных БД, но все же разработчик сталкивается с рядом проблем. Вот уж где прозрачность действительно преуспевает, и причем существенно, так это в области вставки, обновления и уничтожения объектов.

Прежде всего множество проблем возникает из-за отсутствия стандартизации продуктов OODBMS. Стандарт имеется — ODMG, но он фактически охватывает язык запросов OQL и основные средства, которые должна предоставить OODBMS. ODMG не позволяет взаимодействовать и практически не помогает согласовывать очень разные программные интерфейсы различных продуктов OODBMS.

Остальные проблемы возникают из-за того же импедансного несоответствия, из-за которого, прежде всего, и создаются ООБД. Транзакции, клиент/серверное кэширование, сложные ассоциации, ключи и расширения — все они являются концепциями, второстепенными для популярных языков ООП, но существенными для БД. Продукты OODBMS не могут обойти эти проблемы, поэтому все они разными способами на них сказываются.

Третья составляющая сложности преобразования происходит от такого устаревшего поверья, как "переносимость". По мере роста популярности ООП возникают новые языки, бросающие вызов лидерам рынка. Языки ООП начинались со Smalltalk и C++ — двух очень разных образцов программирования. Сейчас возрастает популярность Java, заимствующего в равной степени от Smalltalk и от C++ в программировании и добавляющего новые элементы, такие как интерфейсы, к этой мешанине. Более широко используемые языки, подобные Visual Basic, приобретают ОО-свойства. Более важно то, что с применением технологии распределенных объектов такие языки способны применять объекты, написанные на любом языке, вплоть до интерфейсов CORBA и COM. Написать "переносимую" программу в этом контексте довольно сложно, а переносимое ОО-приложение БД — еще сложнее. Например, рассмотрим стандарта ODMG привязки Java: он не поддерживает отношений, ключей, расширений или доступа к метасхеме ODMG [Cattell and Barry, 1997, p. 230]. Автор предоставляет читателям право решать, насколько полезен стандарт, не рассматривающий ассоциации, ключи или расширения, для прикладного программиста БД на Java.

Таким образом, совершенно не просто преобразовать модель данных UML в схему OODBMS. В следующем разделе рассматриваются некоторые основные проблемы, возникающие в большинстве подобных преобразований, но отдельные продукты всегда могут преподнести сюрпризы.

Классы

Класс находится в центре ОО-преобразования точно так же, как таблица — в центре R-преобразования, а объектный тип — в центре OR-преобразования. Особенности, которые мир OODBMS привносит в определение классов, — это устойчивость, использование интерфейсов, различных типов данных, управление пустыми значениями (или отсутствием таковых), идентификация объектов и расширения.

Устойчивые классы и интерфейсы

Преобразовать класс в схему OODBMS просто: достаточно создать класс. Но дальше все катится по наклонной плоскости.

Первое, о чем надо побеспокоиться: как сделать классы устойчивыми. Число интересных схем для этого весьма ограничено. ODL 2.0 просто предполагает, что создаваемые классы устойчивы. POET требует использования ключевого слова `persistent` перед ключевым словом `class`:

```
persistent class CriminalOrganization : public Organization { . . . };
```

Имеются, по крайней мере, две другие схемы. Например, ObjectStore из ObjectDesign использует класс памяти C++, чтобы указать на устойчивость объекта в противоположность устойчивости класса. Это позволяет использовать средства ObjectStore, позволяющие осуществлять запросы по расширению, которое включает как устойчивые, так и временные объекты. Путем создания устойчивых объектов вместо привязывания определения типа к БД ObjectStore делает осуществление запросов независимым от

устойчивости. Это касается второго основного достижения сообщества OODBMS – превращения типов данных в ортогональные к устойчивости или независимыми от устойчивости.

Стандарт ODMG поддерживает эту возможность с помощью своей концепции привязки C++ классов, которые могут иметь как устойчивые, так и временные экземпляры. Затем стандарт перегружает новый оператор в C++ объектным аргументом Database. Это позволяет объявлять БД временной (в оперативной памяти, неустойчивой), и теперь можно объявить временные, способные к устойчивости объекты. Такой подход прекрасно осуществляет интеграцию устойчивости классов и устойчивости объектов.

Система Objectivity/DB использовала другой подход – наследование. Эта система предоставляет библиотеку классов с классом ooObject. Чтобы сделать класс устойчивым, вы должны наследовать от ooObject. Этот подход устранил необходимость пред- или постобработки и полагался для управления всем непосредственно на C++. Обратная сторона этого подхода – связь создаваемых классов с классом Objectivity и, следовательно, необходимость многократного наследования, а также усложнение наследования классов.

По большому счету, есть два стандартных способа сделать объекты устойчивыми – придать им декларативную устойчивость и устойчивость по достижимости, известную также как транзитивная устойчивость. *Декларативная устойчивость* применяется в системах: вы объявляете, что этот, и только этот, класс (или объект) устойчив. *Транзитивная устойчивость* объявляет какой-то корневой объект устойчивым, затем делает устойчивыми все объекты, на которые этот корневой объект ссылается. Она находится в одном ряду со способами работы языков Smalltalk и Java и, таким образом, является частью стандартных привязок ODMG для этих языков [Cattell and Barry, 1997]. Поскольку эти языки предоставляют возможность "сборки мусора", объекты на самом деле не уничтожаются – уничтожаются только ссылки на них. Затем БД вычищает все объекты, на которые отсутствуют ссылки (постоянная "сборка мусора", метафора, непостижимая уму).

При программировании на Java вы определяете множество интерфейсов в модели данных UML вместо того, чтобы полагаться исключительно на классы и генерализацию классов. Интерфейсы обладают не состоянием, а только спецификациями абстрактного поведения. Вследствие этого интерфейсы – неустойчивые объекты. Привязка Java ODMG, например, определяет, что интерфейсы неустойчивы и DBMS никоим образом не сохраняет интерфейсы. Аналогичным образом абстрактные классы не соответствуют непосредственно объектам; следовательно, любой объект должен быть конкретным классом, и никакой OODBMS не нужно делать абстрактные классы устойчивыми. Таким образом, любой интерфейс UML или любой класс с дескриптором {абстрактный} не осуществляет перевод в устойчивый класс БД для программы Java. Привязка C++, с другой стороны, хранит молчание относительно абстрактных классов [Cattell and Barry, 1997].

Наилучшим решением при преобразовании интерфейсов UML в схему C++ OODBMS служит определение абстрактного класса для данного интерфейса, когда все операции превращаются в чисто виртуальные (инициализатор = 0 для метода, а реализация отсутствует) и полностью отсутствуют члены

данных. Затем можно использовать множественное наследование для представления реализации. Для этого необходимо (как с реализацией интерфейсов Java), чтобы все операции, определенные как чисто виртуальные, в базовом абстрактном классе, были реализованы в том подклассе, который от него наследует. Использование множественного наследования бывает возможно или невозможно в конкретной OODBMS, а также и не рекомендуется в связи с этими теоретическими и практическими проблемами, поэтому следует провести исследование и поставить эксперимент с целевой OODBMS прежде, чем задействовать определенную стратегию преобразования.

Атрибутивные типы данных

Большинство продуктов OODBMS в соответствии с лежащей в их основе философией определяют свои базовые типы как типы языка программирования, унифицирующего системы типов, используемые в программировании, и в БД. Из-за того, что у всех БД есть дополнительные требования к данным, эти продукты также обычно определяют дополнительные типы.

ODL определяет набор основных типов, называемых автоматическими литералами. *Литерал* в стандарте ODMG — это значение, не имеющее объектной идентификации в противоположность объекту, который ее имеет. Эти типы по большей части довольно стандартны: длинные, короткие, длинные без знака, короткие без знака, числа с плавающей запятой, числа двойной точности, булевы, восьмиразрядные (байт), символьные, строковые и перечислимые <>. Последний тип — это генератор типов, создающий тип на основе списка предоставляемых имен элементов. Имеется также набор генераторов литеральных типов коллекции (множество, неупорядоченная совокупность, список, массив и словарь) и структурированные литералы (дата, время, метка времени, интервал и структура<>). ODL определяет также литеральный тип с возможным пустым значением для каждого литерального типа (например, `nullable_float`, `nullable_set<>`, `nullable_date`) [Cattell and Batty, 1997, pp. 32-35].

Пустые значения (`null`) — проблематичные в продуктах OODBMS, поскольку большинство языков ООП не имеют концепции пустого значения. Например, привязки C++, Smalltalk и Java для ODL не поддерживают литеральные типы с возможным пустым значением, а также пустые значения для объектов. На практике большинство OODBMS продуктов также не поддерживают пустые значения. Если используется дескриптор {пустой} в моделях данных UML, зачастую обнаруживается, что нельзя преобразовать это в соответствующий тип данных с возможным пустым значением в схеме OODBMS. Если нельзя представлять пустые значения, нужно найти стандартное значение, означающее пустое значение, когда нет способа полного устранения пустых значений.

Типы коллекций OODBMS позволяют объединять литералы или объекты в:

- *Множество* — неупорядоченная коллекция элементов без дубликатов
- *Неупорядоченная совокупность* — неупорядоченная коллекция элементов, которая может содержать дубликаты
- *Список* — упорядоченная коллекция элементов, индексированная относительной позицией

- *Массив* — упорядоченная коллекция элементов, индексированная цифровой позицией
- *Словарь* — неупорядоченная последовательность пар ключ-значение без дублирования ключей (объект типа Ассоциация)

Коллекция может представлять собой либо объект с объектным идентификатором, либо литерал без идентификатора. Например, для определения отношений между Человеком и Адресом на рис. 13-1, используется следующий синтаксический фрагмент:

```
relationship set<Address> addresses inverse Person::residents;
```

Тип коллекции литералов `set<>` (множество) говорит о том, что каждый человек может иметь 0 или более адресов, что отношение не упорядочено (отсутствует дескриптор {упорядоченный}) и нет дублирующихся адресов. Если у отношения есть дескриптор {упорядоченный}, используйте типы `list<>` (список) или `array<>` (массив). Словарь применяется, если нужен индексированный поиск какого-то значения, которое можно задействовать в качестве индекса. Трудно найти пример использования `bag<>` (неупорядоченной совокупности), поскольку отношения UML предполагают отсутствие дублирующихся связей. Можно посчитать литеральный тип `bag<>` или его объектного кузена `Bag` более полезными в сохранении коллекций временных объектов или указателей.

ВНИМАНИЕ

См. Раздел "Ассоциации" с подробностями об отношениях и их зависимости от объектной идентификации.

Описание типа (*typedef*) — это понятие C и C++, которое позволяет создавать альтернативное имя для типа. Оно не определяет новый тип. В нотации UML соответствие этому понятию отсутствует.

Перечислимый тип (*enum*) — это литеральный тип с именем. Значение, присваиваемое с помощью `enum`, может быть одним из определенных перечислимых значений, которые указываются в списке объявления перечислимого типа. Можно определить перечислимый тип с помощью классификатора "type", составляющего список элементов значений. Затем это преобразуется в перечисление C++ или ODL, которое можно использовать для определения атрибутов и параметров операций:

```
enum LegalStatus{LegallyDefined, OnTrial, Alleged, Unknown};  
attribute LegalStatus status;  
void SetStatus(LegalStatus status);
```

Области и подобласти классов

Стандарт ODMG определяет область типа как "набор всех элементов данного типа в пределах определенной БД" [Cattell and Barry, 1997, p. 16]. Если Криминальная Организация — подтип Организации, тогда набор объектов всех криминальных организаций в БД — это область Криминальной Организации и объект каждой криминальной организации — это также член области Организации.

В реляционных БД область соответствует всем строкам в таблице. В объектно-реляционных — диапазону таблицы, который может включать несколько таблиц. Объектно-ориентированная БД способна автоматически поддерживать множество областей, независимо от принадлежности или местоположения объектов данного типа. OODBMS будет делать это в соответствии с ключевым словом или другим механизмом определения схемы. В ODL определяется ключевое слово `extent` и нмя:

```
class CriminalOrganization extends Organization (extent Mobs)
{ . . . }
```

OODBMS может автоматически проиндексировать область для ускорения запросов, если это определено. Можно всегда самостоятельно осуществлять поддержку областей в коллекции; проблема лишь в том, что это снижает параллелизм из-за блокировки объектов коллекции. Позволив БД выполнить эту работу, вы устраиваете ненужные блокировки объектов как на чтение, так и на запись. Области применяйте в запросах, когда необходим поиск по набору всех объектов какого-то типа и его подтипам [Jordan, 1998]. Можно также динамически создать ссылку на объект, используя шаблон `d_Extent<>`, который создает область с указанным указателем на объект БД [Jordan, 1998, pp. 107–108]:

```
d_Extent<CriminalOrganization> mobs(pDatabase);
```

В UML отсутствует понятие области. При желании можно добавить дескриптор класса к статическим структурным диаграммам UML, чтобы определить имя области, например КриминальнаяОрганизация с дескриптором `{extent=Mobs}`. Затем преобразовать это в соответствующее определение схемы для целевой OODBMS.

Интересное приложение областей появляется в следующем разделе при обсуждении ключей.

Идентификация объектов

Стандарт ODMG и большинство ОО-систем различают два вида данных: значения и объекты. Система распознает это различие, определяя объекты как элементы данных, имеющие объектную идентификацию, которую система может уникальным образом идентифицировать по отношению ко всем другим своим объектам. В случае OODBMS "система" — это домен памяти БД. Обычно существует какой-то способ проверить, являются ли два объекта одинаковыми при сравнении идентификации: например, ODMG предоставляет операцию `same_as()` для всех устойчивых объектов. Многие продукты OODBMS позволяют сравнивать объекты на равенство сравнением значений атрибутов; большинство также сравнивает объекты с помощью идентификации, обычно через операции на ссылках на объекты (эквивалент указателей в OODBMS).

Идентификация объектов превращает понятие первичного ключа в лишнее и ненужное. Ссылки на объект направляются на него с помощью объектного идентификатора, а не значений атрибутов внутри объекта.

Таким образом, в OODBMS не существует концепции первичного либо внешнего ключа. Вместо этого определяемые дескрипторы {OID} и {альтернативный OID} преобразуются в ограничения области класса. ODMG даже предоставляет способ определения значения ключа — предложение `key`:

```
class LawEnforcementID extends Identification
    (extent Badges key BadgeNumber){
    attribute unsigned long BadgeNumber;
    . . .
};
```

В качестве альтернативы целевая DBMS может предоставить некий вид уникального индексирования, как это делает POET [POET, 1997a, p. 114]:

```
persistent class LawEnforcementID : public Identification {
    unsigned long BadgeNumber;
    useindex BadgeNumberIX;
};

unique indexdef BadgeNumberIX : LawEnforcementID
{
    BadgeNumber;
};
```

Другой аспект идентификации объектов — глобальное присвоение имен. Исходные продукты OODBMS не поддерживали OQL или другой подобный тип языка запросов. Вместо этого они разрешали начать с корневого объекта, и затем пройти по отношениям, чтобы найти другие объекты. Для облегчения этого процесса продукты OODBMS ввели понятие глобальных имен объектов в пределах пространства имен БД. Таким образом, стало возможно создать свое собственное альтернативное имя идентификатора объекта и использовать его именования корневого объекта используемой навигационной сети. Затем пользователь извлекает этот объект непосредственно по его глобальному имени и начинает навигацию [Jordan, 1997, pp. 87-88].

ВНИМАНИЕ

Языки запросов предоставляют более общий и легко используемый способ достижения того же эффекта, что и глобальное именование, которое также имеет недостаток: связность данных возрастает из-за соединения с глобальными именами в программах. Если OODBMS предоставляет язык запросов, используйте его вместо глобального именования, если это вообще возможно.

Иногда возникает желание определить эти глобальные объекты, используя объектную нотацию UML, как на рис. 13.4, который демонстрирует различные ключевые криминальные организации, представляющие интерес для приложений каталога. Приведенная конкретная схема имен присваивает имена пяти криминальным организациям. Можно непосредственно извлечь каждый объект КриминальнойОрганизации, затем начать навигацию к ее членам и связанным криминальным организациям с помощью отношений на классе КриминальнаяОрганизация.

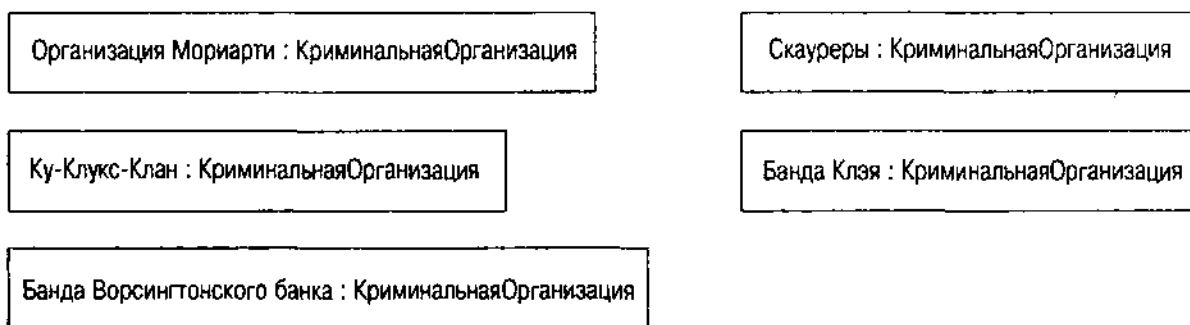


Рис. 13.4. *бъектные имена Криминальной Организации*

Например, чтобы определить статус членов банды Ворсингтонского банка, нужно извлечь этот объект по имени, а затем переместиться в коллекцию людей, исполняющих роли в этой банде.

Если есть такая диаграмма объекта, ее можно непосредственно преобразовать в глобальные объектные имена в целевой DBMS.

Генерализации и реализации

В стандарте ODMG выделяют два вида объектных типов: класс и интерфейс. Класс определяет абстрактное поведение и состояние объектного типа. Интерфейс — только абстрактное поведение. Эти понятия соответствуют понятиям UML-класса и интерфейса (подробности см. в главе 7).

Стандарт ODMG подразделяет интерфейсы на две разновидности — наследование поведения и наследование состояния. Эти понятия более или менее соответствуют понятиям UML отношений генерализации и реализации, но есть кое-какие тонкости, делающие эти две объектные модели различными. Цитата из стандарта, возможно, прояснит сущность отношений,

Классы — это типы, которые могут быть непосредственно реализованы, т.е. экземпляры таких типов могут быть созданы программистом. Интерфейсы — это типы, которые не могут быть непосредственно реализованы. Например, экземпляры классов Сотрудник_С_Зарплатой и Сотрудник_С_Почасовой_Оплатой могут быть созданы, но экземпляры их интерфейса супертипа Сотрудник нет. Порождение подтипов имеет отношение только к наследованию поведения; таким образом, интерфейсы могут наследовать от других интерфейсов и классы могут также наследовать от интерфейсов. Однако из-за неэффективности и двусмысленности множественного наследования состояния интерфейсы не могут наследовать от классов, а классы — от других классов [Cattell and Barry, 1997, p. 15].

В итоге всех этих абстракций ODL предоставляет два вида наследования. *Наследование поведения* возникает в спецификациях интерфейсов и классов, за которыми следует отделенная двоеточием нотация, напоминающая базовые спецификации классов в C++. Это и есть отношение "является", соответствующее реализации или генерализации UML, в зависимости от того, наследуют ли от интерфейса или класса. *Наследование состояния*

возникает в классах в отдельном предложении областей и соответствует генерализации без компонента поведения. Таким образом, можно иметь интерфейсы, наследующие от других интерфейсов (но не классов), и классы, наследующие как от интерфейсов, так и от других классов.

Различие ODMG между наследованием поведения и состояния еще не воплотилось в практике работающей OODBMS, плюс это или минус. API C++ для продуктов OODBMS используют модель наследования C++, которая сочетает две формы со всеми неоднозначностями, проистекающими из множественного наследования. Таким образом, для практических целей продукты OODBMS поддерживают UML-генерализацию. При программировании на Java, который поддерживает концепции интерфейса и наследования по отдельности, можно обнаружить, что Java OODBMS API более точно отражает стандарт ODMG 2.0. Тем не менее модель Java отличается от модели ODMG, которая сильнее напоминает UML-концепции генерализации (наследование класса или интерфейса с использованием ключевых слов области) и реализации (реализация интерфейсов с использованием ключевых слов реализаций).

Практическое преобразование отношений генерализации модели данных UML в схему OODBMS, таким образом, обычно приводит к переводу диаграммы UML в выбранный язык ООП. Обычно это будет отображение один-к-одному, поскольку UML был явно разработан с учетом этого вида преобразования. Интерфейсы вносят некоторое искажение в данную ситуацию, но если не используется Java, следует произвести преобразование один-к-одному генерализаций и реализаций в соответствующий синтаксис Java (предложения `extends` и `implements` соответственно). Если используется C++, нужно создать абстрактные классы и применить множественное наследование для представления реализации (см. предыдущий раздел "Устойчивые классы и интерфейсы", где подробно рассматривается преобразование интерфейсов).

Ассоциации

В ассоциациях продукты OODBMS будут создавать проблемы при формировании схемы. Ассоциации — отношения между объектами — являются центром предоставляемых OODBMS услуг. Хранение данных, управление транзакциями, даже восстановление и обеспечение целостности — стандартные операции БД. Способность к навигации между объектами продукты OODBMS приносят в таблицу как свое основное преимущество.

Вариации на тему

Можно представить связь между двумя объектами целым рядом различных способов, даже в стандарте ODMG. В данном разделе представлены основные варианты, а в следующем — наилучший способ представить различные виды ассоциаций UML с использованием следующих структур:

- *Атрибут.* Непосредственно внедряет отдельный связанный объект как член соотносящегося объекта, которому принадлежит связанный

объект. В C++ означает объявление члена данных с использованием объектного типа без указателя или ссылки. В схемах OODBMS — переводится в объект, который не имеет объектной идентификации, а состоит исключительно из значений, внедренных в соотносящийся объект. Активизировав последний, вы получаете все его внедренные данные, включая связанный объект. Там нет непосредственной ссылочной целостности, но вместе с уничтожением соотносящегося объекта уничтожается и связанный объект.

- *Ссылка.* Внедряет отдельный связанный объект в качестве элемента ссылки соотносящегося объекта, используя шаблон `d_Ref` или его эквивалент (*логический номер* или *постоянную ссылку*). Соответствует элементу данных C++, объявленному как указатель, но устойчивые данные должны иметь ссылку, а не указатель, поскольку указатели не являются устойчивыми. Связанный объект имеет жизненный цикл, изолированный от содержащего его объекта, и ссылка может быть `null`. Когда вы активизируете соотносящийся объект, связанный объект не всегда активизируется, хотя это допустимо при правильной установке параметров. Система не поддерживает ссылочной целостности; приложение должно управлять висящими ссылками.
- *Коллекция объектов.* Внедряет шаблон коллекции объектов как член соотносящегося объекта. Соответствует коллекции объектов, размещенной непосредственно коллекцией C++, и система считает их именно таковыми. Связанные объекты являются частью объекта коллекций и не имеют отдельной объектной идентификации. Активизировав внедренный объект, вы получаете коллекцию и все ее данные как часть значения. Система не поддерживает ссылочной целостности; коллекция управляет объектами как их владелец.
- *Коллекция ссылок.* Внедряет шаблон коллекции объектных ссылок как член соотносящегося объекта. Соответствует коллекции C++ указателей объектов, но ссылки позволяют объектам быть устойчивыми. Связанные объекты имеют жизненные циклы, изолированные от содержащих их объектов, и любая ссылка может оказаться повисшей (указывает на несуществующий объект) или `null`. Коллекция бывает пустой. Система не поддерживает ссылочной целостности, и приложение должно поддерживать объекты, явно уничтожать их и удалять повисшие ссылки.
- *Отношение.* Внедряет специальный шаблон коллекции объектных ссылок как член соотносящегося объекта. Соответствует коллекции C++ объектных указателей, но коллекция шаблонов поддерживает ссылочную целостность с обеих сторон отношения, поэтому повисшие ссылки отсутствуют. Ссылки могут быть пустыми, как и сама коллекция. В некоторых продуктах OODBMS ссылки также допускают "ленивую" активизацию объектов, позволяя извлечь объект с сервера только тогда, когда нужно его использовать. Другие системы активизируют все объекты, которых можно достичь из активизированного объекта (активизация транзитивного замыкания).

На рис. 13.5 представлен синтаксис ODL отношения. Первый идентификатор (или класс коллекции) — это тип отношения. Второй идентификатор — имя отношения. Если используется коллекция, поместите тип отношения внутрь фигурных скобок после имени класса коллекций (*set*, *list* или *bag*).

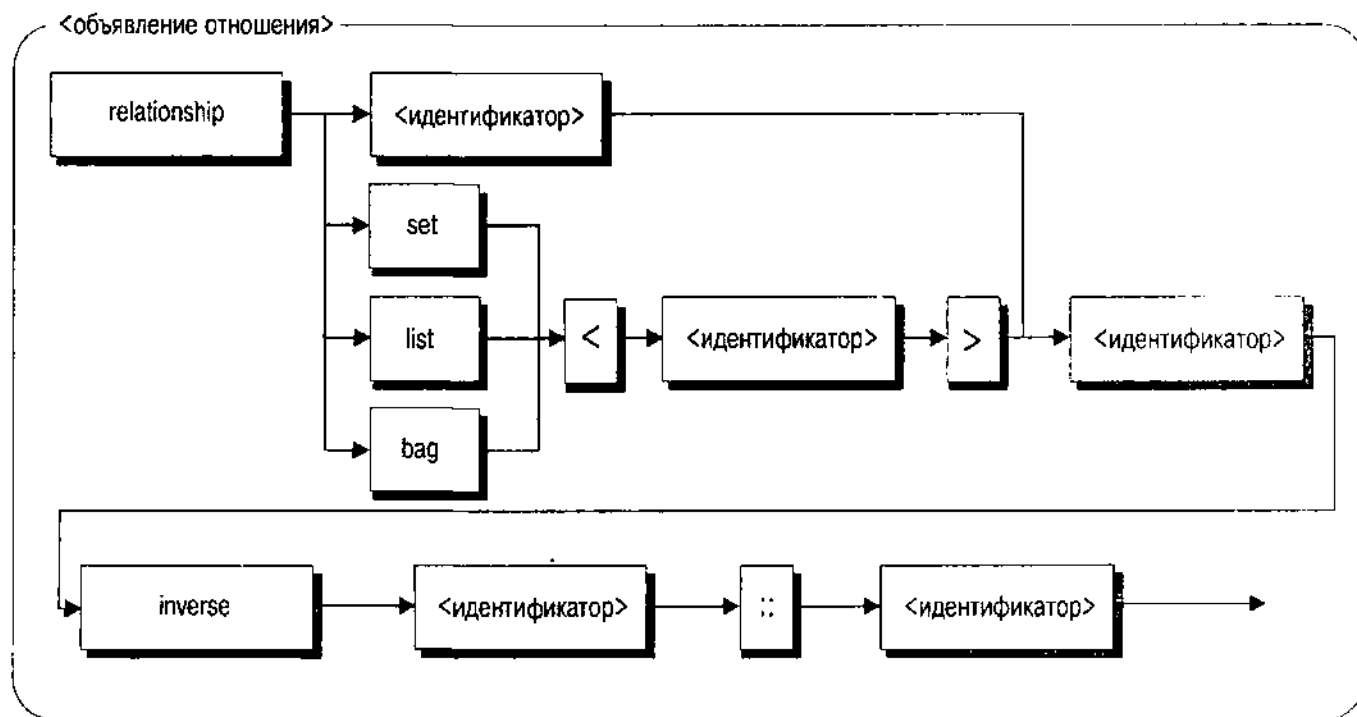


Рис. 13.5. Синтаксис отношения ODL

Синтаксис ODL требует, чтобы отношение содержало предложение *inverse*: это означает, что отношение всегда двунаправленно, т.е. видно из обоих классов. Поскольку только отношения поддерживают ссылочную целостность, можно иметь автоматическую ссылочную целостность только для двунаправленных ассоциаций. Предложение *inverse* содержит два идентификатора: имя целевого класса и имя отношения в этом классе, отделенное с помощью оператора разрешения области действия (::).

Теперь рассмотрим, как преобразуются различные виды конфигурации ассоциаций UML в схемы ODL и C++.

Направленные ассоциации с множественностью 1

Направленные ассоциации UML — это ассоциации со стрелкой, указывающие на класс. Добавление направления к ассоциации устанавливает одностороннюю видимость. Указывающий класс может видеть целевой класс, на который он указывает, но целевой, в свою очередь, не может видеть указывающий. Преимущество определения направленной ассоциации связано с инкапсуляцией и последовательным ограничением связности. Поскольку целевой класс ничего не знает о классе, который его использует, содержимое первого совершенно не затрагивается изменениями в указывающем

классе. Кроме того, в этом разделе ассоциация не является агрегацией, а это простая ассоциация. Это значит, что никакой класс не владеет другим классом.

Если множественность роли целевого класса — 0..1 или 1..1, значит объект указывающего класса ссылается только на один объект целевого класса. Таким образом, преобразование требует атрибута с единственным значением для указывающего класса. Специфическая природа этого атрибута зависит главным образом от наличия типов OODBMS. Обобщенный ODL предоставляет для объявления атрибут типа класса и это преобразуется в методы получить-и-задать, скрывая реализацию базовой структуры данных. C++ ODL предлагает два специфических вида объявлений: внедренный объект, объявленный как часть структуры данных, и устойчивую ссылку на объект. Внедренный объект не имеет идентификации объекта — он становится частью структуры данных внедряющего объекта, а не отдельным объектом в БД. Это подходит для составной агрегированной ассоциации, но не для обычной. При этом остается только ссылка, класс интеллектуального указателя, позволяющий объявить ссылку и направить ее на целевой объект с объектным идентификатором после создания или активации объекта.

Если множественность 0..1, ссылка может быть пустой, и методы должны проверять на null или перехватывать иадекватным образом возникающее исключение для ссылки null. Если множественность 1..1 или просто 1, нужно ввести ограничение, что ссылка всегда должна указывать на действительный объект в методах. Это означает, что конструктор должен ассоциировать объект со ссылкой, и ни один метод не может оставить ссылку как null или повисшей.

Описанная ситуация более распространена с множественностью 0..1, поскольку множественность 1..1 обычно возникает с составной агрегацией вместо обычной ассоциации. Иметь обычную ассоциацию с единственной целью вместо составной агрегации нужно только для того, чтобы позволить ассоциации быть пустой. Это соответствует члену данных C++, которому нужно быть указателем, так как объект может не всегда существовать.

ВНИМАНИЕ

Именно это иллюстрирует основное различие между написанием программы C++ и переносом программы C++ а модель устойчивых данных на основе OODBMS. Нельзя использовать указатели, являющиеся объектами в оперативной памяти — вместо них нужно применить какой-то вид ссылки или логический номер, или интеллектуальный указатель для ссылки на устойчивые объекты. Как следствие, нельзя просто взять программу на C++, пометить некоторые объекты как устойчивые и уйти. Нужно преобразовать члены указатели, параметры указатели, возвращаемые аргументы указатели в устойчивые ссылки. Это выглядит не слишком трудоемко, но на самом деле может обернуться довольно объемной работой по преобразованию кода.

В качестве примера предлагаем специализированное требование в необязательном подробном адресе человека в системе каталога. Подход, представленный на рис. 13. 1, моделирует адреса как последовательность строчных атрибутов в классе Человек. Вместо этого рассмотрим более сложную



Рис. 13.6. Классы Адрес и ПодробныйАдрес

модель, представляющую подробный адрес как отдельный класс (рис. 13.6). Чтобы соответствовать требованиям, предположим, что допустимо использовать один подробный адрес в нескольких объектах адресов (множественность 1..*) и адреса могут видеть подробные адреса, но подробным адресам не нужно видеть адреса. (Трудно представить, как настоящие примеры, имеющие смысл для конкретного сценария, применяются на практике. Обычно эти ситуации в итоге оказываются составной агрегацией, а не отдельными объектами).

Атрибуты, представляющие подробный адрес, сейчас находятся в классе ПодробныйАдрес. Версия ODL класса Адрес теперь выглядит так:

```

class Address (extent Addresses) {
    attribute StreetAddress Street;
    attribute string City;
    attribute string State;
    attribute string PostalCode;
    attribute string Country;
    attribute nullable_string Comment;
    string CreateLabel();
};
  
```

А вариант на C++ — так:

```

class Address : public d_Object {
protected:
    d_Ref< StreetAddress > Street;
    d_String City;
    d_String State;
    d_String PostalCode;
    d_String Country;
    d_String Comment;
public:
    const d_String & CreateLabel (void) const;
};
  
```

Шаблон `d_Ref` — это стандартное представление C++ для устойчивой ссылки OODBMS. Аргумент шаблона (`StreetAddress`) присваивает имя классу объектов, на который ссылается ссылка.

СОВЕТУЕМ

Пробелы между скобками шаблона и именем класса — будущий стандарт, который помогает многим компиляторам идентифицировать имя класса надлежащим образом. В данном случае, поскольку идентификатор — простое имя, пробелы не нужны. Но при использовании сложных имен в C++ (например, включении пространства имен и вложенных классов или `typedef`) можно обнаружить, что пробелы необходимы. Технология библиотеки стандартных шаблонов (STL) все еще имеет определенные проблемы, которые иногда усложняют жизнь.

Направленные ассоциации с множественностью более 1

При множественности `0..*` или `*` в направленной ассоциации вы имеете дело с коллекцией объектов, а не с отдельным объектом. Повторим, что направленная ассоциация подразумевает ограниченную видимость для указывающего объекта, и ассоциация не является составной агрегацией.

Возвращаясь к рис. 13.1, отметим, что перед нами двунаправленная ассоциация многие-ко-многим между Человеком и Адресом. А если для целей разработки ассоциация была создана направленной, со стрелкой, указывающей на адрес? Это означало бы, что можно осуществить доступ к адресу через человека, но нельзя получить доступ к человеку через адрес. Описанная ситуация также не часто встречается в реальных разработках, но несмотря на это, OODBMS предоставляет возможность осуществить задуманное.

```
class Person (extent People) {
    attribute set< Address > Addresses;
    . . .
};
```

На C++:

```
class Person : public Object {
    d_Set< d_Ref< Address > > Addresses;
    . . .
};
```

В варианте для C++ обратите внимание на две вещи. В-первых, множество содержит ссылки на объекты, а не сами объекты. Таким образом OODBMS C++ поддерживает объекты как отдельные, устойчивые объекты вне объекта `d_Set`. Во-вторых, нужно использовать пробелы между символами `>`; иначе у большинства компиляторов возникают проблемы из-за того, что их лексические анализаторы видят оператор `>>`, а не ограничители шаблона и сходят с ума.

Если удаляете адрес, переходите ко всем объектам Человек в БД и просите их уничтожить адрес из их множества адресов. Вероятно, придется

добавить специальную операцию к классу Человек, чтобы управлять обновлениями ссылочной целостности — такие операции надо вызывать только при удалении адреса. Кроме того, нужно сделать видимым все множество людей в том контексте, где удаляется адрес.

СОВЕТУЕМ

С точки зрения баланса, добавленная сложность всей этой ссылочной целостности, вероятно, перевешивает любые преимущества инкапсуляции, приобретенные при использовании направленной связи. В обычной программе на C++ эта структура требует дополнительной сложности операций управления кэшированием, но добавление устойчивости и ссылочной целостности в БД только ухудшает положение. Если нежелательно использовать семантику композитной агрегации, то обычно лучше предпочесть отношения подходу на основе встроенных объектов или коллекций. А значит, применить двунаправленные ассоциации.

Двунаправленные ассоциации с ролевой множественностью 1

Двунаправленная ассоциация переносит нас в мир отношений, а следовательно, и автоматической ссылочной целостности. Сначала рассмотрим ассоциацию из класса с ролевой множественностью 0..1 или 1..1 для роли, присоединенной к другому классу. Определяемый класс осуществляет доступ (вероятно, необязательно) к одиночному объекту целевого класса. Поскольку ассоциация двунаправленная, имеется также видимость ассоциации со стороны целевого класса. И наконец, целевой объект независим от ассоциирующегося объекта, поскольку ассоциация не является агрегацией. Перед нами непосредственно отношение с единственным объектом.

Еще раз обратившись к рис. 13.6, предположим, а что, если бы ассоциация здесь была не направленной, без стрелки, указывающей на Подробный-Адрес? ODL для класса Адрес выглядел бы так:

```
class Address (extent Addresses) {
    relationship StreetAddress Stree inverse StreetAddress::Addresses;
    attribute string City;
    attribute string State;
    attribute string PostalCode;
    attribute string Country;
    attribute nullable_string Comment;
    string CreateLabel();
};
```

Это отличается от первой версии этого класса заменой атрибута отношением, добавлением предложения *inverse*, определяющим имя отношения на стороне ПодробногоАдреса. Вариант на C++ таков:

```
extern const char _addresses []; // Address<->StreetAddress
class Address : public d_Object {
protected:
    d_String City;
```

```

    d_String State;
    d_String PostalCode;
    d_String Country;
    d_String Comment;
public:
    d_Rel_Ref< 2StreetAddress, _addresses > Street;
    const d_String & CreateLabel (void) const;
};

```

Стандартный шаблон отношения с единственным объектом `d_Rel_Ref` на C++ имеет два аргумента, имя целевого класса и символьный массив, содержащий имя отношения в целевом классе (член `inverse`). В данном случае будет программный файл (.cpp), содержащий инициализацию массива символьных констант:

```
const char_address[] = "Addresses"; // rel member of StreetAddress
```

ВНИМАНИЕ

Чтобы понять, насколько сложен этот подход, зададимся вопросом: какую роль играет параметр имени члена `inverse` и почему нельзя просто поместить строку литерала в аргумент шаблона (`d_Rel_Ref< StreetAddress, "address" >`)? Ответ: адрес в памяти объявляемой переменной (т.е. символьный массив `_address`) позволяет компилятору отличить этот экземпляр шаблона от других экземпляров шаблона. Если используется строка литералов, каждый раз после включения файла адрес будет другим. Затем шаблон отношения формирует много экземпляров (по одному для каждого включения шаблона) вместо одного, который ему требуется [Jordan, 1997, p. 123]. Это довольно типично для сложности, которую C++ приносит с собой разработчику БД (а также разработчику шаблонов).

Двунаправленные ассоциации с ролевой множественностью более 1

Теперь, перейдя к последней разновидности, рассмотрим ассоциацию с ролевой множественностью `0..*`, `1..*`, `*` или `n..m`, где `n` и `m` определяют мультиобъектный диапазон. Здесь снова представлен независимый объект, а не агрегат. Если опять обратиться к рис. 13.6 без стрелок, это положение демонстрирует класс ПодробныйАдрес. В ODL объявление класса выглядит так:

```

class StreetAddress (extent StreetAddresses) {
    attribute int Number;
    attribute string Fraction;
    attribute string Name;
    attribute string Suffix;
    relationship set<Address> Addresses;
    string CreateString();
};

```


Вариант на C++ таков:

```
extern const char_streetAddress[]; // Address<-> StreetAddress
class StreetAddress : public d_Object {
    int Number;
    d_String Fraction;
    d_String Name;
    d_String Suffix;
public:
    d_Rel_Set<Address, _streetAddress >;
};
```

Рис. 13.5 предоставляет вам один из трех вариантов для класса коллекций, которые можно использовать при объявлении отношения ко-многим: множество (set), неупорядоченное множество (bag) и список (list). Большинство отношений — множества. В определенных случаях допустимо сохранять связь с объектом более одного раза. Если упорядочивания не требуется в данной ситуации, используется неупорядоченное множество, в противном случае — список. Обе коллекции позволяют иметь дубликаты. Если ассоциация требует упорядочивания (для роли указан дескриптор {упорядоченный}), можно использовать список для задания порядка. Но если дубликаты не разрешены, примените множество, которое не имеет упорядочивания.

Составные агрегации

Теперь рассмотрим ассоциацию, которую до сих пор избегали — составную агрегацию. С ее помощью на рис. 13.1 класс Идентификации связан с классом Человек. Последнему принадлежит идентификация, которая не может существовать без человека.

В OODBMS можно представить эту ситуацию двумя способами: как истинную композицию или как двунаправленное отношение.

Первый способ позволяет непосредственно объявить объект:

```
class Person (extent People) {
    . . .
    attribute set <Identification>;
};
```

На C++:

```
class Person : public Entity {
    . . .
public:
    d_Rel_Set< Identification, _person >;
};
```

Здесь используется объявление атрибута для создания множества объектов идентификации и его объектов как части структуры класса Человек, а

не отдельно стоящих, устойчивых объектов. При активизации объекта Человек значение активизирует все множество объектов идентификации. Объекты идентификации не имеют никакой отдельной идентификации устойчивого объекта, поэтому при уничтожении объекта Человек уничтожаются также и все его объекты идентификации.

Используя второй способ, вы объявляете двунаправленное полное отношение, как это описано в разделе "Двунаправленные ассоциации с ролевой множественностью больше 1". Объекты Идентификации теперь становятся первоклассными устойчивыми объектами со своей собственной идентификацией в БД и получают доступ к ним через ссылки. К преимуществам этого подхода относятся автоматическая ссылочная целостность (удалите идентификацию, и ссылка на нее становится пустой), автоматическая поддержка отношения inverse, автоматическая поддержка области и возможность запроса независимых идентификационных объектов без ссылки на объекты Человек, содержащие их. Основной недостаток — нужно обеспечивать принадлежность только через код. При использовании подхода на основе истинной композиции, если уничтожается объект Человек, DBMS уничтожает также и все идентификационные объекты, принадлежащие этому объекту. При использовании подхода двунаправленных отношений это необходимо выполнить в операции Уничтожить(), поскольку DBMS уничтожит только объектную ссылку, а не объект, который независим.

ВНИМАНИЕ

OODBMS POET, например, предлагает средство наподобие каскадного удаления, имеющееся в большинстве ограничений внешних ключей RDBMS. Можно использовать ключевое слово depend для представления составной агрегации и режимов различной глубины, управляющими распространением удаления (P1SHALLOW и P1DEEP эквивалентны NO ACTION и CASCADE). Эти средства совершенно не стандартны, и разумеется, их нельзя использовать с любой другой OODBMS.

Третичные ассоциации

Продукты OODBMS пока не поддерживают n-арных ассоциаций, как полноценные отношения, и объектная модель ODMG также не позволяет объявлять такие ассоциации [Cattell and Barry, 1997, p. 36]. Следовательно, нужно представлять такие отношения, создавая классы ассоциаций и последовательные двоичные отношения между тремя или более связанными классами и данным классом [Jordan, 1998, pp. 116-117]. Например, представление отношения "виды деятельности" на рис. 13.2 на ODL выглядит так:

```
class Plays (extent PlaysInstances) {
    attribute interval Tenure;
    attribute date StartDate;
    attribute nullable_date EndDate;
    attribute TerminationType TerminationMethod;
    void terminate(TerminationType method);
```

```
relationship Person aPerson inverse Person::playsRoles;  
relationship Organization anOrganization Role  
    inerse Organization::players;  
relationship Role aRole inverse Role::played;  
};
```

Три двоичных отношения представляют собой три роли в качестве отношений к-одному к классу ассоциации. Для таких абстрактных отношений, как "виды деятельности", нужно быть довольно изобретательным в соглашениях об именах атрибутов: они отличаются от имен типов. На стороне связанных классов имеются соответствующие отношения коллекций:

```
class Person (extent People) {  
    . . .  
    relationship set<Plays> playsRoles  
        inverse Plays::aPerson;  
};  
class Organization (extent Organizations) {  
    . . .  
    relationship set<Plays> players  
        inverse Plays::anOrganization;  
};  
class Role (extent Roles) {  
    . . .  
    relationship set<Plays> played  
        inverse Plays::aRole;  
};
```

Версии этих объявлений на C++ довольно просты. Они используют шаблон `d_Rel_Set` с типами и строковыми ограничениями для хранения имен `inverse`.

Дисциплинирование объектов

Поведение объектов и управление им несложны и вместе с тем сильно отличаются от поведения R-таблиц и OR-типов.

Проблемы поведения

Самое очевидное различие между направлением развития большинства поставщиков RDBMS и продуктами OODBMS – место поведения в системе. Продукты OODBMS размещают поведение всех, за исключением немногих, классов на клиентской части. Для определения метода в классе кодируют программу, которая выполняется на клиенте в большей степени, чем на сервере. Это делает OODBMS немного менее гибкой, чем RDBMS с ее храняемыми процедурами, выполняемыми на сервере БД. Задача OODBMS –

материализовать устойчивые объекты в оперативной памяти клиента, затем позволить им сделать свое дело, координируя это с помощью транзакций, общего кэширования и т.д. [Jordan, 1998].

Позитивное следствие этого подхода — существенное упрощение интерфейса поведения. Больше не нужно беспрестанно беспокоиться о выборе места выполнения процедуры и разделять кодирование на C++ и языке программирования хранимых процедур. Все, что требуется — написать полностью программу на C++, Java или Visual Basic и выполнить ее. OODBMS с помощью *активации кэш-памяти* (перемещения объектов в кэш-память), *перемещивания указателей* (преобразования ссылок в объекты в оперативной памяти), *закрепления и освобождения* из памяти (удержания или освобождения объектов из кэш-памяти) и *управления параллелизмом* (совместного использования кэш-памяти и объектов в БД) выстраивает прозрачную систему виртуальной памяти, которая обслуживает объекты и позволяет клиенту обрабатывать все их методы.

Безусловно, обратная сторона этого подхода — невозможность распределить поведение на все части системы. Простые методы, которые могли выполняться на сервере БД, теперь не могут это делать, зато нужно извлекать объект через сеть и работать с ним на клиенте. Это ограничение распространяется и на обработку запросов; когда вы посылаете запрос, то выполняете поведение в клиентской прикладной кэш-памяти, не посылая SQL на сервер для выполнения в общей серверной кэш-памяти.

Таким образом, объектные БД являются системами с толстыми клиентами. Однако по мере расширения использования прикладных серверов, подобных системам CORBA и COM, этот недостаток не столь существен, как это может показаться. Допустимо создать клиенты OODBMS как прикладные серверы, когда реально тонкие клиенты ссылаются на распределенные объекты и поведение, выполняемое на толстых прикладных серверах.

Обычно OODBMS формирует соответствующие классы фабрики или методы, которые нужны прикладному клиенту для преобразования данных из БД в объекты с соответствующей структурой данных. Эти методы должны быть совершенно прозрачны для программиста БД.

Конструкторы объектов, с другой стороны, определенно находятся в области действия программиста. Система вызывает конструктор, когда создается объекта, а не когда его активируют (извлекают в прикладную кэш-память из БД). Таким образом, конструкторы служат той же цели для устойчивых объектов, что и для любых других объектов — инициализации элементов данных, созданию вложенных объектов и любых другие первоначальных установок объекта.

Деструкторы управляют оперативной памятью для устойчивых объектов точно так же, как и для временных объектов. Это не имеет ничего общего с удалением объектов в БД, которое выполняется с помощью метода `delete_object` на постоянной ссылке или удаления указателя на постоянный объект [Cattell and Barry, 1997, pp. 140-141]. Различные реализации OODBM имеют интересные способы управления блокировкой и деактивацией объектов наряду с различными способами управления сбором мусора уничтоженных объектов в БД. Разработчик определенно должен потратить

некоторое время на изучение документации, чтобы понять специфические отношения между удалением объектов, разрушением объектов и деактивацией объектов.

Аналогичным образом, у большинства продуктов OODBMS есть разные способы создания и обновления объектов. Стандарт ODMG призывает использовать перегруженный оператор `new` с некоторыми дополнительными аргументами для поддержки кластеризации БД или созданием в определенной БД. Определяя специальную БД `d_Database::transient_memory`, можно создать, например, устойчивый объект во временной памяти. Это свойство снова делает устойчивость ортогональной типу.

Для обновления нужно вызвать специальный метод `mark_modified()`, который приказывает OODBMS заблокировать объект и обновить устойчивую память при завершении транзакции. Большинство продуктов OODBMS предоставляют способ выполнить это автоматически, поэтому обычно оснований для беспокойства нет. Однако вы все же побеспокойтесь, по крайней мере до тех пор, пока не поймете, чего требует целевой продукт/продукты OODBMS в этой области, и как обеспечить переносимость с несколькими целями [Jordap, 1998, pp. 68-69].

Установка границ

Ограничить поведение в ОО-схеме легко. Для этого пишется программа. Не существует такой вещи, как ограничение БД или триггер в ОО-схеме — это просто другое поведение, которое присоединяют к классу. ОО-модель, таким образом, преобразует сложные ограничения в проблемы программирования, а не в проблемы БД. По желанию разработчика при написании ОО-программ идентифицируют необходимость ограничений в модели UML, затем создают соответствующие операции и запросы определенных методов в соответствующие моменты времени.

Например, если есть ограничение UML на класс, представляющий неизменяемое ограничение класса (предикат, который должен быть всегда истинным для объектов класса), создаются частные или защищенные операции, представляющие ограничение, затем их вызывают из любого метода класса, что может вызвать нарушение ограничения. Обычно это метод мутатора, изменяющий состояние класса, или, возможно, метод `verify()`, который показывает инвариант класса клиентам класса для целей тестирования.

Тот же самый подход применим к ограничениям первичных ключей, поскольку OODBMS заменяет данную концепцию идеей объектной идентификации. Однако, как отмечалось в ранее рассмотренном разделе "Идентификация объектов", ODL и различные продукты OODBMS предоставляют специфические способы реализации ограничений ключей, как первичных, так и потенциальных.

Более сложные ограничения, существующие в классах БД, могут потребовать отдельных классов управления, которые вводят ограничение на объекты, которыми владеют.

Целевой язык

Данный раздел резюмирует то, как можно создать ОО-схему, используя стандарт ODMG. Структура и язык этого раздела аналогичны структуре и языку подобного раздела "Язык мира" в главе 11. Следует обратиться к этой главе за примерами и подробностями R-преобразований и сравнения их с более простыми ООпреобразованиями, здесь описанными. Процесс преобразования показан на примере модели данных, приведенной на рис. с 13.1 по 13.3.

ОО-схема создается в два основных этапа: создание классов из устойчивых типов и создание классов для ассоциаций многие-ко-многим или троячных ассоциаций. Этот раздел указывает основное направление полного преобразования, объединяя все, что было рассмотрено ранее, в единое целое. К концу главы вы поймете, что преобразование разработки UML в ОО-схему гораздо проще, чем в R- или OR-схемы. Тем не менее потребуются приложить некоторые усилия из-за ограничений ODL и ориентированных на конкретный язык программирования привязки определения схемы продуктов OODBMS.

Устойчивые классы и интерфейсы

В качестве первого преобразования создадим устойчивый класс ODL или C++ для каждого класса в модели данных UML, имеющий стереотип "устойчивый", включая классы ассоциаций и классы, наследующие от других классов. Присвоим этому классу имя класса UML. В соответствии с представленной на рис. 13.1 подсистемой Человек создаются 10 классов: Человек, Адрес, Идентификация, ИдентификаторОкончанияСрокаДействия, ВодительскоеУдостоверение, Паспорт, НациональныйИдентификатор, ПринуждениеПоЗакону, КарточкаСоциальногоОбеспечения и СвидетельствоОРождении.

Если на диаграмме UML есть какие-либо интерфейсы, создаем для каждого интерфейс ODL или класс C++. Если используется привязка Java, тогда стоит осуществить непосредственное преобразование интерфейса UML в интерфейс Java. В противном случае просто преобразуем его в класс.

Для каждого атрибута в разделе атрибутов класса UML добавьте атрибут к классу. В качестве примера: для Адреса — НомерДома, НомерКвартиры, НазваниеУлицы, СуффиксУлицы, ОпределительМестоположения, Город, Штат, ПочтовыйИндекс, Страну и Комментарий. Интерфейсы ODL имеют атрибуты и отношения, но все это означает, что нужно выполнить методы получить (get) и задать (set). У интерфейсов нет состояния и, следовательно, настоящих элементов данных для атрибутов и отношений.

В этом месте следует создать отображение преобразования, показывающее, как преобразовать каждый тип данных в модели данных в тип данных ODL. Эти типы данных включают литеральные и структурированные типы (данные на основе значений), встроенные или зависимые объекты (объектные Идентификаторы) и коллекции (идентификаторы множества объектов). Если в диаграмме UML используются типы ODL, стандарт ODMG

предоставляет таблицы отображения для основных ОО-языков программирования [Cattell and Barry, 1997]. Если типы UML уже определены, используйте ключевые слова-объявления типов `typedef` и `enum` для объявления подтипов и перечислимых типов.

Поскольку основанные на C++ (и) DBMS присваивают идентификацию каждому устойчивому объекту, разработчик не создаст первичных или потенциальных ключей для класса C++. Вместо этого нужно ввести любые дескрипторы {OID} или {альтернативный OID} как ограничения, использующие ключ или средства уникального индексирования, если таковые имеются (см. раздел "Идентификация объектов"), или методы, если таких средств нет. Эти методы помещают в классы, агрегирующие область класса. Если временных классов нет, поместите метод в сам класс как статический член (член уровня класса в терминах UML). Обращайтесь с ограничением, как с инвариантом класса, который должен проверяться в каждом методе, изменяющем или добавляющем объекты.

ODL использует ключевую спецификацию для преобразования атрибутов {OID} или {альтернативный OID} в первичный ключ класса. Это также используют для определения альтернативного (потенциального) ключа. НомерУдостоверения, например, — это альтернативный ключ для ИдентификатораПринужденияПоЗакону. Следовательно, можно определить класс ODL, который определяет ключ:

```
class LawEnforcementID extends Identification
    (extent LawEnforcementIDs key BadgeNumber) {
    attribute unsigned long BadgeNumber;
};
```

Допустимо иметь столько ключей, сколько нужно для класса.

Если используется C++ или нежелательно пользоваться спецификацией ключа ODL, нужно ввести ограничения {OID} или {альтернативный OID} с помощью операций, устанавливающих уникальность области класса. Такая операция будет статическим методом в устойчивом C++ объявлении в классе.

```
static void CheckUniqueBadgeNumbers() const; // C++ version
```

В противном случае, если имеется некоторая разновидность агрегатной абстракции, представляющая область класса (кэш или другая коллекция), можно поместить метод в класс вместе с методами добавить (`add`) и удалить (`remove`), которые управляют областью класса.

Конечный результат этой стадии для таблицы Адрес с использованием определения C++ POET таков.

```
persistent class Address {
    long StreetNumber;
    string StreetFraction;
    string StreetName;
    string StreetSuffix;
    string City;
```

```

    string State;
    string PostalCode;
    string Country;
    string Comment;
public:
    // Constructor
    Address();
    // Destructor
    virtual ~Address();
    // Uses memberwise copy constructor, assignment
    // Accessors
    virtual string GetAddress() const; // constructs string
    virtual const string & GetCity() const;
    virtual const string & GetState() const;
    virtual const string & GetPostalCode() const;
    virtual const string & GetCountry() const;
    virtual const string & GetComment() const;
    // Mutators
    virtual void SetStreetAddress(long number,
                                   const string & fraction,
                                   const string & name,
                                   const string & suffix);
    virtual void SetCity(const string & city);
    virtual void SetState(const string & state);
    virtual void SetPostalCode(const string & code);
    virtual void SetCountry(const string & country);
    virtual void SetComment(const string & comment);
};

```

На ODL внешний вид был бы несколько другим:

```

class Address (extent Addresses) {
    attribute long StreetNumber;
    attribute string StreetFraction;
    attribute string StreetName;
    attribute string StreetSuffix;
    attribute string City;
    attribute string State;
    attribute string PostalCode;
    attribute string Country;
    attribute string Comment;
    // Accessors
    string getStreetAddress(); // constructs string
    // Mutators
    void SetStretAddress(in unsigned long number,

```



```

        in string      fraction,
        in string      name,
        in string      suffix);
};

```

Если имеется дескриптор {с возможным пустым значением} для атрибута элементарного литерального типа и применяются спецификации ODL, используйте версии типов с возможным пустым значением (`nullable_string`, например, или `nullable_long`). Это неприменимо к перечислимым и структурированным типам, которые не могут иметь пустого значения (`null`). Если используется C++, пустые значения вообще не могут быть представлены, поскольку C++ не имеет понятия пустого значения. Тем не менее, если все равно вы намерены использовать пустое значение, можно закодировать оболочки, добавляющие управление пустыми значениями к классам, хотя это существенно увеличивает сложность.

Если класс наследует от другого класса или набора классов, включите спецификацию суперкласса в определение класса. На C++ это обычно предложение `": public <superclass>".` На ODL используется ключевое слово `extends`.

Объявление ODL подкласса ИдентификаторПринужденияПоЗакону класса Идентификации выглядит так:

```

class LawEnforcementID extends Identification
    (extent LawEnforcementIDs) {
    attribute unsigned long BadgeNumber;
};

```

А на C++ класс ПринуждениеПоЗакону таков:

```

persistent class LawEnforcementID : public Identification {
    unsigned long BadgeNumber;
public:
    // Constructor
    BadgeNumber();
    //Destructor
    virtual ~BadgeNumber();
    // Memberwise copy constructor and assignment operator
    // Accessors
    virtual unsigned long GetBadgeNumber() const;
    // Mutators
    virtual void SetBadgeNumber(unsigned long number);
};

```

Классы ассоциаций представляют ассоциации с помощью атрибутов, ассоциаций многие-ко-многим и n-арных ассоциаций, которые связывают вместе два или более классов. Класс ассоциаций содержит ассоциации со связанными классами, используя соответствующие коллекции для представления множественности (об этом будет сказано подробнее позже при рассмотрении

преобразования ассоциаций). А пока создадим класс, содержащий атрибуты класса ассоциации:

```
class Plays {
    attribute interval      tenure;
    attribute date          startDate;
    attribute date          endDate;
    attribute terminationMethod termMethod;
    void terminate(in TerminationMethod method);
};
```

Настоящие отношения будут установлены позже, при рассмотрении ассоциаций классов.

Теперь добавьте любые простые ограничения классов-инвариантов к классам и интерфейсам. Если на диаграмме есть прямоугольники ограничений, преобразуйте их в операции уровня класса (статические).

Пора перейти к двоичным ассоциациям. Если ассоциация имеет класс ассоциации, пока ее проигнорируйте. Создайте член отношений для других ассоциаций в каждом из ассоциированных классов и определите предложение *inverse* с помощью имени отношения в другом классе. Если ассоциация направленная (стрелка с одной стороны), создайте член отношения только для целевого класса (предложение *inverse* отношения отсутствует или реверсивное отношение в другом классе). Для стороны *к-одному* — простое отношение идентификатора. Для стороны *ко-многим* — отношение коллекция-класс соответствующего типа. Если есть отношение *многие-ко-многим*, возникнут отношения коллекций в обоих классах.

ВНИМАНИЕ

Существуют и другие способы, но использование двунаправленных отношений — лучший. Обратитесь к разделу "Ассоциации", где подробно рассмотрены все способы преобразования ассоциаций.

Ассоциация *многие-ко-многим* на рис. 13.1 между Адресом и Человеком на ODL выглядит так (элементы класса опущены для ясности):

```
class Address (extent Address) {
    . . .
    relationship set<Person> residents
        inverse Person::addresses;
};
class Person (extent People) {
    . . .
    relationship set<Address> addresses
        inverse Address::residents;
};
```

Для ассоциаций с классами ассоциаций рассматривайте отношения не как непосредственное отношение между связанными классами, но как отношения с классом ассоциации. Например, если к ассоциации многие-ко-многим между людьми и адресами (ЧеловекАдрес) (рис. 13.1) добавлен класс ассоциации, на ODL будут так определены три класса с отношениями (элементы класса опущены для ясности):

```
class PersonAddress (extent PeopleAddresses) {
    attribute date startDate;
    attribute date endDate;
    relationship Person aPerson inverse Person::addresses;
    relationship Address anAddress inverse Address::residents;
};
class Address (extent Addresses) {
    . . .
    relationship set<PersonAddress> residents
        inverse PersonAddress::anAddress;
};
class Person (extent People) {
    . . .
    relationship set<PersonAddress> addresses
        inverse PersonAddress::aPerson;
};
```

Необходимо использовать этот подход для представления n-арных ассоциаций, поскольку нельзя непосредственно соотнести более чем два класса в отдельном отношении. ODL для третичной ассоциации ВидДеятельности и ее ассоциированных классов выглядит так (элементы класса опущены для ясности):

```
class Person (extent People) {
    . . .
    relationship set<Plays> playsRoles
        inverse Plays::aperson;
};
class Organization (extent Organizations) {
    . . .
    relationship set<Plays> players
        inverse Plays::anOrganization;
};
class Role (extent roles) {
    . . .
    relationship set<Plays> played
        inverse Plays::aRole;
};
class Plays {
```

```

attribute interval      Tenure;
attribute date          StartDate;
attribute date          EndDate;
attribute TerminationType TerminationMethod;
void terminate(in terminationType method);
relationship Person aPerson inverse Person::playsRoles;
relationship Organization anOrganization Role
inverse Organization::players;
relationship Role aRole inverse Role::played;
};

```

Для составных агрегаций, имеющих только один агрегированный объект, следует рассмотреть встраивание объекта в тип как альтернативу созданию отношения. Если составная агрегация агрегирует несколько объектов, можно представить ее как коллекцию дочерних объектов вместо отношения, основанного на коллекции. Использование атрибутной спецификации вместо спецификации отношения выражает сильную принадлежность ассоциации составной агрегации. Однако это не обеспечивает поддержку инверсного отношения.

Например, на рис. 13.1 приводится составная агрегация между Человеком и Идентификацией. Можно использовать подход на основе отношений, но в данном случае атрибут коллекций более уместен:

```

class Person (extent People) {
    . . .
    attribute set<Identification> IDs;
};

```

Данный атрибут не имеет инверсии, поэтому класс Идентификации ничего не знает о содержащем его родительском объекте Человек. По какой-то причине нужно осуществлять навигацию от объекта Идентификация к Человеку, которому он принадлежит, затем использовать отношение с инверсией вместо атрибута.

Это все о состоянии. А как насчет поведения?

Операции

Для представления операций в OODBMS создайте член операции для каждой операции, определенной в модели данных. В качестве примера рассмотрим класс КриминальнаяОрганизация, представленный на рис. 13.2. Этот класс экспортирует две операции — ОбновитьСтатус и УстановитьПриоритет, "мутаторы", меняющие состояние определенного объекта криминальной организации. В данном случае имеются два требования: перечислимые типы данных и обновление текущего объекта. Перечислимые типы — классификаторы типа UML, которые уже были преобразованы в объявления перечислимых типов ODL или C++. Затем необходимо добавить операции к классу ODL, а потом реализовать соответствующие методы в тело класса в отдельном файле C++.

Интерфейс ODL (или Java) предоставляет собой другую проблему. Данный интерфейс — это спецификация абстрактного поведения без реализации или состояния. При преобразовании интерфейса UML в интерфейс ODL просто объявляются операции; файл тела или реализация метода отсутствуют. Вместо этого добавляются операции к классам, которые реализуют интерфейс и реализуют их там.

Обычно надо уделять особое внимание конструкторам и деструктору каждого класса. Следует убедиться, что конструктор устанавливает все, что требуется. Проверьте также, что в деструкторе удалены все временные или постоянные объекты из оперативной памяти. Кроме того, можно обнаружить, что большинство продуктов OODBMS предоставляют множество операций по каждому устойчивому объекту, как правило, путем наследования из какого-то корневого устойчивого объектного типа.

Поскольку нет стандарта для создания или уничтожения объектов в устойчивой памяти [Cattell and Barry, 1997, p. 55], каждая OODBMS предоставляет свой собственный метод создания новых объектов и их удаления, когда работа с ними завершена.

OODBMS формируют временные объекты из устойчивых объектов с помощью специальных классов или методов фабрики, которые генерирует система в процессе генерации схемы. Эти фабрики переводят объекты из устойчивых во временные, затем вызывают конструктор для управления любой требуемой временной настройкой.

Удаление устойчивых объектов из оперативной памяти и удаление их из БД — совершенно разные вещи. Точно так же, как в реляционной БД, надо решить, куда поместить поведение удаления и сопровождающее его распространение на объекты, которыми владеет удаляемый объект. В некоторых случаях OODBMS предоставляет специальную операцию типа `delete_object` (удалить объект) в привязке C++ ODMG. Эта операция вызывается на объектах, которые принадлежат удаляемым объектам, а затем вызывает операцию на удаляемом объекте. В других случаях она выполняется автоматически, например с привязкой Java ODMG, удаляющей объекты из БД, когда система перестает на них ссылаться (автоматический сбор мусора). Обычно добавляют некую разновидность операции удаления для класса, который распространяет уничтожение.

Повторим еще раз, что OODBMS не хранит и не выполняет операции. Все поведение в объектной БД осуществляется на клиенте независимо от того, является ли он прикладным сервером, приложением толстого клиента или апплетом веб-браузера.

Таблица 13.1. Преобразование ОО-схемы

Шаг	Преобразование
1	"Устойчивый" класс UML становится устойчивым классом OODBMS.
2	Интерфейс UML, который реализует "устойчивый" класс, становится интерфейсом OODBMS для языков, которые поддерживают интерфейсы (Java, ODL), или абстрактным базовым классом только с чистыми виртуальными элементами и без элементов данных для тех языков, которые не имеют интерфейсов (C++, Smalltalk).
3	Классификатор типа UML становится enum или typedef, как указано.
4	Атрибут UML в классе становится атрибутом в классе OODBMS с помощью преобразований соответствующего типа и ограничений.
5	Используйте литеральный тип (например, nullable_short), если появляется дескриптор {с возможным пустым значением}, и привязка поддерживает пустые значения (ODL); в противном случае проигнорируйте его (C++, Java).
6	Если атрибут UML имеет инициализатор, добавьте код инициализации к конструктору либо как часть метода конструктора, либо как список инициализации элементов (в C++).
7	Для подклассов включите спецификацию суперкласса в объявление класса.
8	Для классов ассоциаций создайте класс с атрибутами класса ассоциаций.
9	Для явной идентификации объектов {OID} или потенциального ключа {альтернативный OID} определите ключевое объявление, если привязка их поддерживает. Если нет (C++, например), предоставьте соответствующие методы для проверки ограничений уникальности на множествах объектов.
10	Добавьте метод к соответствующему классу для каждого явного ограничения и убедитесь, что система вызывает этот метод, когда бы системе не потребовалось, чтобы ограничение было удовлетворено.
11	Создайте отношение для каждой двоичной ассоциации, которая не имеет класса ассоциации с соответствующим объектным или коллекционным типом, порожденным множественностями ассоциации. Используйте инверсные отношения, если нет явных стрелок на ассоциации (и даже в таком случае все равно рассмотрите это).
12	Создайте отношение в классах ассоциаций для каждой ролевой связи на другой класс.
13	Создайте код или используйте средства OODBMS, чтобы распространить удаление на все ассоциации составных агрегаций, как этого требует определенная OODBMS.
14	Создайте класс ассоциаций для третичных ассоциаций и отношения из класса ассоциации в ассоциированные классы с помощью соответствующих типов данных с имеющимися множественностями.

Итоги

Для удобства в таблицу 13.1 сведены все этапы преобразования в OODBMS. Она сильно отличается от таблицы 11.1, которая описывает R-преобразование, или таблицы 12.1, описывающей OR-преобразование. Структура в основном та же, но детали совершенно различны.

ООБД предоставляют совершенно другую точку зрения на объектную устойчивость и разработку схем. Проблемы разработки те же самые: структура классов, структура ассоциаций, поведение и ограничения. Разработчику предоставляется право решать, будет ли ОО-способ управления данными более прозрачным и продуктивным, чем реляционный или объектно-реляционный.

ВНИМАНИЕ

Автор полагает, что OODBMS лучше всего использовать для систем, которые либо используют навигационный доступ в противоположность специальному запросу, либо являются прежде всего средствами хранения объектов, а не для промышленных систем со множеством различных приложений на различных языках. Например, если система представляет собой C++ программу, использующую БД для хранения и извлечения объектов на общий прикладной сервер, следует применить OODBMS. А если система является большой коллекцией программ на Visual Basic, C++ и Developer/2000 с тремя или четырьмя различными типами генераторов отчетов и инструментарием запросов, следует использовать RDBMS или, возможно, ORDBMS.

Итак, мы ознакомились с основами разработки баз данных с использованием UML для представленных на рынке трех основных видов БД. Задача разработчика — выбрать правильный, который удовлетворит потребности, основанные на созданной модели данных UML. Простое или сложное, искусство разработки БД все еще является искусством, а не наукой. Выберите подходящую музыку — и боги баз данных будут к вам благосклонны.

Ссылки на истории о Шерлоке Холмсе

Следующие условные обозначения используются для ссылок на истории о Шерлоке Холмсе:

BLUE	Приключения голубого карбункула
BRUC	Рискованные планы Брюса Партингтона
EMPT	Авантюра с пустым домом
ENGR	Приключение мелкого инженера
FINA	Последняя проблема
FIVE	Пять апельсиновых косточек
HOUN	Собака Баскервиллей
IDEN	Дело об установлении личности
LION	Приключения львиной гривы
MISS	Приключение с потерянными 45 минутами
MUSG	Ритуал Мак-Грейва
PRIO	Приключения в монастырской школе
REDC	Приключения красного круга
SCAN	Скандал в Богемии
SILV	Серебряное великолепие
STUD	Этюд в красных тонах
SUSS	Приключения Суссекского вампира
THOR	Тайна моста Тор
TWIS	Человек со скрученными губами
VALL	Долина страха
VEIL	Приключения скрытного жильца

Библиография

- American National Standards Institute. 1975. ANSI/X3/SPARC Study Group on Data Base Management Systems; Interim Report. *FDT (Bulletin of ACM SIGMOD)* 7:2.
- American National Standards Institute. 1992. American National Standards for Information System—Database Language—SQL. ANSI X3. 135-1992. New York: American National Standards Institute.
- Baarns Consulting Group Inc. 1997. Frequently asked questions, DAO-DDBC Direct. www.baarns.com/access/faq/ad_DAO.asp, accessed on October 26, 1997.
- Beizer, Boris. 1984. *Software System Testing and Quality Assurance*. New York: Van Nostrand Reinhold.
- Berson, Alex. 1992. *Client/Server Architecture*. New York: McGraw-Hill.
- Blaha, Michael R., William J. Premerlani, and James E. Rumbaugh. 1988. Relational database design using an object-oriented methodology. *Communications of the ACM* 31 (4): 414-427.
- Blaha, Michael R., and William Premerlani. 1998. *Object-Oriented Modeling and Design for Database Applications*. Upper Saddle River, NJ: Prentice Hall.
- Booch, Grady. 1994. *Object-Oriented Analysis and Design with Applications, Second Edition*. Reading, MA: Addison-Wesley.
- Booch, Grady. 1996. *Object Solutions: Managing the Object-Oriented Project*. Reading, MA: Addison-Wesley.
- Borges, Jorge Luis. 1962. The library of Babel. In *Labyrinths: Selected Stories and Other Writings*, edited by Donald A. Yates and James E. Irby, pp. 51-58. New York: New Directions Publishing.
- Bruce, Thomas A. 1992. *Designing Quality Databases with IDEFIX Information Models*. New York: Dorset House.
- Buchmann, Alejandro M., Tamer Ozsu, Mark Hornick, Dimitrios Georgakopoulos, and Frank A. Manola. 1991. A transaction model for active distributed object systems. In [Elmagarmid 1991], pp. 123-158.
- Cattell, R. G. G. and Douglas Berry, editors. 1997. *Object Database Standard: ODMG 2.0*. San Francisco: Morgan Kaufmann.
- Chamberlin, Don. 1998. *A Complete Guide to DB2 Universal Database*. San Francisco: Morgan Kaufmann.
- Chappell, David. 1996. *Understanding ActiveX and OLE: A Guide for Managers and Developers*. Redmond, WA: Microsoft Press.
- Chen, Peter Pin-Shan. 1976. The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems* 1 (1): 9-36.
- Chidamber, S. R. and C. K. Kemerer. 1993. A metrics suite for object-oriented design. Working Paper #249. Cambridge, MA: MIT Center for Information Systems Research.
- Codd, E. F. 1970. A relational model of data for large shared data banks. *Communications of the ACM* 13 (6): 377-387.
- Codd, E. F. 1972. Further normalization of the database relational model. In *Data Base Systems*, Englewood Cliffs, NJ: Prentice Hall.
- Codd, E. F. 1990. *The Relational Model for Database Management: Version 2*. Reading, MA: Addison-Wesley.
- Conference on Data Systems Languages, Data Base Task Group (CODASYL DBTG). 1971. *CODASYL DBTG April 71 Report*. New York: Association for Computing Machinery.
- Date, C. J. 1977. *An Introduction to Database Systems, Second Edition*. Reading, MA: Addison-Wesley.
- Date, C. J. 1983. *An Introduction to Database Systems, Volume II*. Reading, MA: Addison-Wesley.
- Date, C. J. 1986. *Relational Database: Selected Writings*. Reading, MA: Addison-Wesley.
- Date, C. J. 1995. *An Introduction to Database Systems, Sixth Edition*. Reading, MA: Addison-Wesley.
- Date, C. J. 1997a. The normal is so... interesting. *Database Programming and Design* 10 (11): 23-25.
- Date, C. J. 1997b. The normal is so... interesting (Part 2 of 2). *Database Programming and Design* 10 (12): 23-24.
- Date, C. J. 1998a. The final normal form! (Part 1 of 2). *Database Programming and Design* 11 (1): 69-71.
- Date, C. J. 1998b. The final normal form! (Part 2 of 2). *Database Programming and Design* 11 (2): 59-61.
- Date, C. J. and Hugh Darwen. 1998. *Foundation for Object/Relational Databases: The Third Manifesto*. Reading, MA: Addison-Wesley.

- Diamond, Jared. 1997. *Guns, Germs, and Steel: The Fates of Human Societies*. New York: W.W. Norton.
- Domer, Dietrich. 1996. *The Logic of Failure: Why Things Go Wrong and What We Can Do to Make Them Right*. New York: Metropolitan Books. Originally published in German in 1989.
- Douglass, Bruce P. 1998. *Real-Time UML: Developing Efficient Objects for Embedded Systems*. Reading, MA: Addison-Wesley.
- Dreger, J. Brian. 1989. *Function Point Analysis*. Englewood Cliffs, NJ: Prentice Hall.
- Dutka, Alan F. And Howard H. Hanson. 1989. *Fundamentals of Data Normalization*. Reading, MA: Addison-Wesley.
- Elmagarmid, Ahmed K., editor. 1991. *Database Transaction Models for Advanced Applications*. San Mateo, CA: Morgan Kaufmann.
- Eriksson, Hans-Erik and Magnus Penker. 1997. *UML Toolkit*. New York: John Wiley and Sons.
- Everest, G.C. 1986. *Database Management: Objectives, System Functions, and Administration*. New York: McGraw-Hill.
- Fagin, Ronald. 1979. Normal forms and relational database operators. *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. May/June.
- Fagin, Ronald. 1981. A normal form for relational databases that is based on domains and keys. *ACM Transactions on Database Systems* 6 (3): 387-415.
- Feldman P., and D. Miller. 1986. Entity model clustering: Structuring a data model by abstraction. *Computer Journal* 29 (4): 348-360.
- Fenton, Norman E. and Shari Lawrence Pfleeger. 1997. *Software Metrics: A Rigorous and Practical Approach, Second Edition*. New York: International Thompson Computer Press.
- Fleming, Candace C. and Barbara von Halle. 1989. *Handbook of Relational Database Design*. Reading, MA: Addison-Wesley.
- Fowler, Martin. 1997. *Analysis Patterns: Reusable Object Models*. Reading, MA: AddisonWesley.
- Fowler, Martin, Ivar Jacobson, and Scott Kendall. 1997. *UML, Distilled: Applying the Standard Object Modeling Language*. Reading, MA: Addison-Wesley.
- Freedman, Daniel P. and Gerald M. Weinberg. 1990. *Handbook of Walkthroughs, Inspections, and Technical Reviews: Evaluating Programs, Projects, and Products, Third Edition*. New York: Dorset House.
- Fuller, Edmund, editor. 1966. *Hamlet*. New York: Dail.
- Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- Garmus, David and David Herron. 1996. *Managing the Software Process: A Practical Guide to Functional Measurements*. Englewood Cliffs, NJ: Prentice Hall.
- Gause, Donald C. and Gerald M. Weinberg. 1989. *Exploring Requirements: Quality Before Design*. New York: Doret House.
- Goldberg, Adele and Kenneth S. Rubin. 1995. *Succeeding with Objects: Decision Frameworks for Project Management*. Reading, MA: Addison-Wesley.
- Gray, James N. and A. Reuter. 1993. *Transaction Processing: Concepts and Techniques*. San Francisco: Morgan Kaufmann.
- Grimes, Richard. 1997. *Professional DCOM Programming with CD*. Olton, Birmingham, England: Wrox Press.
- Groff, James R. and Paul N. Weinberg. 1994. *LAN Times Guide to SQL*. Berkeley, CA: Osborne/McGraw-Hill.
- GUIDE International. 1997. GUIDE Business Rules Project. Final Report. Version 1.2. October. Chicago: GUIDE International. www.guide.org/ap/apbrules.htm (accessed July 24, 1998).
- Halpin, Terry. 1995. *Conceptual Schema and Relational Database Design*. Sydney, Australia: Prentice Hall Australia.
- Harmon, Paul and Mark Watson. 1998. *Understanding UML: The Developer's Guide, with a Web-Based Application in Java*. San Francisco: Morgan Kaufmann.
- Hay, David C. 1996. *Data Model Patterns: Conventions of Thought*. New York: Dorset House.
- Henderson-Sellers, Brian. 1996. *Object-Oriented Metrics: Measures of Complexity*. Englewood Cliffs, NJ: Prentice Hall.
- Hohmann, Luke. 1997. *Journey of the Software Professional: A Sociology of Software Development*. Englewood Cliffs, NJ: Prentice Hall.
- International Function Point Users'Group (IFPUG). 1994. *Function Point Counting Practices Manual, Release 4.0*. Westerville, OH: IFPUG. www.ifpug.org.

- International Standards Organization (ISO). 1997. *Database Language SQL—Part 2: Foundation (SQL/Foundation)*. September. Accessed on May 27, 1998, at [ftp://jerry.ece.umassd.edu/isowg3/dbl/BASEdocs/public/](http://jerry.ece.umassd.edu/isowg3/dbl/BASEdocs/public/).
- Jacobson, Ivar, Magnus Christerson, Patrik Jonsson, and Gunnar Overgaard. 1992. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Reading, MA: Addison-Wesley.
- Jepson, Brian. 1996. *Java Database Programming*. New York: John Wiley and Sons.
- Jones, Capers. 1991. *Applied Software Measurement: Assuring Productivity and Quality*. New York: McGraw-Hill.
- Jordan, David. 1997. *C++ Object Databases: Programming with the ODMG Standard*. Reading, MA: Addison-Wesley.
- Karlsson, Even-Andre, editor. 1996. *Software Reuse: A Holistic Approach*. New York: John Wiley and Sons.
- Kent, William. 1983. A simple guide to five normal forms in relational database theory, *Communications of the ACM* 26 (2): 120-125.
- Koch, George and Kevin Loney. 1997. *Oracle8: The Complete Reference*. Berkeley, CA: Oracle Press/Osborne-McGraw-Hill.
- Koch, George and Kevin Loney. 1998. *Oracle8: The Complete Reference*. Berkeley, CA: Oracle Press.
- Kung, H.T. and J. T. Robinson. 1981. On optimistic methods for concurrency control, *ACM Transactions on Database Systems* 6 (2): 213-226.
- Larman, Craig. 1997. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design*. Englewood Cliffs, NJ: Prentice Hall.
- Lassenen, Ken. 1995. Mapping the Data Access Object: DAO 3.0 www.eu.microsoft.com/oledev/olemap/mapdao30.htm. October 11, 1995. Accessed on October 26, 1997.
- Leach, Paul J. and Rich Salz. 1997. UUIDs and GUIDs and Internet Engineering Task Force Internet-Draft Memo. www.camb.org/tech/dce/info/ietf-draft.txt. February 24, 1997. Accessed on April 22, 1998.
- Lee, Sing. 1997. *Professional Visual C++ ActiveX/COM Control Programming*. Olton, Birmingham, England: Wrox Press.
- Lieberherr, K., I. Holland, and A. Riel. 1988. Object-oriented programming: an objective sense of style. *OOPSLA 1988 (SIGPLAN)* 23 (1): 323-334.
- Logic Works, Inc. 1996. *ERwin Methods Guide*. Princeton, NJ: Logic Works.
- McCabe, T. J. and C.W. Butler. 1989. Design complexity measurement and testing. *Communications of the ACM* 32 (12): 1415-1425.
- McNally, John. 1997. *Informix Unleashed*. Indianapolis, IN: SAMA Publishing.
- Melton, Jim and Alan R. Simon. 1993. *Understanding the New SQL: A Complete Guide*. San Francisco: Morgan Kaufmann.
- Melton, Jim. 1998. SQL3 moves forward. *Database Programming and Design* 11 (6): 63-66.
- Mowbray, Thomas J. and Ron Zahavi. 1995. *The Essential CORBA: Systems Integration Using Distributed Objects*. New York: John Wiley and Sons.
- Muller, Pierre-Alain. 1997. *Instant UML*. Olton, Birmingham, England: Wrox Press.
- Muller, Robert J. 1982. *Data Organization: The Integration of Database Management, Data Analysis, and Software Technology Applied to the National Crime Survey*. Ph.D. Dissertation, Massachusetts Institute of Technology.
- Muller, Robert J. 1996. Testing object-oriented software: Ahlerarchical approach. Paper given at the Ninth International Software Quality Week Conference in San Francisco, May 21-24. Available from the author.
- Muller, Robert J., Janice Nygard, and Paula Crnkovic. 1997. Driving system testing with use cases. *Conference Proceedings of the Sixth International Conference on Software Testing, Analysis and Review*. May 5-9, San Jose, CA. Jacksonville, FL: Software Quality Engineering.
- Muller, Robert J. 1998. *Productive Objects: An Applied Software Project Management Framework*. San Francisco: Morgan Kaufmann.
- O'Day, Dan. 1998. This old house. *IEEE Software* 15 (2): 72-75.
- Object Data Management Group. 1998. *Home Page*. www.odmg.org. Accessed on July 16, 1998.
- Open Group. 1997. The Open Group Architectural Framework—Version 3. www.opengroup.org/architecture (accessed December 12, 1998). Cambridge, MA: The Open Group.
- Oracle Corporation. 1997a. *Oracle8 Spatial Cartridge User's Guide and Reference*. Release 8.0.4. Redwood Shores, CA: Oracle Corporation.
- Oracle Corporation. 1997b. *Oracle8 Visual Information Retrieval Cartridge User's Guide*. Release 1.0.1. Redwood Shores, CA: Oracle Corporation.

- Oracle Corporation. 1997c. *Oracle8 Application Developer's Guide*. Release 8.0. Redwood Shores, CA: Oracle Corporation.
- Oracle Corporation. 1997d. *Oracle8 SQL Reference*. Release 8.0. Redwood Shores, CA: Oracle Corporation.
- Osgood, Charles E., George J. Suci, and Percy H. Tannenbaum. 1957. *The Measurement of Meaning*. Champaign, IL: University of Illinois Press.
- Papadimitriou, Christos. 1986. *The Theory of Database Concurrency Control*. New York: W. H. Freeman.
- POET Software. 1997a. *POET C++ SDK Programmer's Guide*. Version 5.0. San Mateo, CA: POET Software Corporation.
- POET Software. 1997b. *POET ODL Compiler Reference Guide*. Hamburg, Germany: POET Software Corporation.
- Quatrani, Terry. 1997. *Visual Modeling with Rational Rose and UML*. Reading, MA: Addison-Wesley.
- Rational Software Corporation. 1997a. *UML Notation Guide, Version 1.1*. Santa Clara, CA: Rational Software Corporation. www.rational.com.
- Rational Software Corporation. 1997b. *UML Samantics, Version 1.1*. Santa Clara, CA: Rational Software Corporation. www.rational.com.
- Rational Software Corporation. 1997c. *UML Object Constraint Language Specification, Version 1.1*. Santa Clara, CA: Rational Software Corporation. www.rational.com.
- Robarts, Fred S. 1979. *Measurement Theory with Applications to Decisionmaking, Utility, and the Social Sciences*. Encyclopedia of Mathematics and Its Applications, Volume 7. Reading, MA: Addison-Wesley.
- Rumbaugh, James, Michael Blaha, William Premerlani, Frederick Eddy, and William Lorensen. 1992. *Object-Oriented Modeling and Design*. Englewood Cliffs, NJ: Prentice Hall.
- Russell, Bertrand. 1956. *Portraits from Memory*. New York: Simon and Schuster. As quoted in [Weinberg and Weinberg 1988, pp. 8-9]
- Schneidewind, Norman F. 1998. How to evaluate legacy system maintenance. *IEEE Software* 15 (4): 34-42.
- Shlaer, Sally and Stephen Mellor. 1998. *Object-Oriented Systems Analysis: Modeling the World in Data*. Englewood Cliffs, NJ: Yourdon Press.
- Siegal, Jon, editor. 1996. *CORBA Fundamentals and Programming*. New York: John Wiley and Sons.
- Siegel, Shel and Robert J. Muller. 1996. *Object-Oriented Software Testing: A Hierarchical Approach*. New York: John Wiley and Sons.
- Signore, Robert, John Creamer, and Michael O. Stegman. 1995. *The ODBC Solution: Open Database Connectivity in Distributed Environments*. New York: McGraw-Hill.
- Salverston, Len, W.H. Inmon, and Kent Graziano. 1997. *The Data Model Resource Book: A Library of Logical Data Model and Data Warehouse Designs*. New York: John Wiley and Sons.
- Simons, Robert. 1997. Control in an age of empowerment. *IEEE Engineering Management Review* 25 (2): 106-112. Reprinted from the Harvard Business Review, March-April, 1995.
- Soley, Richard M., editor. 1992. *Object Management Architectural Guide, Second Edition*. OMG TC Document 92.11.1. Framingham, MA: Object Management Group.
- Stonebraker, Michael, and Paul Brown with Dorothy Moore. 1999. *Object-Relational DBMSs: Tracking the Next Great Wave*, Second Edition. San Francisco: Morgan Kaufmann.
- Teorey, Toby J. 1999. *Database Modeling and Design, Third Edition*. San Francisco: Morgan Kaufmann.
- Teorey, Toby J., Dongqing Yang, and James P. Fry. 1986. A logical design methodology for relational databases using the extended entity-relationship model. *Computing Surveys* 18 (2): 198-222.
- Texel, Putnam and Charles Williams. 1997. *Use Cases Combined with Booch/OMT/UML: Process and Products*. Englewood Cliffs, NJ: Prentice Hall.
- Ullman, Jeffrey D. 1988. *Principles of Database and Knowledge-Based Systems, Volume I*. New York: W.H. Freeman.
- Ullman, Jeffrey D. 1989. *Principles of Database and Knowledge-Based Systems, Volume II. The New Technologies*. New York: W.H. Freeman.
- Vaughan-Nichols, Steven J., compiler. 1997. Microsoft COMing forward. In Executive Briefs. *Object Magazine* 7 (8): 7.
- Weinberg, Gerald M. 1982. *Rethinking Systems Analysis and Design*. Boston: Little, Brown and Co.
- Weinberg, Gerald M. and Daniela Weinberg. 1988. *General Principles of Systems Design*. New York: Dorset House.
- Weinberg, Gerald M. 1992. *Quality Software Management, Volume 1: Systems Thinking*. New York: Dorset House.
- Yourdon, Ed and Larry Constantine. 1979. *Structured Design*. Englewood Cliffs, NJ: Prentice Hall.